

深度学习：基础与概念

作者:Christopher M. Bishop & Hugh Bishop
组织: 中译本
时间:November 1, 2026
版本:ElegantBook-4.5
全局显示/隐藏证明/解答/笔记:**Show/Hide**

目录

第 1 章 深度学习革命 **1**
1.1 深度学习的影响 1
1.1.1 医疗诊断 1
1.1.2 蛋白质结构预测 2
1.1.3 图像合成 2
1.1.4 大语言模型 4
1.2 一个教学示例 4
1.2.1 合成数据 4
1.2.2 线性模型 5
1.2.3 误差函数 5
1.2.4 模型复杂度 6
1.2.5 正则化 7
1.2.6 模型选择 7
1.3 机器学习简史 8
1.3.1 单层网络 8
1.3.2 反向传播 9
1.3.3 深度网络 10

第 2 章 概率 **11**
2.1 概率法则 11
2.1.1 医学筛查示例 11
2.1.2 加和法则和乘积法则 11
2.1.3 贝叶斯定理 12
2.1.4 再看医学筛查示例 12
2.2 概率密度 13
2.2.1 分布的示例 13
2.2.2 期望和协方差 13
2.3 高斯分布 14
2.3.1 均值和方差 14
2.3.2 似然函数 14
2.3.3 最大似然的偏差 15
2.3.4 线性回归 15
2.4 密度变换 15
2.5 信息论 16
2.5.1 熵 16
2.5.2 微分熵 16
2.5.3 Kullback-Leibler 散度 16
2.5.4 条件熵 17
2.5.5 互信息 17
2.6 贝叶斯概率 17
2.6.1 模型参数 17
2.6.2 正则化 17

2.6.3 贝叶斯机器学习 18

第 3 章 标准分布 **19**
3.1 离散变量 19
3.1.1 伯努利分布 19
3.1.2 二项分布 20
3.1.3 多项分布 20
3.2 多元高斯分布 22
3.2.1 高斯几何 22
3.2.2 矩 23
3.2.3 局限性 23
3.2.4 条件分布 24
3.2.5 边缘分布 25
3.2.6 贝叶斯定理 25
3.2.7 最大似然 25
3.2.8 序贯估计 26
3.2.9 高斯混合 26
3.3 周期变量 27
3.4 指数族分布 28
3.5 非参数化方法 29
3.5.1 直方图 29
3.5.2 核密度 30
3.5.3 最近邻 31

第 4 章 单层网络: 回归 **32**
4.1 线性回归 32
4.1.1 基函数 32
4.1.2 似然函数 33
4.1.3 最大似然 33
4.1.4 最小二乘的几何表示 34
4.1.5 序贯学习 34
4.1.6 正则化最小二乘法 34
4.1.7 多重输出 34
4.2 决策理论 35
4.3 偏差-方差权衡 36

第 5 章 单层网络: 分类 **39**
5.1 判别函数 39
5.1.1 二分类 39
5.1.2 多分类 39
5.1.3 1-of-K 编码方案 39
5.1.4 最小二乘分类 40
5.2 决策理论 40
5.2.1 误分类率 40
5.2.2 预期损失 41
5.2.3 拒绝选项 41

5.2.4 推理和决策 . . . 41
5.2.5 分类器精度 . . . 41
5.3 生成分类器 . . . 42
5.3.1 连续输入 . . . 42
5.3.2 最大似然解 . . . 42
5.3.3 离散特征与指数族 . . . 43
5.4 判别分类器 . . . 43
5.4.1 激活函数 . . . 43
5.4.2 逻辑斯谛回归 . . . 44
5.4.3 多类逻辑斯谛回归 . . . 44
5.4.4 probit 回归与规范连接 . . . 45

第 6 章 深度神经网络 46
6.1 固定基函数的局限性 . . . 46
6.1.1 维度诅咒 . . . 46
6.1.2 高维空间 . . . 46
6.1.3 数据流形 . . . 47
6.1.4 数据依赖的基函数 . . . 47
6.2 多层网络 . . . 48
6.2.1 参数矩阵 . . . 48
6.2.2 通用近似 . . . 49
6.2.3 隐藏单元激活函数 . . . 49
6.2.4 权重空间的对称性 . . . 50
6.3 深度网络 . . . 50
6.3.1 层次化表示 . . . 51
6.3.2 分布式表示 . . . 51
6.3.3 表示学习 . . . 51
6.3.4 迁移学习 . . . 52
6.3.5 对比学习 . . . 52
6.3.6 通用网络结构 . . . 53
6.3.7 张量 . . . 53
6.4 误差函数 . . . 54
6.4.1 回归 . . . 54
6.4.2 二分类 . . . 55
6.4.3 多分类 . . . 55
6.5 混合密度网络 . . . 56
6.5.1 机器人运动学示例 . . . 56
6.5.2 条件混合分布 . . . 56
6.5.3 梯度优化 . . . 57
6.5.4 预测分布 . . . 58

第 7 章 梯度下降 59
7.1 错误平面 . . . 59
7.1.1 局部二次近似 . . . 59
7.2 梯度下降优化 . . . 60

7.2.1 梯度信息的使用 . . . 61
7.2.2 批量梯度下降 . . . 61
7.2.3 随机梯度下降 . . . 61
7.2.4 小批量方法 . . . 62
7.2.5 参数初始化 . . . 62
7.3 收敛 . . . 63
7.3.1 收敛性分析 . . . 63
7.3.2 动量 . . . 63
7.3.3 学习率调度 . . . 64
7.3.4 AdaGrad、RMSProp 与 Adam 算法 . . . 64
7.4 正则化 . . . 65
7.4.1 数据归一化 . . . 65
7.4.2 批量归一化 . . . 66
7.4.3 层归一化 . . . 67

第 8 章 反向传播 **68**
8.1 梯度计算 . . . 68
8.1.1 单层网络 . . . 68
8.1.2 一般前馈网络 . . . 68
8.1.3 简单示例 . . . 69
8.1.4 计算复杂度与数值微分 . . . 70
8.1.5 雅可比矩阵与黑塞矩阵 . . . 71
8.2 自动微分法 . . . 72
8.2.1 前向模式自动微分 . . . 73
8.2.2 逆模式自动微分 . . . 74

第 9 章 正则化 **76**
9.1 归纳偏置 . . . 76
9.1.1 逆问题 . . . 76
9.1.2 无免费午餐定理 . . . 76
9.1.3 对称性和不变性 . . . 77
9.1.4 等变性 . . . 78
9.2 权重衰减 . . . 78
9.2.1 一致性正则化项 . . . 79
9.2.2 广义权重衰减 . . . 80
9.3 学习曲线 . . . 80
9.3.1 早停法 . . . 80
9.3.2 双重下降 . . . 81
9.4 参数共享 . . . 82
9.4.1 软权重共享 . . . 82
9.5 残差连接 . . . 83
9.6 模型平均 . . . 84
9.6.1 Dropout . . . 85

第 10 章 卷积网络 **87**
10.1 计算机视觉 . . . 87

10.1.1 图像数据 . . . 87
10.2 卷积滤波器 . . . 88
10.2.1 特征检测器 . . . 88
10.2.2 平移等变性 . . . 89
10.2.3 填充 . . . 89
10.2.4 跨步卷积 . . . 90
10.2.5 多维卷积 . . . 90
10.2.6 池化 . . . 90
10.2.7 多层卷积 . . . 91
10.2.8 网络架构示例 . . . 92
10.3 可视化训练好的 CNN . . . 92
10.3.1 视觉皮层 . . . 93
10.3.2 可视化训练好的滤波器 . . . 94
10.3.3 显著性图 . . . 94
10.3.4 对抗攻击 . . . 95
10.3.5 合成图像 . . . 95
10.4 目标检测 . . . 96
10.4.1 边界框 . . . 96
10.4.2 交并比 . . . 96
10.4.3 滑动窗口 . . . 96
10.4.4 跨尺度检测 . . . 97
10.4.5 非最大抑制 . . . 97
10.4.6 快速区域卷积神经网络 . . . 97
10.5 图像分割 . . . 98
10.5.1 卷积分割 . . . 98
10.5.2 上采样 . . . 98
10.5.3 全卷积网络 . . . 99
10.5.4 U-Net 架构 . . . 99
10.6 风格迁移 . . . 99

第 11 章 结构化分布 **101**
11.1 概率图模型 . . . 101
11.1.1 有向图 . . . 101
11.1.2 分解 . . . 101
11.1.3 离散变量 . . . 103
11.1.4 高斯变量 . . . 104
11.1.5 二元分类器 . . . 105
11.1.6 参数和观测值 . . . 106
11.1.7 贝叶斯定理 . . . 106
11.2 条件独立性 . . . 107
11.2.1 3 个示例图 . . . 107
11.2.2 相消解释 . . . 109
11.2.3 d 分离 . . . 110
11.2.4 朴素贝叶斯 . . . 110
11.2.5 生成式模型 . . . 111

11.2.6 马尔可夫毯 . . . 112
11.2.7 作为过滤器的图 . . . 112
11.3 序列模型 . . . 113

第 12 章 Transformer 与注意力机制 115
12.1 注意力 . . . 115
12.1.1 Transformer 处理 . . . 115
12.1.2 注意力系数 . . . 116
12.1.3 自注意力 . . . 117
12.1.4 网络参数 . . . 118
12.1.5 缩放自注意力 . . . 119
12.1.6 多头注意力 . . . 119
12.1.7 Transformer 层 . . . 119
12.1.8 计算复杂性 . . . 120
12.1.9 位置编码 . . . 120
12.2 自然语言 . . . 122
12.2.1 词嵌入 . . . 123
12.2.2 分词 . . . 123
12.2.3 词袋模型 . . . 124
12.2.4 自回归模型 . . . 124
12.2.5 递归神经网络 . . . 125
12.2.6 通过时间反向传播 . . . 126
12.3 Transformer 语言模型 . . . 126
12.3.1 解码器型 Transformer . . . 126
12.3.2 抽样策略 . . . 127
12.3.3 编码器型 Transformer . . . 127
12.3.4 序列到序列 Transformer . . . 128
12.3.5 大语言模型 . . . 128
12.4 多模态 Transformer . . . 129
12.4.1 视觉 Transformer . . . 129
12.4.2 图像生成 Transformer . . . 129
12.4.3 音频数据 . . . 130
12.4.4 文本语音转换 . . . 130
12.4.5 视觉和语言 Transformer . . . 130
12.4.6 图像描述 . . . 131

第 13 章 图神经网络 132
13.1 基于图的机器学习 . . . 132
13.1.1 图的属性 . . . 133
13.1.2 邻接矩阵 . . . 133
13.1.3 置换等变性 . . . 134
13.2 神经信息传递 . . . 135
13.2.1 卷积滤波器 . . . 135
13.2.2 图卷积网络 . . . 136
13.2.3 聚合算子 . . . 136

13.2.4 更新算子 . . . 138
13.2.5 节点分类 . . . 138
13.2.6 边分类 . . . 139
13.2.7 图分类 . . . 139
13.3 通用图网络 . . . 139
13.3.1 图注意力网络 . . . 139
13.3.2 边嵌入 . . . 140
13.3.3 图嵌入 . . . 140
13.3.4 过度平滑 . . . 141
13.3.5 正则化 . . . 141
13.3.6 几何深度学习 . . . 141
习题 . . . 142

第 14 章 采样 **144**
14.1 基本采样 . . . 144
14.1.1 期望 . . . 144
14.1.2 标准分布 . . . 145
14.1.3 拒绝采样 . . . 145
14.1.4 适应性拒绝采样 . . . 146
14.1.5 重要性采样 . . . 147
14.1.6 采样–重要性–重采样 . . . 147
14.2 马尔可夫链蒙特卡洛采样 . . . 148
14.2.1 Metropolis 算法 . . . 148
14.2.2 马尔可夫链 . . . 149
14.2.3 Metropolis–Hastings 算法 . . . 150
14.2.4 吉布斯采样 . . . 151
14.2.5 祖先采样 . . . 152
14.3 朗之万采样 . . . 153
14.3.1 基于能量的模型 . . . 153
14.3.2 最大化似然 . . . 154
14.3.3 朗之万动力学 . . . 155
习题 . . . 155

第 15 章 离散潜变量模型 **157**
15.1 K 均值聚类 . . . 157
15.2 高斯混合分布 . . . 159
15.2.1 似然函数 . . . 160
15.2.2 最大似然 . . . 160
15.3 EM 算法 . . . 162
15.3.1 高斯混合模型 . . . 163
15.3.2 EM 算法与 K 均值算法的关系 . . . 164
15.3.3 混合伯努利分布 . . . 164
15.4 证据下界 . . . 166
15.4.1 EM 算法回顾 . . . 167
15.4.2 独立同分布数据 . . . 167

15.4.3 参数先验 . . . 167
15.4.4 广义 EM 算法 . . . 168
15.4.5 顺序 EM 算法 . . . 168
习题 . . . 169

第 16 章 连续潜变量模型 170
16.1 主成分分析 . . . 170
16.1.1 最大方差表述 . . . 171
16.1.2 最小误差表述 . . . 171
16.1.3 数据压缩 . . . 172
16.1.4 数据白化 . . . 173
16.1.5 高维数据 . . . 173
16.2 概率潜变量 . . . 173
16.2.1 生成式模型 . . . 174
16.2.2 似然函数 . . . 174
16.2.3 最大似然法 . . . 175
16.2.4 因子分析 . . . 177
16.2.5 独立成分分析 . . . 177
16.2.6 卡尔曼滤波器 . . . 178
16.3 证据下界 . . . 178
16.3.1 EM 算法 . . . 179
16.3.2 PCA 的 EM 算法 . . . 180
16.3.3 因子分析的 EM 算法 . . . 181
16.4 非线性潜变量模型 . . . 181
16.4.1 非线性流形 . . . 181
16.4.2 似然函数 . . . 183
16.4.3 离散数据 . . . 183
16.4.4 构建生成式模型的四种方法 . . . 183
习题 . . . 184

第 17 章 生成对抗网络 186
17.1 对抗训练 . . . 186
17.1.1 损失函数 . . . 186
17.1.2 实战中的 GAN 训练 . . . 187
17.2 图像的生成对抗网络 . . . 188
17.2.1 CycleGAN . . . 190
17.2.2 表示学习与潜空间插值 . . . 192
17.2.3 理论性质 . . . 193

第 18 章 标准化流 194
18.1 耦合流 . . . 194
18.1.1 实值 NVP . . . 194
18.2 自回归流 . . . 196
18.3 连续流 . . . 198
18.3.1 神经 ODE 的反向传播 . . . 199
18.3.2 神经 ODE 流 . . . 200

第 19 章 自编码器 **203**
19.1 确定性的自编码器 . . . 203
19.1.1 线性自编码器 . . . 203
19.1.2 深度自编码器 . . . 204
19.1.3 稀疏自编码器 . . . 205
19.1.4 去噪自编码器 . . . 205
19.1.5 掩蔽自编码器 . . . 205
19.2 变分自编码器 . . . 207
19.2.1 摊销推理 . . . 208
19.2.2 重参数化技巧 . . . 208

第 20 章 扩散模型 **211**
20.1 前向编码器 . . . 211
20.1.1 扩散核 . . . 212
20.1.2 条件分布 . . . 212
20.2 反向解码器 . . . 213
20.2.1 训练解码器 . . . 214
20.2.2 证据下界 . . . 214
20.2.3 重写 ELBO . . . 215
20.2.4 预测噪声 . . . 215
20.2.5 生成新的样本 . . . 216
20.3 得分匹配 . . . 216
20.3.1 得分损失函数 . . . 218
20.3.2 修改得分损失 . . . 218
20.3.3 噪声方差 . . . 219
20.3.4 随机微分方程 . . . 220
20.4 有引导的扩散 . . . 220
20.4.1 有分类器的引导 . . . 221
20.4.2 无分类器的引导 . . . 221

附录 A 线性代数 **224**
A.0.1 矩阵恒等式 . . . 224
A.0.2 迹和行列式 . . . 224
A.0.3 矩阵导数 . . . 225
A.0.4 特征向量 . . . 226

附录 B 变分法 **229**

附录 C 拉格朗日乘子 **231**

第 1 章 深度学习革命

1.1 深度学习的影响

机器学习是当今最重要、发展最快的技术领域之一. 机器学习的应用遍及各个领域, 基于数据学习的解决方案正逐步取代传统的手工设计算法. 这不仅提升了现有技术的性能指标, 更为一系列全新能力的开发打开了大门. 而如果新算法仍需完全依赖人工设计, 这些能力的实现将是无法想象的.

作为机器学习的一个分支, 深度学习 (Deep Learning, DL) 已经成为一种异常强大且通用的数据学习框架. 深度学习的理论基础是被称为神经网络 (neural network) 的计算模型, 这些模型最初是受到人类大脑中学习和信息处理机制的启发而产生的. 人工智能 (Artificial Intelligence, AI) 领域致力于在机器中重现人脑的强大能力, 而今天" 机器学习" 和" 人工智能" 这两个术语常常被交替使用. 当前使用的许多人工智能系统, 实际上是机器学习的具体应用, 旨在解决特定领域的问题. 这些系统虽然非常有用, 但与人类大脑的巨大能力相比仍有很大差距. 人们引入了" 通用人工智能"(Artificial General Intelligence, AGI) 这一概念, 来描述构建具有更高灵活性的智能系统的愿景. 经过几十年的稳步发展, 机器学习现在已进入快速发展阶段 (参见**??**). 最近, 一种称为大语言模型的庞大深度学习系统开始展现出卓越的能力, 人们把这些能力视为通用人工智能的初步迹象 (Bubeck et al., 2023).

本节从 4 个不同领域的示例来讨论机器学习, 以此说明这一技术应用的广泛性, 同时引入一些相关的基本概念和术语. 值得注意的是, 这些示例以及很多其他示例都是使用深度学习这一基本框架的不同变体来解决的. 这与传统方法形成了鲜明对比, 在传统方法中, 不同的应用需要采用各种迥异且高度专业的技术来处理. 应当强调的是, 我们选择的例子只代表了深度神经网络应用广度的一小部分, 事实上, 几乎所有需要计算的领域都正在或即将受益于深度学习的革命性影响.

1.1.1 医疗诊断

我们先来看看机器学习在皮肤癌诊断中的应用. 黑色素瘤 (melanoma) 是最危险的一种皮肤癌, 但如果发现得早, 是可治愈的.图 1.1展示了皮肤病变的示例图片, 上面的一行图像是危险的恶性黑色素瘤, 下面的一行图像是良性痣. 显然, 区分这两类图片具有极大的挑战性, 几乎不可能通过人工编写出一个算法来实现对这些图像的准确分类.

这个问题已经通过深度学习得到成功解决 (Esteva et al., 2017). 该解决方案是通过一组大型的病变图像集构建的, 该图集称为训练集 (training set), 其中每一幅图像都被标注为恶性或良性, 标注的依据是对病变活检后得到的真实分类. 训练集用来确定深度神经网络中约 2500 万个可调参数 (称为权重) 的取值. 这个根据数据反馈的结果来设置参数值的过程称为学习 (learning) 或训练 (training). 训练的目标是使得训练好的神经网络根据新病变的图像就可以预测病变是良性还是恶性, 而无须执行耗时的活检步骤. 这是一个典型的监督学习 (supervised learning) 问题的例子, 因为在训练过程中, 每个样本都标注了正确的类别标签. 这同时也是一个分类 (classification) 问题的例子, 因为每个输入经过处理后, 都需要被分配到离散的类别集合中 (本例中为" 良性" 或" 恶性"). 如果输出包括一个或多个连续变量, 则称为回归 (regression) 问题. 在化工生产过程中, 根据温度、压力和反应物浓度预测产量, 就是回归问题的一个例子.

这个应用的一个有趣之处是, 可用的带标注的训练图像数量相对较少, 大约只有 12.9 万幅. 因此, 研究人员首先要在一个包含 128 万幅日常物体 (如狗、建筑和蘑菇等) 图像的大型数据集上对深度神经网络进行预训练, 然后在皮肤病变图像数据集上对进行微调 (fine-tune). 这是迁移学习 (transfer learning) 的一个例子, 神经网络从日常物体的大规模数据集中学习自然图像的一般属性, 随后专门针对皮肤病变分类问题进行优化. 使用深度学习技术对皮肤病变图像进行分类的准确率已经超过专业皮肤科医生的水平 (Brinker et al., 2019).

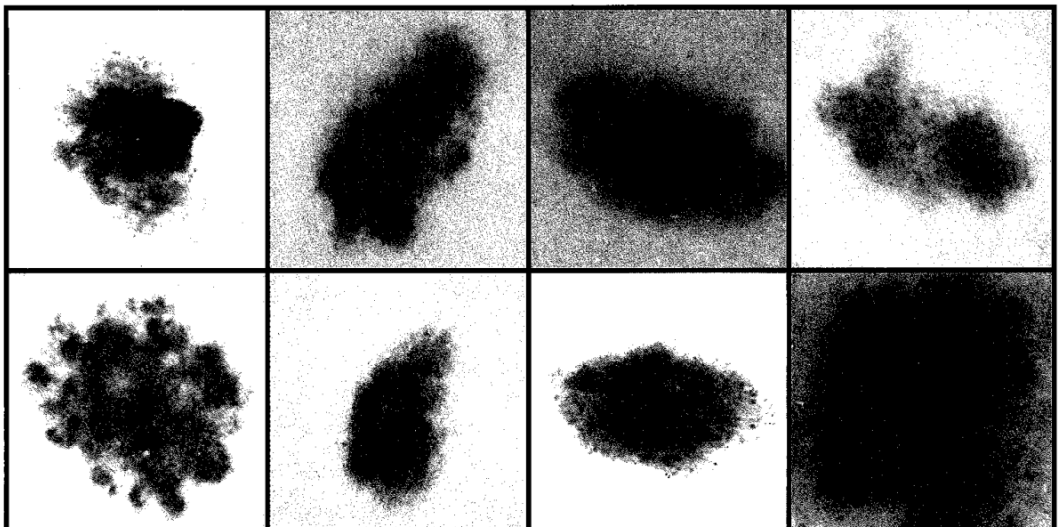


图 1.1: 上面的一行图像是危险的恶性黑色素瘤, 下面的一行图像是良性痣. 未经专业训练的人很难区分这两类皮肤病变类型.

1.1.2 蛋白质结构预测

蛋白质常被称为生物体的基石. 它们是由一个或多个氨基酸的长链组成的生物分子, 蛋白质的特性则由 22 种不同类型的氨基酸序列所决定. 当蛋白质在活细胞内合成后, 就会折叠成复杂的三维结构, 其行为和相互作用很大程度上取决于折叠的形态.

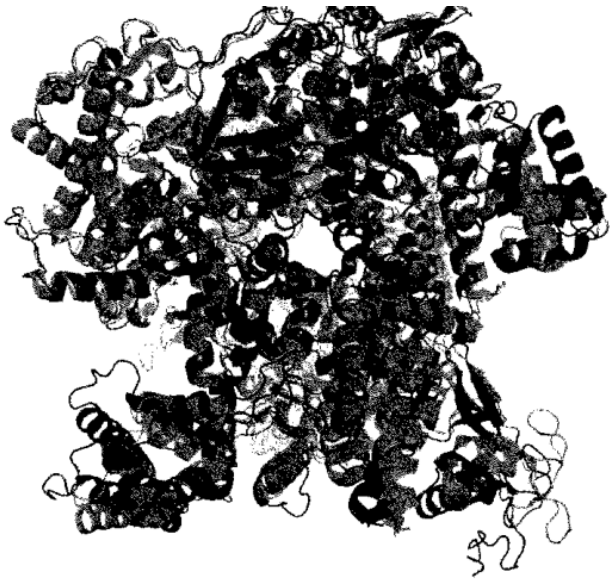
怎么根据氨基酸序列推导出蛋白质的三维结构呢? 这是生物学界几十年来一直想攻克的基础性难题, 但是进展甚微, 直到深度学习技术带来了突破性进展.

蛋白质的三维结构可以通过 X 射线晶体学、低温电子显微镜或核磁共振光谱等实验技术进行测定. 然而, 这个过程通常非常耗时, 而且对某些蛋白质来说, 这种方法极具挑战性. 例如, 很难得到蛋白质的纯净样本, 或者蛋白质的结构依赖于特定环境. 相比之下, 确定蛋白质的氨基酸序列的实验成本较低, 产出较高. 因此, 如何直接从氨基酸序列预测蛋白质的三维结构引起广泛关注, 这有助于更好地了解生物过程, 以及用于新药研发等实际应用. 我们可以以氨基酸序列为输入、以蛋白质的三维结构为输出来训练深度学习模型, 其中的训练数据包含了已知的氨基酸序列和对应的蛋白质三维结构. 因此, 蛋白质结构预测也是监督学习的一个例子. 经过训练的系统能够以新的氨基酸序列为输入, 预测对应的蛋白质三维结构 (Jumper et al., 2021).图 1.2比较了一种蛋白质的预测三维结构和通过 X 射线晶体学获得的实际三维结构.

1.1.3 图像合成

在上面讨论的两个应用中, 神经网络学会了将输入 (皮肤图像或氨基酸序列) 转换成输出 (病变分类或蛋白质的三维结构). 现在我们来看一个不同的例子: 训练数据是一组样本图像, 训练后的神经网络能生成类似的新图像. 与病变分类和蛋白质结构预测的例子不同, 这些训练图像是没有标注的, 这样的训练属于无监督学习 (unsupervised learning).图 1.3展示了一些由深度神经网络生成的合成图像, 该深度神经网络是在拍摄于摄影棚单色背景下的人脸图像集上进行训练的. 这些合成图像质量极高, 很难将它们和真人照片区分开来.

这种模型可以生成与训练数据不同但又有类似统计特性的结果, 所以被称为生成式模型 (generative model). 这种方法的一个变体能根据输入的文本字符串 (称为提示词,prompt) 生成反映文本语义的图像 (参见chapter 10). 我们用生成式 AI(generative AI) 这个术语来描述那些能生成图像、视频、音频、文本、候选药物分子或其他形态信息的深度学习模型.



的蛋白质的三维形状。绿色结构展示了通过 X 射线
深度学习模型 AlphaFold 预测的三维结构 [图片来

图 1.2: 一种名为 T1044/6VR4 的蛋白质的三维形状. 绿色结构展示了通过 X 射线晶体学确定的真实三维结构, 而叠加在上面的蓝色结构展示了深度学习模型 AlphaFold 预测的三维结构 (图片来源于 Jumper et al. (2021), 经许可使用).



图 1.3 由无监督学习方法训练得到的深度神经网络生成的人脸图像 (来自 generated.photos 网站)

图 1.3: 由无监督学习方法训练得到的深度神经网络生成的人脸图像 (来自 generated.photos 网站).

1.1.4 大语言模型

近年来, 机器学习领域最重要的进展之一, 是研发人员开发出了强大的模型来处理自然语言和类似程序代码这样的序列数据. 大语言模型 (Large Language Model, LLM) 利用深度学习技术构建丰富的内部表征, 捕捉语言的语义属性. 有一类重要的大语言模型称为自回归 (autoregressive) 语言模型, 它们能够输出语言, 因此属于生成式 AI 的一种. 这类模型以一个词序列作为输入, 生成这个词序列最可能的下一个词, 我们可以把末尾添加了新词的扩展序列再输入到模型中, 让它生成后续的词, 重复这个过程, 就会得到越来越长的词序列. 这样的模型还可以输出一个特殊的" 停止" 词, 来表示文本生成的结束, 这样就使得模型在输出一定长度的文本后停止. 此时, 用户可以在词序列的末尾添加自定义的词序列, 然后将完整的词序列重新输入模型以触发生成后续的词. 通过这种方式, 人们就可以和神经网络对话了.

这类模型可以在大型文本数据集上通过提取训练对 (training pair) 进行训练. 训练对的输入是随机选定的词序列, 输出是已知的下一个词. 这是自监督学习 (self-supervised learning) 的一个例子, 在自监督学习中, 模型学习从输入到输出映射函数, 有标注的输出是从输入训练数据中自动获取的, 无须进行另外的人工标注. 这种方法能把多种来源的大量文本作为大型训练集, 从而训练出超大规模神经网络.

一些专家认为, 展现出非凡能力的大语言模型是通用人工智能的雏形 (Bubeck et al., 2023), 本书后面将详细讨论这类模型 (参见**??**). 接下来, 我们给出一个基于 GPT-4(OpenAI, 2023) 的语言生成示例.GPT-4 能够针对如下提示做出响应:" 请以莎士比亚戏剧风格, 通过两个角色对话展开争论的方式证明素数有无穷多个", 如**??**所示.

1.2 一个教学示例

对机器学习领域的新人来说, 许多基本概念和术语可以通过一个简单的例子来引入——用多项式拟合一个小型合成数据集 (Bishop, 2006). 这是一个监督学习问题, 在这个问题中, 我们希望根据输入的变量值, 对目标变量进行预测.

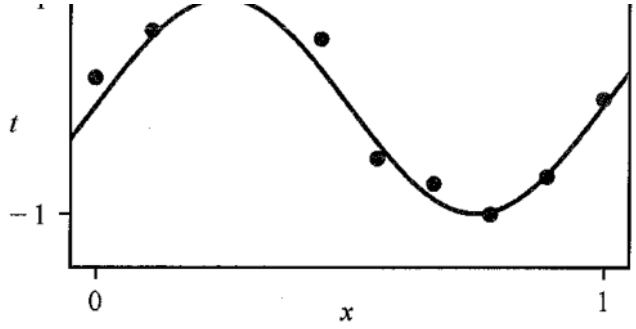
1.2.1 合成数据

我们用 x 表示输入变量, 用 t 表示目标变量, 并假设这两个变量在实数轴上取值连续. 给定一个训练集, 其中包含 N 个 x 的观测值, 记作 $x_1, \cdots, x_N$, 还包含相应 t 的观测值, 记作 $t_1, \cdots, t_N$. 我们的目标是根据 x 的某个新值来预测相应的 t 的值. 机器学习的一个关键目标是对以前未见过的输入进行准确预测, 这种能力称为泛化能力 (generalization).

我们可以通过从正弦函数采样生成的合成数据集来说明这一点.图 1.4展示了由 $N = 10$ 个数据点组成的训练集, 其中输入值 $x_n (n = 1, \cdots, N)$ 是通过在区间 $[0, 1]$ 上均匀采样生成的. 对应的目标值 t_n 则是先计算每个 x_n 所对应的函数 $\sin(2\pi x)$ 的值, 然后向每个数据点添加少量随机噪声 (由高斯分布控制, 参见**??**) 得到的. 通过这种方式生成数据, 我们可以捕获许多现实世界数据集的一个重要特性——它们具有我们希望了解的潜在规律, 但个别观测值会被随机噪声干扰. 这种噪声可能源于它们固有的随机过程 (例如放射性衰变), 但更常见的原因是存在未被观测到的变异源.

我们的目标是利用这个训练集来预测输入变量的新值 x 所对应的目标变量 t 的值, 这涉及隐式地尝试发现潜在的函数 $\sin(2\pi x)$. 这本质上是一个十分困难的问题, 因为我们必须从有限的数据集推广到整个函数. 此外, 观测数据受到噪声干扰, 因此对于给定的 x,t 的适当取值存在不确定性. 概率论 (probability theory)(参见chapter 2) 提供了一个以精确和定量的方式来表达这种不确定性的框架, 而决策论 (decision theory)(参见chapter 5) 允许我们利用这种概率表示, 根据适当的标准做出最优预测. 从数据中学习概率是机器学习的核心, 本书将对此进行详细探讨.

在这个示例中, 我们事先知道真正生成数据是通过一个正弦函数. 在机器学习的实际应用中, 我们的目标是在有限的训练集中发现隐藏的规律. 不过, 了解数据的生成过程有助于我们阐明机器学习中的一些重要概念.



组成的训练集，以蓝色圆点显示，其中每个数据点包含了
显示了用来生成数据的函数 $\sin(2\pi x)$。我们的目标是在不
标变量 t 的值

门事先知道真正生成数据是通过一个正弦函
示是在有限的训练集中发现隐藏的规律。不

图 1.4: 一个由 $N = 10$ 个数据点组成的训练集, 以蓝色圆点显示, 其中每个数据点包含了输入变量 x 及其对应的目标变量 t 的观测值. 绿色曲线显示了用来生成数据的函数 $\sin(2\pi x)$. 我们的目标是在不知道绿色曲线的情况下, 预测新的输入变量 x 所对应的目标变量 t 的值.

1.2.2 线性模型

现在让我们先从一种相对非正式的方式出发, 考虑一种基于曲线拟合的简单方法. 我们将使用多项式函数来拟合数据, 其形式如下:

$$y(x, \mathbf{w}) = w_0 + w_1 x + w_2 x^2 + \cdots + w_M x^M = \sum_{j=0}^{M} w_j x^j \tag{1.1}$$

其中 M 是多项式的阶数 (order),x^j 表示 x 的 j 次幂. 多项式系数 $w_0, \cdots, w_M$ 统称为向量 $\mathbf{w}$. 注意, 尽管多项式函数 $y(x, \mathbf{w})$ 是 x 的非线性函数, 但它也是系数 $\mathbf{w}$ 的线性函数. 在eq. (1.1)中, 像这个多项式一样, 关于未知参数呈线性的函数具有重要的特性, 同时也存在明显的局限性, 它们被称为线性模型 (linear model)(参见chapter 4).

1.2.3 误差函数

多项式系数的值将通过拟合训练数据来确定, 这可以通过最小化误差函数 (error function) 来实现, 该误差函数度量了对于任意给定的 $\mathbf{w}$, 函数 $y(x, \mathbf{w})$ 与训练集中数据点之间的拟合误差. 有一个使用广泛的简单误差函数, 即每个数据点 x_n 的预测值 $y(x_n, \mathbf{w})$ 与相应目标值 t_n 之间的差的平方和的二分之一:

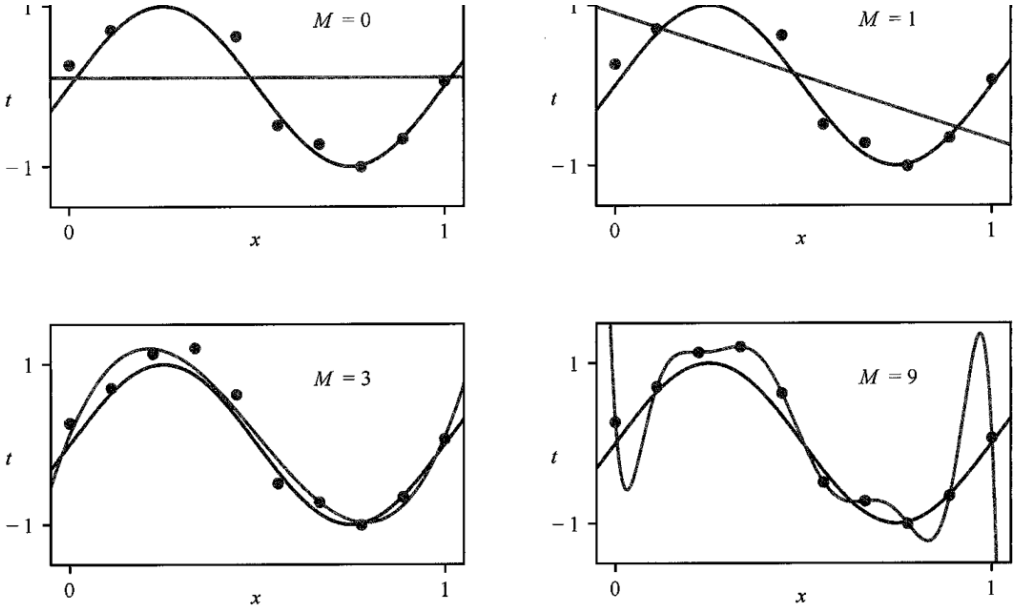
$$E(\mathbf{w}) = \frac{1}{2} \sum_{n=1}^{N} \{y(x_n, \mathbf{w}) - t_n\}^2 \tag{1.2}$$

其中引入系数 1/2 是为了后续计算方便. 稍后我们将从概率论出发推导此误差函数 (参见**??**). 注意, 这个误差函数是非负的, 当且仅当函数 $y(x_n, \mathbf{w})$ 正好通过每个训练数据点时, 其值等于零. 平方和误差函数的几何解释如**??**所示.

我们可以通过选择能够使 $E(\mathbf{w})$ 尽可能小的 $\mathbf{w}$ 值来解决曲线拟合问题. 因为平方和误差函数是系数 $\mathbf{w}$ 的二次函数, 其对系数的导数是系数 $\mathbf{w}$ 的线性函数, 所以该误差函数的最小化有一个唯一解, 记为 $\mathbf{w}^*$, 可以通过解析形式求得封闭解 (也叫解析解)(参见节 4.1). 最终的多项式由函数 $y(x, \mathbf{w}^*)$ 给出.

1.2.4 模型复杂度

我们还面临选择多项式的阶数 M 的问题, 这将引出模型比较 (model comparison) 或模型选择 (model selection) 这一重要概念.



具有不同阶数的多项式图示。多项式如红色曲线所示。这里通过最小化平方和误差函数［见式（1.2）
图 1.5 所示的数据集

可以发现，常数（$M = 0$）和一阶（$M = 1$）多项式对数据的拟合较差，因此对
$\sin(2\pi x)$ 的表示较差。三阶（$M = 3$）多项式似乎对函数 $\sin(2\pi x)$ 给出了最佳拟

图 1.5: 具有不同阶数的多项式图示. 多项式如红色曲线所示. 这里通过最小化平方和误差函数 (见eq. (1.2)) 来拟合图 1.4所示的数据集.

在图 1.5中, 我们展示了 4 个拟合实例, 它们分别使用阶数 $M = 0, 1, 3, 9$ 的多项式来拟合图 1.4所示的数据集.

可以发现, 常数 ($M = 0$) 和一阶 ($M = 1$) 多项式对数据的拟合较差, 因此对函数 $\sin(2\pi x)$ 的表示较差. 三阶 ($M = 3$) 多项式似乎对函数 $\sin(2\pi x)$ 给出了最佳拟合. 高阶 ($M = 9$) 多项式得到了一个对训练数据完美的拟合. 事实上, 这个多项式曲线精确地穿过了每一个数据点, 使得误差 $E(\mathbf{w}^*) = 0$. 然而, 拟合出的曲线却出现了剧烈的波动, 完全不能反映出函数 $\sin(2\pi x)$ 的真实形态. 这种现象称为过拟合 (over-fitting).

我们的目标是让模型获得良好的泛化能力, 使其能够对新的数据做出准确的预测. 为了定量地探究泛化性能与模型复杂度 M 之间的依赖关系, 我们可以引入一个独立的测试集. 该测试集包含 100 个数据点, 其生成方式与训练集相同. 针对每一个 M 值, 我们不仅可以计算出模型在训练集上的残差 $E(\mathbf{w}^*)$(eq. (1.2)), 还可以计算出其在测试集上的残差 $E(\mathbf{w}^*)$. 与评估误差函数 $E(\mathbf{w})$ 相比, 有时使用均方根 (Root Mean Square, RMS) 误差更为方便, 均方根误差定义如下:

$$E_{\mathrm{RMS}} = \sqrt{\frac{1}{N}\sum_{n=1}^{N}\{y(x_n, \mathbf{w}^*) - t_n\}^2} \tag{1.3}$$

公式中的 $1/N$ 是为了让不同大小的数据集能够在相同的基准下进行比较, 而求平方根则是为了确保 E_{RMS} 是在与目标变量 t 相同的尺度上 (以相同的单位) 进行测量的.

??展示了不同 M 值的训练集和测试集的 RMS 误差图. 测试集上的 RMS 误差反映了我们根据新观测数据 x 预测其对应 t 值的能力. 从图中可以看出, 当 M 值较小时测试集误差较大, 这是因为此时的多项式模型灵活性不足, 无法捕捉函数 $\sin(2\pi x)$ 中的振荡. 当 M 取值在 $[3, 8]$ 时, 测试集误差较小, 同时这些模型也能合理地表示出数

据的生成函数 $\sin(2\pi x)$, 如图 1.5中 $M = 3$ 时所示.

此外, 我们或许会认为, 预测新数据的最佳模型就应该是生成这些数据的真实函数 $\sin(2\pi x)$ 本身 (我们后续将验证这一点). 同时, 我们知道 $\sin(2\pi x)$ 的幂级数展开式中包含了所有阶数的项, 所以我们会很自然地推断, 随着模型复杂度 M 的增加, 预测效果应该会持续提升.

通过观察**??**中不同阶数多项式拟合得到的系数 $\mathbf{w}^*$, 我们可以更深入地了解这个问题. 我们注意到, 随着 M 的增加, 系数的幅度越来越大. 特别是当 $M = 9$ 时, 为了让对应的多项式曲线能精准地穿过每一个数据点, 这些系数被精细地调整到了很大的正值或负值. 但在数据点之间, 尤其是在数据范围的两端附近, 曲线却出现了大幅度的摆动, 正如我们在图 1.5中看到的那样. 直观地看, 当多项式模型具有较大的 M 值时, 它变得更加灵活, 从而更容易受到目标值上随机噪声的影响, 并过度拟合了这些噪声.

为进一步探究该现象, 我们观察随着数据集大小的变化, 模型学习效果的相应变化, 如**??**所示. 可以看出, 当模型复杂度固定时, 数据集越大, 过拟合现象就越不明显. 换言之, 数据量越大, 我们就能用越复杂 (即更灵活) 的模型去拟合数据.

1.2.5 正则化

根据可用训练集的大小来限制模型中参数的数量, 其结果有些不尽如人意. 而根据待解决问题的复杂性来选择模型的复杂性似乎更合理. 作为限制参数数量的替代方案, 正则化 (regularization) 技术经常被用于控制过拟合现象, 它通过向误差函数添加一个惩罚项来抑制系数取值过大. 最简单的惩罚项采用所有系数的平方和的形式, 误差函数变为

$$\tilde{E}(\mathbf{w}) = \frac{1}{2}\sum_{n=1}^{N}\{y(x_n, \mathbf{w}) - t_n\}^2 + \frac{\lambda}{2}\sum_{j=0}^{M} w_j^2 \tag{1.4}$$

其中 $\|\mathbf{w}\|^2 = \mathbf{w}^{\mathrm{T}}\mathbf{w} = w_0^2 + w_1^2 + \cdots + w_M^2$, 并且系数 λ 控制着正则化项与平方和误差项之间的相对重要性. 注意, 正则化项中通常不包含系数 w_0, 因为如果包含 w_0, 就会导致最终结果受到目标变量所选原点的影响 (Hastie, Tibshirani, and Friedman, 2009). 当然也可以包含 w_0, 但需要单独为其配置一个正则化系数 (参见**??**). 同样, 我们可以求得eq. (1.4)中误差函数的精确闭式解 (见习题 4.2). 在统计学文献中, 这类方法被称为收缩方法 (shrinkage), 因为它们会使系数的值缩小. 在神经网络领域, 这种方法称为权重衰减 (weight decay), 因为神经网络中的参数通常称为权重, 而这种正则化手段会促使这些权重向零衰减.

??展示了采用正则化误差函数 (eq. (1.4)) 对 9 阶 ($M = 9$) 多项式进行拟合的结果, 所用数据集与之前相同. 观察发现, 当取 $\ln\lambda = -18$ 时, 过拟合现象被有效抑制, 此时的拟合曲线与目标函数 $\sin(2\pi x)$ 相当接近. 反之, 若 λ 取值过大, 则会导致欠拟合, 如**??**中 $\ln\lambda = 0$ 的情形.**??**给出了不同 λ 值下拟合得到的多项式系数, 这些数值表明, 正则化确实发挥了预期作用, 有效减小了系数的幅度.

通过绘制训练集和测试集的 RMS 误差 (eq. (1.3)) 与 $\ln\lambda$ 的关系, 可以看出正则化项对泛化误差的影响, 如**??**所示. 可以看到,λ 现在控制了模型的有效复杂性, 从而决定了过拟合的程度.

在本例中,λ 作为一个超参数 (hyperparameter), 它的值在基于误差函数最小化来确定模型参数 $\mathbf{w}$ 的过程中始终保持不变. 需要注意的是, 我们不能通过同时对 $\mathbf{w}$ 和 λ 最小化误差函数的方式来简单地确定 λ 的取值, 因为这样会导致 λ 趋近于 0, 从而产生过拟合.

1.2.6 模型选择

交叉验证的主要缺点在于所需的训练次数增加了 S 倍, 这对于训练过程本身计算成本较高的模型来说是一个大问题. 交叉验证等使用独立数据评估性能的技术还存在另一个问题, 即对于单个模型可能存在多个复杂度超参数 (例如, 可能有多个正则化超参数). 在最坏的情况下, 探索这些超参数设置的最佳组合可能需要指数级数量的训练次数. 现代机器学习的前沿领域需要非常大的模型和大规模训练数据集. 因此, 超参数设置的探索空间有限, 很大程度上依赖于从小模型获得的经验和启发式方法.

这个将多项式模型拟合到基于正弦函数生成的合成数据集的简单示例, 阐释了机器学习领域的一些核心概念, 在后续章节我们将进一步用这个示例进行讨论. 然而, 现实中的机器学习应用往往要复杂得多: 首先, 用于模型训练的数据集规模可能会非常庞大, 数据量通常会高出几个数量级; 其次, 模型的输入变量的数量通常也会显著增加, 例如, 在图像分析领域, 输入变量的规模可达百万量级, 并且还可能伴随着多个输出变量. 在这些应用中, 将输出映射到输入的可学习函数通常由一类被称为神经网络的模型来表征, 这些模型往往具有海量的参数, 其数量甚至可以达到千亿的规模. 此时, 误差函数将表现为这些模型参数的复杂非线性函数, 无法再通过闭式解的方式进行最小化, 而必须借助迭代优化的方法, 利用误差函数关于模型参数的导数信息来逐步逼近最优解. 这对计算机硬件提出了更高的要求, 并且在实际应用中不可避免地会引入相当高的计算成本.

1.3 机器学习简史

机器学习拥有悠久且丰富的发展历史, 涌现了多种不同的研究路径. 本节将着重介绍基于神经网络的机器学习方法的发展脉络, 因为神经网络构成了深度学习的基石, 并且在实际应用中已被证明是最为有效的机器学习方法.

神经网络模型最初受到人类和其他哺乳动物大脑信息处理机制的启发. 大脑中处理信息的基本单元是称为神经元的电活性细胞, 如**??**所示. 当神经元放电时, 它会沿着轴突发送电脉冲, 到达称为突触的连接点, 这些连接点与其他神经元形成连接. 在突触处会释放称为神经递质的化学信号, 这些信号可以刺激或抑制后续神经元放电.

人脑总共包含约 900 亿个神经元, 每个神经元平均与其他数千个神经元通过突触相连, 形成了一个包含约 100 万亿 (10^{14}) 个突触连接的复杂网络. 如果某个神经元接收到来自其他神经元放电的足够强度的刺激, 那么它自身也可能被触发放电. 然而, 某些突触具有负向调节或抑制作用, 即输入神经元的放电会降低输出神经元放电的可能性. 一个神经元能否引发另一个神经元的放电取决于突触的连接强度, 而这些突触的连接强度的动态变化正是大脑存储信息和进行经验学习的关键机制.

神经元的这些属性已被抽象为简单的数学模型, 即人工神经网络 (Artificial Neural Network, ANN), 它们成为计算学习方法的基础 (McCulloch and Pitts, 1943). 诸多此类模型通过对其他神经元的输出的线性组合来描述单个神经元的特性, 然后使用非线性函数进行转换. 该过程可以用数学表达式表示为:

$$a = \sum_{j=1}^{M} w_j x_j + w_0 \tag{1.5}$$

$$y = f(a) \tag{1.6}$$

其中,$x_1, \cdots, x_M$ 表示与向该神经元发送连接的其他神经元的活动相对应的 M 个输入; 而 $w_1, \cdots, w_M$ 是权重 (weight), 表示相关突触的强度. 量 a 称为预激活 (pre-activation), 非线性函数 $f(\cdot)$ 称为激活函数 (activation function), 输出 y 称为激活 (activation). 可以看出,eq. (1.1)可视为这个模型的一个特例, 其中输入 x_j 由单一变量 x 的各次幂构成, 而激活函数 $f(\cdot)$ 退化为恒等函数 $f(a) = a$.eq. (1.5)和eq. (1.6)给出的简单数学公式构成了 20 世纪 60 年代至今神经网络模型的基础, 其图形表示如**??**所示.

1.3.1 单层网络

人工神经网络的历史大致可以根据其复杂程度划分为三个不同的阶段, 这种复杂性是通过处理的" 层数" 来衡量的. 由eq. (1.5)和eq. (1.6)描述的简单神经网络模型可以视为具有单层处理能力, 对应**??**中的单层连接结构. 在神经计算的历史长河中, 感知机模型 (Rosenblatt, 1962) 无疑占据着举足轻重的地位, 其激活函数 $f(\cdot)$ 是以下形式的阶跃函数:

$$f(a) = \begin{cases} +1, & a \geqslant 0 \\ -1, & a < 0 \end{cases} \tag{1.7}$$

它可以视为一个简化的神经元放电模型, 在该模型中, 当且仅当总加权输入超过 0 的阈值时, 神经元才会放电. 感知机由 Rosenblatt(1962) 首创, 他开发了一种特定的训练算法, 该算法有一个有趣的特点, 即如果存在一组权重值使得感知机能够完美分类其训练数据, 则该算法保证在有限步数内收敛到这一解 (Bishop, 2006). 除了学习算法之外, 感知机还需要一个专用的模拟硬件, 如**??**所示. 典型的感知机配置有多个处理层, 但其中只有一层可以从数据中学习, 因此感知机被认为是一个" 单层" 神经网络.

起初, 大家对神经网络模型抱有很高的期望, 但随后人们发现感知机暴露出重大的局限性.Minsky and Papert(1969) 分析了感知机的特性, 并从数学上严格证明了单层网络的能力局限. 他们还推测, 这样的局限性将会扩展到具有多个可学习参数层的神经网络. 尽管这一猜想后来被证明是极其错误的, 但它在当时极大地打击了人们对神经网络模型的热情, 并直接导致了 20 世纪 70 年代和 80 年代初学术界和工业界对神经网络领域缺乏足够的兴趣和资金投入. 此外, 由于缺乏有效的训练神经网络的算法, 研究人员无法深入研究多层神经网络, 因此像感知机这样的技术仅适用于单层模型. 请注意, 尽管感知机已经从实际机器学习应用中消失, 但这个术语仍然流传下来, 因此现代神经网络有时也称为多层感知机 (Multi Layer Perceptron, MLP).

1.3.2 反向传播

对于训练具有多层可学习参数的神经网络问题, 其解决方案来自微分学的应用以及基于梯度的优化方法. 一项重要的改进是用具有非零梯度的连续可微激活函数取代了原有的阶跃函数 (eq. (1.7)). 另一项关键的改进是引入了可微的误差函数, 该函数能量化地评估在给定参数配置下模型对训练集中目标变量的预测效果. 我们在节 1.2中利用平方和误差函数 (eq. (1.2)) 进行多项式拟合时, 就已经接触到了此类误差函数的一个示例.

通过这些改进, 我们现在得到了一个误差函数, 它可以计算其关于网络模型中每一个参数的偏导数. 现在我们可以着手考虑具有多层参数的网络结构了.**??**展示了一个包含两层参数的简单网络模型. 位于中间层的节点称为隐藏单元 (hidden unit), 因为它们的值不会出现在训练数据中, 训练数据仅提供了模型的输入值和输出值.

??中的每一个隐藏单元和每一个输出单元均计算形如eq. (1.5)和eq. (1.6)的值. 对于一组给定的输入值, 所有隐藏单元和输出单元的状态可以通过迭代地应用eq. (1.5)和eq. (1.6)进行前向计算, 在这个过程中, 信息沿着箭头所指的方向在网络中逐层向前传递. 由于这种信息流动的特点, 此类模型也被称作前馈神经网络 (feed-forward neural network).

能够训练具有多层权重的神经网络是一项重大突破, 自 20 世纪 80 年代中期开始, 这一突破再度引发了对该领域的研究热潮. 这一时期, 研究重心逐渐超越了单纯对神经生物学机制的模拟, 而朝着构建更加严谨、更具原则性的理论基础的方向发展 (Bishop, 1995b). 特别是, 人们逐渐认识到, 概率论和统计学中的核心思想在神经网络及机器学习领域扮演着至关重要的角色. 一个关键的认识是, 从数据中进行学习的过程不可避免地会引入一些背景假设, 这些假设被称为先验知识 (prior knowledge) 或归纳偏置 (inductive bias). 这些先验知识可以显式地融入模型设计之中, 例如, 可以设计一种特定的神经网络结构, 使得皮肤病变的分类结果不依赖于图像中病变的具体位置; 或者, 它们也可以隐式地蕴含在模型的数学形式或其训练方法的选择中.

反向传播算法和基于梯度的优化技术显著提升了神经网络解决实际问题的能力. 然而, 研究人员注意到, 在具有多层结构的神经网络中, 通常只有最后两层的权重参数能够学习到有效的信息. 除了少数例外, 特别是用于图像分析的卷积神经网络模型 (LeCun et al., 1998)(详见chapter 10), 具有两层以上结构的网络模型鲜有成功的应用. 这意味着此类网络模型不能有效解决太复杂的问题. 为了在各种实际应用中获得令人满意的性能, 人们通常要设计预处理步骤, 将原始输入变量变换到一个新的特征空间中, 希望在这个新的空间中机器学习任务能更容易解决. 这种预处理步骤有时也被称为特征提取 (feature extraction). 尽管这种人工特征提取的方法有时见效, 但一个更理想的解决方案显然是直接从数据中自动学习出这些特征, 而不是依赖于人工设计.

到了 2000 年, 当时的神经网络方法再次达到了它们能力的极限. 研究人员开始探索替代神经网络的方法, 如核方法、支持向量机、高斯过程等. 尽管一些热情的研究人员继续追求真正有效训练多层神经网络的目标, 但神经网络再次失宠. 在机器学习领域, 有许多不同的优化算法, 最常用的是随机梯度下降 (stochastic gradient descent) 法 (参见chapter 7). 为了训练这样的神经网络, 我们首先使用随机数生成器初始化参数, 然后使用基于梯度的优化算法自动调整权重值.

1.3.3 深度网络

神经网络发展的第三个阶段, 也是当前阶段, 始于 21 世纪的第二个十年. 一系列的突破使具有多层权重的神经网络得以有效训练, 从而消除了这些技术先前的能力限制. 具有多层权重的网络称为深度神经网络, 专注于这类网络的机器学习子领域称为深度学习 (deep learning)(LeCun, Bengio and Hinton, 2015).

深度学习发展历程中的一个关键性主题, 是神经网络模型规模的显著扩张, 这集中体现在模型参数数量的爆炸式增长上.20 世纪 80 年代, 拥有数百或数千个参数的神经网络模型还较为普遍, 但这一数字随后稳步攀升至数百万乃至数十亿规模. 而目前最前沿的模型, 其参数数量更是达到了惊人的一万亿 (10^{12}) 量级. 拥有海量参数的神经网络模型需要相应规模的庞大数据集进行训练, 才能确保在训练过程中为这些参数有效赋值. 这种庞大模型与海量数据的结合, 反过来也对模型训练过程的计算能力提出了更高的要求.

我们已经了解到, 网络的深度对于神经网络实现卓越性能至关重要. 要理解深度神经网络中隐藏层的作用, 一种有效的视角是表示学习 (representation learning, 也称表征学习)(Bengio, Courville, and Vincent, 2013). 在这种视角下, 网络能够学习把原始输入数据转换成某种新的、富有语义信息的表示形式, 从而显著降低后续网络层需要解决的问题的难度. 这些学习到的内部表示还可以通过迁移学习技术被重新利用来解决其他相关问题, 正如前文在皮肤病变分类任务中所展示的那样. 一个有趣的发现是, 用于处理图像的神经网络模型所学习到的内部表示, 与在哺乳动物视觉皮层中观察到的神经表征惊人地相似 (详见小节 10.3.1). 那些能够针对一系列下游任务进行适配或微调 (fine-tuned) 的大规模神经网络模型, 称为基础模型 (foundation model)(Bommasani et al., 2021). 这些模型可以充分利用海量且多样化的数据集, 从而构建出具有广泛适用性的通用模型.

人们常常发现, 通过架构创新或引入更复杂形式的归纳偏置而带来的性能改进, 很快就会被简单地通过扩大训练数据量替代, 同时伴随着相应的模型规模和用于训练的相关计算能力的扩展 (Sutton, 2019). 大语言模型不仅在特定任务上表现更优, 还可以用同一个训练好的神经网络解决更广泛范围的问题. 大语言模型就是一个显著的例子, 因为单个网络不仅拥有非凡的能力广度, 甚至还能够胜过为解决特定问题而设计的专业网络 (参见**??**).

除了规模化这一因素外, 还有其他若干进展也同样为深度学习的成功做出了重要贡献. 例如, 在传统的简单神经网络中, 训练信号随着其在深度网络中逐层反向传播而逐渐衰减 (详见**??**). 为了解决这一难题, 一种名为残差连接 (residual connection) 的技术被引入网络架构中 (He et al., 2016), 它极大地提高了对数百层深度的网络的训练效率. 另一项具有里程碑意义的进展是自动微分 (automatic differentiation) 方法的提出, 该方法能够基于描述网络前向传播过程的代码, 自动生成用于执行反向传播以计算误差函数梯度的代码. 研究人员只需要显式地编写相对简洁的前向传播函数代码, 就能够快速地探索和尝试不同的神经网络架构, 并快速尝试不同的架构和多种组件的不同组合. 此外, 机器学习领域的很大一部分研究工作都是基于开源模式开展的, 广大研究人员能够借助” 他山之石” 进行构建和拓展, 从而进一步加快这个领域的创新步伐.

第 2 章 概率

2.1 概率法则

几乎所有机器学习应用场景, 都需要处理不确定性. 例如, 区分皮肤病变是良性还是恶性的图像分类系统, 在实际中不可能万无一失. 不确定性可以分为两种. 第一种是认知不确定性 (epistemic uncertainty), 也称系统不确定性 (systematic uncertainty). 这种不确定性源自有限的数据集大小. 随着观测数据的增多, 例如更多良性和恶性皮肤病变图像样本, 系统可以更好地预测新样本的类别. 然而, 即使数据集无限大, 也仍然无法达到完美的准确率, 因为存在第二种不确定性——偶然不确定性 (aleatoric uncertainty), 也称内在 (intrinsic) 不确定性或随机 (stochastic) 不确定性, 有时简称噪声 (noise). 一般来说, 噪声源自人们只能观察到关于世界的部分信息. 因此, 一种从源头减小这种不确定性的方法是收集不同类型的数据.**??**用扩展到二维的正弦曲线为例说明了这个问题 (参见节 1.2).

2.1.1 医学筛查示例

考虑对人群进行筛查以便尽早发现癌症患者的示例. 假设人群中有 1% 的人患有癌症. 理想情况下, 我们的癌症检查对任何患有癌症的人都会给出阳性结果, 而对任何未患癌症的人都会给出阴性结果. 然而, 癌症检查并不完美, 所以我们假设当对未患癌症的人进行检查时, 其中 3% 的人会被误判为阳性, 这就是所谓的假阳性 (false positive). 类似地, 当检查那些确实患有癌症的人时, 其中 10% 的人会被误判为阴性, 这就是所谓的假阴性 (false negative). 检查结果的准确性如**??**所示.

2.1.2 加和法则和乘积法则

我们可以通过考虑随机变量 X(取值为 $\{x_i\}$, 其中 $i = 1, \cdots, L$) 和 Y(取值为 $\{y_j\}$, 其中 $j = 1, \cdots, M$) 来推导概率的加和法则和乘积法则. 其中,$L = 5$,$M = 3$. 如果考虑总共 N 个实例中的这些变量, 则用 n_{ij} 表示 $X = x_i$ 和 $Y = y_j$ 的实例数, 这是阵列中对应单元格的实例数. 用 c_i 表示第 i 列对应于 $X = x_i$ 的实例数, 用 r_j 表示第 j 行对应于 $Y = y_j$ 的实例数.

X 取值为 x_i 且 Y 取值为 y_j 的概率记为 $p(X = x_i, Y = y_j)$, 称作 $X = x_i$ 和 $Y = y_j$ 的联合 (joint) 概率. 它由落在单元格 (i, j) 中的点数占总点数的比例给出, 因此有

$$p(X = x_i, Y = y_j) = \frac{n_{ij}}{N} \tag{2.1}$$

这里隐式地考虑了极限 $N \to \infty$. 类似地, 在不考虑 Y 取值的情况下,X 取值为 x_i 的概率记为 $p(X = x_i)$, 它由落在第 i 列中的点数占总点数的比例给出, 因此有

$$p(X = x_i) = \sum_{j=1}^{M} p(X = x_i, Y = y_j) \tag{2.2}$$

由于 $\sum_i c_i = N$, 可以看出

$$\sum_{i=1}^{L} p(X = x_i) = 1 \tag{2.3}$$

使用这种更简单的记法, 我们可以将概率论的两个基本法则写成如下形式:

$$p(X) = \sum_{Y} p(X, Y) \quad \text{加和法则} \tag{2.4}$$

$$p(X,Y) = p(Y \mid X)p(X) \quad \text{乘积法则} \tag{2.5}$$

这里 $p(X,Y)$ 是联合概率, 可以表达为"X 和 (and)Y 同时成立的概率". 同样,$p(Y \mid X)$ 是条件概率, 可以表述为"给定 (given)X 下 Y 的概率". 最后,$p(X)$ 是边缘概率, 即"X 的概率". 这两个简单的法则构成了我们在本书中使用的所有概率机制的基础.

2.1.3 贝叶斯定理

根据乘积法则以及对称性 $p(X,Y) = p(Y,X)$, 我们可以得到

$$p(Y \mid X) = \frac{p(X \mid Y)p(Y)}{p(X)} \tag{2.6}$$

这就是著名的贝叶斯定理 (Bayes' theorem). 利用加和法则, 贝叶斯定理的分母可以表示为

$$p(X) = \sum_{Y} p(X \mid Y)p(Y) \tag{2.7}$$

在贝叶斯定理中,$p(Y)$ 称为先验概率 (prior probability), 因为它表示在我们观测到 X 之前,Y 的分布. 相应地,$p(Y \mid X)$ 称为后验概率 (posterior probability), 因为它表示在我们观测到 X 之后,Y 的分布.

2.1.4 再看医学筛查示例

让我们回到之前的癌症筛查示例. 设 $Y = 1$ 表示患有癌症,$Y = 0$ 表示未患癌症. 设 $X = 1$ 表示检查结果为阳性,$X = 0$ 表示检查结果为阴性.

我们知道人群中患癌症的先验概率为 $p(Y = 1) = 0.01$, 因此 $p(Y = 0) = 0.99$.

假阳性率为 $p(X = 1 \mid Y = 0) = 0.03$, 因此 $p(X = 0 \mid Y = 0) = 0.97$.

假阴性率为 $p(X = 0 \mid Y = 1) = 0.10$, 因此 $p(X = 1 \mid Y = 1) = 0.90$.

现在, 假设某人的检查结果为阳性, 我们想知道他真正患有癌症的概率, 即 $p(Y = 1 \mid X = 1)$. 根据贝叶斯定理:

$$p(Y = 1 \mid X = 1) = \frac{p(X = 1 \mid Y = 1)p(Y = 1)}{p(X = 1)} \tag{2.8}$$

分母 $p(X = 1)$ 可以通过加和法则计算:

$$p(X = 1) = p(X = 1 \mid Y = 1)p(Y = 1) + p(X = 1 \mid Y = 0)p(Y = 0) \tag{2.9}$$

代入数值:

$$p(X = 1) = 0.90 \times 0.01 + 0.03 \times 0.99 = 0.009 + 0.0297 = 0.0387 \tag{2.10}$$

因此:

$$p(Y = 1 \mid X = 1) = \frac{0.90 \times 0.01}{0.0387} \approx 0.2325 \tag{2.11}$$

这意味着, 即使检查结果为阳性, 真正患有癌症的概率也只有约 23%. 这是因为癌症在人群中的发病率很低 (先验概率小), 而假阳性率虽然只有 3%, 但由于未患癌症的人数很多, 假阳性的绝对数量仍然很大.

2.2 概率密度

我们除了关注在离散数值集合上定义概率, 还关注连续变量的概率. 例如, 我们可能对一个人的身高在某个范围内的概率感兴趣. 对于连续变量, 我们使用概率密度 (probability density) 来描述概率分布.

如果实值变量 x 落在区间 $(x, x+\delta x)$ 内的概率由 $p(x)\delta x$ 给出, 当 $\delta x \to 0$ 时, 则 $p(x)$ 称为 x 的概率密度. 概率密度满足

$$p(x) \geqslant 0 \tag{2.12}$$

并且满足归一化条件

$$\int_{-\infty}^{\infty} p(x)\,\mathrm{d}x = 1 \tag{2.13}$$

变量 x 落在区间 (a, b) 内的概率由

$$P(x \in (a,b)) = \int_a^b p(x)\,\mathrm{d}x \tag{2.14}$$

给出.

2.2.1 分布的示例

2.2.1.1 均匀分布

最简单的概率分布是均匀分布 (uniform distribution), 其中变量 x 在区间 $[a, b]$ 内的密度为常数:

$$p(x \mid a, b) = \frac{1}{b-a}, \quad x \in [a,b] \tag{2.15}$$

2.2.1.2 高斯分布

另一个非常常见的分布是高斯分布 (Gaussian distribution), 也称为正态分布 (normal distribution). 我们将在节 2.3中详细讨论.

2.2.2 期望和协方差

概率分布的一个重要属性是它的期望 (expectation). 对于离散变量 x, 函数 $f(x)$ 的期望为

$$\mathbb{E}[f] = \sum_x f(x)p(x) \tag{2.16}$$

对于连续变量, 期望为

$$\mathbb{E}[f] = \int p(x)f(x)\,\mathrm{d}x \tag{2.17}$$

类似地, 我们可以考虑两个变量的函数的期望. 例如, 函数 $f(x)$ 的方差 (variance) 定义为

$$\mathrm{var}[f] = \mathbb{E}\left[(f(x) - \mathbb{E}[f])^2\right] \tag{2.18}$$

对于两个随机变量 x 和 y, 我们可以定义协方差 (covariance):

$$\operatorname{cov}[x, y] = \mathbb{E}_{x,y}\left[(x - \mathbb{E}[x])(y - \mathbb{E}[y])\right] \tag{2.19}$$

它表示 x 和 y 同时变化的程度. 如果 x 和 y 相互独立, 则协方差为零.

2.3 高斯分布

高斯分布 (Gaussian distribution), 也称为正态分布 (normal distribution), 是概率论中最重要的分布之一. 对于单变量实值变量 x, 高斯分布定义为

$$\mathcal{N}(x \mid \mu, \sigma^2) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left\{-\frac{(x-\mu)^2}{2\sigma^2}\right\} \tag{2.20}$$

其中 μ 称为均值 (mean),σ^2 称为方差 (variance). 方差的平方根 σ 称为标准差 (standard deviation).

2.3.1 均值和方差

高斯分布的均值为

$$\mathbb{E}[x] = \int_{-\infty}^{\infty} x\,\mathcal{N}(x \mid \mu, \sigma^2)\,\mathrm{d}x = \mu \tag{2.21}$$

方差为

$$\operatorname{var}[x] = \mathbb{E}\left[(x-\mu)^2\right] = \sigma^2 \tag{2.22}$$

高斯分布的最大值出现在 $x = \mu$ 处, 其宽度由参数 σ 控制. 当 σ 较大时, 分布较宽; 当 σ 较小时, 分布较窄.

2.3.2 似然函数

假设我们有一组观测数据 $\mathbf{x} = (x_1, \cdots, x_N)^{\mathsf{T}}$, 我们假设这些数据是从高斯分布中独立同分布抽取的. 给定参数 μ 和 σ^2, 观测到这组数据的概率 (即似然函数) 为

$$p(\mathbf{x} \mid \mu, \sigma^2) = \prod_{n=1}^{N} \mathcal{N}(x_n \mid \mu, \sigma^2) \tag{2.23}$$

在实际应用中, 我们通常使用对数似然函数, 因为对数函数是单调递增的, 最大化对数似然等价于最大化似然:

$$\ln p(\mathbf{x} \mid \mu, \sigma^2) = -\frac{N}{2}\ln(2\pi) - \frac{N}{2}\ln(\sigma^2) - \frac{1}{2\sigma^2}\sum_{n=1}^{N}(x_n - \mu)^2 \tag{2.24}$$

最大似然估计通过最大化对数似然来确定参数 μ 和 σ^2.

对 μ 求导并令其为零, 可以得到均值的最大似然估计:

$$\mu_{\mathrm{ML}} = \frac{1}{N}\sum_{n=1}^{N} x_n \tag{2.25}$$

这就是样本均值.

类似地, 对 σ^2 求导并令其为零, 可以得到方差的最大似然估计:

$$\sigma^2_{\mathrm{ML}} = \frac{1}{N}\sum_{n=1}^{N}(x_n - \mu_{\mathrm{ML}})^2 \tag{2.26}$$

这就是样本方差.

2.3.3 最大似然的偏差

需要注意的是, 最大似然估计得到的方差是有偏的 (biased). 具体来说,

$$\mathbb{E}[\sigma^2_{\mathrm{ML}}] = \frac{N-1}{N}\sigma^2 \tag{2.27}$$

因此, 最大似然估计低估了真实的方差. 为了得到无偏估计, 我们通常使用

$$\tilde{\sigma}^2 = \frac{1}{N-1}\sum_{n=1}^{N}(x_n - \mu_{\mathrm{ML}})^2 \tag{2.28}$$

这个偏差问题在小样本时尤为明显, 但当 N 很大时, 偏差可以忽略不计.

2.3.4 线性回归

在节 1.2中, 我们使用平方和误差函数来拟合多项式曲线. 现在我们可以从概率的角度来理解这个误差函数. 假设目标变量 t 是由输入变量 x 通过某个函数生成的, 但带有高斯噪声:

$$t = y(x, \mathbf{w}) + \epsilon \tag{2.29}$$

其中 ϵ 是均值为 0、方差为 σ^2 的高斯噪声. 因此, 给定 x 和参数 $\mathbf{w}$,t 服从高斯分布:

$$p(t \mid x, \mathbf{w}, \sigma^2) = \mathcal{N}(t \mid y(x, \mathbf{w}), \sigma^2) \tag{2.30}$$

现在, 给定训练数据集 $\{\mathbf{x}, \mathbf{t}\}$, 我们可以写出似然函数:

$$p(\mathbf{t} \mid \mathbf{x}, \mathbf{w}, \sigma^2) = \prod_{n=1}^{N}\mathcal{N}(t_n \mid y(x_n, \mathbf{w}), \sigma^2) \tag{2.31}$$

取对数似然, 我们得到

$$\ln p(\mathbf{t} \mid \mathbf{x}, \mathbf{w}, \sigma^2) = -\frac{N}{2}\ln(2\pi) - \frac{N}{2}\ln(\sigma^2) - \frac{1}{2\sigma^2}\sum_{n=1}^{N}\{y(x_n, \mathbf{w}) - t_n\}^2 \tag{2.32}$$

如果我们假设噪声方差 σ^2 是固定的, 那么最大化对数似然等价于最小化平方和误差函数. 这就是为什么我们在节 1.2中使用平方和误差函数的概率解释.

2.4 密度变换

当我们对随机变量进行变换时, 概率密度也会相应地发生变化. 假设我们有一个随机变量 x, 其概率密度为 $p_x(x)$. 现在, 我们通过一个函数 $y = f(x)$ 对 x 进行变换, 得到新的随机变量 y. 我们想知道 y 的概率密度 $p_y(y)$ 是什么.

对于离散变量, 变换是直接的:y 取某个值的概率等于 x 取所有能映射到该值的值的概率之和.

对于连续变量, 情况稍微复杂一些. 概率密度的变换需要满足: 概率质量在变换前后保持不变. 也就是说, 落在 x 的某个区间内的概率, 应该等于落在变换后对应的 y 的区间内的概率.

如果变换 f 是单调的 (即可逆的), 那么概率密度的变换公式为

$$p_y(y) = p_x(x)\left|\frac{\mathrm{d}x}{\mathrm{d}y}\right| \tag{2.33}$$

其中 x 是 y 的反函数, 即 $x = f^{-1}(y)$.

绝对值的出现是因为我们关心的是概率质量, 而不是有向面积. 如果 f 不是单调的, 那么我们需要将所有能映射到同一个 y 值的 x 值的贡献加起来.

2.5 信息论

信息论 (information theory) 研究的是信息的量化、存储和通信. 在机器学习中, 信息论为概率分布的比较和度量提供了重要的工具.

2.5.1 熵

考虑一个离散随机变量 X, 它可以取 $x_1, \cdots, x_K$ 这 K 个值. 当我们观测到这个变量的一个具体值时, 我们获得了多少信息呢?

信息量的大小应该与事件的概率有关. 一个不太可能发生的事件发生了, 比一个很可能发生的事件发生了, 我们获得的信息量更大.

香农 (Shannon) 提出, 信息量可以定义为

$$h(x) = -\log_2 p(x) \tag{2.34}$$

其中对数的底决定了信息量的单位. 以 2 为底时, 单位是比特 (bit).

熵 (entropy) 是信息量的期望:

$$H[X] = -\sum_{k=1}^{K} p(x_k)\log_2 p(x_k) \tag{2.35}$$

熵表示了观测到随机变量 X 之前, 我们对它的不确定性的度量.

2.5.2 微分熵

对于连续变量, 熵的概念可以推广为微分熵 (differential entropy):

$$H[x] = -\int p(x)\ln p(x)\,\mathrm{d}x \tag{2.36}$$

需要注意的是, 与离散熵不同, 微分熵可以是负的.

2.5.3 Kullback-Leibler 散度

Kullback-Leibler 散度 (KL 散度), 也称为相对熵, 是用来度量两个概率分布之间差异的指标.

对于两个分布 $p(x)$ 和 $q(x)$,KL 散度定义为

$$\mathrm{KL}(p\|q) = -\int p(x)\ln\left\{\frac{q(x)}{p(x)}\right\}\mathrm{d}x = \int p(x)\ln\left\{\frac{p(x)}{q(x)}\right\}\mathrm{d}x \tag{2.37}$$

KL 散度满足

$$\mathrm{KL}(p\|q) \geqslant 0 \tag{2.38}$$

当且仅当 p 和 q 完全相同时,KL 散度为零.

需要注意的是,KL 散度不是对称的, 即 $\mathrm{KL}(p\|q) \neq \mathrm{KL}(q\|p)$.

2.5.4 条件熵

给定随机变量 Y, 随机变量 X 的条件熵定义为

$$H[X \mid Y] = -\sum_{x,y} p(x,y) \ln p(x \mid y) \tag{2.39}$$

它表示在知道 Y 之后,X 的不确定性的大小.

2.5.5 互信息

两个随机变量 X 和 Y 之间的互信息 (mutual information) 定义为

$$I[X,Y] = \mathrm{KL}\left(p(x,y)\|p(x)p(y)\right) \tag{2.40}$$

它表示知道其中一个变量之后, 另一个变量的不确定性减少的程度. 互信息满足

$$I[X,Y] = H[X] - H[X \mid Y] \tag{2.41}$$

以及 $I[X,Y] \geqslant 0$, 当且仅当 X 和 Y 相互独立时, 互信息为零.

2.6 贝叶斯概率

在小节 2.1.3中, 我们介绍了贝叶斯定理. 现在我们从更一般的角度来讨论贝叶斯概率 (Bayesian probability).

频率学派的观点认为, 概率是事件发生的长期频率. 而贝叶斯学派则认为, 概率是对不确定性的度量. 这种观点更普遍, 频率学派的概率只是贝叶斯观点下的一种特殊情况.

2.6.1 模型参数

在机器学习中, 我们通常有一个模型, 它由一些参数 $\mathbf{w}$ 控制. 我们的目标是从数据中学习这些参数.

在频率学派的方法中, 参数 $\mathbf{w}$ 被认为是固定的未知量, 而数据 $\mathcal{D}$ 是从某个分布中随机抽取的. 我们通过最大化似然函数 $p(\mathcal{D} \mid \mathbf{w})$ 来确定参数 $\mathbf{w}$.

而在贝叶斯方法中, 参数 $\mathbf{w}$ 本身被认为是随机变量. 我们对参数 $\mathbf{w}$ 有一个先验分布 $p(\mathbf{w})$, 然后通过观测数据 $\mathcal{D}$ 来更新我们对参数的认识, 得到后验分布 $p(\mathbf{w} \mid \mathcal{D})$.

根据贝叶斯定理:

$$p(\mathbf{w} \mid \mathcal{D}) = \frac{p(\mathcal{D} \mid \mathbf{w})p(\mathbf{w})}{p(\mathcal{D})} \tag{2.42}$$

其中 $p(\mathcal{D} \mid \mathbf{w})$ 是似然函数,$p(\mathbf{w})$ 是先验分布, 分母 $p(\mathcal{D})$ 是证据 (evidence), 用于归一化.

2.6.2 正则化

在小节 1.2.5中, 我们引入了正则化项来控制过拟合. 现在我们可以从贝叶斯的角度来理解正则化.

如果我们选择一个均值为零的高斯先验分布 $p(\mathbf{w})$, 那么后验分布的对数为

$$\ln p(\mathbf{w} \mid \mathcal{D}) = \ln p(\mathcal{D} \mid \mathbf{w}) + \ln p(\mathbf{w}) + 常数 \tag{2.43}$$

最大化后验分布等价于最大化似然函数加上先验的对数. 如果先验是高斯的, 那么 $\ln p(\mathbf{w})$ 项就对应于二次正则化项.

因此, 正则化可以被解释为在参数空间中加入了先验知识, 即我们倾向于较小的参数值.

2.6.3 贝叶斯机器学习

在贝叶斯机器学习中, 我们不只是找到参数的一个点估计, 而是计算参数的完整后验分布. 这使得我们可以量化预测的不确定性.

对于新的输入 x, 我们的预测分布为

$$p(t \mid x, \mathcal{D}) = \int p(t \mid x, \mathbf{w}) p(\mathbf{w} \mid \mathcal{D}) \, \mathrm{d}\mathbf{w} \tag{2.44}$$

这个积分考虑了所有可能的参数值, 根据它们的后验概率进行加权.

贝叶斯方法的一个主要优点是它自然地处理了模型复杂度的问题, 不需要显式的模型选择过程. 然而, 后验分布的计算通常是困难的, 需要近似方法.

第 3 章 标准分布

3.1 离散变量

在本章中, 我们假设观测数据是独立同分布的 (independent and identically distributed, i.i.d.). 在后续章节中, 我们还将探索包含结构化数据的更复杂情形, 到那时, 这一假设将不再成立.

参数化方法的一个局限在于, 它将分布假设为一种特定的函数形式, 而这一做法可能在特定的应用中并不合适. 另一种方法是非参数 (nonparametric) 密度估计方法, 此时, 分布的形式通常取决于数据集的大小. 这类模型仍然包含一些参数, 但这些参数将控制模型的复杂度而不是分布的形式. 在本章的最后, 我们将简述三种典型的非参数化方法, 这三种方法分别基于直方图、最近邻和核函数. 诸如此类的非参数化技术的一个主要局限, 在于它们涉及存储所有训练数据; 换言之, 参数的数量将随数据量的增大而增加, 因此这些方法在面对大型数据集时将变得非常低效. 神经网络拥有大量但数量固定的参数, 基于神经网络的灵活分布, 深度学习同时具备参数模型的高效和非参数模型的泛化性.

首先考虑离散变量的简单分布, 以二元变量为起点, 随后扩展到多状态变量.

3.1.1 伯努利分布

考虑一个二元随机变量 $x \in \{0, 1\}$. 例如,x 可能描述的是抛掷硬币的结果, 其中 $x = 1$ 表示" 正面"、$x = 0$ 表示" 反面". 如图 2.2 所示, 如果这枚硬币有所损坏, 则落地时正面朝上的概率和反面朝上的概率未必相等.$x = 1$ 的概率可以用参数 μ 来表示:

$$p(x = 1 \mid \mu) = \mu \tag{3.1}$$

其中 $0 \leqslant \mu \leqslant 1$, 于是可以得到 $p(x = 0 \mid \mu) = 1 - \mu$.$x$ 的概率分布因此可以写成如下形式:

$$\mathrm{Bern}(x \mid \mu) = \mu^x (1 - \mu)^{1-x} \tag{3.2}$$

这就是伯努利 (Bernoulli) 分布 (见习题 3.1). 伯努利分布是归一化的, 且具有如下均值和方差:

$$\mathbb{E}[x] = \mu \tag{3.3}$$

$$\mathrm{var}[x] = \mu(1 - \mu) \tag{3.4}$$

假设我们有一个包含 x 的观测值的数据集 $\mathcal{D} = \{x_1, \cdots, x_N\}$. 基于所有观测值都是独立地从 $p(x \mid \mu)$ 生成的, 我们可以建立以 μ 为参数的似然方程, 使得

$$p(\mathcal{D} \mid \mu) = \prod_{n=1}^{N} p(x_n \mid \mu) = \prod_{n=1}^{N} \mu^{x_n} (1 - \mu)^{1-x_n} \tag{3.5}$$

我们可以通过最大化似然方程, 或等价地最大化对数似然函数 (对数似然函数是单调函数) 来估计 μ 的值. 伯努利分布的对数似然函数形式如下:

$$\ln p(\mathcal{D} \mid \mu) = \sum_{n=1}^{N} \ln p(x_n \mid \mu) = \sum_{n=1}^{N} \{x_n \ln \mu + (1 - x_n) \ln(1 - \mu)\} \tag{3.6}$$

注意对数似然函数仅依赖 x_n 的 N 个观测值的和 $\sum_{n=1}^{N} x_n$. 该值给出了数据在这一分布下的充分统计量 (sufficient statistic)(参见节 3.4). 令 $\ln p(\mathcal{D} \mid \mu)$ 关于 μ 的导数为 0, 就可以得到最大似然估计:

$$\mu_{\mathrm{ML}} = \frac{1}{N} \sum_{n=1}^{N} x_n \tag{3.7}$$

它又称为样本均值 (sample mean). 如果观测到 $x = 1$(正面) 的次数为 m, 则eq. (3.7)又可写作

$$\mu_{\mathrm{ML}} = \frac{m}{N} \tag{3.8}$$

这样硬币落地时正面朝上的概率就可以在上述最大化似然的框架下, 通过在数据集中计算正面朝上次数的比例得到.

3.1.2 二项分布

我们同样可以给出在包含 N 个观测值的数据集中, 有 m 个观测值为 $x = 1$ 的二元变量 x 的分布. 这一分布称为二项 (binomial) 分布, 由eq. (3.5)可以看出, 它与 $\mu^m(1-\mu)^{N-m}$ 成比例. 为得到归一化系数, 注意在抛掷硬币 N 次的过程中, 我们需要将所有可能掷出 m 次正面朝上的情况加总起来, 因此二项分布可写为如下形式:

$$\mathrm{Bin}(m \mid N, \mu) = \binom{N}{m} \mu^m (1-\mu)^{N-m} \tag{3.9}$$

其中

$$\binom{N}{m} \equiv \frac{N!}{(N-m)!\, m!} \tag{3.10}$$

是从 N 个相同物品中无放回地选择 m 个物品的方法总数.图 3.1展示了一个 $N = 10$、$\mu = 0.25$ 的二项分布 (见习题 3.3).

对于独立的事件, 和的均值就是均值的和, 而和的方差就是方差的和. 用这些结果就能找到二项分布的均值和方差 (见习题 2.10). 由于 $m = x_1 + \cdots + x_N$ 且任意观测的均值和方差均可由eq. (3.3)和eq. (3.4)给出, 故有

$$\mathbb{E}[m] \equiv \sum_{m=0}^{N} m\, \mathrm{Bin}(m \mid N, \mu) = N\mu \tag{3.11}$$

$$\mathrm{var}[m] \equiv \sum_{m=0}^{N} (m - \mathbb{E}[m])^2 \mathrm{Bin}(m \mid N, \mu) = N\mu(1-\mu) \tag{3.12}$$

这些结论也可以用微积分进行证明 (见习题 3.4).

3.1.3 多项分布

二元变量可以用于描述只有两种可能取值的量. 然而通常情况下, 我们遇到的离散变量可能包含 K 个相互无关的状态. 尽管存在多种表示这些变量的方法, 但我们很快将会看到一种非常便捷的"K 中之一"(1-of-K) 的表示方案, 有时又称" 独热编码". 在该方案中, 变量可以表示为一个 K 维的向量 $\boldsymbol{x}$,$\boldsymbol{x}$ 中的一个元素 x_k 取值为 1, 其余元素均取值为 0. 因此, 如果我们有一个变量存在 $K = 6$ 种状态, 且该变量特定的一次观测值刚好对应状态 $x_3 = 1$, 则 $\boldsymbol{x}$ 可以表示为

$$\boldsymbol{x} = (0, 0, 1, 0, 0, 0)^{\mathsf{T}} \tag{3.13}$$

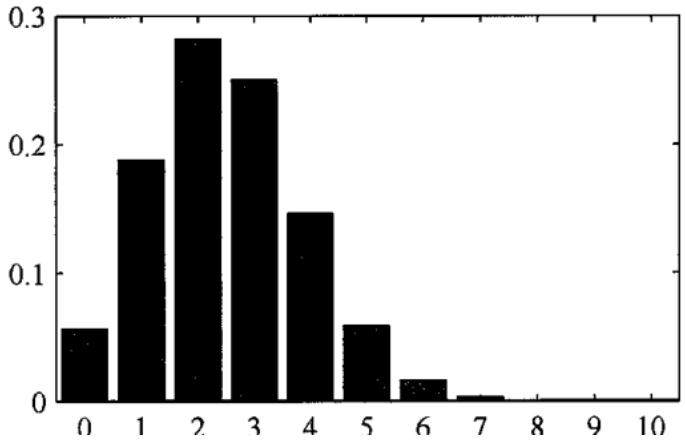


图 3.1: 当 $N = 10$ 且 $\mu = 0.25$ 时, 二项分布式 (3.9) 作为 m 的函数的直方图.

注意, 这些元素满足 $\sum_{k=1}^{K} x_k = 1$. 如果我们将 $x_k = 1$ 的概率记为 μ_k, 则 $\boldsymbol{x}$ 的分布可以由下式给出:

$$p(\boldsymbol{x} \mid \boldsymbol{\mu}) = \prod_{k=1}^{K} \mu_k^{x_k} \tag{3.14}$$

其中 $\boldsymbol{\mu} = (\mu_1, \cdots, \mu_K)^{\mathsf{T}}$, 且因为参数 μ_k 表示的是概率, 它们满足约束 $\mu_k \geqslant 0$ 和 $\sum_{k=1}^{K} \mu_k = 1$.eq. (3.14)所示的分布可以看作将伯努利分布扩展到可以描述超过两种状态. 可以看出, 这一分布是归一化的:

$$\sum_{\boldsymbol{x}} p(\boldsymbol{x} \mid \boldsymbol{\mu}) = \sum_{k=1}^{K} \mu_k = 1 \tag{3.15}$$

且

$$\mathbb{E}[\boldsymbol{x} \mid \boldsymbol{\mu}] = \sum_{\boldsymbol{x}} p(\boldsymbol{x} \mid \boldsymbol{\mu}) \boldsymbol{x} = \boldsymbol{\mu} \tag{3.16}$$

考虑包括 N 个独立观测值 $\boldsymbol{x}_1, \cdots, \boldsymbol{x}_N$ 的数据集 $\mathcal{D}$. 相应的似然函数有如下形式:

$$p(\mathcal{D} \mid \boldsymbol{\mu}) = \prod_{n=1}^{N} \prod_{k=1}^{K} \mu_k^{x_{nk}} = \prod_{k=1}^{K} \mu_k^{(\sum_n x_{nk})} = \prod_{k=1}^{K} \mu_k^{m_k} \tag{3.17}$$

从中可以看出, 包含 N 个数据点的似然函数其实可以只通过 K 个量来定义:

$$m_k = \sum_{n=1}^{N} x_{nk} \tag{3.18}$$

它表示观测值 $x_k = 1$ 的数量, 称为分布的充分统计量 (sufficient statistic)(参见节 3.4). 注意变量 m_k 满足如下约束:$\sum_k m_k = N$. 由 $m_1, \cdots, m_K$ 这组观测计数所定义的分布称为多项 (multinomial) 分布, 它是二项分布在 $K > 2$ 个状态时的推广.

3.2 多元高斯分布

高斯分布又称正态分布, 已广泛用于建模连续变量的分布. 我们已经看到对于单变量 x(参见节 2.2), 高斯分布可以写为如下形式:

$$\mathcal{N}(x \mid \mu, \sigma^2) = \frac{1}{(2\pi\sigma^2)^{1/2}} \exp\left\{-\frac{1}{2\sigma^2}(x-\mu)^2\right\} \tag{3.19}$$

其中 μ 是均值,σ^2 是方差. 对于一个 D 维向量 $\boldsymbol{x}$, 多元高斯分布有如下形式:

$$\mathcal{N}(\boldsymbol{x} \mid \boldsymbol{\mu}, \boldsymbol{\Sigma}) = \frac{1}{(2\pi)^{D/2}|\boldsymbol{\Sigma}|^{1/2}} \exp\left\{-\frac{1}{2}(\boldsymbol{x}-\boldsymbol{\mu})^{\mathsf{T}}\boldsymbol{\Sigma}^{-1}(\boldsymbol{x}-\boldsymbol{\mu})\right\} \tag{3.20}$$

其中 $\boldsymbol{\mu}$ 是 D 维均值向量,$\boldsymbol{\Sigma}$ 是 $D \times D$ 的协方差矩阵,$|\boldsymbol{\Sigma}|$ 表示 $\boldsymbol{\Sigma}$ 的行列式.

在很多不同的场景中都可以看到高斯分布, 因此可以从各种不同的角度来理解它的作用 (参见节 2.2). 例如, 我们已经看到, 对于单个实数变量, 能够使它的熵最大化的分布就是高斯分布, 这一性质对于多元高斯分布同样适用 (见习题 3.8). 当我们考虑多个随机变量之和的时候, 同样会用到高斯分布. 中心极限定理 (central limit theorem) 告诉我们, 在某些温和的条件下, 一组随机变量的和 (它本身也是一个随机变量) 的分布会随着其中随机变量数量的增加而越来越接近于高斯分布 (Walker, 1969). 现实中, 随着 N 的增大, 分布会很快地收敛到高斯分布.

3.2.1 高斯几何

高斯分布有很多重要的分析性质, 本节比前几节更具技术性, 读者需要熟悉多种矩阵相关的等式 (参见附录 A). 高斯分布对 $\boldsymbol{x}$ 的函数依赖是通过二次形式来呈现的:

$$\Delta^2 = (\boldsymbol{x}-\boldsymbol{\mu})^{\mathsf{T}}\boldsymbol{\Sigma}^{-1}(\boldsymbol{x}-\boldsymbol{\mu}) \tag{3.21}$$

其中, 量 Δ 称为 $\boldsymbol{\mu}$ 到 $\boldsymbol{x}$ 的马哈拉诺比斯距离 (Mahalanobis distance). 当 $\boldsymbol{\Sigma}$ 为单位矩阵时, 它退化为欧氏距离. 高斯分布在 $\boldsymbol{x}$ 空间的曲面上是常数, 因为该二次型为常数.

首先, 注意可以不失一般性地假设矩阵 $\boldsymbol{\Sigma}$ 为对称矩阵, 因为任何非对称的成分都会从指数中消失 (见习题 3.11). 考虑协方差矩阵的特征方程:

$$\boldsymbol{\Sigma}\boldsymbol{u}_i = \lambda_i \boldsymbol{u}_i \tag{3.22}$$

其中 $i = 1, \cdots, D$. 由于 $\boldsymbol{\Sigma}$ 是实对称矩阵, 其特征值是实数, 并且其特征向量可以正交化 (见习题 3.12), 故有

$$\boldsymbol{u}_i^{\mathsf{T}}\boldsymbol{u}_j = I_{ij} \tag{3.23}$$

其中 I 是单位矩阵中的元素. 协方差矩阵 $\boldsymbol{\Sigma}$ 可以由其特征向量扩展获得, 并且可以表示为如下形式 (见习题 3.13):

$$\boldsymbol{\Sigma} = \sum_{i=1}^{D} \lambda_i \boldsymbol{u}_i \boldsymbol{u}_i^{\mathsf{T}} \tag{3.24}$$

类似地, 协方差矩阵的逆矩阵 $\boldsymbol{\Sigma}^{-1}$ 可以表示为

$$\boldsymbol{\Sigma}^{-1} = \sum_{i=1}^{D} \frac{1}{\lambda_i} \boldsymbol{u}_i \boldsymbol{u}_i^{\mathsf{T}} \tag{3.25}$$

将eq. (3.25)代入eq. (3.21), 二次型变为

$$\Delta^2 = \sum_{i=1}^{D} \frac{y_i^2}{\lambda_i} \tag{3.26}$$

其中

$$y_i = \boldsymbol{u}_i^{\mathsf{T}}(\boldsymbol{x} - \boldsymbol{\mu}) \tag{3.27}$$

我们可以将 $\{y_i\}$ 解释为一个由正交向量定义的新坐标系, 它由原始 $\boldsymbol{x}$ 的坐标经过平移和旋转得到. 构造向量 $\boldsymbol{y} = (y_1, \cdots, y_D)^{\mathsf{T}}$, 有

$$\boldsymbol{y} = \boldsymbol{U}(\boldsymbol{x} - \boldsymbol{\mu}) \tag{3.28}$$

其中 $\boldsymbol{U}$ 是以 $\boldsymbol{u}_i$ 为行向量的矩阵. 由eq. (3.23)可知 $\boldsymbol{U}$ 是正交矩阵, 满足 $\boldsymbol{U}\boldsymbol{U}^{\mathsf{T}} = \boldsymbol{U}^{\mathsf{T}}\boldsymbol{U} = \boldsymbol{I}$. 由于eq. (3.26)为常数, 因此二次型 (即高斯密度) 在曲面上也为常数. 如果所有特征值 λ_i 均为正, 则这些曲面表示以 $\boldsymbol{\mu}$ 为中心、以 $\boldsymbol{u}_i$ 的方向为坐标轴且以 $\sqrt{\lambda_i}$ 为半轴长的椭球. 为了给出良定的高斯分布, 需要保证协方差矩阵的所有特征值 λ_i 均严格为正, 否则分布就无法被正确归一化. 特征值全部严格为正的矩阵是正定的 (参见第 16 章). 在讨论隐变量模型时, 我们将遇到高斯分布的一个或多个特征值为零的情形, 这时分布是奇异的且限制在一个更低维的子空间中; 如果所有特征值非负, 则称这样的协方差矩阵为半正定的.

3.2.2 矩

下面考虑高斯分布的矩, 并给出对参数 $\boldsymbol{\mu}$ 和 $\boldsymbol{\Sigma}$ 的一种解释.$\boldsymbol{x}$ 在高斯分布下的期望由下式给出:

$$\mathbb{E}[\boldsymbol{x}] = \boldsymbol{\mu} \tag{3.29}$$

$\boldsymbol{\mu}$ 可以视为高斯分布的均值. 接下来考虑高斯分布的二阶矩. 对于多变量的高斯分布, 存在由 $\mathbb{E}[x_i x_j]$ 给出的 D^2 个二阶矩, 我们可以将它们组合成矩阵 $\mathbb{E}[\boldsymbol{x}\boldsymbol{x}^{\mathsf{T}}]$. 这里再一次进行了变量替换 $\boldsymbol{z} = \boldsymbol{x} - \boldsymbol{\mu}$, 交叉项会由于对称性而消去. 最终可以得到

$$\mathbb{E}[\boldsymbol{x}\boldsymbol{x}^{\mathsf{T}}] = \boldsymbol{\mu}\boldsymbol{\mu}^{\mathsf{T}} + \boldsymbol{\Sigma} \tag{3.30}$$

在定义单个随机变量的方差时, 我们在计算二阶矩之前减去了均值. 类似地, 在多元的情形中, 我们再次减去均值, 从而将随机变量 $\boldsymbol{x}$ 的协方差定义为

$$\operatorname{cov}[\boldsymbol{x}] = \mathbb{E}\left[(\boldsymbol{x} - \mathbb{E}[\boldsymbol{x}])(\boldsymbol{x} - \mathbb{E}[\boldsymbol{x}])^{\mathsf{T}}\right] \tag{3.31}$$

对于高斯分布的情形, 利用 $\mathbb{E}[\boldsymbol{x}] = \boldsymbol{\mu}$ 并结合eq. (3.30), 得到

$$\operatorname{cov}[\boldsymbol{x}] = \boldsymbol{\Sigma} \tag{3.32}$$

由于参数矩阵 $\boldsymbol{\Sigma}$ 决定了高斯分布下 $\boldsymbol{x}$ 的协方差, 因此我们称其为协方差矩阵.

3.2.3 局限性

尽管高斯分布式(3.20)经常用作简单的密度模型, 但它也存在一些明显的局限. 考虑分布中自由参数的数量: 一个常规的对称协方差矩阵 $\boldsymbol{\Sigma}$ 含有 $D(D+1)/2$ 个独立的参数 (见习题 3.15),$\boldsymbol{\mu}$ 中还另外存在 D 个独立的参数, 因此总共有 $D(D+3)/2$ 个参数. 对于较大的 D, 参数量按 D 的平方增长, 对较大矩阵的操作和取逆计算代价巨大. 解决这个问题的一种方法是使用协方差矩阵的受限形式. 考虑协方差矩阵是对角的,$\boldsymbol{\Sigma} = \operatorname{diag}(\sigma_i^2)$, 因此密度模型中总共有 $2D$ 个独立参数, 相应的概率密度等值线由轴对齐的椭球体给出. 我们可以进一步将协方差矩阵限制为与

单位矩阵成正比,$\boldsymbol{\Sigma} = \sigma^2\boldsymbol{I}$, 即所谓各向同性的 (isotropic) 协方差, 这时模型中有 $D+1$ 个独立参数, 并且概率密度等值线是一个球面. 常规协方差矩阵、对角协方差矩阵和各向同性协方差矩阵这三种情形如图 3.2所示. 遗憾的是, 虽然这些方法限制了分布中的自由度, 并使协方差矩阵的求逆变得更快, 但它们也极大限制了概率密度的形式, 并限制了模型从数据中捕获有趣相关性的能力.

高斯分布的另一个局限性是, 它本质上是单峰的 (即具有单个最大值), 因此无法为多峰分布提供良好的近似. 稍后我们将看到, 引入潜变量 (也称隐变量或无法观测的变量) 可以同时解决上述问题. 特别地, 通过引入离散潜变量实现高斯混合, 可以获得一大类多峰分布 (参见小节 3.2.9).

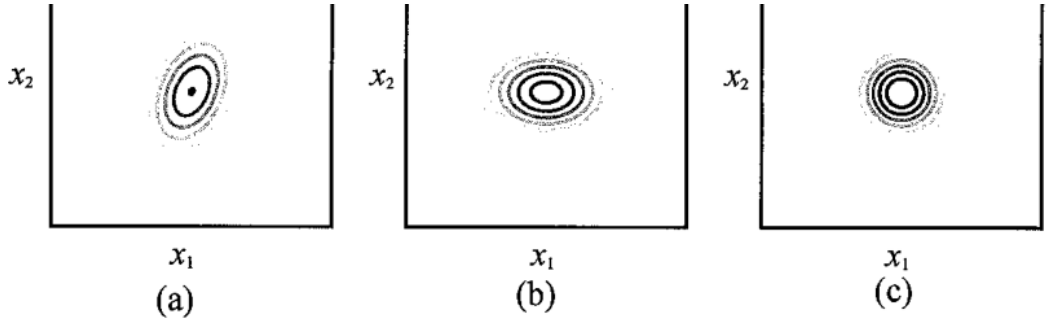


图 3.2: 二维高斯分布的概率密度等值线, 其中协方差矩阵为:(a) 常规协方差矩阵;(b) 对角协方差矩阵, 此时椭圆等值线与坐标轴对齐;(c) 各向同性协方差矩阵, 与单位矩阵成正比, 等值线是同心圆.

3.2.4 条件分布

多元高斯分布的一个重要性质是, 如果两组变量服从联合高斯分布, 那么以其中一组变量为条件的另一组变量的条件分布也是高斯分布; 同样, 其中任意一组变量的边缘分布也是高斯分布.

首先考虑条件分布. 假设 D 维向量 $\boldsymbol{x}$ 服从高斯分布 $\mathcal{N}(\boldsymbol{x} \mid \boldsymbol{\mu}, \boldsymbol{\Sigma})$, 将 $\boldsymbol{x}$ 划分为两个不相交的子集 $\boldsymbol{x}_a$ 和 $\boldsymbol{x}_b$. 在不失一般性的情况下, 取 $\boldsymbol{x}_a$ 为前 M 个分量,$\boldsymbol{x}_b$ 为剩余的 $D-M$ 个分量, 于是

$$\boldsymbol{x} = \begin{pmatrix} \boldsymbol{x}_a \\ \boldsymbol{x}_b \end{pmatrix}, \qquad \boldsymbol{\mu} = \begin{pmatrix} \boldsymbol{\mu}_a \\ \boldsymbol{\mu}_b \end{pmatrix}, \qquad \boldsymbol{\Sigma} = \begin{pmatrix} \boldsymbol{\Sigma}_{aa} & \boldsymbol{\Sigma}_{ab} \\ \boldsymbol{\Sigma}_{ba} & \boldsymbol{\Sigma}_{bb} \end{pmatrix} \tag{3.33}$$

在许多情况下, 使用协方差矩阵的逆矩阵会更方便. 这称为精度矩阵 (precision matrix), 记为

$$\boldsymbol{\Lambda} \equiv \boldsymbol{\Sigma}^{-1} = \begin{pmatrix} \boldsymbol{\Lambda}_{aa} & \boldsymbol{\Lambda}_{ab} \\ \boldsymbol{\Lambda}_{ba} & \boldsymbol{\Lambda}_{bb} \end{pmatrix} \tag{3.34}$$

注意, 协方差矩阵的对称性意味着 $\boldsymbol{\Lambda}_{ab} = \boldsymbol{\Lambda}_{ba}^{\mathsf{T}}$. 从概率的乘积法则中, 只需要将 $\boldsymbol{x}_b$ 固定为观测值并归一化表达式, 就可以基于联合分布 $p(\boldsymbol{x}) = p(\boldsymbol{x}_a, \boldsymbol{x}_b)$ 得到条件分布 $p(\boldsymbol{x}_a \mid \boldsymbol{x}_b)$. 通过在指数中对 $\boldsymbol{x}_a$ 作" 完全平方", 可以得到条件高斯分布的均值和协方差:

$$\boldsymbol{\mu}_{a|b} = \boldsymbol{\mu}_a + \boldsymbol{\Sigma}_{ab}\boldsymbol{\Sigma}_{bb}^{-1}(\boldsymbol{x}_b - \boldsymbol{\mu}_b) \tag{3.35}$$

$$\boldsymbol{\Sigma}_{a|b} = \boldsymbol{\Sigma}_{aa} - \boldsymbol{\Sigma}_{ab}\boldsymbol{\Sigma}_{bb}^{-1}\boldsymbol{\Sigma}_{ba} \tag{3.36}$$

因此条件分布为

$$p(\boldsymbol{x}_a \mid \boldsymbol{x}_b) = \mathcal{N}(\boldsymbol{x}_a \mid \boldsymbol{\mu}_{a|b}, \boldsymbol{\Sigma}_{a|b}) \tag{3.37}$$

注意, 条件分布的均值 $\boldsymbol{\mu}_{a|b}$ 是 $\boldsymbol{x}_b$ 的线性函数, 而协方差 $\boldsymbol{\Sigma}_{a|b}$ 与 $\boldsymbol{x}_b$ 无关. 这是线性-高斯 (linear-Gaussian) 模型的一个示例 (参见**??**).

3.2.5 边缘分布

如果联合分布 $p(\boldsymbol{x}_a, \boldsymbol{x}_b)$ 是高斯分布, 则边缘分布

$$p(\boldsymbol{x}_a) = \int p(\boldsymbol{x}_a, \boldsymbol{x}_b)\, \mathrm{d}\boldsymbol{x}_b \tag{3.38}$$

也是高斯分布. 通过对 $\boldsymbol{x}_b$ 作完全平方以完成积分, 可以得到边缘分布的均值和协方差:

$$\mathbb{E}[\boldsymbol{x}_a] = \boldsymbol{\mu}_a \tag{3.39}$$

$$\mathrm{cov}[\boldsymbol{x}_a] = \boldsymbol{\Sigma}_{aa} \tag{3.40}$$

因此边缘分布为

$$p(\boldsymbol{x}_a) = \mathcal{N}(\boldsymbol{x}_a \mid \boldsymbol{\mu}_a, \boldsymbol{\Sigma}_{aa}) \tag{3.41}$$

可以看到, 对于边缘分布, 均值和协方差用分量协方差矩阵表达最简单; 而对于条件分布, 均值和协方差用分量精度矩阵表达最简单.

3.2.6 贝叶斯定理

在前面两节中, 我们将向量 $\boldsymbol{x}$ 分成两个子向量, 然后找到了条件分布和边缘分布的表达式. 这里考虑更一般的情形: 假设我们有高斯边缘分布 $p(\boldsymbol{x})$ 和高斯条件分布 $p(\boldsymbol{y} \mid \boldsymbol{x})$, 其中 $p(\boldsymbol{y} \mid \boldsymbol{x})$ 的均值是 $\boldsymbol{x}$ 的线性函数, 协方差与 $\boldsymbol{x}$ 无关:

$$p(\boldsymbol{x}) = \mathcal{N}(\boldsymbol{x} \mid \boldsymbol{\mu}, \boldsymbol{\Lambda}^{-1}) \tag{3.42}$$

$$p(\boldsymbol{y} \mid \boldsymbol{x}) = \mathcal{N}(\boldsymbol{y} \mid \boldsymbol{A}\boldsymbol{x} + \boldsymbol{b}, \boldsymbol{L}^{-1}) \tag{3.43}$$

其中 $\boldsymbol{\mu}$、$\boldsymbol{A}$ 和 $\boldsymbol{b}$ 是控制均值的参数,$\boldsymbol{\Lambda}$ 和 $\boldsymbol{L}$ 是精度矩阵. 这种结构在好几种生成式模型中都有出现 (参见第 16 章). 通过对联合分布的对数作完全平方, 可以推导出边缘分布 $p(\boldsymbol{y})$ 和条件分布 $p(\boldsymbol{x} \mid \boldsymbol{y})$:

$$p(\boldsymbol{y}) = \mathcal{N}(\boldsymbol{y} \mid \boldsymbol{A}\boldsymbol{\mu} + \boldsymbol{b},\ \boldsymbol{L}^{-1} + \boldsymbol{A}\boldsymbol{\Lambda}^{-1}\boldsymbol{A}^{\mathsf{T}}) \tag{3.44}$$

$$p(\boldsymbol{x} \mid \boldsymbol{y}) = \mathcal{N}\big(\boldsymbol{x} \mid\ \boldsymbol{\Sigma}\{\boldsymbol{\Lambda}\boldsymbol{\mu} + \boldsymbol{A}^{\mathsf{T}}\boldsymbol{L}(\boldsymbol{y} - \boldsymbol{b})\},\ (\boldsymbol{\Lambda} + \boldsymbol{A}^{\mathsf{T}}\boldsymbol{L}\boldsymbol{A})^{-1}\big) \tag{3.45}$$

其中

$$\boldsymbol{\Sigma} = (\boldsymbol{\Lambda} + \boldsymbol{A}^{\mathsf{T}}\boldsymbol{L}\boldsymbol{A})^{-1} \tag{3.46}$$

对这种条件分布进行估计可以看作贝叶斯定理的一个示例:$p(\boldsymbol{x})$ 称为 $\boldsymbol{x}$ 的先验分布, 条件分布 $p(\boldsymbol{x} \mid \boldsymbol{y})$ 表示观测到 $\boldsymbol{y}$ 之后 $\boldsymbol{x}$ 所对应的后验分布.

3.2.7 最大似然

给定数据集 $X = \{\boldsymbol{x}_1, \cdots, \boldsymbol{x}_N\}$, 假设其中的观测值 $\{\boldsymbol{x}_n\}$ 服从多元高斯分布且相互独立, 则可以通过最大似然来估计分布的参数. 对数似然函数由下式给出:

$$\ln p(X \mid \boldsymbol{\mu}, \boldsymbol{\Sigma}) = -\frac{ND}{2}\ln(2\pi) - \frac{N}{2}\ln|\boldsymbol{\Sigma}| - \frac{1}{2}\sum_{n=1}^{N}(\boldsymbol{x}_n - \boldsymbol{\mu})^{\mathsf{T}}\boldsymbol{\Sigma}^{-1}(\boldsymbol{x}_n - \boldsymbol{\mu}) \tag{3.47}$$

令导数为零, 即可得到均值的最大似然估计解:

$$\boldsymbol{\mu}_{\mathrm{ML}} = \frac{1}{N}\sum_{n=1}^{N}\boldsymbol{x}_n \tag{3.48}$$

这是观测数据点集的平均值. 协方差的最大似然解为

$$\boldsymbol{\Sigma}_{\mathrm{ML}} = \frac{1}{N}\sum_{n=1}^{N}(\boldsymbol{x}_n - \boldsymbol{\mu}_{\mathrm{ML}})(\boldsymbol{x}_n - \boldsymbol{\mu}_{\mathrm{ML}})^{\mathsf{T}} \tag{3.49}$$

如果我们估计真实分布下最大似然解的期望, 则有 $\mathbb{E}[\boldsymbol{\mu}_{\mathrm{ML}}] = \boldsymbol{\mu}$, 而 $\mathbb{E}[\boldsymbol{\Sigma}_{\mathrm{ML}}] = \dfrac{N-1}{N}\boldsymbol{\Sigma}$. 也就是说, 协方差的最大似然估计期望小于真实值, 因此它是有偏的. 通过定义一个不同的估计量可以修正这一偏差:

$$\boldsymbol{\Sigma} = \frac{N}{N-1}\boldsymbol{\Sigma}_{\mathrm{ML}} \tag{3.50}$$

3.2.8 序贯估计

对最大似然解的讨论采用了批处理 (batch processing) 方法, 即一次性考虑整个训练集. 也可以使用序贯 (sequential) 方法, 该方法允许一次处理一个数据点, 然后将其丢弃. 这对于在线应用和大数据非常重要. 考虑eq. (3.48)给出的最大似然估计, 若分解出最末尾的数据点 $\boldsymbol{x}_N$ 的贡献, 则有

$$\boldsymbol{\mu}_{\mathrm{ML}}^{(N)} = \boldsymbol{\mu}_{\mathrm{ML}}^{(N-1)} + \frac{1}{N}(\boldsymbol{x}_N - \boldsymbol{\mu}_{\mathrm{ML}}^{(N-1)}) \tag{3.51}$$

这个结果可以这样解释: 在观测到 $N-1$ 个数据点后, 我们用 $\boldsymbol{\mu}_{\mathrm{ML}}^{(N-1)}$ 来估计 $\boldsymbol{\mu}$; 而在观测到数据点 $\boldsymbol{x}_N$ 后, 我们可以通过将旧的估计值在" 误差信号"$(\boldsymbol{x}_N - \boldsymbol{\mu}_{\mathrm{ML}}^{(N-1)})$ 的方向上移动一个较小的量来获得修订后的估计值, 这个较小的量与 $1/N$ 成正比. 注意, 随着 N 的增加, 来自后续数据点的贡献会越来越小.

3.2.9 高斯混合

尽管高斯分布具有一些重要的分析特性, 但在用于对真实数据集建模时, 它存在很大的局限性. 例如美国黄石国家公园老忠实 (Old Faithful) 喷泉数据集, 包含 272 个观测值, 每个观测值都以分钟为单位给出喷发的持续时间和下一次喷发的间隔时间. 该数据集形成了两个主要的簇, 使用单一的高斯分布无法捕捉这种结构——最大似然拟合的单一高斯会把大部分概率质量放置在数据相对稀疏的簇间区域. 可以预期, 叠加两个高斯分布将能够更好地表示该数据集的结构.

这种叠加是通过对更基本的分布 (如高斯分布) 进行线性组合来实现的, 称作混合分布 (mixture distribution). 考虑形如下式的 K 个高斯密度的叠加:

$$p(\boldsymbol{x}) = \sum_{k=1}^{K}\pi_k\,\mathcal{N}(\boldsymbol{x} \mid \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k) \tag{3.52}$$

这称为高斯混合 (mixture of Gaussians). 每个高斯密度 $\mathcal{N}(\boldsymbol{x} \mid \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k)$ 称为混合的一个分量 (component), 且具有自己的均值 $\boldsymbol{\mu}_k$ 和协方差 $\boldsymbol{\Sigma}_k$. 参数 π_k 称为混合系数 (mixing coefficient). 在eq. (3.52)两边对 $\boldsymbol{x}$ 进行积分, 并注意到 $p(\boldsymbol{x})$ 和单个高斯分量都可以归一化, 便可得到

$$\sum_{k=1}^{K} \pi_k = 1 \tag{3.53}$$

此外, 给定 $\mathcal{N}(\boldsymbol{x} \mid \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k) \geqslant 0$, 则 $p(\boldsymbol{x}) \geqslant 0$ 的充分条件是对所有 k 都有 $\pi_k \geqslant 0$. 将其与eq. (3.53)相结合, 可以得到 $0 \leqslant \pi_k \leqslant 1$, 因此混合系数满足将其作为概率的要求.

根据概率的加和法则与乘积法则, 边缘密度可以写为

$$p(\boldsymbol{x}) = \sum_{k=1}^{K} p(k)\, p(\boldsymbol{x} \mid k) \tag{3.54}$$

这等价于eq. (3.52), 其中 $\pi_k = p(k)$ 可以视为选择第 k 个分量的先验概率. 将密度 $\mathcal{N}(\boldsymbol{x} \mid \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k) = p(\boldsymbol{x} \mid k)$ 视为以 k 为条件时 $\boldsymbol{x}$ 的概率. 根据贝叶斯定理, 相应的后验概率 $p(k \mid \boldsymbol{x})$ 称为责任 (responsibility), 由下式给出:

$$\gamma_k(\boldsymbol{x}) \equiv p(k \mid \boldsymbol{x}) = \frac{\pi_k\, \mathcal{N}(\boldsymbol{x} \mid \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k)}{\sum_{j=1}^{K} \pi_j\, \mathcal{N}(\boldsymbol{x} \mid \boldsymbol{\mu}_j, \boldsymbol{\Sigma}_j)} \tag{3.55}$$

通过使用足够数量的高斯分布, 并调整它们的均值、协方差以及线性组合中的系数, 我们可以把几乎任何连续分布近似到任意准确度. 这种对混合分布的概率解释是非常强大的 (参见第 15 章).

3.3 周期变量

尽管高斯分布本身具有重要的实际意义, 并且作为构建更复杂概率模型的基础也十分有用, 但在某些情况下, 它们并不适合作为连续变量的密度模型. 实际应用中存在的周期变量就属于这种情况.

特定地理位置的风向是周期变量的一个例子. 另一个例子是日历时间, 我们想建模那些可能在 24 小时或一年内具有周期性的量. 这样的量可以使用 $0 \leqslant \theta < 2\pi$ 范围内的角 (极) 坐标来方便地表示. 我们可以选择某个方向作为起始方向, 然后应用高斯分布等常规分布来处理周期变量, 然而这种方法所产生的结果极其依赖于我们随意选择的起始方向. 例如, 设有两个观测值 $\theta_1 = 1°$ 和 $\theta_2 = 359°$, 并且使用标准的一元高斯分布对它们建模. 如果将起始方向设置为 0°, 那么该数据集的样本均值为 180°, 标准差为 179°; 而如果将起始方向设置为 180°, 那么该数据集的样本均值为 0°, 标准差为 1°. 显然, 我们需要一种专门用来处理周期变量的方法.

考虑估计周期变量 θ 的一组观测值 $\mathcal{D} = \{\theta_1, \cdots, \theta_N\}$ 的均值, 其中 θ 的单位为弧度. 简单计算均值 $(\theta_1 + \cdots + \theta_N)/N$ 将与坐标系强相关. 为了找到均值的不变度量, 我们注意到观测值可以视为单位圆上的点, 因此可以用二维单位向量 $\boldsymbol{x}_1, \cdots, \boldsymbol{x}_N$ 来描述它们. 对向量 $\{\boldsymbol{x}_n\}$ 求平均值, 可得

$$\bar{\boldsymbol{x}} = \frac{1}{N} \sum_{n=1}^{N} \boldsymbol{x}_n, \qquad \boldsymbol{x}_n = (\cos\theta_n, \sin\theta_n)^{\mathsf{T}} \tag{3.56}$$

然后找到与平均值对应的角度 $\bar{\theta}$. 显然, 上述定义将确保均值的位置与角坐标的起始方向无关. 注意 $\bar{\boldsymbol{x}}$ 通常位于单位圆内. 观测值的笛卡儿坐标由 $\bar{\boldsymbol{x}} = (\bar{r}\cos\bar{\theta}, \bar{r}\sin\bar{\theta})$ 给出. 代入eq. (3.56)并使 $\bar{x}_1$ 和 $\bar{x}_2$ 分量相等, 对二者取比值, 并使用恒等式 $\tan\theta = \sin\theta/\cos\theta$,$\bar{\theta}$ 可由下式求解:

$$\bar{\theta} = \tan^{-1}\left(\frac{\sum_n \sin\theta_n}{\sum_n \cos\theta_n}\right) \tag{3.57}$$

读者很快就会看到这个结果是如何作为最大似然估计量而产生的.

按照惯例, 考虑分布 $p(\theta)$, 其具有 2π 周期. 任何定义在 θ 上的概率密度 $p(\theta)$ 都必须是非负的且积分为 1, 同时还必须具有周期性. 因此,$p(\theta)$ 必须满足以下三个条件:

$$p(\theta) \geqslant 0, \qquad \int_0^{2\pi} p(\theta)\, \mathrm{d}\theta = 1, \qquad p(\theta + 2\pi) = p(\theta) \tag{3.58}$$

首先定义高斯分布的一个周期性推广, 称为冯 · 米塞斯 (von Mises) 分布. 考虑二维空间上的高斯分布, 其均值为 $\boldsymbol{\mu} = (\mu_1, \mu_2)^{\mathsf{T}}$, 协方差矩阵为 $\sigma^2 \boldsymbol{I}$. 通过从笛卡儿坐标 (x_1, x_2) 转换到极坐标 (r, θ), 并关注沿单位圆 $r = 1$ 这个分布的值, 可以导出周期分布. 利用三角恒等式

$$\cos^2 A + \sin^2 A = 1, \qquad \cos A \cos B + \sin A \sin B = \cos(A - B) \tag{3.59}$$

最终可以得到 $p(\theta)$ 沿单位圆的分布形式:

$$p(\theta \mid \theta_0, m) = \frac{1}{2\pi I_0(m)} \exp\big\{m \cos(\theta - \theta_0)\big\} \tag{3.60}$$

这称为冯 · 米塞斯分布或圆形正态 (circular normal) 分布. 这里的参数 θ_0 对应于分布的均值,m 则称为聚焦 (concentration) 参数, 类似于高斯分布的逆方差 (即精度). 式(3.60)中的归一化系数用 $I_0(m)$ 表示, 它是第一类零阶修正贝塞尔函数 (modified Bessel function), 定义如下:

$$I_0(m) = \frac{1}{2\pi} \int_0^{2\pi} \exp\{m \cos\theta\} \, \mathrm{d}\theta \tag{3.61}$$

当 m 较大时, 该分布近似为高斯分布 (见习题 3.31). 关于参数 θ_0 和 m 的最大似然估计, 可以通过对对数似然函数求导得到, 其中 θ_0 的解与eq. (3.57)一致. 冯 · 米塞斯分布的一个局限性是它是单峰的; 我们可以通过构造包含多个冯 · 米塞斯分布的混合分布, 来获得一个更为灵活的周期变量建模框架, 以处理多峰数据. 最后, 实轴上的任何有效分布 (如高斯分布) 都可以通过将宽度为 2π 的连续区间映射到周期变量 $(0, 2\pi)$ 上, 成为周期分布, 这相当于将实轴" 包裹" 到单位圆上 (环绕分布).

3.4 指数族分布

到目前为止, 本章所研究的概率分布 (混合模型除外) 都是指数族 (exponential family) 分布 (Duda and Hart, 1973; Bernardo and Smith, 1994) 这一大类里的具体例子. 指数族分布的成员具有许多共同的重要性质, 对这些性质的一般性讨论可以带来不少启发.

给定参数 $\boldsymbol{\eta}$, 关于 $\boldsymbol{x}$ 的指数族分布可以定义为以下形式的分布的集合:

$$p(\boldsymbol{x} \mid \boldsymbol{\eta}) = h(\boldsymbol{x})\, g(\boldsymbol{\eta}) \exp\big\{\boldsymbol{\eta}^{\mathsf{T}} \boldsymbol{u}(\boldsymbol{x})\big\} \tag{3.62}$$

其中 $\boldsymbol{x}$ 可以是标量或矢量, 可以是离散的或连续的.$\boldsymbol{\eta}$ 则称为分布的自然参数 (natural parameters),$\boldsymbol{u}(\boldsymbol{x})$ 是 $\boldsymbol{x}$ 的某个函数. 函数 $g(\boldsymbol{\eta})$ 可以解释为用来确保分布被归一化的系数, 因此满足

$$g(\boldsymbol{\eta}) \int h(\boldsymbol{x}) \exp\big\{\boldsymbol{\eta}^{\mathsf{T}} \boldsymbol{u}(\boldsymbol{x})\big\} \, \mathrm{d}\boldsymbol{x} = 1 \tag{3.63}$$

其中 $\boldsymbol{x}$ 若为离散变量, 则积分改为求和.

下面从本章前面介绍的分布示例入手, 说明它们确实是指数族分布的成员. 首先考虑伯努利分布:

$$p(x \mid \mu) = \mathrm{Bern}(x \mid \mu) = \mu^x (1 - \mu)^{1 - x} \tag{3.64}$$

将其表示为对数的指数, 可得 $p(x \mid \mu) = \exp\{x \ln \mu + (1 - x) \ln(1 - \mu)\}$. 与eq. (3.62)比较后, 令 $\eta = \ln\{\mu/(1 - \mu)\}$, 对 μ 进行求解可得 $\mu = \sigma(\eta)$, 其中

$$\sigma(\eta) = \frac{1}{1 + \exp(-\eta)} \tag{3.65}$$

它又称为 sigmoid 函数. 因此伯努利分布可以写成标准指数族形式 $p(x \mid \eta) = \sigma(-\eta) \exp\{\eta x\}$, 其中 $\boldsymbol{u}(x) = x$,$h(x) = 1$.

接下来考虑多项分布. 对于单个观测 $\boldsymbol{x}$, 其形式为 $p(\boldsymbol{x} \mid \boldsymbol{\mu}) = \prod_k \mu_k^{x_k}$. 同样可以写成标准指数族形式, 其中 $\boldsymbol{u}(\boldsymbol{x}) = \boldsymbol{x}, h(\boldsymbol{x}) = 1$. 通过对参数施加 $\sum_k \mu_k = 1$ 的约束, 可以将 μ_k 用其余 $M-1$ 个参数表示, 从而得到

$$\mu_k = \frac{\exp\{\eta_k\}}{\sum_j \exp\{\eta_j\}} \tag{3.66}$$

它又称为 softmax 函数或归一化指数 (normalized exponential). 最后考虑高斯分布, 经过简单的重排, 一元高斯分布同样可以写成eq. (3.62)的标准形式 (见习题 3.35).

考虑使用最大似然来估计一般指数族分布中的参数向量 $\boldsymbol{\eta}$. 在eq. (3.63)两边对 $\boldsymbol{\eta}$ 求导并重排, 可以得到

$$-\nabla \ln g(\boldsymbol{\eta}) = \mathbb{E}[\boldsymbol{u}(\boldsymbol{x})] \tag{3.67}$$

因此, 如果可以对指数族分布进行归一化, 则总是可以通过对归一化函数进行简单的微分来找到它的矩. 考虑一组独立同分布数据 $X = \{\boldsymbol{x}_1, \cdots, \boldsymbol{x}_N\}$, 令对数似然对 $\boldsymbol{\eta}$ 的梯度为零, 可以得出最大似然估计 $\boldsymbol{\eta}_{\mathrm{ML}}$ 必须满足

$$-\nabla \ln g(\boldsymbol{\eta}_{\mathrm{ML}}) = \frac{1}{N} \sum_{n=1}^{N} \boldsymbol{u}(\boldsymbol{x}_n) \tag{3.68}$$

可以看到, 最大似然估计量的解仅通过 $\sum_n \boldsymbol{u}(\boldsymbol{x}_n)$ 依赖于数据, 因此 $\boldsymbol{u}(\boldsymbol{x})$ 又称为分布式(3.62)的充分统计量. 我们不需要存储整个数据集本身, 而只需要存储充分统计量的值. 例如, 对于伯努利分布,$\boldsymbol{u}(x)$ 仅由 x 给出, 因此只需要存储 $\{x_n\}$ 的和; 而对于高斯分布,$\boldsymbol{u}(x) = (x, x^2)^{\mathsf{T}}$, 因此应该保留 $\{x_n\}$ 的和与 $\{x_n^2\}$ 的和.

3.5 非参数化方法

本章专注于使用由少量参数决定的、具有特定函数形式的概率分布, 这些参数的值可从数据集中确定. 这称为参数化 (parametric) 的密度估计方法. 这种方法的一个关键局限是, 所选择的密度模型可能不适合数据分布, 这可能导致预测性能不佳. 例如, 如果生成数据的过程是多峰的, 那么高斯分布永远无法捕捉到分布这一角度的特征, 因为高斯分布必然是单峰的. 本节将考虑一些非参数化 (nonparametric) 的密度估计方法, 这些方法对分布的形式几乎不作任何假设.

3.5.1 直方图

首先讨论密度估计的直方图方法. 标准直方图简单地将 x 划分为互不相交的宽度为 Δ 的分箱, 然后计算落在不同分箱中的观测值 x 的数量 n_j. 为了将此计数值转换为归一化的概率密度, 可以将其简单地除以观测值总数 N 和分箱宽度 Δ 的乘积:

$$p_j = \frac{n_j}{N\Delta} \tag{3.69}$$

这种方法就给出了密度 $p(x)$ 的一种模型, 在每个分箱的宽度 Δ 上, 密度是恒定的. 通常选择具有相同宽度的分箱.图 3.3展示了直方图密度估计的示例: 当 Δ 非常小时, 得到的密度模型非常毛糙, 具有许多在生成数据集的底层分布中不存在的结构; 相反, 如果 Δ 太大, 得到的密度模型将太过平滑, 因此无法捕捉到双峰特性. 最佳结果是 Δ 取某个中间值.

直方图方法的一个主要局限是, 分箱边界导致估计的密度是不连续的, 而且规模会随维度显著改变. 如果将 D 维空间中的每个变量划分为 M 个区间, 那么分箱将有 M^D 个, 这种随 D 指数级扩展的特性是维度诅咒 (curse of dimensionality) 的一个例子. 在实践中, 直方图技术可以用于快速可视化一维或二维数据, 但不适用于大多数数据密度估计应用.

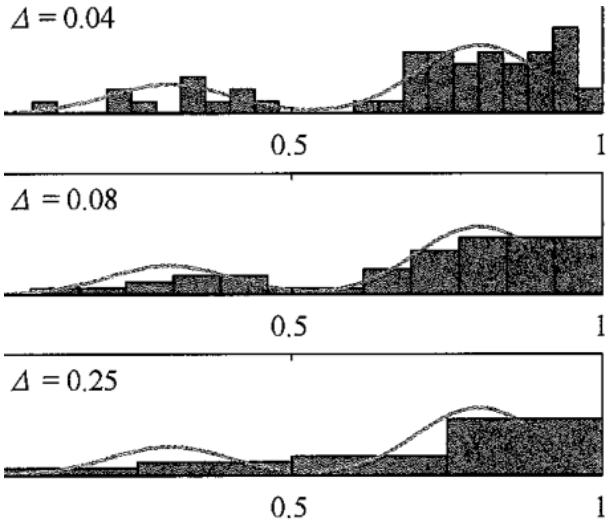


图 3.3: 直方图密度估计的示例, 根据绿色曲线所示的双峰混合分布生成含 50 个数据点的数据集. 三种分箱宽度 $\Delta = 0.04, 0.08, 0.25$ 分别对应过毛糙、适中、过平滑三种结果.

3.5.2 核密度

假设观测值是从某个 D 维空间中的未知概率密度 $p(\boldsymbol{x})$ 中产生的. 考虑包含 $\boldsymbol{x}$ 的一个小区域 $\mathcal{R}$, 与该区域相关联的概率质量为

$$P = \int_{\mathcal{R}} p(\boldsymbol{x})\,\mathrm{d}\boldsymbol{x} \tag{3.70}$$

因为每个数据点在 $\mathcal{R}$ 内的概率为 P, 所以 $\mathcal{R}$ 内的点的总数 K 将服从二项分布. 如果我们假设区域 $\mathcal{R}$ 足够小, 以至于概率密度 $p(\boldsymbol{x})$ 在该区域内大致保持恒定, 则 $P \approx p(\boldsymbol{x})V$, 其中 V 是区域 $\mathcal{R}$ 的体积. 结合起来, 可以得到密度估计的形式:

$$p(\boldsymbol{x}) = \frac{K}{NV} \tag{3.71}$$

注意, 此式的有效性依靠两个相互矛盾的假设: 区域 $\mathcal{R}$ 足够小, 以至于密度在该区域内近似恒定; 但区域 $\mathcal{R}$ 又足够大, 使得落入该区域的点的数量 K 足以使二项分布变得十分尖锐.

可以通过两种不同的方式利用eq. (3.71). 一种是固定 K 并根据数据确定 V 的值, 这将引出 K 近邻技术; 另一种是固定 V 并根据数据确定 K 的值, 这将引出核密度估计技术. 对于核密度, 将区域 $\mathcal{R}$ 取为以 $\boldsymbol{x}$ 为中心、边长为 h 的超小立方体, 并定义核函数 (又称 Parzen 窗)

$$k(\boldsymbol{u}) = \begin{cases} 1, & |u_i| \leqslant 1/2 \text{ 对所有} i \\ 0, & \text{其他} \end{cases} \tag{3.72}$$

它表示以原点为中心的单位立方体. 将其代入eq. (3.71), 可得点 $\boldsymbol{x}$ 处估计的密度为

$$p(\boldsymbol{x}) = \frac{1}{N}\sum_{n=1}^{N} \frac{1}{h^D} k\left(\frac{\boldsymbol{x} - \boldsymbol{x}_n}{h}\right) \tag{3.73}$$

其中 h^D 是 D 维空间中边长为 h 的超小立方体的体积. 如果选择更平滑的核函数 (例如高斯函数), 则可以获得更平滑的密度模型. 使用高斯核时, 导出的核密度模型为

$$p(\boldsymbol{x}) = \frac{1}{N}\sum_{n=1}^{N}\mathcal{N}(\boldsymbol{x} \mid \boldsymbol{x}_n, h^2\boldsymbol{I}) \tag{3.74}$$

其中 h 代表所使用的高斯分量的标准差. 只要核函数满足 $k(\boldsymbol{u}) \geqslant 0$ 且 $\int k(\boldsymbol{u})\,\mathrm{d}\boldsymbol{u} = 1$, 这两个条件就确保了得到的概率分布在任何地方都是非负的并且积分为 1. 参数 h 充当平滑参数:h 太小时对噪声敏感,h 太大时则过度平滑.

3.5.3 最近邻

另一种方法是固定 K 值并允许小球的半径增长, 直至恰好包含 K 个数据点. 对于希望估计密度 $p(\boldsymbol{x})$ 的点 $\boldsymbol{x}$, 考虑以 $\boldsymbol{x}$ 为中心的小球, 并允许小球的半径增长, 直至恰好包含 K 个数据点. 这种技术称为 K 近邻 (K nearest neighbor) 方法. 然后, 密度 $p(\boldsymbol{x})$ 的估计由eq. (3.71)给出, 其中 V 设置为小球的体积. 参数 K 控制了模型的平滑程度, 并且同样存在一个既不太大也不太小的最佳 K 值.

K 近邻技术可以扩展到分类问题. 将 K 近邻密度估计分别应用于每个类别 C_k, 然后利用贝叶斯定理得到类别成员的后验概率. 假设数据集包含 $\sum_k N_k = N$ 个点, 其中 N_k 个属于类别 C_k. 以新点 $\boldsymbol{x}$ 为中心绘制一个恰好包含 K 个点的球体, 该球体的体积为 V 且包含来自类别 C_k 的 K_k 个点. 则条件密度、无条件密度和类别先验分别为

$$p(\boldsymbol{x} \mid C_k) = \frac{K_k}{N_k V}, \qquad p(\boldsymbol{x}) = \frac{K}{NV}, \qquad p(C_k) = \frac{N_k}{N} \tag{3.75}$$

由贝叶斯定理可得后验概率 $p(C_k \mid \boldsymbol{x}) = K_k/K$. 这种方法的一个优点是在” 训练” 阶段不涉及计算, 因为只需要将训练集存储起来即可; 然而, 评估密度的计算成本会随着数据集的增大而线性增长, 这是这类密度模型的一个巨大缺点.

第 4 章 单层网络: 回归

4.1 线性回归

给定 D 维输入变量 $\boldsymbol{x}$, 回归的目标是预测一个或多个连续的目标变量 t. 通常我们会有一个包含 N 个观测值的训练数据集 $\{\boldsymbol{x}_n\}$, 其中 $n = 1, \cdots, N$, 以及相应的目标值 $\{t_n\}$. 目标是为新的 $\boldsymbol{x}$ 值预测对应的 t 值. 为此, 我们定义了函数 $y(\boldsymbol{x}, \boldsymbol{w})$ 来预测新的输入变量 $\boldsymbol{x}$ 所对应的 t 值, 其中 $\boldsymbol{w}$ 表示可以从训练数据中学习的参数向量.

最简单的回归模型是输入变量的线性组合:

$$y(\boldsymbol{x}, \boldsymbol{w}) = w_0 + w_1 x_1 + \cdots + w_D x_D \tag{4.1}$$

其中 $\boldsymbol{x} = (x_1, \cdots, x_D)^{\mathsf{T}}$. 线性回归 (linear regression) 有时特指这种形式的模型. 这种模型的关键性质在于它不仅是参数 $w_0, \cdots, w_D$ 的线性函数, 而且也是输入变量 $\boldsymbol{x}$ 的线性函数. 然而, 正是其作为输入变量 $\boldsymbol{x}$ 的线性函数这一特性, 给模型带来了显著的限制.

4.1.1 基函数

我们可以通过输入变量的固定非线性函数的线性组合来扩展eq. (5.22)定义的模型:

$$y(\boldsymbol{x}, \boldsymbol{w}) = w_0 + \sum_{j=1}^{M-1} w_j \phi_j(\boldsymbol{x}) \tag{4.2}$$

其中 $\phi_j(\boldsymbol{x})$ 称为基函数. 参数 w_0 代表数据中一个任意的固定偏移量, 有时称为偏置参数 (bias). 通过定义一个额外的虚拟基函数 $\phi_0(\boldsymbol{x}) = 1$,eq. (4.2)可以统一为

$$y(\boldsymbol{x}, \boldsymbol{w}) = \sum_{j=0}^{M-1} w_j \phi_j(\boldsymbol{x}) = \boldsymbol{w}^{\mathsf{T}} \boldsymbol{\phi}(\boldsymbol{x}) \tag{4.3}$$

其中 $\boldsymbol{w} = (w_0, \cdots, w_{M-1})^{\mathsf{T}}$,$\boldsymbol{\phi} = (\phi_0, \cdots, \phi_{M-1})^{\mathsf{T}}$. 通过使用非线性基函数, 函数 $y(\boldsymbol{x}, \boldsymbol{w})$ 变为关于输入变量 $\boldsymbol{x}$ 的非线性函数. 尽管如此, 形如eq. (4.3)的函数仍然称为线性模型, 因为它们关于 $\boldsymbol{w}$ 是线性的.

基函数有多种选择. 在第 1 章的多项式曲线拟合中, 我们使用 $\phi_j(x) = x^j$. 其他常见选择包括高斯基函数

$$\phi_j(x) = \exp\left\{-\frac{(x - \mu_j)^2}{2s^2}\right\} \tag{4.4}$$

其中 μ_j 控制基函数在输入空间中的位置, 参数 s 控制它们的空间尺度. 这种形式的基函数通常叫作高斯基函数, 虽然它们在形式上可能类似于高斯分布, 但我们在使用时并不需要概率化的解释. 另一种形式是 sigmoid 型基函数:

$$\phi_j(x) = \sigma\left(\frac{x - \mu_j}{s}\right) \tag{4.5}$$

其中 $\sigma(a)$ 是按如下方式定义的 sigmoid 函数:

$$\sigma(a) = \frac{1}{1 + \exp(-a)} \tag{4.6}$$

我们可以等价地使用 tanh 函数, 因为其可以通过 $\tanh(a) = 2\sigma(2a) - 1$ 与 sigmoid 函数相关联. 此外还有傅里叶基函数 (对应正弦函数展开) 和小波基函数等.

4.1.2 似然函数

假设目标变量 t 由具有加性高斯噪声的确定性函数 $y(\boldsymbol{x}, \boldsymbol{w})$ 给出, 即

$$t = y(\boldsymbol{x}, \boldsymbol{w}) + \epsilon \tag{4.7}$$

其中 ϵ 是方差为 σ^2 的零均值高斯随机变量. 因此可以写出条件分布

$$p(t \mid \boldsymbol{x}, \boldsymbol{w}, \sigma^2) = \mathcal{N}\left(t \mid y(\boldsymbol{x}, \boldsymbol{w}), \sigma^2\right) \tag{4.8}$$

假设数据点独立地从该分布中产生, 我们得到以 $\boldsymbol{w}$ 和 σ^2 为参数的似然函数:

$$p(\boldsymbol{t} \mid X, \boldsymbol{w}, \sigma^2) = \prod_{n=1}^{N} \mathcal{N}\left(t_n \mid \boldsymbol{w}^{\mathsf{T}} \boldsymbol{\phi}(\boldsymbol{x}_n), \sigma^2\right) \tag{4.9}$$

取似然函数的对数, 得到

$$\ln p(\boldsymbol{t} \mid X, \boldsymbol{w}, \sigma^2) = -\frac{N}{2} \ln(2\pi) - \frac{N}{2} \ln \sigma^2 - \frac{1}{2\sigma^2} E_D(\boldsymbol{w}) \tag{4.10}$$

其中平方和误差函数定义为

$$E_D(\boldsymbol{w}) = \frac{1}{2} \sum_{n=1}^{N} \left\{t_n - \boldsymbol{w}^{\mathsf{T}} \boldsymbol{\phi}(\boldsymbol{x}_n)\right\}^2 \tag{4.11}$$

在确定 $\boldsymbol{w}$ 时,eq. (4.10)中的前两项独立于 $\boldsymbol{w}$, 因此可以视为常数. 正如前面所看到的, 在高斯噪声分布下, 最大化似然函数等价于最小化平方和误差函数.

4.1.3 最大似然

首先对于 $\boldsymbol{w}$ 最大化对数似然函数. 其关于 $\boldsymbol{w}$ 的梯度为

$$\nabla_{\boldsymbol{w}} \ln p = -\frac{1}{\sigma^2} \sum_{n=1}^{N} \boldsymbol{\phi}(\boldsymbol{x}_n) \left\{t_n - \boldsymbol{w}^{\mathsf{T}} \boldsymbol{\phi}(\boldsymbol{x}_n)\right\} \tag{4.12}$$

将梯度设置为 0, 可得

$$\sum_{n=1}^{N} \boldsymbol{\phi}(\boldsymbol{x}_n) t_n = \sum_{n=1}^{N} \boldsymbol{\phi}(\boldsymbol{x}_n) \boldsymbol{\phi}(\boldsymbol{x}_n)^{\mathsf{T}} \boldsymbol{w} \tag{4.13}$$

这称为最小二乘问题的正规方程 (normal equation). 引入 $N \times M$ 的设计矩阵 (design matrix)$\boldsymbol{\Phi}$, 其元素 $\Phi_{nj} = \phi_j(\boldsymbol{x}_n)$, 则 $\boldsymbol{\phi}(\boldsymbol{x}_n)^{\mathsf{T}}$ 是 $\boldsymbol{\Phi}$ 的第 n 行. 求解 $\boldsymbol{w}$ 可得

$$\boldsymbol{w}_{\mathrm{ML}} = (\boldsymbol{\Phi}^{\mathsf{T}} \boldsymbol{\Phi})^{-1} \boldsymbol{\Phi}^{\mathsf{T}} \boldsymbol{t} \tag{4.14}$$

其中 $\boldsymbol{\Phi}^{\dagger} \equiv (\boldsymbol{\Phi}^{\mathsf{T}} \boldsymbol{\Phi})^{-1} \boldsymbol{\Phi}^{\mathsf{T}}$ 称为矩阵 $\boldsymbol{\Phi}$ 的摩尔-彭若斯伪逆 (Moore-Penrose pseudo-inverse). 我们还可以使对数似然关于方差 σ^2 最大化, 得到

$$\sigma^2_{\mathrm{ML}} = \frac{1}{N} \sum_{n=1}^{N} \left\{t_n - \boldsymbol{w}_{\mathrm{ML}}^{\mathsf{T}} \boldsymbol{\phi}(\boldsymbol{x}_n)\right\}^2 \tag{4.15}$$

可以看到, 方差参数的最大似然值是由回归函数周围目标值的残差方差给出的.

4.1.4 最小二乘的几何表示

考虑一个 N 维空间, 其轴由 t_n 给出, 因此 $\boldsymbol{t} = (t_1, \cdots, t_N)^{\mathsf{T}}$ 是这个空间中的一个向量. 每个在 N 个数据点上求值的基函数 $\phi_j(\boldsymbol{x})$ 可以表示为相同空间中的一个向量, 用 $\boldsymbol{\phi}_j$ 表示, 它是 $\boldsymbol{\Phi}$ 的第 j 列. 如果基函数的个数 M 小于数据点的个数 N, 则 M 个 $\boldsymbol{\phi}_j$ 将张成一个维度为 M 的线性子空间 $\mathcal{S}$. 预测向量 $\boldsymbol{y} = \boldsymbol{\Phi}\boldsymbol{w}$ 是 $\{\boldsymbol{\phi}_j\}$ 的线性组合, 因此它可以存在于 $\mathcal{S}$ 中的任意位置. 平方和误差等于 $\boldsymbol{y}$ 和 $\boldsymbol{t}$ 之间的欧氏距离的平方. 最小二乘解对应于 $\mathcal{S}$ 中最接近 $\boldsymbol{t}$ 的 $\boldsymbol{y}$, 即 $\boldsymbol{t}$ 在子空间 $\mathcal{S}$ 上的正交投影.

4.1.5 序贯学习

最大似然解eq. (4.14)涉及一次性处理整个训练集, 称为批量学习. 对于大型数据集, 批处理的计算成本很高. 序贯 (sequential) 学习算法一个个地考虑数据点, 在考虑每个数据点时模型参数得到更新. 通过随机梯度下降 (stochastic gradient descent) 技术可以得到序贯学习算法:

$$\boldsymbol{w}^{(\tau+1)} = \boldsymbol{w}^{(\tau)} - \eta \nabla E_n \tag{4.16}$$

其中 τ 表示迭代次数,η 是合适的学习率. 通过平方和误差函数可得

$$\boldsymbol{w}^{(\tau+1)} = \boldsymbol{w}^{(\tau)} + \eta\left\{t_n - \boldsymbol{w}^{\mathsf{T}}\boldsymbol{\phi}(\boldsymbol{x}_n)\right\}\boldsymbol{\phi}(\boldsymbol{x}_n) \tag{4.17}$$

其中 $\phi_n = \phi(\boldsymbol{x}_n)$. 这称为最小二乘 (least-mean-square) 法或 LMS 算法.

4.1.6 正则化最小二乘法

我们之前介绍过在误差函数中添加正则化项来控制过拟合的思想 (参见**??**). 因此可以采用如下形式来最小化总误差函数:

$$E(\boldsymbol{w}) = E_D(\boldsymbol{w}) + \lambda E_W(\boldsymbol{w}) \tag{4.18}$$

其中 λ 是控制数据相关的误差 $E_D(\boldsymbol{w})$ 和正则化项 $E_W(\boldsymbol{w})$ 相对重要性的正则化系数. 正则化项最为简单的形式之一是权重向量的平方和:

$$E_W(\boldsymbol{w}) = \frac{1}{2}\boldsymbol{w}^{\mathsf{T}}\boldsymbol{w} \tag{4.19}$$

总误差函数变为

$$E(\boldsymbol{w}) = \frac{1}{2}\sum_{n=1}^{N}\left\{t_n - \boldsymbol{w}^{\mathsf{T}}\boldsymbol{\phi}(\boldsymbol{x}_n)\right\}^2 + \frac{\lambda}{2}\boldsymbol{w}^{\mathsf{T}}\boldsymbol{w} \tag{4.20}$$

将其关于 $\boldsymbol{w}$ 的梯度设为 0 并求解, 可得

$$\boldsymbol{w} = \left(\lambda\boldsymbol{I} + \boldsymbol{\Phi}^{\mathsf{T}}\boldsymbol{\Phi}\right)^{-1}\boldsymbol{\Phi}^{\mathsf{T}}\boldsymbol{t} \tag{4.21}$$

在统计学中, 正则化方法会将参数值向 0 减小, 这为参数收缩 (parameter shrinkage) 提供了一个示例.

4.1.7 多重输出

在一些应用中, 我们可能希望预测 $K > 1$ 个目标变量, 它们可以用目标向量 $\boldsymbol{t} = (t_1, \cdots, t_K)^{\mathsf{T}}$ 来表示. 更常见的方法是使用相同的一组基函数来对目标向量的所有分量建模:

$$\boldsymbol{y}(\boldsymbol{x}, \boldsymbol{w}) = \boldsymbol{W}^{\mathsf{T}} \boldsymbol{\phi}(\boldsymbol{x}) \tag{4.22}$$

其中 $\boldsymbol{y}$ 是一个 K 维列向量,$\boldsymbol{W}$ 是一个 $M \times K$ 的参数矩阵. 类似地, 对数似然函数可以表示为矩阵形式, 最大化后得到

$$\boldsymbol{W}_{\mathrm{ML}} = (\boldsymbol{\Phi}^{\mathsf{T}} \boldsymbol{\Phi})^{-1} \boldsymbol{\Phi}^{\mathsf{T}} \boldsymbol{T} \tag{4.23}$$

其中 $\boldsymbol{T}$ 是 $N \times K$ 的目标矩阵. 不同的目标变量之间是解耦的, 我们只需要计算被所有向量 $\boldsymbol{w}_k$ 共享的单个伪逆矩阵即可.

4.2 决策理论

我们将回归任务看作条件概率分布 $p(t \mid \boldsymbol{x})$ 来建模, 并且选择了条件概率的特定形式, 即高斯分布, 其中与 $\boldsymbol{x}$ 相关的均值 $y(\boldsymbol{x}, \boldsymbol{w})$ 由参数 $\boldsymbol{w}$ 控制, 方差由参数 σ^2 给出.$\boldsymbol{w}$ 和 σ^2 都可以使用最大似然从数据中学习得到. 预测分布 (predictive distribution) 的结果为

$$p(t \mid \boldsymbol{x}, \boldsymbol{w}_{\mathrm{ML}}, \sigma^2_{\mathrm{ML}}) = \mathcal{N}\left(t \mid y(\boldsymbol{x}, \boldsymbol{w}_{\mathrm{ML}}), \sigma^2_{\mathrm{ML}}\right) \tag{4.24}$$

预测分布表达了我们对某些新输入 $\boldsymbol{x}$ 所对应 t 值的不确定性. 然而, 对于许多实际应用, 我们需要预测 t 的具体值, 而不是返回整个分布, 特别是在需要采取特定行动的情况下. 我们的任务因此分为两个阶段. 第一个阶段称为推理 (inference) 阶段, 我们使用训练数据来确定预测分布 $p(t \mid \boldsymbol{x})$; 第二个阶段称为决策 (decision) 阶段, 我们使用这一预测分布来确定一个特定的值 $f(\boldsymbol{x})$. 这个特定的值依赖于输入向量 $\boldsymbol{x}$, 并且根据某些准则是最优的. 我们可以通过最小化一个既依赖于预测分布又依赖于特定值 $f(\boldsymbol{x})$ 的损失函数 (也称成本函数) 来实现这一点. 该框架称为决策理论 (decision theory).

假设真实值为 t, 我们为预测选择了一个值 $f(\boldsymbol{x})$, 从而得到某种形式的惩罚或成本. 惩罚是由损失 (loss) 决定的, 用 $L(t, f(\boldsymbol{x}))$ 表示. 由于我们不知道 t 的真实值, 因此并不最小化 L 本身, 而是最小化平均或期望的损失:

$$\mathbb{E}[L] = \iint L(t, f(\boldsymbol{x}))\, p(\boldsymbol{x}, t)\, \mathrm{d}\boldsymbol{x}\, \mathrm{d}t \tag{4.25}$$

回归问题中常见的损失函数是平方损失函数 $L(t, f(\boldsymbol{x})) = \{f(\boldsymbol{x}) - t\}^2$. 在这种情况下, 期望损失函数可以写成

$$\mathbb{E}[L] = \iint \{f(\boldsymbol{x}) - t\}^2 p(\boldsymbol{x}, t)\, \mathrm{d}\boldsymbol{x}\, \mathrm{d}t \tag{4.26}$$

我们的目标是找到能使 $\mathbb{E}[L]$ 最小的 $f(\boldsymbol{x})$. 假设 $f(\boldsymbol{x})$ 是一个完全任意的函数, 则可以使用变分法得到

$$\frac{\delta \mathbb{E}[L]}{\delta f(\boldsymbol{x})} = 2 \int \{f(\boldsymbol{x}) - t\} p(\boldsymbol{x}, t)\, \mathrm{d}t = 0 \tag{4.27}$$

求解 $f(\boldsymbol{x})$ 并使用概率的加和法则和乘积法则, 可以得到

$$f(\boldsymbol{x}) = \frac{\int t\, p(t \mid \boldsymbol{x})\, \mathrm{d}t}{\int p(t \mid \boldsymbol{x})\, \mathrm{d}t} = \int t\, p(t \mid \boldsymbol{x})\, \mathrm{d}t = \mathbb{E}_t[t \mid \boldsymbol{x}] \tag{4.28}$$

这是以 $\boldsymbol{x}$ 为条件的条件平均, 称为回归函数 (regression function). 对于形如eq. (4.8)的高斯条件分布, 条件均值可以简化为

$$\mathbb{E}[t \mid \boldsymbol{x}] = \int t\, p(t \mid \boldsymbol{x})\, \mathrm{d}t = y(\boldsymbol{x}, \boldsymbol{w}_{\mathrm{ML}}) \tag{4.29}$$

我们也可以用另一种推导方式得到这个结果. 将平方项展开, 并对 t 进行积分后交叉项消失, 得到

$$\mathbb{E}[L] = \int \{f(\boldsymbol{x}) - \mathbb{E}[t \mid \boldsymbol{x}]\}^2 \, p(\boldsymbol{x}) \, \mathrm{d}\boldsymbol{x} + \int \mathrm{var}[t \mid \boldsymbol{x}] \, p(\boldsymbol{x}) \, \mathrm{d}\boldsymbol{x} \tag{4.30}$$

我们需要确定的函数 $f(\boldsymbol{x})$ 只出现在右侧的第一项中, 当 $f(\boldsymbol{x}) = \mathbb{E}[t \mid \boldsymbol{x}]$ 时该项达到最小值. 右侧的第二项是分布 t 的方差在 $\boldsymbol{x}$ 上取平均后的结果, 代表目标数据的内在变化, 可以视为噪声; 因为它独立于 $f(\boldsymbol{x})$, 所以它代表损失函数不可约的最小值.

平方损失函数并不是回归中唯一可选的损失函数. 平方损失的一种泛化形式称为闵可夫斯基 (Minkowski) 损失, 其期望为

$$\mathbb{E}[L] = \iint |f(\boldsymbol{x}) - t|^q \, p(\boldsymbol{x}, t) \, \mathrm{d}\boldsymbol{x} \, \mathrm{d}t \tag{4.31}$$

当 $q = 2$ 时即为期望平方损失.$\mathbb{E}[L]$ 的最小值由 $q = 2$ 时的条件均值、$q = 1$ 时的条件中位数和 $q \to 0$ 时的条件众数给出. 高斯噪声假设意味着给定 $\boldsymbol{x}$ 时 t 的条件分布是单峰的, 这对于某些应用可能不合适; 在这种情况下, 平方损失可能会导致结果变差, 需要开发更复杂的模型. 本节重点讨论的是回归问题的决策理论, 在第 5 章中, 我们将在分类任务中引入类似的概念 (参见节 5.2).

4.3 偏差-方差权衡

到目前为止, 在对回归问题的线性模型进行讨论时, 我们假设基函数的形式和数量都是给定的 (参见**??**). 我们还看到, 如果使用非常有限的数据集训练复杂模型, 使用最大似然估计可能会导致严重的过拟合问题. 然而, 通过限制基函数的数量来避免过拟合有一个副作用, 就是限制了模型捕捉数据中有趣和重要趋势的灵活性. 正则化项可以控制包含大量参数的模型的过拟合问题, 但这又引出了如何确定合适的正则化系数 λ 的问题. 我们可以从频率学派的视角讨论模型复杂性的问题, 称为偏差-方差 (bias-variance) 权衡.

如果用参数向量 $\boldsymbol{w}$ 控制的函数对 $h(\boldsymbol{x})$ 进行建模, 那么从贝叶斯学派的视角来看, 模型中的不确定性可以通过 $\boldsymbol{w}$ 上的后验分布来表示. 但是频率学派会基于数据集 $\mathcal{D}$ 对 $\boldsymbol{w}$ 进行点估计, 并试图通过思想实验来解释该估计的不确定性. 假设我们有大量的数据集, 每个数据集的大小为 N 且都独立地从分布 $p(t \mid \boldsymbol{x})$ 中产生. 对于任何给定的数据集 $\mathcal{D}$, 都可以通过算法得到预测函数 $f(\boldsymbol{x}; \mathcal{D})$.

在讨论回归问题的决策理论时 (参见节 4.2), 一旦知道条件分布 $p(t \mid \boldsymbol{x})$, 每个损失函数都会得到对应的最优预测. 一个普遍的选择是平方损失函数, 其最优预测由条件期望给出:

$$h(\boldsymbol{x}) = \mathbb{E}[t \mid \boldsymbol{x}] = \int t \, p(t \mid \boldsymbol{x}) \, \mathrm{d}t \tag{4.32}$$

平方损失的期望可以写成

$$\mathbb{E}[L] = \int \{f(\boldsymbol{x}) - h(\boldsymbol{x})\}^2 p(\boldsymbol{x}) \, \mathrm{d}\boldsymbol{x} + \iint \{h(\boldsymbol{x}) - t\}^2 p(\boldsymbol{x}, t) \, \mathrm{d}\boldsymbol{x} \, \mathrm{d}t \tag{4.33}$$

右侧的第二项来自数据上的固有噪声, 与 $f(\boldsymbol{x})$ 无关, 它代表了期望损失的最小值. 右侧的第一项取决于如何选择函数 $f(\boldsymbol{x})$, 因为这一项是非负的, 所以能达到的最小值是 0.

式(4.33)右侧第一项的被积函数, 在一个特定的数据集 $\mathcal{D}$ 上为 $\{f(\boldsymbol{x}; \mathcal{D}) - h(\boldsymbol{x})\}^2$. 由于它的值依赖于特定的数据集 $\mathcal{D}$, 我们取其在数据集上的期望. 在花括号内加减 $\mathbb{E}_{\mathcal{D}}[f(\boldsymbol{x}; \mathcal{D})]$, 然后展开, 可以得到

$$\mathbb{E}_{\mathcal{D}}\left[\{f(\boldsymbol{x}; \mathcal{D}) - h(\boldsymbol{x})\}^2\right] = \{\mathbb{E}_{\mathcal{D}}[f(\boldsymbol{x}; \mathcal{D})] - h(\boldsymbol{x})\}^2 + \mathbb{E}_{\mathcal{D}}\left[\{f(\boldsymbol{x}; \mathcal{D}) - \mathbb{E}_{\mathcal{D}}[f(\boldsymbol{x}; \mathcal{D})]\}^2\right] \tag{4.34}$$

因此期望损失可以分解为三项之和:

$$\mathbb{E}[L] = \text{平方偏差} + \text{方差} + \text{噪声} \tag{4.35}$$

其中

$$\text{平方偏差} = \int \{\mathbb{E}_{\mathcal{D}}[f(\boldsymbol{x};\mathcal{D})] - h(\boldsymbol{x})\}^2 \, p(\boldsymbol{x}) \, \mathrm{d}\boldsymbol{x} \tag{4.36}$$

$$\text{方差} = \int \mathbb{E}_{\mathcal{D}}\left[\{f(\boldsymbol{x};\mathcal{D}) - \mathbb{E}_{\mathcal{D}}[f(\boldsymbol{x};\mathcal{D})]\}^2\right] p(\boldsymbol{x}) \, \mathrm{d}\boldsymbol{x} \tag{4.37}$$

$$\text{噪声} = \iint \{h(\boldsymbol{x}) - t\}^2 p(\boldsymbol{x}, t) \, \mathrm{d}\boldsymbol{x} \, \mathrm{d}t \tag{4.38}$$

第一项称为平方偏差, 表示所有数据集上的平均预测与期望回归函数的差异程度. 第二项称为方差, 衡量函数 $f(\boldsymbol{x};\mathcal{D})$ 对被选的特定数据集的敏感程度.

我们的目标是最小化期望损失, 它已被分解为平方偏差项、方差项和常数噪声项的和. 偏差和方差之间存在权衡: 非常灵活的模型具有低偏差和高方差, 而相对稳定的模型具有高偏差和低方差. 具有最优预测能力的模型能够在偏差和方差之间达到最佳平衡.

我们可以用第 1 章介绍过的正弦数据集来说明. 独立生成 $L = 100$ 个数据集, 每个数据集包含 $N = 25$ 个数据点, 数据点由正弦曲线 $h(x) = \sin(2\pi x)$ 产生. 对每个数据集拟合一个包含 $M = 24$ 个高斯基函数和一个常数偏置基函数的模型 (共 25 个参数), 通过最小化正则化误差函数得到预测函数. 较小的 λ 值允许模型对每个单独数据集上的噪声进行微调, 从而导致较大的方差; 相反, 较大的 λ 值会使权重参数向 0 靠近, 从而导致较大的偏差.

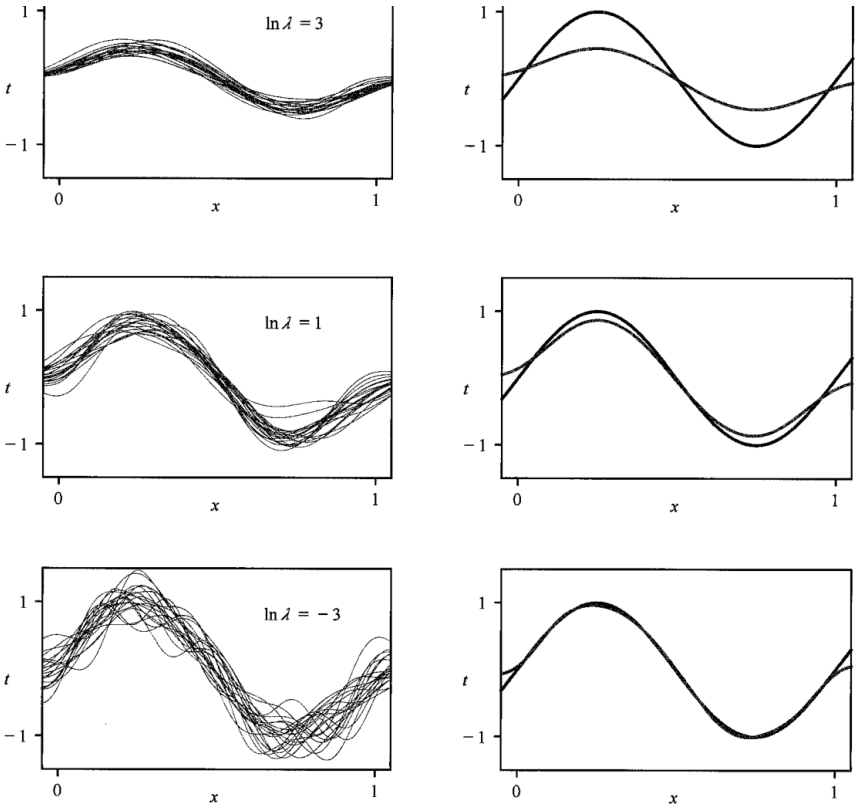


图 4.1: 使用正弦数据展示模型复杂度上偏差和方差的依赖关系, 模型复杂度由正则化参数 λ 控制.$L = 100$ 个数据集, 每个含 $N = 25$ 个数据点, 模型含 24 个高斯基函数及一个偏置基函数. 左列:100 次拟合中的 20 次随 $\ln\lambda$ 变化的结果; 右列:100 次拟合的平均结果 (粗曲线) 与正弦目标函数 (细曲线). 从上到下 $\ln\lambda = 3$(高偏差、低方差)、$\ln\lambda = 1$(折中)、$\ln\lambda = -3$(低偏差、高方差).

平均预测用下式估计:

$$\bar{f}(x) = \frac{1}{L}\sum_{l=1}^{L} f^{(l)}(x) \tag{4.39}$$

而平方偏差和方差则由下式给出:

$$\text{平方偏差} = \frac{1}{L}\sum_{l=1}^{L}\{\bar{f}(x) - h(x)\}^2 \tag{4.40}$$

$$\text{方差} = \frac{1}{L}\sum_{l=1}^{L}\{f^{(l)}(x) - \bar{f}(x)\}^2 \tag{4.41}$$

偏差-方差分解的实际价值是有限的, 因为它基于数据集集合的平均, 而实际上我们只有单个观测数据集. 如果有大量给定大小的独立训练集, 更好的方法是将它们组合成单个更大的训练集, 从而降低给定复杂度模型的过拟合风险. 但尽管如此, 偏差-方差分解仍经常提供对模型复杂性问题的有用洞察.

第 5 章 单层网络: 分类

5.1 判别函数

判别式是一种函数, 它取输入向量 $\boldsymbol{x}$ 并将其归入 K 个类 (用 C_k 表示) 中的一个类. 我们可以大致确定解决分类问题的三种方法 (参见小节 5.2.4). 其中, 最简单的方法是构建一个判别函数 (discriminant function), 该函数直接将每一个输入向量 $\boldsymbol{x}$ 归入一个特定类. 然而, 一种更有效的方法是在推理阶段依据条件概率 $p(C_k \mid \boldsymbol{x})$ 构建分布模型, 然后利用这些分布模型做出最优决策. 将推理和决策分开能带来很多好处.

有两种不同的方法可以确定条件概率 $p(C_k \mid \boldsymbol{x})$. 一种方法是直接建模, 例如将它表示为参数模型, 然后使用训练集对参数进行优化, 这就是判别概率模型 (discriminative probabilistic model). 另一种方法是对类-条件概率密度 $p(\boldsymbol{x} \mid C_k)$ 以及类的先验概率 $p(C_k)$ 进行建模, 然后使用贝叶斯定理计算所需的后验概率:

$$p(C_k \mid \boldsymbol{x}) = \frac{p(\boldsymbol{x} \mid C_k)p(C_k)}{p(\boldsymbol{x})} \tag{5.1}$$

此为生成概率模型 (generative probabilistic model), 它提供了从每个类-条件概率密度 $p(\boldsymbol{x} \mid C_k)$ 生成样本的方法. 本章仅关注线性判别式 (linear discriminant), 即决策面为超平面的判别式.

5.1.1 二分类

线性判别函数的最简单表示是通过取输入向量的线性函数得出的:

$$y(\boldsymbol{x}) = \boldsymbol{w}^{\mathsf{T}}\boldsymbol{x} + w_0 \tag{5.2}$$

其中 $\boldsymbol{w}$ 为权重向量,w_0 为偏置参数. 如果 $y(\boldsymbol{x}) \geqslant 0$, 输入向量 $\boldsymbol{x}$ 将被归入 C_1 类, 否则归入 C_2 类. 因此, 相应的决策边界由 $y(\boldsymbol{x}) = 0$ 定义, 对应于 D 维输入空间中的 $(D-1)$ 维超平面. 偏置参数 w_0 决定了决策面的位置, 而 $\boldsymbol{w}$ 决定了决策面的朝向.

$y(\boldsymbol{x})$ 的值为点 $\boldsymbol{x}$ 到决策面的带号距离的测度. 设 $\boldsymbol{x}_\perp$ 是 $\boldsymbol{x}$ 在决策面上的正交投影, 则点 $\boldsymbol{x}$ 与决策面的带号正交距离为 $y(\boldsymbol{x})/\|\boldsymbol{w}\|$. 如同线性回归模型 (参见小节 4.1.1), 有时使用更简洁的符号较为方便, 即给出一个额外的虚拟" 输入" 值 $x_0 = 1$, 然后定义 $\tilde{\boldsymbol{w}} = (w_0, \boldsymbol{w})$ 和 $\tilde{\boldsymbol{x}} = (1, \boldsymbol{x})$, 从而有 $y(\boldsymbol{x}) = \tilde{\boldsymbol{w}}^{\mathsf{T}}\tilde{\boldsymbol{x}}$.

5.1.2 多分类

下面将线性判别函数扩展到多分类 ($K > 2$). 我们可能会试图通过组合一些二分类判别函数来建立多分类判别式, 然而这将导致分类区域不明确的困难. 我们可以通过采用 K 个线性函数组成的单个 K 类判别式来避免这些困难:

$$y_k(\boldsymbol{x}) = \boldsymbol{w}_k^{\mathsf{T}}\boldsymbol{x} + w_{k0} \tag{5.3}$$

然后在所有 $j \neq k$ 的情况下, 如果 $y_k(\boldsymbol{x}) > y_j(\boldsymbol{x})$, 就将点 $\boldsymbol{x}$ 归入 C_k 类. 因此,C_k 类和 C_j 类之间的决策边界由 $y_k(\boldsymbol{x}) = y_j(\boldsymbol{x})$ 给出, 对应于一个 $(D-1)$ 维超平面. 这种判别式的决策区域总是单连且凸出的.

5.1.3 1-of-K 编码方案

在分类问题中, 目标变量 $\boldsymbol{t}$ 是我们希望预测其类标签的二进制向量. 对于二分类问题, 最方便的是单目标变量 $t \in \{0, 1\}$ 的二进制表示,$t = 1$ 表示 C_1 类,$t = 0$ 表示 C_2 类. 对于多分类 ($K > 2$) 问题, 使用 1-of-K 编码方案较为方便, 其也称独热编码方案, 其中 $\boldsymbol{t}$ 是长度为 K 的向量, 从而如果该类是 C_k 类, 则 $\boldsymbol{t}$ 的所有元素都为 0, 只有元素 t_k 的值为 1. 例如, 假设有 $K = 5$ 个类, 则来自 C_3 类的数据点将得到目标向量 $\boldsymbol{t} = (0, 0, 1, 0, 0)$.

5.1.4 最小二乘分类

在线性回归模型中, 通过最小化平方和误差函数, 可以得到参数值的简单闭式解 (参见小节 4.1.3). 因此我们期待将同样的最小二乘形式应用于分类问题. 考虑一个训练数据集 $\{\boldsymbol{x}_n\}$, 定义目标矩阵 $\boldsymbol{T}$(第 n 行为 $\boldsymbol{t}_n^{\mathsf{T}}$) 和设计矩阵 $\boldsymbol{X}$. 平方和误差函数为

$$E_D(\tilde{\boldsymbol{W}}) = \frac{1}{2}\operatorname{Tr}\left\{(\tilde{\boldsymbol{X}}\tilde{\boldsymbol{W}} - \boldsymbol{T})^{\mathsf{T}}(\tilde{\boldsymbol{X}}\tilde{\boldsymbol{W}} - \boldsymbol{T})\right\} \tag{5.4}$$

将与 $\tilde{\boldsymbol{W}}$ 有关的导数设为 0 并重新排列, 求得解

$$\tilde{\boldsymbol{W}} = (\tilde{\boldsymbol{X}}^{\mathsf{T}}\tilde{\boldsymbol{X}})^{-1}\tilde{\boldsymbol{X}}^{\mathsf{T}}\boldsymbol{T} = \tilde{\boldsymbol{X}}^{\dagger}\boldsymbol{T} \tag{5.5}$$

其中 $\tilde{\boldsymbol{X}}^{\dagger}$ 是矩阵 $\tilde{\boldsymbol{X}}$ 的伪逆 (参见小节 4.1.3), 从而求得判别函数

$$\boldsymbol{y}(\boldsymbol{x}) = \tilde{\boldsymbol{W}}^{\mathsf{T}}\tilde{\boldsymbol{x}} = \tilde{\boldsymbol{X}}^{\dagger\mathsf{T}}\tilde{\boldsymbol{x}} \tag{5.6}$$

最小二乘法给出了判别函数参数的精确闭式解. 然而, 即使作为一个判别函数 (我们直接用它来做决定, 且不需要做任何概率解释), 它也存在一些严重的问题. 特别是, 最小二乘法对离群值 (与大部分数据点相距甚远的数据点) 的存在非常敏感. 平方和误差函数对距离决策边界较远的、即使被正确分类的数据点赋予了过高的权重. 回想一下, 最小二乘法对应的是高斯条件分布假设下的最大似然, 而二进制目标变量的分布显然远非高斯分布. 为便于比较, 使用逻辑斯谛回归 (logistic regression) 技术 (参见小节 5.4.2) 得出的结果对离群值的鲁棒性更强.

5.2 决策理论

假设有一个输入向量 $\boldsymbol{x}$ 和一个相应的目标变量 $\boldsymbol{t}$, 我们想要在给定 $\boldsymbol{x}$ 的新值时预测 $\boldsymbol{t}$. 在讨论线性回归时 (参见节 4.2), 机器学习中的预测过程可分为推理和决策两个阶段. 下面我们结合分类器对这一观点进行更深入的探讨.

5.2.1 误分类率

假设我们的目标只是尽可能减少误分类, 则需要一条规则, 以便将 $\boldsymbol{x}$ 的每个值归入一个可用的类. 该规则会将输入空间划分为若干决策区域 $\mathcal{R}_k$, 每个类拥有一个决策区域, 从而 $\mathcal{R}_k$ 中的所有点都会被归入 C_k 类. 决策区域之间的边界称为决策边界或决策面.

为了得到最优决策规则, 首先考虑二分类. 当属于 C_1 类的输入向量被归入 C_2 类或反之之时, 就会出现错误. 发生这种情况的概率为

$$p(\text{错误}) = p(\boldsymbol{x} \in \mathcal{R}_2, C_1) + p(\boldsymbol{x} \in \mathcal{R}_1, C_2) \tag{5.7}$$

为了将 $p(\text{错误})$ 最小化, 我们应该将每个点 $\boldsymbol{x}$ 归入使被积函数值较小的类. 因此, 对于给定的 $\boldsymbol{x}$ 值, 如果 $p(\boldsymbol{x}, C_1) > p(\boldsymbol{x}, C_2)$, 则归入 C_1 类. 根据概率的乘积法则 $p(\boldsymbol{x}, C_k) = p(C_k \mid \boldsymbol{x})p(\boldsymbol{x})$, 由于 $p(\boldsymbol{x})$ 是公因子, 因此我们可以将这一结果重新表述为: 如果将 $\boldsymbol{x}$ 的每个值归入后验概率 $p(C_k \mid \boldsymbol{x})$ 最大的类, 则犯错的概率最小.

在更一般的多分类情况下, 将正确概率最大化稍显容易:

$$p(\text{正确}) = \sum_k P(\boldsymbol{x} \in \mathcal{R}_k, C_k) = \sum_k \int_{\mathcal{R}_k} p(\boldsymbol{x}, C_k)\,\mathrm{d}\boldsymbol{x} \tag{5.8}$$

5.2.2 预期损失

在许多应用中, 我们的目标相比仅减少误分类数量更为复杂. 让我们再次回到医疗诊断问题: 如果一名未患癌症的病人被误诊为患有癌症, 后果可能是他会经历一些痛苦并接受进一步检查; 相反, 如果癌症患者被误诊为身体健康, 后果可能是他由于没有接受相应治疗而过早死亡. 这两种错误可能导致截然不同的后果.

我们可以通过采用损失函数来将这些问题形式化, 损失函数是对采取任何可供选择之决策或行动所造成损失的单一、总体衡量. 假设一个新的 $\boldsymbol{x}$ 值实际上属于 C_k 类, 而我们却将 $\boldsymbol{x}$ 归入 C_j 类, 我们会遭受损失 L_{kj}, 可以将其视为损失矩阵的 (k, j) 元素. 我们的目标是最大限度地减少总损失. 每个 $\boldsymbol{x}$ 可被独立归入决策区域 $\mathcal{R}_j$ 中的一个类. 预期损失为

$$\mathbb{E}[L] = \sum_k \sum_j \int_{\mathcal{R}_j} L_{kj}\, p(\boldsymbol{x}, C_k)\, \mathrm{d}\boldsymbol{x} \tag{5.9}$$

能够最小化预期损失的决策规则会将每个新的 $\boldsymbol{x}$ 分配给使 $\sum_k L_{kj} p(C_k \mid \boldsymbol{x})$ 最小的类别 j.

5.2.3 拒绝选项

我们已经看到, 分类误差产生于输入空间的区域, 其中最大后验概率 $p(C_k \mid \boldsymbol{x})$ 明显小于 1, 或等价地, 联合分布 $p(\boldsymbol{x}, C_k)$ 存在相近的概率值. 在这些区域, 我们对归属关系相对不确定. 在某些应用中, 应避免对疑难案例做出决策, 以期在做出分类决策更有把握的案例中获得较低的误差率. 这就是所谓的拒绝选项 (rejection option). 为此, 我们可以引入阈值 θ, 并拒绝那些后验概率 $p(C_k \mid \boldsymbol{x})$ 的最大值小于或等于 θ 的输入变量 $\boldsymbol{x}$. 通过设置 $\theta = 1$ 可以确保不拒绝任何示例; 而如果有 K 个类, 那么设置 $\theta < 1/K$ 可以拒绝所有示例.

5.2.4 推理和决策

事实上, 我们可以找出三种不同的方法来解决决策问题, 按复杂程度递减的顺序排列如下:

(a) 首先解决确定每个 C_k 类的类-条件概率密度 $p(\boldsymbol{x} \mid C_k)$ 的推理问题, 分别推断类的先验概率 $p(C_k)$. 然后使用贝叶斯定理

$$p(C_k \mid \boldsymbol{x}) = \frac{p(\boldsymbol{x} \mid C_k) p(C_k)}{p(\boldsymbol{x})} \tag{5.10}$$

求出类的后验概率 $p(C_k \mid \boldsymbol{x})$, 分母可根据归一化条件 $\sum_k p(\boldsymbol{x}) = \sum_k p(\boldsymbol{x} \mid C_k) p(C_k)$ 求出.

(b) 首先解决确定类的后验概率 $p(C_k \mid \boldsymbol{x})$ 的推理问题, 然后使用决策理论将每个新输入的 $\boldsymbol{x}$ 归入其中一个类. 直接对后验概率进行建模的方法称为判别模型.

(c) 计算出一个称为判别函数的函数 $f(\boldsymbol{x})$, 它能将每个新输入的 $\boldsymbol{x}$ 直接映射到类标签上. 例如, 对于二分类问题,$f(\cdot)$ 可能是二进制值, 从而 $f = 0$ 代表 C_1 类, 而 $f = 1$ 代表 C_2 类.

对输入和输出的分布进行显式或隐式建模的方法叫作生成式模型, 因为通过从它们中采样, 我们可以在输入空间中生成合成数据点. 方法 (a) 要求最高, 因为它需要求出 $\boldsymbol{x}$ 和 C_k 的联合分布; 但它的一个优点是, 可以根据边缘密度 $p(\boldsymbol{x})$ 求出新数据点, 这有助于离群值检测 (outlier detection).

5.2.5 分类器精度

衡量分类器性能的最简单方法是计算测试集中被正确分类的数据点所占的比例. 然而, 不同类型的误差会导致不同的后果. 让我们再次回到癌症检查示例: 如果分类器预测一个人患有癌症而这是真实标签, 称这种预测为真阳性 (True Positive, TP); 如果这个人未患癌症, 则称这种预测为假阳性 (False Positive, FP), 假阳性也称第一类误差 (type 1 error). 类似地, 真阴性 (True Negative, TN) 对应正确预测未患癌, 假阴性 (False Negative, FN) 则称第二类误差 (type 2 error).

这可以用混淆矩阵 (confusion matrix) 来表示, 其中行代表真实类, 列代表根据决策准则所做的分类. 准确率为

$$\text{准确率} = \frac{N_{\text{TP}} + N_{\text{TN}}}{N_{\text{TP}} + N_{\text{FP}} + N_{\text{TN}} + N_{\text{FN}}} \tag{5.11}$$

其中 N 是参加测试的总人数. 正如我们所看到的, 如果存在严重失衡的类, 则准确率可能会误导人.

5.3 生成分类器

在为类-条件概率密度 $p(\boldsymbol{x} \mid C_k)$ 指定了一个参数函数形式后, 就可以使用最大似然法确定参数值和类的先验概率 $p(C_k)$. 我们首先讨论连续输入变量 $\boldsymbol{x}$ 的情形, 然后简要讨论离散输入.

5.3.1 连续输入

假设类-条件概率密度是高斯密度. 进一步假设所有类共享相同的协方差矩阵 $\boldsymbol{\Sigma}$, 则 C_k 类的密度为

$$p(\boldsymbol{x} \mid C_k) = \frac{1}{(2\pi)^{D/2}|\boldsymbol{\Sigma}|^{1/2}} \exp\left\{-\frac{1}{2}(\boldsymbol{x} - \boldsymbol{\mu}_k)^{\mathsf{T}}\boldsymbol{\Sigma}^{-1}(\boldsymbol{x} - \boldsymbol{\mu}_k)\right\} \tag{5.12}$$

考虑二分类情形. 根据贝叶斯定理, 后验概率 $p(C_1 \mid \boldsymbol{x})$ 可写成逻辑斯谛 sigmoid 函数作用于 $\boldsymbol{x}$ 的线性函数:

$$p(C_1 \mid \boldsymbol{x}) = \sigma(a(\boldsymbol{x})) = \frac{1}{1 + \exp(-a(\boldsymbol{x}))} \tag{5.13}$$

其中

$$a(\boldsymbol{x}) = (\boldsymbol{\mu}_1 - \boldsymbol{\mu}_2)^{\mathsf{T}}\boldsymbol{\Sigma}^{-1}\boldsymbol{x} - \frac{1}{2}\boldsymbol{\mu}_1^{\mathsf{T}}\boldsymbol{\Sigma}^{-1}\boldsymbol{\mu}_1 + \frac{1}{2}\boldsymbol{\mu}_2^{\mathsf{T}}\boldsymbol{\Sigma}^{-1}\boldsymbol{\mu}_2 + \ln\frac{p(C_1)}{p(C_2)} \tag{5.14}$$

此为 logit 函数, 它表示两个类的概率之比的对数 $\ln[p(C_1 \mid \boldsymbol{x})/p(C_2 \mid \boldsymbol{x})]$, 也称对数几率. 可以看到, 来自高斯密度指数中 $\boldsymbol{x}$ 的二次项已被消去 (缘于共享协方差矩阵的假设), 从而 $a(\boldsymbol{x})$ 是 $\boldsymbol{x}$ 的线性函数. 先验概率 $p(C_k)$ 仅通过偏置参数进入, 因此先验的变化会使决策边界发生平行位移. 决策边界对应于后验概率 $p(C_k \mid \boldsymbol{x})$ 恒定的曲面, 故为线性.

对于 K 个类的一般情况, 后验概率由归一化指数给出, 也叫 softmax 函数:

$$p(C_k \mid \boldsymbol{x}) = \frac{\exp(a_k)}{\sum_j \exp(a_j)} \tag{5.15}$$

其中

$$a_k = \ln\{p(\boldsymbol{x} \mid C_k)p(C_k)\} = \boldsymbol{w}_k^{\mathsf{T}}\boldsymbol{x} + w_{k0} \tag{5.16}$$

如果我们放宽共享协方差矩阵的假设, 允许每个类有自己的协方差矩阵 $\boldsymbol{\Sigma}_k$, 那么先前的二次项消去将不再发生, 我们将得到 $\boldsymbol{x}$ 的二次函数, 从而产生二次判别式 (quadratic discriminant).

5.3.2 最大似然解

假设我们有两个类, 每个类都有一个共享协方差矩阵的高斯类-条件概率密度. 将类的先验概率记为 $p(C_1) = \pi$, 因此 $p(C_2) = 1 - \pi$. 对于属于 C_1 类的数据点 $\boldsymbol{x}_n (t_n = 1)$, 有 $p(\boldsymbol{x}_n, C_1) = \pi\,\mathcal{N}(\boldsymbol{x}_n \mid \boldsymbol{\mu}_1, \boldsymbol{\Sigma})$; 对于 C_2 类 $(t_n = 0)$, 有 $p(\boldsymbol{x}_n, C_2) = (1 - \pi)\,\mathcal{N}(\boldsymbol{x}_n \mid \boldsymbol{\mu}_2, \boldsymbol{\Sigma})$. 似然函数为

$$p(\boldsymbol{t}, X \mid \pi, \boldsymbol{\mu}_1, \boldsymbol{\mu}_2, \boldsymbol{\Sigma}) = \prod_{n=1}^{N}\left[\pi\,\mathcal{N}(\boldsymbol{x}_n \mid \boldsymbol{\mu}_1, \boldsymbol{\Sigma})\right]^{t_n}\left[(1 - \pi)\mathcal{N}(\boldsymbol{x}_n \mid \boldsymbol{\mu}_2, \boldsymbol{\Sigma})\right]^{1 - t_n} \tag{5.17}$$

关于 π 最大化对数似然, 可得

$$\pi = \frac{1}{N}\sum_{n=1}^{N} t_n = \frac{N_1}{N_1 + N_2} \tag{5.18}$$

其中 N_1、N_2 分别为 C_1、C_2 类数据点的总数. 因此 π 的最大似然估计值就是 C_1 类数据点在训练集中所占的比例. 类似地, 关于 $\boldsymbol{\mu}_k$ 最大化, 可得

$$\boldsymbol{\mu}_1 = \frac{1}{N_1}\sum_{n=1}^{N} t_n \boldsymbol{x}_n, \qquad \boldsymbol{\mu}_2 = \frac{1}{N_2}\sum_{n=1}^{N} (1 - t_n)\boldsymbol{x}_n \tag{5.19}$$

此为归入各类的所有输入向量 $\boldsymbol{x}_n$ 的平均值. 最后, 共享协方差矩阵的最大似然解为

$$\boldsymbol{\Sigma} = \frac{N_1}{N}\boldsymbol{\Sigma}_1 + \frac{N_2}{N}\boldsymbol{\Sigma}_2 \tag{5.20}$$

其中

$$\boldsymbol{\Sigma}_1 = \frac{1}{N_1}\sum_{n\in C_1} (\boldsymbol{x}_n - \boldsymbol{\mu}_1)(\boldsymbol{x}_n - \boldsymbol{\mu}_1)^{\mathsf{T}}, \quad \boldsymbol{\Sigma}_2 = \frac{1}{N_2}\sum_{n\in C_2} (\boldsymbol{x}_n - \boldsymbol{\mu}_2)(\boldsymbol{x}_n - \boldsymbol{\mu}_2)^{\mathsf{T}} \tag{5.21}$$

5.3.3 离散特征与指数族

对于离散输入特征, 类似的推导同样适用. 如果特征向量 $\boldsymbol{x}$ 的分量在给定类的条件下相互独立 (朴素贝叶斯假设), 则类-条件密度分解为各分量之积.

更一般地, 当类-条件概率密度属于指数族分布时, 二分类的后验概率同样由作用于 $\boldsymbol{x}$ 线性函数的逻辑斯谛 sigmoid 函数给出, 多分类情形则由 softmax 变换给出. 因此, 尽管生成式方法对类-条件密度的特定选择不同, 其后验概率的形式却具有共同的广义线性模型结构.

5.4 判别分类器

对于二分类情况, 我们已经看到, 针对指数族分布中多种类-条件概率密度 $p(\boldsymbol{x} \mid C_k)$ 的选择,C_k 类的后验概率可以写成作用于 $\boldsymbol{x}$ 线性函数的逻辑斯谛 sigmoid 函数; 对于多分类情况, 后验概率由作用于 $\boldsymbol{x}$ 线性函数的 softmax 变换给出. 对于类-条件概率密度 $p(\boldsymbol{x} \mid C_k)$ 的具体选择, 生成式方法首先使用最大似然法确定密度参数和类的先验概率 $p(C_k)$, 然后使用贝叶斯定理求出类的后验概率.

不过, 还有一种方法是明确使用广义线性模型的函数形式并通过最大似然法直接确定其参数. 在这种直接方法中, 我们将似然函数 (由条件分布 $p(C_k \mid \boldsymbol{x})$ 定义) 最大化, 此为一种判别概率建模形式. 判别法的一个优点是需要确定的可学习参数通常较少, 并且在类-条件概率密度的假定形式与真实分布近似程度较低时, 可以提高预测性能.

5.4.1 激活函数

在线性回归 (参见节 4.1) 中, 模型预测值 $y(\boldsymbol{x}, \boldsymbol{w})$ 由参数的线性函数给出:

$$y(\boldsymbol{x}, \boldsymbol{w}) = \boldsymbol{w}^{\mathsf{T}}\boldsymbol{x} + w_0 \tag{5.22}$$

它给出了范围为 $(-\infty, \infty)$ 的连续值输出. 然而, 对于分类问题, 我们希望预测离散类标签, 或者更一般地说, 预测 $(0, 1)$ 范围内的后验概率. 为此, 我们考虑对这一模型进行泛化——用非线性函数 $f(\cdot)$ 对 $\boldsymbol{w}$ 和 w_0 的线性函数进行转换:

$$p(C_1 \mid \boldsymbol{x}, \boldsymbol{w}) = f(\boldsymbol{w}^\mathsf{T}\boldsymbol{x} + w_0) \tag{5.23}$$

在机器学习文献中,$f(\cdot)$ 称为激活函数 (activation function), 它的反函数在统计学文献中则称为连接函数 (link function). 决策面对应于 $y(\boldsymbol{x}) =$ 常数, 因此是 $\boldsymbol{x}$ 的线性超平面.

5.4.2 逻辑斯谛回归

首先考虑二分类问题. 在小节 5.3.1关于生成方法的讨论中, 我们看到了在一般假设下,C_1 的后验概率可以写成作用于特征空间 $\boldsymbol{\phi}$ 线性函数的逻辑斯谛 sigmoid 函数:

$$p(C_1 \mid \boldsymbol{\phi}) = y(\boldsymbol{\phi}) = \sigma(\boldsymbol{w}^\mathsf{T}\boldsymbol{\phi}) \tag{5.24}$$

其中 $p(C_2 \mid \boldsymbol{\phi}) = 1 - p(C_1 \mid \boldsymbol{\phi})$. 在统计学语境中, 这种模型又称作逻辑斯谛回归 (logistic regression) 模型, 但需要强调的是, 这是一种分类模型, 而不是连续变量模型.

下面使用最大似然法确定逻辑斯谛回归模型的参数. 逻辑斯谛 sigmoid 函数的导数可以方便地用其本身表示:

$$\sigma'(a) = \sigma(a)\big(1 - \sigma(a)\big) \tag{5.25}$$

对于数据集 $\{\boldsymbol{\phi}_n, t_n\}$, 其中 $\boldsymbol{\phi}_n = \boldsymbol{\phi}(\boldsymbol{x}_n)$,$t_n \in \{0, 1\}$, 似然函数可写成

$$p(\boldsymbol{t} \mid \boldsymbol{w}) = \prod_{n=1}^{N} y_n^{t_n}(1 - y_n)^{1-t_n} \tag{5.26}$$

其中 $y_n = p(C_1 \mid \boldsymbol{\phi}_n)$. 取似然的负对数, 得到交叉熵 (cross-entropy) 误差函数:

$$E(\boldsymbol{w}) = -\ln p(\boldsymbol{t} \mid \boldsymbol{w}) = -\sum_{n=1}^{N}\big\{t_n \ln y_n + (1 - t_n)\ln(1 - y_n)\big\} \tag{5.27}$$

关于 $\boldsymbol{w}$ 求梯度, 利用eq. (5.25), 可得

$$\nabla E(\boldsymbol{w}) = \sum_{n=1}^{N}(y_n - t_n)\boldsymbol{\phi}_n \tag{5.28}$$

其中 $y_n = \sigma(a_n)$ 且 $a_n = \boldsymbol{w}^\mathsf{T}\boldsymbol{\phi}_n$. 可以看到, 涉及 sigmoid 导数的因子已被消去, 数据点 n 对梯度的贡献由目标值与模型预测值之间的" 误差"$(y_n - t_n)$ 乘以 $\boldsymbol{\phi}_n$ 给出. 这与线性回归模型的平方和误差函数梯度的形式完全相同.

由于 $y(\cdot)$ 的非线性, 最大似然解 $\nabla E(\boldsymbol{w}) = 0$ 没有闭式解, 但误差函数是参数的凸函数, 可以通过迭代重加权最小二乘 (Iterative Reweighted Least Squares, IRLS) 算法, 或更简单地通过随机梯度下降法来求解. 请注意, 对于线性可分的数据集, 最大似然法可能会表现出严重的过拟合: 当 $\boldsymbol{w}^\mathsf{T}\boldsymbol{\phi} = 0$ 所对应的超平面将两个类分开, 且 $\boldsymbol{w}$ 的大小趋于无穷时,sigmoid 函数在特征空间中变得无限陡峭, 相当于单位阶跃函数, 因此每个训练点都被赋予后验概率 $p(C_1 \mid \boldsymbol{x}) = 1$. 在误差函数中加入正则化项可以避免这种奇异解.

5.4.3 多类逻辑斯谛回归

对于多分类问题,C_k 类的后验概率由作用于特征变量线性函数的 softmax 变换给出:

$$p(C_k \mid \boldsymbol{\phi}) = y_k(\boldsymbol{\phi}) = \frac{\exp(a_k)}{\sum_j \exp(a_j)} \tag{5.29}$$

其中预激活 $a_k = \boldsymbol{w}_k^\mathsf{T}\boldsymbol{\phi}$. 使用 1-of-$K$ 编码方案, 属于 C_k 类的 $\boldsymbol{\phi}_n$ 的目标向量 $\boldsymbol{t}_n$ 是一个二进制向量, 除元素 $t_{nk} = 1$ 外其余元素均为 0. 似然函数为

$$p(\boldsymbol{T} \mid \boldsymbol{w}_1, \cdots, \boldsymbol{w}_K) = \prod_{n=1}^{N} \prod_{k=1}^{K} y_{nk}^{t_{nk}} \tag{5.30}$$

取负对数可得多分类问题的交叉熵误差函数:

$$E(\boldsymbol{w}_1, \cdots, \boldsymbol{w}_K) = -\sum_{n=1}^{N} \sum_{k=1}^{K} t_{nk} \ln y_{nk} \tag{5.31}$$

关于参数向量 $\boldsymbol{w}_k$ 求梯度, 同样得到与平方和误差、逻辑斯谛回归形式相同的结果:

$$\nabla_{\boldsymbol{w}_k} E = \sum_{n=1}^{N} (y_{nk} - t_{nk}) \boldsymbol{\phi}_n \tag{5.32}$$

5.4.4 probit 回归与规范连接

我们已经看到, 对于各种类-条件分布, 得到的后验类率由作用于特征变量线性函数的 logistic(或 softmax) 变换给出. 然而, 这并非唯一的选择.probit 回归 (probit regression) 使用累积高斯分布作为激活函数, 它将线性模型的输出通过标准正态分布的累积分布函数 $y = \Phi(a)$ 映射到 $(0, 1)$ 区间.

更一般地, 广义线性模型框架允许我们为激活函数选择其他形式, 其中最常用的是由指数族分布导出的规范连接函数 (canonical link function). 对于给定的观测噪声分布, 规范连接函数使得对数似然的梯度具有与逻辑斯谛回归相同的简洁形式 (即误差 $(y - t)$ 与基函数激活 $\boldsymbol{\phi}$ 的乘积), 这与我们在线性模型的平方和误差和交叉熵误差中观察到的形式一致.

第 6 章 深度神经网络

6.1 固定基函数的局限性

近年来, 神经网络已迅速成为实际应用中最重要的机器学习技术. 本书前几章已经为这一主题奠定了必要的基础: 基于固定非线性基函数线性组合的回归模型可以被视为单层神经网络 (参见第 4 章), 同样地, 基于基函数线性组合的分类模型也可以视为单层网络 (参见第 5 章). 这使我们能够在讨论更复杂的多层 (神经) 网络之前先引入几个重要概念.

用于分类的线性基函数模型是基函数 $\{\phi_j(\mathbf{x})\}$ 的线性组合 (参见第 5 章), 形式为

$$y(\mathbf{x}, \mathbf{w}) = f\left(\sum_{j=1}^{M} w_j \phi_j(\mathbf{x}) + b\right), \tag{6.1}$$

其中 $f(\cdot)$ 是一个非线性输出激活函数; 用于回归的线性模型具有相同形式, 但将 $f(\cdot)$ 换成了恒等函数. 由于这些基函数的一般性, 这样的模型原则上可以解决任何回归或分类问题——只要其中某个基函数恰好对应所期望的输入–输出映射, 可学习的线性层只需把该基函数的值复制到输出即可.

更一般地, 我们期望有一个足够大且丰富的基函数集合, 从而能以任意精度拟合所需函数. 然而, 遗憾的是, 线性模型存在一些重大缺陷: 它们假设基函数 $\phi_j(\mathbf{x})$ 是固定的、与训练数据无关. 为了理解这些局限, 我们首先观察当输入变量数量增加时线性模型的行为会如何变化.

6.1.1 维度诅咒

考虑由如下 M 阶多项式给出的单输入变量简单回归模型 (参见 1.2 节):

$$v(x, \mathbf{w}) = w_0 + w_1 x + w_2 x^2 + \cdots + w_M x^M. \tag{6.2}$$

如果增加输入变量的数量, 假设有 D 个输入变量 $\{x_1, \cdots, x_D\}$, 那么至多三阶系数的通用多项式形式如下:

$$v(\mathbf{x}, \mathbf{w}) = w_0 + \sum_i w_i x_i + \sum_{i \le j} w_{ij} x_i x_j + \sum_{i \le j \le k} w_{ijk} x_i x_j x_k. \tag{6.3}$$

随着 D 增大, 独立系数的数量增长为 $O(D^3)$, 而对于 M 阶多项式, 系数数量增长为 $O(D^M)$. 我们看到, 在高维空间中多项式会迅速变得笨重且几乎不具实用性.

这种朴素方法在扩展到具有更多输入变量的分类问题时同样存在许多问题. 以鸢尾花数据为例: 将输入空间划分为规则单元, 新测试点的类别被推断为同一单元内训练点最多的类别. 如果将输入空间划分为规则单元, 那么单元数量将随维度 D 呈指数级增长, 这就要求指数级数量的训练数据才能保证单元不空, 否则许多单元将不包含任何训练点, 其中的测试点将得不到分类. 这种在高维空间中出现的巨大困难称为**维度诅咒** (curse of dimensionality) .

6.1.2 高维空间

在高维空间中, 我们在三维空间中形成的几何直觉可能会严重失效. 考虑 D 维空间中一个半径 $r = 1$ 的超球体, 我们想知道介于半径 $r = 1 - \varepsilon$ 和 $r = 1$ 之间的壳层体积占超球体总体积的比例. 半径为 r 的 D 维超球体体积 $V_D(r)$ 必须按 r^D 缩放, 即

$$V_D(r) = K_D r^D, \tag{6.4}$$

其中常数 K_D 仅依赖于 D. 该壳层占总体积的比例由下式给出 (见习题 6.1):

$$1 - (1 - \varepsilon)^D. \tag{6.5}$$

对于较大的 D, 即使 ε 很小, 占比也趋近于 1. 于是得到一个重要结论: 在高维空间中, 超球体的大部分体积集中在靠近表面的薄壳之中.

接下来考虑高维空间中高斯分布的行为. 从笛卡尔坐标变换到极坐标并积分掉方向变量, 可以得到径向密度 $p(r)$ 的表达式 (见习题 6.3).$p(r)\,\delta r$ 是位于半径 r 处厚度为 δr 的薄壳内的概率质量. 对于较大的 D, 高斯分布的大部分概率质量集中在特定半径的薄壳内.

6.1.3 数据流形

虽然维度诅咒确实带来了严重问题, 但这并不妨碍我们找到适用高维空间的有效技术. 背后的一个原因是: 真实数据通常被限制在具有较低有效维度的数据空间区域内.

考虑手写数字图像. 每幅图像都是高维空间中的一个点, 维度由像素数决定; 但由于对象可在图像内不同位置、以不同朝向出现, 图像之间只有三个可变自由度. 一组图像大致处在高维空间中的一个三维**流形** (manifold) 上, 且由于对象位置/朝向与像素强度之间的复杂关系, 该流形是高度非线性的.

实际上, 像素数量只是图像生成过程的人为产物. 以更高分辨率捕获相同图像会增加数据空间维度 D, 但并不改变图像仍然存在于三维流形上这一事实. 如果我们能将局部化的基函数与数据流形关联起来, 而不是与整个高维数据空间相关联, 那么所需基函数数量将随流形维度 (而非数据空间维度) 呈指数级增长——而流形维度通常低得多, 这将是巨大的改进.

另一种观察方式是考虑随机图像生成任务: 逐像素独立从均匀分布采样 RGB 强度得到的合成图像, 没有任何一幅看起来像自然图像, 原因在于合成图像缺乏自然图像所表现出的相邻像素强相关性. 自然图像只覆盖了整个高维空间的一小部分.

6.1.4 数据依赖的基函数

独立于所解决问题而选择的简单基函数在高维空间中具有严重局限. 传统方法是由专家针对每个应用手工设计特定基函数 (通常称为特征), 这种方法在过去数十年中是机器学习的主流.

一种数据依赖的做法是让每个训练数据点都与一个基函数相关联, 从而确保基函数自动适应于底层数据流形. 例如**径向基函数** (radial basis function) : 每个基函数仅依赖于到中心向量的径向距离 (通常是欧氏距离). 若基中心选为输入数据值 $\{\mathbf{x}_n\}$, 则对每个数据点都有一个基函数, 其典型形式为

$$\phi_j(\mathbf{x}) = \exp\left(-\frac{\|\mathbf{x}-\mathbf{x}_j\|^2}{2s^2}\right), \tag{6.6}$$

其中 s 控制基函数宽度. 这类模型的主要问题是对大型数据集计算困难.

相关方法是**支持向量机** (Support Vector Machine, SVM) , 它通过以每个训练点为中心重新定义基函数, 并在训练过程中自动选择这些基函数的一个子集. 最终模型中有效基函数数量通常远少于训练点数, 但仍相对较大, 并随训练集增大而增大;SVM 不产生概率化输出, 无法自然推广到多类情况. 这些方法已被深度神经网络所取代.

为了将形如 (6.1) 的线性模型应用于大规模、高维问题, 我们需要一组针对所解决问题的基函数. 神经网络背后的关键思想是: 选择具有可学习参数的基函数 $\phi_j(\mathbf{x})$, 然后在训练过程中同时调整这些参数以及系数 $\{w_j\}$. 整个模型可通过基于梯度的优化方法最小化误差函数来训练 (参见第 7 章), 例如随机梯度下降.

有许多方法可以构造参数化的非线性基函数, 但有一个关键要求: 这些函数必须关于可学习参数可微, 以便应用基于梯度的优化. 最成功的选择是使用与 (6.1) 形式相同的基函数, 其中每个基函数本身是输入的线性组合的非线性函数, 而组合系数是可学习参数. 这种构造显然可以递归扩展为具有多层的层次化模型, 这正是深度神经网络的基础 (参见 6.3 节).

6.2 多层网络

考虑具有两层可学习参数的基本神经网络. 首先用以下形式构造输入变量 $x_1, \cdots, x_D$ 的 M 个线性组合:

$$a_j^{(1)} = \sum_{i=1}^{D} w_{ji}^{(1)} x_i + b_j^{(1)}, \qquad j = 1, \cdots, M, \tag{6.7}$$

其中 $j = 1, \cdots, M$, 上标 (1) 表示相应参数位于神经网络的第一层. 参数 $w_{ji}^{(1)}$ 称为**权重** (weight),$b_j^{(1)}$ 称为**偏置**参数 (bias),$a_j^{(1)}$ 则称为**预激活** (pre-activation, 参见第 4 章). 然后, 每个 $a_j^{(1)}$ 通过一个可微的非线性激活函数 $h(\cdot)$ 被转换为

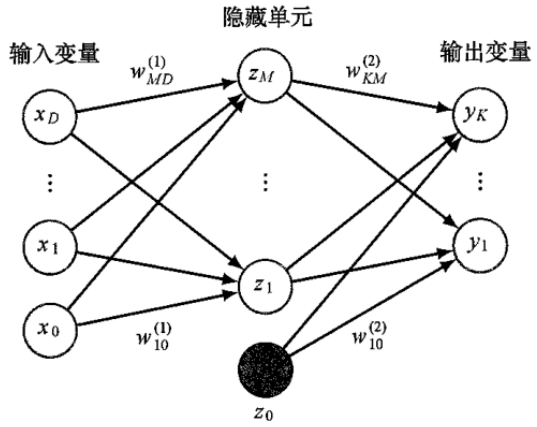
$$z_j^{(1)} = h\left(a_j^{(1)}\right), \tag{6.8}$$

它代表 (6.1) 中基函数的输出. 在神经网络语境中, 这些基函数 $z_j^{(1)}$ 称为**隐藏单元** (hidden unit).

只要导数 $h'(\cdot)$ 可以计算, 整个网络函数就是可微的. 遵循 (6.1), 这些值可以再次被线性组合为

$$a_k^{(2)} = \sum_{j=1}^{M} w_{kj}^{(2)} z_j^{(1)} + b_k^{(2)}, \qquad k = 1, \cdots, K, \tag{6.9}$$

其中 $k = 1, \cdots, K$,K 是输出总数. 该变换对应神经网络的第二层,$w_{kj}^{(2)}$ 同样是权重参数. 最后,$a_k^{(2)}$ 通过适当的输出单元激活函数 $f(\cdot)$ 被转换为网络输出 y_k. 两层神经网络可以用图 6.1 所示的有向图表示.



]示意图。输入变量、隐藏单元和输出变量由节点表示，权重参

图 6.1: 两层神经网络示意图. 输入变量、隐藏单元和输出变量由节点表示, 权重参数由节点之间的连接表示. 偏置参数由来自额外输入 $x_0 = 1$ 和隐藏变量 $z_0 = 1$ 的连接表示; 箭头表示前向传播过程中信息流动的方向.

6.2.1 参数矩阵

如 4.1.1 小节所述, 通过定义一个额外值恒为 1 的输入变量 x_0, 可以将 (6.7) 中的偏置参数吸收到权重参数集合中, 使 (6.7) 变为

$$a_j^{(1)} = \sum_{i=0}^{D} w_{ji}^{(1)} x_i. \tag{6.10}$$

同样也可以把第二层的偏置吸收到第二层权重集合中, 于是整个网络函数变为

$$z_j^{(1)} = h\left(\sum_{i=0}^{D} w_{ji}^{(1)} x_i\right), \qquad y_k = f\left(\sum_{j=0}^{M} w_{kj}^{(2)} z_j^{(1)}\right). \tag{6.11}$$

一种方便的表示法是把输入表示为列向量 $\mathbf{x} = (x_1, \cdots, x_D)^{\mathsf{T}}$, 然后将 (6.11) 中的权重和偏置参数聚集成矩阵. 写 $\mathbf{z}^{(1)} = h(\mathbf{W}^{(1)}\mathbf{x} + \mathbf{b}^{(1)})$,$\mathbf{y} = f(\mathbf{W}^{(2)}\mathbf{z}^{(1)} + \mathbf{b}^{(2)})$, 其中 $h(\cdot)$ 和 $f(\cdot)$ 分别逐元素作用于向量.

6.2.2 通用近似

图 6.10 展示了两层神经网络近似四种不同函数的能力:$f(x) = x^2$、$f(x) = \sin x$、$f(x) = 1/x$ 以及 Heaviside 阶跃函数 $H(x)$. 该图还展示了各个隐藏单元如何协作从而近似最终函数; 图 6.11 则在一个简单二分类问题中说明了隐藏单元的作用.

20 世纪 80 年代, 两层前馈网络的近似性质得到了广泛研究. 各种定理表明: 对于大多数激活函数而言, 这样的网络可以以任意精度近似定义在紧的连续子集上的任何函数 . 对于从任意有限维离散空间到另一空间的映射, 也有类似结论. 因此, 神经网络可以视为**通用近似器** (universal approximator).

虽然这些定理令人安心, 但已有研究只告诉我们存在可表示所需函数的网络, 并未说明这样的网络能否通过学习算法找到; 此外, 某些情况下它们可能需要指数级数量的隐藏单元. 更重要的是, 我们永远无法找到一个真正通用的机器学习算法 (参见 9.1.2 小节). 最后, 尽管两层权重的网络是通用近似器, 但在实际应用中, 远超两层的网络可能带来巨大好处——因为它可以学习分层的内部表示. 以上所有这些论点都支持了深度学习的发展.

6.2.3 隐藏单元激活函数

输出单元的激活函数由被建模分布的类型决定; 而对隐藏单元的唯一要求是它们可微, 这为网络结构设计留下了广泛空间. 虽然原则上网络不同部分可以使用不同激活函数, 但大多数情况下所有隐藏单元使用相同激活函数.

最简单的隐藏单元激活函数是恒等函数, 意味着所有隐藏单元都是线性的. 然而对于任何这样的网络, 我们总能找到一个没有隐藏单元的等价网络——因为连续线性变换的组合本身仍是线性变换, 其表示能力不会强于单个线性层. 但若隐藏单元数量少于输入或输出单元数量, 这样的网络所生成的变换将不是从输入到输出最一般的线性变换: 考虑一个 N 输入、M 隐藏、K 输出且全部使用线性激活的网络, 它有 $M(N + K)$ 个参数, 而直接线性变换有 NK 个参数. 当 M 相对 N 或 K 较小时, 这等价于一个比直接线性映射参数更少的两层线性网络, 对应一个秩亏变换. 这种线性” 瓶颈” 网络对应标准数据分析技术主成分分析 (PCA, 参见第 16 章).

一个简单、非线性且可微的早期选择是 sigmoid 激活函数

$$\sigma(a) = \frac{1}{1 + \exp(-a)}. \tag{6.12}$$

与 sigmoid 密切相关且部分受到生物神经元性质启发的是 tanh 激活函数

$$\tanh(a) = \frac{e^a - e^{-a}}{e^a + e^{-a}}. \tag{6.13}$$

tanh 与 sigmoid 的区别在于输入和输出都是对称的. 由于

$$\tanh(a) = 2\sigma(2a) - 1, \tag{6.14}$$

对于任何具有 sigmoid 隐藏单元的网络, 都存在具有 tanh 激活的等价网络 (见习题 6.4). 然而在训练网络时这些并不一定等价, 因为基于梯度的优化需要初始化权重和偏置, 激活函数变了, 初始化方案也必须相应调整.

tanh 的硬版本 由下式给出:

$$h(a) = \max(-1, \min(1, a)). \tag{6.15}$$

sigmoid 和 tanh 的一个主要缺点是当输入具有很大正值或很小负值时, 梯度会指数级趋于零 (参见 7.4.2 小节), 即” 梯度消失” 问题. 使用非零梯度的激活函数通常更好. 一个选择是 softplus 激活函数 (见习题 6.7)

$$h(a) = \ln(1 + e^a). \tag{6.16}$$

当输入为很大正值时,$h(a) \approx a$, 因此即使输入很大, 梯度仍非零, 有助于缓解梯度消失问题.softplus 可视为 ReLU 的平滑版本, 有时也称 soft ReLU(见习题 6.5).

另一个更简单的选择是 ReLU(Rectified Linear Unit) 激活函数

$$h(a) = \max(0, a). \tag{6.17}$$

从经验上看它是表现最好的激活函数之一, 得到了广泛使用. 严格来说 ReLU 的导数在 $a = 0$ 处未定义, 但实

践中可忽略.ReLU 的引入在训练效率上较之前的 sigmoid 有巨大提升：除了能高效训练更深的网络外，它对权重随机初始化也不太敏感，且非常适合低精度计算 (如 8 位定点数而非 64 位浮点数). 在许多实际应用中 ReLU 是默认选择，除非明确需要探索其他激活函数.

ReLU 的一个问题是对负输入梯度为零，意味着某些隐藏单元在训练期间可能接收不到" 误差信号". 避免该问题的一个办法是使用 ReLU 的修改版 leaky ReLU:

$$h(a) = \max(0, a) + \alpha \min(0, a), \qquad 0 < \alpha < 1. \tag{6.18}$$

一个变体取 $\alpha = -1$，此时 $h(a) = |a|$；另一个变体允许每个隐藏单元有自己的 α_j，从而在训练期间通过计算关于 α_j 以及权重和偏置的梯度来学习.

6.2.4 权重空间的对称性

前馈网络的一个性质是：多个不同权重向量 $\mathbf{w}$ 的选择都可以产生相同的从输入到输出的映射函数 . 考虑图 6.1 所示的两层网络，具有 M 个 tanh 激活的隐藏单元，每一层都是全连接.

符号翻转对称：如果改变输入到某一特定隐藏单元的所有权重和偏置的符号，那么对给定输入数据点，该隐藏单元预激活的符号将被反转，从而激活也被反转——这是因为 tanh 是奇函数,$\tanh(-a) = -\tanh(a)$. 这种反转可以通过改变从该隐藏单元输出的所有权重符号来准确补偿. 因此网络所表示的输入–输出映射保持不变. 对于 M 个隐藏单元，存在 2^M 种这样的" 符号翻转" 对称，任何给定权重向量都属于具有这种交换对称性的 2^M 个等价权重向量集合.

隐藏单元置换对称：类似地，假设把输入到和输出自两个不同隐藏单元的所有权重 (和偏置) 值交换，这显然没有改变网络的输入–输出映射，但对应于不同权重向量. 对于 M 个隐藏单元，任何给定权重向量都属于具有这种交换对称性的 $M!$ 个等价权重向量集合.

对于具有超过两层权重的网络，总对称水平由这样因子的乘积给出，每个隐藏单元层对应一个这样的因子. 事实证明，这些因子解释了权重空间中所有对称 (权重值的特定选择导致的偶然对称除外)，并且这些对称并非 tanh 所特有，而是适用于大多数激活函数 . 一般来说，权重空间中的这些对称在实践中几乎无用，因为网络训练旨在找到参数的特定设置，其他等价设置的存在没有什么用；然而当使用贝叶斯方法评估不同规模网络的概率分布时，权重空间对称确实发挥作用 .

6.3 深度网络

我们通过使用可学习参数控制线性回归/分类模型的基函数，阐述了发展神经网络的动机，从而产生了图 6.1 所示的两层网络. 多年来它一直是最广为使用的架构——主要因为很难有效训练超过两层的网络. 然而最近训练技术的发展使多层网络变得有效 (参见第 7 章).

注意此类网络中" 层数" 的术语存在一些混乱：图 6.1 所示网络有时被描述为" 三层网络"(对单元层数计数，把输入视为单元)，有时被描述为" 单隐藏层网络"(对隐藏单元层数计数). 我们建议将图 6.1 所示网络称为" **两层网络**"，因为可学习权重的层数对网络性质十分重要.

我们可以轻松地把两层网络架构 (6.11) 扩展到任意有限层数 L，其中第 $l = 1, \cdots, L$ 层计算

$$\mathbf{z}^{(l)} = h^{(l)}\left(\mathbf{W}^{(l)}\mathbf{z}^{(l-1)} + \mathbf{b}^{(l)}\right), \tag{6.19}$$

其中 $h^{(l)}$ 是与第 l 层相关联的激活函数,$\mathbf{W}^{(l)}$ 表示相应的权重和偏置参数矩阵. 此外 $\mathbf{z}^{(0)} = \mathbf{x}$ 表示输入向量,$\mathbf{z}^{(L)} = \mathbf{y}$ 表示输出向量.

虽然图 6.1 所示的两层网络具有通用近似能力，但具有超过两层的网络有时可以用比两层网络少得多的参数来表示给定函数.Montufar 等人 (2014) 指出，网络函数将输入空间划分成多个区域，这些区域在网络深度上是指数级增长的，但在隐藏层宽度上却是多项式级的；使用两层网络表示相同函数则需要指数级数量的隐藏单元.

6.3.1 层次化表示

虽然通用近似定理是有趣的结果, 但另一个更令人信服的探索深度网络的理由是: 网络架构编码了一种特定形式的归纳偏置, 即输出空间与输入空间之间通过层次化表示相关联. 这方面一个很好的例子是图像中物体识别 (参见第 10 章): 像素与高级概念 (如" 猫") 之间的关系非常复杂且非线性, 对两层网络而言是极具挑战性的识别任务; 然而深度网络会首先在浅层检测低级特征 (如边缘), 然后在后续层中将它们结合起来以得到更高级的特征 (如眼睛或胡须), 之后继续在深层组合这些特征以检测猫的存在.

这可以看作一种**组合** (compositional) 归纳偏置: 更高级对象 (如猫) 由低级对象 (如眼睛) 组成, 而这些低级对象又由更低级元素 (如边缘) 组成. 也可以从相反角度考虑图像生成: 从低级特征 (如边缘) 开始, 组合起来形成简单形状 (如圆), 之后进一步组合形成更高级对象 (如猫). 每个阶段都有许多不同方式组合不同组件, 随着深度增加, 可能性数量呈指数级增长.

6.3.2 分布式表示

在概念上, 隐藏层中的每个单元都可以视为表示网络在该层的一个" 特征": 激活值高表示相应特征存在, 激活值低表示不存在. 当给定层中有 M 个单元时, 网络可以表示 M 个不同特征. 然而网络也可能学习到不同表示——其中隐藏单元的组合表示特征, 从而使具有 M 个单元的隐藏层能表示 2^M 个不同特征.

例如考虑一个用于处理人脸图像的网络: 每张特定人脸图像可能有 (或没有) 眼镜、有 (或没有) 帽子、有 (或没有) 胡须, 从而导致 $2^3 = 8$ 种组合. 尽管这可以用 8 个单元来表示 (检测到相应组合时对应单元" 打开"), 但也可以更紧凑地只使用 3 个单元, 每个单元代表一个属性. 这些属性可以独立存在 (尽管统计上它们的存在可能一定程度上相关). 稍后我们将在第 10 章详细探讨深度学习网络在训练过程中自行发现的内部表示类型.

6.3.3 表示学习

我们可以把连续经过深度网络不同层视为对数据进行变换, 以便更容易地完成任务. 例如成功学习将皮肤病变分类为良性或恶性 (参见 1.1.1 小节) 的网络必须学会将原始图像数据转换为由最终隐藏层输出表示的新空间, 以便网络最终层可以区分这两类数据. 这个最终层可以视为一个简单线性分类器, 因此在最后隐藏层的表示中, 两类数据必须能被一个线性面很好地分开.

这种发现数据的非线性变换使后续任务更容易完成的能力称为**表示学习** (representation learning) . 学习到的表示有时称为**嵌入空间** (embedding space), 它们由网络中一个隐藏层的输出给出, 从而对于任何输入向量 (无论来自训练集还是新数据集) 都可以通过网络前向传播转换为这个表示.

表示学习异常强大的原因在于它使我们能够利用无标签数据. 通常收集大量无标签数据很容易, 但获取相关标签可能更困难: 例如车辆行驶在城市周边时, 车载摄像头可以收集大量城市场景图像, 但识别相关物体 (如行人、路标) 却需要成本高昂且耗时的人工标注.

从无标签数据中学习称为**无监督学习** (unsupervised learning), 人们已开发出许多不同算法. 例如可以训练神经网络将图像作为输入并创建相同图像作为输出; 为使其成为一项具有挑战性的任务, 网络可能使用比图像像素数更少的隐藏层单元, 从而要求网络学习对图像进行某种压缩. 这只需要无标签数据, 因为训练集中每张图像同时充当输入向量和目标向量. 这种网络又称为**自编码器** (参见 19.1 节), 训练目标是使网络发现对解决其他任务 (如图像分类) 有用的数据内部表示.

无监督学习在成功训练第一个深度网络 (卷积网络除外) 方面扮演了重要角色: 网络每一层首先使用无监督学习进行预训练, 然后整个网络再使用基于梯度的监督训练方式. 后来人们发现预训练阶段可以省略, 在适当条件下可以纯粹使用监督学习从头训练深度网络. 然而预训练和表示学习仍然是深度学习的核心. 最值得一提的预训练是在自然语言处理中进行的 (参见第 12 章):Transformer 模型在大量文本上进行训练, 从而学习到高度复杂的语言内部表示, 使其各项能力达到甚至超越人类水平.

6.3.4 迁移学习

从一个特定任务学习到的内部表示也可能对相关任务有帮助. 例如在一个大型有标签常见对象数据集上训练的网络, 可以学习到如何将图像表示转换为更适合对象分类的表示; 然后可以使用一个较小的有标签数据集对网络最终分类层重新训练, 以创建一个病变分类器 (参见 1.1.1 小节). 这是**迁移学习** (transfer learning) 的一个示例, 相较于仅使用病变图像数据训练可以获得更高准确率, 因为网络可以利用自然图像的通用共性.

一般来说, 迁移学习可以通过使用来自相关任务 B 的数据来提升任务 A 的性能, 其中任务 A 训练数据匮乏而任务 B 训练数据更丰富. 两个任务应具有相同类型输入, 且任务之间应存在一些共性, 使得从任务 B 学到的低级特征或内部表示对任务 A 有帮助. 当研究卷积网络 (参见第 10 章) 时, 我们会发现许多图像处理任务在网络早期层中需要相似低级特征, 而后续层则专门用于特定任务, 因此这样的网络非常适合迁移学习.

当任务 A 训练数据非常稀缺时, 可能只需重新训练网络最终层; 如果训练数据更多, 重新训练多个层也可行. 使用一个任务学习参数, 然后应用于一个或多个其他任务的过程称为**预训练** (pre-training). 对于新任务, 相较于对整个网络应用随机梯度下降, 更高效的方式是将新训练数据一次通过参数固定的预训练网络, 以便在新表示中评估训练输入, 然后在仅包含最终层的较小网络上应用基于迭代梯度的优化. 除了把预训练网络作为固定预处理器之外, 也可以应用**微调** (fine-tuning), 即将整个网络适配到任务 A 的训练数据上; 这通常通过非常小的学习率和有限次迭代完成, 以确保网络不会过拟合新任务中相对较小的可用数据集.

跨多个任务学习可以扩展到**多任务学习** (multitask learning) : 网络同时联合学习多个相关任务. 例如希望建立一个垃圾邮件过滤器, 允许不同用户有不同分类器以适应各自偏好; 训练数据可能包括许多不同用户的垃圾邮件和非垃圾邮件样本, 但任何一个用户的样本数量可能相当有限, 因此为每个用户单独训练分类器可能效果不佳. 相反, 可以将数据集组合并训练一个单一较大网络, 该网络共享早期层, 但对不同用户后续层具有单独可学习参数. 跨任务共享数据允许网络利用任务之间的共性, 从而提高对所有用户的准确率. 大量训练样本使我们能够使用具有更多参数的更深层网络, 进一步提高性能.

跨多个任务学习还可以扩展到**元学习** (meta-learning), 也称" 学会学习"(learning to learn) . 多任务学习旨在对一组固定任务进行预测, 而元学习的目标是对训练期间未见过的未来任务进行预测. 这不仅可以通过跨任务学习共享内部表示来实现, 还可以通过学习学习算法本身来实现. 例如当新类的有标签样本非常少时, 元学习可以用于促进分类模型泛化到新类上, 称为**小样本学习** (few-shot learning); 当只使用一个有标签样本时, 则称为**单样本学习** (one-shot learning).

6.3.5 对比学习

对比学习 (contrastive learning) 是最常见和强大的表示学习方法之一 . 它背后的思想是学习一种表示, 使得在嵌入空间中某些输入对 (称为**正对**) 之间距离近, 而其他输入对 (称为**负对**) 之间距离远. 直觉上, 如果选择语义相似的为正对、语义不相似的为负对, 那么将学习到一个表示空间, 在该空间中相似输入之间距离近, 从而使完成下游任务 (如分类) 更容易.

与其他形式的表示学习一样, 我们通常不直接使用训练好的网络输出, 而是使用某个早期层的激活来构建嵌入空间. 对比学习任务与其他大多数机器学习任务不同: 给定输入的误差函数仅取决于其他输入, 而非每个输入单独的标签或目标输出.

给定数据点 $\mathbf{x}$(称为**锚点** anchor), 指定另一个数据点 $\mathbf{x}^+$ 与之组成一个正对; 还必须指定一组数据点 $\{\mathbf{x}_1^-, \cdots, \mathbf{x}_N^-\}$, 其中每个点都与 $\mathbf{x}$ 组成一个负对. 我们需要一个损失函数: 如果 $\mathbf{x}$ 和 $\mathbf{x}^+$ 的表示较为接近则给予奖励, 同时拉大 $\mathbf{x}$ 与每个 $\mathbf{x}^-$ 之间的距离.

一个示例, 也是对比学习中最常用的损失函数, 称为 InfoNCE 损失函数 , 其中 NCE 表示" 噪声对比估计"(noise contrastive estimation). 假设有一个神经网络函数 $f_{\mathbf{w}}(\mathbf{x})$, 用于将来自输入空间的点 $\mathbf{x}$ 映射到一个由可学习参数 $\mathbf{w}$ 控制的表示空间. 该表示是归一化的, 即 $\|f_{\mathbf{w}}(\mathbf{x})\| = 1$. 对于数据点 $\mathbf{x}$,InfoNCE 损失函数定义如下:

$$\mathcal{L}_{\text{NCE}} = -\ln\left(\frac{\exp(f_{\mathbf{w}}(\mathbf{x}) \cdot f_{\mathbf{w}}(\mathbf{x}^+)/\tau)}{\exp(f_{\mathbf{w}}(\mathbf{x}) \cdot f_{\mathbf{w}}(\mathbf{x}^+)/\tau) + \sum_{j=1}^{N} \exp(f_{\mathbf{w}}(\mathbf{x}) \cdot f_{\mathbf{w}}(\mathbf{x}_j^-)/\tau)}\right). \tag{6.20}$$

在这个函数中, 锚点表示 $f_{\mathbf{w}}(\mathbf{x})$ 和正样本表示 $f_{\mathbf{w}}(\mathbf{x}^+)$ 之间的余弦相似度给出了在习得空间中度量正对样本有多近的方法; 同样的方法也可用于评估锚点与负对样本之间的接近程度. 注意这个函数类似于分类交叉熵误差函数, 其中正对的余弦相似度给出了标签类的对数概率, 而负对的余弦相似度给出了错误类的对数概率. 负对非常重要: 如果没有它们, 嵌入空间将简单地学习将每个点映射到相同表示的退化解.

一个特定的对比学习算法主要由正对和负对的选择方式定义, 也就是如何使用先验知识来指定什么是好的表示. 例如考虑学习图像表示: 一个常见选择是通过破坏输入图像来创建正对, 这种方式保留了图像的语义信息, 同时极大地改变了像素空间中的图像 ; 破坏可以是旋转、平移和颜色转换. 数据集中其他图像可用来创建负对. 这种对比学习方法称为**个体判别** (instance discrimination).

然而如果可以访问类标签, 则可以使用同一类别的图像作为正对, 而使用不同类别的图像作为负对. 这放宽了对指定表征应保持不变的增强方法的依赖, 并且避免了将两个语义相似图像视为负对. 这称为**监督对比学习** (supervised contrastive learning) , 因为它依赖于类标签, 并且通常能得到比仅使用交叉熵分类学习的表示更好的结果.

正对和负对的成员不一定非要来自同一数据模态. 在对比语言–图像预训练 (Contrastive-Language Image Pre-training, CLIP) 中, 一个正对由一幅图像和对应的文本说明组成; 用两个单独函数将每个模态输入分别映射到相同表示空间, 其中 f 表示从图像到表示空间的映射,g 表示从文本输入到表示空间的映射. 还需要从数据集中获取一组图像 $\{\mathbf{x}_1, \cdots, \mathbf{x}_N\}$, 并假设对它们来说文本说明 $\mathbf{y}^+$ 是不恰当的, 还需一组同样不匹配输入图像 $\mathbf{x}$ 的文本说明 $\{\mathbf{y}_1^-, \cdots, \mathbf{y}_N^-\}$. 损失函数由下式给出:

$$\mathcal{L}_{\text{CLIP}} = -\ln\left(\frac{\exp(f(\mathbf{x}^+)\cdot g(\mathbf{y}^+)/\tau)}{\sum_{j=0}^{N}\exp(f(\mathbf{x}^+)\cdot g(\mathbf{y}_j^-)/\tau)}\right) - \ln\left(\frac{\exp(g(\mathbf{y}^+)\cdot f(\mathbf{x}^+)/\tau)}{\sum_{j=0}^{N}\exp(g(\mathbf{y}^+)\cdot f(\mathbf{x}_j^-)/\tau)}\right), \tag{6.21}$$

其中 $\mathbf{x}^+$ 和 $\mathbf{y}^+$ 表示一个正对. 损失函数中的两项确保了:(a) 图像表示与对应的文本说明表示相对于其他图像表示而言是接近的;(b) 文本说明表示与它所描述的图像表示相对于其他文本说明表示而言是接近的. 虽然 CLIP 使用文本–图像对, 但任何具有配对模态的数据集都可以用来学习表示. 这通常称为**弱监督学习**, 因为它依赖于带文本说明的图像, 这些数据通常从互联网上爬取, 比手动标注图像类别更容易.

6.3.6 通用网络结构

到目前为止我们探讨了组织成一系列全连接层的神经网络架构. 然而由于网络图与其数学函数之间存在直接对应关系, 我们可以通过考虑更复杂的网络图来设计更通用的网络映射. 这些网络必须限制为**前馈** (feed forward) 网络, 换句话说不能有闭合的有向环, 以确保输出是输入的确定性函数. 在这样的网络中, 每个 (隐藏或输出) 单元计算的函数为

$$a_k = \sum_{j\in\text{pa}(k)} w_{kj}z_j + b_k, \qquad z_k = h(a_k), \tag{6.22}$$

其中 $\text{pa}(k)$ 表示节点 k 的祖先 (ancestor) 集, 换句话说就是指向单元 k 的单元所构成的集合, 而 $\{b_k\}$ 表示相关联的偏置参数. 对于应用于网络输入的一组给定值, 连续应用 (6.22) 可以计算网络中所有单元的激活, 包括输出单元的激活.

6.3.7 张量

可以看到线性代数在神经网络中发挥着核心作用, 其中数据集、激活和网络参数等表示为标量、向量和矩阵. 此外我们还遇到了更高维度的变量. 例如考虑一个包含 N 张彩色图像的数据集, 每张图像高度为 I 像素, 宽度为 J 像素. 每个像素可通过其在图像内所处的行和列索引, 并具有红、绿、蓝值; 对数据集中每张图像我们都有这样一个值, 因此可以用一个四维数组 $\mathcal{X}$ 来表示:

$$x_{nijk}, \qquad i\in\{1,\cdots,I\},\ j\in\{1,\cdots,J\},\ k\in\{1,2,3\},\ n\in\{1,\cdots,N\}, \tag{6.23}$$

其中 i, j 是图像内行和列的索引,$k\in\{1,2,3\}$ 是红、绿、蓝强度索引,$n\in\{1,\cdots,N\}$ 是数据集中特定图像的索引.

这些高维数组称为**张量** (tensor), 而标量、向量和矩阵是张量的特例. 等到后面讨论更复杂的神经网络架构时, 我们会看到许多张量的例子. 像 GPU 这样的大规模并行处理器特别适合处理张量.

6.4 误差函数

在之前的章节中, 我们探讨了用于回归和分类的线性模型 (参见第 4 、 5 章), 并在此过程中推导出了误差函数的适当形式以及相应的输出单元激活函数. 这些选择误差函数所考虑的因素同样适用于多层网络, 为方便起见我们在此总结关键点.

6.4.1 回归

首先讨论回归问题. 此处暂时考虑可以取任意实值的单目标变量 t. 回到对单层网络中回归的讨论 (参见 2.3.4 小节), 假设 t 服从高斯分布, 其均值是以 $\mathbf{x}$ 为输入的神经网络的输出 $y(\mathbf{x}, \mathbf{w})$:

$$p(t \mid \mathbf{x}, \mathbf{w}) = \mathcal{N}\left(t \,\middle|\, y(\mathbf{x}, \mathbf{w}), \sigma^2\right), \tag{6.24}$$

其中 σ^2 是高斯噪声方差. 当然这是一个有点严格的假设, 在某些应用中我们需要扩展此方法以支持更一般的分布 (参见 6.5 节). 对于由 (6.24) 给出的条件分布, 将输出单元激活函数取为恒等函数就足够了, 因为这样的网络可以近似从 $\mathbf{x}$ 到 y 的任意连续函数.

给定由 N 个独立同分布观测值 $\mathbf{X} = \{\mathbf{x}_1, \cdots, \mathbf{x}_N\}$ 和相应目标值 $\mathbf{t} = \{t_1, \cdots, t_N\}$ 组成的数据集, 可以构造相应的似然函数:

$$p(\mathbf{t} \mid \mathbf{X}, \mathbf{w}, \sigma^2) = \prod_{n=1}^{N} \mathcal{N}\left(t_n \,\middle|\, y(\mathbf{x}_n, \mathbf{w}), \sigma^2\right). \tag{6.25}$$

在机器学习文献中, 通常考虑最小化误差函数而非最大化似然. 对 (6.25) 取负对数, 得到误差函数

$$E(\mathbf{w}) = \frac{1}{2\sigma^2} \sum_{n=1}^{N} \|y(\mathbf{x}_n, \mathbf{w}) - t_n\|^2 + \frac{N}{2} \ln \sigma^2 + \frac{N}{2} \ln(2\pi). \tag{6.26}$$

最大化似然首先用于确定 $\mathbf{w}$, 它等价于最小化下式给出的平方和误差函数 (此处舍弃了加性和乘性常数):

$$E(\mathbf{w}) = \frac{1}{2} \sum_{n=1}^{N} \{y(\mathbf{x}_n, \mathbf{w}) - t_n\}^2. \tag{6.27}$$

找到 $\mathbf{w}$ 后, 可通过最小化 (6.26) 来找到 σ^2 的值 (见习题 6.8):

$$\sigma^{2*} = \frac{1}{N} \sum_{n=1}^{N} \{y(\mathbf{x}_n, \mathbf{w}^*) - t_n\}^2, \tag{6.28}$$

并记为 σ^{2*}. 值得注意的是, 一旦完成找到 $\mathbf{w}$ 所需的迭代优化, 就可以计算该值.

如果有多个目标变量, 并假设它们在给定 $\mathbf{x}$ 和 $\mathbf{w}$ 条件下独立、且共享噪声方差 σ^2, 那么目标值的条件分布可写为

$$p(\mathbf{t} \mid \mathbf{x}, \mathbf{w}) = \mathcal{N}\left(\mathbf{t} \,\middle|\, \mathbf{y}(\mathbf{x}, \mathbf{w}), \sigma^2 \mathbf{I}\right). \tag{6.29}$$

遵循与单目标变量相同的论证, 关于权重最大化似然等价于最小化平方和误差函数 (见习题 6.9):

$$E(\mathbf{w}) = \frac{1}{2} \sum_{n=1}^{N} \|\mathbf{y}(\mathbf{x}_n, \mathbf{w}) - \mathbf{t}_n\|^2. \tag{6.30}$$

噪声方差则由下式给出:

$$\sigma^{2*} = \frac{1}{NK} \sum_{n=1}^{N} \|\mathbf{y}(\mathbf{x}_n, \mathbf{w}) - \mathbf{t}_n\|^2, \tag{6.31}$$

其中 K 是目标变量维度. 目标变量的条件独立性假设在所承受的优化问题稍微变复杂的代价下也是可以舍弃的 (见习题 6.10).

回想一下, 误差函数 (由负对数似然给出) 和输出单元激活函数之间存在自然的配对关系 (参见 5.4.6 小节). 在回归问题中, 可以将网络视为具有恒等输出单元激活函数, 即 $y = a$. 相应的平方和误差函数因此具有以下性质:

$$\frac{\partial E}{\partial a_k} = y_k - t_k. \tag{6.32}$$

6.4.2 二分类

接下来考虑二分类问题, 其中有一个单目标变量 $t \in \{0, 1\}$,$t = 1$ 表示类别 C_1,$t = 0$ 表示类别 C_2. 回到对规范连接函数的讨论 (参见 5.4.6 小节), 考虑具有单个输出的网络, 其激活函数是 sigmoid 函数 (6.12), 因此 $0 \leq y(\mathbf{x}, \mathbf{w}) \leq 1$. 我们可以将 $y(\mathbf{x}, \mathbf{w})$ 解释为条件概率 $p(C_1 \mid \mathbf{x})$, 而 $p(C_2 \mid \mathbf{x})$ 由 $1 - y(\mathbf{x}, \mathbf{w})$ 给出. 给定输入下, 目标的条件分布是伯努利分布:

$$p(t \mid \mathbf{x}, \mathbf{w}) = y(\mathbf{x}, \mathbf{w})^t \{1 - y(\mathbf{x}, \mathbf{w})\}^{1-t}. \tag{6.33}$$

如果考虑观测值独立的训练集, 则由负对数似然给出的误差函数为以下形式的交叉熵误差函数:

$$E(\mathbf{w}) = -\sum_{n=1}^{N} \{t_n \ln y_n + (1 - t_n) \ln(1 - y_n)\}, \tag{6.34}$$

其中 y_n 表示 $y(\mathbf{x}_n, \mathbf{w})$.Simard、Steinkraus and Platt(2003) 指出对于分类问题, 使用交叉熵误差函数而非平方和误差函数能加速训练并提高泛化能力.

注意在 (6.34) 中, 因为假设目标值被正确打标签, 所以不存在类似于噪声方差 σ^2 的概念. 可通过引入一个概率 ε 来表示目标值 t 被错误翻转的可能性, 从而轻松扩展以允许标签错误 (见习题 6.11); 这里 ε 可以预先设定, 或者视为一个超参数, 其值可从数据中推断.

如果有 K 个单独二分类任务要执行, 可以使用具有 K 个输出的网络, 每个输出都有一个 sigmoid 激活函数. 与每个输出相关联的是一个二元类标签 $t_k \in \{0, 1\}$, 其中 $k = 1, \cdots, K$. 如果我们假设在给定输入向量的情况下类标签独立, 则目标的条件分布为

$$p(\mathbf{t} \mid \mathbf{x}, \mathbf{w}) = \prod_{k=1}^{K} y_k(\mathbf{x}, \mathbf{w})^{t_k} \{1 - y_k(\mathbf{x}, \mathbf{w})\}^{1-t_k}. \tag{6.35}$$

取相应似然函数的负对数, 可以得出以下误差函数 (见习题 6.13):

$$E(\mathbf{w}) = -\sum_{n=1}^{N} \sum_{k=1}^{K} \{t_{nk} \ln y_{nk} + (1 - t_{nk}) \ln(1 - y_{nk})\}, \tag{6.36}$$

其中 y_{nk} 表示 $y_k(\mathbf{x}_n, \mathbf{w})$.

如果在误差函数中添加适当正则化项, 这种退化性将被消除 (参见第 9 章). 再次强调, 对于特定输出单元预激活, 误差函数的导数采用了我们熟悉的形式 (6.32), 即 $y_k - t_k$(见习题 6.14).

6.4.3 多分类

最后考虑多分类问题, 其中每个输入被分配到 K 个互斥类别中. 二元目标变量 $t_k \in \{0, 1\}$ 采用 1-of-K 编码方案表示类别 (参见 5.1.3 小节), 而网络输出解释为 $y_k(\mathbf{x}, \mathbf{w}) = p(t_k = 1 \mid \mathbf{x})$, 从而引出误差函数

$$E(\mathbf{w}) = -\sum_{n=1}^{N} \sum_{k=1}^{K} t_{nk} \ln y_k(\mathbf{x}_n, \mathbf{w}). \tag{6.37}$$

对应于规范连接的输出单元激活函数由 softmax 函数给出 (参见 5.4.4 小节):

$$y_k(\mathbf{x}, \mathbf{w}) = \frac{\exp(a_k)}{\sum_{j=1}^{K} \exp(a_j)}, \tag{6.38}$$

(6.38) 满足 $0 \leq y_k \leq 1$ 且 $\sum_k y_k = 1$. 注意, 如果将常数项添加到所有 $a_k(\mathbf{x}, \mathbf{w})$ 上, 导致误差函数在某些权重空间方向上保持不变, 则 $y(\mathbf{x}, \mathbf{w})$ 也不会改变; 添加适当正则化项可消除这种退化 (参见第 9 章).

对于具有 softmax 输出单元的网络, 误差函数关于特定预激活 a_k 的导数取与回归情况相同的形式 (6.32)(见习题 6.15).

总之, 根据所解决问题的类型, 输出单元激活函数和匹配的误差函数都有自然选择: 对于回归问题, 使用线性输出和平方和误差函数; 对于多个独立二分类问题, 使用 sigmoid 输出和交叉熵误差函数; 对于多分类问题, 使用 softmax 输出和相应的多类交叉熵误差函数. 对于涉及两个类别的分类问题 (即二分类问题), 可以使用单个 sigmoid 输出, 也可以使用由 softmax 函数激活的有两个输出的网络. 通过考虑其他形式的条件分布, 可以推导出相应的误差函数作为对应的负对数似然函数.

6.5 混合密度网络

到目前为止, 我们已经讨论了输出表示简单概率分布的神经网络, 包括连续变量的高斯分布以及离散变量的二元分布. 在本章最后一节, 我们将展示神经网络如何通过将网络输出视为更复杂分布 (如高斯混合分布) 的参数来表示更一般的条件概率. 通过这种方式定义关联的误差函数和相应输出单元激活函数, 我们称此类网络为**混合密度网络** (mixture density network).

6.5.1 机器人运动学示例

监督学习的目标是建模条件分布 $p(t \mid \mathbf{x})$. 对于许多简单回归问题, 选择高斯分布较为常见; 然而实际机器学习问题往往具有明显非高斯分布特征. 例如在分布可能是多模态的**逆问题** (inverse problem) 中, 高斯假设可能导致非常糟糕的预测.

作为逆问题的简单说明, 考虑机械臂的运动学 (图 6.16). **正问题** (forward problem) 涉及在给定关节角度时找到末端执行器位置, 并具有唯一解; 然而在实践中, 我们希望将末端执行器移动到特定位置, 为此必须设置适当的关节角度. 我们需要求解一个逆问题, 如图 6.16 所示, 该问题有两个解 (见习题 6.16), 对应于" 肘部向上" 和" 肘部向下" 两种配置.

正问题通常对应于物理系统中的因果关系, 通常具有唯一解——例如人体中特定症状可能由特定疾病引起. 然而在机器学习中, 我们通常需要求解逆问题, 例如在给定一组症状情况下预测疾病. 如果正问题涉及多对一映射, 则逆问题将具有许多解: 例如多种不同疾病可能导致相同症状.

在机器人运动学示例中, 运动学由几何方程定义, 其多模态性显而易见. 但在许多机器学习问题中, 多模态的存在可能不太明显, 特别是涉及高维空间的问题. 出于教学目的, 下面考虑一个简单小问题, 其多模态可以很容易可视化. 该问题数据通过在区间 $(0, 1)$ 上均匀采样变量 x 产生, 得到一组值 $\{x_n\}$; 通过计算函数 $x + 0.3 \sin(2\pi x)$, 然后添加在区间 $(-0.1, 0.1)$ 上均匀采样得到的噪声, 得到相应目标值.

图 6.17 左图展示了正问题数据集, 红色曲线显示了通过最小化平方和误差函数拟合两层神经网络的结果 (该网络具有 6 个隐藏单元和一个线性输出单元). 最小二乘对应于高斯假设下的最大似然: 它对正问题是一个很好的模型, 但对高度非高斯的逆问题却是一个很差的模型. 图 6.17 右图显示了通过交换 x 和 t 角色得到的逆问题——再次通过最小化平方和误差函数训练同样的网络, 由于数据的多模态性, 网络对数据的拟合效果较差.

6.5.2 条件混合分布

我们想要找到建模条件概率分布的一般框架. 这可以通过使用专为建模 $p(t \mid \mathbf{x})$ 设计的混合模型来实现, 其中混合系数和分量密度都是输入向量 $\mathbf{x}$ 的灵活函数, 从而得到**混合密度网络** (mixture density network). 对于任何给定 $\mathbf{x}$ 值, 混合模型为建模任意条件密度函数 $p(t \mid \mathbf{x})$ 提供一般形式; 如果有足够灵活的网络, 我们就有了一个可以近似任意条件概率分布的框架.

在这里我们显式地为高斯分量设计模型如下:

$$p(t \mid \mathbf{x}) = \sum_{k=1}^{K} \pi_k(\mathbf{x}) \mathcal{N}\left(t \,\middle|\, \mu_k(\mathbf{x}), \sigma_k^2(\mathbf{x})\right). \tag{6.39}$$

除了高斯分布, 还可以为分量使用其他分布: 例如如果目标变量是二元变量而非连续变量, 则可以使用伯努利分布. 虽然这里使用各向同性协方差, 但也可以通过使用 Cholesky 因式分解表示协方差来使混合密度网络支持一般协方差矩阵 . 即使使用各向同性分量, 由于混合分布的影响, 条件分布 $p(t \mid \mathbf{x})$ 也不具备对 t 各分量的可分解性 (与标准平方和回归模型不同).

令混合模型的各个参数——即混合系数 $\pi_k(\mathbf{x})$、均值 $\mu_k(\mathbf{x})$ 和方差 $\sigma_k^2(\mathbf{x})$——由以 $\mathbf{x}$ 为输入的神经网络的输出决定. 混合密度网络结构如图 6.18 所示. 混合密度网络与混合专家模型密切相关 : 主要区别在于混合专家模型对自身每个分量模型都有独立参数, 而在混合密度网络中, 我们使用相同的函数来预测所有分量密度的参数和混合系数, 于是非线性隐藏单元在依赖于输入的函数之间是共享的.

如果混合模型 (6.39) 中有 K 个分量, 并且 t 有 L 个分量, 则网络将具有:K 个输出单元预激活 (用 a_k^π 表示) 用于确定混合系数 $\pi_k(\mathbf{x})$;K 个输出 (用 a_k^σ 表示) 用于确定高斯标准差 $\sigma_k(\mathbf{x})$;$K \times L$ 个用 a_{kj}^μ 表示的输出用于确定高斯均值分量 $\mu_{k,j}(\mathbf{x})$. 网络输出总数由 $L + 2K$ 给出, 这与具有 L 个输出、用于预测目标变量条件均值的网络不同. 例如图 6.18 中的神经网络可以是具有 S 形 tanh 隐藏单元的两层网络.

混合系数必须满足下列约束:

$$\sum_{k=1}^{K} \pi_k(\mathbf{x}) = 1, \qquad 0 \leq \pi_k(\mathbf{x}) \leq 1, \tag{6.40}$$

这可以使用一组 softmax 输出来实现:

$$\pi_k(\mathbf{x}) = \frac{\exp(a_k^\pi)}{\sum_{j=1}^{K} \exp(a_j^\pi)}. \tag{6.41}$$

同样, 方差必须满足 $\sigma_k^2(\mathbf{x}) \geq 0$, 因而 $\sigma_k(\mathbf{x})$ 可以使用相应网络预激活的指数来表示:

$$\sigma_k(\mathbf{x}) = \exp(a_k^\sigma). \tag{6.42}$$

最后由于均值 $\mu_k(\mathbf{x})$ 具有实分量, 可以用网络输出直接表示:

$$\mu_{k,j}(\mathbf{x}) = a_{kj}^\mu, \tag{6.43}$$

其中输出单元激活函数由恒等式 $f(a) = a$ 给出.

6.5.3 梯度优化

混合密度网络的可学习参数包括神经网络中的权重和偏置向量 $\mathbf{w}$, 它可以通过最大似然来确定, 也可以等价地通过最小化定义为似然负对数的误差函数来确定. 对于彼此独立的数据, 误差函数采用以下形式 (其中显式给出了关于 $\mathbf{w}$ 的依赖):

$$E(\mathbf{w}) = -\sum_{n=1}^{N} \ln \left\{ \sum_{k=1}^{K} \pi_k(\mathbf{x}_n, \mathbf{w}) \mathcal{N}\left(t_n \,\middle|\, \mu_k(\mathbf{x}_n, \mathbf{w}), \sigma_k^2(\mathbf{x}_n, \mathbf{w})\right) \right\}. \tag{6.44}$$

为了最小化误差函数, 需要计算误差 $E(\mathbf{w})$ 关于参数 $\mathbf{w}$ 的导数. 稍后我们将看到如何自动计算这些导数 (参见第 8 章). 然而显式地为输出单元预激活的误差导数导出合适表达式是有帮助的, 因为这强调了这些量的概率解释. 由于误差函数 (6.44) 由一些项的和组成, 其中每项对应一个训练数据点, 因此可以考虑特定输入向量 $\mathbf{x}_n$ 与相关目标向量 t_n 的导数; 总误差 E 的导数通过对所有数据点求和获得, 每个数据点自身的梯度也可以直接用于基于梯度的优化算法 (参见第 7 章).

引入以下变量将有助于推导:

$$\gamma_{nk} = \frac{\pi_k(\mathbf{x}_n) \mathcal{N}\left(t_n \,\middle|\, \mu_k(\mathbf{x}_n), \sigma_k^2(\mathbf{x}_n)\right)}{\sum_{j=1}^{K} \pi_j(\mathbf{x}_n) \mathcal{N}\left(t_n \,\middle|\, \mu_j(\mathbf{x}_n), \sigma_j^2(\mathbf{x}_n)\right)}, \tag{6.45}$$

其中 $\mathcal{N}_k$ 表示 $\mathcal{N}(t_n \mid \mu_k(\mathbf{x}_n), \sigma_k^2(\mathbf{x}_n))$. 这些量可被自然解释为混合分布分量的后验概率, 其中混合系数 $\pi_k(\mathbf{x})$ 可以视为与 $\mathbf{x}$ 相关的先验概率 (见习题 6.17).

关于控制混合系数的网络输出预激活 a_k^π 的误差函数导数由下式给出 (参见习题 6.18):

$$\frac{\partial E_n}{\partial a_k^\pi} = \pi_k(\mathbf{x}_n) - \gamma_{nk}. \tag{6.46}$$

类似地, 关于控制分量均值的网络输出预激活 a_{kj}^μ 的误差函数导数由下式给出 (见习题 6.19):

$$\frac{\partial E_n}{\partial a_{kj}^\mu} = \gamma_{nk}\frac{\mu_{k,j}(\mathbf{x}_n) - t_{n,j}}{\sigma_k^2(\mathbf{x}_n)}. \tag{6.47}$$

最后, 关于控制分量方差的网络输出预激活 a_k^σ 的误差函数导数由下式给出 (见习题 6.20):

$$\frac{\partial E_n}{\partial a_k^\sigma} = \gamma_{nk}\left\{\frac{\|\mathbf{t}_n - \boldsymbol{\mu}_k(\mathbf{x}_n)\|^2}{\sigma_k^2(\mathbf{x}_n)} - 1\right\}. \tag{6.48}$$

6.5.4 预测分布

混合密度网络训练好后, 给定任意输入向量值, 就可以预测目标数据的条件密度函数. 就预测输出向量值的问题而言, 该条件密度表示对数据生成器的完整描述. 从这个密度函数中, 我们可以计算出不同应用可能关注的更具体量. 其中最简单的一个量是平均值, 它对应于目标数据的条件平均:

$$\mathbb{E}[t \mid \mathbf{x}] = \int t\, p(t \mid \mathbf{x})\, dt = \sum_{k=1}^{K} \pi_k(\mathbf{x})\, \mu_k(\mathbf{x}). \tag{6.49}$$

由于使用最小二乘法训练的标准网络会近似条件均值, 因此我们看到混合密度网络可以作为一个特例再现传统最小二乘结果. 当然正如我们已经指出的, 对于多峰分布, 条件均值的值是有限的.

我们可以类似地评估关于条件均值密度函数的方差 (见习题 6.21):

$$\mathbb{V}[t \mid \mathbf{x}] = \mathbb{E}[\|t - \mathbb{E}[t \mid \mathbf{x}]\|^2 \mid \mathbf{x}], \tag{6.50}$$

展开后为

$$\mathbb{V}[t \mid \mathbf{x}] = \sum_{k=1}^{K} \pi_k(\mathbf{x})\left\{\sigma_k^2(\mathbf{x}) + \|\boldsymbol{\mu}_k(\mathbf{x}) - \mathbb{E}[t \mid \mathbf{x}]\|^2\right\}. \tag{6.51}$$

这里用到了 (6.39) 和 (6.49). 该结果比相应的最小二乘结果更普遍, 因为方差是关于 $\mathbf{x}$ 的函数. 这是一个**异方差** (heteroscedastic) 模型的示例: 数据上的噪声方差是输入向量 $\mathbf{x}$ 的函数.

让我们回到图 6.17 所示的逆问题示例来说明混合密度网络的使用方法. 图 6.19 显示了混合系数 $\pi_k(\mathbf{x})$、均值 $\mu_k(\mathbf{x})$ 和对应于 $p(t \mid \mathbf{x})$ 的条件密度等值线. 神经网络的输出 (即混合模型中的参数) 必然是输入变量的连续单值函数; 然而从图 6.19(c) 可以看到, 该模型能够通过调制混合分量 $\pi_k(\mathbf{x})$ 的振幅来产生条件密度, 该密度对 x 的某些值是单峰的, 而对其他值是三峰的. 该模型有 3 个高斯分量, 并使用一个两层神经网络, 该网络有 5 个 S 形 tanh 隐藏单元和 9 个输出 (分别对应于 3 个高斯分量的均值、 3 个方差以及 3 个混合系数). 在 x 取小值或大值时, 目标数据条件概率密度是单峰的, 只有一个高斯分量先验概率很高; 而在 x 取中间值时, 条件概率密度是三峰的, 3 个混合系数具有差不多的值.

我们已经看到, 对于多峰分布, 条件均值给出了较差的数据表示. 例如在图 6.16 所示简单机械臂控制中, 我们需要从两种可能关节角度设置中选择一种来获得所需末端执行器位置, 但两种解的平均值本身并不是一个解. 在这种情况下, 条件众数可能更有价值. 由于混合密度网络的条件众数没有简单解析解, 因此需要进行数值迭代. 一种简单替代方法是在每个 x 值处取最可能分量 (即混合系数最大的分量) 的平均值. 这一做法已在图 6.19(d) 所示示例数据集上给出.

第 7 章 梯度下降

7.1 错误平面

与前面讨论的回归模型和分类模型一样, 我们通过优化误差函数来选择网络参数. 我们的任务是基于一组训练数据, 为网络参数 (权重和偏置) 寻找合适设置. 在第 6 章中我们看到, 神经网络是一类非常宽泛且灵活的函数, 原则上可以使用足够多隐藏单元以高精度逼近所期望函数; 此外深度网络能编码对应于层次化表示的归纳偏置. 本章中, 我们为便于后续讨论, 首先把这些参数统一表示为向量 $\mathbf{w}$, 并基于预先选定的误差函数 $E(\mathbf{w})$ 来优化 $\mathbf{w}$.

为了直观理解这一优化过程, 可以建立误差函数的几何图像: 将其视为一个位于" 权重空间" 之上的曲面. 首先注意, 如果在权重空间中从 $\mathbf{w}$ 向 $\mathbf{w}+\delta\mathbf{w}$ 迈出一小步, 那么误差函数变化由下式给出:

$$\delta E \approx \delta\mathbf{w}^{\mathsf{T}}\nabla E(\mathbf{w}), \tag{7.1}$$

其中 $\nabla E(\mathbf{w})$ 指向误差函数增长率最大的方向.$\mathbf{w}$ 是一个局部极小值点, 而 $\mathbf{w}^*$ 是全局最小值点, 因此有 $E(\mathbf{w}^*) < E(\mathbf{w})$. 在任意点 $\mathbf{w}$ 处, 误差曲面的局部梯度由 ∇E 给出. 我们可以朝着 $-\nabla E(\mathbf{w})$ 的方向迈出一小步, 从而进一步减小误差. 只要 $E(\mathbf{w})$ 是 $\mathbf{w}$ 的平滑连续函数, 它的最小值就会出现在权重空间中梯度为零的点处, 即

$$\nabla E(\mathbf{w}) = \mathbf{0}. \tag{7.2}$$

梯度为零的点称为**驻点** (stationary point, 参见 7.1.1 小节), 驻点可进一步分类为极小点、极大点和鞍点.

我们需要找到这样一个向量 $\mathbf{w}$ 使 $E(\mathbf{w})$ 取最小值. 然而误差函数通常对权重和偏置参数高度非线性地依赖, 因此权重空间中能使梯度为零 (或数值非常小) 的点有很多. 实际上对于任意局部极小值点, 权重空间中通常存在其他等效的局部极小值点. 例如在图 6.1 所示具有 M 个隐藏单元的两层网络中, 每个点都是 $M!\times 2^M$ 个等效点集合中的一员 (参见 6.2.4 小节).

不仅如此, 权重空间中还可能存在多个本质上并不等效的驻点, 尤其是多个非等效的局部极小值点. 在整个权重空间 $\mathbf{w}$ 中, 对应于误差函数最小取值的那个极小值点称为**全局最小值点** (global minimum), 其他所有对应于相对较高误差函数值的极小值点则称为**局部极小值点** (local minima). 深度网络的误差曲面形态可能极为复杂, 人们曾认为基于梯度的优化方法可能陷入性能欠佳的局部极小值区域; 然而在实践中, 大型网络模型在各种不同初始条件下通常都能收敛到性能相近的解 (见 9.3.2 节).

7.1.1 局部二次近似

可以通过对误差函数进行局部二次近似来深入理解优化问题及各种优化技术. 在权重空间中某点 $\hat{\mathbf{w}}$ 周围对 $E(\mathbf{w})$ 做泰勒展开:

$$E(\mathbf{w}) = E(\hat{\mathbf{w}}) + (\mathbf{w}-\hat{\mathbf{w}})^{\mathsf{T}}\mathbf{b} + \frac{1}{2}(\mathbf{w}-\hat{\mathbf{w}})^{\mathsf{T}}\mathbf{H}(\mathbf{w}-\hat{\mathbf{w}}), \tag{7.3}$$

其中省略了三阶及更高阶项. 这里 $\mathbf{b}$ 定义为我们在 $\hat{\mathbf{w}}$ 处评估的 E 的梯度:

$$\mathbf{b} = \nabla E(\hat{\mathbf{w}}), \tag{7.4}$$

黑塞矩阵 (Hessian) 定义为相应的二阶导数矩阵:

$$\mathbf{H}(\mathbf{w}) = \nabla\nabla E(\mathbf{w}). \tag{7.5}$$

如果网络中总共有 W 个权重和偏置, 那么 $\mathbf{w}$ 和 $\mathbf{b}$ 的长度为 W,$\mathbf{H}$ 的维度为 $W\times W$. 根据 (7.3), 对应的局部梯度近似由下式给出:

$$\nabla E(\mathbf{w}) = \mathbf{b} + \mathbf{H}(\mathbf{w}-\hat{\mathbf{w}}). \tag{7.6}$$

对于足够接近 $\hat{\mathbf{w}}$ 的点 $\mathbf{w}$, 这些表达式将给出误差及其梯度的合理近似.

考虑围绕误差函数最小值点 $\mathbf{w}^*$ 进行局部二次近似这一特定情况. 在这种情况下, 因为在 $\mathbf{w}^*$ 处 $\nabla E = \mathbf{0}$, 所以

没有线性项, (7.3) 变为

$$E(\mathbf{w}) = E(\mathbf{w}^*) + \frac{1}{2}(\mathbf{w} - \mathbf{w}^*)^{\mathsf{T}}\mathbf{H}(\mathbf{w} - \mathbf{w}^*), \tag{7.7}$$

其中黑塞矩阵 $\mathbf{H}$ 在点 $\mathbf{w}^*$ 处求得. 为从几何角度解释这一点, 考虑黑塞矩阵的特征方程:

$$\mathbf{H}\mathbf{u}_i = \lambda_i \mathbf{u}_i, \tag{7.8}$$

其中特征向量 $\mathbf{u}_i$ 形成一组完备正交规范集 (参见附录 A), 所以有

$$\mathbf{u}_i^{\mathsf{T}}\mathbf{u}_j = \delta_{ij}. \tag{7.9}$$

这可以视为坐标系变换: 原点平移到 $\mathbf{w}^*$, 轴与特征向量对齐. 写

$$\mathbf{w} - \mathbf{w}^* = \sum_i \alpha_i \mathbf{u}_i. \tag{7.10}$$

将 (7.10) 代入 (7.7) 并使用 (7.8) 和 (7.9), 误差函数可写成如下形式 (见习题 7.1):

$$E(\mathbf{w}) = E(\mathbf{w}^*) + \frac{1}{2}\sum_i \lambda_i \alpha_i^2. \tag{7.11}$$

假设对所有 $i \neq j$ 设置 $\alpha_j = 0$, 然后变化 α_i, 这相当于把 $\mathbf{w}$ 沿着 $\mathbf{u}_i$ 方向移动、远离 $\mathbf{w}^*$. 从 (7.11) 可见: 若对应特征值 $\lambda_i > 0$, 则误差函数值会增大; 反之误差函数值将减小. 如果所有特征值都是正的, 则 $\mathbf{w}^*$ 对应误差函数的局部极小值; 如果所有特征值都是负的, 则 $\mathbf{w}^*$ 对应局部最大值; 如果特征值有正有负, 则 $\mathbf{w}^*$ 表示一个**鞍点** (saddle point).

如果对所有 $\mathbf{v}$ 有

$$\mathbf{v}^{\mathsf{T}}\mathbf{H}\mathbf{v} > 0, \tag{7.12}$$

则称黑塞矩阵 $\mathbf{H}$ 为**正定矩阵** (positive definite). 因为特征向量 $\mathbf{u}_i$ 形成完备集, 任意向量 $\mathbf{v}$ 可以写成

$$\mathbf{v} = \sum_i c_i \mathbf{u}_i, \tag{7.13}$$

由 (7.8) 和 (7.9) 有

$$\mathbf{v}^{\mathsf{T}}\mathbf{H}\mathbf{v} = \sum_i c_i^2 \lambda_i. \tag{7.14}$$

从而如果所有特征值都是正的,$\mathbf{H}$ 将是正定矩阵 (见习题 7.2). 因此,$\mathbf{w}^*$ 是局部极小值的充分必要条件是: 误差函数的梯度在 $\mathbf{w}^*$ 处为零, 并且在 $\mathbf{w}^*$ 处评估的黑塞矩阵是正定矩阵 (见习题 7.3).

在新的坐标系中, 基向量由特征向量 $\mathbf{u}_i$ 给出, 恒定误差 $E(\mathbf{w})$ 的等高线变为轴对齐的椭圆, 以 $\mathbf{w}^*$ 为中心. 椭圆的轴与黑塞矩阵的特征向量 $\mathbf{u}_i$ 对齐, 轴的长度则与相应特征值 λ_i 的平方根成反比 (见习题 7.6).

对于神经网络这种具有复杂结构的误差函数, 试图找到方程 $\nabla E(\mathbf{w}) = \mathbf{0}$ 的解析解是不现实的, 因此需要借助迭代式数值优化方法.

7.2 梯度下降优化

除了最简单的神经网络模型之外, 绝大多数网络的误差曲面都具有极其复杂的形态, 因此最终求得的解将很大程度上依赖于初始参数值 $\mathbf{w}^{(0)}$ 的选取等因素. 为获得一个足够好的解, 通常需要多次运行基于梯度的优化算法, 每次都选用一个不同的随机初始点, 最终通过在一个独立验证集上进行性能评估来确定最优解.

大多数优化技术首先要为权重向量选取一个初始值 $\mathbf{w}^{(0)}$, 然后按照如下迭代规则在权重空间中逐步搜索:

$$\mathbf{w}^{(\tau)} = \mathbf{w}^{(\tau-1)} + \Delta\mathbf{w}^{(\tau-1)}, \tag{7.15}$$

其中 τ 标记迭代步骤. 不同算法对权重向量如何更新 $\Delta\mathbf{w}^{(\tau)}$ 有不同选择. 连续非线性函数的优化问题一直是学术界广泛研究的课题.

7.2.1 梯度信息的使用

在 (7.3) 给出的误差函数二次近似表达式中, 误差曲面形态由向量 $\mathbf{b}$ 和矩阵 $\mathbf{H}$ 共同决定, 它们总共包含 $W(W+3)/2$ 个独立元素 (由于 $\mathbf{H}$ 具有对称性), 其中 W 表示权重向量 $\mathbf{w}$ 的维度 (即网络中可学习参数总数, 见习题 7.7). 因此该二次近似所对应的最小值位置取决于 $O(W^2)$ 个参数, 这意味着我们至少需要收集到 $O(W^2)$ 个相互独立的信息片段才能准确定位到该最小值. 如果不考虑梯度信息, 预计需要 $O(W^2)$ 次误差函数数值评估, 而每次评估又需要 $O(W)$ 步计算操作. 这样一来, 采用这种策略寻找最小值所需的总计算量将高达 $O(W^3)$.

现在将上述情况与利用梯度信息的优化算法进行对比. 由于梯度向量 ∇E 的长度为 W, 每次计算 ∇E 都能提供 W 个独立信息片段. 据此推断, 我们有望仅通过 $O(W)$ 次梯度评估就能找到函数最小值 (参见第 8 章). 正如后续章节将展示的那样, 利用误差反向传播算法, 每次梯度评估的计算复杂度仅为 $O(W)$, 因此现在能够在 $O(W^2)$ 的总计算复杂度内找到最小值. 尽管二次近似的有效性仅限于最小值附近的局部区域, 但这种效率提升具有普遍性. 正是基于这一关键优势, 利用梯度信息成了当前所有实用神经网络训练算法的基础.

深度网络的误差函数梯度可以通过误差反向传播技术高效计算 (参见第 8 章), 因此应用这些梯度信息可以显著提高网络训练速度.

7.2.2 批量梯度下降

利用梯度信息指导权重更新的一种最朴素方法是在 (7.15) 中按照负梯度方向进行小步长的迭代调整:

$$\mathbf{w}^{(\tau)} = \mathbf{w}^{(\tau-1)} - \eta \nabla E(\mathbf{w}^{(\tau-1)}), \tag{7.16}$$

其中参数 $\eta > 0$ 被定义为**学习率** (learning rate). 在完成每次权重更新后, 需要基于新权重向量 $\mathbf{w}^{(\tau)}$ 重新计算梯度, 并重复上述过程. 在每一次迭代过程中, 权重向量都朝着当前位置处误差函数下降最快的方向进行更新, 因此这种方法称为**梯度下降** (gradient descent), 或称**最速下降** (steepest descent).

注意, 由于误差函数是基于整个训练集进行定义的, 因此每次计算 ∇E 都需要遍历所有训练样本. 这种一次性处理全部训练数据的技术称为**批处理** (batch) 方法.

7.2.3 随机梯度下降

深度学习方法的性能通常会随训练数据集规模扩大而显著提升. 然而当训练集中包含海量数据时, 批处理方法的效率会急剧下降, 因为每一次误差函数或梯度的计算都需要遍历整个数据集. 为寻求更高效优化策略, 我们注意到: 对于一组独立同分布观测样本, 基于最大似然估计构建的误差函数可以自然地分解为每个数据点对应项的和:

$$E(\mathbf{w}) = \sum_{n=1}^{N} E_n(\mathbf{w}). \tag{7.17}$$

在处理大规模数据集时, 应用最广泛的训练算法是基于梯度下降的**随机梯度下降** (Stochastic Gradient Descent, SGD) 逐次迭代算法 . 该算法每次仅基于一个数据点来更新权重向量, 具体更新规则如下:

$$\mathbf{w}^{(\tau)} = \mathbf{w}^{(\tau-1)} - \eta \nabla E_n(\mathbf{w}^{(\tau-1)}), \tag{7.18}$$

其中索引 n 在每次迭代后循环递增. 上述更新过程通过循环遍历训练集中数据点来反复执行. 对整个训练集进行一次完整遍历称为一个**训练轮次** (epoch). 当数据以流的形式持续生成时, 这种技术有时也称为**在线梯度下降** (online gradient descent).

与批量梯度下降相比, 随机梯度下降能更高效地处理数据中的冗余. 为理解这一点, 考虑一个极端例子: 复制某数据集的每个数据点, 把规模扩大一倍. 这种情况下误差函数数值就是原来两倍; 如果相应调整学习率的值, 则仍可使用原来的误差函数. 对于批处理方法, 由于数据集加倍, 评估批误差函数梯度所需计算量也将加倍, 而随机梯度下降算法则完全不受影响.

随机梯度下降的另一个重要特性是其具备逃离局部极小值的能力, 因为整个数据集误差函数的驻点通常并不会是每个单独数据点对应误差函数的驻点.

7.2.4 小批量方法

随机梯度下降的一个缺点是: 基于单个数据点计算出的梯度可能大大偏离整个数据集上误差函数的梯度. 为兼顾计算效率和估计精度, 我们折衷一下, 每次迭代时使用一小批数据点 (称为**小批量**,mini-batch) 来估计梯度. 在确定小批量合适大小时需要注意: 利用 N 个样本估计均值的误差为 $\sigma/\sqrt{N}$(参见练习 7.8), 其中 σ 为生成这些数据的分布的标准差. 这表明通过增加小批量规模来提升梯度估计精度的收益是递减的——例如将小批量规模增加 100 倍, 估计误差仅能降低到原先的十分之一. 选择小批量规模时还需要考虑充分利用运行代码的底层硬件架构: 例如在某些硬件平台上, 小批量规模如果是 2 的幂次 (如 64、128、256 等) 就可以获得更好性能.

在使用小批量方法时, 要注意数据点应当从整个数据集中随机选取. 因为在原始数据集中, 由于数据采集方式 (例如数据点按字母顺序或日期排序), 相邻数据点之间可能存在相关性. 为消除这种相关性的影响, 通常做法是先将整个数据集进行随机打乱, 然后按顺序划分为多个小批量; 此外在每次完整遍历数据集后, 还可以再次打乱数据以确保每个小批量的数据组合都是全新的, 这有助于模型跳出局部最优解. 即使采用了小批量方法, 这种学习算法通常仍被称为" 随机梯度下降".

7.2.5 参数初始化

像梯度下降这样的迭代式优化算法都要求我们为需要学习的参数设定初始值. 参数初始化的具体策略会对算法收敛速度以及训练得到网络的泛化能力产生显著影响; 遗憾的是目前还没有能够系统性指导参数初始化过程的完善理论.

尽管如此, 参数初始化过程中一个关键考虑因素是**对称性破缺** (symmetry breaking). 考虑网络中一组隐藏单元或输出单元, 它们都接收相同输入信号. 如果这些单元的所有参数都被赋予相同值 (例如全部初始化为零), 那么这些参数在训练过程中将始终步调一致地进行更新, 并且每个单元都计算出相同函数, 导致功能上冗余. 为解决这一问题, 可以从某个预先定义分布中随机采样来初始化参数, 以打破这种对称性. 在计算资源允许的情况下, 可以采用不同随机初始化策略多次训练网络, 最终选择在独立验证集上性能最佳的模型.

用于初始化权重的分布通常要么是范围为 $[-\epsilon, \epsilon]$ 的均匀分布, 要么是形式为 $\mathcal{N}(0, \epsilon^2)$ 的零均值高斯分布.ϵ 值的选择很重要; 在各种经验方法中, 何恺明初始化 得到了广泛使用. 考虑一个网络, 其中第 l 层计算以下转换:

$$a_i^{(l)} = \sum_j w_{ij}^{(l)} z_j^{(l-1)}, \qquad z_i^{(l)} = \mathrm{ReLU}(a_i^{(l)}), \tag{7.19}$$

其中 j 是发送连接到单元 i 的单元数,ReLU 激活函数由式 (6.17) 给出. 假设使用高斯分布 $\mathcal{N}(0, \sigma_w^2)$ 初始化权重, 并假设第 $(l-1)$ 层中单元的输出 $z_j^{(l-1)}$ 具有零均值和方差 s^2(见习题 7.9), 则可以容易地证明

$$\mathbb{E}[a_i^{(l)}] = 0, \qquad \mathrm{var}[a_i^{(l)}] = M s^2 \sigma_w^2. \tag{7.20}$$

其中的因子来自 ReLU 激活函数. 理想情况下我们希望确保在从一层传播到下一层时, 预激活的方差既不会衰减到零, 也不会显著增长. 如果因此要求第 l 层单元也具有方差 s^2, 那么对于一个具有 M 个输入的单元, 用于初始化其权重的高斯分布标准差为:

$$\sigma_w = \sqrt{\frac{2}{M}}. \tag{7.21}$$

我们还可以将初始化分布的尺度 ϵ 视为一个超参数, 并在多次训练中探索不同的 ϵ 值. 偏置参数通常设置为小的正值, 以确保大多数预激活在学习开始时处于激活状态——这对 ReLU 特别有帮助, 我们希望预激活为正值, 以便有非零梯度来驱动学习过程.

除上述方法外, 另一类重要参数初始化策略是利用在其他相关任务上预训练得到的网络权重, 或者利用各种无监督学习方法. 这些技术都属于迁移学习 (transfer learning) 的范畴 (参见 6.3.4 节).

7.3 收敛

在实际应用梯度下降算法时, 需要谨慎选择学习率 η 的值. 考虑图 7.3 所示情形: 这是一个假想二维权重空间中的简化误差曲面, 该曲面的曲率在不同方向上存在显著差异, 形成一个狭长的" 山谷" 结构. 对误差曲面上大多数点而言, 批量梯度下降算法计算出的局部负梯度 $-\nabla E$(与该点等高线垂直) 并不直接指向最小值点. 直观上我们可能认为增大学习率 η 应能让算法在权重空间中迈出更大步伐, 从而更快收敛到最小值; 然而如果一味加大步长, 权重更新会在山谷两侧来回震荡, 当 η 超过某个阈值时这种震荡甚至会变得越来越剧烈, 导致算法无法收敛. 为避免这种发散振荡, 学习率 η 必须设定得足够小, 那么沿着山谷方向向最小值靠近的速度又可能变得非常缓慢. 在这种情况下, 梯度下降需要经过多次小步迭代才能逐渐逼近最小值, 导致整体优化效率非常低下.

7.3.1 收敛性分析

通过考虑误差函数在最小值附近的二次近似, 可以深入了解该问题的本质. 根据 (7.7)、(7.8) 和 (7.10), 在这种近似中误差函数梯度可以写为

$$\nabla E = \sum_i \lambda_i \alpha_i \mathbf{u}_i. \tag{7.22}$$

再次使用 (7.10), 可以将权重向量的变化表示为对应系数 α_i 的变化:

$$\Delta \mathbf{w} = \sum_i \Delta \alpha_i \mathbf{u}_i. \tag{7.23}$$

结合 (7.22)、 (7.23) 与梯度下降公式 (7.16), 并利用黑塞矩阵特征向量的正交性关系 (7.9), 可以推导出每次迭代步中 α_i 的变化量:

$$\alpha_i^{(\tau)} = (1 - \eta\lambda_i)\, \alpha_i^{(\tau-1)}. \tag{7.24}$$

定义 α_i 和 $\alpha_i^{(\tau)}$ 分别代表权重更新前后的值. 利用特征向量正交性关系 (7.9) 和式 (7.10), 有

$$\mathbf{u}_i^{\mathsf{T}}(\mathbf{w} - \mathbf{w}^{(\tau)}) = \alpha_i^{(\tau)}. \tag{7.25}$$

因此 α_i 可以解释为沿着方向 $\mathbf{u}_i$ 到最小值的距离. 从 (7.24) 可见, 这些距离独立地演化, 每一步中沿着 $\mathbf{u}_i$ 方向的距离乘以因子 $(1 - \eta\lambda_i)$. 经过总共 T 步之后有

$$\alpha_i^{(T)} = (1 - \eta\lambda_i)^T\, \alpha_i^{(0)}. \tag{7.26}$$

由此可见, 只要满足 $|1 - \eta\lambda_i| < 1$, 当 $T \to \infty$ 时 $\alpha_i^{(T)} \to 0$, (7.25) 表明 $\mathbf{w} \to \mathbf{w}^*$, 因此权重向量已经收敛到误差函数最小值.

注意 (7.26) 证明了梯度下降在最小值附近区域能够带来**线性收敛**. 同样要收敛到驻点, 则要求所有 λ_i 都是正的, 这反过来意味着驻点确实是最小值. 通过提高学习率 η, 可以使因子 $(1 - \eta\lambda_i)$ 更小, 进而提高收敛速度; 然而 η 的设定有一个限制: 虽然允许 $(1 - \eta\lambda_i)$ 为负 (这会产生 α_i 的振荡值), 但必须确保 $|1 - \eta\lambda_i| < 1$, 否则 α_i 值会发散. 于是 η 的值需要限制为 $\eta < 2/\lambda_{\max}$, 其中 $\lambda_{\max}$ 是最大特征值. 然而收敛速度主要由最小特征值控制, 因此当 η 设定为最大允许值时, 沿着对应于最小特征值方向的收敛 (图 7.3 中椭圆长轴) 将由下式控制:

$$\alpha_{\min}^{(T)} = (1 - \eta\lambda_{\min})^T\, \alpha_{\min}^{(0)}, \tag{7.27}$$

其中 $\lambda_{\min}$ 是最小特征值. 如果 $\lambda_{\max}/\lambda_{\min}$ 的比值 (其倒数为黑塞矩阵的条件数) 非常小, 即图 7.3 中椭圆形误差等高线的高度将被拉长, 那么向最小值收敛的速度将极其缓慢.

7.3.2 动量

解决特征值差异悬殊问题的一种简单技巧是在梯度下降公式中引入**动量** (momentum) 项. 这实际上相当于给权重向量在权重空间中的运动增加了惯性, 从而缓解了图 7.3 中所示的振荡现象. 引入动量项后的梯度下降更新公式为:

$$\Delta \mathbf{w}^{(\tau-1)} = -\eta\, \nabla E(\mathbf{w}^{(\tau-1)}) + \mu\, \Delta \mathbf{w}^{(\tau-2)}, \tag{7.28}$$

其中 μ 称为动量参数, 然后使用 (7.15) 更新权重向量.

为理解动量项作用, 首先考虑权重向量在误差曲面上曲率相对较小区域中的运动情况. 如果近似认为梯度在该区域内保持不变, 那么可以对 (7.28) 进行多次迭代, 并将由此产生的一系列权重更新值累加起来:

$$\Delta \mathbf{w} = -\eta \nabla E \left(1 + \mu + \mu^2 + \cdots\right) = -\frac{\eta}{1-\mu} \nabla E. \tag{7.29}$$

可以看到, 添加动量项能有效将学习率从 η 提高到 $\eta/(1-\mu)$. 在曲率较低表面上, 使用固定学习率的梯度下降会导致逐渐变小的步长, 对应于线性收敛; 这种情况下添加动量项的效果类似于提高学习率, 向误差函数谷底的收敛更快.

相反, 在误差曲面有较高曲率区域, 如果不加动量项, 梯度下降会出现振荡现象. 在这种情况下, 动量项的连续贡献将趋于相互抵消, 使得有效学习率接近于 η. 因此动量项能够加速模型向最小值点收敛, 同时又避免了发散的振荡.

尽管引入动量项可以改善梯度下降性能, 但它也引入了一个新参数 μ, 其值需要与学习率参数 η 一起合理选择. 从 (7.29) 可以看出,μ 的取值范围应为 $0 \leq \mu \leq 1$. 实践中常用值是 $\mu = 0.9$.

研究者提出了称作 **Nesterov 动量**的动量改进方法, 可以进一步加速模型收敛 . 在传统带动量项的随机梯度下降中, 我们首先计算当前位置的梯度, 然后利用该梯度与前一步动量项的加权组合来确定当前更新步长. 而在 Nesterov 方法中这两步顺序被交换: 具体来说, 我们首先基于前一步动量项进行一次预更新, 然后在这个预更新后的位置计算梯度, 并利用该梯度来确定最终更新步长, 其更新公式如下:

$$\Delta \mathbf{w}^{(\tau-1)} = -\eta \nabla E\left(\mathbf{w}^{(\tau-1)} + \mu \Delta \mathbf{w}^{(\tau-2)}\right) + \mu \Delta \mathbf{w}^{(\tau-2)}. \tag{7.30}$$

对于批量梯度下降算法,Nesterov 动量通常能够提高收敛速度; 然而在随机梯度下降场景下, 其加速效果可能并不明显.

7.3.3 学习率调度

在随机梯度下降学习算法 (7.18) 中, 需要确定学习率 η 的值. 如果 η 值过小, 学习过程将异常缓慢 (参见 7.3.1 小节); 如果 η 值过大, 则可能导致算法不稳定. 尽管一定程度的振荡是可以接受的, 但这种振荡不应导致发散现象.

在实际应用中, 通常在训练初期使用较大的 η 值, 然后随着迭代进行逐渐减小学习率, 即让学习率成为迭代步数 τ 的函数:

$$\mathbf{w}^{(\tau)} = \mathbf{w}^{(\tau-1)} - \eta^{(\tau-1)} \nabla E_n(\mathbf{w}^{(\tau-1)}). \tag{7.31}$$

学习率衰减的例子包括线性衰减、幂律衰减和指数衰减, 公式依次如下:

$$\eta^{(\tau)} = \left(1 - \frac{\tau}{K}\right)\eta_0 + \frac{\tau}{K}\eta_K, \qquad \tau \leq K, \tag{7.32}$$

$$\eta^{(\tau)} = \eta_0 \left(1 + \tau/s\right)^{-c}, \tag{7.33}$$

$$\eta^{(\tau)} = \eta_0 \, c^{\tau}. \tag{7.34}$$

在 (7.32) 中,η 的值在 K 步内线性减小, 之后保持为 η_K. 超参数 η_0、η_K、K、s 和 c 的最优值通常需要借助经验才能找到. 在实践中, 监督学习曲线 (learning curve)——即误差函数随梯度下降迭代过程的变化情况——对调整学习率衰减策略很有好处.

7.3.4 AdaGrad、RMSProp 与 Adam 算法

前文阐述了最优学习率取决于误差曲面在局部的曲率, 并且这种曲率会随参数空间中方向不同而变化. 这就启发了一系列针对网络中各个参数采用不同学习率的优化算法. 这些学习率的值在训练过程中会自动进行调整.

AdaGrad 是" 自适应梯度"(adaptive gradient) 的缩写 , 其算法核心思想是随着时间推移不断减小每个参数的学习率, 而减小幅度则取决于该参数对应的所有历史梯度值的平方和. 因此与较大曲率相关联的参数, 其对应学习

率会下降得更快. 具体来说其更新规则如下:

$$r_i^{(\tau)} = r_i^{(\tau-1)} + g_i^2, \qquad g_i = -\frac{\partial E}{\partial w_i}, \tag{7.35}$$

$$w_i^{(\tau)} = w_i^{(\tau-1)} - \frac{\eta}{\sqrt{r_i^{(\tau)} + \delta}} g_i, \tag{7.36}$$

其中 η 是学习率,δ 是一个小常数 (比如 10^{-6}), 旨在确保 r_i 接近零时的数值稳定性.$E(\mathbf{w})$ 是特定小批量的误差函数, (7.36) 是标准随机梯度下降公式, 但学习率针对每个参数进行了调整. 该算法初始值是 $r_i^{(0)} = 0$.

AdaGrad 算法的缺点是它从训练一开始就累积平方梯度, 因此相关的权重更新可能变得非常小, 这可能在训练后期大大减缓训练进度.

RMSProp 算法背后的思想是均方根传播 (root mean square propagation), 使用梯度平方的指数加权平均来替代 AdaGrad 中的简单累加 , 更新规则如下:

$$r_i^{(\tau)} = \beta\, r_i^{(\tau-1)} + (1-\beta)\, g_i^2, \tag{7.37}$$

$$w_i^{(\tau)} = w_i^{(\tau-1)} - \frac{\eta}{\sqrt{r_i^{(\tau)} + \delta}} g_i, \tag{7.38}$$

其中 $0 < \beta < 1$, 典型值为 $\beta = 0.9$.

Adam 优化算法 的名字来源于" 自适应矩估计"(adaptive moments). 它是 RMSProp 算法与动量法的结合, 为每个参数分别维护一个动量项. 其更新规则基于梯度及其平方的指数加权移动平均, 具体形式为:

$$s_i^{(\tau)} = \beta_1\, s_i^{(\tau-1)} + (1-\beta_1)\, g_i, \tag{7.39}$$

$$r_i^{(\tau)} = \beta_2\, r_i^{(\tau-1)} + (1-\beta_2)\, g_i^2, \tag{7.40}$$

$$\hat{s}_i = \frac{s_i^{(\tau)}}{1-\beta_1^{\tau}}, \qquad \hat{r}_i = \frac{r_i^{(\tau)}}{1-\beta_2^{\tau}}, \tag{7.41}$$

$$w_i^{(\tau)} = w_i^{(\tau-1)} - \frac{\eta}{\sqrt{\hat{r}_i} + \delta} \hat{s}_i. \tag{7.42}$$

这里的系数 $1/(1-\beta_1^{\tau})$ 和 $1/(1-\beta_2^{\tau})$ 纠正了初始设置 $s_i^{(0)}$ 和 $r_i^{(0)}$ 为零引入的偏差 (见习题 7.12). 由于 $\beta \leq 1$ 且随 τ 变大偏差趋于零, 因此实践中有时会省略这种偏差修正. 权重参数典型值为 $\beta_1 = 0.9$ 和 $\beta_2 = 0.99$.Adam 算法是深度学习应用中最常用的学习算法.

需要注意的是, 这种直观理解实际上仅在主曲率方向与参数空间坐标轴对齐的情况下才真正适用, 这对应于黑塞矩阵局部为对角矩阵的情形, 尽管在实际应用中这种情况不太可能发生. 但这几个算法仍然被广泛使用并发挥作用.

7.4 正则化

在神经网络前向传播过程中, 对计算得到的变量进行归一化处理可以避免网络处理过大或过小的值. 尽管原则上神经网络的权重和偏置可以适应各种不同输入和隐藏变量的值, 但在实践中, 归一化对于确保有效的训练至关重要. 根据归一化操作所针对的对象不同, 我们将归一化技术分为三种类型: 数据归一化、批量归一化和层归一化.

7.4.1 数据归一化

有时会遇到这样的数据集, 其中不同输入变量的取值范围差异很大. 例如在医疗健康数据中, 患者身高可能以米为单位测量 (如 1.8 米), 而他们的血小板计数可能以每微升血液中的血小板数量来衡量 (如每微升 300 000 个血小板). 这种差异会让梯度下降训练过程更困难: 考虑一个具有两个权重的单层回归网络, 其两个对应输入变量具有截然不同取值范围. 其中一个权重的微小变化, 相比于另一个权重的类似变化, 会对输出乃至误差函数产生大得

多的影响. 这会产生一个沿着不同坐标轴曲率差异极大的误差曲面, 如图 7.3 所示.

因此对于连续型输入变量, 将输入值重新缩放使其具有相似取值范围通常是非常有益的. 这很容易做到: 首先分别计算每个输入变量的均值和方差

$$\mu_i = \frac{1}{N}\sum_{n=1}^{N} x_{ni}, \tag{7.43}$$

$$\sigma_i^2 = \frac{1}{N}\sum_{n=1}^{N}(x_{ni} - \mu_i)^2, \tag{7.44}$$

这是在开始任何训练之前都需要进行的计算. 然后使用以下方法缩放输入值:

$$\tilde{x}_{ni} = \frac{x_{ni} - \mu_i}{\sigma_i}, \tag{7.45}$$

这样缩放后的值 $\tilde{x}_{ni}$ 具有零均值和单位方差 (见习题 7.14). 请注意, 必须使用相同的 μ_i 值和 σ_i 值来预处理任何开发集、验证集或测试集数据, 以确保对所有输入都以相同方式进行缩放.

7.4.2 批量归一化

我们已经了解了数据归一化的重要性, 类似思路也可以应用于深度网络中每一隐藏层的变量. 如果某个特定隐藏层的激活值范围存在很大差异, 那么将这些值归一化为零均值和单位方差应该可以简化后续网络层的学习问题. 然而与数据归一化可在训练开始前一次性完成不同, 隐藏单元值的归一化需要在训练过程中每次权重更新后都重复进行. 这种方法称为**批量归一化** (batch normalization) .

引入批量归一化也是为了解决**梯度消失**和**梯度爆炸**问题, 它们通常出现在训练非常深的神经网络时. 根据微积分中的链式法则, 误差函数 E 相对于网络第一层中参数的梯度由下式给出:

$$\frac{\partial E}{\partial w_{ij}^{(1)}} = \sum_k \cdots \sum_{j'} \frac{\partial E}{\partial z_k^{(K)}} \cdots \frac{\partial z_j^{(1)}}{\partial a_i^{(1)}} \frac{\partial a_i^{(1)}}{\partial w_{ij}^{(1)}}, \tag{7.46}$$

其中 $z_k^{(K)}$ 表示第 K 层中节点 k 的激活值, 式 (7.46) 右侧每个偏导数 (参见 8.1.5 小节) 代表那一层雅可比矩阵的元素. 如果这些偏导数项中大多数的绝对值小于 1, 那么大量此类项的乘积将趋于 0; 反之如果大多数项的绝对值大于 1, 乘积将趋于无穷大. 因此随着网络深度增加, 误差函数梯度可能变得非常大或非常小. 批量归一化方法在很大程度上解决了这个问题.

为明确批量归一化的具体定义, 考虑多层网络中某个特定层. 该层中每个隐藏单元都对其输入的预激活值 $z_j = h(a_j)$ 计算一个非线性函数, 因此我们可以选择对预激活值 a_j 或激活值 z_j 进行归一化. 在实践中二者均可, 这里以归一化预激活值 a_j 为例说明过程. 由于权重值在每个小批量之后都会更新, 因此需要对每个小批量分别归一化. 具体来说, 对于一个大小为 K 的小批量, 定义

$$\mu_B = \frac{1}{K}\sum_{n=1}^{K} a_n, \tag{7.47}$$

$$\sigma_B^2 = \frac{1}{K}\sum_{n=1}^{K}(a_n - \mu_B)^2, \tag{7.48}$$

$$\hat{a}_n = \frac{a_n - \mu_B}{\sqrt{\sigma_B^2 + \delta}}, \tag{7.49}$$

其中对 $n = 1, \cdots, K$ 的求和操作针对小批量中的元素进行;δ 是一个很小常数, 引入它是为了避免当 σ_B^2 很小时出现因为除数趋于零而导致的各种数值问题.

通过对网络中某一层的预激活值进行归一化, 我们减少了该层参数自由度, 从而降低了它的表达能力. 我们可以通过将预激活值重新缩放为具有平均值 β_j 和标准差 γ_j 来补偿这一点:

$$y_j = \gamma_j \hat{a}_n + \beta_j, \tag{7.50}$$

其中 β_j 和 γ_j 是可学习的自适应参数, 它们与网络权重和偏置一起通过梯度下降法进行联合学习. 这些可学习参

数是批量归一化与输入数据归一化的一个关键区别: 在式 (7.50) 中, 它们直接由独立参数 β_j 和 γ_j 决定, 这些参数在梯度下降过程中更容易学习.

式 (7.50) 可能看起来只是抵消了批量归一化的效果, 因为均值和方差现在又可以适应任意值了. 然而关键区别在于参数在训练过程中的演变方式: 对于原始网络, 一个小批量的均值和方差由该层所有权重和偏置的复杂函数决定; 而在式 (7.50) 中, 它们直接由独立参数 β_j 和 γ_j 决定. 式 (7.49) 和式 (7.50) 描述了一个相对于小批量数据点的、由可学习参数 β 和 γ 决定的可微变量转换. 这可视为神经网络中的附加层, 因此每个标准隐藏层后面都可以加一个批量归一化层.

当完成网络训练、想要对新数据进行预测时, 我们并没有训练时的小批量数据可用, 也不能仅从一个数据样本确定均值和方差. 为解决这个问题, 原则上可以在对权重和偏置进行最后更新之后, 针对每个层在整个训练集上计算 μ 和 σ^2; 然而为计算这些统计量就需要对整个数据集再处理一遍, 计算成本太高. 于是选择在整个训练阶段计算移动平均值:

$$\mu^{(\tau)} = \alpha\,\mu^{(\tau-1)} + (1-\alpha)\mu_B, \tag{7.51}$$

$$\sigma^{2(\tau)} = \alpha\,\sigma^{2(\tau-1)} + (1-\alpha)\sigma_B^2, \tag{7.52}$$

其中 $0 \le \alpha \le 1$. 这些移动平均值在训练期间不起作用, 仅用于在推理阶段处理新数据点.

尽管批量归一化在实践中非常有效, 但其内在原理目前还不明确. 批量归一化提出的原因是: 人们观察到网络中较早层权重的更新会改变后续层所看到的数值分布, 这种现象称为**内部协变量偏移** (internal covariate shift). 然而后来的研究 表明, 协变量偏移并不是一个关键因素, 批量归一化所带来训练改进实际上在于它平滑了误差函数的崎岖程度.

7.4.3 层归一化

在批量归一化中, 如果批量规模太小, 那么均值和方差的估计会包含太多噪声. 另外, 对于非常大的训练集, 小批量可能会被分散在不同 GPU 上处理, 使得跨小批量的全局归一化效率低下. 一种替代方案是分别针对每个数据点对隐藏单元的值进行归一化, 而不是在小批量中的样本间进行归一化. 这称为**层归一化** (layer normalization) .

层归一化对每个样本在同一层的所有隐藏单元之间归一化, 从而不依赖小批量统计, 对批量大小不敏感, 在循环神经网络和小批量训练场景中尤为有效.

第 8 章 反向传播

8.1 梯度计算

本章旨在寻找一种高效技术, 用于评估前馈神经网络误差函数 $E(\mathbf{w})$ 的梯度. 我们将看到, 这可以通过一种本地消息传递方案来实现: 信息会通过网络向后发送, 称为**误差反向传播** (error backpropagation). 许多具有实际意义的误差函数 (例如由一组独立同分布数据的最大似然定义的误差函数) 都包含一些项的总和 (每一项对应训练集中一个数据点), 因此

$$E(\mathbf{w}) = \sum_{n=1}^{N} E_n(\mathbf{w}). \tag{8.1}$$

在此我们将考虑如何为误差函数中这样一项的 $\nabla E(\mathbf{w})$ 求值. 这可以直接用于随机梯度下降法, 也可以将结果累积到一组训练数据点上, 用于批量处理或小批量处理方法. 下面先为具有任意前馈拓扑结构、任意可微非线性激活函数和一大类误差函数的通用网络推导出反向传播算法, 然后使用一个简单分层网络结构 (具有单层 S 形隐藏单元以及平方和误差) 来说明所得公式.

注意" 反向传播" 一词在神经计算文献中有多种含义: 有的神经计算文献可能把前馈结构称作反向传播网络; 此外" 反向传播" 一词有时也用来描述神经网络端到端训练程序, 包括梯度下降参数更新. 本书专门使用" 反向传播" 来描述用于对导数进行数值计算的计算程序, 例如计算关于网络权重和偏置的误差函数梯度. 这一程序也可用于计算其他重要导数, 如雅可比矩阵和黑塞矩阵.

8.1.1 单层网络

考虑一个简单线性模型, 其中输出 y_k 是输入变量 x_i 的线性组合:

$$a_k = \sum_i w_{ki} x_i. \tag{8.2}$$

对于特定输入数据点 n, 平方和误差函数形式为

$$E_n = \frac{1}{2} \sum_k \{y_k(x, \mathbf{w}) - t_k\}^2, \tag{8.3}$$

其中 $y = y(x, \mathbf{w})$,t_k 是关联目标值. 该误差函数相对于权重 w_{ji} 的梯度为

$$\frac{\partial E_n}{\partial w_{ji}} = (y_j - t_j) \frac{\partial y_j}{\partial a_j} x_i. \tag{8.4}$$

这可以理解为" 本地" 计算: 涉及与链路输出端 w_{ji} 相关的" 误差信号" $(y_j - t_j)$, 以及与链路输入端相关的变量 x_i 的乘积. 5.4.3 小节介绍了逻辑斯谛 sigmoid 激活函数与交叉熵误差函数、softmax 激活函数和与之匹配的多元交叉熵误差函数是如何产生类似公式的. 接下来探究这一简单结果如何扩展到更复杂的多层前馈网络情境.

8.1.2 一般前馈网络

一般来说, 前馈网络由一组单元组成, 每个单元计算其输入的加权和:

$$a_j = \sum_i w_{ji} z_i, \tag{8.5}$$

其中 z_i 是另一个单元的激活值, 或是向单元 j 发送连接的输入单元, 而 w_{ji} 是与该连接相关的权重. 此外, 可以通过引入一个额外单元或输入 (激活值固定为 +1) 来将偏置包含在该总和中, 因此不需要显式处理偏置 (参见 6.2 节). 在通过非线性激活函数 $h(\cdot)$ 进行转换后, 式 (8.5) 中的和 (称为预激活) 给出单元 j 的激活值 z_j:

$$z_j = h(a_j). \tag{8.6}$$

请注意, 式 (8.5) 中参与求和的一个或多个变量 z_i 可以是输入变量; 同样, 式 (8.6) 中的单元 j 也可以是输出变量. 对于训练集中每个数据点, 假设已向网络提供相应输入向量, 并通过连续应用式 (8.5) 和式 (8.6) 计算出网络中所有隐藏单元和输出单元的激活值. 这一过程叫作**前向传播** (forward propagation), 因为它可以被视为信息通过网络的前向流动.

下面计算相对于权重 w_{ji} 的导数 $\partial E_n/\partial w_{ji}$. 各单元输出将取决于特定输入数据点 n; 不过为使符号简洁, 省略网络变量下标 n. 首先注意到 E 对 w_{ji} 的依赖仅体现在输入 a_j 对单元 j 的求和上. 因此可以应用偏导数的链式法则, 得出

$$\frac{\partial E}{\partial w_{ji}} = \frac{\partial E}{\partial a_j}\frac{\partial a_j}{\partial w_{ji}} = \frac{\partial E}{\partial a_j} z_i. \tag{8.7}$$

接下来引入一个有用符号:

$$\delta_j \equiv \frac{\partial E}{\partial a_j}, \tag{8.8}$$

其中 δ_j 通常称为**误差**, 缘由后续很快揭晓. 利用式 (8.5) 可以写出

$$\frac{\partial E}{\partial w_{ji}} = \delta_j z_i. \tag{8.9}$$

将式 (8.8) 和式 (8.5) 代入式 (8.7), 得到式 (8.9). 根据式 (8.9), 只需将权重输出端单元的 δ 值乘以权重输入端单元的 z 值, 即可得到所需导数. 请注意这与式 (8.4) 中简单线性模型形式相同. 因此为了计算导数, 只需计算网络中每个隐藏单元和输出单元的 δ 值, 然后应用式 (8.9) 即可.

如前所述, 对于输出单元有

$$\delta_k = \frac{\partial E}{\partial a_k}, \tag{8.10}$$

前提是使用规范连接函数作为输出单元激活函数 (参见 5.4.6 小节). 为计算隐藏单元的 δ 值, 再次使用偏导数链式法则:

$$\delta_j = \frac{\partial E}{\partial a_j} = h'(a_j)\sum_k w_{kj}\delta_k, \tag{8.11}$$

其中求和操作遍及所有单元 k, 且单元 j 向单元 k 发送连接. 注意在写出式 (8.11) 时, 我们利用了这样一个事实: 只有通过改变变量 a_j 的值,a_j 的变化才会引起误差函数的变化; 所有 k 的单元包括其他隐藏单元和/或输出单元.

如果将式 (8.8) 给出的 δ_j 定义代入式 (8.11), 并利用式 (8.5) 和式 (8.6), 即可得出以下反向传播公式 (见习题 8.1):

$$\delta_j = h'(a_j)\sum_k w_{kj}\delta_k. \tag{8.12}$$

由此可知, 特定隐藏单元的 δ_j 值可以通过从网络中较高单元后向传播误差信息来获得. 注意反向传播 (8.12) 中的求和取自 w_{kj} 上的第一个索引 (对应于信息通过网络后向传播), 而在前向传播式 (8.5) 中求和取自第二个索引 (见习题 8.2). 由于已知输出单元的 δ 值, 因此通过递归应用式 (8.12), 可以计算前馈网络中所有隐藏单元的 δ 值, 而无须考虑其拓扑结构.

在上述推导中, 我们隐式假设网络中每个隐藏单元或输出单元都具有相同激活函数 $h(\cdot)$. 不过只需跟踪 $h(\cdot)$ 的哪种形式与哪个单元相匹配, 就可以轻松地将推导结果拓展到不同单元, 使其拥有单独激活函数.

对于批量处理方法, 对训练集中每个数据点重复上述步骤, 然后对批量处理或小批量处理中所有数据点求和, 即可得到总误差 E 的导数:

$$\frac{\partial E}{\partial w_{ji}} = \sum_n \delta_{j,n} z_{i,n}. \tag{8.13}$$

8.1.3 简单示例

上述推导采用了误差函数、激活函数和网络拓扑的一般形式. 为演示反向传播的应用, 考虑图 6.1 所示两层网络以及平方和误差. 输出单元具有线性激活函数, 因此 $y_k = a_k$; 而隐藏单元具有 sigmoid 型激活函数, 其计算公

式为

$$h(a) = \tanh(a), \tag{8.14}$$

其中 $\tanh(a)$ 由式 (6.13) 定义. 该函数一个有用特征是其导数可以用一种特别简单的形式来表示:

$$h'(a) = 1 - h(a)^2. \tag{8.15}$$

考虑平方和误差函数, 对于数据点 n, 误差计算公式为

$$E_n = \frac{1}{2}\sum_k (y_k - t_k)^2, \tag{8.16}$$

其中 y_k 是输出单元 k 的激活值,t_k 是特定输入向量 $\mathbf{x}$ 对应的目标值.

对于训练集中每个数据点, 首先使用以下方法依次进行前向传播:

$$a_j^{(1)} = \sum_{i=0}^{D} w_{ji}^{(1)} x_i, \qquad z_j^{(1)} = \tanh(a_j^{(1)}), \tag{8.17}$$

$$a_k^{(2)} = \sum_{j=0}^{M} w_{kj}^{(2)} z_j^{(1)}, \qquad y_k = a_k^{(2)}, \tag{8.18}$$

其中 D 是输入向量 $\mathbf{x}$ 的维度,M 是隐藏单元总数. 此外利用了 $x_0 = z_0 = 1$, 以便在权重中包含偏置参数.

用以下方法计算每个输出单元的 δ 值:

$$\delta_k^{(2)} = y_k - t_k. \tag{8.19}$$

然后利用如下方法对误差进行反向传播, 从而得出隐藏单元的 δ 值:

$$\delta_j^{(1)} = (1 - z_j^{(1)2}) \sum_k w_{kj}^{(2)} \delta_k^{(2)}. \tag{8.20}$$

式 (8.20) 可根据式 (8.12) 和式 (8.15) 得出. 最后, 第一层和第二层权重的导数为

$$\frac{\partial E_n}{\partial w_{kj}^{(2)}} = \delta_k^{(2)} z_j^{(1)}, \qquad \frac{\partial E_n}{\partial w_{ji}^{(1)}} = \delta_j^{(1)} x_i. \tag{8.21}$$

8.1.4 计算复杂度与数值微分

反向传播最重要的方面之一是计算效率. 为此考察计算误差函数导数所需的计算操作数如何随网络中权重和偏置总数 W 变化.

在 W 足够大的情况下, 计算误差函数导数 (给定输入数据点) 需要 $O(W)$ 次运算. 这是因为除连接非常稀疏的网络外, 权重数量通常远大于单元数量, 因此前向传播的大部分计算量来自式 (8.5) 中的求和操作, 而计算激活函数的工作量却很小. 式 (8.5) 中参与求和的每一项都需要一次乘法和一次加法运算, 因此总计算成本为 $O(W)$.

计算误差函数导数的另一种反向传播方法是使用有限差分法. 具体做法是依次扰动每个权重, 并利用该表达式近似求导来实现:

$$\frac{\partial E_n}{\partial w_{ji}} \approx \frac{E_n(w_{ji} + \epsilon) - E_n(w_{ji})}{\epsilon}, \qquad \epsilon \ll 1. \tag{8.22}$$

在软件模拟中, 可以通过将 ϵ 最小化来提高导数近似精度, 直到出现数值舍入问题为止. 使用对称中心差分法可以显著提高有限差分法精度:

$$\frac{\partial E_n}{\partial w_{ji}} \approx \frac{E_n(w_{ji} + \epsilon) - E_n(w_{ji} - \epsilon)}{2\epsilon} + O(\epsilon^2). \tag{8.23}$$

在这种情况下, 正如式 (8.23) 右侧泰勒展开所验证的,$O(\epsilon)$ 修正被抵消了, 因此残差修正为 $O(\epsilon^2)$(见习题 8.3). 但请注意, 计算步骤数量相比式 (8.22) 增加了大约一倍.

数值微分法的主要问题是失去了非常理想的 $O(W)$ 缩放. 每次前向传播需要执行 $O(W)$ 个步骤, 而网络中有 W 个权重, 每个权重都必须单独扰动, 因此总计算成本为 $O(W^2)$. 不过数值微分法在实践中也能发挥重要作用, 因为将直接实现反向传播或自动微分法计算出的导数与使用中心差分法计算出的导数进行比较, 可以有力地检验

软件的正确性.

8.1.5 雅可比矩阵与黑塞矩阵

我们已经了解如何通过网络反向传播误差来获得误差函数相对于权重的导数. 反向传播也可用于计算其他导数. 首先对雅可比矩阵进行求值, 其元素由网络输出相对于输入的导数给出:

$$J_{ki} = \frac{\partial y_k}{\partial x_i}. \tag{8.24}$$

我们可以使用反向传播程序来计算雅可比矩阵, 该程序与前面计算误差函数相对于权重导数的程序类似. 首先写下元素 J_{ki} 的形式, 利用式 (8.5):

$$J_{ki} = \frac{\partial y_k}{\partial a_j}\frac{\partial a_j}{\partial x_i} = \sum_j \frac{\partial y_k}{\partial a_j} w_{ji}, \tag{8.25}$$

其中式 (8.25) 中的求和操作涉及所有单元 j(例如前面提到的第一隐藏层所有单元), 且输入单元 i 向单元 j 发送连接.

分层拓扑结构中第一隐藏层所有单元的 $\partial y_k/\partial a_j$ 的递归反向传播公式如下:

$$\frac{\partial y_k}{\partial a_j} = h'(a_j)\sum_l w_{lj}\frac{\partial y_k}{\partial a_l}, \tag{8.26}$$

其中求和操作涉及所有单元 l, 且单元 j 向单元 l 发送连接 (对应于 w_{lj} 的第一个索引). 这种反向传播从输出单元开始, 所需导数可直接从输出单元激活函数的函数形式中得到. 对于线性输出单元, 可以得出

$$\frac{\partial y_k}{\partial a_{k'}} = \delta_{kk'}, \tag{8.27}$$

其中 $\delta_{kk'}$ 是单位矩阵元素. 如果在每个输出单元上都有独立逻辑斯谛 sigmoid 激活函数, 则有

$$\frac{\partial y_k}{\partial a_{k'}} = y_k(\delta_{kk'} - y_{k'}). \tag{8.28}$$

而对于 softmax 输出, 可以得出

$$\frac{\partial y_k}{\partial a_{k'}} = y_k(\delta_{kk'} - y_{k'}). \tag{8.29}$$

由于雅可比矩阵度量了输出变量对每个输入变量变化的局部灵敏度, 因此它还允许与输入相关的任何已知误差 Δx_i 通过经训练网络传播, 从而预估它们对输出误差的贡献 Δy_k:

$$\Delta y_k \approx \sum_i J_{ki}\Delta x_i. \tag{8.30}$$

一般来说, 训练后神经网络所代表的网络映射是非线性的, 因此雅可比矩阵元素不是常数, 而取决于所使用的特定输入向量. 式 (8.30) 只对输入的微小扰动有效, 而且必须为每个新输入向量重新计算雅可比矩阵.

雅可比矩阵在由多个不同模块构建的系统中发挥重要作用. 每个模块都可以包含一个固定或可学习函数, 该函数可以是线性或非线性的, 但必须可微. 假设希望最小化与参数 $\mathbf{w}$ 有关的误差函数 E, 该误差函数的导数为

$$\frac{\partial E}{\partial \mathbf{w}} = \frac{\partial E}{\partial \mathbf{y}}\frac{\partial \mathbf{y}}{\partial \mathbf{z}}\frac{\partial \mathbf{z}}{\partial \mathbf{w}}, \tag{8.31}$$

其中红色模块所代表的雅可比矩阵出现在式 (8.31) 右侧中间项.

反向传播也可用于计算误差的二阶导数:

$$H_{ij} = \frac{\partial^2 E}{\partial w_i \partial w_j}, \tag{8.32}$$

方便起见通常将所有权重和偏置参数视为单个向量 $\mathbf{w}$ 的元素,$i, j \in \{1, \cdots, W\}$ 是权重和偏置总数. 在这种情况下, 二阶导数构成黑塞矩阵 $\mathbf{H}$ 的元素 H_{ij}.

由于神经网络可能包含数百万甚至数十亿参数, 因此对许多模型的全黑塞矩阵进行计算甚至只进行存储都是不可行的. 由于计算量被缩放为 $O(W^2)$, 求黑塞矩阵的逆将变得更加困难. 业内人士对寻找有效的全黑塞矩阵近似值很感兴趣. 一种近似法是只对黑塞矩阵对角元素进行计算, 并隐式地将非对角元素设为零; 这需要 $O(W^2)$ 存储空间, 并允许在 $O(W)$ 步骤内求逆, 但仍需要 $O(W^2)$ 计算步骤. 不过通过进一步近似, 计算步骤可以减少到 $O(W)$

个；但实际上黑塞矩阵通常具有重要非对角项，因此必须谨慎对待这一近似.

另一种更有说服力的方法是**外积近似法**. 考虑使用平方和误差函数的回归应用，平方和误差函数为

$$E = \frac{1}{2}\sum_n (y_n - t_n)^2. \tag{8.33}$$

其中只使用一个单一输出以保持符号简洁 (扩展到多个输出也很简单). 然后写下黑塞矩阵 (见习题 8.8):

$$\mathbf{H} = \nabla\nabla E = \sum_n \nabla y_n (\nabla y_n)^{\mathsf{T}} + \sum_n (y_n - t_n)\nabla\nabla y_n. \tag{8.34}$$

其中 ∇ 表示相对于 $\mathbf{w}$ 的梯度. 如果网络已在数据集上经过训练，则输出 y_n 将非常接近目标值 t_n，式 (8.34) 中末尾项很小，可以忽略不计.

忽略式 (8.34) 右侧二阶导数项，即可得到**列文伯格–马夸尔特** (Levenberg–Marquardt) 近似法，也叫**外积近似法**，因为黑塞矩阵是由向量外积之和构成的:

$$\mathbf{H} \approx \sum_n \nabla y_n (\nabla y_n)^{\mathsf{T}}. \tag{8.35}$$

外积近似法之所以方便，是因为仅涉及误差函数的一阶导数，使用标准反向传播法即可在 $O(W)$ 个步骤内高效计算一阶导数；而通过简单乘法运算，只需 $O(W^2)$ 个步骤即可求出矩阵元素. 需要强调的是，这种近似法可能只适用于经过适当训练的网络，且对于一般网络映射，式 (8.34) 右侧二阶导数项通常不可忽略.

对于具有逻辑斯谛 sigmoid 输出单元激活函数的网络，其交叉熵误差函数的相应近似值为

$$\mathbf{H} \approx \sum_n y_n(1 - y_n)\nabla y_n (\nabla y_n)^{\mathsf{T}}. \tag{8.36}$$

对于具有 softmax 输出单元激活函数的多类网络，均可得到类似结果 (见习题 8.11). 外积近似法还可用于开发一种高效序列程序，用于近似黑塞矩阵的逆 (见习题 8.12).

有时我们并非直接需要黑塞矩阵，而只需要黑塞矩阵与某个向量 $\mathbf{v}$ 的乘积 $\mathbf{Hv}$. 通过扩展反向传播程序，可以在 $O(W)$ 个步骤内高效计算出该乘积 .

8.2 自动微分法

我们已经了解利用梯度信息高效训练神经网络的重要性 (参见 7.2.1 小节). 评估神经网络误差函数梯度的方法主要有 4 种.

第 1 种方法是手工推导反向传播方程，然后在软件中显式实现. 这种方法多年来一直是神经网络的中流砥柱；如果认真实施，就能产生高效代码并给出达到数值精度要求的结果. 然而推导方程和编码过程都十分耗费时间且容易出错. 前向传播方程与反向传播方程是分开编码的，这会导致代码冗余；又由于其中往往涉及重复计算，如果模型发生变化就需要同步改变前向实现和反向实现，这容易限制对不同架构进行经验探索的速度和效率.

第 2 种方法是利用有限差分法对梯度进行数值计算 (参见 8.1.4 小节). 这种方法只需要用软件实现前向传播方程. 数值微分法的一个问题是计算精度有限，不过这对于网络训练来说可能不是问题，因为我们可能会使用随机梯度下降法；在随机梯度下降法中，每次计算都只是对局部梯度非常粗略的估计. 这种方法的主要缺点是在网络规模缩放上表现很差. 不过这种技术对于调试其他方法也很有用，因为梯度计算只使用了前向传播代码，因此可以用来确认反向传播的正确性，以及确认其他用于计算梯度的代码的正确性.

第 3 种方法是符号微分法，这种方法能利用专业软件将我们在第 1 种方法中手工完成的分析操作自动化. 作为计算机代数或符号计算的一个例子，这一过程涉及在一个完全机械化的过程中自动应用微积分规则 (如链式法则). 符号微分法的一个明显优势是避免了人工推导反向传播方程时引入的人为误差；此外梯度计算再次达到了机器精度，避免了数值微分时缩放性差的问题.

然而符号微分法的主要缺点是导数表达式可能比原始函数表达式长很多，其长度呈指数级增长，相应计算时间也很长. 考虑由 $u(x)$ 和 $v(x)$ 乘积给出的函数 $f(x)$，该函数及其导数计算公式为

$$f(x) = u(x)v(x), \tag{8.37}$$

$$f'(x) = u'(x)v(x) + u(x)v'(x). \tag{8.38}$$

可以看到在计算 $f(x)$ 和 $f'(x)$ 时必须同时求出 $u(x)$ 和 $v(x)$, 因此存在冗余计算. 如果因子 $u(x)$ 和 $v(x)$ 本身涉及因子, 就会出现表达式嵌套重复, 复杂性会迅速增加. 这个问题叫作**表达式膨胀** (expression swell): 除了比原始函数复杂得多之外, 还存在冗余计算, 比如像 $w_1x + b_1$ 这样的表达式就出现在了多个地方.

符号微分法的另一个主要缺点是它需要以封闭形式表示被微分的表达式. 因此这种方法排除了重要控制流操作, 如循环、递归、条件执行和过程调用, 而这些都是我们在定义网络函数时可能希望使用的有价值结构.

第 4 种方法是**自动微分法** (automatic differentiation), 也称" 算法微分法" . 与符号微分法不同, 自动微分法旨在让计算机在仅给定前向传播方程代码的条件下自动生成实现梯度计算的代码, 而非得到导数的数学表达式. 与符号微分法一样, 自动微分法也达到了机器精度, 但效率更高, 因为它能够利用我们在定义前向传播方程时使用的中间变量, 从而避免冗余计算. 值得注意的是, 自动微分法不仅能处理传统封闭形式数学表达式, 还能处理流程控制元素, 如分支、循环、递归和过程调用. 因此自动微分法的功能比符号微分法强大得多; 与此同时自动微分法具有广泛适用性, 是在机器学习领域之外发展起来的. 现代深度学习在很大程度上是一个经验过程, 涉及对不同架构的计算和比较, 因此自动微分法在准确高效地完成实验方面发挥着关键作用.

自动微分法的主要思想是采用计算函数的代码 (例如计算神经网络误差函数的前向传播方程代码), 并用附加变量来扩充代码, 这些附加变量的值会在代码执行过程中累加, 从而获得所需导数. 自动微分法主要有两种形式: **前向模式**和**逆模式**.

为进一步说明, 考虑一个结构类似于两层神经网络的函数 , 该函数有单个输入 x、带激活 z 的一个隐藏单元和输出 y, 其中

$$z = h(w_1x + b_1), \tag{8.39}$$

$$y = h(w_2z + b_2), \tag{8.40}$$

其中 $h(a)$ 是 softplus:

$$h(a) = \ln\{1 + \exp(a)\}. \tag{8.41}$$

于是有

$$y(x) = h(w_2h(w_1x + b_1) + b_2)\,. \tag{8.42}$$

网络输出 y 相对于 w_i 的导数为 (见习题 8.13)

$$\frac{\partial y}{\partial w_i} = \cdots\,. \tag{8.43}$$

8.2.1 前向模式自动微分

在前向模式自动微分中, 我们在函数 (如神经网络误差函数) 计算过程中涉及的每个中间变量 z(称为" 原始" 变量) 上添加一个附加变量, 该附加变量代表中间变量的某个导数值. 我们将其记作 $\dot{z}$, 即" 正切" 变量. 正切变量及其相关代码由软件环境自动生成. 现在代码不再简单地通过前向传播来计算 z, 而是传播元组 $\{z, \dot{z}\}$, 从而同时计算变量和导数. 原函数一般用初等算子 (包括算术运算和否定运算) 以及指数函数、对数函数和三角函数等超越函数来定义, 所有这些函数的导数都有简单计算公式. 将这些导数与微积分中链式法则结合使用, 可以自动生成用于计算梯度的代码.

举例来说, 如下函数有两个输入变量:

$$f(x_1, x_2) = x_1x_2 + \exp(x_1x_2) - \sin(x_1). \tag{8.44}$$

在软件中实现时, 代码由一系列操作组成, 这些操作可以用底层基本操作的**求值轨迹** (evaluation trace) 来表示. 此处对以下原始变量进行了定义:

$$v_1 = x_1, \qquad v_2 = x_2, \tag{8.45}$$

$$v_3 = v_1v_2, \qquad v_4 = \exp(v_3), \qquad v_5 = \sin(v_1), \tag{8.46}$$

$$v_6 = v_4 - v_5, \qquad v_7 = v_3 + v_6. \tag{8.47}$$

假设要求导数 $\partial f/\partial x_i$. 我们用 $\dot{v}_j = \partial v_j/\partial x_i$ 来定义正切变量. 利用微积分中链式法则, 可以自动构造出用于计算这些值的表达式:

$$\dot{v}_j = \sum_{i \in \mathrm{pa}(j)} \frac{\partial v_j}{\partial v_i} \dot{v}_i, \tag{8.48}$$

其中 $\mathrm{pa}(j)$ 表示求值轨迹中节点 j 的父节点集合, 即指向节点 j 的变量的集合.

例如在式 (8.45) 至式 (8.47) 中, 节点 v_3 的父节点是节点 v_1 和 v_2. 将式 (8.48) 应用于求值轨迹方程, 可以得到以下正切变量的求值轨迹方程:

$$\dot{v}_1 = 1, \qquad \dot{v}_2 = 0, \tag{8.49}$$

$$\dot{v}_3 = \dot{v}_1 v_2 + v_1 \dot{v}_2, \qquad \dot{v}_4 = \dot{v}_3 \exp(v_3), \qquad \dot{v}_5 = \dot{v}_1 \cos(v_1), \tag{8.50}$$

$$\dot{v}_6 = \dot{v}_4 - \dot{v}_5, \qquad \dot{v}_7 = \dot{v}_3 + \dot{v}_6. \tag{8.51}$$

然后代码会执行原始方程和正切方程, 依次对元组 $(v, \dot{v})$ 进行数值计算, 直至得到所需导数 $\dot{v}_7$(见习题 8.17).

一般来说, 如果一个函数有 D 个输入和 K 个输出, 那么一次前向模式自动微分就会产生 $K \times D$ 大小雅可比矩阵中的一列. 若要计算雅可比矩阵第 j 列, 则需要对正切变量求值方程的前向传播进行初始化, 也就是在 $i \neq j$ 时设置 $\dot{x}_i = 1$ 和 $\dot{x}_i = 0$. 我们可以将其写成向量形式, 即 $\dot{\mathbf{x}} = \mathbf{e}_j$, 其中 $\mathbf{e}_j$ 是第 j 个单位向量. 若要计算全雅可比矩阵, 则需要进行 D 次前向传播.

然而若要计算雅可比矩阵与向量 $\mathbf{r} = (r_1, \cdots, r_D)^{\mathsf{T}}$ 的乘积:

$$\mathbf{J}^{\mathsf{T}}\mathbf{r} = \sum_i r_i \nabla_{\mathbf{x}} f_i, \tag{8.52}$$

那么只需设置 $\dot{\mathbf{x}} = \mathbf{r}$, 就可以通过一次前向传播来完成 (见习题 8.18).

从图中可以看出, 这只涉及对原始变量和正切变量求值方程的一个小扩展, 因此 $\partial f_1/\partial x_i$ 和 $\partial f_2/\partial x_i$ 都可以在一次前向传播中一并求出. 不过这样做的缺点是: 如果想计算相对于不同输入变量 x_2 的导数, 就必须进行一次单独前向传播.

我们可以看到, 前向模式自动微分可以通过 D 次前向传播求出导数的 $K \times D$ 大小全雅可比矩阵. 对于具有少量输入和较多输出 ($K \gg D$) 的网络, 这种方法非常有效. 然而我们经常在这样的环境中操作: 通常只有一个函数, 即用于训练的误差函数, 以及大量想要对其进行微分的变量 (包括网络中的权重和偏置), 这些变量可能有数百万个或数十亿个. 在这种情况下, 前向模式自动微分具有极低效率. 因此我们转而采用另一种自动微分法——逆模式自动微分.

8.2.2 逆模式自动微分

逆模式自动微分可以看作误差反向传播程序的拓展. 和前向模式自动微分一样, 我们也在每个中间变量 v_j 中添加附加变量; 附加变量在此处称为**伴随** (adjoint) 变量, 用 $\bar{v}_j$ 表示. 考虑具有单一输出函数 f 的情况, 其伴随变量定义如下:

$$\bar{v}_j \equiv \frac{\partial f}{\partial v_j}. \tag{8.53}$$

可以利用微积分中链式法则, 从输出开始依次反向对它们进行求值:

$$\bar{v}_i = \sum_{j \in \mathrm{ch}(i)} \bar{v}_j \frac{\partial v_j}{\partial v_i}, \tag{8.54}$$

其中 $\mathrm{ch}(i)$ 表示求值轨迹中节点 i 的子节点集合, 即从节点 i 指向这些子节点的变量的集合. 如前所述, 对伴随变量的连续求值代表了通过求值轨迹进行的反向信息流动 (参见图 8.1).

再次考虑式 (8.45) 至式 (8.47) 给出的具体函数示例, 可以得出以下用于计算伴随变量的方程 (见习题 8.16):

$$\bar{v}_7 = 1, \tag{8.55}$$

$$\bar{v}_3 = \bar{v}_7, \qquad \bar{v}_4 = \bar{v}_7, \qquad \bar{v}_6 = \bar{v}_7, \tag{8.56}$$

$$\bar{v}_1 = \bar{v}_3 v_2 + \bar{v}_6 \cos(v_1), \qquad \bar{v}_2 = \bar{v}_3 v_1, \tag{8.57}$$

$$\bar{v}_5 = -\bar{v}_6. \tag{8.58}$$

请注意它们从输出端开始, 然后通过求值轨迹反向流至输入端. 即使有多个输入, 也只需要一次反向传播就能求出导数. 对于神经网络误差函数,E 关于权重和偏置的导数可作为相应伴随变量; 但是如果出现一个以上输出变量, 则需要为每个输出变量进行一次单独反向传播.

逆模式通常比前向模式占用更多内存, 因为必须存储所有中间变量, 以便在反向传播过程中根据需要使用中间变量计算伴随变量. 与此相反, 在前向模式下, 原始变量和正切变量在前向传播过程中被一起计算, 所以变量一旦用完就可以丢弃. 与逆模式相比, 前向模式一般更容易实现.

无论是前向模式自动微分还是逆模式自动微分, 通过网络的单次传播成本都不会超过单次函数计算成本的 6 倍. 实际上开销通常接近单次函数计算成本的 2 倍或 3 倍 . 前向模式和逆模式的融合也很有趣: 前向模式和逆模式的融合在计算黑塞矩阵与向量乘积的情况下很有用 . 在此我们从一个向量 $\mathbf{v}$ 和一个点 $\mathbf{x}$ 开始计算黑塞矩阵与向量的乘积. 通过设置 $\dot{\mathbf{x}} = \mathbf{v}$ 并使用前向模式, 可以得出方向导数 $\mathbf{v}^{\mathsf{T}}\nabla f$; 然后使用逆模式进行微分, 得到 $\nabla^2 f\,\mathbf{v} = \mathbf{H}\mathbf{v}$.

第 9 章 正则化

9.1 归纳偏置

在讨论多项式曲线拟合时, 我们引入了正则化的概念, 其通过防止模型参数取值过大来减少过拟合 (参见 1.2 节). 这涉及向误差函数中添加一个简单惩罚项, 从而得到正则化误差函数, 形式如下:

$$\widetilde{E}(\mathbf{w}) = E(\mathbf{w}) + \frac{\lambda}{2}\mathbf{w}^{\mathsf{T}}\mathbf{w}, \tag{9.1}$$

其中 $\mathbf{w}$ 是模型参数向量,$E(\mathbf{w})$ 是未正则化误差函数, 正则化超参数 λ 控制正则化效应强度. 在以增加一些偏差为代价减小解的方差可以改善预测准确性, 这可以从偏差–方差权衡角度来解释 (参见 4.3 节).

在比较正弦合成数据问题中各阶多项式预测误差时 (参见 1.2.4 小节), 我们发现使用中等复杂度多项式能够获得最小泛化误差, 既不过于简单也不过于灵活. 当使用形如 (9.1) 的正则化项控制模型复杂度时, 也发现类似结果: 正则化系数 λ 大小中等时实现了对新输入值的最佳预测 (参见 1.2.5 小节). 对这一结果的洞察来自偏差–方差分解: 具有高偏差的简单模型无法捕捉到潜在数据生成过程中的变化, 而具有低偏差的高度灵活模型则容易过拟合, 导致泛化能力差. 随着数据集大小增加, 可以使用具有更小偏差的更灵活模型, 而不会引起过大方差, 从而改善泛化.

任何给定神经网络都只是对真实数据生成器的一种近似. 更大网络可以提供更接近的近似, 但会带来过拟合风险. 实践中最佳泛化结果几乎总是通过使用较大网络结合某种形式正则化来获得. 本章探索几种可替代正则化技术, 包括早停法、模型平均、dropout、数据增强和参数共享. 如果需要也可以结合使用多种正则化技术, 例如正则化误差函数 (9.1) 通常可以与 dropout 一起使用.

9.1.1 逆问题

模型选择问题是机器学习的核心, 可以追溯到大多数机器学习任务都是**逆问题** (inverse problem) 这一事实. 给定条件分布 $p(t \mid \mathbf{x})$ 以及一组有限输入点 $\{\mathbf{x}_1, \cdots, \mathbf{x}_N\}$, 原则上至少可以从中抽样相应目标值 $\{t_1, \cdots, t_N\}$. 然而在机器学习中我们必须解决这个问题的逆问题: 只给定有限数量样本就推断整个分布. 这本质上是不适定 (ill-posed) 的, 因为有无限多分布可能生成所观测到的训练数据. 实际上任何在观测目标值上具有非零概率密度的分布都是候选分布.

然而为使机器学习有实用价值, 需要对新的 $\mathbf{x}$ 值进行预测, 因此需要一种方法从无限多可能分布中选择一个特定分布. 选择这个特定分布而非其他分布的偏好称为**归纳偏置** (inductive bias) 或**先验知识** (prior knowledge), 其在机器学习中起着核心作用. 先验知识可能来自有助于约束解空间的背景信息. 对于许多应用, 输入值的微小变化应该导致输出值微小变化, 因此应该使解偏向于那些具有平滑变化函数的解决方案. 形如 (9.1) 的正则化项鼓励模型权重取值较小, 因此引入了对输入变化较慢函数的偏好. 同样当在图像中检测对象时, 可以利用对象本体通常与对象在图像中位置无关的先验知识 (参见第 10 章). 这称为**平移不变性**, 将其纳入解可以显著简化构建具有良好泛化能力系统的任务.

9.1.2 无免费午餐定理

本书核心关注点是一类重要机器学习模型——深度神经网络. 这些模型高度灵活, 已在计算机视觉、语音识别和自然语言处理等多个领域引发革命; 事实上它们已成为大多数机器学习应用的首选框架, 因此看起来像是能完成所有任务的" 通用" 学习算法. 然而即便是非常灵活的神经网络也包含重要归纳偏置.

无免费午餐定理 (no free lunch theorem) 源自" 天下没有免费午餐" 这一说法, 其表明在所有可能问题上, 平均来看每个学习算法与其他学习算法一样好. 如果某个特定模型或算法在某些问题上优于平均水平, 那么它在其他问题上必定低于平均水平. 然而这更多是一个理论概念, 因为这里所说的可能问题空间包括了任何合理实际应用中都非常不典型的输入与输出之间关系.

尽管无免费午餐定理在某种程度上是理论性的, 但它确实突出了偏差在确定机器学习算法性能方面的重要性. 在没有任何偏差的情况下, 不可能" 纯粹从数据中" 学习. 在实践中偏差可能是隐式的: 例如每个神经网络都有有限数量参数, 这限制了它所能表示的函数. 然而偏差也可以作为解决特定问题的先验知识被显式编码.

在寻找通用学习算法过程中, 实际上是在寻找适合的归纳偏置, 它广泛适用于实践中遇到的不同应用. 然而对于任何给定应用, 如果能够融入特定于该应用的更强归纳偏置, 则可以获得更好结果. 基于模型的机器学习观点 主张使机器学习模型中所有假设显式化, 以便为归纳偏置做出适当选择.

我们已经注意到大多数具有实践背景的例子显示出某种程度平滑性, 其中输入值的微小变化大多与目标值微小变化相关联. 像神经网络这样的模型, 以及大多数广泛使用的机器学习技术, 由于表现出这种形式归纳偏置, 因此在某种程度上具有广泛适用性.

像迁移学习和多任务学习这样的技术也可以从正则化角度来看 (参见 6.3.4 小节). 当某个特定任务训练数据有限时, 可以使用来自不同但相关任务的额外数据来帮助确定神经网络中的可学习参数. 任务之间的相似性假设代表了一种比简单正则化更复杂的归纳偏置形式, 这解释了使用额外数据所带来的性能提升.

重要的是不要引入与数据生成过程不一致的偏差或约束. 例如当输出与输入之间实际存在显著非线性关系时, 如果假设它们关系是线性的, 则可能导致得到一个产生不准确答案的系统.

9.1.3 对称性和不变性

在机器学习许多应用中, 预测应该在输入变量经过一种或多种变换时保持不变, 称为具有**不变性** (invariance). 例如在对二维图像中对象 (如猫或狗) 进行分类时, 无论对象在图像中位置如何, 都应赋予对象相同分类, 称为**平移不变性** (translation invariance). 同样图像中对象大小变化也不应改变对象分类, 称为**尺度不变性** (scale invariance). 利用这种对称性来创建归纳偏置可以大大提高机器学习模型性能, 这是**几何深度学习** (geometric deep learning) 所要研究的主题.

所有对应于特定对称性 (如平移或缩放) 的变换集合构成一个称为**群** (group) 的数学结构. 群由一组元素 $A, B, C, \cdots$ 以及一个用于组合元素对的二元运算 (用 $A \circ B$ 表示) 组成. 以下 4 个公理对于群成立:

- **封闭性**: 对于集合中任何两个元素 A 和 B,$A \circ B$ 也必须在集合中.
- **结合性**: 对于集合中任何三个元素 A、B 和 C, 有 $(A \circ B) \circ C = A \circ (B \circ C)$.
- **单位元**: 集合中存在一个称为单位元的元素 I, 满足对于集合中每个元素 A, 有 $A \circ I = I \circ A = A$.
- **逆元**: 对于集合中每个元素 A, 存在集合中另一个元素 A^{-1}, 满足 $A \circ A^{-1} = A^{-1} \circ A = I$.

简单群如以 90° 倍数旋转正方形, 或在二维平面上连续平移对象的变换集合, 都满足群公理 (见习题 9.1).

原则上神经网络对输入空间变换做出预测的不变性可以从数据中学习, 而无须对网络或训练程序进行特殊修改. 然而实际上这可能极具挑战性, 因为这样的变换组合数量是指数级的. 例如同时要求对二维空间中平移、缩放、旋转、强度变化、色彩平衡变化等保持不变; 对象相对较小的平移即使只有几个像素也可以导致像素值显著变化.

因此需要寻求更有效方法来促使自适应模型展现出所需不变性. 这些方法大致可分为 4 类:

1. **预处理**: 通过在预处理阶段计算在所需变换下保持不变的数据特征来构建不变性. 任何后续使用这些特征作为输入的回归或分类系统必然也保持了这些不变性.
2. **正则化误差函数**: 向误差函数中添加正则化项, 以惩罚输入仅受不变性变换改变时模型输出的变化.
3. **数据增强**: 通过使用所需不变性对训练数据点进行变换, 从而扩展训练集, 同时保持它们目标值与变换前相同.
4. **网络架构**: 通过适当选择网络架构, 将不变性融入神经网络结构中.

上述第 1 类方法面临的一个挑战是设计呈现所需不变性的特征, 同时不丢弃可能有助于确定网络输出的信息. 我们已经看到固定的、手工提取的特征表示能力有限, 并且已被使用深度神经网络习得的表示所取代 (参见第 6 章).

第 2 类方法的一个例子是在训练过程中添加正则化项到误差函数中的**切线传播** (tangent propagation) 技术 , 旨在直接惩罚由某种不变性变换下输入变量的变化引起的输出变化. 除训练额外复杂性外, 这种技术一个局限性

是只能处理小变换 (例如少于一个像素的平移).

第 3 类方法又称**数据集增强** (data set augmentation). 它通常相对容易实现, 并在实践中被证明非常有效. 它通常应用在图像分析场景中, 因为创建变换训练数据非常直接. 通过对原始示例做小改动的变换, 然后按变换幅度幂次对误差函数做泰勒展开, 可以分析使用增强数据的效果 , 并得到一个正则化误差函数, 其中正则化项惩罚网络输出相对于输入变量在变换方向上投影的梯度. 这与前面讨论的切线传播技术相关. 添加随机噪声的变换是一种特殊情况, 在这种情况下正则化项惩罚网络输出相对于输入的导数 (见习题 9.2). 因为鼓励即便在输入变量添加噪声情况下网络输出也保持不变, 所以该惩罚非常合理.

对于序贯训练算法 (如随机梯度下降), 可以在输入模型之前变换每个数据点来增强数据集, 以便如果数据点被循环使用则每次都会应用不同变换 (从适当分布中抽取变换方式). 对于批训练方法, 通过多次复制每个数据点并独立变换每个副本, 可以达到类似效果.

9.1.4 等变性

不变性的一个重要推广称为**等变性** (equivariance). 在这种情况下, 当输入变换时, 网络输出以特定方式变换, 而不是保持不变. 例如考虑一个接收图像作为输入并返回图像分割结果的网络, 其中每个像素分类为属于前景对象或背景. 此时如果图像中对象位置发生平移, 那么希望对象相应分割结果也类似地发生平移.

用 I 表示图像, 用 S 表示分割网络操作, 那么对于一个平移操作 T, 有

$$S(T(I)) = T(S(I)). \tag{9.2}$$

这表明被平移图像的分割结果是通过对原始图像分割结果进行平移得到的.

更一般地, 当应用于输出的变换与应用于输入的变换不同时, 等变性也成立:

$$S(T_1(I)) = T_2(S(I)). \tag{9.3}$$

例如假设分割后图像分辨率低于原始图像,T_1 代表原始图像空间中平移,T_2 代表较低维分割空间中相应平移. 同样如果 S 代表计算图像中对象方向的操作, 而 T_1 代表旋转 (这是图像中所有像素值的复杂非线性变换), 那么 T_2 将依据 S 生成一个标量方向值增大或减小.

还可以看到, 不变性是等变性的一种特殊情况, 其中输出变换是简单恒等变换. 例如如果 C 是一个神经网络用于分类图像中一个对象, 而 T 是一个平移操作, 则有

$$C(T(I)) = C(I), \tag{9.4}$$

这表明对象类别不依赖于该对象在图像中位置.

最后, 第 4 类方法, 也就是在网络结构中融入不变性, 已被证明非常强大且有效, 并且还带来了其他关键好处. 我们将在计算机视觉中卷积神经网络背景下详细讨论这类方法 (参见第 10 章).

9.2 权重衰减

我们在线性回归背景下引入正则化 (参见 1.2.5 小节) 来控制模型复杂性. 这是一种限制模型中参数数量的替代方法. 最简单正则化项由模型参数平方和构成, 从而形成式 (9.1) 的正则化误差函数. 该误差函数会对较大参数值施加惩罚. 有效的模型复杂性由正则化系数 λ 的选择决定.

式 (9.1) 中的平方和正则化项在机器学习文献中称为**权重衰减** (weight decay), 因为在序贯学习算法中它鼓励权重值向零衰减 (除非数据支持其不衰减, 参见习题 9.3). 这种正则化项一个优点是其导数很容易计算, 可用于梯度下降训练. 具体来说对于式 (9.1), 梯度由下式给出:

$$\nabla\widetilde{E}(\mathbf{w}) = \nabla E(\mathbf{w}) + \lambda\mathbf{w}. \tag{9.5}$$

可以看到式 (9.1) 中的因子 1/2 在取导数时会消失.

我们还看到这种加性正则化项可以解释为权重向量 $\mathbf{w}$ 上零均值高斯先验分布的负对数 (参见 2.6.2 小节). 这为将先验知识纳入模型训练过程提供了概率性视角. 遗憾的是这种先验是通过模型参数表示的, 而我们具有的关

于待解决问题领域知识更可能是通过从输入到输出的网络函数形式表示的. 然而参数与网络函数之间关系极其复杂, 因此只有非常有限类型先验知识可以直接轻松地表示为模型参数先验.

通过考虑二维参数空间以及未正则化误差函数 $E(\mathbf{w})$, 可以看到二次正则化项的一般效果. 该误差函数是 $\mathbf{w}$ 的二次函数 (对应于具有平方和误差函数的简单线性回归模型, 参见 4.1.2 小节). 参数空间中轴已经旋转以与黑塞矩阵特征向量对齐, 对应于椭圆误差函数等值线的轴 (参见 8.1.6 小节). 可以看到正则化项效果是缩小权重参数大小; 然而正则化项对于参数 $\mathbf{w}_1$ 的效果更大, 因为未正则化误差对 $\mathbf{w}_1$ 值敏感性远低于对 $\mathbf{w}_2$ 值的敏感性. 直观上只有参数 $\mathbf{w}_1$ 是" 活跃的", 因为输出对 $\mathbf{w}_1$ 相对不敏感, 正则化项会将 $\mathbf{w}_1$ 推向 0.

正则化后仍然活跃的参数数量, 这个概念可以从贝叶斯学派或频率学派角度形式化 . 对于 $\lambda \to \infty$, 所有参数都被推向 0, 有效参数数量因此为 0. 随着 λ 减小, 有效参数数量开始增加, 直至 $\lambda = 0$ 时有效参数数量等于模型中实际参数总数. 通过正则化控制模型复杂性与通过限制参数数量控制模型复杂性有相似之处.

9.2.1 一致性正则化项

简单权重衰减形式 (9.1) 的一个缺陷是它破坏了网络映射的某些我们希望保持的变换特性. 为说明这一点, 考虑一个有两层权重和线性输出单元的多层感知机网络, 该网络从一组输入变量 $\{x_i\}$ 映射到一组输出变量 $\{y_k\}$. 第一隐藏层激活由下式给出:

$$z_j = h\left(\sum_i w_{ji}x_i + b_j\right), \tag{9.6}$$

而输出单元激活由下式给出:

$$y_k = \sum_j w_{kj}z_j + b'_k. \tag{9.7}$$

假设我们对输入数据进行如下线性变换:

$$x_i \longrightarrow \tilde{x}_i = ax_i + b. \tag{9.8}$$

然后可以通过对从输入到隐藏层单元的权重和偏置进行相应线性变换来保证网络执行的映射不变 (见习题 9.4). 如果使用原始数据训练一个网络, 并使用输入和/或目标变量经过上述某种线性变换的数据训练另一个网络, 那么一致性要求我们应该得到等价网络, 它们区别仅仅在于是否对权重做了线性变换. 任何正则化项都应该与这种性质一致, 否则就会任意地偏好一种解而不是另一种等价解. 显然简单权重衰减形式 (9.1) 因为平等地惩罚所有权重和偏置, 所以不满足这一性质.

为此需要寻找一个在线性变换下不变的正则化项. 这要求正则化项对权重重新缩放和偏置移动保持不变. 可以通过以下形式变换第一层权重和偏置:

$$w_{ji} \longrightarrow \tilde{w}_{ji} = aw_{ji}, \tag{9.9}$$

$$b_j \longrightarrow \tilde{b}_j = b_j - \frac{b}{a}w_{ji}. \tag{9.10}$$

类似地网络输出变量的如下线性变换:

$$y_k \longrightarrow \tilde{y}_k = cy_k + d, \tag{9.11}$$

可以通过用以下形式变换第二隐藏层权重和偏置来实现:

$$w'_{kj} \longrightarrow \tilde{w}'_{kj} = cw'_{kj}, \tag{9.12}$$

$$b'_k \longrightarrow \tilde{b}'_k = cb'_k + d. \tag{9.13}$$

这样的正则化项由下式给出:

$$\Omega(\mathbf{w}) = \frac{1}{\alpha_1}\|\mathbf{W}^{(1)}\|^2 + \frac{1}{\alpha_2}\|\mathbf{W}^{(2)}\|^2, \tag{9.14}$$

其中 $\mathbf{W}^{(1)}$ 表示第一层权重集合,$\mathbf{W}^{(2)}$ 表示第二层权重集合, 且偏置被排除在求和之外. 只要使用 $\alpha_1 \to a^2\alpha_1$ 和 $\alpha_2 \to c^2\alpha_2$ 来重新缩放正则化参数, 这个正则化项在权重变换下就会保持不变.

正则化项 (9.14) 对应于参数的先验分布形式:

$$p(\mathbf{w} \mid \alpha_1, \alpha_2) \propto \exp\left(-\frac{1}{\alpha_1}\|\mathbf{W}^{(1)}\|^2 - \frac{1}{\alpha_2}\|\mathbf{W}^{(2)}\|^2\right). \tag{9.15}$$

注意这种形式先验是反常的 (它们不能被规范化), 因为偏置参数是不受约束的. 使用反常先验可能会对选择正则化系数以及在贝叶斯框架内进行模型比较造成困难. 因此通常会为偏置设置单独先验 (从而打破平移不变性), 这些先验都有自己的超参数.

可以通过从先验中抽取样本并绘制相应网络函数来展示这 4 个超参数的效果. 先验由 4 个超参数控制:$\alpha_1, \alpha_2, \alpha_1', \alpha_2'$, 它们分别代表第一层偏置、第一层权重、第二层偏置和第二层权重的高斯分布精度.

9.2.2 广义权重衰减

更一般地, 考虑将权重分成任意多组 $\mathcal{W}_k$ 的正则化项, 使得

$$\Omega(\mathbf{w}) = \sum_k \frac{1}{\alpha_k}\|\mathbf{w}_k\|^2, \tag{9.16}$$

其中:

$$\|\mathbf{w}_k\|^2 = \sum_{w_i \in \mathcal{W}_k} w_i^2. \tag{9.17}$$

例如可以为多层网络中每一层使用不同正则化项. 我们有时使用简单二次正则化项的一般形式:

$$\Omega(\mathbf{w}) = \frac{1}{2}\sum_i |w_i|^q. \tag{9.18}$$

当 $q = 1$ 时, 形如式 (9.18) 的正则化项在统计学领域称为 **lasso**(least absolute shrinkage and selection operator) . 对于二次误差函数, 它的性质是如果 λ 足够大, 系数 w_i 将趋于零, 从而得到一个稀疏模型, 对应基函数不起作用.

为说明这一点, 考虑最小化下式给出的正则化误差函数:

$$\widetilde{E}(\mathbf{w}) = E(\mathbf{w}) + \lambda\|\mathbf{w}\|_q^q. \tag{9.19}$$

这等价于在约束条件 (见习题 9.5)

$$\|\mathbf{w}\|_q^q \leq \eta \tag{9.20}$$

下最小化未正则化误差函数 $E(\mathbf{w})$, 其中 η 有合适取值. 这两种方法可以通过拉格朗日乘子法关联起来 (参见附录 C).

模型稀疏的原因可以通过图 9.6 说明: 该图显示了误差函数在约束条件 (9.20) 下的最小值. 随着 λ 增大, 更多参数将趋于零. 相比之下二次正则化项不会要求权重参数为零, 只会将 w_i 减小到一个较小值.lasso 正则化项提供了一个稀疏解, 其中 $w_i = 0$.

9.3 学习曲线

我们已经探讨了模型泛化能力如何随模型中参数数量、数据集大小以及权重衰减正则化项系数而变化. 这些都允许在偏差和方差之间进行权衡, 以最小化泛化误差. 另一个影响这种权衡的因素是学习过程本身. 在通过梯度下降优化误差函数期间, 训练误差通常随模型参数更新而降低, 而留出数据 (hold-out data) 的误差可能是非单调的. 这种现象可以通过使用**学习曲线**来可视化. 学习曲线显示了在迭代式学习过程 (如随机梯度下降) 中, 性能指标 (如训练集和验证集误差) 关于迭代次数的变化. 学习曲线不仅提供对训练进程的洞察, 还提供一种控制模型有效复杂度的实用方法.

9.3.1 早停法

除正则化外, 控制网络有效复杂度的一种替代方法是**早停法** (early stopping). 深度学习模型训练涉及迭代下降定义在一组训练数据上的误差函数. 虽然在训练集上计算的误差大致上关于迭代次数单调递减, 但在留出数据

(通常称为验证集) 上计算的误差通常首先下降, 然后由于网络开始过拟合而增加. 因此为获得具有良好泛化能力的网络, 应在验证集上的最小误差点处停止训练.

我们可以在二次误差函数上验证这一洞察, 结果显示早停法与使用简单权重衰减项的正则化 作用相似. 权重空间中的轴已经旋转以与黑塞矩阵特征向量平行. 如果在没有权重衰减的情况下 (参见 7.1.1 小节), 权重向量从原点开始, 并在训练期间沿着局部负梯度向量路径前进, 那么权重向量将从最初与 $\mathbf{w}_1^*$ 轴平行, 大约朝着 $\mathbf{w}^*$ 移动, 最后向误差函数最小值移动. 这种移动方法遵循误差表面形状, 并且与黑塞矩阵特征值有很大不同. 因此在 $\mathbf{w}^*$ 附近的某个点停止训练类似于权重衰减.

早停法和权重衰减之间的关系可以被量化 (参见习题 9.6), 从而显示出迭代次数 τ(其中 η 是学习率) 可以像正则化参数 λ 的倒数一样起作用. 因此网络中有效参数数量在训练期间会增长.

学习曲线的这种现象有时可以从网络中有效参数数量角度进行定性解释. 这个数量开始时很少, 然后在训练过程中增长, 对应模型有效复杂度的稳定增加. 在训练误差达到最小值之前停止训练, 是限制网络有效复杂度的一种方法.

9.3.2 双重下降

偏差–方差权衡为可学习模型的泛化能力会随模型参数数量变化而变化提供了解释 (参见 4.3 节). 参数太少的模型会因为有限表现能力 (高偏差) 而有较大测试误差; 随着参数数量增加, 可以预期测试误差会降低. 然而随着参数数量进一步增加, 由于过拟合 (高方差), 测试误差会再次上升. 这引出了经典统计学中一个普遍观念, 即模型中参数数量需要根据数据集大小进行限制: 对于给定训练集, 非常大的模型预期表现不佳.

然而与这种预期相反, 对于现代深度神经网络, 即使参数数量远远超过对训练数据完美拟合所需, 也可以表现出色 . 深度学习社区的普遍看法是, 更大的模型表现会更好. 虽然有时提前停止训练, 但模型也可能被训练到零误差, 并且在测试数据上表现良好.

这些看似矛盾的观点可以通过检查学习曲线和其他模型复杂度对泛化能力的图表来解释, 这些图表揭示了一种称为**双重下降** (double descent) 的更微妙现象 . 图 9.9 显示了一个名为 ResNet18 的大型神经网络模型 在图像分类任务上训练误差和测试误差与模型复杂度的关系. 该网络有 18 层参数. 通过改变控制每层中隐藏单元数量的" 宽度参数", 网络中权重和偏置数量可以变化. 可以看到与预期一致, 训练误差随模型复杂度增加而单调减小; 然而测试误差最初下降, 然后再次上升, 最后再次下降. 虽然训练误差已经达到零, 但测试误差仍旧持续减小.

这种令人惊讶的现象比通常从经典统计学偏差–方差讨论中期望的现象更复杂, 并呈现出两种不同模型拟合模式: 对应于小到中等复杂度的经典偏差–方差权衡, 以及当进入大型模型阶段时测试误差进一步减小. 两种模型拟合模式之间的过渡大约发生在模型中参数数量足够多以至于能够精确拟合训练数据时 .Nakkiran 等人 (2019) 定义了模型有效复杂度为模型可以在上面达到零训练误差的最大训练集大小, 因此当模型有效复杂度超过训练集中数据点数量时, 就会出现双重下降.

如果使用早停法来控制模型复杂度, 则可以看到类似现象: 增加训练轮次会增加模型有效复杂度, 对于足够大的模型将再一次观察到双重下降.

双重下降一个讽刺结果是: 在某种情况下, 增加训练集大小实际上可能会降低性能, 这与常识相悖——因为一般认为更多数据总是好事. 对于处于临界复杂度阶段的模型, 增加训练集大小可以将插值阈值向右推, 导致更高测试误差. 然而当将训练集中数据点数量从 4000 增加到 18000 时, 可以得到整体上更低曲线; 但对于一定范围内嵌入维度 (对应处于临界复杂度阶段的模型), 增加数据集大小实际上可能会降低模型泛化能力.

当在误差函数中使用正则化项来控制模型复杂度时, 可以得到类似结果. 在这里, 一个训练到收敛的大型模型的测试误差显示出关于 $1/\lambda$(即正则化参数倒数) 的双重下降, 因为较高 λ 对应于较低模型复杂度 .

对于这样的模型, 存在许多可能解, 包括那些过度拟合数据的解. 似乎是随机梯度下降引入的隐性偏见产生了良好泛化能力.

9.4 参数共享

正则化项, 如 L_2 正则化项 $\|\mathbf{w}\|^2$, 通过促使权重接近于零从而减少过拟合. 另一种降低网络复杂度的方法是对权值施加硬约束: 将权重分组, 要求每组内所有权重共享相同值, 其中共享值是从数据中学习得到的. 这称为**权重共享** (weight sharing), 又称**参数共享** (parameter sharing) 或**参数捆绑** (parameter tying). 这意味着自由度数量少于网络中连接数量, 通常是一种将归纳偏置编码到网络中的方法, 从而表达一些已知不变性. 评估此类网络误差函数的梯度可以通过稍微修改反向传播来完成 (见习题 9.7), 虽然在实践中这是通过自动微分隐式处理的.

我们讨论卷积神经网络 (参见第 10 章) 时将广泛使用参数共享. 然而参数共享只适用于可以预先指定约束形式的特定问题.

9.4.1 软权重共享

Nowlan 和 Hinton(1992) 并未使用强制模型参数集相等的硬约束, 而是引入了一种称为**软权重共享** (soft weight sharing) 的形式, 其中正则化项促使权重组具有相似值. 权重的分组、每组平均权值以及组内权值的传播都是作为学习过程的一部分来确定的.

前面曾提到, 式 (9.1) 中简单权重衰减正则化项可以看作权重高斯先验分布的负对数. 这促使所有权重都收敛到 0. 但可以通过考虑高斯混合概率分布来促使权重形成几个组, 而非仅仅一个组. 高斯分量均值 μ_j 和方差 σ_j^2, 以及混合系数 π_j, 则视为学习过程中需要确定的可调参数 (参见 3.2.9 小节). 这样就有了如下概率密度形式:

$$p(\mathbf{w}) = \sum_{j=1}^{K} \pi_j \mathcal{N}\left(\mathbf{w} \,\middle|\, \mu_j, \sigma_j^2\right). \tag{9.21}$$

对式 (9.21) 右侧取负对数就会得到如下形式的正则化函数:

$$\Omega(\mathbf{w}) = -\ln\left\{\sum_{j=1}^{K} \pi_j \mathcal{N}\left(\mathbf{w} \,\middle|\, \mu_j, \sigma_j^2\right)\right\}, \tag{9.22}$$

其中 K 是高斯分量数量.

总误差函数由下式给出:

$$\widetilde{E}(\mathbf{w}) = E(\mathbf{w}) + \lambda\Omega(\mathbf{w}), \tag{9.23}$$

其中 λ 为正则化系数.

该误差关于混合模型的权重 $\mathbf{w}$ 和参数 $\{\pi_j, \mu_j, \sigma_j\}$ 被联合最小化. 这可以使用梯度下降来完成, 但需要我们计算 $\Omega(\mathbf{w})$ 对所有可学习参数的导数. 为此我们将 $\{\pi_j\}$ 视为已经生成权值的每个分量的先验概率, 并引入相应后验概率. 后验概率由贝叶斯定理给出 (见习题 9.8).

总误差函数关于权重的导数由下式给出 (见习题 9.9):

$$\frac{\partial \widetilde{E}}{\partial w_i} = \frac{\partial E}{\partial w_i} + \lambda \sum_j r_{ij} \frac{w_i - \mu_j}{\sigma_j^2}, \tag{9.24}$$

其中权重系数由每个权重的分量 j 生成的后验概率给出.

关于高斯分布中心的误差导数也很容易计算得出 (见习题 9.10):

$$\frac{\partial \widetilde{E}}{\partial \mu_j} = \lambda \sum_i r_{ij} \frac{\mu_j - w_i}{\sigma_j^2}. \tag{9.25}$$

这个过程会驱使 μ_j 趋向于相应中心 μ_j 周围权重偏差平方的加权平均值, 其中权重系数由每个权重的分量 j 生成的后验概率给出.

为确保方差 σ_j^2 为正, 引入下式定义的新变量 ξ_j:

$$\sigma_j^2 = \exp(\xi_j), \tag{9.26}$$

并且对 ξ_j 进行无约束最小化. 相关导数为 (见习题 9.11):

$$\frac{\partial \widetilde{E}}{\partial \xi_j} = \lambda \sum_i r_{ij} \left(\frac{(w_i - \mu_j)^2}{\sigma_j^2} - 1 \right). \tag{9.27}$$

关于混合系数 π_j 的导数, 需要考虑如下约束条件:

$$0 \leq \pi_j \leq 1, \qquad \sum_j \pi_j = 1, \tag{9.28}$$

这些约束源于对 π_j 作为先验概率的解释. 可以通过使用 softmax 函数, 将混合系数表示为一组辅助变量 η_j 来实现对它的约束. 然后采用以下形式正则化误差函数关于 η_j 的导数 (见习题 9.12):

$$\frac{\partial \widetilde{E}}{\partial \eta_j} = \lambda \pi_j (1 - r_j(\mathbf{w})). \tag{9.29}$$

可以看到 π_j 将趋向于混合分量 j 的平均后验概率.

因此正则化项的作用是将每个权值拉向第 j 个高斯分布中心, 其力度与给定权重的高斯分布后验概率成正比. 这正是我们想要的结果.

软权重共享的另一种应用 介绍了一种原则性方法: 将生成式模型无监督训练与相应判别模型有监督训练相结合. 这在有大量未标记数据而标记数据短缺的情况下是有用的. 生成式模型优势在于所有数据都可以用来确定模型参数, 而判别模型只有标记数据直接为其提供参数. 当存在模型设定误差时, 也就是当模型没能准确描述所生成数据真实分布时 (通常情况下是这样), 判别模型具备更好泛化能力. 通过引入两个模型参数的软捆绑, 得到一个良定的生成式和判别式混合方法, 从而可以鲁棒地对模型设定误差进行建模. 此外还能够从对模型在未标记数据上进行训练中受益.

9.5 残差连接

深度神经网络 (简称深度网络) 的表征能力 (也称表示能力) 在很大程度上源于模型的多层处理. 据观察, 增加网络层数可以显著提高泛化能力. 我们也看到了批量归一化以及对权重和偏置进行初始化, 可以帮助解决深度网络中梯度消失或梯度爆炸的问题 (参见 7.2.5 小节和 7.4.2 小节). 然而即使使用批量归一化, 训练难度也仍然会随着网络层数增多而增加.

这种现象的一种解释称为**破碎梯度** (shattered gradient) . 我们已经看到神经网络表征能力随网络深度呈指数级增长. 使用 ReLU 激活函数, 网络可以表示的线性区域数量呈指数级增长. 然而这样做的一个后果是误差函数梯度中的不连续现象激增. 在这里输出变量对输入变量的导数 (网络雅可比矩阵) 是输入变量的函数. 从微积分链式法则来看, 这些导数决定了误差表面梯度. 我们看到对于深度神经网络, 网络浅层权重参数即便发生极其微小变化, 也会产生显著梯度变化. 基于梯度的迭代式优化算法假设梯度在参数空间中平滑变化, 因此这种" 破碎梯度" 效应会使训练对非常深的网络无效 (参见 6.3 节).

残差连接 (residual connection) 是对神经网络架构的一个重要修改, 它对训练极深网络有很大帮助. 残差连接是一种特殊形式的**跳层连接** (skip-layer connection). 考虑一个包含如下三层的神经网络:

$$\mathbf{z}^{(1)} = F(\mathbf{x}), \tag{9.30}$$

$$\mathbf{z}^{(2)} = F^{(2)}(\mathbf{z}^{(1)}), \tag{9.31}$$

$$\mathbf{y} = F^{(3)}(\mathbf{z}^{(2)}). \tag{9.32}$$

这里函数 $F(\cdot)$ 可能仅由一个线性变换和一个 ReLU 激活函数组成, 也可能很复杂, 比如包含多个线性层、激活函数和归一化层. 残差连接会将每个函数输入添加到函数输出, 从而有

$$\mathbf{z}^{(1)} = F(\mathbf{x}) + \mathbf{x}, \tag{9.33}$$

$$\mathbf{z}^{(2)} = F^{(2)}(\mathbf{z}^{(1)}) + \mathbf{z}^{(1)}, \tag{9.34}$$

$$\mathbf{y} = F^{(3)}(\mathbf{z}^{(2)}) + \mathbf{z}^{(2)}. \tag{9.35}$$

每个函数和残差连接的组合, 如 $F(\mathbf{x})+\mathbf{x}$, 称为**残差块** (residual block). 残差网络 (residual network), 也称 ResNet, 由多层这样的块按顺序组成. 通过残差连接修改后的网络如图 9.13 所示. 如果非线性函数中参数足够小, 使函数输出接近于零, 则残差块可以很容易地生成恒等变换.

" 残差" 一词指的是, 在每个块中函数学习恒等映射和期望输出之间的残差, 我们可以通过整理残差变换等式得到:

$$F^{(2)}(\mathbf{z}^{(1)}) = \mathbf{z}^{(2)} - \mathbf{z}^{(1)}. \tag{9.36}$$

到目前为止还没有指定可学习非线性函数 $F(\cdot)$ 具体形式. 最简单选择是使用一个标准前馈网络, 它由可学习线性变换层和固定非线性激活函数 (如 ReLU 函数) 层交替构成. 这为残差连接的放置提供了两种可能: 在版本 (a) 中, 被加的量总是非负的, 因为它们由 ReLU 层输出给出, 因此为了允许被加量可正可负, 版本 (b) 更常用.

现在可以代入中间变量 $\mathbf{z}^{(1)}$ 和 $\mathbf{z}^{(2)}$, 从而给出将网络输出作为输入 $\mathbf{x}$ 的函数表达式 (见习题 9.13):

$$\mathbf{y} = F^{(3)}\left(F^{(2)}(F(\mathbf{x}) + \mathbf{x}) + \mathbf{z}^{(1)}\right) + \mathbf{z}^{(2)}. \tag{9.37}$$

如果结合式 (9.33) 至式 (9.35) 给出整个网络统一总体方程, 就可以进一步深入了解残差连接促使误差表面变得平滑的方式:

$$\mathbf{y} = F^{(3)}\left(F^{(2)}(F(\mathbf{x}) + \mathbf{x}) + F^{(1)}(\mathbf{x}) + \mathbf{x}\right) + F^{(2)}(F(\mathbf{x}) + \mathbf{x}) + F(\mathbf{x}) + \mathbf{x}. \tag{9.38}$$

图 9.15 描述了残差网络的这种扩展形式. 可以看到函数整体由多个并行网络组成, 每个网络层数较少. 该网络具有深度网络表征能力, 因为深度网络是它的一个特例. 误差表面被浅层和深层子网络的组合所平滑.

式 (9.38) 定义的跳层连接要求输入变量和所有中间变量具有相同维度, 以使它们能够相加. 可以通过添加以下形式可学习参数的非方阵 $\mathbf{W}_s$ 来改变网络中某处维度:

$$\mathbf{y} = F^{(3)}(\mathbf{z}^{(2)}) + \mathbf{W}_s\mathbf{z}^{(1)}. \tag{9.39}$$

Li 等人 (2017) 开发了一种直接可视化误差表面的方法, 结果表明残差连接的作用是创建更平滑误差表面. 残差网络中通常包含批量归一化层, 它们一同显著减少了梯度消失和梯度爆炸问题.He 等人 (2015a) 证明残差连接可以使非常深的网络 (可能有数百层) 得到有效训练. 与标准深度网络相比, 具有残差连接的网络中梯度对输入值敏感度要低得多.

9.6 模型平均

如果我们训练了几个不同模型来解决相同问题, 而不是尝试选择单一最佳模型, 那么通常可以通过平均单个模型所做预测来提高泛化能力. 这种模型整合方式有时称为**委员会**或**集成** (ensemble). 对于产生概率输出的模型, 预测分布是每个模型预测的平均值:

$$p(t \mid \mathbf{x}) = \frac{1}{L}\sum_{l=1}^{L} p_l(t \mid \mathbf{x}), \tag{9.40}$$

其中 $p_l(t \mid \mathbf{x})$ 是模型 l 的输出,L 是模型总数.

这种平均过程的动机源于偏差和方差之间权衡. 回想图 4.7: 当使用正弦数据训练多个多项式, 然后对得到的函数进行平均时, 方差项产生的贡献几乎会被抵消, 从而提升预测性能 (参见 4.3 节). 当然在实践中只有一个数据集, 因此必须找到一种方法, 以便在集成模型 (一些旨在解决相同问题的模型集合) 内部不同模型之间引入可变性.

可以使用**自助** (bootstrap) 数据集, 其中多个数据集创建过程如下. 假设原始数据集由 N 个数据点 $\mathbf{X} = \{\mathbf{x}_1, \cdots, \mathbf{x}_N\}$ 组成. 可以通过从 $\mathbf{X}$ 中有放回地抽样 N 个数据点来创建新数据集 $\mathbf{X}_B$, 这样 $\mathbf{X}$ 中一些数据点可能在 $\mathbf{X}_B$ 中出现多次, 而 $\mathbf{X}$ 中其他数据点则可能不包含在 $\mathbf{X}_B$ 中. 这个过程可以重复 L 次, 从而生成 L 个大小为 N 的数据集, 其中每个数据集都是通过从原始数据集 $\mathbf{X}$ 中采样得到. 每个数据集可以用来训练一个模型, 并对得到的模型预测进行平均. 这个过程称为**自助聚合** (bootstrap aggregation) 或**袋装** (bagging) .

形成集成模型的另一种方法是使用原始数据集训练多个具有不同架构的模型.

我们可以通过考虑一个仅涉及输入向量 $\mathbf{x}$ 和单输出变量 y 的回归问题来分析集成预测的好处. 假设我们有一

组训练好的模型 $y_1(\mathbf{x}), \cdots, y_L(\mathbf{x})$, 构造由下式给出的集成模型预测:

$$y_{\text{ens}}(\mathbf{x}) = \frac{1}{L}\sum_{l=1}^{L} y_l(\mathbf{x}). \tag{9.41}$$

如果试图预测的真实函数由 $h(\mathbf{x})$ 给出, 那么每个模型输出都可以写成真实值加上一个误差的形式:

$$y_l(\mathbf{x}) = h(\mathbf{x}) + \epsilon_l(\mathbf{x}). \tag{9.42}$$

平均平方和误差则采用以下形式:

$$\mathbb{E}_{\mathbf{x}}\left[\{y_l(\mathbf{x}) - h(\mathbf{x})\}^2\right] = \mathbb{E}_{\mathbf{x}}[\epsilon_l(\mathbf{x})^2], \tag{9.43}$$

其中 $\mathbb{E}_{\mathbf{x}}[\cdot]$ 表示关于输入向量 $\mathbf{x}$ 分布的期望. 因此集合中模型分别预测产生的平均误差为

$$\mathbb{E}_{\text{avg}} = \frac{1}{L}\sum_{l=1}^{L} \mathbb{E}_{\mathbf{x}}[\epsilon_l(\mathbf{x})^2]. \tag{9.44}$$

类似地, 来自集成模型 (9.41) 的期望误差由下式给出:

$$\mathbb{E}_{\text{ens}} = \mathbb{E}_{\mathbf{x}}\left[\left(\frac{1}{L}\sum_{l=1}^{L} \epsilon_l(\mathbf{x})\right)^2\right]. \tag{9.45}$$

如果假设误差均值为零且不相关, 则有

$$\mathbb{E}_{\mathbf{x}}[\epsilon_l(\mathbf{x})] = 0, \tag{9.46}$$

$$\mathbb{E}_{\mathbf{x}}[\epsilon_m(\mathbf{x})\epsilon_l(\mathbf{x})] = 0, \quad m \neq l. \tag{9.47}$$

于是可以得到 (见习题 9.14):

$$\mathbb{E}_{\text{ens}} = \frac{1}{L}\mathbb{E}_{\text{avg}}. \tag{9.48}$$

这个结果表明, 只需要对 L 个不同版本模型进行平均, 就可以将模型平均误差降低到原来 $1/L$. 遗憾的是它依赖于一个关键假设, 即各个模型造成的误差是不相关的. 在实践中误差通常高度相关, 总体误差减小通常要小得多. 然而可以证明, 预期的集成模型误差不会超过集合中模型的预期误差, 因此

$$\mathbb{E}_{\text{ens}} \leq \mathbb{E}_{\text{avg}}. \tag{9.49}$$

另一种称为**提升** (boosting) 的模型组合方法, 将多个" 基" 分类器组合起来, 产生一种性能明显优于任何基分类器的集成模型. 即使基分类器表现只比随机好一点点, 提升也可以给出好结果. 提升方法和前面讨论的集成模型方法 (如袋装) 的主要区别在于: 基分类器是按顺序训练的, 且每个基分类器都是使用数据集的加权形式进行训练的, 其中与每个数据点相关的权重系数取决于前一个分类器的性能. 特别是被其中一个基分类器误分类的点, 在训练序列中下一个分类器时会被赋予更大权重. 一旦所有分类器都训练完毕, 就可以对它们预测通过加权多数投票方式进行整合 (见习题 9.15).

在实践中所有模型组合方法的主要缺点是必须训练多个模型, 然后必须对所有模型预测进行评估, 因而增加了训练和推理计算开销, 开销大小取决于具体应用场景.

9.6.1 Dropout

Dropout 是一种使用广泛且非常有效的正则化形式, 它可以视为一种在不需要单独训练多个模型的情况下, 对指数级数量规模模型隐式地进行模型平均的方法. 它具有广泛适用性, 并且计算开销低.

dropout 核心思想是在训练过程中随机删除网络中的节点, 包括它们之间的连接. 每当一个新数据点进入网络, 模型就会随机选择一些节点并忽略它们.dropout 可以同时应用于隐藏节点和输入节点, 但不能应用于输出节点, 这相当于将被删节点输出设置为零. 它可以通过定义一个掩码向量 $R_i \in \{0, 1\}$ 来实现: 对于数据点 n, 该掩码将被乘以非输出节点 i, 其值以概率 p 设置为 1. 对于隐藏节点 $p = 0.5$ 很有效; 而对于输入节点, 通常使 $p = 0.8$.

在训练过程中, 当每个数据点进入网络时就会创建一个新掩码, 并在该剪枝网络上应用前向和后向传播步骤来创建误差函数梯度, 然后用于更新权重, 例如采用随机梯度下降法. 如果数据点被分组成多个小批次, 则对梯度

在每个小批次 (参见 7.2.4 小节) 中的数据点上进行平均后, 再应用权重更新. 对于具有 M 个非输出节点的网络, 有 2^M 个剪枝网络, 因此在训练期间只会考虑这些网络中的一少部分. 这与传统集成方法不同: 在传统集成方法中集成中的每个网络都是独立训练至收敛的. 它们之间另一个区别是, 指数级数量规模的网络隐式地用 dropout 进行训练, 它们不是独立的, 而是与完整网络共享它们的参数值, 因此剪枝网络间参数也是彼此共享的.

需要注意的是, 使用 dropout 训练后, 单个参数更新包含较大噪声, 因此训练可能需要花费更长时间. 此外由于误差函数本身是有噪声的, 仅通过在训练过程中减小误差函数很难确保优化算法正常工作.

一旦训练完成, 原则上就可以通过应用集成规则 (9.40) 来进行预测, 这种情况下的集成规则如下:

$$p(t \mid \mathbf{x}) = \sum_R p(R)\, p(t \mid \mathbf{x}, R), \tag{9.50}$$

其中求和是在掩码的指数级空间上进行的,$p(t \mid \mathbf{x}, R)$ 是来自具有掩码 R 的网络的预测分布. 因为这种求和难以处理, 所以可以通过对少量掩码进行采样来近似. 在实践中只需要 10 或 20 个掩码就足以获得良好结果. 这一过程称为蒙特卡洛 dropout.

一种更简单方法是在不掩盖训练好的网络节点情况下进行预测, 并重新缩放网络中权重, 使每个节点在测试期间的预期输入与训练期间输入大致相同, 以补偿在训练中会忽略一部分节点的事实. 因此如果一个节点在训练期间以概率 p 的形式存在, 那么在测试期间在使用网络进行预测之前, 需要对来自该节点的输出权重乘以 p.

应用 dropout 的另一种动机来自贝叶斯学派. 在按照贝叶斯学派观点处理问题时, 我们将通过对所有 2^M 个网络模型进行平均来预测, 其中每个网络模型都由其后验概率加权 (参见 2.6 节). 这样做计算成本很高, 无论是在评估后验概率的训练期间, 还是在计算加权预测的测试期间.dropout 通过给每个可能模型赋予相同权重来近似这个模型平均.

dropout 具有良好表现背后的更深层次原因, 在于 dropout 在减少过拟合方面发挥了重要作用. 在一个标准神经网络中, 参数可以调整为错误适应单个数据点, 其隐藏节点被过度特化: 给定其他节点输出, 每个节点会调整其权重以最小化误差, 导致节点以一种可能无法泛化到新数据的方式共同调整参数. 使用 dropout, 每个节点不能依赖其他特定节点存在, 而必须在广泛场景下做出有用贡献, 从而减少共同调整和特化. 对于使用最小二乘法训练的简单线性回归模型,dropout 正则化等价于二次正则化的一种修改形式 (见习题 9.18).

第 10 章 卷积网络

10.1 计算机视觉

最简单的机器学习模型假设观测数据值是“非结构化”的, 这意味着我们事先对数据向量 $\mathbf{x} = (x_1, \cdots, x_p)$ 中各个元素之间可能存在的关系一无所知. 如果我们对这些变量进行随机排列, 并在所有训练数据和测试数据上一致地应用这种固定排列, 那么到目前为止我们所考虑过的模型的性能将不会有任何差异.

然而, 机器学习的许多应用涉及结构化数据 (structured data), 其中输入变量之间存在额外的关系. 例如, 自然语言中的单词形成一个序列 (sequence)(参见第 12 章): 如果我们将语言建模为一个生成式自回归过程, 那么我们将期望每个单词与直接邻近的单词更相关, 而不是依赖于序列中更早出现的单词. 同样, 图像的像素之间具有良定的空间关系, 其中输入变量被安排在二维网格中, 相邻像素具有高度相关的值.

我们已经可以通过在训练目标的误差函数中添加正则化项 (参见 9.1 节), 以及通过数据增强或修改模型架构来利用我们对特定数据模态结构的知识. 这些方法可以帮助引导模型尊重输入数据在变换方面的某些属性, 如不变性和等变性 (参见 9.1.4 小节). 在本章中, 我们将介绍一种称为卷积神经网络 (Convolutional Neural Network, CNN) 的架构方法.CNN 可以视为具有参数共享的稀疏连接多层网络, 并用于编码特定于图像数据的不变性和等变性.

10.1.1 图像数据

图像数据的自动分析和解释是计算机视觉 (computer vision) 领域的焦点, 代表了机器学习的一个主要应用领域 (Szeliski, 2022). 计算机视觉主要基于三维投影几何, 人们曾手工构建特征并用作简单学习算法的输入 (Hartley and Zisserman, 2004). 然而, 计算机视觉是最早被深度学习革命改变的领域之一, 这主要归功于 CNN 架构. 虽然该架构最初是在图像分析场景下开发的, 但它也被应用于序列数据分析等其他场景. 最近, 基于 Transformer(参见第 12 章) 的替代架构在一些应用中已经与 CNN 架构形成竞争态势.

机器学习在计算机视觉领域的应用有很多, 其中最常见的一些应用如下:

(1) 图像分类, 例如将一幅皮肤病变的图像分类为良性或恶性 (参见图 1.1), 这有时称为“图像识别”;
(2) 目标检测, 检测图像中的目标并确定它们在图像中的位置, 例如从自动驾驶汽车收集的相机数据中检测行人 (参见图 10.19);
(3) 图像分割, 其中每个像素被单独分类, 从而将图像划分为共享相同标签的区域. 例如, 自然场景可能分割成天空、草地、树木和建筑物 (参见图 10.26), 而医学扫描图像可以分割成癌变组织和正常组织;
(4) 标题生成, 即从图像中自动生成文本描述 (参见图 12.27);
(5) 图像合成, 例如生成人脸图像 (参见图 1.3), 也可以根据描述所需图像内容的文本输入来合成图像;
(6) 图像修补, 将图像的一个区域替换为与图像其余部分一致的合成像素, 这可用于在图像编辑过程中删除不需要的对象 (参见图 20.9);
(7) 风格迁移, 即将一种风格的输入图像 (例如一张照片) 转换为不同风格的相应图像, 例如一幅油画 (参见图 10.32);
(8) 图像超分, 通过增加像素数量以及与合成相关的高频信息来提高图像的分辨率 (参见图 20.8);
(9) 深度预测, 即使用一个或多个视图来预测目标图像中每个像素处场景与相机的距离;
(10) 场景重建, 即使用一个或多个场景的二维图像来重建三维表示.

图像由像素的矩形数组组成, 其中每个像素要么具有灰度强度, 要么具有更常见的红、绿、蓝通道的三元组强度值. 这些强度值是非负数, 同时也有最大值, 对应于相机或用于捕捉图像的其他硬件设备的限制. 大多数情况下, 我们将强度视为连续变量, 但在实践中它们是用有限精度表示的, 例如用 8 比特表示范围 0 ~ 255 内的整数. 一些图像, 例如医学诊断中使用的磁共振成像 (Magnetic Resonance Imaging, MRI) 扫描图像, 是包含体素的三维网格. 类似地, 视频包含一系列二维图像, 因此也可以视为三维结构, 其中连续的帧随着时间堆叠在一起.

考虑在图像数据上应用神经网络所面临的挑战. 图像通常具有高维特性, 通过相机捕获的图像包含数千万像素. 因此, 将图像数据视为非结构化数据可能需要一个具有大量参数的模型, 而这会导致模型难以训练. 更重要的是, 这样的方法没有考虑到图像数据的高度结构化性质, 其中不同像素的相对位置起着至关重要的作用. 我们之所以关注这一点, 是因为如果我们将图像的像素随机排列, 那么结果就不再像自然图像了. 如果我们为每个像素独立地随机采样像素强度来生成合成图像, 那么生成的合成图像看起来像自然图像 (参见图 6.8) 的可能性基本为零. 局部相关性很重要: 在自然图像中, 与两个相距很远的像素相比, 两个邻近像素具有相似颜色和强度的概率要高得多. 这表明我们可以将图像的先验知识通过强大的归纳偏置编码到神经网络中, 从而得到参数少得多、泛化精度高得多的模型.

引入卷积神经网络的一个动机是, 图像数据是 CNN 所处理的模态, 由于其高维特性, 标准的全连接架构将需要大量的参数. 要了解这一点, 考虑具有 $10^3 \times 10^3$ 像素的彩色图像, 每个像素都有三个值分别对应红、绿、蓝的强度. 如果网络的第一个隐藏层有 1000 个隐藏单元, 那么我们在第一层就已经有 3×10^9 个权重. 此外, 这样的网络必须通过示例来学习任何不变性和等变性, 这将需要一些庞大的数据集. 通过设计一个包含图像结构归纳偏置的架构, 我们可以大大降低对数据集的要求, 并提升模型关于图像空间对称性的泛化能力.

为了利用图像数据的二维结构来创建归纳偏置, 我们可以使用 4 个相互关联的概念: 层次性、局部性、等变性和不变性. 在检测图像中人脸的任务中, 存在一个自然的层次结构. 因为一幅图像可能包含几张人脸, 每张人脸都包含眼睛等元素, 而每只眼睛都有虹膜等结构, 虹膜本身则具有边缘等结构. 在层次结构的最低级别, 神经网络中的节点可以使用图像小区域的局部信息来检测特征 (例如边缘) 的存在, 因此只需要看到图像像素的一少部分即可. 更复杂的结构可以通过组合在前层发现的多个特征来检测. 然而, 一个关键点是, 尽管我们希望在模型中构建层次结构的一般概念, 但我们希望包括在每个级别提取的特征类型在内的层次结构细节, 都是从数据中学习的, 而不是手动编码的. 分层模型可以很自然地与深度学习框架结合, 其已经能够通过一系列 (可能非常多的) 处理 “层” 从原始数据中提取非常复杂的概念. 整个系统是端到端训练的.

10.2 卷积滤波器

10.2.1 特征检测器

为简单起见, 我们最初将重点聚焦在灰度图像 (即具有单个通道的图像) 上. 考虑神经网络第一层中的单个单元, 该单元仅将图像中的一个小矩形区域或小块内的像素值作为输入, 如图 10.1(a) 所示. 这个小矩形区域或小块称为该单元的感受野 (receptive field), 它体现了局部性的概念. 我们希望与此单元相关的权重值 (简称权重) 能够学习检测一些有用的低级特征. 该单元的输出以函数的形式给出: 该函数首先对输入值进行加权线性组合, 然后使用非线性激活函数进行变换:

$$z = \mathrm{ReLU}\left(\mathbf{w}^{\mathsf{T}}\mathbf{x} + w_0\right) \tag{10.1}$$

其中 $\mathbf{x}$ 是感受野的像素值向量, 我们假设使用了 ReLU 激活函数. 由于每个输入像素都有一个权重相关联, 因此权重本身会形成一个小的、称为滤波器 (filter) 的二维网格, 有时也称核 (kernel), 其本身可以像图像一样可视化, 如图 10.1(b) 所示.

假设式 (10.1)中的 $\mathbf{w}$ 和 w_0 是固定的, 我们想知道对于哪一个输入图像小块 $\mathbf{x}$, 这个隐藏单元将给出最大输出响应. 为了回答这个问题, 我们需要以某种方式约束 $\mathbf{x}$. 假设它的范数 $\|\mathbf{x}\|^2$ 是固定的, 那么对于某个系数 α, 使 $\mathbf{w}^{\mathsf{T}}\mathbf{x}$ 最大化并因此最大化隐藏单元响应的 $\mathbf{x}$ 的解的形式为 $\mathbf{x} = \alpha\mathbf{w}$(见习题 10.1). 也就是说, 当检测到一个在整体缩放下像核图像的图像小块时, 就会产生来自这个隐藏单元的最大输出响应. 注意, 只有当 $\mathbf{w}^{\mathsf{T}}\mathbf{x}$ 超过 $-w_0$ 的阈值时,ReLU 才会生成非零输出. 因此该隐藏单元充当了特征检测器的角色, 它会在找到与其卷积核匹配足够好的图像小块时发出信号.

10.2.2 平移等变性

接下来请注意, 如果人脸图像中的小块表示处在该位置的眼睛, 那么图像不同部分的同一组像素值表示不同位置的眼睛. 我们的神经网络需要能够将其在一个位置学到的知识泛化到图像中的所有可能位置, 而不需要在训练集中在每个可能位置见过对应特征的示例. 为了实现这一点, 我们可以简单地在图像的多个位置复制相同的隐藏单元权值, 如图 10.2 中的一维输入空间所示.

如果图像的局部小块在连接到该小块的单元中产生特定响应, 则不同位置的同一组像素值将在特征图中的相应变换位置产生相同的响应. 这是等变性 (equivariance) 的一个例子 (参见 9.1.3 小节). 可以看到这个网络中的连接是稀疏的 (sparse), 因为大多数连接不存在. 此外, 权值由所有隐藏单元共享. 这一变换是卷积 (convolution) 的一个例子. 隐藏层的单元形成一个特征图 (feature map), 其中所有单元共享相同的权重.

我们可以将卷积的想法扩展到二维图像 (Dumoulin and Visin, 2016). 对于像素强度为 $I(j,k)$ 的图像 I 和像素值为 $K(l,m)$ 的滤波器 K, 特征图 C 的激活值由下式给出:

$$C(j,k) = \sum_{l,m} I(j+l,\ k+m)\, K(l,m) \tag{10.2}$$

为了清晰起见, 我们省略了非线性激活函数. 这也是卷积的一个例子, 且有时可写作 $C = I * K$. 请注意, 严格来说, 式 (10.2)所示的操作应称为互相关 (cross-correlation), 它与卷积的传统数学定义略有不同, 但在这里我们将遵循机器学习文献中的常见实践, 并将式 (10.2)所示的操作称为卷积 (见习题 10.4). 图 10.3 以 3×3 图像和 2×2 滤波器为例, 对这种操作做了说明 (参见 7.4.2 小节). 重要的是, 当在卷积网络中使用批量归一化时, 在归一化单元状态过程中, 必须在特征图中的每个空间位置使用相同的均值和方差, 以确保特征图的统计量与位置无关.

通过将这种卷积结构与标准的全连接网络作比较, 我们看到了前者的 3 个优点: 首先, 连接是稀疏的, 即使对于较大的图像, 权重也要少得多; 其次, 权值是共享的, 这大大减少了独立参数的数量, 从而减小了学习这些参数所需的训练集大小; 最后, 相同的网络可以应用于不同大小的图像, 而无需重新训练. 关于卷积网络的最后一个观察是, 通过利用图形处理单元 (Graphics Processing Unit, GPU) 高计算吞吐量的大规模并行特性, 卷积在实现中计算效率非常高.

作为卷积应用的一个例子, 考虑使用固定的、手工制作的卷积滤波器检测图像边缘的问题. 直观上, 可以认为竖直边缘是我们在图像上水平移动时, 因像素之间的强度发生显著的局部变化而产生的. 我们可以通过使用以下形式的 3×3 滤波器对图像进行卷积来测量这一点:

$$K = \begin{bmatrix} 1 & 0 & -1 \\ 1 & 0 & -1 \\ 1 & 0 & -1 \end{bmatrix} \tag{10.3}$$

类似地, 我们也可以使用此滤波器的转置来对图像进行卷积, 从而检测图像的水平边缘:

$$K = \begin{bmatrix} 1 & 1 & 1 \\ 0 & 0 & 0 \\ -1 & -1 & -1 \end{bmatrix} \tag{10.4}$$

图 10.4 显示了将这两个卷积滤波器应用于示例图像的结果. 注意在图 10.4(b) 中, 如果竖直边缘对应于像素强度的增加, 则特征图上的相应点为正 (用浅色表示); 而如果竖直边缘对应于像素强度的降低, 则特征图上的相应点为负 (用深色表示). 图 10.4(c) 所示水平边缘的结果具有类似的性质.

10.2.3 填充

从图 10.3 中可以看出, 卷积图比原始图像小. 如果原始图像的维度为 $J \times K$, 并且我们用维度为 $M \times M$ 的核对其进行卷积 (滤波器通常是正方形的), 则生成的特征图的维度为 $(J - M + 1) \times (K - M + 1)$. 在某些情况下, 我

们希望特征图具有与原始图像相同的尺寸. 这可以通过在原始图像外部填充 (padding) 额外像素来实现, 如图 10.5 所示. 如果我们用 P 个像素填充原始图像, 则输出的特征图的维度为 $(J+2P-M+1)\times(K+2P-M+1)$. 如果没有对原始图像进行填充, 即 $P=0$, 则称为有效 (valid) 卷积. 当选择的 P 值使得输出数组的大小与输入数组的大小相同时, 对应于 $P=(M-1)/2$, 则称为等大 (same) 卷积, 因为原始图像和特征图具有相同的维度 (见习题 10.6). 在计算机视觉中, 滤波器的 M 通常为奇数, 因此填充可以在原始图像的所有边上对称进行, 并且滤波器具有一个与位置关联的明确定义的中心像素. 最后, 我们需要为填充像素选择一个合适的强度值. 典型的选择是将填充值设置为零 (在先从每个图像中减去平均值之后), 这样零就是像素强度的平均值. 填充也可以应用于特征图, 以便由后续卷积层处理.

10.2.4 跨步卷积

在典型的图像处理应用中, 图像可以有非常多的像素, 并且由于卷积核通常相对较小, 因此 $M<J,K$, 特征图的大小将与原始图像相似 (如果进行等大填充, 则大小相同). 有时, 我们希望使用比原始图像小得多的特征图, 以便在设计卷积网络时更灵活. 实现此目的的一种方法是使用跨步卷积 (strided convolution), 即不再将滤波器一次移动一个像素, 而是以较大的步长 S 移动滤波器. 我们称 S 为步幅 (stride). 如果在水平和竖直方向上使用相同的步幅, 那么特征图中的元素数量为 (见习题 10.7):

$$\left\lfloor \frac{J+2P-M}{S}+1 \right\rfloor \times \left\lfloor \frac{K+2P-M}{S}+1 \right\rfloor \tag{10.5}$$

其中 $\lfloor x \rfloor$ 表示将 x 向下取整, 得到小于或等于 x 的最大整数. 对于大图像和小尺寸滤波器, 特征图将是原始图像大小的 $1/S$.

10.2.5 多维卷积

到目前为止, 我们已经考虑了单个灰度图像上的卷积. 彩色图像有三个通道, 对应于红、绿、蓝. 通过扩展滤波器的维度, 我们可以轻松地扩展卷积以支持多个通道. 具有 $J\times K$ 个像素和 C 个通道的图像可以用维度为 $J\times K\times C$ 的张量来描述 (见 6.3.7 小节). 我们可以引入一个用维度为 $M\times M\times C$ 的张量来描述的滤波器, 该张量对于 C 个通道中的每一个通道都有一个独立的 $M\times M$ 滤波器. 假设对原始图像没有进行填充且步幅为 1, 则同样可以得到大小为 $(J-M+1)\times(K-M+1)$ 的特征图, 如图 10.6 所示.

下面让我们对卷积进一步进行重要扩展. 目前, 我们已经创建了单一的特征图, 并且特征图中的所有点共享同一组可学习参数. 对于维度为 $M\times M\times C$ 的滤波器, 无论图像大小如何, 都将有 M^2C 个权重参数. 此外, 还有一个与该单元相关的偏置参数. 这种滤波器类似于全连接网络中的单个隐藏节点, 它只能学习检测一种特征, 因此作用非常有限. 为了构建更灵活的模型, 我们需要使用多个这样的滤波器, 其中每个滤波器都有互不相关的参数集, 从而产生独立的特征图, 如图 10.7 所示. 我们再次将这些独立的特征图称为通道 (channel). 滤波器张量现在的维度为 $M\times M\times C\times C_{\text{out}}$, 其中 C 是输入通道数,C_{out} 是输出通道数. 每个输出通道都有自己的相关偏置参数, 因此参数总数为 $(M^2C+1)C_{\text{out}}$.

设计卷积网络时的一个有用概念是 1×1 卷积 (Lin, Chen, and Yan, 2013), 它是一个滤波器大小为单个像素的卷积层. 滤波器有 C 个权重 (每个输入通道一个权重), 外加一个偏置.1×1 卷积通过将输出通道数设置为与输入通道数不同, 可以简单地用来改变通道数 (通常用来减少通道数), 而不改变特征图的大小. 因此, 它是对跨步卷积或池化的补充, 因为它减少了通道的数量而不是通道的维度.

10.2.6 池化

卷积层编码了平移等变性 (equivariance): 如果将一小块像素 (代表隐藏单元的感受野) 移动到图像中的不同位置, 则特征图的关联输出将移动到特征图中的相应位置. 这对于在图像中找到目标的位置等应用很有价值. 对于其他应用, 例如图像分类, 我们希望在输入平移时输出保持不变. 然而, 在所有情况下, 我们都希望网络能够学习分层结构, 其中特定级别的复杂特征是由前一级的简单特征构建而成的. 在许多情况下, 这些简单特征之间的空

间关系非常重要. 例如, 眼睛、鼻子和嘴巴的相对位置有助于确定面部的存在, 而非仅仅表明这些特征在图像中存在. 但是, 相对位置的微小变化不会影响分类, 我们希望对单个特征的这种小平移保持输出不变. 这可以通过在卷积层输出之后应用池化 (pooling) 来实现.

池化与卷积有相似之处, 因为单元阵列排列在网格中, 且其中的每个单元都从前一个特征图的感受野中获取输入. 此外, 池化同样需要选择滤波器大小和步幅. 然而, 池化与卷积的不同之处在于, 池化单元的输出是其输入的简单、固定的函数, 因此池化中没有可学习参数. 池化的一个常见示例是最大池化 (max-pooling)(Zhou and Chellappa, 1988), 其中每个单元仅输出对于输入值的“取最大值”函数. 图 10.8 所示的简单示例说明了这一过程, 其中使用“max”运算符将特征图中的 2×2 像素块组合在一起, 从而生成较小维度的新特征图. 这里的步幅等于滤波器的宽度, 因此感受野没有重叠.

我们可以将特征图中单元的激活解释为检测相应特征强度的度量, 因此最大池化保留了有关特征是否存在以及特征强度的信息, 但丢弃了一些位置信息. 池化还有许多其他选择, 例如平均池化 (average pooling), 其中池化函数计算特征图中相应感受野值的平均值. 这些都引入了一定程度的局部平移不变性. 除了构建一些局部平移不变性外, 池化通过对特征图下采样, 还可用于降低表示的维度. 注意, 在卷积层中使用大于 1 的步幅也可以实现对特征图下采样的效果.

例如, 如果我们有一个 8 通道的特征图, 每个通道的维度为 64×64, 应用感受野大小为 2×2 且步幅为 2 的最大池化, 则池化运算的输出将是一个维度为 $32 \times 32 \times 8$ 的张量. 我们还可以在特征图的多个通道中应用池化, 这将使网络有潜力学习简单平移不变性之外的其他不变性. 例如, 如果卷积层中的多个通道学习检测不同方向上的相同特征, 则这些特征图的最大池化将对于旋转变换 (几乎) 保持不变. 池化还允许卷积网络处理不同大小的图像. 归根结底, 卷积网络的输出以及通常的一些中间层必须具有固定尺寸. 根据图像大小改变池化的步幅大小, 可以使卷积网络适应可变大小的输入, 以确保池化输出的数量保持不变.

10.2.7 多层卷积

到目前为止我们描述的卷积网络结构类似于标准全连接神经网络中的单层网络结构. 为了允许网络发现和表示数据中的分层结构, 现在我们通过使用多个上述类型的卷积层来扩展架构. 每个卷积层由维度为 $M \times M \times C_{\text{in}} \times C_{\text{out}}$ 的滤波器张量表示, 其中独立权重和偏置参数的数量为 $(M^2 C_{\text{in}} + 1) C_{\text{out}}$. 每个这样的卷积层都允许我们选择性地在通道数量上进行扩展, 后续加上一个池化层. 现在, 我们可以连续应用多个这样的卷积层和池化层, 其中某个层的输出通道 C_{out}(类似于输入图像的 RGB 通道) 形成下一层的输入通道. 注意, 特征图中的通道数有时称为特征图的“深度”, 但我们更愿意用“深度”来表示网络中的层数.

卷积框架的一个关键性质是局部性, 特征图中的给定单元仅从前一层的一小块 (即感受野) 中获取信息. 当我们构建一个每一层都是卷积层的深度神经网络时, 后面层中单元的有效感受野会比前面层中单元的有效感受野大得多, 如图 10.9 所示. 早期的 CNN 通常具有较少的卷积层, 因为它们具有较大的感受野. 例如,AlexNet(Krizhevsky, Sutskever, and Hinton, 2012) 具有步幅为 4、尺寸为 11×11 的感受野. 从图 10.9 中可以看出, 通过使用每层只有较小感受野的多层结构, 也可以隐式地实现较大的感受野. 后者的优点是需要的参数要少得多, 并且有效地对较大的滤波器施加了归纳偏置, 因为它们必定由卷积子滤波器组成. 尽管这是一个高度复杂的架构, 但只需要对网络函数本身进行显式编码即可, 因为我们可以使用自动微分 (参见 8.2 节) 来计算损失函数的导数, 并使用随机梯度下降来优化损失函数.

在许多应用中, 网络的输出单元需要预测图像整体的信息. 例如在图像分类任务中, 网络需要组合来自整个输入图像的信息. 这通常是通过在网络的最后阶段引入一个或两个标准的全连接层来实现的, 其中每个单元都连接到前一层中的每个单元. 这种架构中的参数数量是可控的, 因为中间池化层的存在, 最终卷积层的维度通常比输入层的维度少得多. 然而, 即使网络中 (共享的) 连接数量比卷积层中的多, 最终全连接层也可能包含网络中大多数独立自由度. 因此, 一个完整的 CNN 包括多层卷积, 这些卷积穿插着池化操作, 并且通常在网络的最后阶段有传统的全连接层. 在设计这样的架构时, 需要做出许多选择, 包括层数、每层中的通道数、滤波器尺寸、步幅大小以及多个其他此类超参数. 在实践中, 训练每个候选配置的计算成本很高, 很难使用留出法对超参数的值进行系统的比较, 研究人员为此探索了各种不同的架构.

10.2.8 网络架构示例

卷积网络是第一个成功部署在应用中的深度神经网络 (即具有两个以上可学习参数层的神经网络). 一个早期的例子是 LeNet, 它用于对低分辨率的单色手写数字图像进行分类 (LeCun et al., 1989; LeCun et al., 1998). 通过引入名为 ImageNet(Deng et al., 2009) 的大规模基准数据集, 开发更强大的卷积网络得到了加速. 该数据集包含约 1400 万幅自然图像, 其中每幅图像都被手工归为近 22000 个类别之一. 这是一个比我们以前使用的所有数据集都大得多的数据集,ImageNet 推动的领域进步强调了大规模数据以及设计具有适当归纳偏置的良好模型在构建成功深度学习方案时的重要性.

ImageNet 中包含 1000 个非重叠类别的图像子集构成了年度 ImageNet 大规模视觉识别挑战赛 (ImageNet Large Scale Visual Recognition Challenge) 的基础. 同样, 这比之前通常考虑几十个类别要多得多. 拥有如此多的类别使问题变得更加棘手, 因为如果类别分布均匀, 随机猜测的错误率将达到 99.9%. 该数据集包含超过 128 万幅训练图像、5 万幅验证图像和 10 万幅测试图像. 分类器旨在为测试图像生成预测输出类别的排序列表, 并根据 Top 1 和 Top 5 的错误率报告结果. 这意味着如果真实类别出现在列表顶部或位于 5 个排名最靠前的类别预测之中, 则视为图像分类正确. 该数据集的早期结果实现了 Top 5 的错误率约为 25.5%.AlexNet 卷积网络架构 (Krizhevsky, Sutskever, and Hinton, 2012) 取得了重要进展, 赢得了 2012 年的比赛, 并将 Top 5 的错误率降低到 15.3% 的新纪录.AlexNet 模型的关键之处在于使用 ReLU 函数作为激活函数、应用 GPU 训练网络以及使用 dropout 正则化 (参见 9.6.1 小节). 该模型随后几年取得了进一步的成功, 错误率降低至约 3%, 在某些数据上比人类水平 (约 5%) 表现得要好一些 (Dodge and Kara, 2017). 这可以归因于人类难以区分只有微妙差异的不同类别 (例如不同的蘑菇品种).

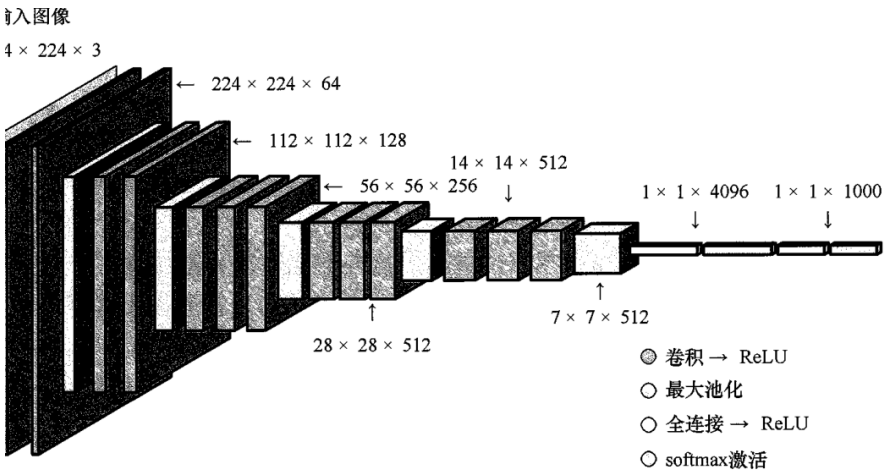
作为典型卷积网络架构的一个示例, 我们来详细了解一下 VGG-16 模型 (Simonyan and Zisserman, 2014), 其中 VGG 代表开发该模型的 Visual Geometry Group,16 表示模型中可学习层的数量.VGG-16 模型有一些简单的设计原则, 旨在得到相对一致的架构, 如图 10.10 所示, 即最大限度地减少需要选择的超参数的数量.

该模型以 224×224 像素和三颜色通道的图像作为输入, 然后接入穿插有下采样的卷积层集合. 所有卷积层都具有尺寸为 3×3、步幅为 1 的滤波器, 执行等大填充并使用 ReLU 激活函数; 而最大池化操作都使用步幅为 2、尺寸为 2×2 的滤波器, 从而将单元数下采样为原先的四分之一. 第 1 个可学习层是一个卷积层, 其中每个单元从输入通道堆叠得到的 $3 \times 3 \times 3$ 的“立方体”获取输入, 因此包括偏置在内共有 28 个参数, 这些参数在该通道的特征图中的所有单元之间共享. 第 1 层有 64 个这样的特征通道, 从而使输出张量的大小为 $224 \times 224 \times 64$. 第 2 层也是卷积层, 同样有 64 个通道. 紧接着是最大池化层, 给出的特征图大小为 112×112. 第 3 层和第 4 层又是卷积层, 维度为 112×112, 每层都有 128 个通道. 通道数量的增加在一定程度上抵消了最大池化层中的下采样, 以确保每层表示中的变量数量不会随着所通过网络层数的增加而过快地减少. 同样, 接下来是最大池化操作, 以产生大小为 56×56 的特征图. 接下来是另外三个卷积层, 每个卷积层有 256 个通道, 从而再次将与下采样相关的通道数量增加一倍. 接下来是另一个最大池化操作, 得到大小为 28×28 的特征图. 紧接着又是三个卷积层, 每个卷积层有 512 个通道, 后面跟着另一个最大池化操作, 下采样得到大小为 14×14 的特征图. 接下来是另外三个卷积层, 尽管这三个卷积层中的特征图数量保持在 512, 但接下来再次进行最大池化操作, 从而将特征图的大小减小到 7×7. 最后, 还有三层是全连接层, 这意味着它们是具有完全连接且不共享参数的标准神经网络层. 最终的最大池化层有 512 个通道, 每个通道的大小为 7×7, 总共 25088 个单元. 第一个全连接层有 4096 个单元, 每个单元都连接到每个最大池化单元. 接下来是第二个全连接层, 它同样有 4096 个单元. 最后是第三个全连接层, 它有 1000 个单元, 以便将网络应用于包含 1000 个类别的分类问题. 网络中除输出层外的所有可学习层都具有非线性 ReLU 激活函数, 输出层具有 softmax 激活函数.

总体而言,VGG-16 模型中总共有大约 1.38 亿个可独立学习的参数, 其中大部分 (近 1.03 亿个) 位于第一个全连接层中, 而大多数连接位于第一个卷积层中 (参见习题 10.8).

10.3 可视化训练好的 CNN

本节介绍探索现代深度 CNN 所学习到的特征, 你将看到它们与哺乳动物视觉皮层性质的一些显著相似之处.



一个典型卷积网络（在本例中是 VGG-16 模型）的架构

图 10.1: 一个典型卷积网络 (在本例中是 VGG-16 模型) 的架构.

10.3.1 视觉皮层

研发 CNN 的大部分动机来自神经科学的开创性研究, 这些研究深入了解了包括人类在内的哺乳动物视觉处理的本质. 来自视网膜的电信号通过位于大脑后部的视觉皮层中的一系列处理层进行转换, 其中神经元组织成二维片, 每个二维片形成二维视野的映射. 在这些开创性研究中,Hubel and Wiesel (1959) 测量了当向猫的眼睛呈现视觉刺激时, 猫的视觉皮层中单个神经元的电响应. 他们发现, 一些称为 “简单细胞” 的神经元对视觉输入有强烈的响应, 这些视觉输入朝向特定角度的简单边缘, 位于视野内的特定位置. 而其他刺激在这些神经元中产生的响应相对较少. 更详细的研究表明, 这些简单细胞的响应可以使用 Gabor 滤波器来建模,Gabor 滤波器是由下式定义的二维函数:

$$G(x, y) = A \exp\left(-\alpha \tilde{x}^2 - \beta \tilde{y}^2\right) \sin(\omega \tilde{y} + \phi) \tag{10.6}$$

其中

$$\tilde{x} = (x - x_0)\cos\theta + (y - y_0)\sin\theta \tag{10.7}$$

$$\tilde{y} = -(x - x_0)\sin\theta + (y - y_0)\cos\theta \tag{10.8}$$

式 (10.7)和式 (10.8)表示坐标系旋转角度 θ, 因此式 (10.6)中的 $\sin(\cdot)$ 项表示朝向极角 θ 方向的正弦空间振荡, 频率为 ω, 相位角为 ϕ. 式 (10.6)中的指数因子旨在创建一个衰减包络, 将滤波器置于点 (x_0, y_0) 的邻域内, 衰减速率由 α 和 β 控制. 式 (10.6)定义的 Gabor 滤波器示例如图 10.11 所示.

Hubel 和 Wiesel 还发现了 “复杂细胞” 的存在, 这些细胞会响应更复杂的刺激, 并且该刺激似乎是通过组合和处理简单细胞的输出得到的. 这些响应对输入中的微小变化 (例如位置偏移) 表现出一定程度的不变性, 类似于深度卷积网络中的池化单元. 哺乳动物视觉处理系统的更深层对视觉输入的变换具有更具体的响应和更大的不变性. 这种细胞称为 “祖母细胞”, 因为不论场景的位置、大小、照明或其他变换如何, 当且仅当视觉输入与某些特定的概念 (如祖母的头像) 相对应时, 这种细胞才能在理论上做出响应. 这项工作直接启发了一种称为新认知机 (neocognitron)(Fukushima, 1980) 的深度神经网络的早期形式, 它是卷积神经网络的先驱. 新认知机支持多层处理, 包括了具有共享权重的局部感受野和紧接着的局部平均或最大池化, 以确保位置不变性. 然而, 新认知机缺乏端到端的训练过程, 而依赖于无监督聚类算法进行贪婪的逐层学习.

10.3.2 可视化训练好的滤波器

假设我们有一个训练好的深度 CNN, 我们希望了解隐藏单元学会了检测什么. 对第一个卷积层中的滤波器, 这是相对直接的, 因为它们对应于原始输入图像空间中的小块, 我们可以将这些滤波器的网络权重直接可视化为小的图像块. 图 10.12 展示了一些在 ImageNet 数据集上训练好的 CNN 第一层滤波器示例. 我们看到这些滤波器与图 10.11 中的 Gabor 滤波器之间存在显著的相似性. 然而, 这并不意味着卷积神经网络是大脑工作的良好模型, 因为我们可以从各种统计方法中获得非常相似的结果 (Hyvärinen, Hurri, and Hoyer, 2009). 这些特征滤波器表示的是统计自然图像的一般性质, 它们对于自然和人工系统中的图像理解都被证明是有用的.

虽然我们可以直接可视化第一层中的滤波器, 但网络中的后续层更难解释, 因为它们的输入不是图像小块, 而是滤波器响应. 一种方法 (类似于 Hubel 和 Wiesel 使用的方法) 是向网络提供大量图像小块, 并观察哪些图像小块在任何特定隐藏单元中产生最大的激活值. 图 10.13 显示了在网络的隐藏单元中产生最强激活的图像小块示例. 该网络有 5 个卷积层, 紧接着是两个全连接层, 在包含 1000 个类别的 130 万幅 ImageNet 图像上完成训练. 我们看到一个自然的分层结构, 第 1 层对边缘响应, 第 2 层对纹理和简单形状响应, 第 3 层显示对象的组成部分 (如轮子), 第 5 层显示整个对象.

我们可以扩展这种技术, 使其不仅仅从验证集中选择图像小块, 而且对输入变量进行数值优化, 以最大限度地激活特定单元 (Zeiler and Fergus, 2013; Simonyan, Vedaldi, and Zisserman, 2013; Yosinski et al., 2015). 如果我们选择某单元作为输出之一, 则可以寻找最能代表相应类标签的图像. 由于输出单元通常具有 softmax 激活函数, 因此最好最大化输入 softmax 激活函数的预激活值, 而不是直接最大化类概率, 因为这样可以确保优化仅依赖于一个类. 例如, 假设我们想要寻找对类别 “狗” 产生最强烈响应的图像, 那么如果我们优化 softmax 输出, 由于 softmax 激活函数中的分母的存在, 可能会使图像变得不那么像猫. 然而, 输出单元激活的无约束优化会导致单个像素值趋于无穷大, 并且还会产生难以解释的高频结构, 因此需要某种形式的正则化来找到更接近自然图像的解.Yosinski et al. (2015) 使用了一种正则化函数, 该函数涉及像素值的平方和以及一个变换过程. 该过程将基于梯度的更新替换为删除高频结构的模糊操作, 并引入剪裁操作来将那些对类标签贡献很小的像素值设置为零, 示例结果如图 10.14 所示. 这种方法与对抗训练有关 (参见第 17 章).

10.3.3 显著性图

深入了解卷积网络所用特征的另一种方法是使用显著性图 (saliency map), 目的是在确定类标签时识别图像中最重要的区域. 这最好通过研究最终卷积层来完成, 因为该卷积层仍然保留了空间定位信息, 该信息在随后的全连接层中会丢失, 但它们具有最高级别的语义表示. 对于给定的输入图像,Grad-CAM(Gradient Class Activation Mapping, 梯度类激活图) 方法 (Selvaraju et al., 2016) 首先在 softmax 输出之前, 计算对于给定类别 c 的输出单元预激活 a^c 关于最终卷积层通道 k 中所有单元预激活 a_{ij}^k 的导数. 对于该层中的每个通道, 这些导数的平均值形式如下:

$$\alpha_k^c = \frac{1}{M} \sum_{i,j} \frac{\partial a^c}{\partial a_{ij}^k} \tag{10.9}$$

其中 i 和 j 用于索引通道 k 的行和列,M 是通道 k 中的单元总数. 然后, 这些平均值被用于形成以下形式的加权和:

$$L_{\text{Grad-CAM}}^c = \text{ReLU}\left(\sum_k \alpha_k^c A^k\right) \tag{10.10}$$

其中 A^k 是包含元素 a_{ij}^k 的矩阵. 生成的数组与最终卷积层具有相同的维度, 例如图 10.10 所示的 VGG-16 模型的维度为 14×14, 并且可以用 “热力图” 的形式叠加在原始图像上, 如图 10.15 所示.

10.3.4 对抗攻击

与输入图像像素值变化相关的梯度也可用于创建针对卷积网络的对抗攻击 (adversarial attack)(Szegedy et al., 2013). 这类攻击以人类无法察觉的水平对图像进行非常小的修改, 从而导致神经网络对图像分类错误. 创建对抗性图像的一种简单方法是快速梯度符号 (fast gradient sign) 法 (Goodfellow, Shlens, and Szegedy, 2014). 这涉及将图像 $\mathbf{x}$ 中的每个像素值更改为固定量 ϵ, 符号由误差函数 $E(\mathbf{x}, t)$ 相对于像素值的梯度确定. 我们将得到一幅由下式定义的修改后的图像:

$$\tilde{\mathbf{x}} = \mathbf{x} + \epsilon \operatorname{sign}(\nabla_{\mathbf{x}} E(\mathbf{x}, t)) \tag{10.11}$$

这里的 t 是 $\mathbf{x}$ 的真实标签, 误差 $E(\mathbf{x}, t)$ 举例来说可以是 $\mathbf{x}$ 的负对数似然. 使用反向传播可以有效地计算所需梯度 (参见第 8 章). 在神经网络的常规训练中, 可以通过调整网络参数来最小化误差, 而式 (10.11)定义的修改会改变图像 (同时保持训练好的网络参数固定), 从而导致误差增加. 通过保持 ϵ 为较小的值, 可以确保人眼无法察觉图像的变化. 值得注意的是, 这可以使网络以较高的置信度将图像分类错误, 如图 10.16 中的示例所示: 左边的图像以 57.7% 的置信度归类为熊猫, 添加少量随机扰动 (这些扰动本身以 8.2% 的概率归类为熊猫), 导致右边的图像以 99.3% 的置信度归类为长臂猿.

以这种方式欺骗神经网络会引发潜在的安全问题, 因为它为攻击训练好的分类器创造了机会. 这个问题似乎是由过拟合引起的, 在过拟合中, 具有较高拟合能力的模型已经精确地适应了特定的图像, 使得输入中的微小变化会导致预测类概率的大幅变化. 然而事实证明, 一幅经过调整能使特定训练好的网络产生虚假输出的图像, 在被输入其他网络时也会导致类似的虚假输出 (Goodfellow, Shlens, and Szegedy, 2014). 此外, 使用灵活性低得多的线性模型可以得到类似的对抗结果. 人工对图像进行物理修改也可能导致这种现象: 当把一幅规则的、未损坏的人工图像呈现给训练好的神经网络时, 该网络也会给出错误的预测, 如图 10.17 所示. 虽然这些对抗攻击的基本类型可以通过对网络训练过程进行简单修改来解决, 但更复杂的对抗攻击方法处理起来依旧较为困难. 了解这些结果的含义并缓解隐患仍然是一个开放的研究领域.

10.3.5 合成图像

在关于图像修改的最后一个示例中, 我们将讨论一种称为 DeepDream 的技术 (Mordvintsev, Olah, and Tyka, 2015), 它为运行训练好的卷积网络提供了额外的见解. 它的目标是生成具有夸张特征的合成图像. 为此, 我们需要确定网络特定隐藏层中的哪些节点对特定图像有较强响应, 然后修改图像从而放大这些响应. 例如, 我们将一些云的图像输入一个在对象检测任务上训练好的网络, 并且如果一个特定节点在图像的特定区域检测到类似猫的模式, 则将该区域修改得更像“猫”.

为此, 我们首先将图像输入网络, 并前向传播到某个特定层. 然后将该层的反向传播 δ 变量设置为等于节点的预激活, 并向输入层反向传播, 从而获得图像像素的梯度向量 (参见 8.1.2 小节). 最后沿着梯度向量前进一小步来修改图像. 该过程可以视作利用一种基于梯度的方法来增大函数的值 (见习题 10.10). 要最大化的函数为

$$F(I) = \sum_{i,j,k} a_{ij}^{k}(I)^2 \tag{10.12}$$

其中 $a_{ij}^{k}(I)$ 是当输入图像为 I 时, 所选网络层中通道 k 的第 i 行和第 j 列单元的预激活, 求和操作则涉及该层中所有的单元和通道. 为了生成平滑的图像, 可以使用空间平滑和像素裁剪形式的某些正则化. 如果需要更强的增强效果, 可以多次重复此过程. 将 DeepDream 技术应用于图像的示例如图 10.18 所示. 有趣的是, 即使卷积网络被训练以区分对象类别, 但它们似乎至少能够捕获从这些类生成图像所需的一些信息. 这种技术可以应用于照片, 或者我们也可以将随机噪声作为输入, 从而完全由训练好的网络生成图像. 尽管 DeepDream 技术提供了一些关于训练好的网络运行的见解, 但它主要用于生成艺术风格的有趣图像.

10.4 目标检测

前面主要以图像分类问题为例介绍了 CNN 的设计动机, 其中整幅图像被分类为单一类别, 例如“猫”或“自行车”. 这在诸如 ImageNet 的数据集上是合理的, 因为每幅图像都设计为主要包含单一对象. 然而, 还有许多其他应用可以利用 CNN 内建的归纳偏置. 更一般地说, 如果 CNN 中的卷积层已经在大型图像数据集的特定任务上训练好, 那么 CNN 就可以学习具有广泛适用性的内部表示. 因此, 我们可以针对大量的特定任务对 CNN 进行微调 (参见 6.3.4 小节). 你已经看到了一个在 ImageNet 数据上训练好的卷积网络的例子, 其通过迁移学习能够在皮肤病变图像的分类上达到人类水平 (参见 1.1.1 小节).

图像中有多个对象属于一个或多个类别的情况很常见, 我们可能希望检测其中每个对象的存在及其所属的类别. 此外, 在许多计算机视觉应用中, 我们还需要确定在图像中检测到的任何对象的位置. 例如, 使用 RGB 摄像头的自动驾驶车辆可能需要检测行人的存在和位置, 并识别道路标志、其他车辆等.

10.4.1 边界框

考虑明确图像中对象位置的问题. 一种广泛使用的方法是定义一个边界框 (bounding box), 它是一个紧密贴合对象边界的矩形, 如图 10.19 所示. 边界框可以由其中心坐标以及宽度和高度定义, 形式为向量 $\mathbf{b} = (b_x, b_y, b_w, b_h)$. 这里 $\mathbf{b}$ 中的元素可以以像素为单位指定, 也可以用连续数字表示. 按照惯例, 图像的左上角对应坐标 $(0, 0)$, 右下角对应坐标 $(1, 1)$.

当假设图像有且仅有一个对象, 且该对象来自一个预定义的类别集合 (其中的类别数量为 C) 时,CNN 通常会有 C 个输出单元, 其激活函数由 softmax 函数定义 (见 5.3 节). 对象可以由 4 个额外的具有线性激活函数的输出来定位, 用于训练预测边界框坐标 (b_x, b_y, b_w, b_h). 由于这些量是连续的, 因此可以使用对应输出的平方和误差函数. 例如,Redmon et al. (2015) 使用此方法首先将图像划分成一个 7×7 的网格; 然后对于每个网格单元, 使用卷积网络并基于从整个图像中提取的特征, 输出与网格单元关联的任意对象的类别和边界框坐标.

10.4.2 交并比

我们需要一种有意义的方式来衡量训练好的网络在预测边界框方面的性能. 在图像分类中, 网络的输出是类别标签的概率分布, 我们可以通过查看测试集上真实类别标签的对数似然来衡量性能. 然而, 在对象定位中, 我们需要某种方法来衡量边界框相对于某些真值的准确性, 其中后者可以通过像人工标注这样的方式获得. 预测框与目标框的重合程度可以用作这种度量的基础, 但重叠区域的面积会取决于图像中对象的大小. 此外, 如果预测的边界框超出实际的边界框, 则应该受到惩罚. 一种能够较好地解决这两个问题的度量方法称为交并比 (Intersection-over-Union, IoU), 它是这两个边界框的交集区域面积与并集区域面积的比值, 如图 10.20 所示. 注意,IoU 度量的取值范围是 0 ~ 1. 如果 IoU 度量超过某个阈值 (通常为 0.5), 则可以将预测标记为正确. 注意,IoU 通常不直接用作训练的损失函数, 因为想要通过梯度下降优化它很困难, 因此通常使用中心对象来训练, 而 IoU 分数主要用作评价指标.

10.4.3 滑动窗口

一种进行目标检测和目标定位的方法是, 首先创建一个训练集, 其中既包括所要检测对象的紧密裁剪的样本, 也包括不包含任何对象 (“背景” 类别) 的图像类似裁剪区域的样本. 这个训练集用于训练分类器, 例如一个深度 CNN, 其输出表示输入窗口中存在特定对象类别的概率. 然后, 训练好的模型通过在图像上 “扫描” 输入窗口来检测新图像中的对象, 并针对每个位置, 将图像的结果集合用作分类器的输入. 这称为滑动窗口 (sliding window). 当以较高概率检测到对象时, 窗口位置就是相应的边界框.

这种方法的一个明显缺点是, 由于图像中潜在窗口位置的数量十分庞大, 因此计算成本可能非常高. 此外, 为了适应图像中不同大小的对象, 可能需要使用不同尺度的窗口重复该过程. 我们可以通过在图像上以大于 1 像素的步幅在水平和垂直方向上移动输入窗口来节省计算成本. 然而, 使用较小步幅提高定位精度与使用较大步幅降

低计算成本之间存在权衡. 滑动窗口方法的计算成本对于简单分类器而言可能是合理的, 但对于潜在包含数百万参数的深度神经网络来说, 计算成本难以承受.

幸运的是, 神经网络的卷积结构可以大大提高效率 (Semanet et al., 2013). 我们注意到, 这种网络中的卷积层本身就涉及在输入图像中以跨步的方式滑动共享权重的特征检测器. 因此, 当使用滑动窗口通过卷积网络进行多次前向传播时, 计算中存在大量冗余, 如图 10.21 所示. 由于滑动窗口的计算结构与卷积的计算结构相似, 因此在卷积网络中高效地实现滑动窗口非常简单. 考虑图 10.22 中一个简化的卷积网络, 它包括卷积层, 后跟最大池化层, 然后是全连接层. 网络的输入图像大小为 6×6, 卷积层中的滤波器大小为 3×3, 步幅为 1, 最大池化层具有大小为 2×2、步幅为 1 的非重叠感受野. 为简便起见, 我们在每层中只显示一个通道, 但扩展到多个通道也很容易.

假设该网络是在对象的中心图像上训练的, 然后应用于 8×8 大小的更大图像, 如图 10.23 所示. 其中, 我们只是通过增大卷积层和最大池化层来扩大网络. 现在, 卷积层的大小为 6×6, 池化层的大小为 3×3, 有 4 个输出单元, 每个输出单元都有自己的 softmax 函数. 输入其中一个输出单元的权重将在这 4 个输出单元之间共享. 我们看到, 处理位于输入图像左上角的窗口位置所需的计算与处理训练阶段原始 6×6 输入的计算相同. 对于其余的窗口位置, 则只需要少量的额外计算, 如蓝色方块所示. 与简单地重复应用完整卷积网络相比, 这显著提高了效率 (见习题 10.12). 注意, 现在全连接层本身具有卷积结构.

10.4.4 跨尺度检测

除了在图像中寻找不同位置的对象之外, 还需要在不同尺度和不同的宽高比下寻找对象. 例如, 猫直立时为其绘制的边界框, 与其躺下时为其绘制的边界框具有不同的宽高比. 与使用输入窗口具有不同大小和形状的多个检测器相比, 使用固定输入窗口并制作多个具有不同水平和垂直缩放因子的输入图像副本的方式更加简单, 但是效果相同. 后者在每个图像副本上扫描输入窗口以检测对象, 并使用关联的缩放因子将边界框坐标转换回原始图像空间, 如图 10.24 所示.

10.4.5 非最大抑制

通过在图像上扫描训练好的卷积网络, 可以检测图像中同一类别对象的多个实例, 以及来自其他类别对象的实例. 然而, 这也倾向于在相似位置产生同一对象的多个检测结果, 如图 10.25 所示. 可以使用非最大抑制 (non-max suppression) 来解决这个问题. 对于每个对象类别, 非最大抑制的工作方式如下: 首先在整个图像上运行滑动窗口, 并计算每个位置存在该对象类别的概率; 然后消除所有概率低于某个阈值 (例如 0.7) 的边界框, 得到类似于图 10.25 所示的结果——具有最高概率的框被认为是有效检测, 并且将相应的边界框记录作为预测结果; 接下来, 将任何与有效检测的边界框的 IoU 超过某个阈值 (例如 0.5) 的其他框丢弃, 从而消除针对同一对象的多个邻近检测结果; 最后在剩余的框中, 将具有最高概率的框选为另一个有效检测, 并且重复上述消除步骤. 将这个过程持续下去, 直至所有边界框要么丢弃, 要么选为有效检测.

10.4.6 快速区域卷积神经网络

滑动窗口方法的一个缺点是, 如果想要非常精确地定位对象, 则需要考虑大量只有细微间隔的窗口位置, 这将使计算成本变得很高. 滑动窗口方法将深度卷积网络的全部力量应用于图像的所有区域, 即使一些区域不太可能包含对象. 另一种方法是应用一些计算成本较低的技术, 例如分割算法, 来识别图像中存在对象可能性较高的区域, 然后仅对这些区域应用完整的卷积网络. 这种方法称为带有 CNN 的快速区域提议 (fast Region proposals with CNN, fast R-CNN)(Girshick, 2015). 也可以使用区域提议 (region proposal) 卷积网络来识别最有可能的区域, 这种方法称为 faster R-CNN(Ren et al., 2015), 该方法允许对区域提议网络和检测定位网络进行端到端训练.

10.5 图像分割

在图像分类问题中, 整个图像被分配了单一类别标签. 我们已经看到, 如果检测多个对象并使用边界框记录它们的位置, 则可以提供更详细的信息 (参见 10.4 节). 语义分割 (semantic segmentation) 可以让我们获得更详细的分析, 其中图像的每个像素被分配给预设的类别之一. 这意味着输出空间的维度与输入图像的维度相同, 因此可以方便地表示为具有相同像素数量的图像. 尽管输入图像通常具有 R、G、B 三个通道, 但输出数组具有 C 个通道 (假设有 C 个类别) 以表示每个类别的概率. 如果我们将每个类别与不同的 (任意选择的) 颜色关联起来, 那么分割网络的预测可以表示为一幅图像, 其中每个像素根据具有最高概率的类别着色, 如图 10.26 所示.

10.5.1 卷积分割

一种解决语义分割问题的简单方法是构建一个卷积网络, 该网络将图像上以某个像素为中心的矩形部分作为输入, 并借助单一 softmax 输出对该像素进行分类. 通过将这样的网络轮流应用于每个像素, 便可以分割整个图像 (这需要根据输入窗口的大小在图像周围进行边缘填充). 然而重叠部分会导致冗余计算, 使得这种方法十分低效 (参见图 10.21). 我们可以通过将不同输入位置的前向传播计算组合输入单一网络来消除这种低效性, 得到一个最终的全连接层也是卷积层的模型 (参见图 10.23). 因此, 我们可以构建一个在每层都使用步幅为 1、做了等大填充且没有池化的 CNN, 从而使得每一层的维度都与输入图像的维度相同. 每个输出单元都具有 softmax 激活函数, 权重在所有输出之间共享. 虽然可能有效, 但这样的网络仍然需要许多层, 且每层中有多个通道, 以学习实现高精度所需的复杂内部表示. 如果这样做, 即使对于常规分辨率的图像, 计算成本也是难以承受的.

10.5.2 上采样

大多数卷积网络使用多层下采样, 从而在通道数量增加的同时, 使特征图的大小减小, 以及在确保网络整体大小和计算成本可控的同时, 允许网络从图像中提取高阶语义特征. 我们可以利用这个概念来创建一个更高效的语义分割架构. 该架构采用标准的深度卷积网络, 并添加了一些额外的可学习层, 这些额外的可学习层用于将低维的内部表示转换回原始图像分辨率 (Long, Shelhamer, and Darrell, 2014; Noh, Hong, and Han, 2015; Badrinarayanan, Kendall, and Cipolla, 2015), 如图 10.27 所示.

为了做到这一点, 我们需要一种方法来逆转跨步卷积和池化操作的下采样效果. 首先考虑类比于池化的上采样操作, 其中输出层的单元数比输入层的单元数多, 例如每个输入单元对应一个 2×2 的输出单元块. 接下来的问题是输出应该使用什么值. 为了找到类比于平均池化的上采样操作, 我们可以简单地将每个输入值复制到所有对应输出单元中, 如图 10.28(a) 所示. 可以看到, 对这个操作的输出应用平均池化即可重新生成输入.

对于最大池化, 我们可以考虑图 10.28(b) 中的操作, 其中将每个输入值复制到相应输出块的第一个单元中, 并将每个输出块中的其余值置零. 同样, 我们看到对输出层应用最大池化操作会重新得到输入层. 这有时称为最大上采样 (max-unpooling). 将非零值分配给输出块的第一个单元似乎是任意的, 因此可以使用一种保留了更多来自下采样层的空间信息的改进方法 (Badrinarayanan, Kendall, and Cipolla, 2015). 这是通过设计一种网络架构来实现的, 该架构在每个最大池化下采样层之后都有一个相应的上采样层. 在下采样过程中, 记录每个输出块中的哪个单元具有最大值, 接着在相应的上采样层中, 将非零元素设置到相同的位置, 如图 10.29 中的 2×2 最大池化所示.

前面讨论的上采样操作都是固定的函数, 类似于平均池化和最大池化的下采样操作. 我们也可以使用一种称为学习到的 (learned) 上采样操作, 类比于用于下采样的跨步卷积. 在跨步卷积中, 输出图上的每个单元通过共享可学习权重与输入图上的一小块相连. 当我们在输出数组中移动一步时, 滤波器在输入数组上将移动两步或更多步, 因此输出数组的维度小于输入数组的维度. 对于上采样, 我们使用一个滤波器将输入数组中的一个像素连接到输出数组中的一小块. 然后选择架构, 使得当在输入数组中移动一步时, 在输出数组中移动两步或更多步 (Dumoulin and Visin, 2016). 图 10.30 说明了应用 3×3 滤波器和输出步幅为 2 的情况. 注意, 存在多个滤波器位置重合的输出单元, 在这种情况下, 相应的输出值可以通过对来自各个滤波器位置的贡献进行求和或求平均来计算. 这种上采样称为转置卷积 (transpose convolution), 因为如果将下采样卷积表示为矩阵形式, 则相应的上采样卷积可以由转置矩阵给出 (见习题 10.13). 注意, 有时也称这种上采样为 “反卷积”, 但最好避免使用这个术语, 因为在数学中 “反

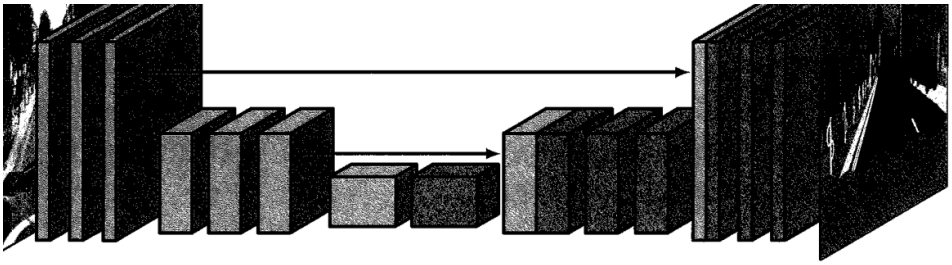
卷积”通常表示泛函分析中卷积的逆操作, 概念完全不同. 转置卷积又称“分数跨步卷积”, 因为标准卷积的步幅是输出层步长与输入层步长的比值.

10.5.3 全卷积网络

如果我们的网络架构没有池化层, 即下采样和上采样纯粹使用卷积来完成, 则称这样的网络为全卷积网络 (fully convolutional network)(Long, Shelhamer, and Darrell, 2014). 全卷积网络可以接收任意大小的图像, 并输出相同大小的分割图. 这可以看作 3×3 卷积的逆运算.

10.5.4 U-Net 架构

我们已经看到, 与跨步卷积和池化相关的下采样允许增加通道的数量, 而不会使网络的规模变得过大. 但这也会降低空间分辨率, 导致信号流过网络时丢失位置信息. 虽然这对于图像分类来说是可以接受的, 但对于图像语义分割来说, 丢失空间信息是一个大问题, 因为我们想要分类每个像素. 解决这个问题的一种方法是使用 U-Net 架构 (Ronneberger, Fischer, and Brox, 2015), 如图 10.31 所示, 其名称来自示意图所示的 U 形结构. 其核心概念是, 对于每个下采样层, 都有一个对应的上采样层, 并且每个下采样层的输出被拼接到相应的上采样层, 从而使这些上采样层能够获得更高分辨率的空间信息. 注意, 可以在 U-Net 架构的最后一层使用 1×1 卷积将通道数量减少至类别数量, 然后接一个 softmax 激活函数.



10.31 U-Net 架构具有下采样层和上采样层的对称排列结构，每个下采样层的输出被拼接到相应的上采样层

图 10.2: U-Net 架构具有下采样层和上采样层的对称排列结构, 每个下采样层的输出被拼接到相应的上采样层.

10.6 风格迁移

深度卷积网络中的早期层学习检测边缘和纹理等简单特征, 而后续层则学习检测更复杂的对象等实体 (参见 10.3 节). 我们可以利用这一性质, 使用一种称为神经风格迁移 (neural style transfer) 的过程来以另一种图像的风格重新渲染图像 (Gatys, Ecker, and Bethge, 2015), 如图 10.32 所示.

我们的目标是生成合成图像 G, 其“内容”由图像 C 定义, 其“风格”来自另一幅图像 S. 这可以通过定义由以下两项的和组成的误差函数 $E(G)$ 来实现, 其中一项促使 G 具有与 C 相似的内容, 另一项则促使 G 具有与 S 相似的风格:

$$E(G) = E_{\text{content}}(G, C) + E_{\text{style}}(G, S) \tag{10.13}$$

图像内容和风格的概念由这两项的函数形式隐含定义. 然后, 我们可以通过从随机初始化的图像开始, 并使用梯度下降最小化 $E(G)$ 来得到 G.

为了定义 $E_{\text{content}}(G, C)$, 我们可以选择网络中特定的卷积层, 并将图像 G 用作输入, 计算该层中单元的激活, 对图像 C 也执行相同的操作. 然后我们可以促使相应的预激活值变得相似, 方法是使用如下形式的平方和误差函数:

$$E_{\text{content}}(G, C) = \sum_{i,j,k} \left(a_{ij}^k(G) - a_{ij}^k(C)\right)^2 \tag{10.14}$$

其中 $a_{ij}^k(G)$ 表示当输入图像为 G 时, 该层中通道 k 位置 (i, j) 处单元的预激活值; 类似地,$a_{ij}^k(C)$ 表示当输入图像为 C 时, 该层中通道 k 位置 (i, j) 处单元的预激活值. 选择哪层用于定义预激活会影响最终结果, 早期层旨在匹配边缘等低级特征, 后续层则旨在匹配更复杂的结构甚至整个对象.

在定义 $E_{\text{style}}(G, S)$ 时, 直觉上图像风格是由卷积层中不同通道的特征共现来确定的. 在定义 E_{style} 时, 图像 S 中的垂直边缘通常与橙色斑块相关联, 那么我们希望生成的图像 G 也是如此. 虽然 $E_{\text{content}}(G, C)$ 尝试匹配 G 的特征与 C 中相同位置的相应特征, 但 $E_{\text{style}}(G, S)$ 使用通道之间相关性的 Gram 矩阵来度量风格, 从而与空间位置无关.

第 11 章 结构化分布

11.1 概率图模型

我们已经看到, 概率是深度学习最重要的基础概念之一. 例如, 用于二元分类的神经网络可以用一个条件概率分布来描述, 其形式为

$$p(t \mid \mathbf{x}, \mathbf{w}) = y(\mathbf{x}, \mathbf{w})^t \left(1 - y(\mathbf{x}, \mathbf{w})\right)^{1-t} \tag{11.1}$$

其中,$y(\mathbf{x}, \mathbf{w})$ 表示神经网络函数, 它以向量 $\mathbf{x}$ 作为输入, 并受可学习参数向量 $\mathbf{w}$ 的控制. 相应的交叉熵似然是构建用于训练神经网络的误差函数的基础. 虽然神经网络函数可能极其复杂, 但式 (11.1)中的条件分布形式却很简单. 有许多重要的深度学习模型都具有更丰富的概率结构, 如大语言模型、归一化流、变分自编码器、扩散模型等. 为了描述和利用这种结构, 我们引入了一个强大的框架, 称为概率图模型 (probabilistic graphical model) 或简称图模型 (graphical model), 它允许以图的形式表达结构化概率分布. 当与神经网络相结合来定义相关的概率分布时, 图模型为创建可以端到端训练的复杂模型提供了巨大的灵活性, 其中梯度可通过自动微分来进行有效计算, 并使用随机梯度下降进行训练. 本章将重点讨论应用深度学习所需的概率图模型的核心概念, 而对机器学习的概率图模型更为全面的讨论可参阅 Bishop (2006).

概率可以用两个简单的法则来表示, 即加和法则和乘积法则 (参见 2.1 节). 本书讨论的所有概率操作, 无论多么复杂, 都相当于重复应用这两个法则. 因此, 原则上我们可以纯粹通过代数操作来制定和使用复杂的概率模型. 不过, 我们发现使用概率分布的图示法来扩充分析是有好处的, 因为图示法具有以下 3 个有用的特性:

1. 它们提供了一种可视化概率模型结构的简单方法, 可用于设计和构思新的模型;
2. 通过对图进行检查, 可以深入了解模型的特性, 包括条件独立性;
3. 在复杂模型中进行推理和学习所需的复杂计算可以用图形操作来表示, 如消除消息传递, 其中隐式地进行了基本的数学运算.

11.1.1 有向图

图由节点 (也称顶点) 构成, 这些节点由边来连接. 在图模型中, 每个节点代表一个随机变量, 而连接则表示这些变量之间的概率关系. 然后, 图就可以记录所有随机变量的联合分布是如何分解成多个因子 (每个因子都仅依赖变量的一个子集) 的乘积的. 本章将重点讨论图模型, 在图模型中, 图的连接具有箭头指示的特定方向. 这些模型称为有向图模型 (directed graphical model), 也称贝叶斯网络. 有向图适用于表达随机变量之间的因果关系, 而无向图更适合表达随机变量之间的软约束. 图模型的另一大类是马尔可夫随机场 (Markov random field), 也称无向图模型 (undirected graphical model), 其中的连接不带箭头, 无方向意义. 有向图和无向图都可以看作一种叫作因子图 (factor graph) 表示法的特例. 接下来, 我们将重点关注有向图. 不过, 请注意, 在我们讨论图神经网络 (Graph Neural Network, GNN)(参见第 13 章) 时 (其中节点代表标准神经网络中的确定性变量), 也会出现无概率解释的无向图.

虽然这种概率图模型的节点和边与神经网络图很相似, 但它们具有特别的概率性解释, 并含有更丰富的语义. 为了避免混淆, 本书用蓝色表示神经网络图, 用红色表示概率图模型 (后文简称图模型).

11.1.2 分解

为了有利于使用有向图描述概率分布, 首先考虑关于 3 个变量 a、b 和 c 的任意联合分布 $p(a, b, c)$. 请注意, 在这个阶段, 我们不需要进一步说明它们都是些什么变量, 比如它们是离散变量还是连续变量. 事实上, 图模型的一个强大之处在于, 一个特定的图可以为多个分布类别做出概率说明. 通过应用概率的乘积法则 (式 (2.9)), 我们可以写下联合分布, 形式如下:

$$p(a,b,c) = p(c \mid a,b)\, p(a,b) \tag{11.2}$$

再次应用乘积法则对式 (11.2)右边的第二项进行计算, 得到

$$p(a,b,c) = p(c \mid a,b)\, p(b \mid a)\, p(a) \tag{11.3}$$

请注意, 无论选择何种联合分布, 以上分解都是成立的. 下面让我们用一个简单的图模型来表示式 (11.3)的右侧, 如图 11.1 所示. 首先为随机变量 a、b 和 c 分别引入一个节点, 并将每个节点与式 (11.3)右侧相应的条件分布联系起来. 然后为每个条件分布添加有向连接 (如箭头所示), 这些连接来自与分布条件变量相对应的节点. 因此, 对于因子 $p(c \mid a,b)$, 将存在从节点 a 和 b 到节点 c 的连接; 而对于因子 $p(a)$, 则不存在传入连接. 如果节点 a 和节点 b 之间存在一个连接, 那么节点 a 是节点 b 的父节点, 相应地, 节点 b 是节点 a 的子节点. 请注意, 我们不会对节点以及节点所对应的变量进行任何形式上的区分, 而只是使用相同的符号指代两者.



$c_1, \cdots, x_7$ 联

1.5) 给出

图 11.1: 一个描述变量 $x_1, \cdots, x_7$ 联合分布的有向图, 对应于式 (11.5)的分解.

对于式 (11.3), 重要的一点是, 它的左侧变量与变量 a、b、c 是对称的, 而右侧变量则不对称. 在对式 (11.3)进行分解时, 我们隐式地选择了一种特定的排序, 即 a、b、c; 如果我们选择不同的排序, 就会得到不同的分解, 从而得到不同的图形化表示.

下面让我们通过 $p(x_1, \cdots, x_K)$ 给出的 K 个变量的联合分布来扩展图 11.1 中的示例. 通过反复应用乘积法则, 该联合分布可以写成条件分布的乘积,K 个变量中的每一个变量都有一个条件分布:

$$p(x_1, \cdots, x_K) = p(x_K \mid x_1, \cdots, x_{K-1}) \cdots p(x_2 \mid x_1)\, p(x_1) \tag{11.4}$$

对于给定的 K, 我们可以再次将这表示为一个有向图, 其中有 K 个节点. 式 (11.4)右侧的每个条件分布都有一

个节点, 且每个节点都有来自所有较低编号节点的输入连接. 我们称这个图是全连接的 (fully connected), 因为每一对节点之间都有一个连接.

到目前为止, 我们使用的都是完全通用的联合分布, 因此它们的分解和相关的全连接图表示适用于任何分布. 我们很快就会发现, 正是图中没有 (absence) 的连接传达了关于图所代表的分布类属性的有趣信息. 参见图 11.2 所示的有向图. 请注意, 这并不是一个全连接图, 因为从节点 x_1 到 x_3 或者从节点 x_3 到 x_4 并不存在连接. 从这个有向图中提取出联合 (概率) 分布的相应表示, 即图 11.2 给出了联合分布的相应分解. 例如, 节点 x_6 将以节点 x_2 和 x_4 为条件. 因此, 所有 7 个变量的联合分布为

$$p(x_1,\cdots,x_7) = p(x_1)\,p(x_2)\,p(x_3)\,p(x_4 \mid x_1,x_2,x_3)\,p(x_5 \mid x_1)\,p(x_6 \mid x_4)\,p(x_7 \mid x_4,x_5) \tag{11.5}$$

我们现在所讨论的有向图有一个重要的限制条件, 即禁止存在有向环 (directed cycle). 换句话说, 图中不存在闭环路径, 这意味着我们不能沿着箭头的方向从一个节点移动到另一个节点, 最后回到起始节点. 这种图也称有向无环图, 或简称 DAG(Directed Acyclic Graph). 这表示存在一种节点排序, 使得不存在从任何节点到任何较低编号节点的连接 (见习题 11.2).

现在, 我们可以用一般性术语来说明给定有向图与相应变量分布之间的关系. 图定义的联合分布由图中所有节点的条件分布的乘积给出, 每个节点的条件分布以图中该节点的父节点所对应的变量为条件. 因此, 对于一个有 K 个节点的图, 联合分布的计算公式为

$$p(\mathbf{x}) = \prod_{k=1}^{K} p\big(x_k \mid \mathrm{pa}(x_k)\big) \tag{11.6}$$

其中,$p\big(\mathrm{pa}(x_k)\big)$ 表示节点 x_k 的父节点集. 这个关键公式表达了有向图模型联合分布的分解特性. 虽然每个节点对应一个变量, 但我们同样可以将变量集、向量值变量或张量值变量与图中的节点联系起来. 不难看出, 只要各个条件分布被归一化, 式 (11.6)右侧的表示就总是能被正确地归一化 (见习题 11.1). 建议读者花点时间仔细研究一下式 (11.5)与图 11.2 之间的对应关系.

11.1.3 离散变量

我们已经讨论过作为指数族分布成员的概率分布的重要性, 并看到指数族分布中包括许多知名分布的特例 (参见 3.4 节). 虽然这些分布相对简单, 但它们构成了构建更复杂概率分布的有用构件, 而图模型非常有助于表达这些构件是如何相互连接在一起的. 对于广泛使用的组件分布, 有两种特殊的选择, 分别对应于离散变量和高斯变量. 我们先来看看离散变量.

具有 K 种可能状态 (使用 1-of-K 表示法) 的单个离散变量 x 的概率分布 $p(x \mid \boldsymbol{\mu})$ 由下式给出并由参数 $\boldsymbol{\mu} = (\mu_1,\cdots,\mu_K)$ 控制:

$$p(x \mid \boldsymbol{\mu}) = \prod_{k=1}^{K} \mu_k^{x_k} \tag{11.7}$$

由于约束 $\sum_k \mu_k = 1$, 因此只需要指定 $\boldsymbol{\mu}$ 的 $K-1$ 个值即可定义分布. 假设有两个离散变量 x_1 和 x_2, 其中每个变量都有 K 种状态, 希望模拟它们的联合分布. 用参数 μ_{ij} 表示同时观测到 $x_1 = i$ 和 $x_2 = j$ 的概率, 其中 x_{1i} 表示 x_1 的第 i 个组件,x_{2j} 表示 x_2 的第 j 个组件. 联合分布可以写为 $p(x_1,x_2 \mid \boldsymbol{\mu}) = \prod_{i,j} \mu_{ij}^{x_{1i}x_{2j}}$. 由于参数 μ_{ij} 受 $\sum_{i,j} \mu_{ij} = 1$ 的约束, 因此以上联合分布受 K^2-1 个参数的控制.

利用乘积法则, 我们可以将联合分布 $p(x_1,x_2)$ 分解为 $p(x_2 \mid x_1)\,p(x_1)$ 的形式, 对应于一个两节点图, 其中有一个从节点 x_1 到节点 x_2 的连接, 如图 11.3(a) 所示. 边缘分布 $p(x_1)$ 和之前一样受 $K-1$ 个参数的控制. 同样, 条件分布 $p(x_2 \mid x_1)$ 需要为 x_2 的 K 个可能取值中的每个值指定 $K-1$ 个参数. 因此, 在联合分布中必须指定的参数总数依然是 $(K-1)+K(K-1) = K^2-1$.

假设变量 x_1 和 x_2 是独立的, 对应于图 11.3(b) 所示的图模型. 然后, 将每个变量都用一个独立的离散分布来描述, 参数总数为 $2(K-1)$. 对于 M 个独立离散变量的分布 (每个变量有 K 个状态), 参数总数为 $M(K-1)$, 因此参数总数随着变量数量的增加而呈线性增长. 从图的角度来看, 我们已经通过放弃图中的连接来减少参数的数量, 但这样做的代价是分布的类更加有限了.

更一般地说, 如果有 M 个离散变量 $x_1, \cdots, x_M$, 则可以使用有向图对联合分布进行建模, 每个节点有一个变量. 每个节点上的条件分布由一组非负参数给出, 这些参数受限于归一化约束. 如果图是全连接的, 那么我们就有了一个完全通用的分布, 该分布有 $K^M - 1$ 个参数; 而如果图中没有连接, 联合分布就会分解为边缘分布的乘积, 参数总数为 $M(K-1)$. 具有中间水平连通性的图比完全分解的图更易于实现一般分布, 且比一般分布需要更少的参数. 如图 11.4 所示, 边缘分布 $p(x_1)$ 需要 $K-1$ 个参数, 而对于 $i = 2, \cdots, M$, 条件分布 $p(x_i \mid x_{i-1})$ 中的每个分布都需要 $K(K-1)$ 个参数. 因此这个具有 M 个离散节点 (每个节点有 K 个状态) 的节点链需要指定 $K-1+(M-1)K(K-1)$ 个参数, 总的参数数量为 $K-1+(M-1)K(K-1)$, 与节点链长度 M 的增加呈线性增长 (而非指数增长). 相比之下, 由 M 个节点组成的全连接图则有 $K^M - 1$ 个参数, 参数的数量随着 M 的增加量指数增长.

共享参数 (也称捆绑参数) 是减少模型中独立参数数量的另一种方法. 例如, 在图 11.4 中, 我们可以安排在 $i = 2, \cdots, M$ 的情况下, 所有条件分布 $p(x_i \mid x_{i-1})$ 都受同一组 $K(K-1)$ 个参数的控制, 从而得到图 11.5 所示的模型. 再加上控制 x_1 分布的 $K-1$ 个参数, 总共需要指定 $K^2 - 1$ 个参数来定义联合分布.

在离散变量模型中, 控制参数数量增长的另一种方法是使用参数化的条件分布表, 而非使用完整的条件概率值表. 为了阐明这一观点, 请看图 11.6 所示的模型, 其中所有节点都代表二进制变量. 每个父变量 x_i 由代表概率 $p(x_i = 1)$ 的单个参数 μ_i 控制, 其中所有节点都代表二进制变量. 每个父变量 x_i 由代表概率 $p(x_i = 1)$ 的单个参数 μ_i 控制, 因此父节点共有 M 个参数. 然而, 条件分布 $p(y \mid x_1, \cdots, x_M)$ 需要 2^M 个参数, 用于表示父变量 2^M 种可能设置中每一种设置的概率 $p(y = 1)$. 因此, 一般来说, 指定这一条件分布所需的参数数量将随着 M 的增加呈指数增长. 通过使用作用于父变量线性组合的逻辑斯谛 sigmoid 函数 (参见 3.4 节), 我们可以得到如下更简洁的条件分布形式:

$$p(y = 1 \mid x_0, x_1, \cdots, x_M) = \sigma\left(\sum_{i=0}^{M} w_i x_i\right) = \sigma\left(\mathbf{w}^{\mathsf{T}}\mathbf{x}\right) \tag{11.8}$$

其中,$\sigma(a) = (1 + \exp(-a))^{-1}$ 是逻辑斯谛 sigmoid 函数,$\mathbf{x} = (x_0, x_1, \cdots, x_M)$ 是一个 $(M+1)$ 维的父状态向量, 其中增加了一个变量 x_0, 其值限制为 1, 而 $\mathbf{w} = (w_0, w_1, \cdots, w_M)$ 是一个包含 $M+1$ 个参数的向量. 与一般情况相比, 这种条件分布的形式更受限制, 但它现在受一些随 M 的增加呈线性增长的参数的控制. 从这个意义上讲, 这类似于在多元高斯分布中选择协方差矩阵的限制形式 (例如对角矩阵).

11.1.4 高斯变量

让我们回到图模型, 其中节点代表具有高斯分布的连续变量. 每个分布都以图中父节点的状态为条件. 这种依赖关系可以有多种形式, 这里我们重点讨论一种特定的选择, 即每个高斯分布的均值是高斯父变量状态的某个线性函数. 这就产生了另一类模型——线性高斯模型, 其中包括许多具有实际意义的案例, 如概率主成分分析 (见 16.2 节)、因子分析和线性动态系统等 (Roweis and Ghahramani, 1999).

考虑 D 个变量上的任意一个有向无环图, 其中节点 i 代表具有高斯分布的单个连续随机变量 x_i. 该高斯分布的均值是节点 i 的父节点 $\mathrm{pa}(i)$ 状态的线性组合:

$$p\left(x_i \mid \mathrm{pa}(i)\right) = \mathcal{N}\left(x_i \;\middle|\; \sum_{j \in \mathrm{pa}(i)} w_{ij} x_j + b_i,\ v_i\right) \tag{11.9}$$

其中 w_{ij} 和 b_i 是控制均值的参数,v_i 是 x_i 条件分布的方差. 联合分布的对数就是图中所有节点的这些条件式乘积的对数, 因此其形式为

$$\ln p(\mathbf{x}) = -\frac{1}{2}\sum_{i=1}^{D}\frac{\left(x_i - \sum_j w_{ij}x_j - b_i\right)^2}{v_i} + \text{const} \tag{11.10}$$

其中 $\mathbf{x} = (x_1, \cdots, x_D)$, 且 const 表示与 $\mathbf{x}$ 无关的常数项. 可以看到, 这是 $\mathbf{x}$ 的组件的二次函数, 因此联合分布 $p(\mathbf{x})$ 是多元高斯分布.

我们可以使用以下方法求出该联合分布的均值和协方差. 每个变量的均值由递推公式给出 (见习题 11.6):

$$\mathbb{E}[x_i] = \sum_j w_{ij}\,\mathbb{E}[x_j] + b_i \tag{11.11}$$

因此我们可以从最低编号的节点开始, 通过图递归地找到 $\mathbb{E}[\mathbf{x}] = (\mathbb{E}[x_1], \cdots, \mathbb{E}[x_D])$. 同样, 联合分布的协方差矩阵元素也满足以下形式的递推公式 (见习题 11.7):

$$\text{cov}[x_i, x_j] = \sum_k w_{ik}\,\text{cov}[x_k, x_j] + \delta_{ij}v_i \tag{11.12}$$

其中我们假设每个节点的编号大于其父节点的编号. 因此, 协方差同样可以从最低编号的节点开始递归求值.

考虑两个可能出现的图结构极端案例. 首先, 假设图中无连接, 因此图由 D 个孤立节点构成. 在这个情况下, 参数 w_{ij} 不存在, 因此只有 D 个参数 b_i 和 D 个参数 v_i. 从递推公式 (11.11)和 (11.12)中可以看出,$p(\mathbf{x})$ 的均值为 $(b_1, \cdots, b_D)$, 协方差矩阵为 $\text{diag}(v_1, \cdots, v_D)$ 形式的对角矩阵. 联合分布共有 $2D$ 个参数且代表一组 D 个独立的单变量高斯分布. 其次, 考虑一个全连接图, 其中每个节点都将所有较低编号的节点作为其父节点. 在这种情况下, 协方差矩阵中独立参数 w_{ij} 和 v_i 的总数为 $D(D+1)/2$, 对应于一般的对称协方差 (见习题 11.8).

具有某种中等复杂程度的图对应于具有部分约束协方差矩阵的联合高斯分布. 例如, 图 11.7 所示的图在高斯变量 x_1 和 x_3 之间缺少一个连接. 利用递推公式 (11.11)和式 (11.12), 我们可以得出联合分布的均值和协方差为 (见习题 11.9)

$$\boldsymbol{\mu} = (b_1,\ b_2 + W_{21}b_1,\ b_3 + W_{31}b_1 + W_{32}(b_2 + W_{21}b_1)) \tag{11.13}$$

我们很容易将线性高斯图模型扩展到图节点代表多元高斯变量的情况. 在这种情况下, 我们可以写下节点 i 的条件分布, 形式如下:

$$p\left(\mathbf{x}_i \mid \text{pa}(i)\right) = \mathcal{N}\left(\mathbf{x}_i \;\middle|\; \sum_{j\in\text{pa}(i)} \mathbf{W}_{ij}\mathbf{x}_j + \mathbf{b}_i,\ \mathbf{V}_i\right) \tag{11.14}$$

其中 $\mathbf{W}_{ij}$ 是一个矩阵 (如果 $\mathbf{x}_i$ 和 $\mathbf{x}_j$ 的维度不同, 则这个矩阵是非正方形的). 同样, 我们很容易验证所有变量的联合分布是高斯分布 (见习题 11.10).

11.1.5 二元分类器

下面我们用一个二元分类器来说明如何使用有向图对概率分布进行描述, 该模型的可学习参数具有高斯先验 (参见 2.6.2 小节). 我们可以将其写成如下形式:

$$p(\mathbf{t}, \mathbf{w} \mid \mathbf{X}, \lambda) = p(\mathbf{w} \mid \lambda)\prod_{n=1}^{N} p(t_n \mid \mathbf{x}_n, \mathbf{w}) \tag{11.15}$$

其中,$\mathbf{t} = (t_1, \cdots, t_N)$ 是目标值的向量,$\mathbf{X}$ 是行 $\mathbf{x}_1, \cdots, \mathbf{x}_N$ 的数据矩阵. 分布 $p(t_n \mid \mathbf{x}_n, \mathbf{w})$ 由式 (11.1)给出. 我们还假设参数向量 $\mathbf{w}$ 的高斯先验为

$$p(\mathbf{w} \mid \lambda) = \mathcal{N}(\mathbf{w} \mid \mathbf{0}, \lambda^{-1}\mathbf{I}) \tag{11.16}$$

该模型中的随机变量为 $\{t_1, \cdots, t_N\}$ 和 $\mathbf{w}$. 此外, 该模型还包含噪声方差 σ^2 和超参数 λ, 它们是模型的参数而非随机变量. 如果我们暂时只考虑随机变量, 那么式 (11.15)给出的分布可以用图 11.8 所示的图模型来表示.

当开始处理更复杂的模型时, 要像图 11.8 那样明确写出 $t_1, \cdots, t_N$ 形式的多个节点就会变得很不方便. 因此, 我们引入了一种图形符号, 以便更简洁地表示这样的多个节点. 绘制一个具有代表性的节点, 然后在它的周围用一个方框 (称为板块) 将其包围, 并标为 N 以表示有 N 个此类节点. 用此方式修改图 11.8, 便可得到图 11.9.

11.1.6 参数和观测值

有时我们会发现, 用图形化表示法明确表示模型的参数及随机变量是很有帮助的. 为此, 我们将遵照以下惯例: 随机变量用开圆表示, 确定性参数用浮动变量表示. 如果我们采用图 11.9 所示的图模型并加入确定性参数, 即可得到图 11.10 所示的图模型.

我们有 3 种变量. 首先, 存在未观测到 (也称潜在或隐藏的) 的随机变量, 用红色圆圈表示. 其次, 当观测到随机变量并将其设置为特定值时, 用蓝色阴影的红圈表示. 最后, 如图 11.11 所示, 非随机参数用浮动变量表示. 请注意,$\mathbf{w}$ 的值是无法观测到的, 因此 $\mathbf{w}$ 是一个潜 (latent) 变量 (也称隐变量). 这些变量在本书讨论的许多模型中都发挥了至关重要的作用.

当我们将图模型应用于机器学习问题时, 通常会将一些随机变量设置为特定的观测值. 例如, 线性回归模型中的随机变量 t_n 将设置为等于训练集中给出的特定值. 在图模型中, 可通过给相应的节点添加阴影来表示这些观测变量. 因此, 与图 11.10 所示的图模型 (其中变量 t_n 被观测) 相对应的图模型如图 11.11 所示.

11.1.7 贝叶斯定理

当概率模型中的随机变量设定为与观测值相等时, 其他未观测到的随机变量的分布也会相应地改变. 计算这些更新分布的过程称为推理 (inference). 我们可以通过贝叶斯定理的图形解释来说明这一点. 假设我们把两个变量 x 和 y 的联合分布 $p(x, y)$ 分解成因子的乘积, 形式为 $p(x, y) = p(x)p(y \mid x)$. 这可以用图 11.13(a) 中的有向图来表示. 假设我们观测到了 y 的值, 如图 11.13(b) 中的阴影节点所示. 我们可以把边缘分布 $p(x)$ 看作潜变量 x 的一个先验概率, 而我们的目标就是推断出相应的后验概率. 利用加和法则与乘积法则, 我们可以求出边缘分布 $p(y)$:

$$p(y) = \sum_x p(y \mid x)\, p(x) \tag{11.17}$$

然后可以用贝叶斯定理计算出

$$p(x \mid y) = \frac{p(y \mid x)\, p(x)}{p(y)} \tag{11.18}$$

因此, 联合分布现在可以用 $p(x \mid y)$ 和 $p(y)$ 来表示. 从图的角度看, 联合分布 $p(x, y)$ 可以用图 11.13(c) 所示的图来表示, 其中箭头的方向是相反的. 这是图模型推理问题的最简单示例.

对于捕获到丰富概率结构的复杂图模型来说, 一旦观测到某些随机变量, 计算后验分布的过程就会变得复杂而巧妙. 从概念上讲, 它只涉及系统地应用概率的加和法则与乘积法则, 或等同于贝叶斯定理. 然而在实践中, 利用图结构可以大大提高计算和管理的效率. 这些计算可以用图 (涉及在节点间发送本地信息) 上优雅的计算来表示. 这些方法不仅对树状结构的图给出了精确答案, 也对有环图给出了近似迭代算法. 我们不会在此进一步讨论这些问题, 感兴趣的读者请参阅 Bishop (2006), 以了解机器学习方面更全面的探讨.

请注意, 由于我们最终的目标是对新的输入值进行预测, 因此诸如 $\mathbf{w}$ 的模型参数一般不会与它们自身有直接关系. 假设我们得到了一个新的输入值 $\hat{\mathbf{x}}$, 并希望以观测到的数据为条件找到的相应概率分布. 以确定性参数为条件, 该模型中所有随机变量的联合分布由式 (11.15)给出. 相应的图模型如图 11.12 所示. 然后, 通过对模型参数 $\mathbf{w}$ 进行积分, 就能从加和法则中得到 $\hat{t}$ 所需的预测分布. 这种对参数的积分是一种完全贝叶斯式的处理方法, 在实践中很少使用, 尤其是在深度神经网络中. 相反, 我们通常首先找到能使后验分布最大化的最可能值 $\mathbf{w}_{\text{MAP}}$ 来近似这个积分, 然后仅使用这个单一值并应用 $p(\hat{t} \mid \hat{\mathbf{x}}, \mathbf{w}_{\text{MAP}})$ 进行预测.

11.2 条件独立性

多变量概率分布的一个重要概念是条件独立性 (Dawid, 1979). 考虑 3 个变量 a、b、c, 假设在给定 b 和 c 的条件下,a 的条件分布不依赖于 b 的值, 即

$$p(a \mid b, c) = p(a \mid c) \tag{11.19}$$

我们称之为, 在给定 c 的条件下,a 与 b 是条件独立的. 如果我们考虑以 c 为条件的 a 和 b 的联合分布, 就可以用稍微不同的方式来表示, 形式如下:

$$p(a, b \mid c) = p(a \mid b, c)\, p(b \mid c) = p(a \mid c)\, p(b \mid c) \tag{11.20}$$

其中利用了概率的乘积法则和式 (11.19). 我们看到, 以 c 为条件的 a 和 b 的联合分布可以分解为 a 的边缘分布和 b 的边缘分布的乘积 (后者同样都以 c 为条件). 也就是说, 在给定 c 的条件下, 变量 a 和 b 在统计上是独立的. 请注意, 我们对条件独立性的定义要求式 (11.19)或式 (11.20)必须对 c 的每一个可能值都成立, 而不仅仅对 c 的某些值成立. 我们有时也会使用条件独立性的速记符号 (Dawid, 1979), 其中

$$a \perp b \mid c \tag{11.21}$$

表示在给定 c 的条件下,a 和 b 是条件独立的. 条件独立性在机器学习的概率模型中发挥着重要作用, 因为它既简化了模型的结构, 也简化了在模型下执行推理和学习所需的计算.

如果能用条件分布的乘积 (即有向图的数学表达式) 得出一组变量的联合分布表达式, 则原则上可以通过反复应用概率的加和法则与乘积法则来检验任何潜在的条件独立性是否成立. 实际上, 该方法十分耗时. 图模型的一个重要而优雅的特点是, 可以直接从图中读取联合分布的条件独立性, 而无须执行任何分析操作. 实现此过程的一般框架称为 d 分离 (d-separation), 其中“d”代表“有向”(Pearl, 1988). 本节将介绍 d 分离的概念并给出 d 分离准则的一般说明, 形式证法见 Lauritzen (1996).

11.2.1 3 个示例图

在开始讨论有向图的条件独立性之前, 我们先来看 3 个简单的例子, 每个例子都涉及只有 3 个节点的图. 这些将共同促进和说明 d 分离的关键概念. 第 1 个示例如图 11.14 所示, 利用一般结果 [式 (11.6)] 可以很容易地写出该图所对应的联合分布, 即

$$p(a, b \mid c) = p(a \mid c)\, p(b \mid c) \tag{11.22}$$

如果未观测到任何变量, 则可以通过将式 (11.22)的两边相对于 c 边缘化来研究 a 和 b 是否独立, 即

$$p(a, b) = \sum_{c} p(a \mid c)\, p(b \mid c)\, p(c) \tag{11.23}$$

一般来说, 这不会分解为乘积 $p(a)p(b)$, 因此

$$a \not\perp b \mid \varnothing \tag{11.24}$$

其中 $\varnothing$ 表示空集, 符号上表示条件独立性一般不成立. 当然, 对于特定的分布来说, 条件独立性也可能由于与各种条件概率相关的特定数值而成立, 但一般来说, 这并不是由图的结构得出的.

假设我们以节点 c 为条件, 如图 11.15 中的有向图所示. 根据式 (11.22), 我们可以很容易地写出在给定 c 的条件下 a 和 b 的条件分布, 其形式为

$$p(a, b \mid c) = p(a \mid c)\, p(b \mid c) \tag{11.25}$$

于是, 我们得到如下条件独立性:

$$a \perp b \mid c \tag{11.26}$$

可以考虑从节点 a 经过节点 c 到达节点 b 的路径, 从而对这一结果进行简单的图解释. 节点 c 之所以相对于该路径称为尾尾相接 (tail-to-tail) 节点, 就是因为该节点与两个箭头的尾部相连, 而连接节点 a 和 b 的路径的存在会导致这两个节点相互依赖. 然而, 如图 11.15 所示, 当我们以节点 c 为条件时, 条件节点会 “阻塞” 从 a 到 b 的路径, 并使 a 和 b 变得 (条件) 独立.

同样, 我们也可以考虑图 11.16 所示的有向图. 根据式 (11.6), 我们可以得到与该有向图相对应的联合分布, 即

$$p(a, b, c) = p(a)\, p(c \mid a)\, p(b \mid c) \tag{11.27}$$

假设未观测到任何变量. 我们可以通过对 c 进行边缘化来检验 a 和 b 是否独立,

$$p(a, b) = p(a) \sum_{c} p(c \mid a)\, p(b \mid c) = p(a)\, p(b \mid a) \tag{11.28}$$

从而得出

$$a \not\perp b \mid \varnothing \tag{11.29}$$

假设以节点 c 为条件, 如图 11.17 中的有向图所示. 利用贝叶斯定理和式 (11.27), 即可得出

$$p(a, b \mid c) = \frac{p(a)\, p(c \mid a)\, p(b \mid c)}{p(c)} = p(a \mid c)\, p(b \mid c) \tag{11.30}$$

于是, 我们再次得到如下条件独立性:

$$a \perp b \mid c \tag{11.31}$$

和以前一样, 我们可以用图形来解释这些结果. 就节点 a 到节点 b 的路径而言, 节点 c 称为头尾相接 (head-to-tail) 节点. 该路径连接了节点 a 和 b, 并使它们相互依赖. 假设我们现在观测到了节点 c, 如图 11.17 所示, 那么这个观测结果就会 “阻塞” 从节点 a 到 b 的路径, 从而得到条件独立性 $a \perp b \mid c$.

最后, 让我们来看看第 3 个示例, 如图 11.18 中的有向图所示. 利用一般结果 [式 (11.6)] 写出该图所对应的联合分布, 即

$$p(a, b, c) = p(a)\, p(b)\, p(c \mid a, b) \tag{11.32}$$

首先考虑未观测到任何变量的情况. 在 c 上对式 (11.32)的两边进行边缘化, 得到

$$p(a, b) = p(a)\, p(b) \tag{11.33}$$

因此, 与前两个例子所不同的是,a 和 b 是独立的, 且未观测到任何变量. 我们可以将这一结果写成

$$a \perp b \mid \varnothing \tag{11.34}$$

假设我们以节点 c 为条件, 如图 11.19 所示, 则 a 和 b 的条件分布如下:

$$p(a, b \mid c) = \frac{p(a)\, p(b)\, p(c \mid a, b)}{p(c)} \tag{11.35}$$

一般来说, 这不会分解为乘积 $p(a \mid c)\, p(b \mid c)$, 因此

$$a \not\perp b \mid c \tag{11.36}$$

第 3 个示例的行为与前两个示例相反. 观察图 11.18, 节点 c 相对于从节点 a 到节点 b 的路径是头头相接 (head-to-head) 的. 节点 c 有时也称碰撞节点. 当未观测到节点 c 时, 就会 “阻塞” 路径, 且导致 a 和 b 是独立的. 然而, 以 c 为条件会 “疏通” 路径, 并使 a 和 b 相互依赖.

11.2.2 相消解释

我们有必要花点时间进一步了解图 11.19 中的有向图的异常行为. 如图 11.20 所示, 考虑这样一个图, 该图对应的问题与汽车燃油系统有关的 3 个二进制随机变量. 其中, 变量 B 代表电池的状态, 即满电 ($B = 1$) 或没电 ($B = 0$); 变量 F 代表油箱的状态, 即满油 ($F = 1$) 或没油 ($F = 0$); 变量 G 代表电动燃油表的读数,$G = 1$ 表示油箱满油,$G = 0$ 表示油箱没油, 电池要么满电要么没电, 油箱要么满油要么没油, 且先验概率为

$$p(F = 1) = 0.9, \qquad p(B = 1) = 0.9 \tag{11.37}$$

在给定油箱状态和电池状态的情况下, 电动燃油表读数为 1 的概率为

$$p(G = 1 \mid B = 0, F = 0) = 0.1, \quad p(G = 1 \mid B = 0, F = 1) = 0.2, \quad p(G = 1 \mid B = 1, F = 0) = 0.2, \quad p(G = 1 \mid B = 1, F = 1) = 0.8 \tag{11.38}$$

可见这是一个相当不靠谱的燃油表! 因为剩下的所有概率都由概率和为 1 这一要求决定.

在我们观测到任何数据之前, 油箱为空的先验概率为 $p(F = 0) = 0.1$. 假设我们观测燃油表并发现它的读数为零, 即 $G = 0$, 与图 11.20 的中间图相对应. 我们可以使用贝叶斯定理来计算油箱为空的后验概率. 首先计算

$$p(G = 0) = \sum_{B} \sum_{F} p(G = 0 \mid B, F)\, p(B)\, p(F) = 0.315 \tag{11.39}$$

然后计算

$$p(F = 0 \mid G = 0) = \frac{p(G = 0 \mid F = 0)\, p(F = 0)}{p(G = 0)} \approx 0.257 \tag{11.40}$$

所以 $p(F = 0 \mid G = 0) > p(F = 0)$, 从而只要测到燃油表读数为零, 就更有可能确定油箱确实是空的, 正如我们潜意识里所认为的那样. 接下来, 假设我们还检查了电池的状态并发现电池没电, 即 $B = 0$. 如图 11.20 的右图所示, 我们现在已经观测到燃油表和电池的状态. 根据对燃油表状态和电池状态的观测, 油箱为空的后验概率为

$$p(F = 0 \mid G = 0, B = 0) = \frac{p(G = 0 \mid B = 0, F = 0)\, p(F = 0)}{p(G = 0 \mid B = 0)} \approx 0.111 \tag{11.41}$$

其中, 先验概率 $p(B = 0)$ 在式 (11.41)的分子和分母之间相互抵消. 因此, 由于观测到电池的状态, 油箱为空的概率降低了 (从 0.257 降至 0.111). 这与我们的直觉相吻合, 即电池没电可以相消解释 (explain away) 燃油表读数为零的观测结果. 通过观测燃油表的读数, 我们发现油箱和电池的状态确实是相互依赖的. 事实上, 如果我们不直接观测燃油表, 而是观测节点 G 的某个后代节点的状态, 例如一个相当不可靠的目击者说他看到燃油表的读数为零, 则情况也会如此 (见习题 11.14). 请注意, 概率 $p(F = 0 \mid G = 0, B = 0) \approx 0.111$ 大于先验概率 $p(F = 0) = 0.1$, 因为观测到燃油表读数为零仍然提供了一些支持油箱为空的证据.

总而言之, 在未观测到尾对尾节点或头对尾节点的情况下, 路径会变得畅通; 而一旦尾对尾节点或头对尾节点被观测到, 路径就会变得阻塞. 相比之下, 头对头节点在未被观测到时会阻塞路径, 而一旦头对头该节点和/或其至少一个后代被观测到, 路径就会变得畅通.

第 3 个例子还有一个微妙之处. 首先, 我们来介绍一些术语. 如果存在一条从 x 到 y 的路径, 且这条路径上的每一步都遵循箭头的方向, 则可以说节点 y 是节点 x 的后代. 然后可以证明, 如果节点或其任何后代被观测到, 则头头相接的路径就会变得畅通 (见习题 11.13).

11.2.3 d 分离

本小节将对有向图的 d 分离特性 (Pearl, 1988) 进行一般性阐述. 考虑一个有向图, 其中 **A**、**B**、**C** 是任意不相交的节点集 (但它们的并集可能小于图中的完整节点集). 我们希望确定给定的有向无环图是否隐含特定的条件独立性 $\mathbf{A} \perp \mathbf{B} \mid \mathbf{C}$. 为此, 我们需要考虑从 **A** 中任何节点到 **B** 中任何节点的所有可能路径. 如果任何这样的路径包含一个节点, 则任何这样的路径可称为被阻塞路径, 并且可能出现如下两种情况之一:

1. 路径上的箭头在节点处头尾相接或尾尾相接, 且节点位于集合 **C** 中;
2. 箭头在节点处头头相接, 且节点及其任何后代节点都不在集合 **C** 中.

如果所有路径都被阻塞, 那么 **A** 和 **B** 就会被 **C** 分离, 图中所有变量的联合分布将满足 $\mathbf{A} \perp \mathbf{B} \mid \mathbf{C}$.

d 分离法如图 11.21 所示. 在图 11.21(a) 中, 从 a 到 b 的路径既没有被节点 f 阻塞, 因为节点 f 是这条路径的尾对尾节点, 且未被观测到; 也没有被节点 e 阻塞, 因为尽管节点 e 是头对头节点, 但它在条件集中只有一个后代节点 c. 因此, 条件独立性 $\mathbf{A} \perp \mathbf{B} \mid \mathbf{C}$ 从该图中无法得出. 在图 11.21(b) 中, 从 a 到 b 的路径被节点 f 阻塞, 因为这是一个被观测到的尾对尾节点, 所以根据该图进行分解的任何分布都将满足条件独立性 $a \perp b \mid f$. 请注意, 这条路径也被节点 e 阻塞了, 因为节点 e 是一个头对头节点, 节点 e 及其子节点都不在条件集合中. 在 d 分离中, 图 11.12 中以浮动变量表示的 λ 等这些节点在 d 分离中不起作用. 参数的行为与观测节点的行为相同. 然而, 这些节点不存在边缘分布, 因此参数节点本身永远不会有父节点, 经过这些节点的所有路径永远是尾尾相接的, 因此会被阻塞.

独立同分布数据是条件独立性和 d 分离的另一个示例 (参见 2.3.2 小节). 观察图 11.12 所示的二元分类模型. 这里的随机节点分别对应 t_n、**w** 和 t. 我们可以看到, 对于从子到任何一个节点 t_n 的路径来说,**w** 的节点是尾对尾的, 因此我们具有以下条件独立性:

$$t \perp \{t_n\} \mid \mathbf{w} \tag{11.42}$$

因此, 以网络参数 **w** 为条件,$\hat{t}$ 的预测分布与训练数据 $\{t_1, \cdots, t_N, \mathbf{x}_n\}$ 无关. 我们可以首先使用训练数据来确定 **w** 的后验分布 (或后验分布的近似值), 然后丢弃训练数据, 并使用 **w** 的后验分布来预测新输入的观测结果中的 t.

11.2.4 朴素贝叶斯

有一种相关的图结构出现在朴素贝叶斯 (Naive Bayes) 模型中, 其中我们使用条件独立性假设来简化模型结构. 假设我们的数据由向量 **x** 的观测值组成, 我们希望将 **x** 的观测值归入 K 个类别中的一个类别 (后文简称类). 我们可以为每个类定义类-条件密度 $p(\mathbf{x} \mid C_k)$ 以及先验类概率 $p(C_k)$. 朴素贝叶斯模型的关键假设是, 以类 C_k 为条件, 将输入变量的分布分解为两个或多个密度的乘积. 假设我们把 **x** 分成 L 个元素, 即 $\mathbf{x} = (x_1, \cdots, x_L)$, 则朴素贝叶斯模型的形式为

$$p(\mathbf{x} \mid C_k) = \prod_{j=1}^{L} p(x_j \mid C_k) \tag{11.43}$$

假设式 (11.43)分别对每个类 C_k 成立. 图 11.22 是该模型的图形化表示. 我们可以看到, 在 $j \neq i$ 的情况下, 给定 C 时,x_i 和 x_j 是条件独立的. 因为这种路径在节点 C 处是尾尾相接的, 所以当给定 C 时, 对 C 的观测会阻塞 x_i

和 x_j 之间的路径. 然而, 如果我们将 C 去边缘化, 从 x_i 到 x_j 的尾尾相接路径将不再受阻. 所以一般来说, 边缘密度 $p(\mathbf{x})$ 不会相对于元素 $(x_1, \cdots, x_L)$ 进行分解.

如果我们得到一个标记训练集, 其中包括观测值 $\{\mathbf{x}_1, \cdots, \mathbf{x}_N\}$ 以及它们的类标签, 我们就可以假设数据是从模型中独立提取的, 从而使用最大似然法将朴素贝叶斯模型拟合到训练数据中 (见习题 11.15). 通过使用相应的标记数据分别拟合每个类的模型, 然后将先验类概率 $p(C_k)$ 设置为等于每个类中训练数据点的比例, 从而得到解决方案. 向量 $\mathbf{x}$ 属于类 C_k 的概率由贝叶斯定理给出, 其形式为

$$p(C_k \mid \mathbf{x}) = \frac{p(\mathbf{x} \mid C_k)\, p(C_k)}{p(\mathbf{x})} \tag{11.44}$$

其中 $p(\mathbf{x} \mid C_k)$ 由式 (11.43)给出,$p(\mathbf{x})$ 可以用下式求出:

$$p(\mathbf{x}) = \sum_k p(\mathbf{x} \mid C_k)\, p(C_k) \tag{11.45}$$

用于二维数据空间的朴素贝叶斯模型如图 11.23 所示, 其中 $\mathbf{x} = (x_1, x_2)$. 此处, 我们假设两个类中每个类的类-条件密度 $p(\mathbf{x} \mid C_k)$ 都是轴对齐的高斯密度, 因此它们各自将对 x_1 和 x_2 进行分解, 从而

$$p(\mathbf{x} \mid C_k) = p(x_1 \mid C_k)\, p(x_2 \mid C_k) \tag{11.46}$$

然而, 边缘密度 $p(\mathbf{x})$ 由下式给出:

$$p(\mathbf{x}) = \sum_k p(\mathbf{x} \mid C_k)\, p(C_k) \tag{11.47}$$

它现在是高斯混合模型且不会对 x_1 和 x_2 进行分解. 在融合不同来源的数据 (如用于医疗诊断的血液检测和皮肤图像) 时, 将会涉及朴素贝叶斯模型的简单应用 (参见 5.2.4 小节). 当输入空间的维度 D 较高时, 在全 D 维空间中进行密度估计将更具挑战性, 朴素贝叶斯假设会有所帮助. 如果输入向量同时包含离散变量和连续变量, 这种方法也会很有用, 因为每种变量都可以用适当的模型来表示 (例如, 对于二元观测可以使用伯努利分布, 对于实值变量可以使用高斯分布). 该模型的条件独立性假设显然是强假设, 可能会导致对类-条件密度的表示相当糟糕. 然而, 即使这一假设没有得到精确的满足, 模型在实际应用中也仍然可以提供良好的分类性能, 因为决策边界可能对类-条件密度中的某些细节不敏感.

11.2.5 生成式模型

机器学习的许多应用都可以看作逆问题 (inverse problem) 的示例, 其中有一个产生数据的底层过程 (通常是物理过程), 我们现在的目标是学习如何反转这个过程. 一种方法是训练深度神经网络, 如卷积网络, 从而将图像作为输入并生成描述对象的类型、位置和大小的输出. 因此, 作为判别模型的一个示例, 这种方法试图直接解决逆问题. 只要有足够多的标记图像, 这种方法就能达到很高的准确率. 在实践中, 未标记的图像往往很多, 而在获得训练集的过程中, 大部分工作量耗费在证明标记上, 这个证明过程可能需要手动完成. 我们的判别模型在训练过程中无法直接使用未标记的图像.

另一种方法是对生成过程进行建模, 然后进行计算反转. 在我们的图像示例中, 如果假设对象的类、位置和大小都是独立选择的, 则可以使用有向图模型来表示生成过程, 如图 11.24 所示. 请注意, 箭头的方向与生成步骤的顺序相对应, 因此该模型代表了观测数据生成的因果过程 (Pearl, 1988). 这是生成式模型的一个示例, 因为一旦经过训练, 它就可以用来生成合成图像, 方法是首先从学习到的先验分布中选择对象的类、位置和大小, 然后从学习到的条件分布中抽取图像样本. 对象所属的类 (离散变量) 与对象的位置和方向 (连续变量) 具有独立的先验概率. 图像 (像素强度阵列) 的概率分布取决于对象所属的类及其位置和方向.

根据图 11.24 中的图模型, 在未观测到图像时, 类、位置和大小变量是独立的. 这是因为这些变量中的任何两个变量之间的每条路径就图像变量而言是头头相接的, 这是未观测到的. 然而, 当我们观测到一幅图像时, 这些路

径就会变得畅通, 类、位置和大小变量也不再独立. 从直觉上讲, 这是合理的, 因为被告知图像中对象所属的类可以为我们提供相关信息, 从而帮助我们确定对象的位置和大小. 例如, 一个对象的图像可以看作一个生成过程的输出, 在这个生成过程中, 对象的类型、位置和方向也从一些先验分布中选择, 然后创建图像. 在给定一个大型图像数据集的情况下, 其中的图像已标记所含对象的类型、位置和大小, 我们的目标是训练一个机器学习模型, 该模型可以获取新的、未标记的图像, 并检测对象是否存在, 以及对象在图像中的位置和大小.

然而, 概率模型中的隐变量并不需要有任何明确的物理解释, 而引入这些隐变量可能只是为了从更简单的组件中构造出更复杂的联合分布. 例如, 归一化流、变分自编码器和扩散模型等都使用深度神经网络, 从而通过转换具有简单高斯分布的隐变量, 在数据空间中创建复杂的分布. 稍后我们将看到扩散模型和其他生成式模型如何根据对所需图像内容和风格的文字描述, 合成令人赞叹的高分辨率图像 (参见第 20 章).

11.2.6 马尔可夫毯

在讨论更复杂的有向图时, 一个很有用的条件独立性称为马尔可夫毯 (Markov blanket) 或马尔可夫边界 (Markov boundary). 考虑由具有 D 个节点的有向图表示的联合分布 $p(x_1, \cdots, x_D)$, 并考虑以所有其余变量 $x_{j\neq i}$ 为条件的变量 x_i 的特定节点的条件分布. 利用分解性质 [式 (11.6)], 我们可以将这一条件分布表示为如下形式:

$$p(x_i \mid x_{j\neq i}) = \frac{p(\mathbf{x})}{\int p(\mathbf{x})\,\mathrm{d}x_i} \tag{11.48}$$

其中积分被离散变量的求和取代. 现在我们注意到, 任何与 x_i 无函数相关的因子 $p(x_k \mid \mathrm{pa}(k))$ 都可以在对 x_i 的积分之外取值, 因此分子和分母之间会抵消. 剩下的因子就是节点 x_i 本身的条件分布 $p(x_i \mid \mathrm{pa}(i))$, 以及任何其他节点 x_k 的条件分布, 从而节点 x_i 在 $p(x_k \mid \mathrm{pa}(k))$ 的条件集中, 即节点 x_i 是 x_k 的父节点. 条件分布 $p(x_i \mid \mathrm{pa}(i))$ 将取决于节点 x_i 的父节点, 而条件分布 $p(x_k \mid \mathrm{pa}(k))$ 将取决于 x_k 的子节点和共父节点 [即节点 x_i 的父节点 (节点 x_i 除外) 所对应的变量]. 由父节点、子节点和共父节点组成的节点集称为马尔可夫毯, 如图 11.25 所示. 它具有这样一个特性, 即节点 x_i 的条件分布 (以图中所有其余节点为条件) 但取决于马尔可夫毯中的变量. 我们可以考虑把节点 x_i 的马尔可夫毯看作能将节点 x_i 与图中的其他节点隔离开来的最小节点集. 请注意, 仅包含节点 x_i 的父节点和子节点是不够的, 因为相消解释意味着观测到子节点不会阻塞通往共父节点的路径. 因此, 我们还必须观测到共父节点.

11.2.7 作为过滤器的图

我们已经看到, 一个特定的有向图不仅代表了将联合概率分布分解为条件概率的乘积, 还代表了通过 d 分离准则获得的一组条件独立性.d 分离定理实际上表达了这两个特性的等价性. 为了说明这一点, 不妨把有向图看作一个过滤器. 假设我们考虑的是图中 (未观测到的) 节点所对应的变量 $\mathbf{x}$ 的特定联合概率分布 $p(\mathbf{x})$. 只有当且仅当该分布可以用图所隐含的分解 [式 (11.6)] 表示时, 过滤器才会允许该分布通过. 将变量 $\mathbf{x}$ 上所有可能的分布 $p(\mathbf{x})$ 的集合提交给过滤器, 对于有向分解 (directed factorization), 将过滤器允许通过的分布子集记为 D_F, 如图 11.26 所示. 我们也可以根据是否满足图的 d 分离特性时才能通过过滤器. 将通过过滤器的所有可能概率分布 $p(\mathbf{x})$ 的集合记为 D_F.d 分离特性所隐含的所有条件独立性, 使用图 (作为第二种过滤器) 来过滤分布.d 分离定理表明, 同样的一组分布 D_F 将被允许通过第二种过滤器. 如果我们将所有条件独立性, 然后只允许满足所有这些条件独立性的分布通过.

需要强调的是, 从 d-分离得到的条件独立性适用于该特定有向图所描述的任何概率模型. 例如, 无论变量是离散的还是连续的, 抑或是这些变量的组合, 情况都是如此. 在一种极端情况下, 我们有一个全连接图, 它完全不显示条件独立性, 并且可以表示给定变量上任何可能的联合概率分布, 集合 D_F 将包含所有可能的分布 $p(\mathbf{x})$. 在另一种极端情况下, 我们有一个全不连接图, 即不包含任何连接的图. 这相当于联合分布是一个可以分解为组成图节点的变量的边缘分布的乘积. 请注意, 对于任何给定的图, 集合 D_F 都将包括除图所描述的分布外, 还具有其他独立特性的任何分布. 例如, 一个全分解分布总能通过相应变量集上的任何图所隐含的过滤器.

11.3 序列模型

机器学习有许多重要的应用, 其中的数据都由一系列的值组成, 称为序列数据. 例如, 文本由一系列单词组成, 而蛋白质则由一系列氨基酸组成. 序列数据通常是按时间排序的, 例如麦克风的音频信号或特定地点的日降雨量. 有时, “时间”“过去”“未来”等术语也可以用于描述其他类型的序列数据, 而不仅仅是时间序列. 涉及序列数据的应用包括语音识别、不同语言间的自动翻译、DNA 中的基因检测、合成音乐、编写计算机代码、与现代搜索引擎进行对话等.

我们用 $x_1, \cdots, x_N$ 表示序列数据, 其中的每个元素 x_n 都包含一个值向量. 请注意, 我们可能会有好几个这样的序列数据 (后文简称序列), 它们分别独立地取自同一个分布. 在这种情况下, 所有序列的联合分布可以分解为每个序列分布的乘积. 由此, 我们可以仅关注其中一个序列的建模.

根据式 (11.4), 我们已经看到, 通过反复应用概率的乘积法则,N 个变量的一般分布可以写成条件分布的乘积, 而这种分解的形式取决于变量的特定排序. 对于向量值变量, 如果我们选择与序列中变量顺序相对应的排序, 则可以写出

$$p(x_1, \cdots, x_N) = p(x_1) \prod_{n=2}^{N} p(x_n \mid x_1, \cdots, x_{n-1}) \tag{11.49}$$

这对应于一个有向图, 其中每个节点都从序列中的每个前一节点接收一个连接, 如图 11.27 中的 4 个节点所示. 这就是所谓的自回归 (autoregressive) 模型.

这种表示法具有完全的通用性, 因此从建模的角度来看没有任何价值, 因为它没有表达任何假设. 我们可以通过从图中移除连接, 或者等效地从式 (11.49)右侧因子的条件集中去除变量, 来引入条件独立性, 从而限制模型的空间.

最有力的假设是去除所有条件变量, 从而得出联合分布, 其形式为

$$p(x_1, \cdots, x_N) = \prod_{n=1}^{N} p(x_n) \tag{11.50}$$

式 (11.50)将观测变量视为独立变量, 因此完全忽略了排序信息. 这对应于一个无连接的概率图模型, 如图 11.28 所示.

既能捕获到顺序特性, 又能引入建模假设的有趣模型介于以上两个极端之间. 一个强有力的假设是, 每个条件分布都只依赖于序列中的前一个变量, 从而得出如下联合分布:

$$p(x_1, \cdots, x_N) = p(x_1) \prod_{n=2}^{N} p(x_n \mid x_{n-1}) \tag{11.51}$$

请注意, 序列中第一个变量的处理方式略有不同, 因为它没有条件变量. 图 11.29 所示图模型的函数形式 [式 (11.51)] 称为马尔可夫模型或马尔可夫链, 其可用简单节点链组成的图来表示. 通过使用 d 分离 (参见 11.2.3 小节), 我们可以看到, 在时间 n 之前的所有观测值中, 观测值 x_{n-1} 的条件分布为

$$p(x_n \mid x_1, \cdots, x_{n-1}) = p(x_n \mid x_{n-1}) \tag{11.52}$$

从式 (11.52)开始, 利用概率的乘积法则, 直接求值即可对此轻松验证 (见习题 11.16). 因此, 如果我们使用这样一个模型来预测序列中的下一个观测值, 则预测值的分布将只取决于前一个观测值, 而与之前的所有观测值无关.

更具体地说, 形式为式 (11.51)的图模型称为一阶马尔可夫模型, 因为每个条件分布中只出现了一个条件变量. 我们可以对这个模型进行扩展, 方法是让每个条件分布都依赖于前面的两个变量, 从而得到一个二阶马尔可夫模型, 其形式为

$$p(x_1, \cdots, x_N) = p(x_1)\, p(x_2 \mid x_1) \prod_{n=3}^{N} p(x_n \mid x_{n-1}, x_{n-2}) \tag{11.53}$$

请注意, 前两个变量的处理方法不同, 因为它们的条件变量少于两个. 图 11.30 以有向图的形式对该模型进行了展示. 通过使用 d 分离 (或使用概率规则进行直接计算), 我们可以看到, 在二阶马尔可夫模型中, 给定所有先前观测值 $x_1, \cdots, x_{n-1}$ 的 x_n 的条件分布与观测值 $x_1, \cdots, x_{n-2}$ 无关 (见习题 11.17). 同样, 我们可以考虑扩展到 M 阶马尔可夫链, 其中某个变量的条件分布取决于前 M 个变量.

假设观测值是具有 K 种状态的离散变量, 此时一阶马尔可夫链中的条件分布 $p(x_n \mid x_{n-1})$ 将由一组参数 (共 $K-1$ 个) 指定; 在 x_{n-1} 的 K 个状态中, 每个状态都有 $K-1$ 个参数, 共有 $K(K-1)$ 个参数. 假设我们将模型扩展为 M 阶马尔可夫模型, 这样联合分布便可用条件式 $p(x_n \mid x_{n-1}, \cdots, x_{n-M})$ 来构造. 如果变量是离散的且条件分布用一般条件概率表来表示, 模型将有 $K^{M+1} - 1$ 个参数. 因此, 参数数量会随着 M 的增加而呈指数增长, 一般来说,M 的值越大, 这种方法就越不实用.

假设我们希望为序列建立一个模型, 这个模型不受马尔可夫假设的任何顺序的限制, 但可以使用有限数量的自由参数来指定. 我们可以通过引入额外的潜变量 (又称隐变量) 来实现这一目标, 从而允许从简单组件中构建出类别丰富的模型. 对于每个观测值 x_n, 我们引入一个相应的潜变量 z_n(可能与观测变量的类型或维度不同). 现在, 我们假设潜变量构成了马尔可夫链, 从而产生了状态空间模型的图结构, 如图 11.31 所示. 它满足关键的条件独立性要求, 即给定 z_n 时,z_{n-1} 和 z_{n+1} 是独立的, 因此

$$z_{n+1} \perp z_{n-1} \mid z_n \tag{11.54}$$

状态空间模型用潜变量 $z_1, \cdots, z_N$ 表示一系列观测变量 $x_1, \cdots, x_N$ 的联合概率分布, 它的形式如式 (11.55)所示:

$$p(x_1, \cdots, x_N, z_1, \cdots, z_N) = p(z_1) \prod_{n=2}^{N} p(z_n \mid z_{n-1}) \prod_{n=1}^{N} p(x_n \mid z_n) \tag{11.55}$$

通过使用 d 分离, 我们可以发现, 在状态空间模型中, 总有一条路径能通过潜变量连接任意两个观测变量 x_i 和 x_m, 而且这条路径永远不会阻塞. 因此, 在给定所有先前观测值的情况下, 观测值 x_{n+1} 的预测分布 $p(x_{n+1} \mid x_1, \cdots, x_n)$ 并不会表现出任何条件独立性. 可学习参数的数量由潜变量的维度决定, 而给定观测值 x_n 的分布原则上取决于之前的所有观测值. 但是, 较远的观测值具有非常有限的影响, 因为它们的影响必须通过潜态链来实现, 而潜态链本身也在被较新的观测值更新. 在避免 n 元模型参数数量呈指数增长的同时, 允许存在远距离依赖关系的一种方法是使用隐马尔可夫模型 (hidden Markov model), 其图结构如图 11.31 所示 (参见 11.3.1 小节).

第 12 章 Transformer 与注意力机制

12.1 注意力

Transformer 是深度学习领域最重要的进展之一. 基于一种称为注意力 (attention) 的处理概念,Transformer 能使网络对不同的输入赋予不同的权重, 而这些加权系数本身又取决于输入值, 从而捕捉到与序列数据和其他形式数据相关的强大归纳偏差. 这类模型之所以叫 Transformer, 是因为它们能将某个表示空间中的一组向量变换成某个新空间中具有相同维度的相应向量的集合. 变换旨在使新空间具有更丰富的内部表示, 该表示更适合解决下游任务. 对于 Transformer 而言, 其输入可以是非结构化的向量集、有序序列或其他更一般的表示形式, 从而使 Transformer 具有广泛的适用性.

Transformer 最初是在自然语言处理 (Natural Language Processing, NLP) 领域引入的, 这里的“自然”语言指的是像英语或汉语这样的语言, 它大大超越了之前基于递归神经网络 (Recurrent Neural Network, RNN) 的那些最先进的方法. 随后, 人们发现 Transformer 在许多其他领域也取得了优异成果. 例如, 视觉 Transformer 在图像处理任务中的表现往往优于卷积神经网络 (Convolutional Neural Network, CNN), 而结合文本、图像、音频和视频等多种类型数据的多模态 Transformer 则是最强大的深度学习模型之一.

Transformer 的一大优势在于迁移学习效果显著, 所以 Transformer 可以在大量数据上进行训练, 然后通过某种形式的微调, 将经过训练的模型应用于众多下游任务. 一个大规模模型随后可以进行调整以解决多个不同的任务, 它就是所谓的基础模型 (foundation model). 此外,Transformer 可以使用无标签数据以自监督的方式进行训练 (foundation model). 此外,Transformer 可以使用无标签数据以自监督的方式进行训练, 这对语言模型尤其有效, 因为 Transformer 可以利用来自互联网和其他来源的海量文本. 缩放假设 (scaling hypothesis) 认为, 只要扩大模型的规模 (以可学习参数的数量来衡量), 并在相应的大规模数据集上进行训练, 即使不改变架构, 也能显著提高性能. 此外,Transformer 特别适用于大规模并行处理硬件, 如图形处理单元 (Graphics Processing Unit, GPU), 从而能在合理的时间内训练出拥有万亿 (10^{12}) 规模参数的超大型神经网络语言模型. 这些模型具有非凡的能力, 并展现出被描述为通用人工智能早期迹象的涌现特性 (Bubeck et al., 2023).

对于初学者来说,Transformer 架构可能看起来很复杂, 甚至让人望而却步, 因为它涉及多个不同组件的协同工作, 其中各种设计构思可能看起来很随意. 因此, 本章将逐步全面地介绍 Transformer 背后的所有关键思想, 并提供清晰的直观认识以引出各种元素的设计. 在介绍完 Transformer 架构后, 我们将重点介绍自然语言处理, 并在最后探讨其他应用领域.

12.1.1 Transformer 处理

虽然注意力具有非常广泛的适用性, 但我们仍将以自然语言为例引出注意力的应用. 请看如下两个句子:

I swam across the river to get to the other bank.
I walked across the road to get cash from the bank.

“bank”一词在这两个句子中具有不同的含义. 不过, 这只能通过查看句子中其他词提供的上下文才能发现. 我们还看到, 在决定“bank”如何翻译时, 有些词发挥着尤为重要的作用. 在第 1 个句子中, “swam”和“river”两个词最能说明“bank”指的是河边; 而在第 2 个句子中, “cash”一词最能说明“bank”指的是金融机构. 我们发现, 为了对“bank”进行适当的翻译, 处理这样一个句子的神经网络应该关注 (换句话说, 更依赖于) 句子中其余部分的特定词. 注意力的概念如图 12.1 所示.

作为 Transformer 的核心概念, 注意力最初是作为机器翻译领域递归神经网络 (参见 12.2.5 小节) 的增强版而开发的 (Bahdanau, Cho, and Bengio, 2014). 不过,Vaswani et al. (2017) 后来证明, 如果去除递归结构, 转而只专注于注意力机制, 就能显著提高性能. 如今, 在几乎所有应用中, 基于注意力的 Transformer 已完全取代递归神经网络.

在讨论自然语言处理时, 我们将了解如何使用词嵌入 (word embedding) 将词投射到嵌入空间中的向量. 这些向量可用作后续神经网络处理的输入. 这些词嵌入可以捕捉到基本的语义属性, 例如, 将具有相似含义的词映射到嵌入空间中的邻近位置. 这种嵌入的一个特点是, 给定的词总是被映射到同一个嵌入向量.Transformer 可以视为一种更丰富的嵌入形式, 其中一个给定向量被映射到一个依赖序列中其他向量的位置. 因此, 上述示例中代表“bank”的向量可以映射到两个不同句子的新嵌入空间中的不同位置. 例如, 在第 1 个句子中, 变换后的表示可能会将“bank”在嵌入空间中靠近“water”; 而在第 2 个句子中, 变换后的表示可能会将“bank”放在靠近“money”的位置.

下面我们以蛋白质建模为例来说明注意力的重要性. 我们可以把蛋白质看作由分子单元 (称为氨基酸) 组成的一维序列. 一个蛋白质可能由数百或数千个这样的分子单元组成, 每个分子单元由 22 种可能性中的一种给定. 在活细胞中, 蛋白质折叠成三维结构, 一维序列中互相远离的氨基酸可以在三维空间中变得接近, 从而产生相互作用.Transformer 模型允许这些互相远离的氨基酸相互“关注”, 从而大大提高了三维结构建模的准确性 (Vig et al., 2020).

在清楚地了解 Transformer 的工作原理之前, 我们必须准确地使用符号. 我们将按照标准惯例, 将数据向量组合成一个 $N \times D$ 维的矩阵 $\mathbf{X}$, 其中第 n 行是 token 向量 $\mathbf{x}_n$,$n = 1, \cdots, N$, 如图 12.3 所示. 请注意, 该矩阵代表一组输入 token, 而在大多数应用中, 我们需要一个包含多组 token 的数据集, 例如每个词都是由一个 token 表示的独立文本段落.

Transformer 的输入数据是一组 D 维向量 $\{\mathbf{x}_n\}$, 其中 $n = 1, \cdots, N$. 我们将这些数据向量称为 token, 一个 token 可能对应句子中的一个词、图像中的一个小块或者蛋白质中的一个氨基酸.token 中的元素 x_{ni} 称为特征. 稍后我们将了解如何为自然语言数据和图像构建这些 token 向量.Transformer 的一个强大特性是, 我们无须设计新的网络架构来处理不同类型的数据, 而只需要将数据变量组合成一组联合 token 即可.

Transformer 的基本构件是一个函数, 用于将数据矩阵作为输入并创建一个具有相同维度的变换矩阵 $\bar{\mathbf{X}}$ 作为输出. 我们可以写下该函数, 形式如下:

$$\bar{\mathbf{X}} = \text{TransformerLayer}[\mathbf{X}] \tag{12.1}$$

其中 $\bar{\mathbf{X}}$ 的第 n 行代表变换后的数据向量 $\bar{\mathbf{x}}_n$. 然后, 我们可以连续叠加多个 Transformer 层来构建能够学习丰富内部表示的深度网络. 每个 Transformer 层都有自己的权重和偏置, 可以通过梯度下降法使用适当的损失函数进行学习, 这将在本章后续部分 (参见 12.3 节) 详细讨论.Transformer 层本身包含两个阶段. 在实现注意力机制的第一阶段, 将数据矩阵各列中不同 token 向量的相应特征混合在一起; 第二阶段则独立作用于每一行, 并对每个 token 向量中的特征进行变换.

12.1.2 注意力系数

假设嵌入空间中有一组输入 token $\mathbf{x}_1, \cdots, \mathbf{x}_N$, 我们希望将它们映射到另一组具有相同数量 token 但位于一个新嵌入空间的 $\mathbf{y}_1, \cdots, \mathbf{y}_N$, 这一新的嵌入空间能捕捉到更丰富的语义结构. 考虑一个特定的输出向量 $\mathbf{y}_n$.$\mathbf{y}_n$ 的值不仅取决于相应的输入向量 $\mathbf{x}_n$, 还取决于集合中所有其他的输入向量. 凭借注意力, 对于那些在确定 $\mathbf{y}_n$ 的修正表示中发挥特别重要作用的输入向量 $\mathbf{x}_m$ 来说, 这种依赖性应该更强. 实现这一目标的简单方法是将每个输出向量 $\mathbf{y}_n$ 定义为输入向量 $\mathbf{x}_1, \cdots, \mathbf{x}_N$ 与加权系数 a_{nm} 的线性组合:

$$\mathbf{y}_n = \sum_{m=1}^{N} a_{nm} \mathbf{x}_m \tag{12.2}$$

其中,a_{nm} 称为注意力权重 (attention weight), 又称注意力系数或权重系数. 对于对输出 $\mathbf{y}_n$ 影响较小的输入 token, 注意力系数应接近于零; 而对于对输出 $\mathbf{y}_n$ 影响最大的输入 token, 注意力系数应最大. 因此, 我们限制注意力系数为非负值, 以避免出现一个注意力系数变大为正值, 而另一个注意力系数补偿变小为负值的情况. 我们还希望确保如果一个输出对某一特定输入给予更多注意力, 则应该以减少对其他输入的注意力为代价, 因此我们限制注意力系数的总和为 1: 加权系数必须满足以下两个约束条件:

$$a_{nm} \geq 0 \tag{12.3}$$

$$\sum_{m=1}^{N} a_{nm} = 1 \tag{12.4}$$

这意味着每个注意力系数都在 $0 \leq a_{nm} \leq 1$ 的范围内, 因此这些注意力系数定义了一个“单位分解”(见习题 12.1). 在 $a_{nm} = 1$ 且当 $n = m$ 时 $a_{nm} = 0$ 的特殊情况下, 由于 $\mathbf{y}_n = \mathbf{x}_n$, 输入向量在变换后保持不变. 一般来说, 输出向量 $\mathbf{y}_n$ 是输入向量的混合体, 其中一些输入向量的权重系数高于其他输入向量.

请注意, 我们为每个输出向量 $\mathbf{y}_n$ 设置了一组不同的系数, 而约束条件 [式 (12.3)和式 (12.4)] 分别适用于每个 n 值. 这些系数取决于输入数据, 后续我们将展示如何计算它们.

12.1.3 自注意力

接下来的问题是如何确定权重系数 a_{nm}. 在展开详细讨论之前, 我们不妨先了解一下信息检索领域的一些术语. 以通过线上电影流媒体服务选择观看哪部电影为例. 一种方法是将每部电影与一系列属性关联起来, 这些属性描述了电影的类型 (喜剧、动作片等)、主演姓名、片长等. 用户可以通过目录搜索找到自己喜欢看的电影. 我们可以将每部电影的属性编码成一个称为键 (key) 的向量, 从而将该过程自动化. 相应的电影文件本身叫作值 (value). 同样, 用户也可以为所需属性提供自己的个人值向量, 我们称之为查询 (query). 然后, 线上电影流媒体服务可以将查询向量与所有键向量作比较, 从而找出最佳匹配, 并以值文件的形式将相应的电影发送给用户. 用用户所“关注”的特定电影, 其键向量最匹配于用户的查询向量. 这可以视为硬注意力 (hard attention) 的一种形式, 其会返回一个单一值向量. 对于 Transformer, 我们可以将其泛化到软注意力 (soft attention) 中, 即使用连续变量来度量查询和键之间的匹配程度, 然后使用这些变量来权衡值向量对输出的影响. 这也将确保 Transformer 函数是可微分的, 从而可以通过梯度下降法进行训练.

参照信息检索的方法, 我们可以将每个输入 $\mathbf{x}_n$ 视为一个值向量, 用来创建输出 token. 我们还可以直接使用 $\mathbf{x}_n$ 作为输入 token n 的键向量. 这就好比用电影本身来概括电影的特点. 最后, 我们可以使用 $\mathbf{x}_n$ 作为输出 $\mathbf{y}_n$ 的查询向量, 然后将其与每个键向量作比较. 要想知道 $\mathbf{x}_n$ 代表的 token 与 $\mathbf{x}_m$ 代表的 token 有多大的相似度, 我们需要计算出这些向量的相似度. 度量向量相似度的一种简单方法是求它们的点积 $\mathbf{x}_n^T \mathbf{x}_m$. 为了施加约束条件 [式 (12.3)和式 (12.4)], 使用 softmax 函数变换点积 (参见 5.3 节), 以定义权重系数 a_{nm}:

$$a_{nm} = \frac{\exp\left(\mathbf{x}_n^T \mathbf{x}_m\right)}{\sum_{m'=1}^{N} \exp\left(\mathbf{x}_n^T \mathbf{x}_{m'}\right)} \tag{12.5}$$

请注意, 在这种情况下,softmax 函数不存在概率解释, 它只是用来对注意力权重进行适当的归一化.

因此, 总的来说, 每个输入向量 $\mathbf{x}_m$ 都是通过形式为式 (12.2)的输入向量线性组合而转换为相应的输出向量 $\mathbf{y}_n$ 的, 其中应用于输入向量 $\mathbf{x}_m$ 的权重系数 a_{nm} 由 softmax 函数给出 [式 (12.5)], 该 softmax 函数是根据输入 n 的查询向量 $\mathbf{x}_n$ 与输入 m 相关的键向量 $\mathbf{x}_m$ 之间的点积 $\mathbf{x}_n^T \mathbf{x}_m$ 进行定义的. 请注意, 如果所有输入向量都是正交的, 那么每个输出向量都等于相应的输入向量, 从而在 $m = 1, \cdots, N$ 的情况下,$\mathbf{y}_m = \mathbf{x}_m$(见习题 12.3).

我们可以使用数据矩阵 $\mathbf{X}$ 和类似的 $N \times D$ 输出矩阵 $\mathbf{Y}$(其行由 $\mathbf{y}_m$ 给出) 将式 (12.2)用矩阵符号写出:

$$\mathbf{Y} = \text{Softmax}\left[\mathbf{X}\mathbf{X}^T\right]\mathbf{X} \tag{12.6}$$

其中,Softmax[·] 是一个算子, 作用是取矩阵 $\mathbf{L}$ 中每个元素的指数, 然后将每一行独立归一化, 使它们的和为 1. 这个过程叫作自注意力 (self-attention), 因为我们使用相同的序列来确定查询、键和值. 在后续内容中, 我们还会遇到这种注意力的变体. 此外, 由于查询向量和键向量之间的相似度度量是由点积给出的, 因此这个过程又叫作点积自注意力 (dot-product self-attention).

12.1.4 网络参数

目前, 由于没有可调整的参数, 从输入向量 $\mathbf{x}_n$ 到输出向量 $\mathbf{y}_n$ 的变换是固定的, 且不具备从数据中学习的能力. 此外,token 向量 $\mathbf{x}_n$ 的每个特征值在确定注意力系数时发挥相同的作用, 而我们希望网络在确定 token 相似度时能够灵活地将注意力更多地集中在某些特征上. 通过定义由原始向量的线性变换给出的修正特征向量, 我们可以解决这两个问题:

$$\tilde{\mathbf{X}} = \mathbf{X}\mathbf{U} \tag{12.7}$$

其中 $\mathbf{U}$ 是可学习权重参数的 $D \times D$ 矩阵, 类似于标准神经网络中的“层”, 从而得到一个修正后的变换, 其形式为

$$\mathbf{Y} = \text{Softmax}\left[\tilde{\mathbf{X}}\tilde{\mathbf{X}}^{\mathsf{T}}\right]\tilde{\mathbf{X}} \tag{12.8}$$

虽然这种方法更灵活, 但它有一个特性, 即如下矩阵是对称的:

$$\tilde{\mathbf{X}}\tilde{\mathbf{X}}^{\mathsf{T}} = \mathbf{X}\mathbf{U}\mathbf{U}^{\mathsf{T}}\mathbf{X}^{\mathsf{T}} \tag{12.9}$$

虽然 softmax 函数意味着得到的注意力权重矩阵本身并不对称, 但我们可以通过允许查询和键具有独立参数来创建一个更加灵活的模型. 然而, 式 (12.8)使用相同的参数矩阵 $\mathbf{U}$ 来定义值向量和注意力系数, 这似乎又是一个不受欢迎的限制. 此外, 注意力权重矩阵本身并不对称, 因为我们可能会认为“斧头”与“工具”的关联性很强, 因为每一把斧头都是一个工具; 而“工具”与“斧头”的关联性很弱, 因为除了斧头, 还有许多其他种类的工具. 而我们希望注意力机制支持明显的不对称.

通过定义单独的查询矩阵、键矩阵和值矩阵, 并让每个矩阵都有自己独立的线性变换, 我们可以克服这些限制:

$$\mathbf{Q} = \mathbf{X}\mathbf{W}^{(Q)} \tag{12.10}$$

$$\mathbf{K} = \mathbf{X}\mathbf{W}^{(K)} \tag{12.11}$$

$$\mathbf{V} = \mathbf{X}\mathbf{W}^{(V)} \tag{12.12}$$

其中, 权重矩阵 $\mathbf{W}^{(Q)}$、$\mathbf{W}^{(K)}$ 和 $\mathbf{W}^{(V)}$ 表示在训练最终的 Transformer 架构时将学习到的参数. 矩阵 $\mathbf{W}^{(K)}$ 的维度为 $D \times D_k$, 其中 D_k 是键向量的长度. 矩阵 $\mathbf{W}^{(Q)}$ 的维度为 $D \times D_k$ 必须与矩阵 $\mathbf{K}$ 的维度相同, 因为只有这样才能在查询向量和键向量之间形成点积. 为此, 可以设置 $D_q = D_k$. 同样,$\mathbf{W}^{(V)}$ 是一个维度为 $D \times D_v$ 的矩阵, 其中 D_v 是输出向量的长度. 如果设置 $D_v = D$, 使输出表示和输入表示具有相同的维度, 则有助于加入残差连接, 我们稍后将对此进行讨论. 此外, 如果每个层的维度相同, 则多个 Transformer 层可以叠加在一起 (参见 12.1.7 小节). 通过对式 (12.8)进行泛化, 我们可以得到

$$\mathbf{Y} = \text{Softmax}\left[\mathbf{Q}\mathbf{K}^{\mathsf{T}}\right]\mathbf{V} \tag{12.13}$$

其中矩阵 $\mathbf{Q}\mathbf{K}^{\mathsf{T}}$ 的维度为 $N \times N$, 且矩阵 $\mathbf{Y}$ 的维度为 $N \times D_v$. 矩阵 $\mathbf{Q}\mathbf{K}^{\mathsf{T}}$ 的计算如图 12.4 所示, 而矩阵 $\mathbf{Y}$ 的计算如图 12.5 所示.

实际上, 我们还可以在这些线性变换中加入偏置参数. 不过, 偏置参数也可以包含在权重矩阵中, 就像我们对标准神经网络所做的那样 (参见 6.2.1 小节), 方法是在数据矩阵 $\mathbf{X}$ 中增加一列 1, 然后在权重矩阵中增加额外一行参数来表示偏置. 自此, 我们将把偏置参数视为隐参数, 以免混淆符号.

12.1.5 缩放自注意力

我们还可以对自注意力层进行最后一次改进. 回想一下, 当输入量较大时,softmax 函数的梯度会呈指数减小, 就像 tanh 函数或逻辑斯谛 sigmoid 函数一样. 为了避免出现这种情况, 我们可以在应用 softmax 函数之前重新缩放查询向量和键向量的点积. 为了得到合适的缩放结果, 请注意, 如果查询向量和键向量的元素都是均值为 0 且方差为单位方差的独立随机数, 则点积的方差就是 D_k(见习题 12.4). 因此, 我们使用 D_k 的平方根所给出的标准差对 softmax 的参数进行归一化处理, 从而注意力层的输出形式为

$$\text{Attention}(\mathbf{Q},\mathbf{K},\mathbf{V}) = \text{Softmax}\left[\frac{\mathbf{Q}\mathbf{K}^{\mathsf{T}}}{\sqrt{D_k}}\right]\mathbf{V} \tag{12.14}$$

这就是所谓的缩放点积自注意力 (scaled dot-product self-attention), 也是自注意力神经网络层的最终形式. 这一层的结构如图 12.6 所示, 算法过程见算法 12.1.

12.1.6 多头注意力

注意力层允许输出向量关注输入向量的依赖数据模式, 称作注意力头 (attention head). 然而, 可能同时存在多种相关的注意力模式. 例如, 在自然语言中, 有些注意力模式可能与时态有关, 而另一些注意力模式则可能与词汇有关. 使用单一注意力头会导致这些效应的平均化. 对此, 我们可以并行使用多个注意力头. 这些注意力头由结构相同的单个注意力头的副本组成, 具有独立的可学习参数, 用于控制查询矩阵、键矩阵和值矩阵的计算. 这类似于在卷积网络的每一层中使用多个不同的过滤器.

假设我们有 H 个注意力头, 以 $h = 1, \cdots, H$ 进行编号, 形式为

$$\mathbf{H}_h = \text{Attention}(\mathbf{Q}_h, \mathbf{K}_h, \mathbf{V}_h) \tag{12.15}$$

其中 $\text{Attention}(\cdot,\cdot,\cdot)$ 由式 (12.14)给出, 使用如下方法为每个注意力头定义单独的查询矩阵、键矩阵和值矩阵:

$$\mathbf{Q}_h = \mathbf{X}\mathbf{W}_h^{(Q)}, \qquad \mathbf{K}_h = \mathbf{X}\mathbf{W}_h^{(K)}, \qquad \mathbf{V}_h = \mathbf{X}\mathbf{W}_h^{(V)} \tag{12.16}$$

首先将各个注意力头合并成一个单一矩阵, 然后使用矩阵 $\mathbf{W}^{(O)}$ 对结果进行线性变换, 从而得到一个合并输出, 形式为

$$\mathbf{Y}(\mathbf{X}) = \text{Concat}\left[\mathbf{H}_1, \cdots, \mathbf{H}_H\right]\mathbf{W}^{(O)} \tag{12.17}$$

多头注意力网络的结构如图 12.7 所示, 算法过程见算法 12.2.

每个矩阵 $\mathbf{H}_h$ 的维度为 $N \times D_v$, 因此合并矩阵的维度为 $N \times HD_v$. 在使用维度为 $HD_v \times D$ 的线性矩阵 $\mathbf{W}^{(O)}$ 进行变换后, 最终得到维度为 $N \times D$ 的输出矩阵 $\mathbf{Y}$, 它与原始输入矩阵 $\mathbf{X}$ 的维度相同. 矩阵 $\mathbf{W}^{(O)}$ 的元素是在训练阶段与查询矩阵、键矩阵和值矩阵一起学习的. 通常情况下,D_v 选择为与 D/H 相等, 从而使得到的合并矩阵的维度为 $N \times D$. 图 12.8 展示了多头注意力层的信息流.

请注意, 前文给出的多头注意力表述与研究文献中使用的表述相同, 我们在每个注意力头的 $\mathbf{W}^{(V)}$ 矩阵与输出矩阵 $\mathbf{W}^{(O)}$ 的连续乘法中包含了一些冗余. 在消除这些冗余后, 多头自注意力层就可以写成每个注意力头的贡献之和 (见习题 12.5).

12.1.7 Transformer 层

多头自注意力构成了 Transformer 网络的核心架构元素. 我们知道, 神经网络从深度结构中获益匪浅, 因此我们希望将多个自注意力层堆叠在一起. 为了提高训练效率, 我们可以引入绕过多头结构的残差连接 (参见 9.5 节). 为此, 我们要求输出维度与输入维度相同 (参见 7.4.3 小节). 然后进行层归一化 (layer normalization)(Ba, Kiros, and Hinton, 2016), 从而提高训练效率. 由此产生的变换可以写成

$$\mathbf{Z} = \text{LayerNorm}\,[\mathbf{Y}(\mathbf{X}) + \mathbf{X}] \tag{12.18}$$

其中 $\mathbf{Y}$ 由式 (12.17)定义. 有时, 层归一化会被预归一化取代, 即层归一化在多头注意力之前而不是之后进行, 因为这可以带来更有效的优化. 在这种情况下, 我们得出

$$\mathbf{Z} = \mathbf{Y}(\mathbf{X}') + \mathbf{X}, \qquad \text{其中} \quad \mathbf{X}' = \text{LayerNorm}\,[\mathbf{X}] \tag{12.19}$$

此外, 这些值向量是输入向量的线性函数, 因此我们可以看到, 注意力层的输出受限于输入的线性组合. 非线性通过注意力权重引入, 因此输出将通过 softmax 函数非线性地依赖于输入, 但输出向量仍然受限于输入向量所跨的子空间, 这限制了注意力层的表达能力. 我们可以使用一个标准的非线性神经网络 (有 D 个输入和 D 个输出) 对每一层的输出进行后处理, 用 MLP[·] 表示 “多层感知机” , 从而提高 Transformer 的灵活性. 例如, 这可能包括一个具有 ReLU 隐藏单元的两层全连接网络. 当我们这样做时, 需要保持 Transformer 处理不同长度序列的能力. 为此, 我们需要对每个输出向量 (与 $\mathbf{Z}$ 的行相对应) 应用相同的共享网络. 同样, 也可以通过使用残差连接来改进该神经网络层. 它还包括层归一化, 从而 Transformer 层的最终输出形式为

$$\mathbf{X}' = \text{LayerNorm}\,[\text{MLP}[\mathbf{Z}] + \mathbf{Z}] \tag{12.20}$$

在这种情况下,Transformer 层的最终输出形式为

$$\mathbf{X}' = \text{MLP}[\mathbf{Z}'] + \mathbf{Z}, \qquad \text{其中} \quad \mathbf{Z}' = \text{LayerNorm}\,[\mathbf{Z}] \tag{12.21}$$

这就产生了 Transformer 层的整体架构, 如图 12.9 所示, 算法过程见算法 12.3.

同样, 我们也可以使用预归一化. 在典型的 Transformer 模型中, 有多个这样的层叠加在一起. 虽然不同层之间不共享权重和偏置, 但各层通常具有相同的结构.

12.1.8 计算复杂性

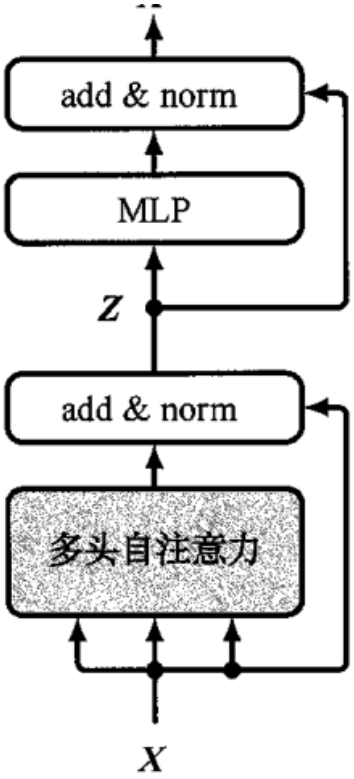
到目前为止我们所讨论的注意力层包含一个向量集 (其中有 N 个向量, 每个向量的长度为 D), 我们将它映射到了另一个具有相同维度的向量集 (其中也有 N 个向量). 因此, 输入和输出的总维度均为 ND. 如果我们使用标准的全连接神经网络将输入值映射到输出值, 就会有 $O(N^2D^2)$ 个独立参数. 同样, 计算通过这样一个网络进行一次前向传递的成本也是 $O(N^2D^2)$.

在注意力层中, 矩阵 $\mathbf{W}^{(Q)}$、$\mathbf{W}^{(K)}$ 和 $\mathbf{W}^{(V)}$ 在输入 token 之间共享, 因此独立参数的数量为 $O(D^2)$, 这里假设 $D_q \approx D_v = D$. 由于有 N 个输入 token, 在自注意力层中计算点积的步骤数为 $O(N^2D)$. 我们可以把自注意力层看作一个稀疏矩阵, 其中的参数在矩阵的特定块之间共享. 随后的神经网络层 (它有 D 个输入和 D 个输出) 的成本为 $O(D^2)$. 由于它在 token 之间共享, 其复杂度与 N 成线性关系, 因此这一层的总成本为 $O(ND^2)$.

根据 N 和 D 的相对大小,Transformer 层或多层感知机层有可能支配计算成本. 与全连接网络相比,Transformer 层的计算效率更高.Transformer 架构的许多变体已被提出 (Lin et al., 2021; Phuong and Hutter, 2022), 包括旨在提高效率的修正版 (Tay et al., 2020).

12.1.9 位置编码

在 Transformer 架构中, 矩阵 $\mathbf{W}^{(Q)}$、$\mathbf{W}^{(K)}$ 和 $\mathbf{W}^{(V)}$ 由输入的所有 token 共用. 同样, 紧随其后的神经网络 (前馈神经网络) 也由输入的所有 token 共用. 因此,Transformer 具有这样的特性, 即对输入 token(也就是 $\mathbf{X}$ 的行) 进行排序会导致输出矩阵 $\bar{\mathbf{X}}$ 的行也被排序. 换句话说,Transformer 在输入排列方面是等变化的 (equivariant). 网络架构中的参数共享有利于 Transformer 的大规模并行处理, 也使网络能够像学习短距离依赖关系一样有效地学习远距离依赖关系. 然而, 当我们考虑序列数据 (如自然语言中的词) 时, 不依赖 token 顺序就成了一个主要的限制, 因为 Transformer 学习到的表示法将与输入 token 的顺序无关. 句子 “The food was bad, not good at all” 和 “The food was



2.9 Transformer 层的整体
]。此处“MLP”代表多层
机，“add & norm”则代
差连接和随后的层归一化

$\tilde{\mathbf{X}} = \mathrm{ML}$

图 12.1: Transformer 层的整体架构. 此处“MLP”代表多层感知机，“add & norm”则代表残差连接和随后的层归一化.

good, not bad at all" 包含相同的 token, 但由于 token 的排序不同, 这两个句子的含义也截然不同. 显然,token 顺序对于包括自然语言处理在内的大多数顺序处理任务至关重要, 因此我们需要找到一种方法来将有关 token 顺序的信息注入网络.

由于想要保留精心构建的注意力层的强大特性, 因此我们希望在数据中对 token 顺序进行编码, 而不是在网络架构中表示 token 顺序. 为此, 我们将构建一个与每个输入位置 n 相关的位置编码向量 $\mathbf{r}_n$, 然后将其与相关输入 token 的嵌入向量 $\mathbf{x}_n$ 结合起来. 增加所有后续注意力空间的维度, 进而显著增加计算成本. 相反, 我们可以简单地将位置向量加到 token 向量上, 从而得出

$$\tilde{\mathbf{x}}_n = \mathbf{x}_n + \mathbf{r}_n \tag{12.22}$$

这就要求位置编码向量的维度与 token 嵌入向量的维度相同.

接下来的任务是构建嵌入向量 $\mathbf{r}_n$. 一种简单的方法是为每个位置关联一个整数. 但这样做的问题是, 数值的大小会无限制地增加, 因此可能会严重破坏嵌入向量. 此外, 对于比训练时所用输入序列更长的新输入序列, 它可能无法很好地泛化, 因为这些输入序列涉及的编码值将超出训练时所用编码值的范围. 我们也可以为序列中的每个 token 分配一个介于 0 和 1 之间的数字, 从而保持表示法的有界性. 然而, 这种表示法对于特定位置并不是唯一的, 因为它取决于整个序列的长度.

理想的位置编码应该为每个位置提供唯一的表示, 其应该是有界的, 能泛化为较长的序列, 且应该有一致的方法来表示任意两个输入向量之间的步数, 而不管它们的绝对位置如何, 因为 token 的相对位置往往比绝对位置更重要. 位置编码的方法有很多 (Dutker, Schmitt, and Schütze, 2021). 在此, 我们介绍一种由 Vaswani et al. (2017) 引入的基于正弦函数的技术. 对于给定的位置 n, 相关的编码向量具有组件 r_{ni}, 其计算公式为

$$r_{n,2i} = \sin\left(\frac{n}{10000^{2i/D}}\right), \qquad r_{n,2i+1} = \cos\left(\frac{n}{10000^{2i/D}}\right) \tag{12.23}$$

当 i 是偶数时取第一式, 当 i 是奇数时取第二式. 这种编码的特点是, 向量 $\mathbf{r}_n$ 的元素都位于 $(-1, 1)$ 范围内. 这不禁让人联想到二进制数的表示方法, 最低阶位高频率地交替出现, 而随后的位则以逐渐降低的频率交替出现. 如图 12.10(a) 所示, 嵌入向量 $\mathbf{r}_n$ 的元素由一系列波长逐渐增大的正弦和余弦函数给出.

式 (12.23)给出的正弦表示法有一个很好的特性, 即对于任何固定偏移 k, 位置 $n + k$ 处的编码可以表示为位置 n 处编码的线性组合, 其中系数不取决于绝对位置, 而只取决于 k 值 (见习题 12.10). 因此, 网络应该能够学会关注相对位置. 请注意, 这一特性要求编码同时使用正弦和余弦函数. 不过, 对于式 (12.23)来说, 向量元素是连续变量而非二进制变量. 位置编码向量的热图说明见图 12.10(b).

另一种常用的位置表示方法是使用习得的位置编码. 这是通过在每个 token 位置设置一个权重向量来实现的, 该权重向量可在训练过程中与模型的其他参数共同学习, 从而避免使用人工表示法. 由于 token 位置之间不共享参数, 因此 token 在排列时不再是不变的, 而这正是位置编码的目的. 不过, 这种方法并不符合前述对较长的输入序列进行泛化的标准, 因为训练过程中没有出现过的位置编码未经训练. 因此, 当输入序列的长度在训练和推理过程中都相对固定时, 这种方法通常最适用.

12.2 自然语言

在研究了 Transformer 架构之后, 我们将探讨如何用它来处理由词、句子和段落组成的语言数据. 尽管 Transformer 模型最初就是针对这种模态开发的, 但事实证明,Transformer 模型是一类非常通用的模型, 并已成为大多数输入数据类型的最先进模型. 本章稍后将探讨 Transformer 在其他领域的应用 (参见 12.4 节).

包括英语在内的许多语言都由一系列词 (用空格分隔) 和标点符号组成, 语言数据是序列数据的典型代表. 我们目前将重点关注词, 稍后再讨论标点符号.

12.2.1 词嵌入

我们面临的第一个挑战是将词转换成适合用作深度神经网络输入的数字表示. 一种简单的方法是定义一个固定的字典, 然后引入长度与这个字典大小相等的向量以及每个词的“独热”表示, 其中字典中的第 k 个词用一个向量编码, 该向量在第 k 个位置为 1, 在其他位置为 0. 例如, 如果“aardwolf”是这个字典中的第三个词, 那么它的向量表示就是 $(0, 0, 1, 0, \cdots, 0)$.

独热编码有一个明显的问题, 即现实中的字典可能有几十万个条目, 从而导致向量的维度非常高. 此外, 独热编码无法捕捉词与词之间可能存在的任何相似性或关系. 这两个问题都可以通过将词映射到一个低维空间来解决, 这个过程叫作词嵌入 (word embedding). 在这个过程中, 每个词都可以表示为一个通常只有几百个维度的空间中的密集向量.

嵌入过程可以用一个大小为 $D \times K$ 的矩阵 $\mathbf{E}$ 来定义, 其中 D 是嵌入空间的维度,K 是字典的维度. 对于每个独热编码的输入向量 $\mathbf{x}_n$, 我们可以用下式计算出相应的嵌入向量:

$$\mathbf{v}_n = \mathbf{E}\mathbf{x}_n \tag{12.24}$$

由于 $\mathbf{x}_n$ 具有独热编码, 因此向量 $\mathbf{v}_n$ 可以由矩阵 $\mathbf{E}$ 的相应列给出.

我们可以使用很多方法来从文本语料库 (即大型数据集) 中学习矩阵 $\mathbf{E}$. 在此, 我们将研究一种名为 word2vec 的流行技术 (Mikolov et al., 2013), 它可以视为一种简单的双层神经网络. 构建一个训练集, 其中每个样本都是通过考虑文本中 M 个相邻词的“窗口”获得的, 一个典型的值可能是 $M = 5$. 样本被认为是独立的, 误差函数则定义为每个样本的误差函数之和. 这种方法有两个模型: 连续词袋模型和“跳字模型”. 在连续词袋 (continuous bag of words) 模型中, 网络训练的目标变量是中间词, 其余的上下文 (context) 词构成输入, 因此网络被训练用于“填空”. 而“跳字 (skip-gram) 模型”将输入和输出互换了, 也就是将中心词作为输入, 而将目标值作为上下文词. 这两个模型如图 12.11 所示.

模型训练完成后, 嵌入矩阵 $\mathbf{E}$ 在连续词袋模型中由第二层权重矩阵的转置矩阵给出, 而在“跳字模型”中则由第一层权重矩阵给出. 在嵌入空间中, 语义相关的词会被映射到附近的位置. 这在意料之中, 因为与不相关的词相比, 相关的词更有可能与相似的上下文词一起出现. 例如, “城市”和“首都”两个词作为“巴黎”或“伦敦”等目标词的上下文词出现的频率可能较高, 而作为“橙色”或“多项式”的上下文词出现的频率较低. 如果将“巴黎”和“伦敦”映射到附近的嵌入向量上, 网络就能更容易地预测缺失词的概率.

事实证明, 习得的嵌入空间往往比相关词的邻近性具有更丰富的语义结构, 这使得进行简单的向量运算成为可能. 例如, “巴黎之于法国就像罗马之于意大利”这一概念可以通过对嵌入向量的运算来表达. 如果我们用 $\mathbf{v}$(word) 来表示“词”的嵌入向量, 则可以得到

$$\mathbf{v}(\text{Paris}) - \mathbf{v}(\text{France}) + \mathbf{v}(\text{Italy}) \approx \mathbf{v}(\text{Rome}) \tag{12.25}$$

词嵌入本身最初是作为自然语言处理工具开发的. 如今, 它们更多地用作深度神经网络的预处理步骤. 在这方面, 它们可以被视为深度神经网络的第一层. 它们可以被一些标准的预训练嵌入矩阵来固定, 也可以作为一个自适应层来处理, 作为系统整体端到端训练的一部分来学习. 在后一种情况下, 可以使用随机权值或标准嵌入矩阵对嵌入层进行初始化.

12.2.2 分词

使用固定字典的一个问题是, 我们无法处理字典中没有的词或拼写错误的词. 此外, 这种方法不考虑标点符号或其他字符序列, 如计算机代码. 解决这些问题的另一种方法是在字符层面进行操作而不是使用词来操作, 这样我们的字典就包含了大小写字母、数字、标点符号以及其他符号 (如空格和制表符等). 但这种方法的缺点是, 它忽略了语言中语义上重要的词结构, 因此后续的神经网络必须学会从基本字符中重新组合词. 此外, 对于给定的文本, 还需要执行更多的连续步骤, 从而增加了处理序列的计算成本.

我们可以通过使用预处理步骤将一串词和标点符号转换成一串 token, 从而将字符级和词级表示的优势结合起来. 通过进行这种分词, 系统便能处理其他类型的序列, 如计算机代码, 甚至处理其他模态, 如图像 (参见 12.4.1 小节). 这也意味着同一个词的变体可以有相关的表示. 例如, “cook” “cooks” “cooked” “cooking” 和 “cooker” 都是相关的并共享共同元素 “cook” , 而 “cook” 本身也可以表示为 token.

有许多方法可以实现分词. 例如, 用于数据压缩的字节对编码 (byte pair encoding) 技术可以通过连接字符而非字节来适应文本分词 (Sennrich, Haddow, and Birch, 2015). 首先用单个字符列表对 token 列表进行初始化. 然后搜索文本中出现频率最高的相邻 token 对, 并用新 token 替换这些 token. 为确保词不被连接, 如果第二个 token 以空格开头, 则新 token 不是由两个 token 组成的. 反复进行上述过程, 如图 12.12 所示. 最初,token 数量等于字符数量, 因而相对较少. 随着 token 的形成,token 的总数也会增加, 如果持续时间足够长,token 最终将与文本中的词集相对应. 作为字符级和词级表示之间的折中方案,token 的总数通常是预先确定的. 当达到这个数量时, 算法就会停止.

在将深度学习运用到自然语言的实际操作中时, 输入文本通常首先映射为 token 化的表示. 不过, 在本章的其余部分, 我们将使用词级表示法, 因为这样更容易说明和引出关键概念.

12.2.3 词袋模型

现在, 我们要对自然语言中的词 (或 token) 等有序向量序列的联合分布 $p(x_1, \cdots, x_N)$ 进行建模. 最简单的方法是假定词是从同一分布中独立抽取的, 因此联合分布被全分解, 形式为

$$p(x_1, \cdots, x_N) = \prod_{n=1}^{N} p(x_n) \tag{12.26}$$

这可以表示为一个概率图模型, 其中节点是孤立的, 相互之间无连接. 我们可以使用词袋法构建一个简单的文本分类器. 例如, 在情感分析中, 可以将一段代表餐厅评论的文本分为正面评论和负面评论. 朴素贝叶斯分类器假定每个类 C_k 中的词是独立的, 但每个类的词分布不同, 因此

$$p(x_1, \cdots, x_N \mid C_k) = \prod_{n=1}^{N} p(x_n \mid C_k) \tag{12.27}$$

在给定先验类概率 $p(C_k)$ 的前提下, 新序列的后验类概率为

$$p(C_k \mid x_1, \cdots, x_N) \propto p(C_k) \prod_{n=1}^{N} p(x_n \mid C_k) \tag{12.28}$$

类-条件密度 $p(x_n \mid C_k)$ 和先验概率 $p(C_k)$ 都可以使用训练集的频率进行估计. 只需要将这些概率中的每个概率设置为该词在训练集中出现的次数, 就能得到该模型的最大似然解. 这就是所谓的词袋 (bag-of-words) 模型, 因为它完全忽略了词的排序. 对于一个新的序列, 将表格条目相乘以得到所需要的后验概率. 请注意, 如果测试集中出现一个训练集中没有的词, 那么相应的概率估计值将为零, 因此通常会在训练后对这些估计值进行 “平滑” , 即在所有条目中均匀地重新分配一个较小的概率水平, 以避免出现零值.

12.2.4 自回归模型

词袋模型的一个主要局限在于完全忽略了词序. 为了解决这个问题, 我们可以采用自回归方法. 在不失通用性的前提下, 我们可以将词序列的分布分解为条件分布的乘积, 其形式为

$$p(x_1, \cdots, x_N) = p(x_1) \prod_{n=2}^{N} p(x_n \mid x_1, \cdots, x_{n-1}) \tag{12.29}$$

这可以用一个概率图模型 (参见图 11.29) 来表示, 其中序列里的每个节点都会从各自的前一个节点接收一个连接. 我们可以用一个表格来表示式 (12.29)右侧的每一项, 该表格中的条目也是通过训练集的简单频率计数估算

出来的. 然而, 这些表格的大小会随着序列长度的增加呈指数增长 (见习题 12.12), 因此这种方法的成本会高得令人望而却步.

我们可以假设式 (12.29)右侧的每个条件分布都与之前的所有观测值无关, 但最近的 L 个词除外, 从而大大简化模型. 例如, 如果 $L = 2$, 那么在该模型下, 一组 N 个观测值的联合分布为

$$p(x_1, \cdots, x_N) = p(x_1)\, p(x_2 \mid x_1) \prod_{n=3}^{N} p(x_n \mid x_{n-1}, x_{n-2}) \tag{12.30}$$

在相应的图模型中, 每个节点都有前两个节点的连接 (参见图 11.30). 此处, 我们假设所有变量共享条件分布 $p(x_n \mid x_{n-L:n-1})$. 同样, 式 (12.30)右侧的每个分布都可以用表格来表示, 其值是从抽取自训练语料的连续词的三元组统计中估算出来的.$L = 1$ 的情况称为二元 (bi-gram) 模型, 因为它取决于相邻的词对. 同样,$L = 2$ 的情况涉及相邻词的三元组, 叫作三元 (tri-gram) 模型. 一般来说, 这些模型都叫作 n 元 (n-gram) 模型.

本节到目前为止讨论的所有模型都可以生成式运行, 以合成新文本. 例如, 如果我们提供一个序列中的第 1 和第 2 个词, 那么我们就可以从三元统计 $p(x_n \mid x_{n-1}, x_{n-2})$ 中抽样生成第 3 个词, 然后我们可以使用第 2 和第 3 个词来抽样生成第 4 个词, 以此类推. 然而, 由于每个词都是根据前两个词预测出来的, 因此产生的文本将是不连贯的. 高质量的文本模型必须考虑语言中的远距离依赖关系. 另一方面, 我们不能简单地类推. 然而, 由于每个词都是根据前两个词预测出来的, 因此产生的文本将是不连贯的. 另一方面, 我们不能简单地通过增加 L 的值, 因为概率表的大小会呈指数级增长, 所以成本远远超过三元模型. 不过, 当我们考虑现代语言模型时, 自回归表示法将发挥核心作用, 这些模型并不基于概率表, 而基于配置为 Transformer 的深度神经网络.

12.2.5 递归神经网络

n 元模型等技术在序列长度上的缩放性很差, 因为它们存储的是完全通用的条件概率表, 而基于神经网络的参数化模型, 我们可以实现更好的缩放. 设想我们仅仅用标准的前馈神经网络来处理自然语言中的词序列. 此时便会出现一个问题, 即网络有固定数量的输入和输出, 而我们需要网络能够处理训练集和测试集中长度可变的序列. 此外, 如果序列中某个位置的一个词或一组词代表某个概念, 那么处在不同位置的同一个词或同一组词很可能在新的位置代表相同的概念. 这会让我们想起之前处理图像数据时遇到的等变性 (参见第 10 章). 如果我们能够构建一个能在序列中共享参数的网络架构, 那么我们不仅可以捕捉到等变性, 还可以大大减少模型中自由参数的数量, 并处理不同长度的序列.

为了解决这个问题, 我们可以借鉴隐马尔可夫模型, 引入一个与序列中每一步 n 相关的显式隐变量 $\mathbf{z}_n$. 神经网络将当前词 $\mathbf{x}_n$ 和当前隐状态 $\mathbf{z}_{n-1}$ 作为输入, 并产生输出词 $\mathbf{y}_n$ 以及隐变量的下一个状态 $\mathbf{z}_n$. 然后, 我们可以将该网络的各个副本连接起来, 其中权重值在各个副本之间共享. 由此产生的架构称为递归神经网络 (Recurrent Neural Network, RNN), 如图 12.13 所示. 此处隐状态的初始值可以初始化为某个默认值, 例如 $\mathbf{z}_0 = (0, 0, \cdots, 0)$.

图 12.13 显示了带有参数 $\mathbf{w}$ 的一般递归神经网络. 它将 $\mathbf{x}_1, \cdots, \mathbf{x}_N$ 作为输入, 并生成一个序列 $\mathbf{y}_1, \cdots, \mathbf{y}_N$ 作为输出. 图中的每个方框对应一个具有非线性隐藏单元的多层网络. 关于如何在实践中使用递归神经网络, 请参考将句子从英语翻译成荷兰语的具体任务. 句子的长度是可变的, 且每个输出句子的长度可能与相应的输入句子的具体任务. 输入句子的长度是可变的, 且网络在开始生成输出句子之前, 可能需要看到整个输入句子.

输入句子. 我们可以使用递归神经网络解决这个问题, 方法是输入完整的英文句子, 然后输入一个特殊的 token(用 ⟨start⟩ 表示), 以触发翻译过程的开始. 在训练过程中, 网络将学会如何将 ⟨start⟩ 与输出句子的开头联系起来. 我们还可以将每个连续生成的词输入下一个时间步的输入中, 如图 12.14 所示. 训练网络以生成特定的 ⟨stop⟩token, 以表示翻译完成. 网络的前几个阶段用于吸收输入序列, 而相关的输出向量则被忽略. 网络的这一部分可以看作一个 “编码器”, 其中整个输入句子都压缩成了隐变量的状态 $\mathbf{z}^*$. 其余的网络阶段则充当 “解码器”, 旨在逐字生成翻译好的句子作为输出. 请注意, 每个输出词都会作为输入词输入网络的下一阶段, 因此这种方法具有类似于式 (12.29)的自回归结构.

解决梯度消失和梯度爆炸问题以及有限远距离依赖关系的一种方法是修正神经网络架构, 从而允许额外的信号路径, 该路径可以绕过网络每个阶段的许多处理步骤, 进而允许在更多的时间步内记忆信息. 长短时记忆

(Long Short-Term Memory, LSTM) 模型 (Hochreiter and Schmidhuber, 1997) 和门控循环单元 (Gated Recurrent Unit, GRU) 模型 (Cho et al., 2014) 是最广为人知的示例. 虽然与标准递归神经网络相比, 它们的性能有所提高, 但在远距离依赖关系建模方面, 它们的能力仍然有限, 而且每个单元的额外复杂性意味着长短时记忆模型的训练速度相比标准递归神经网络更慢. 此外, 所有递归神经网络的信号路径都与序列中的步数成线性关系. 最后, 由于处理过程具有连续性, 它们不支持在单个训练示例内进行并行计算. 特别是, 这意味着递归神经网络难以有效利用基于图形处理单元的现代高度并行硬件. 用 Transformer 取代递归神经网络可以解决这些问题.

12.2.6 通过时间反向传播

与普通神经网络一样, 递归神经网络可根据反向传播计算出的梯度, 通过随机梯度下降法进行训练, 并通过自动微分法进行计算. 误差函数包括所有输出单元对每个单元误差的求和, 其中每个输出单元都有一个 softmax 激活函数和一个相关的交叉熵误差函数. 在前向传播过程中, 激活值从序列中的第一个输入节点一直传播到序列中的所有输出节点, 然后误差信号沿着相同的路径反向传播. 这一过程称为通过时间反向传播 (backpropagation through time), 其原理简单明了. 然而在实践中, 由于深度网络架构会出现梯度消失或梯度爆炸的问题, 因此训练很长的序列可能会很困难.

标准递归神经网络的另一个问题是, 其无法很好地处理远距离依赖关系. 这在自然语言中尤为突出, 因为这种依赖关系非常普遍. 在较长的文本段落中, 不妨引入一个概念, 这个概念对预测文本后面出现的词起着重要作用. 在图 12.14 所示的架构中, 英文句子的整个概念必须在长度固定的单个隐向量 $\mathbf{z}^*$ 内被捕捉到, 而如果序列较长, 这个问题就会越来越严重. 这就是所谓的瓶颈问题 (bottleneck problem), 因为任意长度的序列都必须总结为单个激活的隐向量, 而网络只有在处理完整个输入序列后才能开始生成输出.

12.3 Transformer 语言模型

递归神经网络的自回归特性意味着只能按顺序生成每个输出词, 因此网络在为每个输出位置提供输入时, 会看到所有先前的输出.Transformer 提供了一种更高效的机制来实现类似的效果, 因为它可以实现对序列中所有位置的并行处理.

12.3.1 解码器型 Transformer

到目前为止, 我们已经讨论了自注意力, 在自注意力中, 序列中的每个输入向量 (包括与句子结尾相对应的向量) 都会关注序列中的所有其他向量. 对于语言模型, 我们需要将输出向量限制为只关注在其之前出现的词 (即从序列开头直到并包括当前位置). 这可以通过将未来输入向量的注意力系数设置为零来实现. 例如, 对于一个由 5 个向量组成的序列, 我们可以使用下三角矩阵 $\mathbf{A}$ 作为注意力系数矩阵:

$$\mathbf{A} = \begin{bmatrix} 1 & 0 & 0 & 0 & 0 \\ 1 & 1 & 0 & 0 & 0 \\ 1 & 1 & 1 & 0 & 0 \\ 1 & 1 & 1 & 1 & 0 \\ 1 & 1 & 1 & 1 & 1 \end{bmatrix} \tag{12.31}$$

其中, 注意力系数矩阵的每一行都归一化为总和 1. 我们可以对注意力系数矩阵 $\mathbf{A}$ 进行掩码处理, 方法是将所有需要置零的元素的 softmax 输入设置为某个非常大的负值. 然后, 在这些元素上计算 softmax 函数, 使其输出趋近于零. 由此得到的注意力层称为遮掩自注意力 (masked self-attention), 因为它阻止模型关注当前位置之后的词.

对于包含 N 个词的句子的语言模型, 我们可以通过提供嵌入的词序列 $\mathbf{x}_1, \cdots, \mathbf{x}_N$(每个前面都有位置编码 $\mathbf{r}_1, \cdots, \mathbf{r}_N$) 来定义. 然后将这些输入向量送入一系列的 L 个遮掩自注意力层中. 网络的输出是一个向量序列 $\mathbf{y}_1, \cdots, \mathbf{y}_N$, 其中每个输出向量都是长度为 K 的向量,K 为词表的大小. 在训练过程中, 模型要从这些向量中学习预测序列中的下一个词. 为此, 我们将向量 $\mathbf{y}_n$ 表示为输入词 $\mathbf{x}_{n+1}$ 的概率分布, 并将其形式定义为 softmax 函数:

$$p(\mathbf{x}_{n+1} \mid \mathbf{x}_1, \cdots, \mathbf{x}_n) = \text{Softmax}\left[\mathbf{y}_n\right] \tag{12.32}$$

在给定词嵌入向量 $\mathbf{x}_{n+1}$(独热编码) 的情况下, 网络的参数将通过最小化对数概率或负对数似然来进行训练. 因此, 在给定嵌入序列 $\mathbf{x}_1, \cdots, \mathbf{x}_N$ 的情况下, 训练一个这样的模型的误差函数为

$$E = -\sum_{n=1}^{N} \ln p(\mathbf{x}_{n+1} \mid \mathbf{x}_1, \cdots, \mathbf{x}_n) \tag{12.33}$$

语言模型可以自回归地生成文本, 方法是首先给出初始的起始词或词序列 (该序列可以为空), 并在给定当前序列的情况下, 从式 (12.32)给出的分布中采样下一个词. 然后将这个新词添加到序列中, 并用新的序列再次运行该过程. 这种模型通常被称为仅解码器 (decoder-only)Transformer 或 GPT 模型.GPT 代表生成式预训练 Transformer. 该架构如图 12.15 所示, 算法过程见算法 12.4.

请注意, 该网络需要在所有序列长度 N 上进行训练, 因为我们希望它能够预测长度可变的序列. 实践中经常会固定 N, 然后将不同长度的输入序列填入序列长度 N, 从而处理所有序列. 如果序列中存在大量零向量而不是词嵌入向量, 网络就可能学会预测这些零向量的表示. 因此, 更常见的做法是将特殊的填充 (padding) 向量 ⟨pad⟩ 添加到序列中, 以获得固定的序列长度. 请注意, 在遮掩自注意力机制中, 这些填充向量是无法被关注到的. 此外, 当我们训练这些模型时, 我们通常将语言中自然出现的词序列混合在一起, 这是因为没有真实的目标序列可供比较. 因此, 在误差函数的定义中, 我们直接将输出与下一个词进行比较, 这种方法通常被称为自监督学习 (self-supervised learning), 因为在生成损失函数时, 输入序列中的词本身充当了目标值.

在遮掩注意力层中, 注意系数矩阵 $\mathbf{A}$ 的每行在定义时都必须总和为 1, 其中的元素都是非负的, 并且位于下三角矩阵. 因此, 该矩阵在 $\mathbf{A}$ 中是严格下三角的, 其行和列的序号分别为 $n = 1, \cdots, N$, 如图 12.16 所示.

由于注意力层的计算涉及到的点积是并行计算的, 网络中的多个注意力层也是并行运行的, 因此该架构在硬件利用方面比递归神经网络效率高得多. 此外, 我们还可以从架构本身了解到, 远距离依赖关系可以得到更好的处理, 因为信息不必在每一步都通过递归神经网络的内部状态 $\mathbf{z}_n$ 来传递. 相反, 词之间的依赖关系直接建模.

12.3.2 抽样策略

仅解码器 Transformer 的训练使用了所谓的自监督方法. 给定一组输入词, 我们不需要外部标签, 因为序列中的后续词本身就充当了学习目标. 因此, 我们可以使用任何大型文本语料库来训练这些模型, 例如维基百科或整个互联网. 然后可以使用大量的参数和数据来优化这些模型, 从而创建出具有显著涌现能力的模型.

在生成新文本的阶段, 我们可以从 Transformer 的输出中反复抽样. 例如, 在给定当前文本序列的情况下, 从最可能的后续词中进行选择, 并将这个新词附加到序列中, 然后重复这个过程. 这种方法叫作贪心解码 (greedy decoding), 因为我们总是选择序列中下一个最可能出现的词. 在给定当前序列的情况下, 每个后续词都以概率 $p(\mathbf{x}_{n+1} \mid \mathbf{x}_1, \cdots, \mathbf{x}_n)$ 被采样. 然而, 这种方法的一个问题是, 模型将在每个阶段选择最可能的下一个词, 从而忽略了那些一开始可能看起来不太可能, 但后来会导致非常合理的高质量文本的词序列. 由于这些模型是概率模型, 因此可以通过从概率分布中采样来产生更加真实且更具多样性的文本序列, 从而得到比贪心解码更好的结果.

我们还可以使用更复杂的技术, 例如在采样下一个词之前, 通过将温度参数引入 softmax 函数的参数中, 来缩放概率分布. 这样做的目的是为了调节输出序列的多样性. 我们也可以将注意力限制在某个阈值内最可能的后续词中, 或者使用其他专门的解码算法来提高生成文本的质量.

12.3.3 编码器型 Transformer

仅解码器 Transformer 使用自监督学习来训练网络. 这种模型的一个常见应用是语言建模, 它指的是给定一系列输入词, 预测下一个词. 我们可以使用由网络生成的嵌入序列来完成下游任务, 例如文本分类.

另一种流行的应用程序叫作掩码语言建模 (masked language modeling), 其目标是从序列上下文预测出序列中被掩码的词. 例如, 给定句子 “The capital of France is ⟨mask⟩” , 目标是预测出缺失的词 “Paris” . 这种任务需要一

个双向 (bidirectional) 网络, 其可以访问被掩码词两侧的上下文. 为此, 我们可以在 Transformer 中使用非遮掩的注意力层, 从而允许每个输入词在确定其自身的表示时, 同时使用序列中所有其他词的表示. 这种模型通常称为仅编码器 (encoder-only)Transformer, 它由一系列未遮掩的注意力层组成. 该架构如图 12.18 所示.

我们可以使用大量未标记的文本语料库, 通过自监督学习训练仅编码器 Transformer. 在文本语料库中, 随机选择输入序列中的一些词进行掩码处理, 然后训练网络来预测这些被掩码的词. 我们可以通过训练网络来预测两个相邻输入序列之间的关系, 从而改善网络输出的表示. 这个学习过程也被称为预训练 (pre-training). 预训练完成后, 我们可以将这些网络的输出嵌入向量用作下游网络的输入向量, 并根据少量标记数据进行微调. 这种方法也被称为迁移学习 (transfer learning)(参见 6.3.4 小节).

在自然语言处理领域中, 最著名的仅编码器 Transformer 示例是 BERT, 即来自 Transformer 的双向编码器表示 (Bidirectional Encoder Representations from Transformers)(Devlin et al., 2018).BERT 模型是一个仅解码器 Transformer 架构的示例, 它包括 12 个注意力层 (称为 Transformer 块)、768 个隐藏单元和 12 个注意力头, 总共有 1.1 亿个可学习参数.

12.3.4 序列到序列 Transformer

我们已经看到, 仅解码器 Transformer 是一种自然的、自回归生成文本序列的方法, 它使用了一组并行遮掩自注意力层. 而仅编码器 Transformer 则允许双向输入, 从而可以根据输入序列生成嵌入表示. 同样, 我们可以将这些方法结合起来, 创建一个序列到序列 (sequence-to-sequence) 或编码器-解码器 (encoder-decoder)Transformer, 其中, 模型的编码器一侧是一个非遮掩的注意力 Transformer, 它从输入序列中生成嵌入向量; 而解码器一侧则是一个自回归生成式仅解码器 Transformer. 在这种模型中, 遮掩注意力层的输出会与注意力层的查询和键向量进行额外的注意力层组合, 以预测输出序列中的下一个词. 这种模型的一个早期示例是 T5, 即文本到文本转移 Transformer(Text-to-Text Transfer Transformer)(Raffel et al., 2019). 该架构如图 12.20 所示.

12.3.5 大语言模型

我们已经在 12.1 节中介绍了 Transformer 架构的基本组件. 最初的 Transformer 模型 (Vaswani et al., 2017) 由 6 个编码器层和 6 个解码器层组成. 每个编码器层都有 512 个隐藏单元、8 个注意力头和 2048 个隐藏单元的前馈网络. 每个解码器层都有相同数量的隐藏单元和注意力头, 还有一个额外的注意力层, 用于关注之前由编码器产生的输出. 因此, 总共有 6500 万个可学习参数. 这些模型经过训练后, 可以在多种语言之间进行翻译. 值得注意的是, 这些模型的原始描述现在被认为还相对简单, 但这种简单性正是它们成功的关键.

在后续研究中, 随着计算资源的不断增加和数据集的急剧扩大,Transformer 模型的规模也开始急剧扩大. 缩放假设 (scaling hypothesis) 认为, 只要模型的参数数量和训练数据量都增加, 模型的性能就会得到提高. 对于给定的数据集大小 (以词数为单位), 模型的参数数量有一个最佳数量. 如果可用的参数过多, 模型就会过度拟合训练数据; 而如果可用的参数太少, 模型就无法捕捉到训练数据中的全部结构. 这种关系在 Hoffmann et al. (2022) 研究的各种不同模型中都得到了体现. 这种缩放假设似乎能够相当准确地预测模型的性能, 其中模型的损失函数可以用模型大小的幂律来表示.

在数据有限的情况下, 我们可以对经过预训练的模型进行微调. 然而, 微调模型通常会改变模型参数, 以适应下游任务. 如果我们在多个不同任务上训练一个模型, 就必须保留所有这些参数的副本. 随着 Transformer 模型规模的扩大, 我们希望同时优化参数的使用, 并适应不同的任务. 参数高效微调 (Parameter-Efficient Fine-Tuning, PEFT)(Houlsby et al., 2019) 是一组专门设计的方法, 其目的是用尽可能少的额外可训练参数使经过预训练的 Transformer 模型适应下游任务. 其中一种方法叫作低秩适应 (Low-Rank Adaptation, LoRA)(Hu et al., 2021), 其通过用较小的低秩矩阵的乘积来近似较大的权重矩阵. 这些矩阵如图 12.21 所示.

大语言模型 (Large Language Model, LLM) 是在庞大的文本数据集上进行预训练的, 这些数据集通常包括来自互联网的文本. 然后, 它们可以被用于多种下游任务, 而无须对模型参数进行任何额外的训练. 我们可以通过所谓的提示词 (prompt) 来引导模型解决这些任务. 例如, 为了让大语言模型执行情感分析任务, 我们可以在输入提示词中给出指令和少量示例, 其中包含输入文本和所需的分类输出. 然后, 大语言模型就可以根据用户给出的提示词,

生成对应的输出. 在这种情况下, 用户是在与模型对话 (in-context), 而不是在对模型进行微调 (fine-tuned). 在一些大语言模型中, 用户也可以在模型进行推理时使用提示词来增强模型的输出, 而无须对模型进行任何微调. 这种方法也被称为提示词工程 (prompt engineering).

由于这些模型在互联网规模的文本数据集上进行了训练, 因此它们能够学习到各种有用的模式. 例如, 通过学习如何将文本翻译成不同语言, 这些模型可以在没有任何额外训练的情况下执行翻译任务. 通过提示词, 这些模型还可以执行文本摘要、问答和文本分类等任务. 通过进一步的训练, 大语言模型还可以学习如何在给定提示词的情况下, 生成与用户期望一致的输出.

然而, 这些大语言模型也会产生一些不期望 (undesirable) 的行为, 例如生成幻觉 (hallucination) 或生成不适当的文本. 我们可以通过人工标注的方式来对模型进行训练, 使其生成更符合人类偏好的输出 (Ouyang et al., 2022). 这个过程通常被称为人类反馈强化学习 (Reinforcement Learning from Human Feedback, RLHF)(参见第 19 章).

虽然这些大语言模型主要用于自然语言处理领域, 但我们还是可以将它们应用于其他领域, 例如通过将图像表示为 token, 从而将图像用作输入或输出 (参见 12.4.1 小节).

12.4 多模态 Transformer

虽然最初 Transformer 架构是在自然语言语境下开发的, 但自注意力机制在其他领域也有广泛应用.Transformer 模型已经成功地应用于图像分类、语音识别、文本到语音生成、基因组学和蛋白质建模等问题, 通常在基准测试中表现优于传统模型. 只要将原始输入转换成一系列 token, 我们就可以在各种应用中使用 Transformer. 如果输入本身是一组向量, 我们就能直接使用这些向量; 而如果输入是图像或音频等其他形式的数据, 我们就必须先将其转换成 token 序列.

12.4.1 视觉 Transformer

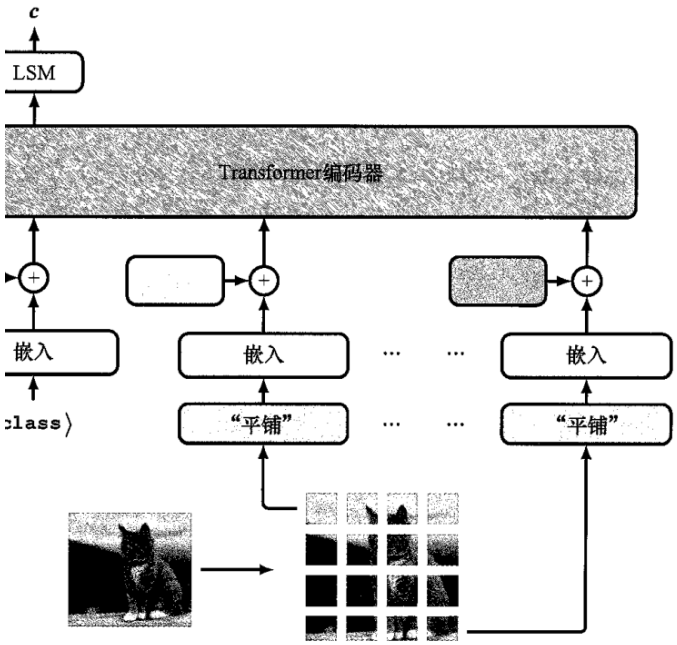
我们将用视觉 Transformer(Vision Transformer, ViT)(Dosovitskiy et al., 2020) 说明如何将 Transformer 应用于图像分类任务. 首先, 我们可以将一幅大小为 224×224 像素、有 3 个颜色通道的图像划分成多个小块, 每个小块的大小为 16×16 像素. 因此, 总共有 $14 \times 14 = 196$ 个小块, 每个小块有 $16 \times 16 \times 3 = 768$ 个像素值. 我们将每个小块展平成一个长度为 768 的向量, 这就是我们的输入 token. 最后, 我们使用一个可学习的线性投影层 (维度为 $768 \times D$), 将这些输入向量映射到一个维度为 D 的嵌入空间. 这样我们就得到了一个由 196 个长度为 D 的向量组成的序列. 由于 Transformer 是对 token 顺序等变化进行等变, 所以我们还为每个图像小块添加了一个可学习的位置嵌入向量 (参见 12.1.9 小节).

除了图像块的 token, 我们还为输入序列增加了一个可学习的分类 token(用 $\mathbf{c}$ 表示), 如图 12.22 所示. 我们将整个序列输入一个仅编码器 Transformer(参见 12.3.3 小节). 这个序列有 196 个图像 token 和 1 个分类 token, 其中分类 token 最终编码了图像的整体表示, 可用于下游分类. 在经过 L 个 Transformer 层处理后, 我们可以从最终输出序列中提取出分类 token, 并使用全连接层进行处理以执行分类任务. 图 12.22 所示的视觉 Transformer 架构使用了 $L = 12$ 层、维度为 $D = 768$ 和 12 个注意力头, 总共有大约 8600 万个可学习参数.

值得注意的是, 虽然视觉 Transformer 是在 ImageNet 数据集上进行预训练的, 但它们在小型数据集上的表现通常不如卷积网络 (例如 ResNet), 因为它们没有内建的图像结构归纳偏置. 当在大型数据集上进行预训练时, 视觉 Transformer 的性能优于卷积网络. 因此, 卷积网络通常可以作为特征提取器, 为 Transformer 提供输入.

12.4.2 图像生成 Transformer

正如可以将句子中的每个词都表示为 token 一样, 我们也可以将图像表示为一系列 token. 这样一来, 我们就能用一个仅解码器 Transformer 对图像进行建模, 从而像语言模型一样自回归生成图像 (Chen et al., 2020). 为此, 我们需要将图像转换成一串离散值, 然后将这些值表示为独热向量. 我们可以使用离散变分自编码器 (DVAE) 或离散归一化流 (NF) 等技术对图像进行编码 (参见第 16 章和第 18 章).



觉 Transformer 的架构。此处将一个可学习的 token <class> 作为额外的输入，
x 激活函数的线性层（记作 LSM）进行转换，从而得到最终的类 - 向量输出 *c*

图 12.2: 用于分类任务的视觉 Transformer 架构. 此处将一个可学习的 token ⟨class⟩ 作为额外的输入, 相关的输出由一个具有 softmax 激活函数的线性层进行转换, 从而得到最终的类-向量输出 **c**.

当我们使用仅解码器 Transformer 来生成图像时, 该模型可以根据给定的提示词 (prompt) 生成高质量的图像. 这里的提示词是描述所需图像的文本. 这与语言模型中根据提示词生成文本的过程类似. 扩散模型 (参见第 20 章) 已成为图像生成的主要方法, 但 Transformer 也可以用于图像生成任务.

12.4.3 音频数据

音频数据也可以表示为 token 序列. 例如, 我们可以将一段录音的声波表示为一个时间序列. 然后, 我们可以使用仅编码器 Transformer 对音频进行建模, 从而完成语音识别任务, 即转录口语 (语音到文本). 我们还可以将音频信号分割成小块, 然后将这些小块表示为 token. 这样一来, 一个仅解码器 Transformer 就可以用于语音合成 (文本到语音) 任务.

12.4.4 文本语音转换

我们已经看到,Transformer 在序列到序列应用 (如语言翻译) 中表现良好, 因此用它们来解决文本语音转换 (Text-To-Speech, TTS) 问题也就不足为奇了, 该问题旨在从文本输入中生成人类语音的音频信号. 首先, 我们需要将音频信号表示为一串 token, 例如梅尔频谱图 (mel spectrogram). 梅尔频谱图是一种频谱图, 其中频率轴是根据梅尔尺度进行缩放的 (参见 4.5 节). 然后, 我们使用一个仅解码器 Transformer 从文本输入中生成梅尔频谱图. 最后, 我们使用声码器 (vocoder) 将梅尔频谱图转换为音频信号. 这种方法已被证明能生成高质量的语音 (Kim et al., 2021).

12.4.5 视觉和语言 Transformer

我们也可以将文本和图像作为输入, 将它们组合成一个序列. 例如, 视觉语言模型 (Vision-Language Model, VLM) 可以根据文本描述生成图像 (Radford et al., 2021), 或者根据图像生成描述 (caption). 这些模型通常使用仅编码器 Transformer, 将文本 token 和图像 token 作为输入.

例如,CLIP(Contrastive Language-Image Pre-training) 模型 (Radford et al., 2021) 使用两个独立的编码器, 分别处理图像和文本. 图像编码器是一个视觉 Transformer(参见 12.4.1 小节), 文本编码器是一个仅解码器 Transformer. 这两个编码器被联合训练, 使配对的图像和文本在嵌入空间中彼此接近, 而不配对的图像和文本则远离. 这种方法使得该模型可以根据图像查询最匹配的文本, 或根据文本查询最匹配的图像, 从而实现强大的零样本 (zero-shot) 图像分类能力.

12.4.6 图像描述

图像描述 (image captioning) 任务旨在根据输入图像生成一段描述性文本. 这可以通过一个序列到序列 Transformer 来实现, 其中图像 token(由视觉编码器产生) 作为输入, 文本描述作为输出 (Liu et al., 2021). 该模型在大量图像-文本对上进行训练, 从而可以为新的图像生成合理的描述. 图 12.27 给出了图像描述的示例.

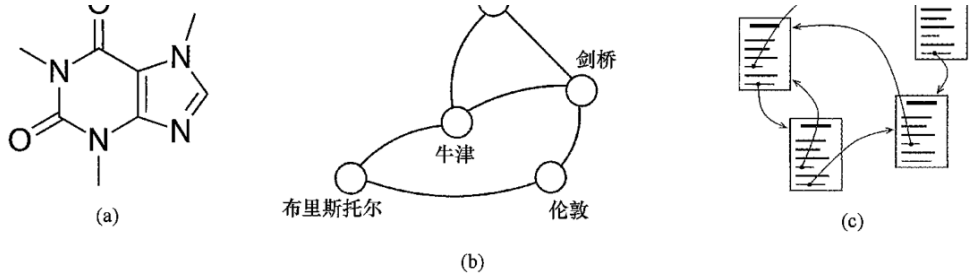
第 13 章 图神经网络

13.1 基于图的机器学习

在前几章中, 我们学习了以序列和图像形式出现的结构化数据, 分别对应于用一维数组和二维数组存储的变量. 在更广泛的应用中, 有许多类型的结构化数据更适合通过图来描述, 如图 13.1所示. 通常, 一个图由一组称为节点的对象组成, 它们通过边来连接. 节点和边都可以与数据相关联. 例如, 在一个分子中, 节点和边分别对应于不同原子类型 (碳原子、氮原子、氢原子等) 和不同化学键类型 (单键、双键等) 的离散变量相关联. 对于铁路网络, 每条铁路线可能与两个城市间平均旅行时间的连续变量相关联. 这里我们假设边是对称的, 例如从伦敦到剑桥的旅行时间与从剑桥到伦敦的旅行时间相同. 这样的边可以表示为节点之间的无向连接. 对于万维网而言, 边是有向的, 因为页面 A 上有指向页面 B 的超链接, 并不一定意味着页面 B 上有指回页面 A 的超链接.

其他图结构化数据的例子有: 蛋白质相互作用网络, 其中节点是蛋白质, 边表示蛋白质对之间相互作用的强度; 电路图, 其中节点是组件, 边是导线; 社交网络, 其中节点是人, 边是“友谊”. 此外, 还有可能存在更复杂的图结构, 例如公司内部的知识图谱包含多种不同类型的节点 (如人员、文档和会议), 以及多种类型的边用于描述不同的属性 (如一个人出席会议或一个文档引用另一个文档).

在本章中, 我们将探索如何将深度学习应用于图结构化数据. 在讨论图像时, 我们已经遇到了结构化数据的一个例子. 在图像中, 数据向量 $\boldsymbol{x}$ 的单个元素对应于规则网格上的像素. 因此, 图像是图结构化数据的特殊实例, 其中节点是像素, 边描述了哪些像素是相邻的. 卷积神经网络 (CNN) 考虑了这种结构 (参见第 10 章), 并纳入了像素之间相对位置的先验知识及一些保持属性的等变性操作 (如分割), 以及保持属性的不变性的操作 (如分类). 我们将以图像的卷积神经网络作为启发, 构建更通用的称为图神经网络的深度学习方法 (Zhou et al., 2018; Wu et al., 2019; Hamilton, 2020; Veličković, 2023). 我们将看到, 把深度学习应用于图结构化数据时的一个关键思路是确保对图中节点重新排序的等变性或不变性.



(a) (b) (c)

图 13.1: 图结构化数据的 3 个例子:(a) 咖啡因分子由化学键连接的原子组成;(b) 由铁路线连接的城市构成的铁路网络;(c) 由超链接连接的页面构成的万维网.

我们可以使用图结构化数据来完成许多类型的应用, 这些应用可以根据目标是预测节点、边还是整个图的属性来大致分组. 节点预测 (node prediction) 的一个例子是根据文档之间的超链接和引用关系对文档进行主题分类.

对于边来说, 举个例子, 我们可能已知蛋白质网络中的一些相互作用, 想要预测其他潜在的相互作用. 这类任务称为边预测 (edge prediction) 或图补全 (graph completion) 任务. 还有一些任务, 其中边是已知的, 我们的目标是在图中发现簇或“社群”.

最后一类是预测与整个图相关的属性. 例如, 我们可能希望预测特定分子是否能溶解于水. 这里, 我们得到的不是单个图, 而是不同图的数据集, 我们可以把它们看成从某个共同分布抽取而来, 换句话说, 我们假设图本身是独立同分布的 (independent and identically distributed). 这类任务可以视为图回归 (graph regression) 或图分类 (graph classification) 任务.

图神经网络为每个节点定义了一个嵌入向量 (embedding vector), 该嵌入向量通常使用观测到的节点属性进行初始化, 然后通过一系列可学习层进行转换, 以创建学习表示. 这与词嵌入或 token 在 Transformer 中的处理方式类似, 通过一系列层的处理 (参见第 12 章), 便可以更好地捕获单词在文本其余部分上下文中的语义. 图神经网

络还可以学习表示图中每条边的特征向量 (边嵌入) 以及整个图的全局特征向量 (图嵌入).

除了直接完成预测任务外, 我们还可以在图上使用深度学习来发现有用的内部表示, 这些表示随后可以促进一系列下游任务的完成. 这称为图表示学习 (graph representation learning). 例如, 我们可以通过在大量分子结构语料库上训练深度学习系统来寻求构建分子的基础模型 (foundation model). 这样的模型一旦训练成功, 我们就可以通过使用小型标注数据集对它进行微调以完成各种特定任务.

作为节点级预测任务的一个标准例子, 假设我们要区分一个大型社交网络中的每个用户节点是真实人物还是自动化机器人. 在一个庞大且不断变化的社交网络中逐一核实并标注每个节点是不可行的, 我们只能手工标注少数节点. 因此, 在训练期间, 我们可以访问包含一部分节点标签的整个图, 并试图预测剩余节点的标签. 这是半监督学习的一种形式. 对于分子溶解性分类的例子, 我们可能会得到一个带标签的分子训练集, 以及需要预测溶解性的新分子测试集. 这是我们在前面章节中多次见到的典型归纳 (inductive) 任务. 然而, 另一些图预测任务是直推式 (transductive) 的, 我们得到整个图的结构以及一些节点的标签, 要去预测剩余节点的标签.

13.1.1 图的属性

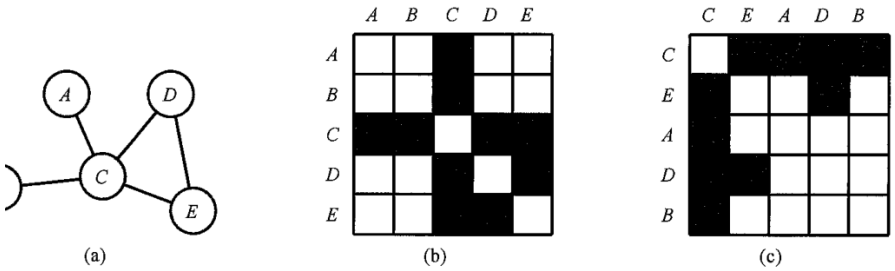
在本章中, 我们将关注简单图, 其中任意两个节点之间最多有一条边, 边是无向的, 并且没有连接到自身节点的环. 这足以让我们介绍图神经网络的关键概念, 同时也涵盖了图神经网络广泛的实际应用. 这些概念之后可以应用于更复杂的图结构.

我们首先引入与图相关的一些符号, 并定义一些重要的属性. 一个图 $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ 由一组节点或顶点构成, 用 $\mathcal{V}$ 表示, 其中还包含一组边或连接, 用 $\mathcal{E}$ 表示. 我们用 $n = 1, \cdots, N$ 来索引节点, 并将从节点 n 到节点 m 的边写为 (n, m). 如果两个节点被一条边连接, 则称它们为邻居节点, 所有节点 n 的邻居节点的集合用 $\mathcal{N}(n)$ 表示.

除了图结构, 通常还有与节点相关联的观测数据. 对于每个节点 n, 我们可以将相应的节点变量表示为一个 D 维的列向量 $\boldsymbol{x}_n$, 将这些变量组合成一个 $N \times D$ 维的数据矩阵 $\boldsymbol{X}$, 其中第 n 行由 $\boldsymbol{x}_n^{\mathsf{T}}$ 给出. 图中的边也可能与数据变量相关联, 但在当下我们只关注节点变量 (参见小节 13.3.2).

13.1.2 邻接矩阵

指定图中边的一种便捷方式是使用邻接矩阵 (adjacency matrix), 用 $\boldsymbol{A}$ 表示. 要定义邻接矩阵, 我们首先需要为节点分配一个顺序. 如果图中有 N 个节点, 我们可以使用 $n = 1, \cdots, N$ 来索引它们. 邻接矩阵的维度是 $N \times N$, 在其中的每个元素位置 n, m, 只要从节点 n 到节点 m 的边存在, 就在该处填入 1, 所有其他元素位置都填入 0. 对于具有无向边的图, 邻接矩阵是对称的, 因为存在从节点 n 到节点 m 的边意味着也存在从节点 m 到节点 n 的边, 对于所有的 n 和 m, 都有 $A_{nm} = A_{mn}$. 邻接矩阵的一个例子如图 13.2所示.



邻接矩阵的一个例子: (a) 一个包含 5 个节点的图; (b) 对应于特定节点顺序的邻接矩阵; (c) 对应于另一

图 13.2: 邻接矩阵的一个例子:(a) 一个包含 5 个节点的图;(b) 对应于特定节点顺序的邻接矩阵;(c) 对应于另一不同节点顺序的邻接矩阵.

邻接矩阵定义了图的结构, 我们可以考虑直接将其作为神经网络的输入. 为此, 我们可以 “展平” 邻接矩阵, 例如将列连接成一个长的列向量. 然而, 这种方法的一个主要问题在于邻接矩阵取决于节点排序的任意选择, 如图 13.2所示. 假设我们想要预测一个分子的溶解性, 这显然不应该取决于写下邻接矩阵时给节点分配的顺序. 由

于节点的排序方式的数量随节点数的增加而阶乘式增长, 尝试通过使用大数据集或数据增强来学习置换不变性是不切实际的. 相反, 在构建网络架构时, 我们应当将这种不变性视为归纳偏置.

13.1.3 置换等变性

我们可以引入置换矩阵 $\boldsymbol{P}$ 的概念, 以便在数学上表达节点标签的排列. 置换矩阵与邻接矩阵大小相同, 并指定了节点排序的特定排列. 置换矩阵在每行和每列中各含有一个 1, 其余元素都是 0, 于是置换矩阵中元素位置 n, m 的 1 表示节点 n 在重新排列后将标记为 m. 例如, 考虑从 (A, B, C, D, E) 到 (C, E, A, D, B) 的排列, 它对应于图 13.2中的两种节点排序选择.

相应的置换矩阵形式为 (见习题 13.1)

$$\boldsymbol{P} = \begin{pmatrix} 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 & 0 \end{pmatrix}. \tag{13.1}$$

我们可以更加正式地定义置换矩阵如下. 首先引入标准单位向量 $\boldsymbol{u}_n$, 其中 $n = 1, \cdots, N$. 这是一个列向量, 其中除了第 n 个元素等于 1 之外, 其他所有元素都等于 0. 在这种表示法中, 单位矩阵如下所示:

$$\boldsymbol{I} = \begin{pmatrix} \boldsymbol{u}_1^{\mathsf{T}} \\ \boldsymbol{u}_2^{\mathsf{T}} \\ \vdots \\ \boldsymbol{u}_N^{\mathsf{T}} \end{pmatrix}. \tag{13.2}$$

引入一个置换函数 $\pi(\cdot)$, 将 n 映射到 $m = \pi(n)$. 对应的置换矩阵如下所示:

$$\boldsymbol{P} = \begin{pmatrix} \boldsymbol{u}_{\pi(1)}^{\mathsf{T}} \\ \boldsymbol{u}_{\pi(2)}^{\mathsf{T}} \\ \vdots \\ \boldsymbol{u}_{\pi(N)}^{\mathsf{T}} \end{pmatrix}. \tag{13.3}$$

当我们重新排序一个图的节点上的标签时, 对应的节点数据矩阵 $\boldsymbol{X}$ 的效果是根据 $\pi(\cdot)$ 进行置换操作, 这可以通过在其前面乘上 $\boldsymbol{P}$ 来实现 (见习题 13.4), 即

$$\widetilde{\boldsymbol{X}} = \boldsymbol{P}\boldsymbol{X}. \tag{13.4}$$

对于邻接矩阵, 行和列都会被置换. 同样, 行可以通过在前面乘上 $\boldsymbol{P}$ 来置换, 而列则可以通过在后面乘上 $\boldsymbol{P}^{\mathsf{T}}$ 来置换 (见习题 13.5), 从而得到一个新的邻接矩阵:

$$\widetilde{\boldsymbol{A}} = \boldsymbol{P}\boldsymbol{A}\boldsymbol{P}^{\mathsf{T}}. \tag{13.5}$$

当我们将深度学习应用于图结构化数据时, 需要以数值形式表示图结构, 这样才能将它们输入神经网络, 这就要求我们为节点指定顺序. 然而, 我们选择的具体节点排序是任意的, 所以必须确保图的任何全局属性不依赖于这种排序方式. 换言之, 网络预测必须对节点标签的重新排序具有不变性, 即

$$y(\widetilde{\boldsymbol{X}}, \widetilde{\boldsymbol{A}}) = y(\boldsymbol{X}, \boldsymbol{A}) \qquad \text{不变性} \tag{13.6}$$

其中 $y(\cdot, \cdot)$ 是网络的输出.

有些情况下, 我们要对单个节点进行预测. 在这种情况下, 如果我们重新排序节点标签, 则相应的预测应显示相同的次序, 这样不论如何选择顺序, 给定的预测总是与同一个节点相关联. 换言之, 节点级预测应该对节点标签重新排序保持等变性. 即

$$\boldsymbol{y}(\widetilde{\boldsymbol{X}}, \widetilde{\boldsymbol{A}}) = \boldsymbol{P}\,\boldsymbol{y}(\boldsymbol{X}, \boldsymbol{A}) \qquad \text{等变性} \tag{13.7}$$

其中 $\boldsymbol{y}(\cdot, \cdot)$ 是网络输出的一个向量, 每个节点对应其中的一个元素.

13.2 神经信息传递

确保节点标签置换下的不变性或等变性是我们将深度神经网络应用于图结构化数据时的一个关键考虑. 另一个关键考量是我们想要利用深度神经网络的表示能力, 所以我们保留了“层”的概念, 即可以对每一层进行重复的计算变换. 如果网络的每一层都在节点重新排序后保持等变性, 那么多层叠加后的网络也将保持等变性, 同时网络的每一层都受益于图结构信息.

对于输出代表节点级预测的网络, 整个网络将按要求表现出等变性. 如果网络用来预测图级属性, 那么可以在网络的最后添加一个对输入的排列顺序不敏感 (置换不变) 的层. 我们还想确保网络的每一层都是一个高度灵活的非线性函数, 并且对其参数具有可微分性, 以便可以使用自动微分计算梯度, 从而通过随机梯度下降进行训练.

图的规模各不相同. 例如, 不同的分子可以有不同数量的原子, 因此标准神经网络使用固定长度表示是不合适的. 因此, 网络应该能够处理变长输入, 正如我们在 Transformer 网络中看到的那样 (参见第 12 章). 一些图可能非常大, 例如有数百万参与者的社交网络, 所以我们也希望构建具有良好扩展性的模型. 为了将不变性和等变性内置于网络架构, 并促进模型在大型图上的应用, 参数共享将发挥重要作用, 这一点毋庸置疑.

13.2.1 卷积滤波器

为了开发符合所有这些要求的框架, 我们可以从使用卷积神经网络的图像处理中寻找灵感 (参见第 10 章). 首先我们注意到, 图像可以视为图结构化数据的一个特定实例, 在这个实例中, 节点是像素, 边代表图像中相邻的像素对, 这里的相邻包括对角相邻的节点以及水平或垂直相邻的节点.

在卷积网络中, 我们对图像域进行逐层变换, 某个层的一个像素的状态值是由前面层的相关像素的状态值通过计算决定的, 用于计算的局部函数称为滤波器 (参见 10.2 节). 考虑一个使用 3×3 滤波器的卷积层, 如图 13.3(a) 所示. 单个的滤波器在第 $(l+1)$ 层的一个像素上执行的计算可以表示为

$$z_i^{(l+1)} = f\left(\sum_j w_j z_j^{(l)} + b\right) \tag{13.8}$$

其中 $f(\cdot)$ 是一个可微分的非线性激活函数, 比如 ReLU 函数, 而对 j 的求和则是对第 l 层中一个小块内的所有 9 个像素进行的. 相同的函数可以应用于图像中的多个小块, 因此权重 w_j 和偏置 b 在各个小块间共享 (因此它们不带索引 i).

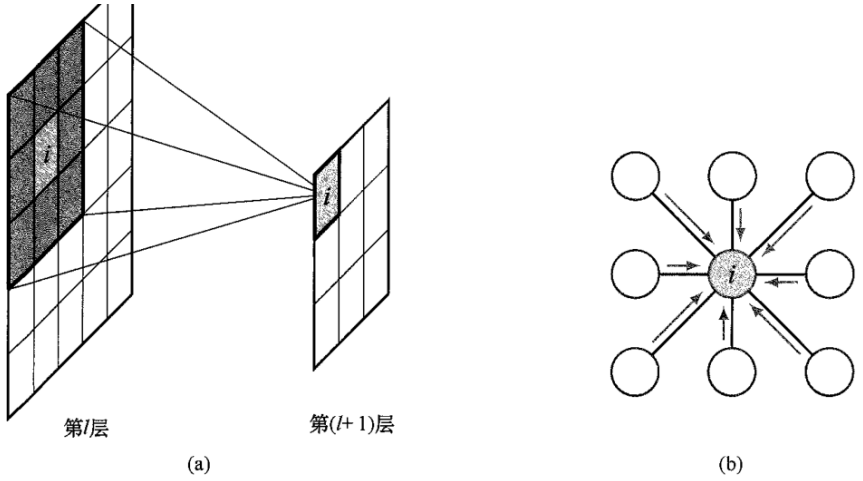


图 13.3: 图像的卷积滤波器可以表示为一种图结构计算:(a) 在深度卷积网络的第 $(l+1)$ 层, 节点 i 的值是由第 l 层局部像素区域激活值通过滤波器函数计算得来的;(b) 将同样的计算结构以图的形式表达, “消息” 从节点 i 的邻居节点流向节点 i.

式(13.8)在第 l 层节点置换下不是等变的, 因为权重向量中的 w_j 元素在排列上不是不变的. 然而, 我们可以通过进行一些简单的修改来实现等变性, 具体步骤如下. 首先将滤波器视为一个图, 如图 13.3(b) 所示, 并且从中分离

出来自节点 i 的贡献. 其他 8 个节点构成节点 i 的邻居节点集合 $\mathcal{N}(i)$. 然后假设单个权重参数 w_{neigh} 在邻居节点间共享, 因此有

$$z_i^{(l+1)} = f\left(w_{\text{neigh}} \sum_{j\in\mathcal{N}(i)} z_j^{(l)} + w_{\text{self}}\, z_i^{(l)} + b\right) \tag{13.9}$$

其中节点 i 有自己的权重参数 w_{self}.

我们可以将式(13.9)解释为通过从邻居节点收集信息来更新节点 i 处的局部表示 z_i, 这通过从邻居节点传递给节点 i 的消息来实现. 在这种情况下, 消息直接是其他节点的激活状态. 随后将这些消息与节点 i 的信息结合, 并使用非线性函数进行转换. 来自邻居节点的信息通过式(13.9)中的简单求和聚合起来, 这显然对于与这些节点相关联的标签的任何排列都是不变的. 将以上操作同步应用于图中的每个节点, 因此如果节点被排序, 则最终的计算将保持不变, 但它们也将被相应地置换, 因此这个计算在节点重新排序下是等变的. 注意, 上述推导依赖于参数 w_{neigh}、w_{self} 和 b 在所有节点之间共享这一假设.

13.2.2 图卷积网络

我们以卷积操作为例, 构建用于处理图结构数据的深度神经网络模型. 我们的目标是定义一种灵活的、非线性的节点嵌入转换, 它对一组权重和偏置参数可微分, 并且将第 l 层中的变量映射到第 $(l+1)$ 层的相应变量. 对于图中的每个节点 n 以及网络中的每一层 l, 引入一个 D 维的列向量 $\boldsymbol{h}_n^{(l)}$ 作为节点嵌入变量, 其中 $n = 1, \cdots, N$ 且 $l = 1, \cdots, L$.

式(13.9)给出的转换首先从邻居节点收集和合并信息, 然后用当前节点嵌入和传入消息的函数更新节点. 因此, 我们可以将每一层的处理视为两个连续阶段. 首先是聚合阶段, 对于每个节点 n, 将消息从其邻居节点传递到该节点, 并以置换不变的方式组合成一个新的向量 $\boldsymbol{z}_n^{(l)}$. 然后是更新阶段, 将邻居节点的聚合信息与节点本身的局部信息结合, 计算出该节点更新后的嵌入向量.

考虑图中的一个特定节点 n. 首先聚合来自节点 n 的所有邻居节点的节点向量:

$$\boldsymbol{z}_n^{(l)} = \text{Aggregate}\left(\{\boldsymbol{h}_m^{(l)} : m \in \mathcal{N}(n)\}\right) \tag{13.10}$$

然后使用另一种操作来更新节点 n 处的嵌入向量:

$$\boldsymbol{h}_n^{(l+1)} = \text{Update}\left(\boldsymbol{h}_n^{(l)}, \boldsymbol{z}_n^{(l)}\right). \tag{13.11}$$

如果聚合函数对变化的邻居节点数量有良好的定义, 并且不依赖于这些节点的排序, 那么该聚合函数的形式将非常灵活. 只要它是一个对这些参数可微分的函数, 便于进行梯度下降训练, 它就可以包含可学习参数. 同样, 这可以是一组可学习参数的可微分函数. 并行地对图中的每个节点执行聚合操作, 然后执行更新操作. 节点嵌入通常使用观测到的节点数据初始化, 因此 $\boldsymbol{h}_n^{(0)} = \boldsymbol{x}_n$.

注意, 每一层通常都有自己的参数, 尽管参数也可以跨层共享. 这个框架称为消息传递神经网络 (message-passing neural network)(Gilmer et al., 2017), 算法 13.1 对它做了总结.

13.2.3 聚合算子

聚合函数有许多可能的形式, 但它必须仅依赖于输入的集合, 而不依赖于输入的顺序. 它还必须是任何可学习参数的可微分函数. 根据式(13.9), 最简单的聚合函数如下:

$$\text{Aggregate}\left(\{\boldsymbol{h}_m^{(l)} : m \in \mathcal{N}(n)\}\right) = \sum_{m\in\mathcal{N}(n)} \boldsymbol{h}_m^{(l)}. \tag{13.12}$$

简单求和显然独立于邻居节点的排序, 并且无论邻居节点集合中有多少节点, 它都是明确定义的. 注意这里没有可学习参数.

相对于那些邻居节点较少的节点, 拥有众多邻居的节点会被求和操作赋予更强的影响力, 这可能导致数值计算问题, 特别是在像社交网络这样的应用中, 邻居节点集合的大小可能会相差好几个数量级. 这种方法的一个变体

是将聚合操作定义为邻居嵌入向量的平均值, 即

$$\text{Aggregate}\Big(\{\boldsymbol{h}_m^{(l)} : m \in \mathcal{N}(n)\}\Big) = \frac{1}{|\mathcal{N}(n)|} \sum_{m \in \mathcal{N}(n)} \boldsymbol{h}_m^{(l)} \tag{13.13}$$

其中 $|\mathcal{N}(n)|$ 表示邻居节点集合 $\mathcal{N}(n)$ 中的节点数量. 然而, 这种归一化抛弃了关于网络结构的信息, 并且被证明比简单的求和 (Hamilton, 2020) 要弱, 因此是否使用它取决于节点特征和图结构相比谁更重要.

这种方法的另一个变体 (Kipf and Welling, 2016) 则考虑到了每个节点的邻居的数量:

$$\text{Aggregate}\Big(\{\boldsymbol{h}_m^{(l)} : m \in \mathcal{N}(n)\}\Big) = \sum_{m \in \mathcal{N}(n)} \frac{\boldsymbol{h}_m^{(l)}}{\sqrt{|\mathcal{N}(n)|\,|\mathcal{N}(m)|}}. \tag{13.14}$$

最后一种可能性是取邻居嵌入向量的逐元素最大值 (或最小值), 这同样满足所需特性, 即对数量可变的邻居节点能作出良好定义, 并且和邻居节点顺序无关.

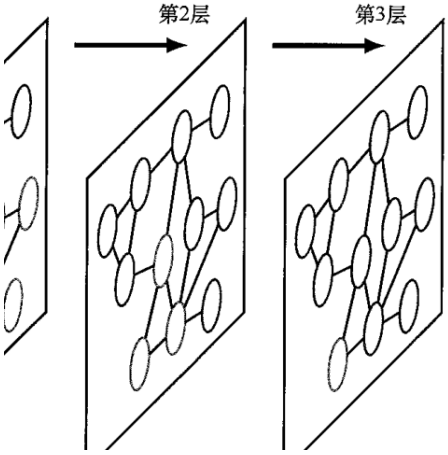


图 13.4: 通过图神经网络连续层的信息流示意图. 在第 3 层中, 只有一个节点以红色突出显示. 该节点从前一层的两个邻居节点接收信息, 而这两个邻居节点又从第 1 层的邻居节点接收信息. 就像用于图像的卷积神经网络一样, 有效感受野——对应于显示为红色的节点数量——会随着处理层数的增加而扩大.

图神经网络中给定层的每个节点通过聚合前一层的邻居节点信息来更新, 这就出现了感受野 (receptive field) 的概念, 类似于第 10 章中 CNN 所使用的滤波器的感受野. 随着信息通过连续的层被处理, 对当前层某个节点的更新就会依赖于越来越多的早期层的其他节点, 直到有效的感受野几乎覆盖整个图为止, 如图 13.4所示. 然而, 大的稀疏图可能需要非常多的层, 才能使每个输出受每个输入的影响. 因此, 有些架构引入了一个额外的 “超级节点”, 它直接连接到原始图中的每个节点, 使信息得到快速传播.

迄今为止我们所讨论的聚合算子都没有可学习参数. 如果我们在组合邻居节点的嵌入向量之前, 先使用一个多层神经网络 (用 MLP_ϕ 表示) 对每个嵌入向量进行变换, 那么我们就可以引入这样的参数. 其中 MLP 代表 “多层感知机”, 其下标 ϕ 表示网络的参数. 只要网络具有跨节点共享的结构和参数值, 那么这个聚合算子就是排列不变的. 我们还可以使用另一个参数为 θ 的神经网络 MLP_θ 来转换组合向量, 从而得到如下一般性的聚合操作:

$$\text{Aggregate}\Big(\{\boldsymbol{h}_m^{(l)} : m \in \mathcal{N}(n)\}\Big) = \text{MLP}_\theta\left(\sum_{m \in \mathcal{N}(n)} \text{MLP}_\phi(\boldsymbol{h}_m^{(l)})\right) \tag{13.15}$$

其中 MLP_ϕ 和 MLP_θ 在网络的第 l 层是共享的. 由于 MLP 的灵活性, 式(13.15)定义的转换代表了任何置换不变函数的通用逼近器, 该函数能将一组嵌入映射到单个嵌入 (Zaheer et al., 2017). 注意, 这里的求和操作可以替换为其他的置换不变函数, 例如平均值, 逐元素取最大值或最小值.

如果我们考虑一个没有边的图, 它就退化成了一个简单的无结构节点集合, 这时的图神经网络就成为一个特例. 在这种情况下, 如果我们对集合中的每个向量 $\boldsymbol{h}_n^{(l)}$ 应用式(13.15), 其中求和是在除 $\boldsymbol{h}_n^{(l)}$ 外的所有其他向量上进行的, 那么我们就有了一个关于无结构变量集合 [称为深度集合 (deep set)] 的学习函数的通用框架.

13.2.4 更新算子

在选择了合适的聚合算子之后, 我们同样需要决定更新算子的形式. 类似于 CNN 的公式(13.9), 一种简单的更新算子形式如下:

$$\text{Update}\left(\boldsymbol{h}_n^{(l)}, \boldsymbol{z}_n^{(l)}\right) = f\left(\boldsymbol{W}_{\text{self}}\boldsymbol{h}_n^{(l)} + \boldsymbol{W}_{\text{neigh}}\boldsymbol{z}_n^{(l)} + \boldsymbol{b}\right) \tag{13.16}$$

其中 $f(\cdot)$ 是一个非线性激活函数, 例如可以逐元素应用到向量参数上的 ReLU 函数;$\boldsymbol{W}_{\text{self}}$ 和 $\boldsymbol{W}_{\text{neigh}}$ 是可学习权重;$\boldsymbol{b}$ 是偏置;$\boldsymbol{z}_n^{(l)}$ 由式(13.10)定义.

如果我们选择简单的求和 [式(13.12)] 作为聚合函数, 并且在节点及其邻居节点之间共享相同的权重矩阵, 使得 $\boldsymbol{W}_{\text{self}} = \boldsymbol{W}_{\text{neigh}}$, 我们就能得到如下特别简单的更新算子形式:

$$\boldsymbol{h}_n^{(l+1)} = \text{Update}\left(\boldsymbol{h}_n^{(l)}, \boldsymbol{z}_n^{(l)}\right) = f\left(\boldsymbol{W}_{\text{neigh}} \sum_{m\in\mathcal{N}(n),n} \boldsymbol{h}_m^{(l)} + \boldsymbol{b}\right). \tag{13.17}$$

消息传递算法通常通过设置 $\boldsymbol{h}_n^{(0)} = \boldsymbol{x}_n$ 来初始化. 然而, 有时我们可能希望每个节点都有一个内部表示向量, 其维度不同于 $\boldsymbol{x}_n$ 的维度. 这样的表示可以通过用额外的零填充节点向量 $\boldsymbol{x}_n$(以达到更高的维度), 或者简单地使用可学习的线性变换将节点向量转换到所需维度的空间来初始化. 当节点没有与之关联的数据变量时, 一种替代的初始化形式是使用一个标记了每个节点度数 (即邻居节点数量) 的独热向量.

总的来说, 我们可以将图神经网络表示为一系列连续变换节点嵌入的层. 如果我们将这些嵌入分组到一个矩阵 $\boldsymbol{H}$ 中, 其第 n 行是向量 $\boldsymbol{h}_n^{\mathsf{T}}$, 并初始化为数据矩阵 $\boldsymbol{X}$, 则可以将这些连续的变换写成以下形式:

$$\boldsymbol{H}^{(1)} = \boldsymbol{F}\left(\boldsymbol{X}, \boldsymbol{A}, \boldsymbol{W}^{(1)}\right) \tag{13.18}$$

$$\boldsymbol{H}^{(2)} = \boldsymbol{F}\left(\boldsymbol{H}^{(1)}, \boldsymbol{A}, \boldsymbol{W}^{(2)}\right) \tag{13.19}$$

$$\vdots$$

$$\boldsymbol{H}^{(L)} = \boldsymbol{F}\left(\boldsymbol{H}^{(L-1)}, \boldsymbol{A}, \boldsymbol{W}^{(L)}\right) \tag{13.20}$$

其中 $\boldsymbol{A}$ 是邻接矩阵,$\boldsymbol{W}^{(l)}$ 表示网络第 l 层中全部权重和偏置的集合. 在置换矩阵 $\boldsymbol{P}$ 定义的节点重新排序下, 第 l 层计算的节点嵌入的变换是等变的:

$$\boldsymbol{P}\boldsymbol{H}^{(l)} = \boldsymbol{F}\left(\boldsymbol{P}\boldsymbol{H}^{(l-1)}, \boldsymbol{P}\boldsymbol{A}\boldsymbol{P}^{\mathsf{T}}, \boldsymbol{W}^{(l)}\right). \tag{13.21}$$

因此, 整个网络计算出一个等变的转换 (见习题 13.7).

13.2.5 节点分类

图神经网络可以看作一系列层, 其中的每一层都将一组节点嵌入向量 $\{\boldsymbol{h}_n^{(l)}\}$ 转换成同样大小和维度的新向量集合 $\{\boldsymbol{h}_n^{(l+1)}\}$. 在网络的最后一个卷积层之后, 我们需要得到预测结果, 以便定义一个用于训练的损失函数, 以及使用训练好的网络对新数据进行预测.

考虑对图中节点分类的任务, 这是图神经网络最常见的用途之一. 我们可以定义一个输出层, 有时称为读出层 (readout layer), 该层为每个节点计算一个与 C 个分类相对应的 softmax 函数, 形式如下:

$$y_{ni} = \frac{\exp\left(\boldsymbol{w}_i^{\mathsf{T}}\boldsymbol{h}_n^{(L)}\right)}{\sum_j \exp\left(\boldsymbol{w}_j^{\mathsf{T}}\boldsymbol{h}_n^{(L)}\right)} \tag{13.22}$$

其中 $\{\boldsymbol{w}_i\}$ 是一组可学习的权重向量,$i = 1, \cdots, C$. 我们可以将损失函数定义为所有节点和所有类别的交叉熵损失之和:

$$\mathcal{L} = -\sum_{n\in\mathcal{V}_{\text{train}}} \sum_{i=1}^{C} t_{ni} \ln y_{ni} \tag{13.23}$$

其中 t_{ni} 是目标值, 每个 n 值都有一个独热编码. 因为权重向量 $\boldsymbol{w}_i$ 在输出节点间是共享的, 且输出 y_{ni} 对于节点排序是等变的, 因此损失函数 [式(13.23)] 是不变的. 如果目标是在输出处预测连续值, 则可以将简单的线性变换与平方和误差结合起来, 来定义一个合适的损失函数.

式(13.23)中对 n 的求和是在用于训练的节点子集 $\mathcal{V}_{\text{train}}$ 上进行的. 我们需要区分以下 3 种类型的节点.

(1) 节点子集 $\mathcal{V}_{\text{train}}$ 中的节点是有标签的, 既参与图神经网络的消息传递操作, 也用于计算训练的损失函数.

(2) 可能还有一个名为 $\mathcal{V}_{\text{trans}}$ 的节点子集, 里面是未标记的直推式 (transductive) 节点并且不参与用于训练的损失函数的计算. 然而, 它们仍然参与训练和推理阶段的消息传递, 它们的标签可能会作为推理过程的一部分被预测.

(3) 剩余的节点标记为 $\mathcal{V}_{\text{induct}}$, 是一组归纳节点, 它们不用于计算损失函数, 并且这些节点及其关联的边在训练阶段不参与消息传递. 然而, 它们在推理阶段参与消息传递, 它们的标签预测为推理的结果.

如果没有直推式节点, 即测试节点 (及其关联的边) 在训练阶段不可用, 那么通常将这种训练称为归纳学习 (inductive learning), 这可以视为一种监督学习 (supervised learning) 的形式. 然而, 如果存在直推式节点, 则称这种训练为直推式学习 (transductive learning), 这可以视为半监督 (semi-supervised) 学习的一种形式.

13.2.6 边分类

在某些应用中, 我们希望对图的边而不是节点进行预测. 一种常见的边分类任务是边补全. 在这种任务中, 目标是确定两个节点之间是否应该存在一条边. 给定一组节点嵌入, 可以使用嵌入向量之间的点积, 通过逻辑斯谛 sigmoid 函数定义两个节点 n 和 m 之间存在边的概率 $p(n, m)$:

$$p(n, m) = \sigma\left(\boldsymbol{h}_n^{\mathsf{T}} \boldsymbol{h}_m\right). \tag{13.24}$$

一个典型的应用实例是, 预测社交网络中任意两个用户是否拥有共同的兴趣爱好, 并愿意建立联系.

13.2.7 图分类

在图神经网络的某些应用中, 我们要在给定一组带标签的训练图 $\mathcal{G}_1, \cdots, \mathcal{G}_N$ 的情况下, 预测新图的属性. 这要求我们以不依赖于任意节点排序的方式组合最终层的嵌入向量, 从而确保输出的预测结果与节点的排序无关. 这任务有点像聚合函数, 只不过这里考虑的是图中所有节点, 而不仅仅是单个节点的邻域集. 最简单的方法是对节点嵌入向量进行求和:

$$y = f\left(\sum_{n \in \mathcal{V}} \boldsymbol{h}_n^{(L)}\right) \tag{13.25}$$

其中, 函数 f 可能包含可学习参数, 如线性变换或神经网络. 也可以使用其他不变的聚合函数, 例如平均值、逐元素的最小值或最大值.

对于分类问题, 例如将候选药物分子标记为有毒或安全, 通常采用交叉熵损失函数; 而对于回归问题, 例如预测候选药物分子的溶解度, 则通常采用平方误差损失函数. 图级别的预测对应于归纳任务, 因为必须有不同的图集合用于训练和推理.

13.3 通用图网络

迄今为止我们所考虑的图网络有许多变体和扩展. 这里我们概述一些关键概念以及一些实践考虑.

13.3.1 图注意力网络

注意力机制在作为 Transformer 架构的基础时展现出强大的能力 (参见 12.1 节). 在图神经网络中, 注意力机制可以用于构建一种能够整合来自邻居节点信息的聚合函数. 在这种机制中, 传入的消息会根据注意力系数 A_{nm} 加权, 从而得到

$$\boldsymbol{z}_n^{(l)} = \text{Aggregate}\left(\{\boldsymbol{h}_m^{(l)} : m \in \mathcal{N}(n)\}\right) = \sum_{m \in \mathcal{N}(n)} A_{nm}\, \boldsymbol{h}_m^{(l)} \tag{13.26}$$

其中, 注意力系数满足

$$A_{nm} \geq 0 \tag{13.27}$$

$$\sum_{m \in \mathcal{N}(n)} A_{nm} = 1. \tag{13.28}$$

这种网络称为图注意力网络 (graph attention network)(Veličković et al., 2017), 它可以捕获一种归纳偏置, 该归纳偏置表明在确定最佳更新的过程中, 某些相邻节点比其他节点更重要, 而这种重要性取决于数据本身.

构建注意力系数的方法有很多种, 但通常都会使用 softmax 函数. 例如, 我们可以使用双线性形式

$$A_{nm} = \frac{\exp\left(\boldsymbol{h}_n^\top \boldsymbol{W} \boldsymbol{h}_m\right)}{\sum_{m' \in \mathcal{N}(n)} \exp\left(\boldsymbol{h}_n^\top \boldsymbol{W} \boldsymbol{h}_{m'}\right)} \tag{13.29}$$

其中 $\boldsymbol{W}$ 是一个 $D \times D$ 的可学习参数矩阵. 一种更通用的选择是使用神经网络组合边的两个端点的节点的嵌入向量:

$$A_{nm} = \frac{\exp\{\text{MLP}(\boldsymbol{h}_n, \boldsymbol{h}_m)\}}{\sum_{m' \in \mathcal{N}(n)} \exp\{\text{MLP}(\boldsymbol{h}_n, \boldsymbol{h}_{m'})\}} \tag{13.30}$$

其中 MLP 具有单个的连续输出变量, 并且当输入向量交换时, 这个输出变量的值不变. 只要 MLP 在网络中的所有节点之间是共享的 (见习题 13.8), 这个聚合函数就会在节点重新排序下保持等变性.

图注意力网络可以通过引入多头注意力机制来进行扩展, 在这个机制中, 我们可以定义 H 组不同的注意力权重 $A_{nm,h}^{(h)} = 1, \cdots, H$. 每个注意力头都使用上面描述的机制之一进行计算, 并且具有自己的独立参数 (参见 12.1.6 节). 然后在聚合步骤中使用拼接和线性投影将它们组合起来. 需要注意的是, 对于全连接网络, 多头图注意力网络就变成标准的 Transformer 编码器 (见习题 13.9).

13.3.2 边嵌入

前面讨论的图神经网络使用的嵌入向量都与节点相关联. 我们已经看到一些网络也具有与边相关联的数据. 即使不存在与边相关联的观测值, 我们也仍然可以维护和更新基于边的隐藏变量, 这些变量可以为图神经网络学习的内部表示做出贡献.

除了由 $\boldsymbol{h}_n^{(l)}$ 给出的节点嵌入, 我们还引入了边嵌入 $\boldsymbol{e}_{nm}^{(l)}$. 然后我们可以使用以下形式定义一般的消息传递方程:

$$\boldsymbol{e}_{nm}^{(l+1)} = \text{Update}_{\text{edge}}\left(\boldsymbol{e}_{nm}^{(l)}, \boldsymbol{h}_n^{(l)}, \boldsymbol{h}_m^{(l)}\right) \tag{13.31}$$

$$\boldsymbol{z}_n^{(l+1)} = \text{Aggregate}_{\text{node}}\left(\{\boldsymbol{e}_{nm}^{(l+1)} : m \in \mathcal{N}(n)\}\right) \tag{13.32}$$

$$\boldsymbol{h}_n^{(l+1)} = \text{Update}_{\text{node}}\left(\boldsymbol{h}_n^{(l)}, \boldsymbol{z}_n^{(l+1)}\right). \tag{13.33}$$

最终层学习到的边嵌入 $\boldsymbol{e}_{nm}^{(l)}$ 可以直接用来预测与边相关的内容.

13.3.3 图嵌入

除了节点嵌入和边嵌入, 我们还可以维护和更新与整个图相关的嵌入向量 $\boldsymbol{g}^{(l)}$. 将所有这些方面结合在一起, 我们便能够为图结构化应用定义更通用的一组消息传递函数, 以及更丰富的学习表示集合. 具体来说, 我们可以定义如下消息传递方程 (Battaglia et al., 2018):

$$\boldsymbol{e}_{nm}^{(l+1)} = \text{Update}_{\text{edge}}\left(\boldsymbol{e}_{nm}^{(l)}, \boldsymbol{h}_n^{(l)}, \boldsymbol{h}_m^{(l)}, \boldsymbol{g}^{(l)}\right) \tag{13.34}$$

$$\boldsymbol{z}_n^{(l+1)} = \text{Aggregate}_{\text{node}}\left(\{\boldsymbol{e}_{nm}^{(l+1)} : m \in \mathcal{N}(n)\}\right) \tag{13.35}$$

$$\boldsymbol{h}_n^{(l+1)} = \text{Update}_{\text{node}}\left(\boldsymbol{h}_n^{(l)}, \boldsymbol{z}_n^{(l+1)}, \boldsymbol{g}^{(l)}\right) \tag{13.36}$$

$$\boldsymbol{g}^{(l+1)} = \text{Update}_{\text{graph}}\left(\boldsymbol{g}^{(l)}, \{\boldsymbol{h}_n^{(l+1)} : n \in \mathcal{V}\}, \{\boldsymbol{e}_{nm}^{(l+1)} : (n, m) \in \mathcal{E}\}\right). \tag{13.37}$$

这些消息传递方程首先更新边嵌入向量 $\boldsymbol{e}_{nm}^{(l+1)}$, 该操作是基于边嵌入向量的先前状态, 每条边连接的节点的嵌入向量, 以及图级嵌入向量 $\boldsymbol{g}^{(l)}$ 进行的, 参见式(13.34). 然后将这些更新的边嵌入向量在连接到每个节点的每条边

上进行聚合, 参见式(13.35). 接下来转而利用当前节点的嵌入向量和图级嵌入向量更新节点嵌入向量 $\boldsymbol{h}_n^{(l+1)}$, 参见式(13.36). 最后基于图中所有节点和所有边的信息, 以及从前一层获得的图级嵌入向量, 更新当前层的图级嵌入向量.图 13.5演示了上述过程, 算法 13.2 对图消息传递更新做了总结.

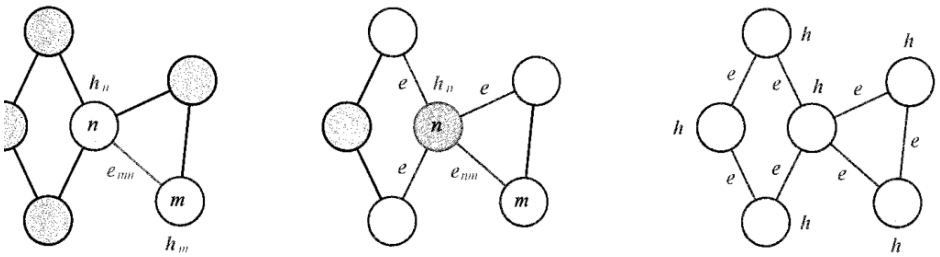


图 13.5: 式(13.34) ~ 式(13.37)定义的图消息传递更新的示意图, 展示了 (a) 边更新、(b) 节点更新, 以及 (c) 全局图更新. 在每种情况下, 正在更新的变量用红色显示, 而那些对该更新有贡献的变量则用蓝色显示.

13.3.4 过度平滑

某些图神经网络会出现的一个显著问题, 就是所谓的过度平滑 (over-smoothing), 在多次迭代地进行消息传递后, 节点嵌入向量变得彼此非常相似, 这实际上限制了网络的深度. 缓解这个问题的一种方法是引入残差连接 (见 9.5 节). 例如, 我们可以修改更新运算符 [式(13.36)]:

$$\boldsymbol{h}_n^{(l+1)} = \text{Update}_{\text{node}}\left(\boldsymbol{h}_n^{(l)}, \boldsymbol{z}_n^{(l+1)}, \boldsymbol{g}^{(l)}\right) + \boldsymbol{h}_n^{(l)}. \tag{13.38}$$

减少过度平滑的另一种方法是让输出层不仅从网络的最后一个卷积层获取信息, 也从所有先前层获取信息. 这可以通过拼接先前层的表示来完成:

$$\boldsymbol{y}_n = f\left(\boldsymbol{h}_n^{(1)} \oplus \boldsymbol{h}_n^{(2)} \oplus \cdots \oplus \boldsymbol{h}_n^{(L)}\right) \tag{13.39}$$

其中 $\boldsymbol{a} \oplus \boldsymbol{b}$ 表示对向量 $\boldsymbol{a}$ 和 $\boldsymbol{b}$ 进行拼接. 这种方法的一个变体是使用最大池化 (max pooling) 而不是拼接来组合向量. 在这种情况下, 输出向量的每个元素由前几层中所有对应嵌入向量的元素的最大值决定.

13.3.5 正则化

标准的正则化技术可以用于图神经网络 (参见第 9 章), 包括将惩罚项 (例如参数值的平方和) 添加到损失函数中. 此外, 一些专门针对图神经网络开发的正则化方法也已经出现.

图神经网络已经采用权重共享来实现置换的等变性和不变性, 但通常它们在每层都有独立的参数. 然而, 权重和偏置也可以跨层共享, 以减少独立参数的数量.

在图神经网络的背景下进行 dropout 指的是在训练期间省略图节点的随机子集, 对于每次前向传播则选择新的随机子集. 这同样适用于图中的边, 即在训练过程中, 随机移除或掩盖邻接矩阵中的部分元素.

13.3.6 几何深度学习

在设计针对图结构数据的深度学习模型时, 我们已经看到排列对称性是一个至关重要的因素. 它作为一种归纳偏置的形式, 在大幅减少数据需求的同时也提高了预测性能. 在与空间属性相关的图神经网络应用中, 如图形网格、流体流动模拟或分子结构, 我们还可以在网络架构中构建额外的等变性和不变性.

例如, 考虑预测分子属性的任务. 当探索候选药物的空间时, 分子可以表示为一个列表, 其中包括给定类型的原子 (碳原子、氢原子、氮原子等) 以及每个原子的空间坐标 (表示为三维的列向量). 我们可以为每个原子 n 在每一层 l 引入一个相关的嵌入向量, 表示为 $\boldsymbol{r}_n^{(l)}$, 并且这些向量可以用已知的原子坐标进行初始化. 然而, 这些向量的元素值取决于坐标系统的任意选择, 而分子的属性则不依赖于它. 例如, 如果分子在三维空间中旋转或相对于坐标

系统原点移动到新位置, 或者如果坐标系统本身反射得到分子的镜像版本, 那么分子的溶解度保持不变. 因此, 分子的属性也应该在这样的变换下保持不变.

通过仔细选择更新操作和聚合操作的函数形式 (Satorras, Hoogeboom, and Welling, 2021), 新的嵌入向量 $\boldsymbol{r}_n^{(l)}$ 可以纳入图神经网络的更新方程 [式(13.31) ~ 式(13.33)] 中, 以实现所需的对称性:

$$\boldsymbol{e}_{nm}^{(l+1)} = \text{Update}_{\text{edge}}\left(\boldsymbol{e}_{nm}^{(l)}, \boldsymbol{h}_n^{(l)}, \boldsymbol{h}_m^{(l)}, \|\boldsymbol{r}_n^{(l)} - \boldsymbol{r}_m^{(l)}\|^2\right) \tag{13.40}$$

$$\boldsymbol{r}_n^{(l+1)} = \boldsymbol{r}_n^{(l)} + C \sum_{(n,m)\in\mathcal{E}} \left(\boldsymbol{r}_n^{(l)} - \boldsymbol{r}_m^{(l)}\right) \phi\left(\boldsymbol{e}_{nm}^{(l+1)}\right) \tag{13.41}$$

$$\boldsymbol{z}_n^{(l+1)} = \text{Aggregate}_{\text{node}}\left(\{\boldsymbol{e}_{nm}^{(l+1)} : m \in \mathcal{N}(n)\}\right) \tag{13.42}$$

$$\boldsymbol{h}_n^{(l+1)} = \text{Update}_{\text{node}}\left(\boldsymbol{h}_n^{(l)}, \boldsymbol{z}_n^{(l+1)}\right). \tag{13.43}$$

注意,$\|\boldsymbol{r}_n^{(l)} - \boldsymbol{r}_m^{(l)}\|^2$ 表示坐标 $\boldsymbol{r}_n^{(l)}$ 和 $\boldsymbol{r}_m^{(l)}$ 之间的平方距离, 它并不依赖于平移、旋转或反射. 同样, 坐标 $\boldsymbol{r}_n^{(l)}$ 可以通过相对差异 $(\boldsymbol{r}_n^{(l)} - \boldsymbol{r}_m^{(l)})$ 的线性组合进行更新. 这里的 $\phi(\boldsymbol{e}_{nm}^{(l+1)})$ 是边嵌入的一般标量函数, 由神经网络表示, 系数 C 通常设置为求和项数目的倒数. 因此在这种变换下, 消息传递方程 [式(13.40)、式(13.42)和式(13.43)] 是不变的, 而式(13.41)给出的坐标嵌入是等变的 (见习题 13.10).

我们已经看到了结构化数据中许多对称性的例子, 从图像中物体的平移和图中节点顺序的排列, 到三维空间中分子的旋转和平移. 将这些对称性捕获在深度神经网络的结构中是一种强大的归纳偏置形式, 也是进行丰富的几何深度学习 (geometric deep learning) 研究的基础 (Bronstein et al., 2017; Bronstein et al., 2021).

习题

13.1(⋆) 证明: 对应于图 13.2中两种节点排序选择的置换 $(A, B, C, D, E) \rightarrow (C, E, A, D, B)$, 可以表示为式(13.5)定义的形式, 其中置换矩阵由式(13.1)给出.

13.2(⋆⋆) 证明: 对应于图 13.2中两种节点排序选择的图 $\mathcal{G}$ 中与每个节点相连的边数可以由矩阵 $\boldsymbol{A}^2$ 中相应的对角元素给出, 其中 $\boldsymbol{A}$ 是邻接矩阵.

13.3(⋆) 由下式给出的邻接矩阵, 画出其对应的图:

$$\boldsymbol{A} = \begin{pmatrix} 0 & 1 & 1 & 0 & 1 \\ 1 & 0 & 1 & 1 & 1 \\ 1 & 1 & 0 & 1 & 0 \\ 0 & 1 & 1 & 0 & 0 \\ 1 & 1 & 0 & 0 & 0 \end{pmatrix}. \tag{13.44}$$

13.4(⋆⋆) 证明在数据矩阵 $\boldsymbol{X}$ 前乘以式(13.3)定义的置换矩阵 $\boldsymbol{P}$ 的效果是创建一个新的数据矩阵 $\widetilde{\boldsymbol{X}}$[由式(13.4)给出], 其行已根据置换函数 $\pi(\cdot)$ 进行置换.

13.5(⋆⋆) 证明转换后的邻接矩阵 $\widetilde{\boldsymbol{A}}$, 它由式(13.5)定义, 其中 $\boldsymbol{P}$ 由式(13.3)定义, 使得新矩阵的行和列都根据置换函数 $\pi(\cdot)$ 相对于原始邻接矩阵 $\boldsymbol{A}$ 进行置换.

13.6(⋆⋆) 将式(13.16)所示的更新方程写成使用矩阵的图级别方程. 为了保持符号简洁, 省略层索引 l. 为此, 首先将节点嵌入向量 $\boldsymbol{h}_n^{(l)}$ 收集到一个矩阵 $\boldsymbol{H}$ 中, 其行 n 由 $\boldsymbol{h}_n^\mathsf{T}$ 给出. 然后证明下式

$$\boldsymbol{Z} = \boldsymbol{A}\boldsymbol{H} \tag{13.45}$$

给出的邻域聚合向量 $\boldsymbol{z}_n$ 可以以矩阵形式写成 $\boldsymbol{Z} = \boldsymbol{A}\boldsymbol{H}$,$\boldsymbol{Z}$ 是一个 $N \times D$ 大小的矩阵, 其中的行 n 由 $\boldsymbol{z}_n^\mathsf{T}$ 给出. 最后证明式(13.16)中非线性激活函数的参数可以以矩阵形式写成

$$\boldsymbol{H}^{(l+1)} = f\left(\boldsymbol{H}\boldsymbol{W}_{\text{self}}^\mathsf{T} + \boldsymbol{A}\boldsymbol{H}\boldsymbol{W}_{\text{neigh}}^\mathsf{T} + \boldsymbol{1}\boldsymbol{b}^\mathsf{T}\right) \tag{13.46}$$

其中 $\boldsymbol{1}$ 是一个 D 维的列向量, 其中的所有元素都是 1.

13.7(⋆⋆) 利用深度图卷积网络第 l 层的等变性 [式(13.21)] 和节点变量的置换性质 [式(13.4)], 证明式(13.20)定义的完整深度图卷积网络也是等变的.

13.8(⋆⋆) 证明在式(13.26)定义的聚合函数中 [注意力权重由式(13.30)给出], 该聚合函数在图中节点重新排序下是等变的.

13.9(⋆) 证明一个图注意力网络 (其中图是全连接的, 即每对节点之间都有一条边), 等同于标准的 Transformer 架构.

13.10(⋆⋆) 当坐标系统被平移时, 对坐标系统定义的物体的位置向量使用

$$\boldsymbol{r}' = \boldsymbol{r} + \boldsymbol{c} \tag{13.47}$$

进行转换, 其中 $\boldsymbol{c}$ 是描述平移的固定向量. 类似地, 如果坐标系统发生旋转和/或镜像反射, 则对坐标系统定义的物体的位置向量使用

$$\boldsymbol{r}' = \boldsymbol{R}\boldsymbol{r} \tag{13.48}$$

进行转换, 其中 $\boldsymbol{R}$ 是一个正交矩阵, 它的逆由其转置给出, 所以

$$\boldsymbol{R}\boldsymbol{R}^{\mathsf{T}} = \boldsymbol{R}^{\mathsf{T}}\boldsymbol{R} = \boldsymbol{I}. \tag{13.49}$$

利用这些属性, 证明在平移、旋转和反射下, 消息传递方程 [式(13.40)、式(13.42)和式(13.43)] 是不变的, 并且式(13.41)给出的坐标嵌入向量是等变的.

第 14 章 采样

14.1 基本采样

在深度学习领域, 我们经常需要从概率分布 $p(z)$ 中生成 z 的样本. 这里的 z 可能是一个标量, 分布可能是单变量高斯分布; 或者 z 可能是一幅高分辨率的图像, 而 $p(z)$ 可能是由深度神经网络定义的生成式模型. 这种生成样本的过程称为采样 (sampling), 也称蒙特卡洛采样 (Monte Carlo sampling). 对于许多简单的分布, 我们可以使用数值技术直接生成合适的样本. 而对于更复杂的分布, 包括那些隐式定义的分布, 我们则需要使用更复杂的方法. 我们把每个实例化的值称为一个样本, 而不像传统统计学那样把一组值称为一个样本.

在本章中, 我们将重点关注与深度学习最相关的采样方面的内容. 关于蒙特卡洛采样方法的更通用的介绍, 可以参考 Gilks, Richardson, and Spiegelhalter (1996) 以及 Robert and Casella (1999).

在本节中, 我们将介绍一系列相对简单的采样策略, 用于从给定分布中生成随机样本. 由于样本是由计算机算法生成的, 它们是伪随机 (pseudo-random) 的, 也就是说, 它们将使用确定性算法来运算, 但仍必须通过适当的随机性测试. 这里我们假设已经存在一个确定性算法, 它能生成在 $(0, 1)$ 区间均匀分布的伪随机数, 事实上, 大多数软件平台都内置了这样的功能.

14.1.1 期望

虽然在某些应用中我们可以直接使用样本本身, 但在更多情况下, 我们的目标是计算某个概率分布的期望. 假设我们希望找到函数 $f(z)$ 关于概率分布 $p(z)$ 的期望. 在这里,z 的分量可能包括离散变量、连续变量或它们两者的组合. 对于连续变量, 期望定义为

$$\mathbb{E}[f] = \int f(z)\, p(z)\, \mathrm{d}z \tag{14.1}$$

其中, 积分在离散变量的情况下被求和替代.

我们在此假设这样的期望过于复杂, 无法使用解析方法准确计算, 这时候就需要用采样方法近似计算. 采样方法的基本思路是获得从分布 $p(z)$ 中独立地抽取的一组样本 $z^{(l)}$(其中 $l = 1, \cdots, L$), 使得期望(14.1)可以通过有限和的形式来近似:

$$\overline{f} = \frac{1}{L} \sum_{l=1}^{L} f\left(z^{(l)}\right). \tag{14.2}$$

如果样本 $z^{(l)}$ 是从概率分布 $p(z)$ 中抽取的, 则 $\mathbb{E}[\overline{f}] = \mathbb{E}[f(z)]$, 因此估计量 $\overline{f}$ 具有正确的均值 (见习题 14.1). 我们也可以把它改成

$$\mathbb{E}[f(z)] \simeq \frac{1}{L} \sum_{l=1}^{L} f\left(z^{(l)}\right) \tag{14.3}$$

其中符号 $\simeq$ 表示右侧是左侧的无偏估计量, 也就是说, 当对噪声分布进行平均时, 两侧是相等的 (见习题 14.2). 估计量(14.2)的方差由下式给出:

$$\mathrm{var}[\overline{f}] = \frac{1}{L} \mathbb{E}\left[(f - \mathbb{E}[f])^2\right] \tag{14.4}$$

该方差是函数 $f(z)$ 在概率分布 $p(z)$ 下的方差. 注意, 这种方差会随着 L 的增加而线性减小, 并且不依赖于 z 的维度, 从原则上讲, 使用相对较小的样本集合 $\{z^{(l)}\}$ 就有可能获得高精度. 然而, 问题在于样本集合 $\{z^{(l)}\}$ 中的样本可能不是独立的, 所以有效样本量可能比表面上的样本量要小得多. 另外, 如果 $f(z)$ 在 $p(z)$ 较大的区域很小, 在 $p(z)$ 较小的地方反而很大, 则期望可能被小概率区域主导, 这意味着为了达到足够的准确度, 可能需要较大的样本量.

14.1.2 标准分布

现在假设我们已经有了一个均匀分布随机数的来源, 我们讨论如何从简单的非均匀分布中生成随机数. 假设 z 在区间 $(0,1)$ 上均匀分布, 并且我们使用某个函数 $g(\cdot)$ 来变换 z 的值, 使得 $y = g(z)$.y 的分布将由下式决定:

$$p(y) = p(z)\left|\frac{\mathrm{d}z}{\mathrm{d}y}\right| \tag{14.5}$$

在这里,$p(z) = 1$. 我们的目标是选择合适的函数 $g(\cdot)$, 让生成的 y 具有某个特定的期望分布 $p(y)$. 对式(14.5)进行积分, 可以得到

$$z = \int_{-\infty}^{y} p(\tilde{y})\,\mathrm{d}\tilde{y} \equiv h(y) \tag{14.6}$$

这是 $p(y)$ 的不定积分.$y = h^{-1}(z)$, 所以我们需要使用一个变换函数, 该函数是目标分布不定积分的反函数.

以指数分布为例

$$p(y) = \lambda \exp(-\lambda y), \qquad 0 \leq y < \infty. \tag{14.7}$$

在这种情况下, 式(14.6)中的积分的下限是 0, 因此 $h(y) = 1 - \exp(-\lambda y)$. 如果我们使用 $y = -\dfrac{1}{\lambda}\ln(1-z)$ 变换均匀分布的随机变量 z, 那么 y 将服从指数分布.

可以应用变量变换方法的另一种分布是柯西分布:

$$p(y) = \frac{1}{\pi}\frac{1}{1+y^2} \tag{14.8}$$

在这种情况下, 不定积分的反函数可以用正切函数表示 (见习题 14.4).

作为变换方法的最后一个例子, 我们介绍用于生成高斯分布样本的 Box–Muller 方法. 首先, 我们生成一对均匀分布的随机数 $z_1, z_2 \in (-1,1)$, 这可以通过对在区间 $(0,1)$ 上均匀分布的变量 z 进行 $z \to 2z-1$ 线性变换来实现. 接下来, 通过丢弃所有不满足条件 $z_1^2 + z_2^2 \leq 1$ 的数对, 这将产生一个在单位圆内均匀分布的点集, 且其概率密度为 $p(z_1, z_2) = 1/\pi$. 最后, 对于每对 z_1, z_2, 我们计算量

$$r^2 = z_1^2 + z_2^2 \tag{14.9}$$

其中 $r^2 = z_1^2 + z_2^2$(见习题 14.5).y_1 和 y_2 的联合概率密度函数为

$$p(y_1, y_2) = \frac{1}{2\pi}\exp\left(-\frac{1}{2}(y_1^2 + y_2^2)\right) \tag{14.10}$$

因此,y_1 和 y_2 是独立的, 并且每个变量都服从均值为零、方差为 1 的高斯分布.

如果 y 服从均值为零、方差为单位方差的高斯分布, 那么 $\sigma y + \mu$ 将服从均值为 μ、方差为 σ^2 的高斯分布. 为了生成具有多元高斯分布的向量值变量, 假设多元高斯分布的均值为 $\boldsymbol{\mu}$、协方差矩阵为 $\boldsymbol{\Sigma}$, 我们可以利用楚列斯基分解, 其形式为 $\boldsymbol{\Sigma} = \boldsymbol{L}\boldsymbol{L}^{\mathsf{T}}$, 其中 $\boldsymbol{L}$ 是一个下三角矩阵. 然后, 如果 $\boldsymbol{z}$ 是一个随机向量, 其分量是独立的且具有零均值、单位方差的高斯分布 (见习题 14.6), 那么 $\boldsymbol{y} = \boldsymbol{\mu} + \boldsymbol{L}\boldsymbol{z}$ 将具有均值为 $\boldsymbol{\mu}$、协方差矩阵为 $\boldsymbol{\Sigma}$ 的高斯分布.

显然, 变换技术的成功依赖于计算所需分布的不定积分与求得其反函数的能力. 这样的操作只对几种简单分布可行, 因此我们必须转向替代方法, 寻找更通用的策略. 这里我们考虑两种替代方法: 拒绝采样 (rejection sampling) 和重要性采样 (importance sampling). 尽管这两种方法受限于单变量分布, 并不能直接应用于多维的复杂问题, 但它们是更通用策略的重要组成部分.

14.1.3 拒绝采样

假设我们想要从一个分布 $p(z)$ 中进行采样, 该分布不属于目前为止考虑过的简单标准分布, 并且直接从 $p(z)$ 中采样很困难. 此外, 假设我们能够轻易地计算任意给定值 z 的 $\widetilde{p}(z)$, 但计算结果需要乘以归一化常数 Z_p 的倒数才能得到真实的 $p(z)$, 即

$$p(z) = \frac{1}{Z_p}\widetilde{p}(z) \tag{14.11}$$

其中 $\widetilde{p}(z)$ 很容易计算, 但 Z_p 是未知的.

拒绝采样框架允许我们从相对复杂的分布中采样, 但有一定的限制条件. 首先考虑单变量分布, 然后考虑扩展

到多维的情形. 要应用拒绝采样, 我们需要一些更简单的分布 $q(z)$, 它们有时称为提议分布 (proposal distribution), 我们可以容易地从中抽取样本. 接下来引入常数 k, 其值的选择需要满足对于所有的 z 都有 $kq(z) \geqslant \tilde{p}(z)$. 函数 $kq(z)$ 称为比较函数.

拒绝采样的每一步都需要生成两个随机数. 首先, 我们从分布 $q(z)$ 中生成一个随机数 $z^\star$. 接下来, 我们从区间 $[0, kq(z^\star)]$ 上的均匀分布中生成另一个随机数 u. 这对随机数在函数 $kq(z)$ 的曲线下具有均匀分布. 最后, 如果 $u > \tilde{p}(z^\star)$, 则拒绝该样本, 否则保留它. 因此, 这对随机数如果位于未归一化分布 $\tilde{p}(z)$ 和缩放分布 $kq(z)$ 之间的灰色区域内, 则会被拒绝. 剩余的随机数对在 $\tilde{p}(z)$ 曲线下具有均匀分布 (见习题 14.7), 因此相应的 z 值服从我们期望的分布 $p(z)$.

接受样本的概率由下式给出:

$$p(\text{accept}) = \int \min\left\{\frac{\tilde{p}(z)}{kq(z)}, 1\right\} q(z)\,\mathrm{d}z \tag{14.12}$$

原始的 z 值是从分布 $q(z)$ 中生成的, 然后这些样本以 $\tilde{p}(z)/kq(z)$ 的概率得以接受, 因此这种方法拒绝的点的比例取决于未归一化分布 $\tilde{p}(z)$ 曲线下的面积与缩放分布 $kq(z)$ 曲线下的面积的比值. 由此可见, 常数 k 应尽可能小, 同时满足对于任意 z,$kq(z)$ 处处都不小于 $\tilde{p}(z)$ 的限制条件.

为了演示拒绝采样的用法, 考虑从伽马分布中采样的任务:

$$\text{Gam}(z|a, b) = \frac{b^a z^{a-1} e^{-bz}}{\Gamma(a)} \tag{14.13}$$

当 $a > 1$, 伽马分布具有钟形形状. 因此, 合适的提议分布是柯西分布(14.8), 因为它也是钟形的, 而且我们可以使用前面讨论的变换方法从中采样. 我们需要稍微扩展一下柯西分布, 以确保其在任何位置的值都不小于伽马分布的值. 这可以通过使用 $z = b\tan y + c$ 变换均匀分布的随机变量来实现, 这会产生服从下式分布的随机数

$$q(z) = \frac{1}{\pi}\frac{b}{(z-c)^2 + b^2} \tag{14.14}$$

通过设置 $c = a - 1$、$b^2 = 2a - 1$, 并选择尽可能小的常数 k 来满足 $kq(z) \geqslant \tilde{p}(z)$ 的要求, 可以获得最小的拒绝率.

为了使拒绝采样具有实用价值, 我们要求比较函数与目标分布尽可能接近, 以便将拒绝率保持在最低限度. 下面看看当我们试图在高维空间中使用拒绝采样时会发生什么. 我们考虑一个有点儿故意设计的问题, 希望从一个均值为零、协方差为 $\sigma_1^2\boldsymbol{I}$ 的高斯分布中采样 (其中 $\boldsymbol{I}$ 是单位矩阵), 而提议分布本身是一个均值为零、协方差为 $\sigma_2^2\boldsymbol{I}$ 的高斯分布. 显然, 必须有 $\sigma_2 \geqslant \sigma_1$ 以确保存在一个常数 k 使得 $kq(z) \geqslant p(z)$. 在 D 维空间中,k 的最优值由 $k = (\sigma_2/\sigma_1)^D$ 给出. 样本接受率将是 $p(z)$ 和 $kq(z)$ 曲线下面积的比值, 因为这两个分布都是归一化的, 所以结果为 $1/k$. 对于 σ_2 仅比 σ_1 高出 1%, 在 $D = 1000$ 维时, 接受率大约为 1/20 000. 在这个例子中, 比较函数接近目标分布. 目标分布可能是多模态且具有尖锐峰值的, 要找到一个好的提议分布和比较函数将极其困难. 此外, 接受率随维度呈指数下降是拒绝采样的一个普遍特性. 尽管拒绝采样在一维或两维空间中可能是一种有用的技术, 但它不适用于高维问题. 然而, 它可以作为在高维空间采样的更复杂算法中的一个子程序发挥作用.

14.1.4 适应性拒绝采样

许多时候, 我们希望使用拒绝采样方法却很难确定包络分布 $q(z)$ 的合适解析形式. 一种替代方法是基于分布 $\tilde{p}(z)$ 的测量值构造包络函数 (Gilks and Wild, 1992). 当 $p(z)$ 是对数凹函数时, 即 $\ln p(z)$ 的导数是 z 的非增函数时, 构造合适的包络函数尤其简单.

首先, 在一些初始网格点上计算 $\ln \tilde{p}(z)$ 及其梯度, 然后利用得到的切线交点来构造包络函数. 接下来, 从包络分布中抽取一个样本值 (见习题 14.10). 这一步很简单, 因为包络分布的对数是一系列线性函数, 因此包络分布本身是由以下形式的分段指数分布构成的:

$$q(z) = \sum_j k_j \exp\{-a_j(z - z_j)\}, \qquad z_j < z \leqslant z_{j+1}. \tag{14.15}$$

一旦抽取了一个样本, 就可以应用常规的拒绝标准. 样本如果被接受, 它将是期望分布的一个抽取. 样本如果被拒绝, 它就被纳入网格点集合中, 用于计算新的切线, 并由此完善包络函数. 随着网格点数量的增加, 包络函数会越来越逼近期望分布 $p(z)$, 并且样本被拒绝的概率就会降低.

Gilks 提出了旨在避免计算导数的自适应拒绝采样 (Gilks, 1992). 自适应拒绝采样框架也可以扩展到非对数凹分布, 只需要在每个拒绝采样步骤之后跟随一个 Metropolis–Hastings 步骤 (将在小节 14.2.3中讨论), 从而产生自适应拒绝 Metropolis 采样 (Gilks, Best, and Tan, 1995).

14.1.5 重要性采样

我们希望从复杂概率分布中进行采样的一个原因是为了计算形如(14.1)的期望. 重要性采样技术提供了一个直接近似期望的框架, 但它本身并不提供从分布 $p(z)$ 中抽取样本的方法.

假设直接从 $p(z)$ 中采样是不现实的, 但对于任意给定的 z 值, 我们能够容易地计算出 $\widetilde{p}(z)$ 的值. 一种简单的计算期望方法是将 z 空间离散化为均匀网格, 并将期望表示成求和的形式. 然而, 这种方法的一个明显问题是, 求和项的数量会随着 z 的维度的增加呈指数增长. 此外, 正如我们已经注意到的, 我们感兴趣的概率分布通常会将它们的大部分概率密度限制在 z 空间的相对较小的区域内, 所以在高维问题中, 均匀采样将非常低效, 因为只有很少一部分样本会对求和做出显著贡献. 我们真正希望的是在 $p(z)$ 较大的区域, 或理想情况下, 从 $p(z)f(z)$ 较大的区域中选择样本点.

与拒绝采样一样, 重要性采样也基于易于抽取样本的提议分布 $q(z)$. 我们可以利用从 $q(z)$ 中抽取的样本集 $\{z^{(l)}\}$, 将期望表达为样本上的有限和形式:

$$\mathbb{E}[f] = \int f(z)p(z)\,\mathrm{d}z \simeq \frac{1}{L}\sum_{l=1}^{L}\frac{\widetilde{p}(z^{(l)})}{q(z^{(l)})}f\left(z^{(l)}\right). \tag{14.16}$$

权重 $r^{(l)} = \widetilde{p}(z^{(l)})/q(z^{(l)})$ 称为重要性权重 (importance weights), 用于纠正从错误分布中采样而引入的偏差. 请注意, 与拒绝采样不同, 这里生成的所有样本都会被保留.

通常分布 $p(z)$ 的计算只能达到一个归一化常数的精度, 所以 $p(z) = \widetilde{p}(z)/Z_p$, 其中 $\widetilde{p}(z)$ 可以轻易计算出来, 而 Z_p 是未知的. 类似地, 我们可能希望使用的重要性采样分布 $q(z) = \widetilde{q}(z)/Z_q$ 也具有同样的性质. 我们可以使用相同的样本集来计算比值 Z_p/Z_q, 结果为

$$\mathbb{E}[f] = \int f(z)p(z)\,\mathrm{d}z \simeq \frac{\sum_l r^{(l)}f(z^{(l)})}{\sum_l r^{(l)}}. \tag{14.17}$$

因此, 式(14.17)中的期望可用加权和来表示:

$$\mathbb{E}[f] = \sum_l w_l f\left(z^{(l)}\right) \tag{14.18}$$

其中

$$w_l = \frac{r^{(l)}}{\sum_{l'} r^{(l')}}. \tag{14.19}$$

请注意,$\{w_l\}$ 都是非负数且和为 1.

与拒绝采样一样, 重要性采样的成功严重依赖于采样分布 $q(z)$ 与目标分布 $p(z)$ 的匹配程度. 如果 $p(z)f(z)$ 变化剧烈并且其大部分概率密度集中在 z 空间的相对较小的区域内 (这种情况很常见), 则一组重要性权重 $\{r^{(l)}\}$ 可能会被少数几个较大权重的支配, 剩余的权重则相对无关紧要. 因此, 有效样本的数量可能比表面上的样本数 L 少得多. 如果没有任何样本落在 $p(z)f(z)$ 较大的区域内, 问题将更为严重. 在那种情况下, 即使 $r^{(l)}$ 和 $r^{(l)}f(z^{(l)})$ 的表面方差可能很小, 对期望的估计也可能产生严重错误. 因此, 重要性采样的一个主要缺点是, 它可能产生任意错误的结果, 并且没有诊断指标. 这也突出了采样分布 $q(z)$ 的一个关键要求, 即在 $p(z)$ 可能显著的区域内, 它的值不应该太小或为零.

14.1.6 采样–重要性–重采样

小节 14.1.3讨论的拒绝采样部分地依赖于成功确定一个适当的正值 k. 对于分布 $p(z)$ 和 $q(z)$ 的各种组合, 想要确定一个适当的 k 值是不切实际的, 因为当 k 值大到能保证目标分布被包络的时候, 对应的样本接受率会小到没有实用价值.

采样–重要性–重采样方案也使用一个采样分布 $q(z)$, 但避免了确定 k 值的要求. 该方案分两个阶段. 在第一阶段, 从 $q(z)$ 中抽取 L 个样本 $z^{(1)}, \cdots, z^{(L)}$. 然后在第二阶段, 使用式(14.19)构建权重 $(w_1, \cdots, w_L)$. 最后从离散分布 $w_1, \cdots, w_L$ 中抽取第二组 L 个样本, 其被抽中的概率由权重 $(w_1, \cdots, w_L)$ 给出.

为了理解这一点, 考虑单变量情况, 并注意重采样值的累积分布由下式给出:

$$p(z < a) = \sum_l w_l \mathbb{I}\left(z^{(l)} \leq a\right) \simeq \int \mathbb{I}(z \leq a)\, p(z)\, \mathrm{d}z \tag{14.20}$$

其中 $\mathbb{I}(\cdot)$ 是指示函数 (如果其参数为真, 则函数值为 1, 否则为 0). 使 L 趋于无穷大, 并假设分布的适当的正则性, 我们就可以用根据原始采样分布 $q(z)$ 加权的积分来替换求和:

$$p(z < a) \simeq \int \mathbb{I}(z \leq a) \frac{\widetilde{p}(z)}{q(z)} q(z)\, \mathrm{d}z \tag{14.21}$$

这就是 $p(z)$ 的累积分布函数. 同样, 我们可以看到 $p(z)$ 的归一化并不是必需的.

对于有限值 L 和给定的初始样本集, 重采样值只能近似地从目标分布中抽取. 与拒绝采样一样, 随着采样分布 $q(z)$ 更接近目标分布 $p(z)$, 近似结果的效果也会越来越好. 当 $q(z) = p(z)$ 时, 初始样本 $\{z^{(1)}, \cdots, z^{(L)}\}$ 具有目标分布, 且权重 $w_l = 1/L$, 因此重采样值也具有目标分布.

如果需要关于分布 $p(z)$ 的矩, 则可以直接使用原始样本连同权重来计算, 因为

$$\mathbb{E}[f(z)] = \int f(z) p(z)\, \mathrm{d}z. \tag{14.22}$$

14.2 马尔可夫链蒙特卡洛采样

在上一节中, 我们讨论了用于计算函数期望的拒绝采样和重要性采样策略, 并且我们看到它们在高维空间中有严重局限性. 本节将讨论一种非常通用和强大的采样策略, 称为马尔可夫链蒙特卡洛采样, 它允许从多种分布中进行采样并且可以很好地扩展到高维样本空间. 马尔可夫链蒙特卡洛采样起源于物理学 (Metropolis and Ulam, 1949), 直至 20 世纪 80 年代末期才开始在统计学领域产生显著影响.

与拒绝采样和重要性采样一样, 我们依然从提议分布中采样. 然而这一次, 我们保持当前状态 $z^{(\tau)}$ 的记录, 提议分布 $q(z|z^{(\tau)})$ 以当前状态为条件, 因此样本序列 $z^{(1)}, z^{(2)}, \cdots$ 形成一个马尔可夫链 (参见小节 14.2.2). 同样, 尽管 Z_p 的值可能是未知的, 如果我们将 $p(z)$ 写成 $p(z) = \widetilde{p}(z)/Z_p$, 我们假设对于任意给定的 z 值,$\widetilde{p}(z)$ 可以很容易计算出来. 我们的提议分布足够简单, 可以直接从中抽取样本. 在算法的每一个迭代中, 我们从提议分布中生成一个候选样本 $z^\star$, 然后根据一个适当的准则来接受该样本.

14.2.1 Metropolis 算法

在基础的 Metropolis 算法 (Metropolis et al., 1953) 中, 我们假设提议分布是对称的, 即对于所有的 z_A 和 z_B 值, 有 $q(z_A|z_B) = q(z_B|z_A)$. 然后候选样本以如下概率被接受:

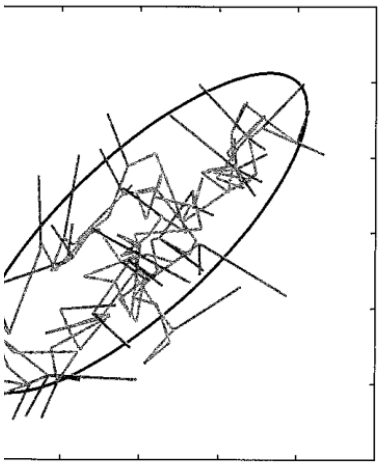
$$A\left(z^\star, z^{(\tau)}\right) = \min\left(1, \frac{\widetilde{p}(z^\star)}{\widetilde{p}(z^{(\tau)})}\right) \tag{14.23}$$

这可以通过选择一个在单位区间 $(0, 1)$ 上均匀分布的随机数 u 来实现. 如果 $A(z^\star, z^{(\tau)}) > u$, 则接受该样本. 请注意, 如果从 $z^{(\tau)}$ 到 $z^\star$ 的步骤导致 $p(z)$ 的值增加, 那么候选样本一定会被保留.

如果候选样本被接受, 则 $z^{(\tau+1)} = z^\star$, 否则候选样本 $z^\star$ 将被丢弃. 将 $z^{(\tau+1)}$ 设置为 $z^{(\tau)}$, 并从分布 $q(z|z^{(\tau+1)})$ 中抽取另一个候选样本. 这与拒绝采样形成了鲜明对比, 在拒绝采样中, 被拒绝的样本将直接被丢弃. 在 Metropolis 算法中, 当一个候选样本被拒绝时, 它的前一个样本将包含在最终的样本列表中, 导致样本有多个副本. 当然, 在实际应用中, 每个保留的样本只会有一个副本, 并附带一个整数权重因子, 用于记录该状态出现的次数.

正如我们将看到的, 如果对于任何的 z_A 和 z_B 值,$q(z_A|z_B)$ 都是正值 (这是一个充分但非必要条件), 则 $z^{(\tau)}$ 的分布趋向于 $p(z)$(当 $\tau \to \infty$ 时). 然而, 应该强调的是, 序列 $z^{(1)}, z^{(2)}, \cdots$ 不是来自 $p(z)$ 的独立样本集, 因为连续的样本高度相关. 如果希望获得独立的样本, 那么我们可以丢弃大部分序列, 每次只保留第 M 个样本. 对于足够大的 M, 保留的样本将在实际意义上是独立的.图 14.1显示了一个简单的示例, 它使用 Metropolis 算法从二维高斯分布

色采样形成了鲜明对比，在
算法中，当一个候选样本被



用 Metropolis 算法从高斯分布中采
图，该高斯分布的一个标准差等高

图 14.1: 使用 Metropolis 算法从高斯分布中采样的简单示意图, 该高斯分布的一个标准差等高线是用椭圆显示的. 提议分布是一个各向同性的高斯分布, 其标准差为 0.2. 接受的候选样本以绿线显示, 拒绝的候选样本以红线显示. 共生成 150 个候选样本, 其中 43 个被拒绝.

中进行采样, 其中提议分布是各向同性的高斯分布.

14.2.2 马尔可夫链

在更详细地讨论马尔可夫链蒙特卡洛采样之前, 我们先研究一下马尔可夫链的一般性质. 特别地, 我们想知道马尔可夫链在何种情况下会收敛到期望的分布. 一阶马尔可夫链定义为随机变量序列 $z^{(1)}, \cdots, z^{(M)}$, 使得对于 $m \in \{1, \cdots, M-1\}$, 下面的条件独立性成立:

$$p\left(z^{(m+1)} \,|\, z^{(1)}, \cdots, z^{(m)}\right) = p\left(z^{(m+1)} \,|\, z^{(m)}\right) \tag{14.24}$$

这可以用一个链式的有向图模型来表示 (见图 11.29). 然后, 我们可以通过给出初始变量的概率分布 $p(z^{(0)})$ 和以转移概率 (transition probabilities) 形式表达的后续变量条件分布 $T_m(z^{(m)}, z^{(m+1)}) \equiv p(z^{(m+1)}|z^{(m)})$, 即可完整定义马尔可夫链. 如果转移概率对所有 m 都相同, 则称马尔可夫链是齐次的或同质 (homogeneous) 的.

链中的某个特定变量的边缘概率可以用马尔可夫链中前一个变量的边缘概率表示为

$$p\left(z^{(m+1)}\right) = \int p\left(z^{(m+1)} \,|\, z^{(m)}\right) p\left(z^{(m)}\right) \, \mathrm{d}z^{(m)} \tag{14.25}$$

其中的积分对于离散变量来说可以替换为求和. 如果一个分布在马尔可夫链的每个步骤都保持该分布不变, 那么该分布对于这个马尔可夫链来说就是不变的或者说稳定的. 因此, 对于具有转移概率 $T(z', z)$ 的齐次马尔可夫链来说, 如果满足以下条件, 则分布 $p^\star(z)$ 是不变的:

$$p^\star(z) = \int T(z', z)\, p^\star(z') \, \mathrm{d}z' \tag{14.26}$$

请注意, 给定的马尔可夫链可能有多个不变分布. 例如, 如果转移概率由恒等变换给出, 那么任何分布都将是不变的.

确保目标分布 $p(z)$ 是不变的一个充分 (但非必要) 条件是选择满足详细平衡 (detailed balance) 性质的转移概

率, 该性质由下式定义

$$p^\star(z)\,T(z,z') = p^\star(z')\,T(z',z) \tag{14.27}$$

对于特定的分布 $p^\star(z)$, 容易看出, 如果转移概率满足对于特定分布的详细平衡性质, 则保持该分布不变, 因为

$$\int p^\star(z')T(z',z)\,\mathrm{d}z' = \int p^\star(z)T(z,z')\,\mathrm{d}z' \tag{14.28}$$

$$= p^\star(z)\int p(z'|z)\,\mathrm{d}z' \tag{14.29}$$

$$= p^\star(z). \tag{14.30}$$

满足详细平衡性质的马尔可夫链被称为可逆的 (reversible).

我们的目标是使用马尔可夫链从给定分布中采样. 如果能建立一个马尔可夫链使得目标分布是不变的, 就可以达到目的. 然而, 我们还必须要求不管初始分布 $p(z^{(0)})$ 的选择如何, 当 $m \to \infty$ 时, 分布 $p(z^{(m)})$ 都收敛到所需的不变分布 $p^\star(z)$. 这个性质称为遍历性 (ergodicity), 此时不变分布称为平衡 (equilibrium) 分布. 显然, 一个遍历 (ergodic) 的马尔可夫链只能有一个平衡分布. 可以证明, 一个齐次马尔可夫链是遍历的, 只需要对不变分布和转移概率施加较弱的约束条件 (Neal, 1993).

在实践中, 我们经常利用一组 “基础” 转移 $B_1,\cdots,B_K$ 来构造转移概率. 这可以通过混合分布的形式来实现:

$$T(z',z) = \sum_{k=1}^{K}\alpha_k B_k(z',z) \tag{14.31}$$

混合系数 $\alpha_1,\cdots,\alpha_K$ 必须满足 $\alpha_k \geq 0$ 且 $\sum_k \alpha_k = 1$. “基础” 转移可以通过连续应用结合起来, 于是有

$$T(z',z) = \sum_{z_1}\cdots\sum_{z_{K-1}} B_1(z',z_1)\cdots B_{K-1}(z_{K-2},z_{K-1})B_K(z_{K-1},z). \tag{14.32}$$

如果一个分布对每个基础转移都是不变的, 那么很明显它也会对式(14.31)或式(14.32)给出的 $T(z',z)$ 保持不变. 对于混合分布 [式(14.31)], 如果每个基础转移都满足详细平衡条件, 那么复合转移概率 T 也将满足详细平衡条件. 这对于使用式(14.32)构造的转移概率并不成立, 尽管通过对基础转移的应用顺序进行对称化 (即 $B_1,B_2,\cdots,B_K,B_K,\cdots,B_2,B_1$), 可以恢复详细平衡. 使用复合转移概率的一个常见例子是让每个基础转移仅改变变量的一个子集.

为了深入了解马尔可夫链蒙特卡洛采样算法的本质, 考虑一个由整数构成的状态空间 $\mathbb{Z}$, 已知概率

$$p\left(z^{(\tau+1)} = z^{(\tau)} + 1\right) = 0.25 \tag{14.33}$$

$$p\left(z^{(\tau+1)} = z^{(\tau)} - 1\right) = 0.25 \tag{14.34}$$

$$p\left(z^{(\tau+1)} = z^{(\tau)}\right) = 0.5. \tag{14.35}$$

其中 $z^{(\tau)}$ 表示在时间步 τ 的状态. 如果初始状态是 $z^{(0)} = 0$, 那么根据对称性, 在时间步 τ 有 $\mathbb{E}[z^{(\tau)}] = 0$, 同样也容易看出 $\mathbb{E}[(z^{(\tau)})^2] = \tau/2$(见习题 14.11). 因此, 在时间步 τ 之后, 随机游走平均而言仅行进了与 $\sqrt{\tau}$ 成比例的距离. 这种平方根依赖性是随机游走行为的典型特征, 表明随机游走在探索状态空间时效率非常低. 正如我们将看到的, 设计马尔可夫链蒙特卡洛采样算法的一个核心目标就是避免随机游走行为.

14.2.3 Metropolis–Hastings 算法

前面我们介绍了基础的 Metropolis 算法, 但没有实践证明它能从所需分布中进行采样. 在给出证明之前, 我们首先讨论一种更泛化的形式, 称为 Metropolis–Hastings 算法 (Hastings, 1970), 该算法适用于提议分布不再是其参数的对称函数的情况. 特别是在算法的第 τ 步, 当前状态为 $z^{(\tau)}$, 首先从分布 $q_k(z|z^{(\tau)})$ 中抽取一个样本 $z^\star$, 然后以概率 $A_k(z^\star,z^{(\tau)})$ 接受它, 其中

$$A_k\left(z^\star,z^{(\tau)}\right) = \min\left(1,\frac{\widetilde{p}(z^\star)\,q_k(z^{(\tau)}|z^\star)}{\widetilde{p}(z^{(\tau)})\,q_k(z^\star|z^{(\tau)})}\right) \tag{14.36}$$

这里的 k 标记了正在考虑的可能转换集合的成员. 同样, 计算接受准则不需要知道概率分布 $p(\mathbf{z}) = \widetilde{p}(\mathbf{z})/Z_p$ 中的归一化常数 Z_p. 对于对称的提议分布,Metropolis–Hastings 准则 [式(14.36)] 可简化为标准的 Metropolis 准则 [式(14.23)].

我们可以通过证明式(14.27)定义的详细平衡条件得以满足, 从而进一步证明 $p(\mathbf{z})$ 是由 Metropolis–Hastings 算法所定义的马尔可夫链的不变分布. 利用(14.36), 我们有

$$p(\mathbf{z})\, q_k(\mathbf{z}^\star|\mathbf{z})\, A_k(\mathbf{z}, \mathbf{z}^\star) = \min\left(\widetilde{p}(\mathbf{z}) q_k(\mathbf{z}^\star|\mathbf{z}),\ \widetilde{p}(\mathbf{z}^\star) q_k(\mathbf{z}|\mathbf{z}^\star)\right) \tag{14.37}$$

这个表达式在交换 $\mathbf{z}$ 和 $\mathbf{z}^\star$ 时是对称的, 因此详细平衡成立.

提议分布的具体选择对算法的性能有显著影响. 对于连续状态空间, 一个常见的选择是以当前状态为中心的高斯分布, 在确定此分布的方差时有一个重要的权衡——如果方差较小, 则接受转换的比例将会很高, 但通过状态空间的过程表现为缓慢的随机游走, 导致相关时间较长. 然而, 如果方差较大, 则拒绝率将会很高, 因为在我们考虑的复杂问题中, 许多被提出的步骤将处于概率 $p(\mathbf{z})$ 较低的状态. 考虑一个有强相关性变量的多变量分布 $p(\mathbf{z})$. 提议分布的尺度 σ 应该与最小标准差 $\sigma_{\min}$ 的数量级相匹配, 这导致了随机游走行为, 其中大致独立的状态之间分隔的步数是 $(\sigma_{\max}/\sigma_{\min})^2$ 数量级的. 实际上, 在二维问题中, 随着 σ 增加而导致的拒绝率增加已被接受转换的更大步长抵消. 更一般地说, 对于多元高斯分布, 获得独立样本所需的步数和 $(\sigma_{\min}/\sigma_2)^2$ 成正比, 其中 σ_2 是第二小的标准差 (Neal, 1993). 撇开这些细节不谈, 如果分布变化的长度尺度在不同方向上差异很大, 那么 Metropolis–Hastings 算法可能会收敛得非常慢.

14.2.4 吉布斯采样

吉布斯采样 (Geman and Geman, 1984) 是一种简单且使用广泛的采样策略, 可以看作 Metropolis–Hastings 算法的一个特例. 考虑分布 $p(\mathbf{z}) = p(z_1, \cdots, z_M)$, 我们希望从中采样, 假设我们已经为马尔可夫链选择了一些初始状态. 吉布斯采样过程中的每一步都包含用我们从一个变量的条件分布中抽取的值替换该变量的值, 该条件分布是以其余变量的值为条件的. 因此, 可通过从分布 $p(z_i|\mathbf{z}_{\setminus i})$ 中采样来替换 z_i, 其中 $\mathbf{z}_{\setminus i}$ 表示 $\{z_1, \cdots, z_M\}$ 但其中不包括 z_i, 这个过程要么按某种特定顺序循环遍历变量, 要么在每一步随机选择要更新的变量.

例如, 假设我们有一个关于 3 个变量的分布 $p(z_1, z_2, z_3)$, 在时间步 τ, 我们选定了值 $z_1^{(\tau)}$、$z_2^{(\tau)}$ 和 $z_3^{(\tau)}$. 首先将 z_1 替换为通过从如下条件分布中采样获得的新值 $z_1^{(\tau+1)}$:

$$p\left(z_1 \,|\, z_2^{(\tau)}, z_3^{(\tau)}\right). \tag{14.38}$$

然后将 $z_2^{(\tau)}$ 替换为通过从如下条件分布中采样获得的新值 $z_2^{(\tau+1)}$:

$$p\left(z_2 \,|\, z_1^{(\tau+1)}, z_3^{(\tau)}\right). \tag{14.39}$$

这样新的 z_1 值便可立即在后续的采样步骤中使用. 用我们从如下条件分布中抽取的样本 $z_3^{(\tau+1)}$ 更新 $z_3^{(\tau)}$:

$$p\left(z_3 \,|\, z_1^{(\tau+1)}, z_2^{(\tau+1)}\right) \tag{14.40}$$

以此类推, 依次循环遍历这 3 个变量.

我们可以将吉布斯采样过程看作是 Metropolis–Hastings 算法的一个特定实例. 考虑一个涉及变量 z_i 的 Metropolis–Hastings 采样步骤, 其余变量 $\mathbf{z}_{\setminus i}$ 保持不变, 从 $\mathbf{z}$ 到 $\mathbf{z}'$ 的转移概率由 $q(\mathbf{z}'|\mathbf{z}) = p(z_i|\mathbf{z}_{\setminus i})$ 给出. 注意 $\mathbf{z}_{\setminus i} = \mathbf{z}'_{\setminus i}$, 因为这些分量在采样步骤中未发生改变. 同样,$p(\mathbf{z}) = p(z_i|\mathbf{z}_{\setminus i})p(\mathbf{z}_{\setminus i})$, 因此, Metropolis–Hastings 算法 [式(14.36)] 中确定接受概率的因子由下式给出:

$$\frac{p(\mathbf{z}')q(\mathbf{z}|\mathbf{z}')}{p(\mathbf{z})q(\mathbf{z}'|\mathbf{z})} = \frac{p(z_i'|\mathbf{z}_{\setminus i})p(\mathbf{z}_{\setminus i})\, p(z_i|\mathbf{z}_{\setminus i})}{p(z_i|\mathbf{z}_{\setminus i})p(\mathbf{z}_{\setminus i})\, p(z_i'|\mathbf{z}_{\setminus i})} = 1 \tag{14.41}$$

其中 $\mathbf{z}_{\setminus i} = \mathbf{z}'_{\setminus i}$. 因此,Metropolis–Hastings 采样步骤总是可以接受.

为了证明该过程从目标分布中采样, 我们首先注意到分布 $p(\mathbf{z})$ 是吉布斯采样步骤中的一个单独的不变量, 因此它也是整个马尔可夫链的不变量. 这是因为当我们从 $p(z_i|\mathbf{z}_{\setminus i})$ 中采样时, 边缘分布 $p(\mathbf{z}_{\setminus i})$ 显然是不变的, 因为 $\mathbf{z}_{\setminus i}$ 的值没有发生改变. 同样, 定义的每一步采样都从正确的条件分布 $p(z_i|\mathbf{z}_{\setminus i})$ 中采样. 因为这些条件分布和边缘分布共同指定了联合分布, 所以我们可以看到联合分布本身是不变的.

为了完整地描述该算法, 还需要指定初始状态的分布, 尽管在多次迭代后抽取的样本实际上独立于此分布. 确保能从正确分布中进行吉布斯采样的第二个条件是要具有遍历性 (ergodic). 具有遍历性的一个充分条件是条件分布不存在零概率区域. 如果是这种情况, 那么 $\boldsymbol{z}$ 空间中的任意一点都可以通过有限步骤到达其他任意一点, 每一步都包括更新每个分量变量中的一个. 如果不满足这个要求, 即某些条件分布存在零概率区域, 那么遍历性 (如果适用) 必须被显式地证明.

与 Metropolis 算法类似, 我们可以通过研究吉布斯采样在特定分布 (例如高斯分布) 上的应用来深入了解其行为. 考虑两个变量中相关高斯分布, 其条件分布的宽度为 1, 边缘分布的宽度为 L. 典型的步长由条件分布决定, 并且将是 L 阶的. 由于状态根据随机游走演化, 因此为了获得来自分布的独立样本, 所需的步数将是 L^2 阶的. 当然, 如果高斯分布是不相关的, 那么吉布斯采样过程将是最优的. 对于这个简单问题, 我们可以旋转坐标系, 使得新变量不相关. 但是, 在实际应用中, 我们通常不可能找到这样的转换.

由于基本的吉布斯采样技术一次只考虑一个变量, 因此连续样本之间存在强依赖关系. 在另一种极端情况下, 如果我们能够直接从联合分布中采样 (实际上这种操作很难实现), 则连续样本将是独立的. 我们可以采用一种折中策略来改进简单的吉布斯采样, 即连续地从变量组而不是单个变量中采样. 这可以通过分块吉布斯 (blocking Gibbs) 采样算法实现, 该算法选择变量块 (不一定是互斥的), 然后依次对每个块中的变量进行联合采样, 条件是其他所有变量保持其当前值 (Jensen, Kong, and Kjaerulff, 1995).

减少吉布斯采样中随机游走行为的一种方法称为超松弛 (over-relaxation)(Adler, 1981). 在它的原始形式中, 它适用于条件分布是高斯分布的问题, 就是说, “条件分布为高斯分布” 所涵盖的分布类型比 “多元高斯分布” 更广泛. 例如, 非高斯分布 $p(z, y) \propto \exp(-z^2y^2)$ 具有条件高斯分布. 在吉布斯采样的每步, 特定分量 z_i 具有均值为 μ_i、方差为 σ_i^2 的条件分布. 在超松弛框架中,z_i 的值被替换为

$$z_i' = \mu_i + \alpha_i(z_i - \mu_i) + \sigma_i\sqrt{1-\alpha_i^2}\,v \tag{14.42}$$

其中 v 是一个有零均值和单位方差的高斯随机变量,α 是一个参数, 满足 $-1 < \alpha < 1$. 当 $\alpha = 0$ 时, 该方法等同于标准的吉布斯采样; 当 $\alpha < 0$ 时, 该步骤偏向于均值的相反侧. 这一步骤保持了目标分布的不变性, 因为如果 z_i 有均值 μ 和方差 σ^2, 那么 z_i' 也将如此 (见习题 14.14). 超松弛的作用是在变量高度相关时, 鼓励状态空间中的定向运动. 有序超松弛框架 (ordered over-relaxation)(Neal, 1999) 将这种方法推广到了非高斯分布.

吉布斯采样的实用性取决于从条件分布 $p(z_i|\boldsymbol{z}_{\setminus i})$ 中采样的难度. 对于使用有向图模型表示的概率分布, 单个节点的条件分布仅依赖于相应马尔可夫毯中的变量. 对于有向图, 这个条件分布只是马尔可夫毯中节点状态的函数.

14.2.5 祖先采样

对于许多模型来说, 联合分布 $p(\boldsymbol{z})$ 可以方便地用图模型的术语来表示. 对于一个没有观测变量的有向图, 可以直接使用祖先采样 (ancestral sampling) 方法从联合分布中采样. 联合分布由下式指定:

$$p(\boldsymbol{z}) = \prod_{i=1}^{M} p(\boldsymbol{z}_i \mid \mathrm{pa}(i)) \tag{14.43}$$

其中 $\boldsymbol{z}_i$ 是与节点 i 关联的一组变量,pa(i) 表示与节点 i 的父节点关联的一组变量.

为了获得联合分布的一个样本, 我们按照 $z_1, \cdots, z_M$ 的顺序遍历这组变量, 并从条件分布 $p(\boldsymbol{z}_i|\,\mathrm{pa}(i))$ 中采样. 这总是可能的, 因为在每个步骤, 所有父变量都会被实例化. 遍历图一次后, 我们将获得联合分布的一个样本. 这里假设我们可以从每个节点的各条件分布中采样.

考虑这样一个有向图, 其部分节点组成了证据集 $\boldsymbol{\varepsilon}$, 并被观测值实例化过. 原则上可以扩展上述过程, 至少对于代表离散变量的节点, 我们可以给出以下逻辑采样方法 (Henrion, 1988), 它可以看作重要性采样的一种特殊情况 (见小节 14.1.5). 在每一步, 当为一个观测值已知的变量 z_i 获得一个采样值时, 对采样值与观测值进行比较, 如果它们一致, 则保留采样值, 并继续依次处理下一个变量. 然而, 如果它们不一致, 则丢弃到目前为止的整个样本, 重新从图的第一个节点开始. 这种算法可以正确地从后验分布中采样, 因为它仅仅相当于从隐变量和数据变量的联合分布中抽取样本, 然后丢弃那些与观测数据不一致的样本 (在发现一个不一致的值后就不再从联合分布中继

续采样, 从而稍微提高了效率). 然而, 随着观测变量数量的增加和这些变量可以取的状态数量的增加, 从后验分布中接受一个样本的整体概率迅速降低, 因此这种方法在实践中很少使用.

这种方法的改进版称为似然加权采样 (likelihood weighted sampling)(Fung and Chang, 1990; Shachter and Peot, 1990), 它基于祖先采样并结合了重要性采样. 对于每一个变量, 如果该变量在证据集中, 那么把它设置为其实例化的值. 如果它不在证据集中, 则从条件分布 $p(z_i|\,\mathrm{pa}(i))$ 中采样, 其中条件变量设置为其当前采样的值. 随后产生的样本 $\boldsymbol{z}$ 的权重由下式给出 (见习题 14.15):

$$w = \prod_{i\in\mathcal{E}} p\left(z_i = \hat{z}_i \mid \mathrm{pa}(i)\right). \tag{14.44}$$

这种方法可以进一步使用自重要性采样 (self-importance sampling)(Shachter and Peot, 1990) 扩展, 其中重要性采样分布不断更新, 以反映当前估计的后验分布.

14.3 朗之万采样

Metropolis–Hastings 算法通过使用提议分布创建候选样本的马尔可夫链, 并使用式(14.36)所示的准则接受或拒绝它们, 从而从概率分布中抽取样本. 这可能相对低效, 因为提议分布通常是一个简单且固定的分布, 它可以在数据空间的任何方向上生成更新, 从而导致随机游走的行为.

我们已经看到, 在训练神经网络时, 利用关于模型可学习参数的对数似然梯度来最大化似然函数是极其有利的. 类似地, 我们可以引入利用关于数据向量的概率密度梯度的马尔可夫链采样算法, 以便优先向概率较大的区域移动. 这种技术称为哈密顿蒙特卡洛 (Hamiltonian Monte Carlo) 采样, 也称混合蒙特卡洛 (hybrid Monte Carlo) 采样. 该采样方法使用了 Metropolis 接受准则 (Duane et al., 1987; Bishop, 2006). 在这里, 我们研究一种在深度学习中使用广泛的新的采样方法——朗之万采样. 尽管它避免了使用 Metropolis 接受准则, 但我们必须仔细设计算法, 以确保产生的结果样本是无偏的. 在基于能量函数的机器学习模型中, 朗之万采样发挥了至关重要的作用.

14.3.1 基于能量的模型

许多生成式模型可以表示为条件概率分布 $p(\boldsymbol{x}|\boldsymbol{w})$, 其中 $\boldsymbol{x}$ 是数据向量,$\boldsymbol{w}$ 代表可学习参数的向量. 这样的模型可以通过最大化相对于训练集定义的对应似然函数来进行训练. 然而, 为了表示一个有效的概率分布, 模型必须满足如下要求:

$$\int p(\boldsymbol{x}|\boldsymbol{w})\,\mathrm{d}\boldsymbol{x} = 1 \tag{14.45}$$

进行训练. 然而, 为了表示一个有效的概率分布, 模型必须满足如下要求; 这样的归一化要求大大限制了适用模型的形式. 在不考虑归一化约束的情况下, 我们可以考虑一个更广泛的模型类别, 称为基于能量的模型 (LeCun et al., 2006). 假设我们有一个称为能量函数的函数 $E(\boldsymbol{x},\boldsymbol{w})$, 它是其参数的实值函数, 但没有其他约束. 指数 $\exp\{-E(\boldsymbol{x},\boldsymbol{w})\}$ 是一个非负数, 因此可以看作未归一化的 $\boldsymbol{x}$ 上的概率分布. 这里在指数中引入负号只是一个惯例, 它意味着能量的较高值对应于概率的较低值. 然后我们可以使用下式定义一个归一化的分布 (见习题 14.16):

$$p(\boldsymbol{x}|\boldsymbol{w}) = \frac{1}{Z(\boldsymbol{w})}\exp\{-E(\boldsymbol{x},\boldsymbol{w})\} \tag{14.46}$$

其中归一化常数 $Z(\boldsymbol{w})$ 称为配分函数 (partition function), 它由下式定义:

$$Z(\boldsymbol{w}) = \int \exp\{-E(\boldsymbol{x},\boldsymbol{w})\}\,\mathrm{d}\boldsymbol{x}. \tag{14.47}$$

能量函数通常使用深度神经网络进行建模, 输入向量为 $\boldsymbol{x}$, 标量输出为 $E(\boldsymbol{x},\boldsymbol{w})$, 其中 $\boldsymbol{w}$ 代表网络中权重和偏置的集合.

注意配分函数依赖于 $\boldsymbol{w}$, 这给训练带来了一些问题. 例如, 一组独立同分布数据 $\mathcal{D} = \{\boldsymbol{x}_1,\cdots,\boldsymbol{x}_N\}$ 的对数似然函数具有以下形式:

$$\ln p(\mathcal{D}|\boldsymbol{w}) = \sum_{n=1}^{N}\ln p(\boldsymbol{x}_n|\boldsymbol{w}) = -N\ln Z(\boldsymbol{w}) - \sum_{n=1}^{N} E(\boldsymbol{x}_n,\boldsymbol{w}). \tag{14.48}$$

对于具有许多选择的能量函数 $E(\boldsymbol{x}, \boldsymbol{w})$, 计算配分函数 [式(14.47)] 是不切实际的, 因为这需要对整个 $\boldsymbol{x}$ 空间进行积分 (或对离散变量求和). “基于能量的模型” 指的就是这种积分难以处理的模型. 然而请注意, 概率模型可以视为基于能量的模型的特例, 因此本书讨论的许多模型可以视为基于能量的模型. 基于能量的模型的最大优势是, 它们在不需要归一化的情况下具有灵活性. 相应的缺点是, 由于归一化常数未知, 它们可能更难以训练.

14.3.2 最大化似然

我们已经开发了各种近似方法来训练基于能量的模型, 而无须计算配分函数 (Song and Kingma, 2021). 本小节探讨基于马尔可夫链蒙特卡洛采样的技术. 另一种方法称为分数匹配, 我们将在扩散模型的背景下进行讨论 (见第 20 章).

使用式(14.46), 对于一个基于能量的模型, 对数似然函数关于模型参数的梯度可以写成以下形式:

$$\nabla_{\boldsymbol{w}} \ln p(\boldsymbol{x}|\boldsymbol{w}) = -\nabla_{\boldsymbol{w}} E(\boldsymbol{x}, \boldsymbol{w}) - \nabla_{\boldsymbol{w}} \ln Z(\boldsymbol{w}). \tag{14.49}$$

这是针对单个数据点 $\boldsymbol{x}$ 的似然函数, 但实际上我们希望最大化训练数据集上的似然函数, 该数据集是从某个未知分布 $p_D(\boldsymbol{x})$ 中抽样得到的. 如果我们假设数据点是独立同分布的, 那么我们可以考虑与 $p_D(\boldsymbol{x})$ 相关的似然对数期望的梯度, 其由下式给出:

$$\mathbb{E}_{p_D}\left[\nabla_{\boldsymbol{w}} \ln p(\boldsymbol{x}|\boldsymbol{w})\right] = -\mathbb{E}_{p_D}[\nabla_{\boldsymbol{w}} E(\boldsymbol{x}, \boldsymbol{w})] - \nabla_{\boldsymbol{w}} \ln Z(\boldsymbol{w}). \tag{14.50}$$

这里我们利用了 $-\nabla_{\boldsymbol{w}} \ln Z(\boldsymbol{w})$ 不依赖于 $\boldsymbol{x}$ 的性质, 因而可以把它移到期望之外. 假设配分函数 $Z(\boldsymbol{w})$ 是未知的, 但我们可以利用式(14.47)并重组得到

$$-\nabla_{\boldsymbol{w}} \ln Z(\boldsymbol{w}) = \int \nabla_{\boldsymbol{w}} E(\boldsymbol{x}, \boldsymbol{w})\, p(\boldsymbol{x}|\boldsymbol{w})\, \mathrm{d}\boldsymbol{x}. \tag{14.51}$$

式(14.51)右侧的部分对应于模型分布 $p(\boldsymbol{x}|\boldsymbol{w})$ 的期望, 它由下式给出 (见习题 14.18):

$$\mathbb{E}_{p(\boldsymbol{x}|\boldsymbol{w})}[\nabla_{\boldsymbol{w}} E(\boldsymbol{x}, \boldsymbol{w})]\,. \tag{14.52}$$

结合式(14.50) ~ 式(14.52), 我们得到

$$\nabla_{\boldsymbol{w}} \mathbb{E}_{p_D}\left[\ln p(\boldsymbol{x}|\boldsymbol{w})\right] = -\mathbb{E}_{p_D}[\nabla_{\boldsymbol{w}} E(\boldsymbol{x}, \boldsymbol{w})] + \mathbb{E}_{p(\boldsymbol{x}|\boldsymbol{w})}[\nabla_{\boldsymbol{w}} E(\boldsymbol{x}, \boldsymbol{w})]\,. \tag{14.53}$$

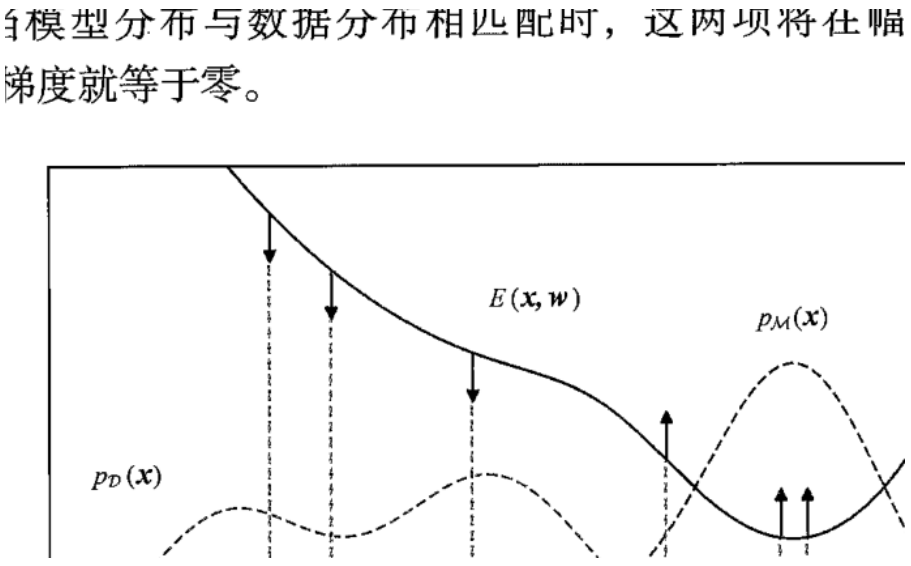


图 14.2: 通过最大化似然对基于能量的模型进行训练的示意图, 其中显示了能量函数 $E(\boldsymbol{x}, \boldsymbol{w})$(绿色) 及相关的模型分布 $p_M(\boldsymbol{x})$ 和真实数据分布 $p_D(\boldsymbol{x})$. 利用式(14.53)增加期望的对数似然, 会在与模型样本 (用蓝点表示) 对应的点处推高能量函数, 并在与数据集样本 (用红点表示) 对应的点处拉低能量函数.

我们的目标是找到参数 $\boldsymbol{w}$ 的值, 让似然函数的值最大化. 为此, 我们朝梯度 $\nabla_{\boldsymbol{w}} \ln p(\boldsymbol{x}|\boldsymbol{w})$ 的方向对 $\boldsymbol{w}$ 进行了微小调整. 从式(14.53)中可以看出, 这个梯度的期望值可以分解为两部分, 且这两部分符号相反. 式(14.53)右侧第一项的作用是减小 $E(\boldsymbol{x}, \boldsymbol{w})$, 因而会增大模型定义的数据点 $\boldsymbol{x}$ 的概率密度, 这里的 $\boldsymbol{x}$ 是从 $p_D(\boldsymbol{x})$ 中抽取的. 式(14.53)右

侧第二项的作用则是增大 $E(\boldsymbol{x}, \boldsymbol{w})$, 从而降低由模型定义的来自模型本身的数据点的概率密度. 在模型密度超过训练数据密度的区域, 净效果将是增加能量, 因此降低了概率密度. 相反, 在训练数据密度超过模型密度的区域, 净效果将是降低能量, 因此增大了概率密度. 正如我们所希望的那样, 这两项一起将概率密度从训练数据密度低的区域移向了训练数据密度高的区域. 当模型分布与数据分布相匹配时, 这两项将在幅度上相等, 此时式(14.53)左侧的梯度就等于零.

我们已经看到, 对于基于能量的模型, 由于存在未知的配分函数 $Z(\boldsymbol{w})$, 似然函数不能显式求值. 然而, 我们可以利用蒙特卡洛采样方法, 来计算对数似然函数关于模型参数的梯度. 一旦以任何方式训练了基于能量的模型, 就可以使用采样方法从模型中抽取样本了, 比如使用蒙特卡洛采样方法.

14.3.3 朗之万动力学

当把式(14.53)作为一种实际的训练方法应用时, 我们需要近似计算式(14.53)右侧的两项. 对于任何给定的 $\boldsymbol{x}$ 值, 我们可以使用自动微分来评估 $\nabla_{\boldsymbol{w}} E(\boldsymbol{x}, \boldsymbol{w})$. 对于式(14.53)中的第一项, 我们可以使用训练集来估计关于 $\boldsymbol{x}$ 的期望:

$$\mathbb{E}_{x \sim p_D}[\nabla_{\boldsymbol{w}} E(\boldsymbol{x}, \boldsymbol{w})] \approx \frac{1}{N} \sum_{n=1}^{N} \nabla_{\boldsymbol{w}} E(\boldsymbol{x}_n, \boldsymbol{w}). \tag{14.54}$$

式(14.53)中的第二项更具挑战性, 因为我们需要从能量函数定义的模型分布中抽取样本, 而相应的配分函数难以处理. 这可以使用马尔可夫链蒙特卡洛采样来完成. 另一种受欢迎的采样方法称为随机梯度朗之万动力学, 简称朗之万采样 (Parisi, 1981; Welling and Teh, 2011). 这一项仅通过得分函数 (score function) 依赖于分布 $p(\boldsymbol{x}|\boldsymbol{w})$, 得分函数定义为对数似然函数关于数据向量 $\boldsymbol{x}$ 的梯度, 并由下式给出:

$$\boldsymbol{s}(\boldsymbol{x}, \boldsymbol{w}) = \nabla_{\boldsymbol{x}} \ln p(\boldsymbol{x}|\boldsymbol{w}) \tag{14.55}$$

值得强调的是, 这是针对数据点 $\boldsymbol{x}$ 的梯度, 而非通常意义上针对可学习参数 $\boldsymbol{w}$ 的梯度. 将式(14.46)代入式(14.55), 我们可以得到

$$\boldsymbol{s}(\boldsymbol{x}, \boldsymbol{w}) = -\nabla_{\boldsymbol{x}} E(\boldsymbol{x}, \boldsymbol{w}). \tag{14.56}$$

可以看到, 配分函数不再出现, 因为它与 $\boldsymbol{x}$ 无关.

让我们从一个先验分布中开始抽取一个初始值 $\boldsymbol{x}^{(0)}$, 然后执行以下马尔可夫链步骤:

$$\boldsymbol{x}^{(\tau+1)} = \boldsymbol{x}^{(\tau)} + \eta \nabla_{\boldsymbol{x}} \ln p\left(\boldsymbol{x}^{(\tau)}, \boldsymbol{w}\right) + \sqrt{2\eta}\, \boldsymbol{\varepsilon}^{(\tau)}, \qquad \tau = 1, \cdots, \mathcal{T} \tag{14.57}$$

其中 $\boldsymbol{\varepsilon}^{(\tau)} \sim \mathcal{N}(\boldsymbol{0}, \boldsymbol{I})$ 是从零均值、单位协方差的高斯分布中独立抽取的样本, 参数 η 控制步长的大小. 朗之万方程的每次迭代都会沿着对数似然的梯度方向迈出一步, 然后添加高斯噪声. 可以证明, 当 $\eta \to 0$ 和 $\mathcal{T} \to \infty$ 时,$\boldsymbol{x}^{(\mathcal{T})}$ 的值是从分布 $p(\boldsymbol{x})$ 中独立抽取的样本.

我们可以通过重复这个过程来生成一组样本 $\{\boldsymbol{x}_1, \cdots, \boldsymbol{x}_M\}$, 然后使用下式来近似计算式(14.53)中的第二项:

$$\mathbb{E}_{x \sim p_M(x)}[\nabla_{\boldsymbol{w}} E(\boldsymbol{x}, \boldsymbol{w})] \approx \frac{1}{M} \sum_{m=1}^{M} \nabla_{\boldsymbol{w}} E(\boldsymbol{x}_m, \boldsymbol{w}). \tag{14.58}$$

运行长的马尔可夫链来生成独立样本的计算成本可能很高, 所以我们需要考虑实际的近似方法. 有一种方法称为对比散度 (contrastive divergence)(Hinton, 2002). 在这里, 用来评估式(14.58)的样本是通过从训练数据点 $\boldsymbol{x}_n$ 开始的蒙特卡洛链获得的. 如果蒙特卡洛链运行了大量步骤, 那么得出的值基本上就是来自模型分布的无偏样本. 相反, 对比散度方法建议只运行几步的蒙特卡洛链, 甚至只有一步, 这在计算上成本要低得多. 相应的结果样本将远非无偏样本, 并且会靠近数据流形. 因此, 使用梯度下降的效果将是仅在数据流形的邻域内塑造能量曲面, 从而塑造概率密度. 这对于诸如分类的任务可能是有效的, 但预计在学习生成式模型方面效果会较差.

习题

14.1(★★) 证明式(14.2)定义的 $\overline{f}$ 是无偏估计, 换句话说, 其右边项的期望等于 $\mathbb{E}[f(\boldsymbol{z})]$.

14.2(★★) 证明式(14.2)定义的 $\overline{f}$ 的方差由式(14.4)给出.

14.3(★) 假设 z 是在区间 $(0,1)$ 上均匀分布的随机变量, 使用 $y = h^{-1}(z)$ 变换 z, 其中 $h(y)$ 由式(14.6)给出. 证明 y 具有分布 $p(y)$.

14.4(★★) 给定一个在区间 $(0,1)$ 上均匀分布的随机变量 z, 找到一个转换 $y = f(z)$, 使得 y 具有式(14.8)给出的柯西分布.

14.5(★★) 假设变量 z_1 和 z_2 在单位圆上均匀分布, 进行式(14.9)和式(14.10)给出的变量变换. 证明 (y_1, y_2) 具有式(14.10)所示的联合分布.

14.6(★★) 假设 $\boldsymbol{z}$ 是一个具有零均值和单位协方差矩阵的 D 维高斯分布的随机变量, 并且假设正定对称矩阵 $\boldsymbol{\Sigma}$ 具有楚列斯基分解 $\boldsymbol{\Sigma} = \boldsymbol{L}\boldsymbol{L}^{\mathsf{T}}$, 其中 $\boldsymbol{L}$ 是一个下三角矩阵 (即主对角线以下的元素非零, 主对角线以上的元素为零). 证明变量 $\boldsymbol{y} = \boldsymbol{\mu} + \boldsymbol{L}\boldsymbol{z}$ 服从均值为 $\boldsymbol{\mu}$ 和协方差为 $\boldsymbol{\Sigma}$ 的高斯分布. 这提供了一种利用服从零均值、单位方差的一元高斯样本来生成服从一般多元高斯分布样本的技术.

14.7(★★) 证明拒绝采样确实是从期望的分布 $p(z)$ 中抽取样本. 假设提议分布是 $q(z)$. 证明样本值 z 被接受的概率由 $\widetilde{p}(z)/kq(z)$ 给出, 其中 $\widetilde{p}$ 是与 $p(z)$ 成比例的任何未归一化的分布, 并且常数 k 设置为能够确保 $kq(z) \geq \widetilde{p}(z)$ 的最小值. 注意, 抽取值 z 的概率等于从 $q(z)$ 中抽取该值的概率乘以给定已经抽取了该值的情况下接受该值的概率. 利用这一点, 结合概率的加法和乘法规则, 写出关于 z 的归一化分布形式, 并证明它等于 $p(z)$.

14.8(★) 假设变量 z 在区间 $[0,1]$ 上均匀分布. 证明变量 $y = b\tan(\pi(z-1/2)) + c$ 具有式(14.14)给出的柯西分布.

14.9(★★) 使用连续性和归一化的要求, 确定自适应拒绝采样中包络分布 [式(14.15)] 的系数 k_j.

14.10(★★) 通过使用小节 14.1.2讨论的从单一指数分布中采样的技术, 设计一种算法, 用来从式(14.15)定义的分段指数分布中采样.

14.11(★) 证明式(14.33) ~ 式(14.35)定义的整数上的简单随机游走具有性质 $\mathbb{E}[z^{(\tau)}] = \mathbb{E}[z^{(\tau-1)}] + 1/2$, 并因此通过归纳得出 $\mathbb{E}[(z^{(\tau)})^2] = \tau/2$.

14.12(★★) 证明小节 14.2.4讨论的吉布斯采样满足式(14.27)定义的详细平衡性质.

14.13(★) 考虑一个双峰分布, 其中两个模式之间被一个概率为零的狭窄区域隔开. 讨论标准的吉布斯采样是不是遍历的, 并由此判断是否能正确地从此分布中采样.

第 15 章 离散潜变量模型

15.1 K 均值聚类

我们首先考虑在多维空间中识别数据点群组或聚类的问题. 假设有一个数据集 $\{\boldsymbol{x}_1, \cdots, \boldsymbol{x}_N\}$, 其中包含了 N 个 D 维欧几里得变量 $\boldsymbol{x}$ 的观测值. 我们的目标是将该数据集划分成 K 个聚类, 可以暂定 K 的值是已知的. 直觉上, 我们会认为一个聚类包含一组数据点, 这些数据点之间的距离相较于聚类外的数据点之间的距离要短. 这里引入一组 D 维向量 $\boldsymbol{\mu}_k$, 其中 $k = 1, \cdots, K$, 并且 $\boldsymbol{\mu}_k$ 是第 k 个聚类的 “原型” . 可以认为 $\boldsymbol{\mu}_k$ 代表了聚类中心. 然后要找到一组聚类向量 $\{\boldsymbol{\mu}_k\}$, 以及数据点到聚类的分配, 使得每个数据点到与其最近的聚类向量 $\boldsymbol{\mu}_k$ 的距离平方和最小.

为了方便起见, 需要定义一些符号来描述数据点到聚类的分配. 对于每个数据点 $\boldsymbol{x}_n$, 我们引入相应的一组二进制指示变量 $r_{nk} \in \{0, 1\}$, 其中 $k = 1, \cdots, K$. 这些指示变量描述了数据点 $\boldsymbol{x}_n$ 会被分配到哪个聚类, 因此如果数据点 $\boldsymbol{x}_n$ 被分配到聚类 k, 那么 $r_{nk} = 1$, 并且对于 $j \neq k$,$r_{nj} = 0$. 这是 1-of-K 编码方案的一个例子. 接下来可以定义如下误差函数:

$$J = \sum_{n=1}^{N} \sum_{k=1}^{K} r_{nk} \|\boldsymbol{x}_n - \boldsymbol{\mu}_k\|^2 \tag{15.1}$$

该误差函数代表每个数据点到分配给它的聚类向量 $\boldsymbol{\mu}_k$ 的距离平方和. 我们的目标是找到 $\{r_{nk}\}$ 和 $\{\boldsymbol{\mu}_k\}$ 的值, 让 J 最小. 这可以通过一个迭代过程来实现, 其中每次迭代包括两个连续步骤, 分别对应于对 $\{r_{nk}\}$ 和 $\{\boldsymbol{\mu}_k\}$ 进行的连续优化. 首先为 $\{\boldsymbol{\mu}_k\}$ 选择一些初始值. 然后在第一步, 保持 $\{\boldsymbol{\mu}_k\}$ 固定, 并优化 $\{r_{nk}\}$ 以最小化 J; 而在第二步, 保持 $\{r_{nk}\}$ 固定, 并优化 $\{\boldsymbol{\mu}_k\}$ 以最小化 J. 重复这两步优化直至收敛.

在第一步, 保持 $\{\boldsymbol{\mu}_k\}$ 固定, 并关于 $\{r_{nk}\}$ 最小化 J.J 是关于 $\{r_{nk}\}$ 的线性函数, 所以对于这个优化可以轻松地给出闭合形式的解. 涉及不同 n 的项是独立的, 因此对于任何能给出 $\|\boldsymbol{x}_n - \boldsymbol{\mu}_k\|^2$ 最小值的 k 值, 可以通过选择 $r_{nk} = 1$ 来单独为每个 n 进行优化. 换句话说, 我们只是简单地将第 n 个数据点分配给与其最近的聚类, 这可以表示为

$$r_{nk} = \begin{cases} 1, & k = \arg\min_j \|\boldsymbol{x}_n - \boldsymbol{\mu}_j\|^2 \\ 0, & \text{其他} \end{cases} \tag{15.2}$$

接下来考虑在 $\{r_{nk}\}$ 固定的情况下优化 $\{\boldsymbol{\mu}_k\}$(M 步骤). 目标函数 J 是关于 $\boldsymbol{\mu}_k$ 的二次函数, 可通过将其关于 $\boldsymbol{\mu}_k$ 的导数设为零来最小化它, 从而得到

$$\sum_{n=1}^{N} r_{nk}(\boldsymbol{x}_n - \boldsymbol{\mu}_k) = 0 \tag{15.3}$$

我们可以很容易地解出 $\boldsymbol{\mu}_k$, 得到

$$\boldsymbol{\mu}_k = \frac{\sum_n r_{nk}\boldsymbol{x}_n}{\sum_n r_{nk}}. \tag{15.4}$$

观察式(15.4)的右侧, 分母等于分配给聚类 k 的数据点的数量, 运算结果 $\boldsymbol{\mu}_k$ 可以简单地看作等于所有分配给聚类 k 的数据点 $\boldsymbol{x}_n$ 的平均值. 这种方法因而得名 K 均值算法 (Lloyd, 1982). 因为 $\{r_{nk}\}$ 是离散的, 并且每次迭代都不会导致误差函数增大, 所以 K 均值算法一定可以在有限的步骤中收敛 (见习题 15.1).

将数据点重新分配到聚类和重新计算聚类均值这两个阶段轮流重复进行, 直至分配不再变化 (或者直至迭代次数超过某预定值). 然而, 这种方法可能会收敛到目标函数 J 的局部极小值而不是全局最小值.MacQueen (1967) 研究了 K 均值算法的收敛性.

图 15.1使用源自美国黄石国家公园老忠实喷泉 (Old Faithful geyser) 数据集的喷发数据对 K 均值算法进行了说明 (参见 3.2.9 小节). 该数据集包含 272 个数据点, 每个数据点都在水平轴上给出了一次喷发的持续时间, 并在垂直轴上给出了下一次喷发的持续时间. 在这里, 我们已经对数据进行了线性重新缩放, 也就是标准化, 使得每个变量都具有零均值和单位标准差.

具有零均值和单位标准差。

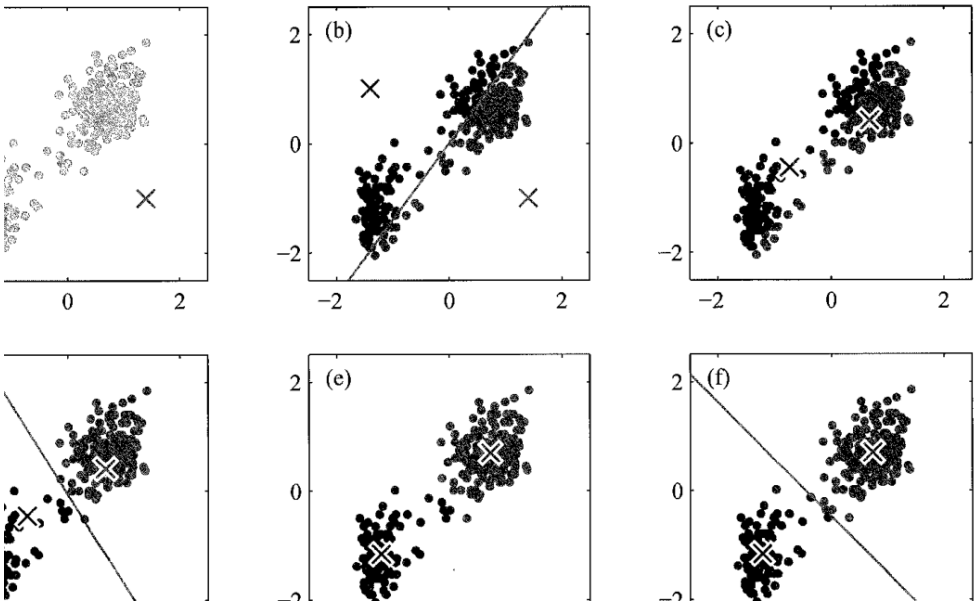


图 15.1: 利用线性重新缩放的老忠实喷泉数据集来说明 K 均值算法.(a) 绿色点代表二维欧几里得空间中的数据集, 红色和蓝色的叉号表示初始选择的聚类中心;(b) 在初始的 E 步骤中, 每个数据点要么分配给红色聚类, 要么分配给蓝色聚类;(c) 在随后的 M 步骤中, 每个聚类中心被重新计算为分配给对应聚类的所有数据点的平均值;(d)–(i) 显示了直至最终收敛前的连续 E 步骤和 M 步骤.

到目前为止, 考虑的是批量版本的 K 均值算法, 其中整个数据集一起用来更新原型向量. 我们也可以推导出一个顺序更新的策略, 在这个策略中, 对于每个数据点 $\boldsymbol{x}_n$, 更新离它最近的原型向量 $\boldsymbol{\mu}_k$:

$$\boldsymbol{\mu}_k^{\text{new}} = \boldsymbol{\mu}_k^{\text{old}} + \frac{1}{N_k}(\boldsymbol{x}_n - \boldsymbol{\mu}_k^{\text{old}}) \tag{15.5}$$

其中 N_k 是到目前为止用来更新 $\boldsymbol{\mu}_k$ 的数据点的数量. 这允许每个数据点在使用一次后就被丢弃, 然后处理下一个数据点.

K 均值算法的一个显著特点是, 在每次迭代中, 每个数据点都只被分配给一个聚类. 尽管有些数据点会比其他任何聚类中心 $\boldsymbol{\mu}_j$ 更接近特定聚类中心 $\boldsymbol{\mu}_k$, 但也可能有其他数据点大约处于聚类中心之间的中点位置. 在后一种情况下, 其实并不清楚将数据点硬性分配给最近的聚类是否是最合适的方法. 我们将看到, 通过采用概率方法, 我们将能够以一种反映最适当分配的不确定性程度的方式, 获得数据点到聚类的“软”分配. 这种概率方法有许多好处.

作为 K 均值算法应用的例子, 我们考虑图像分割和图像压缩这两个相关问题. 图像分割的目标是将图像划分成区域, 以便每个区域都具有相对一致的视觉外观或者对应于物体或物体的某一部分 (Forsyth and Ponce, 2003). 图像中的每个像素都是由红、绿、蓝通道的强度组成的三维空间中的一个点, 图像分割算法简单地将图像中的每个像素作为一个单独的数据点来处理. 然后展示了对任何特定的 K 值运行 K 均值算法直至收敛的结果, 方法是重新绘制图像, 并替换图像中的每个像素向量——用其分配到的聚类中心 $\boldsymbol{\mu}_k$ 给出的 $\{R, G, B\}$ 强度三元组进行替换. 各种不同 K 值的结果表明, 对于给定的 K 值, 该算法只使用 K 种颜色来表示图像.

我们也可以使用聚类算法进行数据压缩. 首先我们要区分无损数据压缩 (其目标是从压缩表示中精确地重建原始数据) 和有损数据压缩 (也就是说, 我们接受数据重建过程中的一些误差以换取相比无损压缩情况下更高的压缩率). 我们可以将 K 均值算法应用于有损数据压缩问题. 对于 N 个数据点中的每一个数据点, 我们只存储其分配到的聚类的身份 k. 我们还需要存储 K 个聚类中心 $\{\boldsymbol{\mu}_k\}$ 的值, 在我们选择 $K \ll N$ 的前提下, 这种方法明显减少了需要的数据量, 然后每个数据点都可以通过与其最近的聚类中心 $\boldsymbol{\mu}_k$ 来近似表示. 新的数据点同样可以通过首先找到最近的 $\boldsymbol{\mu}_k$, 然后存储标签 k 而不是原始的数据向量来进行压缩. 这个框架通常称为向量量化 (vector quantization), 向量 $\{\boldsymbol{\mu}_k\}$ 称为码本向量 (codebook vector).

15.2 高斯混合分布

我们之前已经将高斯混合分布描述为高斯分量的简单线性叠加, 目的是提供比单个高斯分布更丰富的密度模型类别 (参见 3.2.9 小节). 在本节中, 将转向用离散潜变量的形式来描述高斯混合分布. 这将帮助我们更深入地理解这个重要分布, 并有助于激励我们去探究 EM(Expectation–Maximization, 最大期望) 算法.

回想一下式 (3.111), 高斯混合分布可以写成高斯分量的线性叠加形式:

$$p(\boldsymbol{x}) = \sum_{k=1}^{K} \pi_k \mathcal{N}(\boldsymbol{x}|\boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k) \tag{15.6}$$

让我们引入一个 K 维的二进制随机向量 $\boldsymbol{z}$, 它采用 1-of-K 表示形式, 其中一个元素等于 1, 其他所有元素等于 0. 因此 z_k 的值满足 $z_k \in \{0, 1\}$ 以及 $\sum_k z_k = 1$, 并且根据哪个元素非零, 向量 $\boldsymbol{z}$ 有 K 种可能的状态. 将联合分布 $p(\boldsymbol{x}, \boldsymbol{z})$ 定义为边缘分布 $p(\boldsymbol{z})$ 和条件分布 $p(\boldsymbol{x}|\boldsymbol{z})$ 的形式. 关于 $\boldsymbol{z}$ 的边缘分布是由混合系数 π_k 指定的, 即

$$p(z_k = 1) = \pi_k \tag{15.7}$$

其中参数 $\{\pi_k\}$ 必须满足

$$0 \leq \pi_k \leq 1 \tag{15.8}$$

和

$$\sum_{k=1}^{K} \pi_k = 1. \tag{15.9}$$

因为 $\boldsymbol{z}$ 使用 1-of-K 表示形式, 我们也可以将该分布写成

$$p(\boldsymbol{z}) = \prod_{k=1}^{K} \pi_k^{z_k} \tag{15.10}$$

同样, 给定 $\boldsymbol{z}$ 的特定值时 $\boldsymbol{x}$ 的条件分布也是一个高斯分布:

$$p(\boldsymbol{x}|z_k = 1) = \mathcal{N}(\boldsymbol{x}|\boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k) \tag{15.11}$$

它可以写成

$$p(\boldsymbol{x}|\boldsymbol{z}) = \prod_{k=1}^{K} \mathcal{N}(\boldsymbol{x}|\boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k)^{z_k} \tag{15.12}$$

联合分布由 $p(\boldsymbol{z})p(\boldsymbol{x}|\boldsymbol{z})$ 给出, 它可以用图模型来描述 (见习题 15.3). 可以通过对所有可能的 $\boldsymbol{z}$ 的状态求和联合分布来获得 $\boldsymbol{x}$ 的边缘分布:

$$p(\boldsymbol{x}) = \sum_{\boldsymbol{z}} p(\boldsymbol{z})p(\boldsymbol{x}|\boldsymbol{z}) = \sum_{k=1}^{K} \pi_k \mathcal{N}(\boldsymbol{x}|\boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k) \tag{15.13}$$

其中我们利用了式(15.10)和式(15.12). 因此,$\boldsymbol{x}$ 的边缘分布是形式为式(15.6)的高斯混合分布. 如果我们有多个观测值 $\boldsymbol{x}_1, \cdots, \boldsymbol{x}_N$, 那么因为边缘分布的形式是 $p(\boldsymbol{x}) = \sum_{\boldsymbol{z}} p(\boldsymbol{x}, \boldsymbol{z})$, 所以每个观测到的数据点 $\boldsymbol{x}_n$ 都对应一个潜变量 $\boldsymbol{z}_n$.

另一个起到重要作用的量是给定 $\boldsymbol{x}$ 时 $\boldsymbol{z}$ 的条件概率. 我们使用 $\gamma(z_k)$ 来表示 $p(z_k = 1|\boldsymbol{x})$, 其值可以使用贝叶斯定理找到:

$$\gamma(z_k) = \frac{\pi_k \mathcal{N}(\boldsymbol{x}|\boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k)}{\sum_{j=1}^{K} \pi_j \mathcal{N}(\boldsymbol{x}|\boldsymbol{\mu}_j, \boldsymbol{\Sigma}_j)}. \tag{15.14}$$

π_k 可以视为 $z_k = 1$ 的先验概率,$\gamma(z_k)$ 可以视为观测到 $\boldsymbol{x}$ 之后的相应后验概率. 正如我们稍后将看到的,$\gamma(z_k)$ 也可以视为分量 k 对 “解释” 观测值 $\boldsymbol{x}$ 所要承担的责任 (responsibility).

我们找到了一个涉及显式潜变量的高斯混合分布的等效表示, 其中联合分布以图模型的形式给出. 这么做的收益看起来并不大. 然而, 现在我们已经能够用联合分布 $p(\boldsymbol{x}, \boldsymbol{z}) = p(\boldsymbol{z})p(\boldsymbol{x}|\boldsymbol{z})$ 代替边缘分布 $p(\boldsymbol{x})$ 工作, 这将带来显著的简化 (特别是在引入 EM 算法时).

15.2.1 似然函数

假设我们有一组观测数据 $\{\boldsymbol{x}_1, \cdots, \boldsymbol{x}_N\}$, 我们希望使用高斯混合模型来对这些数据进行建模. 我们可以将这个数据集表示为一个 $N \times D$ 的矩阵 $\boldsymbol{X}$, 其中第 n 行由 $\boldsymbol{x}_n^{\mathsf{T}}$ 给出.

由式(15.6)可知, 似然函数的对数由下式给出:

$$\ln p(\boldsymbol{X}|\boldsymbol{\pi}, \boldsymbol{\mu}, \boldsymbol{\Sigma}) = \sum_{n=1}^{N} \ln \left\{ \sum_{k=1}^{K} \pi_k \mathcal{N}(\boldsymbol{x}_n|\boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k) \right\}. \tag{15.15}$$

最大化这个对数似然函数比针对单个高斯分布更复杂. 难点在于式(15.15)中的对数内部的求和项会导致对数不再直接作用于高斯分布. 如果将对数似然函数关于模型参数的导数设为零, 我们将不再获得闭合解, 这一点我们很快就会看到.

在讨论如何最大化这个函数之前, 值得强调的是, 在应用到高斯混合模型时, 最大似然框架存在一个显著的问题, 就是有奇异性. 为了简单起见, 考虑一个高斯混合模型, 其组成部分的协方差矩阵由 $\boldsymbol{\Sigma}_k = \sigma_k^2 \boldsymbol{I}$ 给出, 其中 $\boldsymbol{I}$ 是单位矩阵, 但所得结论对于一般的协方差矩阵也是成立的. 假设该高斯混合模型的一个分量, 比如第 j 个分量, 其均值 $\boldsymbol{\mu}_j$ 恰好等于某个数据点, 即 $\boldsymbol{\mu}_j = \boldsymbol{x}_n$, 则这个数据点将在似然函数中贡献一个如下形式的项:

$$\mathcal{N}(\boldsymbol{x}_n|\boldsymbol{x}_n, \sigma_j^2 \boldsymbol{I}) = \frac{1}{(2\pi\sigma_j^2)^{D/2}} \tag{15.16}$$

考虑极限 $\sigma_j \to 0$, 我们会看到该项趋向于无穷大, 所以对数似然函数也将趋向于无穷大. 由此可见, 最大化对数似然函数并不是一个良定义的问题, 因为这种奇异性总是存在的, 并且每当一个高斯分量 “坍缩” 到一个特定的数据点时就会发生. 这种奇异性提供了一个最大似然方法中可能出现过拟合的例子. 当把最大似然方法应用到高斯混合模型时, 我们必须采取措施以避免找到这样的病态 (pathological) 解, 比如转而寻求似然函数的局部最大值, 这些局部最大值通常表现良好. 我们可以试图通过使用适当的启发式方法来避免奇异性, 例如检测到高斯分量在坍缩时将其均值重置为随机选择的值, 同时将其协方差重置为某个较大值, 然后继续优化. 通过添加一个对应于参数先验分布的正则化项到对数似然函数中, 也可以避免奇异性 (参见小节 15.4.3).

在寻找最大似然解时还有一个问题, 因为对于任何给定的最大似然解, 一个包含 K 个组分的高斯混合模型将有总共 $K!$ 个等效解, 对应于将 K 组参数分配给 K 个组分的 $K!$ 种方式. 换句话说, 对于参数值空间中的任何给定 (非退化) 点, 都会存在额外的 $K! - 1$ 个点. 所有这些点都将产生完全相同的分布. 这个问题称为可识别性 (identifiability) 问题 (Casella and Berger, 2002), 当我们希望解释模型发现的参数值时, 这是一个重要问题. 当我们讨论具有连续潜变量的模型 (参见第 16 章) 时, 可识别性问题也会出现. 然而, 当我们寻找一个好的密度模型时, 这个问题无关紧要, 因为任何等效解都和其他解一样好.

15.2.2 最大似然

在为含有隐变量的模型寻找最大似然解的探索中,EM 算法是一种优雅而强大的方法 (Dempster, Laird and Rubin, 1977; McLachlan and Krishnan, 1997). 在本章中, 我们将先后给出三种不同的 EM 算法推导, 每一种都比前一种更加通用. 下面让我们在高斯混合模型的背景下以相对非正式的方式开始. 首先需要强调的是,EM 算法具有广泛的适用性, 它的底层概念我们将在本书涉及的多个模型中遇到.

我们选择从写下似然函数最大值处必须满足的条件开始. 将式(15.15)中的 $\ln p(\boldsymbol{X}|\boldsymbol{\pi}, \boldsymbol{\mu}, \boldsymbol{\Sigma})$ 关于高斯分量均值 $\boldsymbol{\mu}_k$ 的导数设为零, 便可得到

$$0 = \sum_{n=1}^{N} \underbrace{\frac{\pi_k \mathcal{N}(\boldsymbol{x}_n|\boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k)}{\sum_j \pi_j \mathcal{N}(\boldsymbol{x}_n|\boldsymbol{\mu}_j, \boldsymbol{\Sigma}_j)}}_{\gamma(z_{nk})} \boldsymbol{\Sigma}_k^{-1}(\boldsymbol{x}_n - \boldsymbol{\mu}_k) \tag{15.17}$$

其中我们利用了高斯分布的形式. 注意, 后验概率或责任 $\gamma(z_{nk})$ 由式(15.14)给出, 因而自然地出现在公式的右侧. 乘以 $\boldsymbol{\Sigma}_k^{-1}$(我们假设它是非奇异的) 并重新排列后, 便可得到

$$\boldsymbol{\mu}_k = \frac{1}{N_k} \sum_{n=1}^{N} \gamma(z_{nk}) \boldsymbol{x}_n \tag{15.18}$$

其中我们定义了

$$N_k = \sum_{n=1}^{N} \gamma(z_{nk}). \tag{15.19}$$

可以将 N_k 解释为分配给聚类 k 的有效点数. 仔细观察这个解的形式, 可以看到, 第 k 个高斯分量的均值 $\boldsymbol{\mu}_k$ 是通过对数据集中的所有点进行加权平均得到的. 在这个加权因子中, 数据点 $\boldsymbol{x}_n$ 的权重是由后验概率 $\gamma(z_{nk})$ 给出的, 即第 k 个高斯分量负责生成 $\boldsymbol{x}_n$ 的概率.

如果将 $\ln p(\boldsymbol{X}|\boldsymbol{\pi}, \boldsymbol{\mu}, \boldsymbol{\Sigma})$ 关于 $\boldsymbol{\Sigma}_k$ 的导数设为零并遵循类似的推理线索, 利用单个高斯分布的最大似然解的结果 (参见 3.2.7 小节), 便可得到

$$\boldsymbol{\Sigma}_k = \frac{1}{N_k} \sum_{n=1}^{N} \gamma(z_{nk})(\boldsymbol{x}_n - \boldsymbol{\mu}_k)(\boldsymbol{x}_n - \boldsymbol{\mu}_k)^{\mathsf{T}}. \tag{15.20}$$

这与用单个高斯分布拟合数据集的对应结果形式相同, 但每个数据点都由相应的后验概率加权, 并且分母由与相应分量相关联的有效点数给出.

最后, 我们调整混合系数 π_k 的值, 目的是最大化对数似然函数 $\ln p(\boldsymbol{X}|\boldsymbol{\pi}, \boldsymbol{\mu}, \boldsymbol{\Sigma})$. 此处我们必须考虑一个约束 [式(15.9)], 该约束要求混合系数的和为 1. 这可以通过使用拉格朗日乘子 λ 并最大化以下量来实现:

$$\ln p(\boldsymbol{X}|\boldsymbol{\pi}, \boldsymbol{\mu}, \boldsymbol{\Sigma}) + \lambda \left(\sum_{k=1}^{K} \pi_k - 1 \right) \tag{15.21}$$

这样便可得到

$$0 = \sum_{n=1}^{N} \frac{\mathcal{N}(\boldsymbol{x}_n|\boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k)}{\sum_j \pi_j \mathcal{N}(\boldsymbol{x}_n|\boldsymbol{\mu}_j, \boldsymbol{\Sigma}_j)} + \lambda. \tag{15.22}$$

可以看到 “责任” 再次出现了. 如果对式(15.22)的两边乘以 π_k 并利用式(15.9)对 k 进行求和, 就会发现 $\lambda = -N$. 消除 λ 并重新排列后, 便可得到

$$\pi_k = \frac{N_k}{N}. \tag{15.23}$$

因此, 第 k 个高斯分量的混合系数由该分量对解释数据点所要承担的平均责任给出.

请注意, 式(15.18)、式(15.20)和式(15.23)并不构成高斯混合模型参数的闭合解, 因为责任 $\gamma(z_{nk})$ 通过式(15.14)以复杂的方式依赖于这些参数. 然而, 这些结果确实提出了一种简单的迭代方案来寻找最大似然问题的解. 正如我们将看到的, 对于高斯混合模型的特定情况, 这实际上是 EM 算法的一个实例. 我们首先选择均值、协方差和混合系数的一些初始值, 然后交替进行以下两个步骤——期望步骤 (E 步骤) 和最大化步骤 (M 步骤). 在 E 步骤中, 我们使用参数的当前值来估计后验概率或责任, 其由式(15.14)给出. 然后在 M 步骤中使用这些概率重新估计均值、协方差和混合系数, 它们分别由式(15.18)、式(15.20)和式(15.23)给出.

在实践中, 当对数似然函数的值的变化, 或者说参数的值的变化低于某个阈值时, 算法就被认为已经收敛. 我们将证明每次进行 E 步骤之后, 紧跟着进行的 M 步骤导致的参数更新都会增大对数似然函数的值.

请注意, 与 K 均值算法相比,EM 算法需要进行更多的迭代才能达到 (近似) 收敛, 而且每个循环都需要进行更多的计算. 因此, 人们通常会运行 K 均值算法来找到高斯混合模型的初始状态, 随后使用 EM 算法进行调整. 一种便捷的做法就是把协方差矩阵初始化为 K 均值算法以找到各个聚类的样本协方差, 而混合系数可以设置为数据点被分配给相应聚类的比例. 为了规避似然函数的奇异性——高斯分量坍缩至特定数据点的情况——我们必须采取诸如参数正则化的技术. 需要强调的是, 对数似然函数通常会有多个局部最大值, 并且 EM 算法不能保证找到全局最大值.

高斯混合模型非常灵活, 如果模型参数选择得当, 只需要给定足够数量的分量, 就可以高精度地逼近复杂的数据分布. 然而在实践中, 尤其在高维空间中, 分量的数量通常会有多个局部最大值, 并且精确建模分布所需的分量的数量可能会大到不可接受. 尽管如此, 高斯混合模型仍十分有用. 此外, 理解高斯混合模型能为我们理解具有连续潜变量的模型和基于深度神经网络的生成式模型奠定基础, 后者对高维空间的扩展性要好得多 (参见第 16 章).

的变化，或者说参数的值的变化低于某个阈值时，算法就收敛了已经收敛。

我们用重缩放的老忠实喷泉数据集来说明应用于高斯混合模型的 EM 算法，所示。这里使用了两个高斯分量，其聚类中心使用与图 15.1 中 K 均值算法进行初始化，协方差矩阵则初始化为与单位矩阵成比例。图 15.7(a) 显示了绿点以及高斯混合模型的初始配置，其中两个高斯分量的一倍标准差轮廓线分蓝色和红色圆圈。图 15.7(b) 显示了 E 步骤的初始结果，其中每个数据点使用分量生成的后验概率相等的蓝色墨水比例绘制，并且绘制时还使用了由红色的后验概率所给出的相应比例的红色墨水。因此，那些属于两个聚类的概率的点呈现为紫色。第一次进行 M 步骤后的情况如图 15.7(c) 所示，蓝色分量移动到由每个数据点属于蓝色聚类的概率加权的数据集的均值上。换句话说，

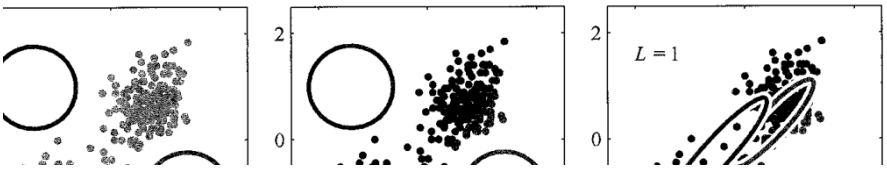


图 15.2: 将 EM 算法应用于老忠实喷泉数据集, 该数据集在图 15.1中用于说明 K 均值算法. 这里使用了两个高斯分量, 其聚类中心使用与图 15.1 中 K 均值算法相同的值进行初始化, 协方差矩阵则初始化为与单位矩阵成比例.(a) 初始配置;(b) E 步骤;(c)–(f) 分别展示了进行 2 次、 5 次和 20 次完整的 EM 循环后的结果. 在图 (f) 中, 算法接近收敛.

15.3 EM 算法

本节将讨论更一般的 EM 算法, 并特别关注隐变量的作用. 如前所述, 我们用 $\boldsymbol{X}$ 表示所有观测数据点的集合, 其中的第 n 行代表 $\boldsymbol{x}_n$. 同样, 相应的隐变量则用一个 $N \times K$ 的矩阵 $\boldsymbol{Z}$ 来表示, 每一行记为 $\boldsymbol{z}_n$. 如果我们假设这些数据点是从分布中独立抽取的, 则可以使用图形来表示这个独立同分布数据集的高斯混合模型. 所有模型参数的集合记为 θ, 因此对数似然函数由下式给出:

$$\ln p(\boldsymbol{X}|\theta) = \sum_{n=1}^{N} \ln \left\{ \sum_{\boldsymbol{z}_n} p(\boldsymbol{x}_n, \boldsymbol{z}_n|\theta) \right\}. \tag{15.24}$$

注意, 我们的讨论同样适用于连续隐变量, 只需要将对 $\boldsymbol{Z}$ 的求和替换为积分即可.

现在假设对于 $\boldsymbol{X}$ 中的每个观测值, 我们都被告知对应的隐变量 $\boldsymbol{Z}$ 的值. 此时将 $\{\boldsymbol{X}, \boldsymbol{Z}\}$ 称为完整数据集, 并将实际观测到的数据 $\boldsymbol{X}$ 视为不完整的 (incomplete). 完整数据集的似然函数可以简单地采取 $\ln p(\boldsymbol{X}, \boldsymbol{Z}|\theta)$ 的形式, 我们假设这个完整数据集的对数似然函数的最大化是比较简单的.

然而在实践中, 我们没有得到完整的数据集 $\{\boldsymbol{X}, \boldsymbol{Z}\}$, 而只有不完整的数据 $\boldsymbol{X}$. 隐变量 $\boldsymbol{Z}$ 的状态信息仅由后验分布 $p(\boldsymbol{Z}|\boldsymbol{X}, \theta)$ 给出. 由于无法使用完整数据集的对数似然函数, 我们转而考虑其在隐变量的后验分布下的期望值, 这对应于 (正如我们将看到的)EM 算法的 E 步骤. 在随后的 M 步骤中, 我们最大化这个期望值. 如果当前对参数的估计记为 θ^{old}, 那么一对连续的 E 步骤和 M 步骤将产生修正后的估计 θ^{new}.

EM 算法通过选择参数的某个起始值 θ^{old} 来进行初始化. 尽管这种期望的使用看起来有些武断, 但是当我们在节 15.4对 EM 算法进行更深入的讨论时, 我们将看到做出这个选择的动机. 在 E 步骤中, 我们使用当前的参数值 θ^{old} 来找出 $p(\boldsymbol{Z}|\boldsymbol{X}, \theta^{\text{old}})$ 给出的隐变量的后验分布. 然后使用这个后验分布找出用于对一些通用参数值 θ 进行估计的完整数据对数似然函数的期望. 将这个期望记为 $\boldsymbol{Q}(\theta, \theta^{\text{old}})$, 其由下式给出:

$$\boldsymbol{Q}(\theta, \theta^{\text{old}}) = \sum_{\boldsymbol{Z}} p(\boldsymbol{Z}|\boldsymbol{X}, \theta^{\text{old}}) \ln p(\boldsymbol{X}, \boldsymbol{Z}|\theta). \tag{15.25}$$

在 M 步骤中, 我们通过最大化式(15.25)所示的函数来确定修正后的参数估计:

$$\theta^{\text{new}} = \arg\max_{\theta} \boldsymbol{Q}(\theta, \theta^{\text{old}}). \tag{15.26}$$

请注意, 在函数 $\boldsymbol{Q}(\theta, \theta^{\text{old}})$ 的定义中, 对数直接作用于联合分布 $p(\boldsymbol{X}, \boldsymbol{Z}|\theta)$, 因此相应的 M 步骤根据我们的假设

是可行的. 正如我们稍后将展示的, 每个 EM 循环都会增大不完整数据对数似然函数的值 (除非它已经处于局部最大值)(参见小节 15.4.1).

EM 算法也可以用来寻找 MAP(最大后验概率) 解, 这适用于定义参数时引入先验分布 $p(\theta)$ 的模型 (见习题 15.5). 在这种情况下,E 步骤与最大似然示例中的情形相同, 而在 M 步骤中, 要最大化的量由 $Q(\theta, \theta^{\text{old}}) + \ln p(\theta)$ 给出. 适当的先验选择能消除**??**所示类型的奇异性.

当未观测变量对应于数据集中缺失的值时, 也可以应用 EM 算法. 观测值的分布是通过取所有变量的联合分布, 然后在缺失的变量上进行边缘化得到的. 接下来便可以使用 EM 算法来最大化相应的似然函数. 如果数据值是随机缺失的, 则意味着导致缺失值的机制不依赖于未观测到的值. 但在许多情况下, 事实并非如此. 例如, 当传感器测量的量超过某个阈值时, 传感器就无法返回值.

15.3.1 高斯混合模型

现在我们考虑将这种隐变量视角的 EM 算法应用于高斯混合模型的特定案例. 回顾一下, 我们的目标是最大化对数似然函数(15.15), 该函数可以使用观测数据集 $\boldsymbol{X}$ 来计算, 并且我们发现由于在对数内部进行了关于 k 的求和, 这比单个高斯分布的情况要困难得多. 假设除了观测数据集 $\boldsymbol{X}$, 我们还得到了相应的离散变量 $\boldsymbol{Z}$ 的值. 回想一下, 图 15.5(a) 显示了一个完整数据集 (即包含标签来显示每个数据点是由哪个分量生成的), 图 15.5(b) 则显示了相应的不完整数据集.

现在考虑对完整数据集 $\{\boldsymbol{X}, \boldsymbol{Z}\}$ 的似然函数进行最大化的问题. 根据式(15.10)和式(15.12), 这个似然函数的形式如下:

$$p(\boldsymbol{X}, \boldsymbol{Z}|\boldsymbol{\mu}, \boldsymbol{\Sigma}, \boldsymbol{\pi}) = \prod_{n=1}^{N}\prod_{k=1}^{K} \pi_k^{z_{nk}} \mathcal{N}(\boldsymbol{x}_n|\boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k)^{z_{nk}} \tag{15.27}$$

其中 z_{nk} 表示 $\boldsymbol{z}_n$ 的第 k 个分量. 取对数后, 可以得到

$$\ln p(\boldsymbol{X}, \boldsymbol{Z}|\boldsymbol{\mu}, \boldsymbol{\Sigma}, \boldsymbol{\pi}) = \sum_{n=1}^{N}\sum_{k=1}^{K} z_{nk}\left\{\ln \pi_k + \ln \mathcal{N}(\boldsymbol{x}_n|\boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k)\right\}. \tag{15.28}$$

与不完整数据的对数似然函数 [式(15.15)] 相比, 可以看出 k 的求和项与对数项已经交换了位置. 对数现在直接作用于高斯分布, 而高斯分布本身就是指数族分布的成员. 显然, 这使得最大似然问题的解决方案简单多了. 首先考虑均值和协方差的最大化问题. 因为 $\boldsymbol{z}_n$ 是一个 K 维向量, 除了其中一个元素的值为 1, 其他所有元素的值都为 0, 所以完整数据的对数似然函数仅仅是 K 个独立贡献的和, 每个分量对应一个贡献. 因此, 均值或协方差的最大化处理实际上就像对单个高斯分布的处理一样, 只不过前者仅涉及“分配”给该分量的数据点子集. 对于混合系数的最大化, 注意, 由于式(15.9)中的求和约束, 不同 k 值的这些系数是耦合的. 另外, 和之前一样, 这也可以使用拉格朗日乘子来保证, 从而导致:

$$\pi_k = \frac{1}{N}\sum_{n=1}^{N} z_{nk}. \tag{15.29}$$

因此, 混合系数等于分配给相应分量的数据点的比例.

然而, 实际上我们没有隐变量的值. 如前所述, 我们考虑了关于隐变量后验分布的完整数据对数似然函数的期望. 使用式(15.10)和式(15.12)并结合贝叶斯定理, 这个后验分布的形式如下:

$$p(\boldsymbol{Z}|\boldsymbol{X}, \boldsymbol{\mu}, \boldsymbol{\Sigma}, \boldsymbol{\pi}) \propto \prod_{n=1}^{N}\prod_{k=1}^{K} \left[\pi_k \mathcal{N}(\boldsymbol{x}_n|\boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k)\right]^{z_{nk}}. \tag{15.30}$$

可以看到这个分布在 n 上是可分解的, 所以在这个后验分布下,$\boldsymbol{z}_n$ 是独立的 (见习题 15.6), 这可以很容易地通过检查图 15.9 中的有向图并使用 d 分离准则来验证 (参见 11.2 节). 在这个后验分布下, 指示变量 z_{nk} 的期望由下式给出:

$$\mathbb{E}[z_{nk}] = \gamma(z_{nk}) = \frac{\pi_k \mathcal{N}(\boldsymbol{x}_n|\boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k)}{\sum_j \pi_j \mathcal{N}(\boldsymbol{x}_n|\boldsymbol{\mu}_j, \boldsymbol{\Sigma}_j)}. \tag{15.31}$$

这正是第 k 个高斯分量对数据点 $\boldsymbol{x}_n$ 所要承担的责任. 因此, 完整数据对数似然函数的期望由下式给出:

$$\mathbb{E}_{\boldsymbol{Z}}\left[\ln p(\boldsymbol{X}, \boldsymbol{Z}|\boldsymbol{\mu}, \boldsymbol{\Sigma}, \boldsymbol{\pi})\right] = \sum_{n=1}^{N}\sum_{k=1}^{K}\gamma(z_{nk})\left\{\ln \pi_k + \ln \mathcal{N}(\boldsymbol{x}_n|\boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k)\right\}. \tag{15.32}$$

我们现在可以按照以下步骤进行操作. 首先为参数 $\boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k$ 和 π_k 选择一些初始值, 并使用这些值来估计责任 (E 步骤). 然后保持责任固定, 并通过最大化 $\mathbb{E}_{\boldsymbol{Z}}[\ln p(\boldsymbol{X}, \boldsymbol{Z}|\cdot)]$ 来最大化期望, 见式(15.32)(M 步骤). 因此前面提到的闭合解 $\boldsymbol{\mu}_k^{\text{new}}$、$\boldsymbol{\Sigma}_k^{\text{new}}$ 和 π_k^{new} 分别由式(15.18)、式(15.20)和式(15.23)给出. 这正是我们早些时候推导出的应用于高斯混合模型的 EM 算法.

15.3.2 EM 算法与 K 均值算法的关系

对 K 均值算法与 EM 算法进行比较后, 可以发现它们之间有很大的相似性.K 均值算法把数据点硬 (hard) 分配到聚类中, 每个数据点唯一地与一个聚类相关联, 而 EM 算法基于后验概率进行软 (soft) 分配. 事实上, 我们可以从 EM 算法推导出 K 均值算法.

考虑一个高斯混合模型, 其中混合分量的协方差矩阵由 $\epsilon\boldsymbol{I}$ 给出, 其中 ϵ 是所有分量共享的方差参数,$\boldsymbol{I}$ 是单位矩阵, 因此

$$p(\boldsymbol{x}|\boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k) = \frac{1}{(2\pi\epsilon)^{D/2}}\exp\left\{-\frac{1}{2\epsilon}\|\boldsymbol{x} - \boldsymbol{\mu}_k\|^2\right\}. \tag{15.33}$$

考虑将要应用于这种形式的包含 K 个分量的高斯混合模型的 EM 算法, 其中我们将 ϵ 当作固定常数而不是重新估计的参数处理. 根据式(15.31), 特定数据点 $\boldsymbol{x}_n$ 的后验概率或责任由下式给出:

$$\gamma(z_{nk}) = \frac{\pi_k \exp\left\{-\frac{1}{2\epsilon}\|\boldsymbol{x}_n - \boldsymbol{\mu}_k\|^2\right\}}{\sum_j \pi_j \exp\left\{-\frac{1}{2\epsilon}\|\boldsymbol{x}_n - \boldsymbol{\mu}_j\|^2\right\}}. \tag{15.34}$$

考虑极限 $\epsilon \to 0$, 式(15.34)右侧的分母中包含了以 j 索引的多个趋于零的项. 假设使得 $\|\boldsymbol{x}_n - \boldsymbol{\mu}_j\|^2$ 最小的特定项 (例如 $j = 1$ 的项) 将会最慢地趋于零并支配该平方和. 因此, 数据点 $\boldsymbol{x}_n$ 的责任 $\gamma(z_{nk})$ 除了第 1 项外都会趋于零, 第 1 项的责任 $\gamma(z_{n1})$ 将趋于 1. 注意, 这独立于 π_k 的值, 只要没有任何 π_k 为零即可. 因此, 在这个极限下, 我们获得了数据点到聚类的硬分配, 就像在 K 均值算法中一样, 所以 $\gamma(z_{nk}) \to r_{nk}$, 其中 r_{nk} 由式(15.2)定义. 每个数据点因此被分配到与其最近的均值所代表的聚类. 然后 EM 算法中 $\boldsymbol{\mu}_k$ 的重估方程由式(15.18)给出, 并简化为 K 均值算法的结果 [式(15.4)]. 注意, 混合系数的重估方程 [式(15.23)] 仅是将 π_k 的值重置为分配给聚类 k 的数据点的比例, 尽管这些参数在算法中已不再起积极作用.

最后, 在极限 $\epsilon \to 0$ 下, 用来给出完整数据对数似然函数期望的式(15.32)(见习题 15.12), 就可以变为

$$\mathbb{E}_{\boldsymbol{Z}}\left[\ln p(\boldsymbol{X}, \boldsymbol{Z}|\boldsymbol{\mu}, \boldsymbol{\Sigma}, \boldsymbol{\pi})\right] \to -\frac{1}{2}\sum_{n=1}^{N}\sum_{k=1}^{K} r_{nk}\|\boldsymbol{x}_n - \boldsymbol{\mu}_k\|^2 + \text{常数}. \tag{15.35}$$

因此, 我们看到在这个极限下, 完整数据对数似然函数的最大化期望等价于最小化 K 均值算法的误差度量 J,J 由式(15.1)给出. 注意, K 均值算法不估计聚类的协方差, 只估计聚类的均值.

15.3.3 混合伯努利分布

在本章中, 目前主要关注的是由高斯混合分布描述的连续变量模型. 为了在不同的情境中阐述 EM 算法, 现在讨论混合建模的另一个例子——由伯努利分布描述的离散二元变量混合模型. 这个模型也称潜在类别分析模型 (Lazarsfeld and Henry, 1968; McLachlan and Peel, 2000).

考虑一组 D 个二元变量 $x_i \in \{0, 1\}$,$i = 1, \cdots, D$, 其中的每一个二元变量都服从参数为 μ_i 的伯努利分布, 因此

$$p(\boldsymbol{x}|\boldsymbol{\mu}) = \prod_{i=1}^{D}\mu_i^{x_i}(1 - \mu_i)^{1-x_i} \tag{15.36}$$

其中 $\boldsymbol{x} = (x_1, \cdots, x_D)$,$\boldsymbol{\mu} = (\mu_1, \cdots, \mu_D)$. 可以看出, 在给定 $\boldsymbol{\mu}$ 的情况下, 各个变量 x_i 是独立的, 这个分布的均值和

协方差可以很容易地得到 (见习题 15.13)

$$\mathbb{E}[\boldsymbol{x}] = \boldsymbol{\mu} \tag{15.37}$$

$$\mathrm{cov}[\boldsymbol{x}] = \mathrm{diag}\{\mu_i(1-\mu_i)\}. \tag{15.38}$$

考虑这些分布的有限混合分布, 表示为

$$p(\boldsymbol{x}|\boldsymbol{\mu},\boldsymbol{\pi}) = \sum_{k=1}^{K} \pi_k\, p(\boldsymbol{x}|\boldsymbol{\mu}_k) \tag{15.39}$$

其中 $\boldsymbol{\mu} = \{\boldsymbol{\mu}_1, \cdots, \boldsymbol{\mu}_K\}$,$\boldsymbol{\pi} = \{\pi_1, \cdots, \pi_K\}$, 并且

$$p(\boldsymbol{x}|\boldsymbol{\mu}_k) = \prod_{i=1}^{D} \mu_{ki}^{x_i}(1-\mu_{ki})^{1-x_i}. \tag{15.40}$$

混合系数满足式(15.8)和式(15.9)所示的条件. 这个混合分布的均值和协方差分别如下 (见习题 15.14):

$$\mathbb{E}[\boldsymbol{x}] = \sum_{k=1}^{K} \pi_k \boldsymbol{\mu}_k \tag{15.41}$$

$$\mathrm{cov}[\boldsymbol{x}] = \sum_{k=1}^{K} \pi_k \left\{\boldsymbol{\Sigma}_k + \boldsymbol{\mu}_k \boldsymbol{\mu}_k^{\mathsf{T}}\right\} - \mathbb{E}[\boldsymbol{x}]\mathbb{E}[\boldsymbol{x}]^{\mathsf{T}} \tag{15.42}$$

其中 $\boldsymbol{\Sigma}_k = \mathrm{diag}\{\mu_{ki}(1-\mu_{ki})\}$. 由于协方差矩阵 $\mathrm{cov}[\boldsymbol{x}]$ 不再是对角矩阵, 这个混合分布能够捕捉变量之间的相关性, 这与单个伯努利分布不同.

如果我们得到一个数据集 $\boldsymbol{X} = \{\boldsymbol{x}_1, \cdots, \boldsymbol{x}_N\}$, 那么该模型的对数似然函数可以表示为

$$\ln p(\boldsymbol{X}|\boldsymbol{\mu},\boldsymbol{\pi}) = \sum_{n=1}^{N} \ln\left\{\sum_{k=1}^{K} \pi_k p(\boldsymbol{x}_n|\boldsymbol{\mu}_k)\right\}. \tag{15.43}$$

我们再次看到对数内部出现了求和, 因此最大似然解不再有闭合形式.

下面让我们为混合伯努利分布的似然函数最大化推导 EM 算法. 为此, 首先引入一个与每个实例 $\boldsymbol{x}_n$ 关联的显式离散隐变量 $\boldsymbol{z}_n$. 与高斯混合模型一样,$\boldsymbol{z}_n$ 采用 1-of-K 编码方案, 所以 $\boldsymbol{z}_n = (z_{n1}, \cdots, z_{nK})$ 是一个二元的 K 维向量, 有一个分量等于 1, 所有其他分量等于 0. 我们可以将给定隐变量的 $\boldsymbol{x}$ 的条件分布写为

$$p(\boldsymbol{x}|\boldsymbol{z},\boldsymbol{\mu}) = \prod_{i=1}^{D}\prod_{k=1}^{K} p(x_i|\mu_{ki})^{z_{nk}} \tag{15.44}$$

而隐变量的先验分布与高斯混合模型中的相同, 因此

$$p(\boldsymbol{z}|\boldsymbol{\pi}) = \prod_{k=1}^{K} \pi_k^{z_{nk}}. \tag{15.45}$$

将 $p(\boldsymbol{x}|\boldsymbol{z},\boldsymbol{\mu})$ 和 $p(\boldsymbol{z}|\boldsymbol{\pi})$ 相乘, 然后对 $\boldsymbol{z}$ 进行边缘化, 即可回到式(15.39)(见习题 15.16).

为了推导 EM 算法, 我们首先写下完整数据对数似然函数:

$$\ln p(\boldsymbol{X},\boldsymbol{Z}|\boldsymbol{\mu},\boldsymbol{\pi}) = \sum_{n=1}^{N}\sum_{k=1}^{K} z_{nk}\left\{\ln \pi_k + \sum_{i=1}^{D}\left[x_i \ln \mu_{ki} + (1-x_i)\ln(1-\mu_{ki})\right]\right\}. \tag{15.46}$$

其中 $\boldsymbol{X} = \{\boldsymbol{x}_n\}$,$\boldsymbol{Z} = \{\boldsymbol{z}_n\}$. 接下来取完整数据对数似然函数关于隐变量后验分布的期望, 得到

$$\mathbb{E}_{\boldsymbol{Z}}\left[\ln p(\boldsymbol{X},\boldsymbol{Z}|\boldsymbol{\mu},\boldsymbol{\pi})\right] = \sum_{n=1}^{N}\sum_{k=1}^{K} \gamma(z_{nk})\left\{\ln \pi_k + \sum_{i=1}^{D}\left[x_{ni} \ln \mu_{ki} + (1-x_{ni})\ln(1-\mu_{ki})\right]\right\}, \tag{15.47}$$

其中 $\gamma(z_{nk}) = \mathbb{E}[z_{nk}]$ 是给定数据点 $\boldsymbol{x}_n$ 时分量 k 的后验概率或责任. 在 E 步骤中, 使用贝叶斯定理计算这些责任, 形式如下:

$$\gamma(z_{nk}) = \frac{\pi_k p(\boldsymbol{x}_n|\boldsymbol{\mu}_k)}{\sum_j \pi_j p(\boldsymbol{x}_n|\boldsymbol{\mu}_j)}. \tag{15.48}$$

在 M 步骤中, 针对参数 $\boldsymbol{\mu}_k$ 和 π_k 最大化完整数据对数似然函数的期望. 如果我们把式(15.47)关于 μ_{ki} 求导并

将导数设为零 (见习题 15.17), 然后重新排列各项, 便可得到

$$\mu_{ki} = \frac{\sum_n \gamma(z_{nk}) x_{ni}}{N_k} \tag{15.49}$$

其中 $N_k = \sum_n \gamma(z_{nk})$ 是与分量 k 相关联的数据点的有效数量. 分量 k 的均值设置为数据的加权均值, 权重系数由分量 k 对每个数据点所要承担的责任给出. 在关于 π_k 进行最大化的过程中, 我们需要引入一个拉格朗日乘子来确保约束条件 $\sum_k \pi_k = 1$ 得到满足. 遵循与高斯混合类似的步骤 (见习题 15.18), 我们得到

$$\pi_k = \frac{N_k}{N}. \tag{15.50}$$

这代表了一个直观合理的结果, 即分量 k 的混合系数由数据集中被该分量解释的数据点的有效比例给出.

注意, 与高斯混合模型相比, 这里不存在使似然函数趋于无穷大的奇异点. 这可通过似然函数上界有限 (见习题 15.19) 看出来, 因为 $0 \leq p(\boldsymbol{x}|\boldsymbol{\mu}) \leq 1$. 虽然存在使似然函数为零的解, 但只要 EM 算法不是初始化到一个病态起点, 这些解就不会被 EM 算法找到, 因为 EM 算法总是增大似然函数的值, 直至找到一个局部最大值 (参见节 15.4).

15.4 证据下界

本节从更一般的角度介绍 EM 算法. 推导对数似然函数的一个下界, 这个下界称为证据下界 (Evidence Lower BOund,ELBO), 有时也称变分下界. 这里的 “证据” 是指 (对数) 似然函数, 在贝叶斯设置中有时也称 “模型证据”, 因为它允许在不使用验证集的情况下比较不同的模型 (Bishop, 2006). 为了说明这个下界, 我们使用它从第三个角度重新推导 EM 算法.ELBO 将在后续章节讨论的几个深度生成式模型中发挥重要作用. 它还提供了一个关于变分框架的样板, 在这个框架中, 我们引入了一个关于隐变量的分布 $q(\boldsymbol{Z})$, 然后使用变分法针对这个分布进行优化 (见附录 B).

考虑一个概率模型, 在该模型中我们将所有观测变量统称为 $\boldsymbol{X}$, 并将所有隐变量统称为 $\boldsymbol{Z}$. 联合分布 $p(\boldsymbol{X}, \boldsymbol{Z}|\theta)$ 由一组参数 θ 控制. 我们的目标是最大化似然函数:

$$p(\boldsymbol{X}|\theta) = \sum_{\boldsymbol{Z}} p(\boldsymbol{X}, \boldsymbol{Z}|\theta). \tag{15.51}$$

在这里我们假设 $\boldsymbol{Z}$ 是离散的, 当 $\boldsymbol{Z}$ 包含连续变量, 或者包含离散变量与连续变量的组合时, 要讨论的内容是相同的, 将求和替换为适当的积分即可.

假设直接优化 $p(\boldsymbol{X}|\theta)$ 是困难的, 但优化完整数据似然函数 $p(\boldsymbol{X}, \boldsymbol{Z}|\theta)$ 则简单得多. 接下来我们引入一个定义在隐变量上的分布 $q(\boldsymbol{Z})$, 我们观察到, 对所选择的任何分布 $q(\boldsymbol{Z})$, 以下分解成立:

$$\ln p(\boldsymbol{X}|\theta) = \mathcal{L}(q, \theta) + \mathrm{KL}(q(\boldsymbol{Z}) \,\|\, p(\boldsymbol{Z}|\boldsymbol{X}, \theta)) \tag{15.52}$$

其中

$$\mathcal{L}(q, \theta) = \sum_{\boldsymbol{Z}} q(\boldsymbol{Z}) \ln \left\{ \frac{p(\boldsymbol{X}, \boldsymbol{Z}|\theta)}{q(\boldsymbol{Z})} \right\} \tag{15.53}$$

而

$$\mathrm{KL}(q(\boldsymbol{Z}) \,\|\, p(\boldsymbol{Z}|\boldsymbol{X}, \theta)) = -\sum_{\boldsymbol{Z}} q(\boldsymbol{Z}) \ln \left\{ \frac{p(\boldsymbol{Z}|\boldsymbol{X}, \theta)}{q(\boldsymbol{Z})} \right\}. \tag{15.54}$$

注意 $\mathcal{L}(q, \theta)$ 既是分布 $q(\boldsymbol{Z})$ 的泛函, 也是参数 θ 的函数. 仔细研究式(15.53)和式(15.54)的形式, 并特别注意它们在符号上的差异,$\mathcal{L}(q, \theta)$ 包含 $\boldsymbol{X}$ 和 $\boldsymbol{Z}$ 的联合分布, 而 $\mathrm{KL}(q\|p)$ 包含给定 $\boldsymbol{X}$ 的 $\boldsymbol{Z}$ 的条件分布 (见习题 15.21). 为了验证式(15.52), 我们首先利用概率的乘积法则给出:

$$\ln p(\boldsymbol{X}, \boldsymbol{Z}|\theta) = \ln p(\boldsymbol{Z}|\boldsymbol{X}, \theta) + \ln p(\boldsymbol{X}|\theta). \tag{15.55}$$

然后将其代入 $\mathcal{L}(q, \theta)$ 的表达式中. 这产生了两项: 其中一项抵消了 $\mathrm{KL}(q\|p)$; 而由于 $q(\boldsymbol{Z})$ 是一个归一化分布 (其总和为 1), 另一项给出了所需的对数似然函数 $\ln p(\boldsymbol{X}|\theta)$.

从式(15.54)中我们看到,$\mathrm{KL}(q\|p)$ 是 $q(\boldsymbol{Z})$ 与后验分布 $p(\boldsymbol{Z}|\boldsymbol{X}, \theta)$ 之间的 Kullback–Leibler 散度.Kullback–

Leibler 散度满足 $\mathrm{KL}(q\|p) \geq 0$, 当且仅当 $q(\boldsymbol{Z}) = p(\boldsymbol{Z}|\boldsymbol{X}, \theta)$ 时等号成立. 因此, 从式(15.52)可以得出 $\mathcal{L}(q, \theta) \leq \ln p(\boldsymbol{X}|\theta)$, 换句话说,$\mathcal{L}(q, \theta)$ 是 $\ln p(\boldsymbol{X}|\theta)$ 的一个下界.

15.4.1 EM 算法回顾

我们可以使用式(15.52)来推导 EM 算法, 并证明它确实能够最大化对数似然函数. 假设当前的参数向量值为 θ^{old}. 在 E 步骤中, 固定 θ^{old} 并通过优化 $q(\boldsymbol{Z})$ 来最大化下界 $\mathcal{L}(q, \theta)$. 这个最大化问题的解可以很容易地看出来, 因为 $\ln p(\boldsymbol{X}|\theta)$ 的值不依赖于 $q(\boldsymbol{Z})$, 所以当 Kullback–Leibler 散度消失时, 即 $q(\boldsymbol{Z})$ 等于后验分布 $p(\boldsymbol{Z}|\boldsymbol{X}, \theta^{\mathrm{old}})$ 时,$\mathcal{L}(q, \theta)$ 的值最大. 在这种情况下, 下界将等于对数似然函数的值, 同时 KL 散度消失.

在随后的 M 步骤中, 固定分布 $q(\boldsymbol{Z})$ 并通过优化 θ 来最大化下界 $\mathcal{L}(q, \theta)$, 得到新的参数值 θ^{new}. 这将导致下界增大 (除非它已经达到最大值), 这必然也会导致相应的对数似然函数增大. 由于我们确定分布 $q(\boldsymbol{Z})$ 使用的是旧值而不是新值, 并且在 M 步骤中保持固定, 它不等于新的后验分布 $p(\boldsymbol{Z}|\boldsymbol{X}, \theta^{\mathrm{new}})$, 因此会有一个非零的 Kullback–Leibler 散度. 对数似然函数的增量将大于下界的增量.

如果将 $q(\boldsymbol{Z}) = p(\boldsymbol{Z}|\boldsymbol{X}, \theta^{\mathrm{old}})$ 代入式(15.53), 则可以看到, 在 E 步骤之后, 下界采取以下形式:

$$\mathcal{L}(q, \theta) = \sum_{\boldsymbol{Z}} p(\boldsymbol{Z}|\boldsymbol{X}, \theta^{\mathrm{old}}) \ln p(\boldsymbol{X}, \boldsymbol{Z}|\theta) - \sum_{\boldsymbol{Z}} p(\boldsymbol{Z}|\boldsymbol{X}, \theta^{\mathrm{old}}) \ln p(\boldsymbol{Z}|\boldsymbol{X}, \theta^{\mathrm{old}}) \tag{15.56}$$

其中的常数项 const 只是分布 $q(\boldsymbol{Z})$ 的负熵, 因此与 θ 无关. 在这里我们将 $Q(\theta, \theta^{\mathrm{old}})$ 识别为式(15.25)定义的完整数据对数似然函数的期望, 因此它是 M 步骤中被最大化的量, 正如我们早先在高斯混合模型中看到的那样.

EM 算法也可以从参数空间的角度来观察. 这里红色曲线描述了我们希望最大化的 (不完整数据的) 对数似然函数. 我们从某个初始参数值 θ^{old} 开始. 在第一次 E 步骤中, 我们评估隐变量的后验分布, 产生了一个下界 $\mathcal{L}(q, \theta)$, 其值在 θ^{old} 处等于对数似然函数的值. 在 M 步骤中, 下界被最大化, 得到新的参数值 θ^{new}, 它能给出比 θ^{old} 更大的对数似然值. 随后的 E 步骤构建了一个在 θ^{new} 处相切的下界. 注意, 这个下界曲线在 θ^{old} 处与对数似然函数曲线呈切线接触, 因此这两条曲线具有相同的梯度. 这个下界是一个凸函数, 具有唯一的最大值 (用于来自指数族分布的模型).

我们已经看到,EM 算法的 E 步骤和 M 步骤都会使良定义的对数似然函数的下界值增大, 并且完整的 EM 循环将以这样一种方式改变模型参数; 使对数似然值增大 (除非它已经达到最大值, 在这种情况下, 参数保持不变).

15.4.2 独立同分布数据

对于一个独立同分布的数据集,$\boldsymbol{X}$ 将包含 N 个数据点 $\{\boldsymbol{x}_n\}$, 而 $\boldsymbol{Z}$ 将包含相应的 N 个隐变量 $\{\boldsymbol{z}_n\}$, 其中 $n = 1, \cdots, N$. 根据独立性假设, 我们有 $p(\boldsymbol{X}, \boldsymbol{Z}) = \prod_n p(\boldsymbol{x}_n, \boldsymbol{z}_n)$. 通过对 $\{\boldsymbol{z}_n\}$ 进行边缘化, 我们得到 $p(\boldsymbol{X}) = \prod_n p(\boldsymbol{x}_n)$. 使用概率的求和法则和乘积法则, 则, 在 E 步骤中计算的后验概率具有以下形式:

$$p(\boldsymbol{Z}|\boldsymbol{X}, \boldsymbol{\mu}, \boldsymbol{\Sigma}, \boldsymbol{\pi}) = \prod_{n=1}^{N} p(\boldsymbol{z}_n|\boldsymbol{x}_n, \boldsymbol{\mu}, \boldsymbol{\Sigma}, \boldsymbol{\pi}) \tag{15.57}$$

则后验分布可以针对不同的数据点进行因子分解, 即

$$p(\boldsymbol{Z}|\boldsymbol{X}, \theta) = \prod_{n=1}^{N} p(\boldsymbol{z}_n|\boldsymbol{x}_n, \theta). \tag{15.58}$$

因此, 后验分布也可以针对 n 进行分解. 对于高斯混合模型, 这简单地表明每个混合分量对特定数据点 $\boldsymbol{x}_n$ 所要承担的责任仅依赖于 $\boldsymbol{x}_n$ 的值以及混合分量的参数 θ, 而不依赖于其他数据点的值.

15.4.3 参数先验

我们也可以使用 EM 算法来最大化引入了参数先验 $p(\theta)$ 的模型中的后验分布 $p(\theta|\boldsymbol{X})$. 为了理解这一点, 注意作为 θ 的函数, 我们有 $p(\theta|\boldsymbol{X}) = p(\theta, \boldsymbol{X})/p(\boldsymbol{X})$, 所以

$$\ln p(\theta|\boldsymbol{X}) = \ln p(\theta, \boldsymbol{X}) - \ln p(\boldsymbol{X}). \tag{15.59}$$

利用式(15.52), 我们有

$$\ln p(\theta|\boldsymbol{X}) = \mathcal{L}(q,\theta) + \mathrm{KL}(q\|p) + \ln p(\theta) - \ln p(\boldsymbol{X}) \tag{15.60}$$

其中 $\ln p(\boldsymbol{X})$ 是一个常数. 我们可以再次交替地针对 q 和 θ 优化式(15.60)右侧的表达式. 针对 q 的优化导致与标准 EM 算法相同的 E 步骤方程, 因为 q 只出现在 $\mathcal{L}(q,\theta)$ 中. 通过引入先验项 $\ln p(\theta)$,M 步骤方程被修改, 这通常只需要对标准最大似然的 M 步骤方程进行小幅度的修改即可 (参见第 9 章). 附加项代表着一种正则化形式, 并且具有消除高斯混合模型似然函数奇异性的作用.

15.4.4 广义 EM 算法

EM 算法将最大化似然函数这个潜在的困难问题分解为两个阶段:E 步骤和 M 步骤, 每个步骤通常都更容易实现. 但尽管如此, 对于复杂模型来说,E 步骤或 M 步骤, 或者说两者都可能变得难以处理. 因此 EM 算法有两种扩展.

广义 EM(Generalized EM,GEM) 算法解决了难以处理的 M 步骤问题.GEM 算法不对 θ 最大化 $\mathcal{L}(q,\theta)$, 而是寻求以这样一种方式改变参数: 增大其值. 因为 $\mathcal{L}(q,\theta)$ 是对数似然函数的下界, 所以 GEM 算法的每一个完整循环都保证了能够增大对数似然函数的值 (除非参数已经对应于局部最大值). 利用 GEM 算法的一种方式是在 M 步骤中使用基于梯度的迭代优化算法. 另一种形式的 GEM 算法称为期望条件最大化算法 (expectation conditional maximization algorithm), 涉及在每个 M 步骤中进行多个约束优化 (Meng and Rubin, 1993). 例如, 参数可能划分为几组,M 步骤则拆分为多个步骤, 每个步骤涉及优化一个组, 同时固定其他组.

我们同样可以通过对 $q(\boldsymbol{Z})$ 进行部分优化而非完全优化来推广 EM 算法的 E 步骤 (Neal and Hinton, 1999). 正如我们所见, 对于任何给定的 θ 值,$\mathcal{L}(q,\theta)$ 关于 $q(\boldsymbol{Z})$ 有一个唯一的最大值, 对应于后验分布 $q^\star(\boldsymbol{Z}) = p(\boldsymbol{Z}|\boldsymbol{X},\theta)$, 并且对于这个选择的 $q(\boldsymbol{Z})$, 下界 $\mathcal{L}(q,\theta)$ 等于对数似然函数 $\ln p(\boldsymbol{X}|\theta)$. 因此, 任何收敛到 $\mathcal{L}(q,\theta)$ 全局最大值的算法都能找到一个同时也是对数似然函数 $\ln p(\boldsymbol{X}|\theta)$ 全局最大值的 θ 值. 只要 $p(\boldsymbol{X},\boldsymbol{Z}|\theta)$ 是 θ 的连续函数, 那么根据连续性,$\mathcal{L}(q,\theta)$ 的任何局部最大值也将是 $\ln p(\boldsymbol{X}|\theta)$ 的局部 (或全局) 最大值. 这种增量版本中的每个 E 步骤或 M 步骤都会增加 $\mathcal{L}(q,\theta)$ 的值, 如果算法收敛到 $\mathcal{L}(q,\theta)$ 的局部 (或全局) 最大值, 这将对应于对数似然函数 $\ln p(\boldsymbol{X}|\theta)$ 的局部 (或全局) 最大值.

15.4.5 顺序 EM 算法

考虑 N 个独立的数据点 $\boldsymbol{x}_1,\cdots,\boldsymbol{x}_N$ 及相应的隐变量 $\boldsymbol{z}_1,\cdots,\boldsymbol{z}_N$. 将联合分布 $p(\boldsymbol{X},\boldsymbol{Z}|\theta)$ 在数据点上分解, 这种结构可以在增量形式的 EM 算法中利用, 在每一个 EM 循环中, 一次只处理一个数据点. 在 E 步骤中, 我们不是重新计算所有数据点的责任, 而是只重新计算一个数据点的责任. 看起来随后的 M 步骤可能涉及对所有数据点的责任的计算. 然而, 如果混合分量是指数族分布, 则责任仅存在于简单的充分统计量 (sufficient statistic), 并且可以高效地更新. 例如, 考虑一个高斯混合模型, 假设我们对数据点 $\boldsymbol{x}_n$ 更新, 其中相应的旧责任值和新责任值分别表示为 $\gamma(z_{nk})$ 和 $\gamma^{\mathrm{new}}(z_{nk})$. 在 M 步骤中, 所需的充分统计量可以递增地更新. 例如, 对于均值, 充分统计量由式(15.18)和式(15.19)定义, 我们可以得到

$$N_k^{\mathrm{new}} = N_k^{\mathrm{old}} + \gamma^{\mathrm{new}}(z_{nk}) - \gamma(z_{nk}) \tag{15.61}$$

并且

$$\boldsymbol{\mu}_k^{\mathrm{new}} = \boldsymbol{\mu}_k^{\mathrm{old}} + \frac{\gamma^{\mathrm{new}}(z_{nk}) - \gamma(z_{nk})}{N_k^{\mathrm{new}}}(\boldsymbol{x}_n - \boldsymbol{\mu}_k^{\mathrm{old}}). \tag{15.62}$$

协方差和混合系数的相应结果是类似的. 因此,E 步骤和 M 步骤花费的时间都是固定的, 与数据点的数量无关. 因为参数在每个数据点之后就修正了, 而不是等到整个数据集处理完毕之后才修正, 所以这种增量版本比批处理版本收敛得更快.

习题

15.1(⋆) 考虑节 15.1讨论的 K 均值算法. 证明由于离散指示变量 r_{nk} 存在有限数量的可能取值, 对于每个这样的取值都有一个唯一的最优集合 $\{\boldsymbol{\mu}_k\}$, 因此 K 均值算法在有限次迭代后必定会收敛.

15.2(⋆⋆) 推导 K 均值算法的顺序形式. 在每一步, 考虑一个新的数据点 $\boldsymbol{x}_n$, 并且只更新与数据点 $\boldsymbol{x}_n$ 最近的原型向量. 从批处理设置中用于原型向量的式(15.4)出发, 将来自最后一个数据点 $\boldsymbol{x}_n$ 的贡献单独分离出来. 通过重新排列各项, 证明这种更新采取的是式(15.5)的形式. 注意, 由于在此推导过程中没有做任何近似, 得到的原型向量具有以下性质: 它们各自等于分配给它们的所有数据向量的均值.

15.3(⋆) 考虑一个高斯混合模型, 其中潜变量的边缘分布 $p(\boldsymbol{z})$ 由式(15.10)给出, 观测变量的条件分布 $p(\boldsymbol{x}|\boldsymbol{z})$ 由式(15.12)给出. 证明边缘分布 $p(\boldsymbol{x})$(通过对所有可能的 $\boldsymbol{z}$ 值求和 $p(\boldsymbol{z})p(\boldsymbol{x}|\boldsymbol{z})$ 而获得), 是形式为式(15.6)的高斯混合分布.

15.4(⋆) 证明在具有 K 个分量的混合模型中, 交换对称性导致的等价参数设置的数量是 $K!$.

15.5(⋆⋆) 假设我们希望使用 EM 算法来最大化包含潜变量的模型的参数后验分布 $p(\theta|\boldsymbol{X})$, 其中 $\boldsymbol{X}$ 是观测数据集. 证明 E 步骤在最大似然情况下保持不变, 而 M 步骤中要最大化的量由 $\mathcal{Q}(\theta, \theta^{\text{old}})+\ln p(\theta)$ 给出, 其中 $\mathcal{Q}(\theta, \theta^{\text{old}})$ 由式(15.25)定义.

15.6(⋆) 考虑图 15.9 所示的高斯混合模型的有向图. 利用 d 分离准则, 证明潜变量的后验分布可以关于不同的数据点进行因子分解, 即

$$p(\boldsymbol{Z}|\boldsymbol{X}, \boldsymbol{\mu}, \boldsymbol{\Sigma}, \boldsymbol{\pi}) = \prod_{n=1}^{N} p(\boldsymbol{z}_n|\boldsymbol{x}_n, \boldsymbol{\mu}, \boldsymbol{\Sigma}, \boldsymbol{\pi}). \tag{15.63}$$

15.7(⋆⋆) 考虑高斯混合模型的一种特殊情况, 其中每个分量的协方差矩阵 $\boldsymbol{\Sigma}_k$ 受限于它们所具有的一个共同的值 $\boldsymbol{\Sigma}$. 在这样的模型下推导用于最大化似然函数的 EM 方程.

15.8(⋆⋆) 验证对高斯混合模型的完整数据对数似然函数 [式(15.28)] 进行最大化给出的 EM 方程.

第 16 章 连续潜变量模型

16.1 主成分分析

在第 15 章中, 我们讨论了拥有离散潜变量的概率模型, 例如高斯混合模型. 本章探讨部分或全部潜变量为连续变量的模型. 研究这类模型的一个重要原因是很多数据集都有这样的特点: 数据点都挤在一个比起原始数据空间维度低得多的流形 (manifold) 上. 在实践中, 数据点不会精确地局限于一个光滑的低维流形上, 我们可以将数据点偏离流形的现象解释为" 噪声". 我们可以自然地从生成性视角 (generative view) 来理解此类模型: 首先根据某种潜变量分布在流形内选择一个点, 然后添加从给定潜变量的条件分布中抽取的噪声, 生成一个观测数据点.

对于最简单的连续潜变量模型 (continuous latent-variable model), 可以假设潜变量和观测变量都遵循高斯分布, 并利用线性–高斯依赖性 (linear-Gaussian dependence) 来描述观测变量对潜变量状态的依赖. 这引出了著名的主成分分析 (Principal Component Analysis, PCA) 概率公式, 还有称为因子分析 (factor analysis) 的相关模型. 本章首先介绍如何使用标准非概率方法来实现 PCA, 随后揭示 PCA 如何自然而然地对应于线性–高斯潜变量模型的最大似然解 (maximum likelihood solution). 这种概率化表述带来了许多优势, 比如使用 EM 算法进行参数估计、规范化地扩展成 PCA 混合模型, 以及通过贝叶斯公式自动从数据中确定主成分的数量.

主成分分析已广泛应用于降维、有损数据压缩、特征提取和数据可视化等领域, 它也称为 Kosambi–Karhunen–Loève 变换. 考虑将一个数据集正交投影到一个低维的线性空间 [即主子空间 (principal subspace)], 如图 16.1所示.

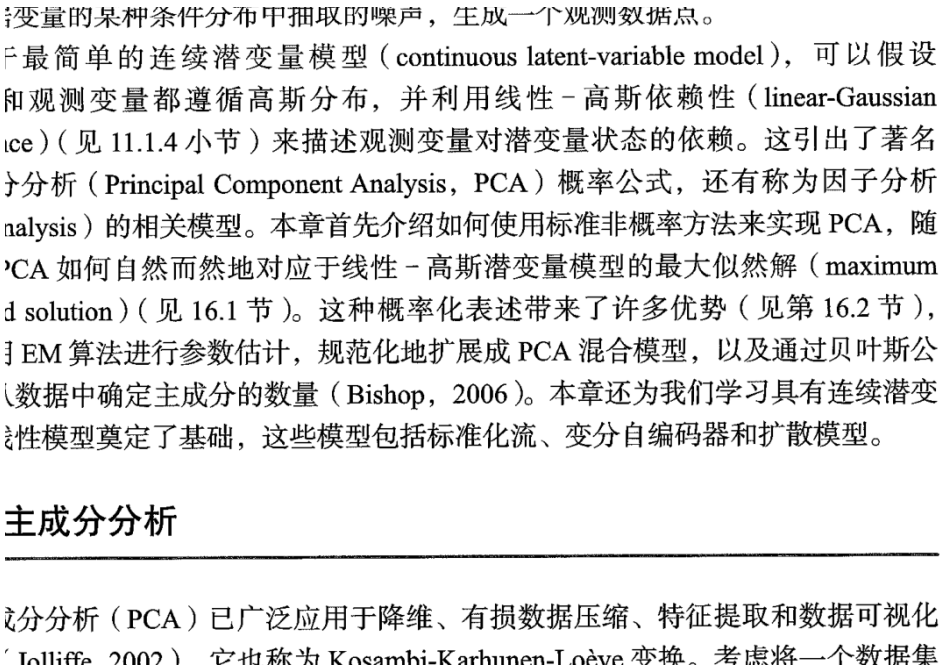


图 16.1: 利用 PCA 寻求一个低维空间, 称为主子空间 (principal subspace), 用洋红色线条表示, 以使数据点 (红色点) 在主子空间中的正交投影能够最大化投影点 (绿色点) 的方差.PCA 的另一种定义是基于最小化投影误差平方和给出的, 这些误差用蓝色线条表示.

PCA 可以定义为最大化投影数据方差的线性投影 (Hotelling, 1933). 同样,PCA 也可以定义为最小化平均投影代价的线性投影, 平均投影代价则定义为数据点与其投影之间的均方距离 (Pearson, 1901). 我们将依次学习这些定义.

考虑一个观测数据集 $\{\boldsymbol{x}_n\}$, 其中 $n = 1, \cdots, N$, 并且 $\boldsymbol{x}_n$ 是一个维度为 D 的欧几里得变量. 我们想要把数据投影到维度为 $M < D$ 的空间, 同时最大化投影数据的方差. 在开始之前, 我们假设 M 的值是给定的.

16.1.1 最大方差表述

首先考虑投影到一维空间 ($M = 1$). 我们可以使用一个 D 维向量 $\boldsymbol{u}$ 来定义这个空间的方向. 为了方便 (且不失一般性), 我们会选择单位向量, 因此 $\boldsymbol{u}^{\mathsf{T}}\boldsymbol{u} = 1$. 然后将每个数据点 $\boldsymbol{x}_n$ 投影到一个标量值 $\boldsymbol{u}^{\mathsf{T}}\boldsymbol{x}_n$ 上.

数据的平均值是 $\bar{\boldsymbol{x}}$, 其中 $\bar{\boldsymbol{x}}$ 是样本集的平均值, 由下式给出:

$$\bar{\boldsymbol{x}} = \frac{1}{N}\sum_{n=1}^{N}\boldsymbol{x}_n. \tag{16.1}$$

投影数据的方差由下式给出:

$$\frac{1}{N}\sum_{n=1}^{N}\left\{\boldsymbol{u}^{\mathsf{T}}(\boldsymbol{x}_n - \bar{\boldsymbol{x}})\right\}^2 = \boldsymbol{u}^{\mathsf{T}}\boldsymbol{S}\boldsymbol{u} \tag{16.2}$$

其中 $\boldsymbol{S}$ 是数据协方差矩阵, 定义为

$$\boldsymbol{S} = \frac{1}{N}\sum_{n=1}^{N}(\boldsymbol{x}_n - \bar{\boldsymbol{x}})(\boldsymbol{x}_n - \bar{\boldsymbol{x}})^{\mathsf{T}}. \tag{16.3}$$

下面我们将投影数据的方差 $\boldsymbol{u}^{\mathsf{T}}\boldsymbol{S}\boldsymbol{u}$ 相对于 $\boldsymbol{u}$ 最大化. 显然, 这必须是一个受约束的最大化操作, 以防止 $\|\boldsymbol{u}\| \to \infty$. 适当的约束来源于归一化条件 $\boldsymbol{u}^{\mathsf{T}}\boldsymbol{u} = 1$. 为了强制执行这个约束, 我们引入一个拉格朗日乘子 λ, 并对下式进行无约束的最大化:

$$\boldsymbol{u}^{\mathsf{T}}\boldsymbol{S}\boldsymbol{u} + \lambda(1 - \boldsymbol{u}^{\mathsf{T}}\boldsymbol{u}). \tag{16.4}$$

通过将式(16.4)对 $\boldsymbol{u}$ 的导数设为零, 我们发现当以下条件满足时, 目标函数将取驻点:

$$\boldsymbol{S}\boldsymbol{u} = \lambda\boldsymbol{u}. \tag{16.5}$$

这意味着 $\boldsymbol{u}$ 必须是 $\boldsymbol{S}$ 的一个特征向量. 如果将式(16.5)左乘 $\boldsymbol{u}^{\mathsf{T}}$ 并利用归一化条件 $\boldsymbol{u}^{\mathsf{T}}\boldsymbol{u} = 1$, 可得方差为

$$\boldsymbol{u}^{\mathsf{T}}\boldsymbol{S}\boldsymbol{u} = \lambda. \tag{16.6}$$

因此, 当我们将 $\boldsymbol{u}$ 设置为具有最大特征值 λ 的特征向量时, 方差将达到最大. 这个特征向量称为第一主成分.

我们可以通过逐步的方式定义更多的主成分, 方法就是选取一个新方向, 使其在所有与已考虑方向正交的方向中最大化投影后的方差. 考虑 M 维投影空间的一般情况, 投影数据方差最大化的最优线性投影由数据协方差矩阵 $\boldsymbol{S}$ 对应于最大特征值 $\lambda_1, \cdots, \lambda_M$ 的 M 个特征向量 $\boldsymbol{u}_1, \cdots, \boldsymbol{u}_M$ 定义 (见习题 16.1).

综上, PCA 涉及计算数据集的均值 $\bar{\boldsymbol{x}}$ 和数据协方差矩阵 $\boldsymbol{S}$, 然后找到 $\boldsymbol{S}$ 对应于最大特征值的 M 个特征向量. 计算一个 $D \times D$ 矩阵的完整特征向量分解的计算成本是 $O(D^3)$. 如果打算将数据投影到前 M 个主成分上, 则只需要找到前 M 个特征值和特征向量. 这可以通过使用更有效的方法来完成, 比如幂方法 (Golub and Van Loan, 1996), 复杂度为 $O(MD^2)$, 当然也可以使用 EM 算法.

16.1.2 最小误差表述

本小节讨论基于投影误差最小化的 PCA 的另一种表述. 为此, 引入一组 D 维的完备正交规范的基向量 $\{\boldsymbol{u}_i\}$, 其中 $i = 1, \cdots, D$, 它们满足

$$\boldsymbol{u}_i^{\mathsf{T}}\boldsymbol{u}_j = \delta_{ij}. \tag{16.7}$$

由于这组基向量是完备的, 每个数据点都可以通过基向量的线性组合精确表示为

$$\boldsymbol{x}_n = \sum_{i=1}^{D} a_{ni}\boldsymbol{u}_i \tag{16.8}$$

其中系数 a_{ni} 对于不同的数据点是不同的. 这只是相当于将坐标系旋转到 $\{\boldsymbol{u}_i\}$ 定义的新系统, 原始的 D 个分量 $\{\boldsymbol{x}_{n1}, \cdots, \boldsymbol{x}_{nD}\}$ 被等价的一组分量 $\{a_{n1}, \cdots, a_{nD}\}$ 替代. 与 $\boldsymbol{u}_i$ 内积并利用正交性质, 可以得到 $a_{ni} = \boldsymbol{x}_n^{\mathsf{T}}\boldsymbol{u}_i$.

不过, 我们的目标是用一个较小的变量数 M(小于原始维度 D) 来近似这个数据点, 相当于将它投影到一个低维的子空间. 这个 M 维的线性子空间可以用前 M 个基向量表示, 这样我们就可以用这些基向量来近似每一个数

据点 $\boldsymbol{x}_n$:

$$\hat{\boldsymbol{x}}_n = \bar{\boldsymbol{x}} + \sum_{m=1}^{M} z_{nm}\boldsymbol{u}_m + \sum_{j=M+1}^{D} b_j \boldsymbol{u}_j \tag{16.9}$$

其中 $\{z_{nm}\}$ 依赖于特定的数据点, 而 $\{b_j\}$ 是对所有数据点都相同的常数. 我们可以自由选择 $\{\boldsymbol{u}_j\}$、$\{z_{nm}\}$ 和 $\{b_j\}$ 以最小化由维度降低引入的误差.

作为误差度量, 我们将使用原始数据点 $\boldsymbol{x}_n$ 与其近似值 $\hat{\boldsymbol{x}}_n$ 之间的平方距离, 并在数据集上取平均, 因此我们的目标是最小化

$$J = \frac{1}{N}\sum_{n=1}^{N} \|\boldsymbol{x}_n - \hat{\boldsymbol{x}}_n\|^2. \tag{16.10}$$

首先考虑关于量 $\{z_{nm}\}$ 的最小化. 将 $\hat{\boldsymbol{x}}_n$ 代入 J, 将 J 关于 z_{nm} 的导数设为零, 利用正交关系, 我们得到

$$z_{nm} = (\boldsymbol{x}_n - \bar{\boldsymbol{x}})^{\mathsf{T}}\boldsymbol{u}_m. \tag{16.11}$$

类似地, 将 J 关于 b_j 的导数设为零, 并再次利用正交关系, 得到

$$b_j = \bar{\boldsymbol{x}}^{\mathsf{T}}\boldsymbol{u}_j. \tag{16.12}$$

代入 z_{nm} 和 b_j, 并利用一般展开式, 得到

$$\boldsymbol{x}_n - \hat{\boldsymbol{x}}_n = \sum_{j=M+1}^{D} \{(\boldsymbol{x}_n - \bar{\boldsymbol{x}})^{\mathsf{T}}\boldsymbol{u}_j\}\boldsymbol{u}_j. \tag{16.13}$$

我们看到, 从 $\boldsymbol{x}_n$ 到 $\hat{\boldsymbol{x}}_n$ 的位移向量位于主子空间的正交空间中, 因为它是 $i = M+1, \cdots, D$ 的 $\boldsymbol{u}_i$ 的线性组合. 这在预期之内, 因为投影点 $\hat{\boldsymbol{x}}_n$ 必须位于主子空间内, 但我们可以在这个子空间内自由移动它们, 因此最小误差由正交投影给出.

剩下的任务是调整 $\{\boldsymbol{u}_j\}$ 从而最小化 J, 这也必须是一个受约束的最小化操作. 误差度量 J 作为纯粹依赖于 $\{\boldsymbol{u}_j\}$ 的函数的表达式如下:

$$J = \frac{1}{N}\sum_{n=1}^{N} \| \sum_{j=M+1}^{D} \{(\boldsymbol{x}_n - \bar{\boldsymbol{x}})^{\mathsf{T}}\boldsymbol{u}_j\}\boldsymbol{u}_j\|^2 = \sum_{j=M+1}^{D} \boldsymbol{u}_j^{\mathsf{T}}\boldsymbol{S}\boldsymbol{u}_j. \tag{16.14}$$

针对任意 D 和任意 $M < D$ 最小化 J 的一般解, 均可以选择 $\{\boldsymbol{u}_j\}$ 为协方差矩阵的特征向量, 由下式给出:

$$\boldsymbol{S}\boldsymbol{u}_i = \lambda_i \boldsymbol{u}_i \tag{16.15}$$

其中 $i = 1, \cdots, D$, 并且像往常一样, 特征向量选择为正交向量. 然后, 误差度量的相应值由下式给出:

$$J = \sum_{i=M+1}^{D} \lambda_i. \tag{16.16}$$

这仅仅是正交于主子空间的那些特征向量的特征值之和. 因此, 我们通过选择这些特征向量为具有 $D-M$ 个最小特征值的那些特征向量, 得到了 J 的最小值, 定义主子空间的特征向量是对应于 M 个最大特征值的那些特征向量. 这个结果符合我们的直觉: 为了最小化平均平方投影距离, 应该选择主成分子空间, 使其通过数据点的平均值并与最大方差的方向对齐.

16.1.3 数据压缩

如果我们将式(16.11)和式(16.12)代入式(16.9), 就可以将 PCA 近似数据向量 $\boldsymbol{x}_n$ 的形式写为

$$\hat{\boldsymbol{x}}_n = \bar{\boldsymbol{x}} + \sum_{m=1}^{M} \{(\boldsymbol{x}_n - \bar{\boldsymbol{x}})^{\mathsf{T}}\boldsymbol{u}_m\}\boldsymbol{u}_m. \tag{16.17}$$

这代表了数据集的压缩, 因为对于每个数据点, 我们都会将 D 维向量 $\boldsymbol{x}_n$ 替换为具有分量 $(\boldsymbol{x}_n - \bar{\boldsymbol{x}})^{\mathsf{T}}\boldsymbol{u}_m$ 的 M 维向量.M 值越小, 压缩程度越大.

PCA 的另一个应用是数据压缩, 我们用手写数字图像数据集来举例. 因为协方差矩阵的每个特征向量都是原始 D 维空间中的向量, 所以我们可以将特征向量表示成与数据点大小相同的图像.

16.1.4 数据白化

PCA 的另一个应用是数据预处理, 其目标不是降低维度, 而是转换数据集以标准化某些属性. 变化后的数据集能更好地为机器学习算法服务. 通常, 当原始变量以不同的单位测量或具有显著不同的可变性时, 就需要进行此类处理.

将 K 均值算法应用于这个数据集时, 我们首先对各个变量进行了单独的线性重缩放, 使得每个变量都具有零均值和单位方差. 这称为标准化数据.

利用 PCA, 可以对数据进行更实质性的标准化, 使其具有零均值和单位协方差, 从而使不同的变量变得不相关. 为此, 首先将特征向量方程写成以下形式:

$$\boldsymbol{SU} = \boldsymbol{U}\boldsymbol{\Lambda} \tag{16.18}$$

其中 $\boldsymbol{\Lambda}$ 是一个 $D \times D$ 的对角矩阵, 矩阵元素为 λ_i;$\boldsymbol{U}$ 是一个 $D \times D$ 的正交矩阵, 其列由 $\boldsymbol{u}_i$ 给出. 然后我们定义, 对于每个数据点 $\boldsymbol{x}_n$, 转换后的值由下式给出:

$$\boldsymbol{y}_n = \boldsymbol{\Lambda}^{-1/2}\boldsymbol{U}^{\mathsf{T}}(\boldsymbol{x}_n - \bar{\boldsymbol{x}}). \tag{16.19}$$

显然, 集合 $\{\boldsymbol{y}_n\}$ 具有零均值, 并且其协方差由单位矩阵给出:

$$\frac{1}{N}\sum_{n=1}^{N}\boldsymbol{y}_n\boldsymbol{y}_n^{\mathsf{T}} = \boldsymbol{\Lambda}^{-1/2}\boldsymbol{U}^{\mathsf{T}}\left[\frac{1}{N}\sum_{n}(\boldsymbol{x}_n - \bar{\boldsymbol{x}})(\boldsymbol{x}_n - \bar{\boldsymbol{x}})^{\mathsf{T}}\right]\boldsymbol{U}\boldsymbol{\Lambda}^{-1/2} = \boldsymbol{I}. \tag{16.20}$$

这个操作称为白化 (whitening) 或球化 (sphering) 数据.

16.1.5 高维数据

在 PCA 的某些应用中, 数据点的数量小于数据空间的维度. 例如, 我们可能想对一个有几百张图像的数据集应用 PCA, 其中的每个图像对应于潜在的几百万维空间中的一个向量 (对应于图像中每个像素的三个颜色值). 注意, 在 D 维空间中,N 个点的集合 ($N < D$) 定义了一个线性子空间, 其维度最多为 $N-1$, 因此当 M 的值大于 $N-1$ 时, 应用 PCA 没有多大意义. 实际上, 如果我们执行 PCA, 就会发现至少有 $D - N + 1$ 个特征值是零, 在这些特征值对应的特征向量方向上, 数据的方差为零.

可以按照以下方式来解决这个问题. 首先将 $\boldsymbol{X}$ 定义为 $N \times D$ 维的中心化数据矩阵, 其第 n 行由 $(\boldsymbol{x}_n - \bar{\boldsymbol{x}})^{\mathsf{T}}$ 给出. 然后可以将协方差矩阵写作 $\boldsymbol{S} = N^{-1}\boldsymbol{X}^{\mathsf{T}}\boldsymbol{X}$, 相应的特征向量方程变为

$$\frac{1}{N}\boldsymbol{X}^{\mathsf{T}}\boldsymbol{X}\boldsymbol{u} = \lambda\boldsymbol{u}. \tag{16.21}$$

如果定义 $\boldsymbol{v} = \boldsymbol{X}\boldsymbol{u}$, 则可以得到

$$\frac{1}{N}\boldsymbol{X}\boldsymbol{X}^{\mathsf{T}}\boldsymbol{v} = \lambda\boldsymbol{v}. \tag{16.22}$$

这是 $N \times N$ 矩阵 $N^{-1}\boldsymbol{X}\boldsymbol{X}^{\mathsf{T}}$ 的特征向量方程, 它与原始协方差矩阵具有相同的 $N-1$ 个特征值 (它本身还有额外的 $D - N + 1$ 个特征值为零). 因此, 我们可以在维度较低的空间中解决特征向量问题, 计算成本为 $O(N^3)$ 而不是 $O(D^3)$. 要确定特征向量, 我们可以对式(16.22)的两边乘以 $\boldsymbol{X}^{\mathsf{T}}$, 得到

$$\left(\frac{1}{N}\boldsymbol{X}^{\mathsf{T}}\boldsymbol{X}\right)(\boldsymbol{X}^{\mathsf{T}}\boldsymbol{v}) = \lambda(\boldsymbol{X}^{\mathsf{T}}\boldsymbol{v}) \tag{16.23}$$

从中可以看出 $(\boldsymbol{X}^{\mathsf{T}}\boldsymbol{v})$ 是具有特征值 λ 的 $\boldsymbol{S}$ 的特征向量. 总之, 要应用这种方法, 我们需要首先计算 $\boldsymbol{X}\boldsymbol{X}^{\mathsf{T}}$, 然后找到它的特征向量和特征值, 最后使用式(16.23)在原始数据空间中计算特征向量.

16.2 概率潜变量

我们在节 16.1中已经看到 PCA 可以定义为数据在低维子空间上的线性投影, 这个低维子空间的维度比原始数据空间的维度小. 每个数据点被映射到由式(16.11)定义的唯一的量 z_{nm}, 我们可以将这些量视为确定性的潜变

量. 为了引入和论证连续概率潜变量, 本节会展示, PCA 也可以表示为概率潜变量模型的最大似然解. 这种 PCA 的重新表述, 称为概率 PCA, 与传统 PCA 相比, 有如下优势:

- 概率 PCA 模型表示了高斯分布的一种约束形式, 在仍然能够捕捉数据集中主要相关性的同时, 还可以限制自由参数的数量.
- 我们可以为 PCA 推导出一种 EM 算法, 该算法在只需要少数几个主特征向量的情况下计算效率很高, 而且能避免将计算数据协方差矩阵作为中间步骤进行计算.
- 概率 PCA 模型与 EM 算法的结合使我们能够处理数据集中缺失的值.
- 可以原则性地构造概率 PCA 模型的混合模型并使用 EM 算法进行训练.
- 概率 PCA 模型可以生成性地运行以提供来自分布的样本.
- 概率 PCA 可用于建模类条件密度, 因此可应用于分类问题.
- 概率 PCA 构成了基于贝叶斯方法处理 PCA 的基础, 我们可以从数据中自动找到主成分子空间的维度.
- 似然函数的存在使得我们可以与其他概率密度模型直接进行比较. 相比之下, 传统 PCA 模型会给予靠近主成分子空间且距离训练数据任意远的数据点一个较低的重建成本.

将 PCA 作为概率模型的这种表述是由 Tipping and Bishop (1997; 1999) 以及 Roweis (1998) 独立提出的. 正如我们稍后将看到的, 它与因子分析 (factor analysis)(Basilevsky, 1994) 密切相关.

16.2.1 生成式模型

概率 PCA 是线性–高斯框架的一个简单例子, 其中所有的边缘分布和条件分布都是高斯分布. 我们可以首先通过引入对应于主成分子空间的、明确的 M 维潜变量 $\boldsymbol{z}$ 来构建概率 PCA. 接下来, 我们定义一个关于潜变量的高斯先验分布 $p(\boldsymbol{z})$, 以及基于潜变量值的 D 维观测变量 $\boldsymbol{x}$ 的高斯条件分布 $p(\boldsymbol{x}|\boldsymbol{z})$. 具体来说, 关于 $\boldsymbol{z}$ 的先验分布由零均值、单位协方差的高斯分布给出:

$$p(\boldsymbol{z}) = \mathcal{N}(\boldsymbol{z}|\boldsymbol{0}, \boldsymbol{I}). \tag{16.24}$$

请注意, 潜分布 $p(\boldsymbol{z})$ 是零均值、单位协方差的高斯分布, 这一假设不失一般性, 因为更普遍的高斯分布也会产生一个等价的概率模型 (见习题 16.4).

同样, 观测变量 $\boldsymbol{x}$ 的条件分布, 基于潜变量 $\boldsymbol{z}$ 的值, 也符合高斯分布:

$$p(\boldsymbol{x}|\boldsymbol{z}) = \mathcal{N}(\boldsymbol{x}|\boldsymbol{W}\boldsymbol{z} + \boldsymbol{\mu}, \sigma^2\boldsymbol{I}) \tag{16.25}$$

其中 $\boldsymbol{x}$ 的均值是 $\boldsymbol{z}$ 的一般线性函数, 受 $D \times M$ 维矩阵 $\boldsymbol{W}$ 和 D 维向量 $\boldsymbol{\mu}$ 的控制. 注意, 关于 $\boldsymbol{x}$ 的元素是因子化的. 换句话说, 这是一个朴素贝叶斯模型. 正如我们稍后将看到的,$\boldsymbol{W}$ 的列跨越了数据空间内的线性子空间, 对应于主子空间. 这个模型中的另一个参数是控制条件分布方差的标量 σ^2.

我们可以从生成性视角来看概率 PCA 模型. 首先选择一个潜变量, 然后基于这个潜变量的值采样观测变量, 得到观测变量的样本值. 具体来说,D 维观测变量 $\boldsymbol{x}$ 是通过为 M 维潜变量 $\boldsymbol{z}$ 加上加性高斯噪声的线性变换来定义的, 所以有

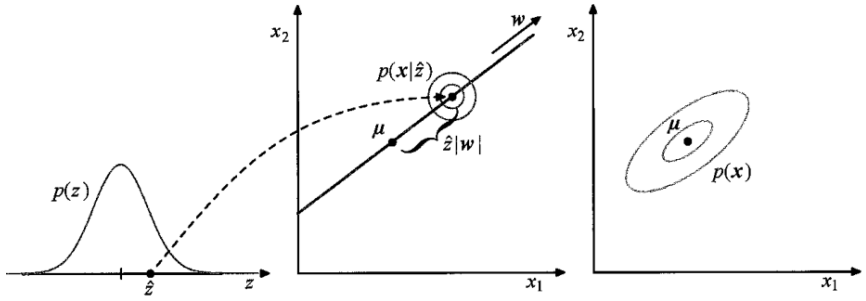
$$\boldsymbol{x} = \boldsymbol{W}\boldsymbol{z} + \boldsymbol{\mu} + \boldsymbol{\epsilon} \tag{16.26}$$

其中 $\boldsymbol{z}$ 是一个 M 维的高斯潜变量,$\boldsymbol{\epsilon}$ 是一个 D 维的零均值高斯分布噪声变量, 协方差为 $\sigma^2\boldsymbol{I}$. 这个生成过程如图 16.2所示. 注意, 这个框架基于从潜空间到数据空间的映射, 而不是前面讨论的常规 PCA 视角. 稍后, 我们将利用贝叶斯定理得出从数据空间到潜空间的反向映射.

16.2.2 似然函数

假设我们希望通过最大似然法确定参数 $\boldsymbol{W}$、$\boldsymbol{\mu}$ 和 σ^2 的值. 为了写下似然函数, 我们需要观测变量的边缘分布 $p(\boldsymbol{x})$ 的表达式. 根据概率的加和法则与乘积法则, 这可以表示为

$$p(\boldsymbol{x}) = \int p(\boldsymbol{x}|\boldsymbol{z})p(\boldsymbol{z})\,\mathrm{d}\boldsymbol{z}. \tag{16.27}$$



常规 PCA 视角看一个针对两维数据空间和一维潜空间的概率 PCA 模型的生成过程。生成观测数据点
首先从其先验分布 $p(z)$ 中抽取潜变量的一个值 $\hat{z}$，然后从具有均值 $w\hat{z}+\mu$ 和协方差 $\sigma^2 I$ 的各向同
（见红色圆圈）中抽取 x 的值。绿色椭圆显示了边缘分布 $p(x)$ 的密度等高线

似然函数

图 16.2: 从常规 PCA 视角看一个针对两维数据空间和一维潜空间的概率 PCA 模型的生成过程. 生成观测数据点 $\boldsymbol{x}$ 的过程是: 首先从其先验分布 $p(\boldsymbol{z})$ 中抽取潜变量的一个值 $\hat{\boldsymbol{z}}$, 然后从具有均值 $\boldsymbol{W}\hat{\boldsymbol{z}}+\boldsymbol{\mu}$ 和协方差 $\sigma^2\boldsymbol{I}$ 的各向同性高斯分布 (见红色圆圈) 中抽取 $\boldsymbol{x}$ 的值. 绿色椭圆显示了边缘分布 $p(\boldsymbol{x})$ 的密度等高线.

由于对应于线性–高斯模型, 这个边缘分布同样是高斯分布, 由下式给出:

$$p(\boldsymbol{x}) = \mathcal{N}(\boldsymbol{x}|\boldsymbol{\mu}, \boldsymbol{C}) \tag{16.28}$$

其中 $D \times D$ 的协方差矩阵 $\boldsymbol{C}$ 定义如下:

$$\boldsymbol{C} = \boldsymbol{W}\boldsymbol{W}^{\mathrm{T}} + \sigma^2\boldsymbol{I}. \tag{16.29}$$

注意预测分布也是高斯分布, 因此通过使用式(16.26)计算其均值和协方差就能更直接地得出同样的结果:

$$\mathbb{E}[\boldsymbol{x}] = \mathbb{E}[\boldsymbol{W}\boldsymbol{z} + \boldsymbol{\mu} + \boldsymbol{\epsilon}] = \boldsymbol{\mu} \tag{16.30}$$

$$\mathrm{cov}[\boldsymbol{x}] = \mathbb{E}[(\boldsymbol{W}\boldsymbol{z} + \boldsymbol{\epsilon})(\boldsymbol{W}\boldsymbol{z} + \boldsymbol{\epsilon})^{\mathrm{T}}] = \mathbb{E}[\boldsymbol{W}\boldsymbol{z}\boldsymbol{z}^{\mathrm{T}}\boldsymbol{W}^{\mathrm{T}}] + \mathbb{E}[\boldsymbol{\epsilon}\boldsymbol{\epsilon}^{\mathrm{T}}] = \boldsymbol{W}\boldsymbol{W}^{\mathrm{T}} + \sigma^2\boldsymbol{I} \tag{16.31}$$

这里我们利用了如下事实:$\boldsymbol{z}$ 和 $\boldsymbol{\epsilon}$ 是独立随机变量, 因此它们不相关.

直观上, 我们可以认为分布 $p(\boldsymbol{x})$ 是通过取一个各向同性的高斯" 喷雾罐", 然后在主成分子空间上移动并喷洒以 σ^2 决定密度的高斯墨水, 并通过先验分布来加权的. 累积的墨水密度将产生一个代表边缘密度 $p(\boldsymbol{x})$ 的" 松饼" 状分布.

16.2.3 最大似然法

接下来考虑使用最大似然法确定模型参数. 给定一个数据集, 概率 PCA 模型可以表示为一个有向图. 相应的对数似然函数为

$$\ln p(\boldsymbol{X}|\boldsymbol{W}, \boldsymbol{\mu}, \sigma^2) = \sum_{n=1}^{N} \ln \mathcal{N}(\boldsymbol{x}_n|\boldsymbol{\mu}, \boldsymbol{C}) \tag{16.32}$$

其中 $\boldsymbol{C}$ 由式(16.29)给出.

将对数似然函数关于 $\boldsymbol{\mu}$ 的导数设为零便可得到预期结果 $\boldsymbol{\mu} = \bar{\boldsymbol{x}}$, 其中 $\bar{\boldsymbol{x}}$ 是由式(16.1)定义的数据均值. 代入原式后, 我们可以将对数似然函数写成以下形式:

$$\ln p(\boldsymbol{X}|\boldsymbol{W}, \boldsymbol{\mu}, \sigma^2) = -\frac{N}{2}\left\{D\ln(2\pi) + \ln|\boldsymbol{C}| + \mathrm{tr}(\boldsymbol{C}^{-1}\boldsymbol{S})\right\} \tag{16.33}$$

其中 $\boldsymbol{S}$ 是由式(16.3)定义的数据协方差矩阵.

虽然似然函数关于 $\boldsymbol{W}$ 和 σ^2 的最大化更为复杂, 但仍有精确的闭合解.Tipping and Bishop (1999) 指出, 对数似

然函数的所有驻点都可以写成

$$\boldsymbol{W} = \boldsymbol{U}_M(\boldsymbol{\Lambda} - \sigma^2\boldsymbol{I})^{1/2}\boldsymbol{R} \tag{16.34}$$

其中 $\boldsymbol{U}_M$ 是一个 $D \times M$ 矩阵, 其列由数据协方差矩阵 $\boldsymbol{S}$ 的任意子集 (大小为 M) 的特征向量给出.$M \times M$ 对角矩阵 $\boldsymbol{\Lambda}$ 的元素由对应的特征值 λ_i 给出.$\boldsymbol{R}$ 是一个任意的 $M \times M$ 正交矩阵.

此外,Tipping and Bishop (1999) 还指出, 当选择那些对应于前 M 个最大的特征值的特征向量时, 就会得到似然函数的最大值 (所有其他解都是鞍点). 同样, 假设特征向量已按照对应特征值的值递减的顺序排列, 则 M 个主特征向量是 $\boldsymbol{u}_1, \cdots, \boldsymbol{u}_M$. 在这种情况下,$\boldsymbol{W}$ 的列定义了标准 PCA 的主子空间. 对应的 σ^2 的最大似然解为

$$\sigma^2 = \frac{1}{D-M}\sum_{i=M+1}^{D}\lambda_i \tag{16.35}$$

这是与被丢弃维度相关联的平均方差.

我们有必要花些时间研究式(16.29)给出的数据协方差矩阵的形式. 考虑预测分布沿某个由单位向量 $\boldsymbol{v}$ 指定的方向的方差, 其中 $\boldsymbol{v}^{\mathsf{T}}\boldsymbol{v} = 1$, 由 $\boldsymbol{v}^{\mathsf{T}}\boldsymbol{C}\boldsymbol{v}$ 给出. 首先假设 $\boldsymbol{v}$ 与主子空间正交, 换句话说,$\boldsymbol{v}$ 是由被丢弃特征向量的某种线性组合给出的. 由于 $\boldsymbol{U}_M^{\mathsf{T}}\boldsymbol{v} = 0$, 因此 $\boldsymbol{v}^{\mathsf{T}}\boldsymbol{C}\boldsymbol{v} = \sigma^2$. 该模型预测了正交于主子空间的噪声方差. 假设 $\boldsymbol{v} = \boldsymbol{u}_i$, 其中 $\boldsymbol{u}_i$ 是定义主子空间的保留特征向量之一, 则有 $\boldsymbol{v}^{\mathsf{T}}\boldsymbol{C}\boldsymbol{v} = (\lambda_i - \sigma^2) + \sigma^2 = \lambda_i$.

换句话说, 这个模型正确地捕捉了数据沿主轴的方差, 并用单个平均值 σ^2 近似所有其余方向的方差. 一旦认识到独立高斯分布 (在这种情况下是潜空间分布和噪声模型) 的卷积的方差是可加的, 这些缩放因子的解释也就清楚了. 因此, 沿特征向量 $\boldsymbol{u}_i$ 方向的方差 λ_i 由以下两部分之和构成: 一是单位方差潜空间分布通过 $\boldsymbol{W}$ 的对应列投影到数据空间的贡献 $\lambda_i - \sigma^2$, 二是噪声模型在所有方向上添加的各向同性方差贡献 σ^2.

在 M=D, 也就是没有降维的情况下, 利用正交性质,$\boldsymbol{x}$ 的边缘分布的数据协方差矩阵 $\boldsymbol{C}$ 将变为

$$\boldsymbol{C} = \boldsymbol{U}(\boldsymbol{\Lambda} - \sigma^2\boldsymbol{I})^{1/2}\boldsymbol{R}\boldsymbol{R}^{\mathsf{T}}(\boldsymbol{\Lambda} - \sigma^2\boldsymbol{I})^{1/2}\boldsymbol{U}^{\mathsf{T}} + \sigma^2\boldsymbol{I} = \boldsymbol{U}\boldsymbol{\Lambda}\boldsymbol{U}^{\mathsf{T}} = \boldsymbol{S} \tag{16.36}$$

于是我们得到无约束高斯分布的标准最大似然解, 其中数据协方差矩阵由样本协方差矩阵给出.

传统 PCA 通常构造为从 D 维数据空间到 M 维线性子空间的点的投影. 然而, 概率 PCA 可以最自然地表示为从潜空间到数据空间的映射. 对于可视化和数据压缩等应用, 我们可以使用贝叶斯定理反转这个映射. 随后, 数据空间中的任何点就可以由其在潜空间中的后验均值和协方差来概括. 从式(16.25)可知, 均值由下式给出:

$$\mathbb{E}[\boldsymbol{z}|\boldsymbol{x}] = \boldsymbol{M}^{-1}\boldsymbol{W}^{\mathsf{T}}(\boldsymbol{x} - \bar{\boldsymbol{x}}) \tag{16.37}$$

其中 $\boldsymbol{M}$ 由下式给出:

$$\boldsymbol{M} = \boldsymbol{W}^{\mathsf{T}}\boldsymbol{W} + \sigma^2\boldsymbol{I}. \tag{16.38}$$

这会将点投影到由下式给出的数据空间中:

$$\boldsymbol{W}\,\mathbb{E}[\boldsymbol{z}|\boldsymbol{x}] + \bar{\boldsymbol{x}}. \tag{16.39}$$

取极限 $\sigma^2 \to 0$, 后验均值可以简化为

$$(\boldsymbol{W}^{\mathsf{T}}\boldsymbol{W})^{-1}\boldsymbol{W}^{\mathsf{T}}\boldsymbol{W}(\boldsymbol{x} - \bar{\boldsymbol{x}}) = \boldsymbol{W}(\boldsymbol{W}^{\mathsf{T}}\boldsymbol{W})^{-1}\boldsymbol{W}^{\mathsf{T}}(\boldsymbol{x} - \bar{\boldsymbol{x}}) \tag{16.40}$$

这代表数据点在潜空间中的正交投影, 因此我们恢复了标准 PCA 模型.

观测变量的条件分布 $p(\boldsymbol{x}|\boldsymbol{z})$ 的协方差独立于 $\boldsymbol{x}$. 同样, 从式(16.37)可以看出, 后验协方差由 $\sigma^2\boldsymbol{M}^{-1}$ 给出, 且与 $\boldsymbol{x}$ 无关.

潜空间中的旋转不变性表示了一种统计上的不可识别性, 类似于我们在离散潜变量混合模型中遇到的情况. 这里, 参数的连续性使得任何值都会导致相同的预测密度, 这与离散混合模型中组件重标记相关的离散不可识别性形成了对比.

16.2.4 因子分析

因子分析模型是一个线性–高斯潜变量模型, 它与概率 PCA 模型密切相关. 因子分析的独特性体现为, 其使得给定潜变量 $\boldsymbol{z}$ 的观测变量 $\boldsymbol{x}$ 的条件分布具有对角协方差, 而不是各向同性协方差, 即

$$p(\boldsymbol{x}|\boldsymbol{z}) = \mathcal{N}(\boldsymbol{x}|\boldsymbol{W}\boldsymbol{z} + \boldsymbol{\mu}, \boldsymbol{\Psi}) \tag{16.41}$$

其中 $\boldsymbol{\Psi}$ 是一个 $D \times D$ 的对角矩阵. 注意, 因子分析模型和概率 PCA 模型一样, 也假设观测变量 $x_1, \cdots, x_D$ 在给定潜变量 $\boldsymbol{z}$ 时是相互独立的. 因子分析模型的核心在于, 它通过用矩阵 $\boldsymbol{\Psi}$ 表示与每个坐标相关的独立方差, 并在矩阵 $\boldsymbol{W}$ 中捕获变量之间的协方差, 来解释观测到的数据协方差结构. 在因子分析文献中, 捕获观测变量之间相关性的 $\boldsymbol{W}$ 的列称为因子载荷 (factor loading), 对角矩阵 $\boldsymbol{\Psi}$ 里代表每个变量独立噪声方差的对角元素则称为唯一性 (uniquenesses) 元素.

利用式(16.27), 我们可以看到观测变量的边缘分布是由 $p(\boldsymbol{x}) = \mathcal{N}(\boldsymbol{x}|\boldsymbol{\mu}, \boldsymbol{C})$ 给出的, 其中

$$\boldsymbol{C} = \boldsymbol{W}\boldsymbol{W}^{\mathrm{T}} + \boldsymbol{\Psi}. \tag{16.42}$$

与概率 PCA 模型一样, 该模型对潜空间中的旋转是不变的. 历史上, 当人们尝试用因子分析方法对各个因子 ($\boldsymbol{z}$ 空间中的坐标) 进行解释时, 曾引发争议. 因子分析与这个空间中的旋转相关联, 导致不可识别性. 然而, 从我们的角度看, 我们将因子分析视为一种潜变量密度模型, 关注的重点是其中潜空间的结构形式, 而不是用于描述该空间的具体坐标.

概率 PCA 和因子分析的另一个区别是它们在数据集变换下的行为不同. 对于 PCA 和概率 PCA, 如果旋转数据空间中的坐标系, 则可以得到与数据完全相同的拟合结果, 只不过 $\boldsymbol{W}$ 矩阵会由相应的旋转矩阵变换而成的. 然而, 对于因子分析, 相似的性质是: 如果我们对数据向量逐个分量地进行重新缩放, 则这个操作会被吸收进相应的 $\boldsymbol{\Psi}$ 元素的重新缩放中.

16.2.5 独立成分分析

线性高斯潜变量模型的一种扩展是这样的模型: 观测变量与潜变量线性相关, 但潜变量分布是非高斯分布. 这类模型的一个重要分支是独立成分分析 (Independent Component Analysis, ICA) 模型, 其潜变量的联合分布能够分解为各分量的独立分布之积, 即

$$p(\boldsymbol{z}) = \prod_j p(z_j). \tag{16.43}$$

这种方法的成功依赖于潜变量具有非高斯分布. 为了理解这一点, 请回想一下, 在概率 PCA(以及因子分析) 中, 潜空间分布由零均值、各向同性的高斯分布给出. 因此模型无法区分仅仅通过潜空间中的旋转就可以互相转换的两个不同的潜变量. 我们可以直接验证这一点, 注意边缘密度及由此产生的似然函数在我们做 $\boldsymbol{W} \to \boldsymbol{W}\boldsymbol{R}$ 变换时会保持不变 (其中 $\boldsymbol{R}$ 是一个满足 $\boldsymbol{R}\boldsymbol{R}^{\mathrm{T}} = \boldsymbol{I}$ 的正交矩阵), 这是因为式(16.29)给出的矩阵 $\boldsymbol{C}$ 本身是不变的. 即使扩展模型以允许更通用的高斯潜变量分布, 此结论也不会改变.

对于为什么高斯潜变量分布在线性模型中不足以找到独立成分, 我们还可以用另一种方式来理解. 我们注意到, 主成分代表了数据空间中坐标系的旋转, 目的是使协方差矩阵对角化, 从而使新坐标中的数据分布不再相关. 尽管零相关是独立性的必要条件, 但它不是充分条件.

为了理解这些模型的作用, 可以设想以下场景: 有两个人在同时说话, 我们使用两个麦克风录制他们的声音. 如果忽略时间延迟和回声等效果, 则任何时刻麦克风接收到的信号都将由这两个声音的振幅的线性组合给出. 这个线性组合的系数是恒定的. 如果我们能从样本数据中推断出它们的值, 我们就可以反转混合过程 (假设它是非奇异的), 从而得到两个干净的信号, 每个信号只包含一个人的声音. 这是一个盲源分离的问题," 盲" 指的是我们只给出了混合数据, 而没有观察到原始来源或混合系数.

在实践中, 常用的潜变量分布由下式给出:

$$p(z_j) = \prod_j \pi(z_j) \tag{16.44}$$

其中 $\pi(z_j)$ 通常选择具有重尾 (heavy tail) 的分布, 与高斯分布相比, 我们在许多真实世界的分布中也能观察到这种特性.

16.2.6 卡尔曼滤波器

到目前为止, 我们假设数据值是独立同分布的. 一种常见的非独立同分布情况是, 数据点形成一个有序序列. 连续潜变量模型可以通过连接潜变量形成马尔可夫链来扩展, 以处理序列数据, 如图 16.3所示. 这种模型称为线性动态系统 (linear dynamical system) 或卡尔曼滤波器.

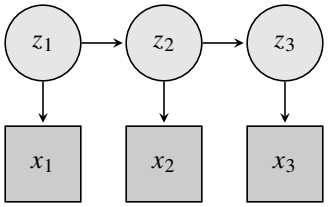


图 16.3: 用于序列数据的概率图模型称为线性动态系统或卡尔曼滤波器, 其中的潜变量形成了一个马尔可夫链.

在这样的最简单模型中, 图中分布 $p(\boldsymbol{x}_n|\boldsymbol{z}_n)$ 代表了该特定观测结果的线性–高斯潜变量模型, 就像我们之前讨论的独立同分布数据一样. 然而, 潜变量 $\boldsymbol{z}_n$ 不再视为独立的, 而是形成了一个马尔可夫链, 链中每个潜变量的分布 $p(\boldsymbol{z}_n|\boldsymbol{z}_{n-1})$ 取决于链中前一个潜变量的状态. 在线性–高斯潜变量模型中,$\boldsymbol{z}_n$ 的分布是高斯分布, 均值则由 $\boldsymbol{z}_{n-1}$ 的线性函数给出. 通常, 分布 $p(\boldsymbol{x}_n|\boldsymbol{z}_n)$ 的所有参数是共享的, 分布 $p(\boldsymbol{z}_n|\boldsymbol{z}_{n-1})$ 的所有参数也是共享的, 因此模型中总的参数数量是固定的, 不依赖于序列的长度. 这些参数可以通过最大似然法从数据中来学习. 这需要用到在图模型上传播消息的有效算法.

16.3 证据下界

在讨论离散潜变量模型时, 我们推导出了边缘对数似然的证据下界 (ELBO), 并展示了它是如何构成期望最大化 (EM) 算法的基础的. 相同的框架同样适用于连续潜变量模型. 这里我们提供一个稍微不同的 ELBO 推导, 并且假设潜变量 $\boldsymbol{z}$ 是连续的.

考虑一个带有观测变量 $\boldsymbol{x}$、潜变量 $\boldsymbol{z}$ 和可学习参数向量 $\boldsymbol{w}$ 的模型 $p(\boldsymbol{x}, \boldsymbol{z}|\boldsymbol{w})$. 只要引入潜变量上的任意一个分布 $q(\boldsymbol{z})$, 我们就可以将对数似然函数 $\ln p(\boldsymbol{x}|\boldsymbol{w})$ 写成两项之和的形式:

$$\ln p(\boldsymbol{x}|\boldsymbol{w}) = \mathcal{L}(q, \boldsymbol{w}) + \mathrm{KL}(q(\boldsymbol{z}) \,\|\, p(\boldsymbol{z}|\boldsymbol{x}, \boldsymbol{w})) \tag{16.45}$$

其中

$$\mathcal{L}(q, \boldsymbol{w}) = \int q(\boldsymbol{z}) \ln \left\{ \frac{p(\boldsymbol{x}, \boldsymbol{z}|\boldsymbol{w})}{q(\boldsymbol{z})} \right\} \mathrm{d}\boldsymbol{z} \tag{16.46}$$

而

$$\mathrm{KL}(q(\boldsymbol{z}) \,\|\, p(\boldsymbol{z}|\boldsymbol{x}, \boldsymbol{w})) = -\int q(\boldsymbol{z}) \ln \left\{ \frac{p(\boldsymbol{z}|\boldsymbol{x}, \boldsymbol{w})}{q(\boldsymbol{z})} \right\} \mathrm{d}\boldsymbol{z}. \tag{16.47}$$

$\mathrm{KL}(q\|p)$ 是一种 Kullback–Leibler 散度, 满足 $\mathrm{KL}(q\|p) \geq 0$, 从中可以得出

$$\ln p(\boldsymbol{x}|\boldsymbol{w}) \geq \mathcal{L}(q, \boldsymbol{w}) \tag{16.48}$$

式(16.46)给出的 $\mathcal{L}(q, \boldsymbol{w})$ 构成了对数似然的下界, 称为证据下界 (ELBO).

我们可以使用 EM 算法的两阶段迭代过程来最大化对数似然函数. 在 EM 算法中, 我们交替地针对 $q(\boldsymbol{z})$(E 步骤) 和 $\boldsymbol{w}$(M 步骤) 最大化 $\mathcal{L}(q, \boldsymbol{w})$. 首先初始化参数 $\boldsymbol{w}$. 然后在 E 步骤中保持 $\boldsymbol{w}$ 固定, 并针对 $q(\boldsymbol{z})$ 最大化下界. 我们注意到, 可以通过最小化式(16.47)中的 Kullback–Leibler 散度来获得下界的最高值. 换言之, 当 Kullback–Leibler 散度为零时,$q(\boldsymbol{z}) = p(\boldsymbol{z}|\boldsymbol{x}, \boldsymbol{w})$.

在 M 步骤中, 我们保持所选的 $q(\boldsymbol{z})$ 固定, 并针对 $\boldsymbol{w}$ 最大化 $\mathcal{L}(q, \boldsymbol{w})$. 将 $q(\boldsymbol{z}) = p(\boldsymbol{z}|\boldsymbol{x}, \boldsymbol{w}^{\mathrm{old}})$ 代入式(16.46), 可以

得到

$$\mathcal{L}(q, \boldsymbol{w}) = \int p(\boldsymbol{z}|\boldsymbol{x}, \boldsymbol{w}^{\text{old}}) \ln p(\boldsymbol{x}, \boldsymbol{z}|\boldsymbol{w}) \, \mathrm{d}\boldsymbol{z} - \int p(\boldsymbol{z}|\boldsymbol{x}, \boldsymbol{w}^{\text{old}}) \ln p(\boldsymbol{z}|\boldsymbol{x}, \boldsymbol{w}^{\text{old}}) \, \mathrm{d}\boldsymbol{z}. \tag{16.49}$$

右侧的第二项独立于 $\boldsymbol{w}$, 因此在 M 步骤中可以忽略. 式(16.49)右侧的第一项则是完整数据对数似然的期望, 该期望来自我们用 $\boldsymbol{w}^{\text{old}}$ 计算得到的 $\boldsymbol{z}$ 的后验分布.

如果我们有一个由独立同分布观测值组成的数据集 $\boldsymbol{x}_1, \cdots, \boldsymbol{x}_N$, 则似然函数的形式为

$$\ln p(\boldsymbol{X}|\boldsymbol{w}) = \sum_{n=1}^{N} \ln p(\boldsymbol{x}_n|\boldsymbol{w}) \tag{16.50}$$

其中数据矩阵 $\boldsymbol{X}$ 包含 $\boldsymbol{x}_1, \cdots, \boldsymbol{x}_N$, 并且参数 $\boldsymbol{w}$ 在所有数据点之间是共享的. 对于每一个数据点, 引入一个相应的潜变量 $\boldsymbol{z}_n$ 及其关联的分布 $q(\boldsymbol{z}_n)$, 并且通过遵循类似于推导式(16.45)时使用的步骤, 我们得到了以下形式的 ELBO:

$$\mathcal{L}(q, \boldsymbol{w}) = \sum_{n=1}^{N} \int q(\boldsymbol{z}_n) \ln p(\boldsymbol{x}_n, \boldsymbol{z}_n|\boldsymbol{w}) \, \mathrm{d}\boldsymbol{z}_n - \sum_{n=1}^{N} \int q(\boldsymbol{z}_n) \ln q(\boldsymbol{z}_n) \, \mathrm{d}\boldsymbol{z}_n. \tag{16.51}$$

当讨论变分自编码器 (见第 19 章) 时, 我们将遇到一个模型, 其 E 步骤的精确解无法实现, 因此我们改为使用深度神经网络对 $q(\boldsymbol{z})$ 进行建模, 然后使用 ELBO 来学习网络的参数.

16.3.1 EM 算法

我们可以按照 EM 算法的通用框架来导出概率 PCA 的 EM 算法. 为此, 我们写下完整数据对数似然函数并根据用" 旧" 参数值评估的潜变量分布的后验分布来获取期望, 然后最大化这个完整数据对数似然函数的期望以产生" 新" 的参数值. 因为我们假定数据点是独立的, 所以完整数据对数似然函数的形式为

$$\ln p(\boldsymbol{X}, \boldsymbol{Z}|\boldsymbol{W}, \boldsymbol{\mu}, \sigma^2) = -\frac{ND}{2} \ln(2\pi\sigma^2) - \frac{1}{2\sigma^2} \sum_{n=1}^{N} \|\boldsymbol{x}_n - \boldsymbol{\mu} - \boldsymbol{W}\boldsymbol{z}_n\|^2 - \frac{N}{2} \ln(2\pi) - \frac{1}{2} \sum_{n=1}^{N} \|\boldsymbol{z}_n\|^2 \tag{16.52}$$

其中矩阵 $\boldsymbol{Z}$ 的第 n 行由 $\boldsymbol{z}_n^{\mathsf{T}}$ 给出. 我们已经知道 $\boldsymbol{\mu}$ 的精确最大似然解是由样本均值 $\bar{\boldsymbol{x}}$[式(16.1)] 给出的, 在这个阶段替代 $\boldsymbol{\mu}$ 是很方便的. 利用式(16.24)和式(16.25)所示的潜变量分布和条件分布, 并且考虑到对潜变量的后验分布的期望, 我们得到

$$\mathbb{E}_{\boldsymbol{Z}}\left[\ln p(\boldsymbol{X}, \boldsymbol{Z}|\boldsymbol{W}, \sigma^2)\right] = -\frac{ND}{2} \ln(2\pi\sigma^2) - \frac{1}{2\sigma^2} \sum_{n=1}^{N} \left\{\|\boldsymbol{x}_n - \bar{\boldsymbol{x}}\|^2 - 2(\boldsymbol{x}_n - \bar{\boldsymbol{x}})^{\mathsf{T}} \boldsymbol{W} \mathbb{E}[\boldsymbol{z}_n] + \operatorname{tr}(\boldsymbol{W}^{\mathsf{T}} \boldsymbol{W} \, \mathbb{E}[\boldsymbol{z}_n \boldsymbol{z}_n^{\mathsf{T}}])\right\} - \frac{1}{2} \sum_{n=1}^{N} \mathbb{E}[\boldsymbol{z}_n^{\mathsf{T}} \boldsymbol{z}_n] + \text{常数}. \tag{16.53}$$

注意, 此情况仅通过高斯分布的充分统计量与后验分布产生关联. 因此, 在 E 步骤中, 我们使用旧的参数值来计算期望:

$$\mathbb{E}[\boldsymbol{z}_n] = \boldsymbol{M}^{-1} \boldsymbol{W}^{\mathsf{T}} (\boldsymbol{x}_n - \bar{\boldsymbol{x}}) \tag{16.54}$$

$$\mathbb{E}[\boldsymbol{z}_n \boldsymbol{z}_n^{\mathsf{T}}] = \sigma^2 \boldsymbol{M}^{-1} + \mathbb{E}[\boldsymbol{z}_n] \mathbb{E}[\boldsymbol{z}_n]^{\mathsf{T}} \tag{16.55}$$

这里的 $\boldsymbol{M}$ 由式(16.38)定义. 以上这些可以直接从后验分布和标准结果 $\mathbb{E}[\boldsymbol{z}_n \boldsymbol{z}_n^{\mathsf{T}}] = \operatorname{cov}[\boldsymbol{z}_n] + \mathbb{E}[\boldsymbol{z}_n] \mathbb{E}[\boldsymbol{z}_n]^{\mathsf{T}}$ 中得出.

在 M 步骤中, 我们固定后验统计量, 并针对 $\boldsymbol{W}$ 和 σ^2 进行最大化. 针对 σ^2 的最大化相对简单, 针对 $\boldsymbol{W}$ 的最大化则需要我们利用矩阵导数获得 M 步骤的如下方程:

$$\boldsymbol{W}^{\text{new}} = \left[\sum_{n=1}^{N} (\boldsymbol{x}_n - \bar{\boldsymbol{x}}) \mathbb{E}[\boldsymbol{z}_n]^{\mathsf{T}}\right] \left[\sum_{n=1}^{N} \mathbb{E}[\boldsymbol{z}_n \boldsymbol{z}_n^{\mathsf{T}}]\right]^{-1} \tag{16.56}$$

$$(\sigma^2)^{\text{new}} = \frac{1}{ND} \sum_{n=1}^{N} \left\{\|\boldsymbol{x}_n - \bar{\boldsymbol{x}}\|^2 - 2(\boldsymbol{x}_n - \bar{\boldsymbol{x}})^{\mathsf{T}} \boldsymbol{W}^{\text{new}} \mathbb{E}[\boldsymbol{z}_n] + \operatorname{tr}(\boldsymbol{W}^{\text{new}\mathsf{T}} \boldsymbol{W}^{\text{new}} \, \mathbb{E}[\boldsymbol{z}_n \boldsymbol{z}_n^{\mathsf{T}}])\right\}. \tag{16.57}$$

概率 PCA 的 EM 算法将首先初始化参数, 然后交替地在 E 步骤中使用式(16.54)和式(16.55)计算潜空间后验分布的充分统计量, 并在 M 步骤中使用式(16.56)和式(16.57)修正参数值.

概率 PCA 的 EM 算法的一个好处是, 其在大规模应用中的计算效率较高. 与基于样本协方差矩阵的特征向量

分解的传统 PCA 不同, EM 算法是迭代的. 传统 PCA 的计算效率要高得多: 协方差矩阵的特征分解需要 $O(D^3)$ 次计算. 通常我们只对前 M 个特征向量及对应的特征值感兴趣, 在这种情况下, 我们可以使用计算成本仅为 $O(MD^2)$ 的算法. 然而, 计算协方差矩阵需要 $O(ND^2)$ 的计算成本. 这里描述的 EM 算法也没有显式构造协方差矩阵. 计算最密集的步骤涉及数据集的求和, 其复杂度为 $O(NDM)$. 对于很大的维度 D,M 要远小于 D, 与 $O(ND^2)$ 相比, 这可以显著节省计算成本.

16.3.2 PCA 的 EM 算法

EM 算法的另一个优雅特性是, 我们可以取极限 $\sigma^2 \to 0$, 这对应于传统 PCA, 并且仍然是一个有效的类 EM 算法. 从式(16.54)中可以看到, 在 E 步骤中唯一需要计算的量是 $\mathbb{E}[\boldsymbol{z}_n]$. 此外, 因为 $\boldsymbol{M} = \boldsymbol{W}^{\mathsf{T}}\boldsymbol{W}$, 所以 M 步骤也简化了. 为了强调算法的简洁性, 定义 $\tilde{\boldsymbol{X}}$ 为一个大小为 $N \times D$ 的矩阵, 其第 n 行由向量 $(\boldsymbol{x}_n - \bar{\boldsymbol{x}})^{\mathsf{T}}$ 给出. 类似地, 定义 $\boldsymbol{\Omega}$ 为一个大小为 $M \times N$ 的矩阵, 其第 n 列由 $\mathbb{E}[\boldsymbol{z}_n]$ 给出. PCA 的 EM 算法的 E 步骤变为

$$\boldsymbol{\Omega} = (\boldsymbol{W}^{\mathsf{T}}\boldsymbol{W})^{-1}\boldsymbol{W}^{\mathsf{T}}\tilde{\boldsymbol{X}}^{\mathsf{T}} \tag{16.58}$$

EM 算法的 M 步骤变为

$$\boldsymbol{W}^{\text{new}} = \tilde{\boldsymbol{X}}^{\mathsf{T}}\boldsymbol{\Omega}^{\mathsf{T}}(\boldsymbol{\Omega}\boldsymbol{\Omega}^{\mathsf{T}})^{-1}. \tag{16.59}$$

已这些也可以通过在线学习的方式实现。这些方程的简单解释如下。根据我们
寸论，E 步骤涉及将数据点正交投影到当前估计的主子空间。相应地，M 步骤
到定投影的情况下重新估计主子空间以最小化重建误差。
门可以用一个简单的物理类比理解这个 EM 算法，它在 $D=2$ 和 $M=1$ 时很容易
考虑二维空间中的一系列数据点，用一根坚硬的棒子代表一维主子空间。然后
合胡克定律（力与弹簧伸长量成正比，因此储能与弹簧伸长量的平方成正比）
将每个数据点连接到棒子上。在 E 步骤中，我们保持棒子固定并允许连接点沿
上下滑动以最小化能量。这会导致每个连接点（独立地）将自己定位在相应数
棒子的正交投影位置。在 M 步骤中，我们保持连接点固定，然后释放棒子，并
子移动到最小能量位置。然后重复 E 步骤和 M 步骤，直至满足适当的收敛标准，
10 所示。

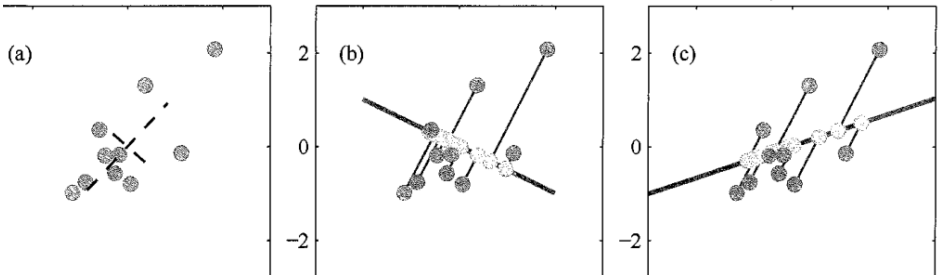


图 16.4: 用一个简单的物理类比来说明 PCA 的 EM 算法.(a) 一组绿色的数据点以及真实的主成分 (显示为按特征值平方根缩放的特征向量);(b) 由 $\boldsymbol{W}$ 定义的主子空间的初始配置 (显示为红色), 以及潜在点 $\boldsymbol{Z}$ 在数据空间中的投影 (由 $\boldsymbol{Z}\boldsymbol{W}^{\mathsf{T}}$ 给出, 显示为青色);(c) 经过第一个 M 步骤后,$\boldsymbol{W}$ 已经在 $\boldsymbol{Z}$ 保持固定的情况下得到更新;(d) 在随后的 E 步骤中,$\boldsymbol{Z}$ 的值已更新并给出了正交投影, 并且 $\boldsymbol{W}$ 保持不变;(e) 经过第二个 M 步骤之后的结果;(f) 收敛后的解.

我们可以用一个简单的物理类比理解这个 EM 算法, 它在 $D = 2$ 和 $M = 1$ 时很容易描绘. 考虑二维空间中的一系列数据点, 用一根坚硬的棒子代表一维主子空间. 然后通过符合胡克定律 (力与弹簧伸长量成正比, 因此储能与弹簧伸长量的平方成正比) 的弹簧将每个数据点连接到棒子上. 在 E 步骤中, 我们保持棒子固定并允许连接点沿着棒子上下滑动以最小化能量. 这会导致每个连接点 (独立地) 将自己定位在相应数据点到棒子的正交投影位置. 在 M 步骤中, 我们保持连接点固定, 然后释放棒子, 并允许棒子移动到最小能量位置. 然后重复 E 步骤和 M 步

骤, 直至满足适当的收敛标准.

以上这些也可以通过在线学习的方式实现. 这些方程的简单解释如下: 根据我们之前的讨论,E 步骤涉及将数据点正交投影到当前估计的主子空间. 相应地,M 步骤表示在固定投影的情况下重新估计主子空间以最小化重建误差.

16.3.3 因子分析的 EM 算法

我们可以使用最大似然法来确定因子分析模型中的参数 $\boldsymbol{\mu}$、$\boldsymbol{W}$ 和 $\boldsymbol{\Psi}$.$\boldsymbol{\mu}$ 的解再次由样本均值给出. 然而, 与概率 PCA 不同的是,$\boldsymbol{W}$ 不再有闭式的最大似然解, 只能通过迭代来找到. 由于因子分析模型是一个潜变量模型, 因此可以使用类似于概率 PCA 的 EM 算法来完成这项工作. 具体来说,E 步骤方程为

$$\mathbb{E}[\boldsymbol{z}_n] = \boldsymbol{G}\boldsymbol{W}^{\mathsf{T}}\tilde{\boldsymbol{X}}_n^{-1}(\boldsymbol{x}_n - \bar{\boldsymbol{x}}) \tag{16.60}$$

$$\mathbb{E}[\boldsymbol{z}_n\boldsymbol{z}_n^{\mathsf{T}}] = \boldsymbol{G} + \mathbb{E}[\boldsymbol{z}_n]\mathbb{E}[\boldsymbol{z}_n]^{\mathsf{T}} \tag{16.61}$$

其中

$$\boldsymbol{G} = (\boldsymbol{I} + \boldsymbol{W}^{\mathsf{T}}\boldsymbol{\Psi}^{-1}\boldsymbol{W})^{-1} \tag{16.62}$$

注意这是以一种涉及对 $M \times M$ 而不是 $D \times D$ 的矩阵 (除了 $D \times D$ 对角矩阵, 其逆在 $O(D)$ 步内可以轻松计算) 求逆的形式来表示的, 这很方便, 因为通常情况下 $M < D$.

同样,M 步骤方程为

$$\boldsymbol{W}^{\text{new}} = \left[\sum_{n=1}^{N}(\boldsymbol{x}_n - \bar{\boldsymbol{x}})\mathbb{E}[\boldsymbol{z}_n]^{\mathsf{T}}\right]\left[\sum_{n=1}^{N}\mathbb{E}[\boldsymbol{z}_n\boldsymbol{z}_n^{\mathsf{T}}]\right]^{-1} \tag{16.63}$$

$$\boldsymbol{\Psi}^{\text{new}} = \text{diag}\left\{\tilde{\boldsymbol{S}} - \frac{1}{N}\sum_{n=1}^{N}(\boldsymbol{x}_n - \bar{\boldsymbol{x}})\mathbb{E}[\boldsymbol{z}_n]^{\mathsf{T}}\boldsymbol{W}^{\text{new}\mathsf{T}}\right\} \tag{16.64}$$

其中 diag 运算符会将矩阵的所有非对角元素设置为零.

16.4 非线性潜变量模型

到目前为止, 在本章中, 我们关注的是基于从潜空间到数据空间的线性变换的潜变量模型. 自然地, 我们会问: 是否可以利用深度神经网络的灵活性来表示更复杂的变换, 同时利用深度神经网络的学习能力让得到的分布适配于一个数据集?

考虑一个简单的向量变量 $\boldsymbol{z}$ 上的分布, 例如高斯分布:

$$p(\boldsymbol{z}) = \mathcal{N}(\boldsymbol{z}|\boldsymbol{0}, \boldsymbol{I}) \tag{16.65}$$

假设我们使用一个由深度神经网络给出的函数 $\boldsymbol{x} = \boldsymbol{g}(\boldsymbol{z}, \boldsymbol{w})$ 来变换 $\boldsymbol{z}$, 其中 $\boldsymbol{w}$ 代表权重和偏置.$\boldsymbol{z}$ 上的分布与神经网络的结合定义了 $\boldsymbol{x}$ 上的分布. 从这样的模型中采样是直截了当的, 因为我们可以从 $p(\boldsymbol{z})$ 中生成样本, 然后使用神经网络函数对每个样本进行变换, 以获得对应的 $\boldsymbol{x}$ 的样本. 这是一个高效的过程, 因为它不涉及迭代.

16.4.1 非线性流形

要求潜空间和数据空间具有相同的维度是一个重大的限制. 考虑这样一种情况:$\boldsymbol{z}$ 的维度是 M,$\boldsymbol{x}$ 的维度是 D, 其中 $M < D$. 在这种情况下,$\boldsymbol{x}$ 上的分布局限于维度为 M 的流形或子空间. 许多机器学习应用中都出现了低维流形, 例如在模拟自然图像的分布时. 非线性潜变量模型 (nonlinear latent variable model) 在建模此类数据时可能非常有用, 因为它们表达了强烈的归纳偏置, 即数据并不” 填满” 数据空间, 而是被限制在一个流形上, 尽管这个流形的形状和维度通常是未知的.

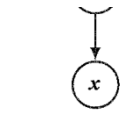
为了从数据中学习 $\boldsymbol{g}(\boldsymbol{z}, \boldsymbol{w})$, 考虑如何计算似然函数 $p(\boldsymbol{x}|\boldsymbol{w})$.$\boldsymbol{x}$ 上的分布由密度的变量变换公式给出:

$$p(\boldsymbol{x}) = p(\boldsymbol{z}(\boldsymbol{x}))\,|\det \boldsymbol{J}(\boldsymbol{x})| \tag{16.66}$$

其中 $\boldsymbol{J}$ 是偏导数的雅可比矩阵.

要计算式(16.66)右侧的分布 $p(\boldsymbol{z}(\boldsymbol{x}))$(对于给定的数据向量 $\boldsymbol{x}$) 和同一点的雅可比矩阵, 我们需要使用神经网络函数的逆函数 $\boldsymbol{z} = \boldsymbol{g}^{-1}(\boldsymbol{x}, \boldsymbol{w})$. 对于大多数神经网络来说, 这个逆函数将不是良好定义的. 例如, 神经网络可能实现一种多对一的函数关系, 在这种情况下, 不同的输入值会被映射到相同的输出值, 变量变换公式并不能给出良好定义的密度. 此外, 如果潜空间的维度与数据空间的维度不同, 那么变换将是不可逆的.

解决这个问题的一种方法是限制函数 $\boldsymbol{g}(\boldsymbol{z}, \boldsymbol{w})$ 是可逆的, 这要求 $\boldsymbol{z}$ 和 $\boldsymbol{x}$ 具有相同的维度. 我们将在介绍标准化流 (normalizing flow) 的技术时更详细地探索这种方法.



[式（16.77）]中抽取一个样本。然后将这个值作为神经网络的输入，得到输出值 $\boldsymbol{g}(\boldsymbol{z},\boldsymbol{w})$。最后从均值为 $\boldsymbol{g}(\boldsymbol{z},\boldsymbol{w})$、协方差为 $\sigma^2\boldsymbol{I}$ 的高斯分布中抽取一个样本[式（16.80）]。重复上述过程以生成多个独立样本。

式（16.77）和式（16.80）定义 $p(\boldsymbol{x},\boldsymbol{z})=p(\boldsymbol{x}|\boldsymbol{z})p(\boldsymbol{z})$ 的图模型

量分布 $p(\boldsymbol{z})$ 与条件分布 $p(\boldsymbol{x}|\boldsymbol{z})$ 的组合定义了数据空间上的边缘分布：

$$p(\boldsymbol{x})=\int p(\boldsymbol{z})p(\boldsymbol{x}|\boldsymbol{z})\mathrm{d}\boldsymbol{z} \tag{16.81}$$

可以用一个涉及一维潜空间和二维数据空间的简单例子来说明这一点，见

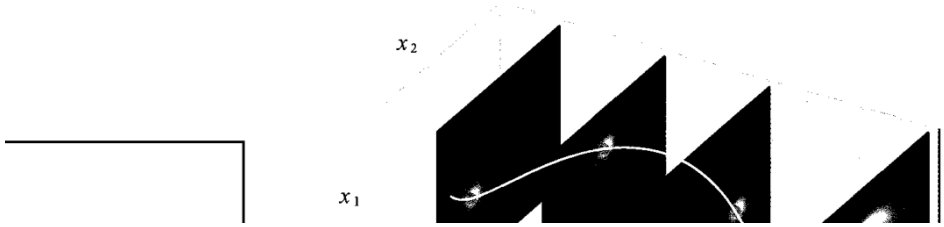


图 16.5: 一个涉及一维潜空间和二维数据空间的非线性潜变量模型的示意图.(a) 潜空间中的先验分布由零均值、单位方差的高斯分布给出;(b) 最左侧的三个图显示了不同 $\boldsymbol{z}$ 值下的高斯条件分布 $p(\boldsymbol{x}|\boldsymbol{z})$ 的示例, 最右侧的图显示了边缘分布 $p(\boldsymbol{x})$ 的示例. 定义条件分布均值的非线性函数 $\boldsymbol{g}(\boldsymbol{z})$ 由 $g_1(z) = \sin(z)$ 和 $g_2(z) = \cos(z)$ 给出, 条件分布的标准差由 $\sigma = 0.3$ 给出.

我们可以用一个涉及一维潜空间和二维数据空间的简单例子来说明这一点, 如图 16.5所示. 潜变量分布 $p(\boldsymbol{z})$ 与条件分布 $p(\boldsymbol{x}|\boldsymbol{z})$ 的组合定义了数据空间上的边缘分布:

$$p(\boldsymbol{x}) = \int p(\boldsymbol{z})p(\boldsymbol{x}|\boldsymbol{z})\,\mathrm{d}\boldsymbol{z}. \tag{16.67}$$

其中神经网络 $\boldsymbol{g}(\boldsymbol{z}, \boldsymbol{w})$ 的输出单元激活函数是线性的. 生成式模型由 $\boldsymbol{z}$ 上的潜变量分布和 $\boldsymbol{x}$ 上的条件分布指定:

$$p(\boldsymbol{x}|\boldsymbol{z}, \boldsymbol{w}) = \mathcal{N}(\boldsymbol{x}|\boldsymbol{g}(\boldsymbol{z}, \boldsymbol{w}), \sigma^2\boldsymbol{I}). \tag{16.68}$$

注意, 从这个分布中独立抽取样本不仅简单而且效率高. 首先使用标准方法从高斯分布中抽取一个样本, 然后将这个值作为神经网络的输入, 得到输出值 $\boldsymbol{g}(\boldsymbol{z}, \boldsymbol{w})$. 最后从均值为 $\boldsymbol{g}(\boldsymbol{z}, \boldsymbol{w})$、协方差为 $\sigma^2\boldsymbol{I}$ 的高斯分布中抽取一个样本.

16.4.2 似然函数

我们已经看到从这个非线性潜变量模型中抽取样本是很容易的. 假设我们希望通过最大化似然函数来拟合观测到的数据集. 可以通过对 $\boldsymbol{z}$ 进行积分并使用概率的加和法则和乘积法则来得到似然值:

$$p(\boldsymbol{x}|\boldsymbol{w}) = \int p(\boldsymbol{x}|\boldsymbol{z},\boldsymbol{w})p(\boldsymbol{z})\,\mathrm{d}\boldsymbol{z}. \tag{16.69}$$

尽管积分内部的两个分布都是高斯分布, 但由神经网络定义的高度非线性函数 $\boldsymbol{g}(\boldsymbol{z},\boldsymbol{w})$ 却使得积分在解析上是不可处理的. 一种计算似然函数的方法是从潜空间分布 $p(\boldsymbol{z})$ 中抽取 K 个样本, 并用这些样本来近似式(16.69), 从而得到

$$p(\boldsymbol{x}|\boldsymbol{w}) \approx \frac{1}{K}\sum_{k=1}^{K} p(\boldsymbol{x}|\boldsymbol{z}^{(k)},\boldsymbol{w}), \qquad \boldsymbol{z}^{(k)} \sim p(\boldsymbol{z}) \tag{16.70}$$

这将 $\boldsymbol{z}$ 的分布表示为具有固定混合系数 $1/K$ 的高斯混合分布. 当样本数量趋于无穷时, 该方法给出了真实的似然函数. 然而, 实际训练中所需的 K 值通常会大得不切实际.

为什么会这样? 考虑三个手写数字图像的例子. 假设图 (a) 代表我们希望计算似然函数的向量 $\boldsymbol{x}$. 如果一个训练好的模型生成了图 (b), 我们会认为这是一个比较差的模型, 因为这个图像并不是数字"2" 的一个很好的表示, 所以应该被赋予一个更低的似然值. 相反, 图 (c) 是通过将图 (a) 中的数字向下和向右移动 0.5 个像素获得的, 是数字"2" 的一个很好的表示, 因此应该被赋予一个更高的似然值. 由于是高斯分布, 似然函数与网络输出和数据向量 $\boldsymbol{x}$ 之间负平方距离的指数成正比. 如果方差参数 σ^2 设置得足够小, 使得图 (b) 具有低似然值, 那么图 (c) 就会有更低的似然值. 即使模型在生成数字方面做得很好, 我们也必须抽取极大量的 $\boldsymbol{z}$ 样本, 才能找到一个足够接近图 (a) 的样本.

然而, 这个框架将零概率密度赋予那些不完全位于流形上的任何数据向量, 这对基于梯度的学习算法是一个问题, 因为对于真实的数据集, 似然函数在每一个数据点都将为零, 并且对于小的 $\boldsymbol{w}$ 变化将保持不变. 为了解决这个问题, 我们使用之前用于解决回归和分类问题的策略, 定义了整个数据空间上的条件分布, 其参数由神经网络的输出决定. 例如, 如果 $\boldsymbol{x}$ 是连续变量的一个向量, 则可以选择条件分布为高斯分布.

16.4.3 离散数据

如果观测到的数据集包含独立的二进制变量, 则可以使用以下形式的条件分布:

$$p(\boldsymbol{x}|\boldsymbol{z},\boldsymbol{w}) = \prod_{i=1}^{D} \mu_i(\boldsymbol{z},\boldsymbol{w})^{x_i}(1-\mu_i(\boldsymbol{z},\boldsymbol{w}))^{1-x_i} \tag{16.71}$$

其中 $\mu_i(\boldsymbol{z},\boldsymbol{w}) = \sigma(a_i(\boldsymbol{z},\boldsymbol{w}))$ 是输出单元 i 的激活值, 激活函数 $\sigma(\cdot)$ 由逻辑斯谛 sigmoid 函数给出,$a_i(\boldsymbol{z},\boldsymbol{w})$ 是输出单元 i 的预激活值. 类似地, 对于独热编码 (one-hot encoded) 的分类变量, 可以使用多项式分布:

$$p(\boldsymbol{x}|\boldsymbol{z},\boldsymbol{w}) = \prod_{i=1}^{D} \mathrm{softmax}(\boldsymbol{a}(\boldsymbol{z},\boldsymbol{w}))^{x_i} \tag{16.72}$$

我们还可以通过构造相关条件分布的乘积来考虑离散变量和连续变量的组合.

在实际应用中, 连续变量通常以离散值表示, 例如在图像中, 红色、绿色和蓝色通道的强度可能取自 8 位二进制数字所能表示的数值集 $\{0,\cdots,255\}$. 当我们采用基于深度神经网络的高度灵活模型时, 这可能会导致一个问题: 如果密度坍塌到一个或多个离散值上, 似然函数就有可能变为零, 导致异常解. 这个问题可以通过使用一种称为去量化 (dequantization) 的技术来解决, 该技术涉及向变量添加噪声, 这些噪声在连续离散值之间的区域内通常是均匀分布的. 训练集的去量化就是用随机抽取的样本替换每个观测值, 这些样本来自与特定离散值相关联的连续分布. 采用这种方法, 模型找到异常解的可能性就会降低.

16.4.4 构建生成式模型的四种方法

我们已经看到, 基于深度神经网络的非线性潜变量模型提供了一个高度灵活的框架来构建生成式模型. 由于神经网络变换的普适性, 这些模型原则上能够以高精度近似任何期望的分布. 此外, 一旦训练好, 这些模型还可以

使用一种高效、非迭代的过程从分布中生成样本. 然而, 我们也面临着一些与训练此类模型相关的挑战, 迫切需要找到比线性模型更复杂的方法. 业内已经提出了不少方法, 每种方法都有自身的优势和局限性, 它们大致可以分为以下四种:

1. 基于生成对抗网络 (Generative Adversarial Network, GAN)(见第 17 章) 放宽" 网络映射必须是可逆的" 这一要求, 允许潜空间的维度低于数据空间的维度. 这里也放弃了似然函数的概念, 引入了第二个神经网络, 其功能是为生成网络提供训练信号. 由于缺乏明确定义的似然函数, 训练过程可能不稳定, 但一旦训练完成, 就很容易从模型中生成样本, 并且所得结果的质量也高.
2. 变分自编码器 (Variational AutoEncoder, VAE)(见第 19 章) 也使用了第二个神经网络, 其作用是近似潜变量的后验分布, 从而允许计算似然函数的近似值. 与 GAN 相比, 其能使训练更为稳健, 而且能直接从训练好的模型中采样, 但是要获得高质量的结果可能更困难.
3. 在标准化流 (normalizing flow)(见第 18 章) 中, 将潜空间的维度设置为等于数据空间的维度, 然后修改神经网络, 使其变得可逆." 网络必须是可逆的" 这一要求限制了其函数形式, 但也使得无须近似就能计算似然函数, 而且能进行高效采样.
4. 扩散模型 (diffusion model)(见第 20 章) 使用一个网络学习如何通过一系列去噪步骤将先验分布的样本转换为数据分布的样本. 这在许多应用中带来了先进的性能, 缺点是由于多次经历网络的去噪过程, 采样成本可能较高.

习题

16.1(⋆⋆) 使用归纳法证明, 将数据投影到 M 维子空间以最大化投影数据的方差所定义的线性投影是由数据协方差矩阵 $\boldsymbol{S}$ 的 M 个特征向量定义的, 对应于 M 个最大特征值.

16.2(⋆⋆) 证明在满足正交性质 [式(16.7)] 的情况下,PCA 误差度量 J[由式(16.10)给出] 的最小值是在 $\boldsymbol{u}_j$ 为数据协方差矩阵 $\boldsymbol{S}$ 的特征向量时获得的.

16.3(⋆) 假设式(16.23)定义的特征向量 $\boldsymbol{v}$ 具有单位长度, 验证这些特征向量已归一化至单位长度.

16.4(⋆) 假设在概率 PCA 模型中将零均值、单位协方差的潜空间分布 [式(16.24)] 替换为一般形式的高斯分布 $\mathcal{N}(\boldsymbol{z}|\boldsymbol{m}, \boldsymbol{\Sigma})$. 通过重新定义模型的参数, 证明这种替换会使得任意有效的 $\boldsymbol{m}$ 和 $\boldsymbol{\Sigma}$ 都无法改变观测变量边缘分布 $p(\boldsymbol{x})$ 的形式.

16.6(⋆⋆) 利用一般高斯分布的均值和协方差公式, 推导概率 PCA 模型中边缘分布 $p(\boldsymbol{x})$ 的结果 [式(16.28)].

16.7(⋆) 绘制 16.2 节描述的概率 PCA 模型的有向图, 并将观测变量 $\boldsymbol{x}$ 的各个组成部分明确显示为单独的节点. 验证概率 PCA 模型具有与朴素贝叶斯模型相同的独立结构.

16.8(⋆⋆) 利用线性–高斯模型的结果, 证明概率 PCA 模型的后验分布 $p(\boldsymbol{z}|\boldsymbol{x})$ 可以由式(16.37)给出.

16.9(⋆) 验证对概率 PCA 模型的对数似然函数 [式(16.33)] 关于参数 $\boldsymbol{\mu}$ 进行最大化会得到结果 $\boldsymbol{\mu} = \bar{\boldsymbol{x}}$, 其中 $\bar{\boldsymbol{x}}$ 是数据向量的均值.

16.10(⋆⋆) 通过计算对数似然函数对参数 $\boldsymbol{\mu}$ 的二阶导数, 证明驻点 $\boldsymbol{\mu} = \bar{\boldsymbol{x}}$ 代表唯一的最大值.

16.11(⋆⋆) 证明当 $\sigma^2 \to 0$ 时, 概率 PCA 模型的后验均值相当于对主子空间的正交投影, 这与传统 PCA 模型的结果一致.

16.12(⋆⋆) 对于 $\sigma^2 > 0$, 证明概率 PCA 模型中的后验均值会相对于正交投影向原点移动.

16.13(⋆⋆) 证明在概率 PCA 下, 根据传统 PCA 的最小二乘投影成本, 数据点的最优重建由式(16.39)给出.

16.14(⋆) 概率 PCA 模型中协方差矩阵的独立参数数量由式(16.29)给出. 证明对于 $M = D - 1$, 独立参数的数量与一般高斯分布中的相同; 而对于 $M = 0$, 独立参数的数量与各向同性协方差高斯分布相同.

16.15(⋆) 推导出小节 16.2.4描述的因子分析模型中独立参数数量的计算表达式.

16.16(⋆⋆) 证明小节 16.2.4描述的因子分析模型不随潜空间坐标旋转而变化.

16.17(⋆⋆) 考虑一个线性–高斯潜变量模型, 它具有潜空间分布 $p(\boldsymbol{z}) = \mathcal{N}(\boldsymbol{z}|\boldsymbol{0}, \boldsymbol{I})$ 和观测变量的条件分布 $p(\boldsymbol{x}|\boldsymbol{z}) = \mathcal{N}(\boldsymbol{x}|\boldsymbol{W}\boldsymbol{z} + \boldsymbol{\mu}, \boldsymbol{\Psi})$. 对数据变量进行非奇异线性变换 $\boldsymbol{x} \to \boldsymbol{A}\boldsymbol{x}$, 其中 $\boldsymbol{A}$ 是一个 $D \times D$ 矩阵. 证明在两种

情况下模型的形式得以保留:(i) $\boldsymbol{A}$ 是对角矩阵且 $\boldsymbol{\Psi}$ 也是对角矩阵 (对应于因子分析);(ii) $\boldsymbol{A}$ 是正交矩阵且 $\boldsymbol{\Psi}$ 与单位矩阵成比例 (对应于概率 PCA).

16.18(⋆) 验证连续潜变量模型对数似然函数可以写成两项之和 [式(16.45)], 这两项分别由式(16.46)和式(16.47)定义.

16.19(⋆) 证明对于一组独立同分布数据, 证据下界 (ELBO) 可以采用式(16.51)的形式.

16.21(⋆⋆) 通过最大化式(16.53)给出的完整数据对数似然函数的期望来推导概率 PCA 模型的 M 步骤方程 [式(16.56)] 和 [式(16.57)].

16.22(⋆⋆⋆) 概率 PCA 的一个优点是, 它可以应用于某些值缺失的数据集, 前提是这些值是随机缺失的. 在这种情况下, 推导出最大化概率 PCA 模型似然函数的 EM 算法.

16.25(⋆⋆) 给出因子分析模型的完整数据对数似然函数的期望的表达式, 从而导出相应的 M 步骤方程 [式(16.63)] 和 [式(16.64)].

16.26(⋆⋆) 通过考虑二阶导数, 证明小节 16.2.4讨论的因子分析模型的对数似然函数相对于参数 $\boldsymbol{\mu}$ 的唯一驻点是由式(16.1)定义的样本均值给出的. 此外, 证明这个驻点是一个最大值.

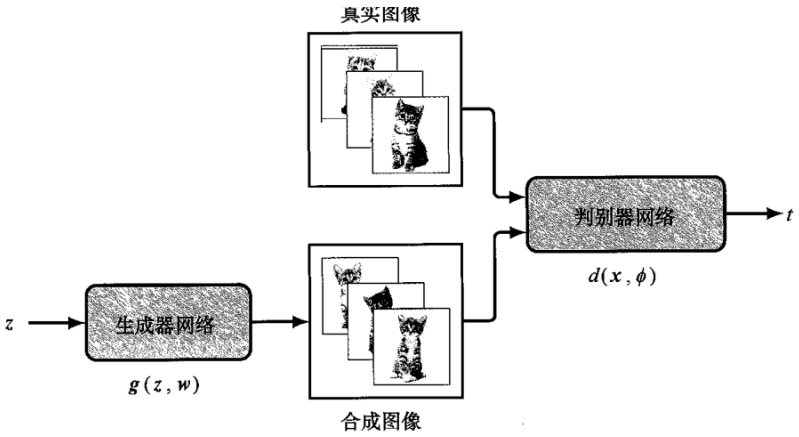
第 17 章 生成对抗网络

17.1 对抗训练

考虑一个基于从潜空间 z 到数据空间 x 的非线性变换的生成式模型. 引入一个潜空间分布 $p(z)$, 它可能是一个简单的高斯分布:

$$p(z) = \mathcal{N}(z \mid \mathbf{0}, \mathbf{I}) \tag{17.1}$$

同时定义一个由深度神经网络实现的非线性变换 $x = g(z, w)$, 其中可学习参数 w 对应的网络称为**生成器** (generator) 网络. 它们共同隐式地定义了 x 上的分布, 我们的目标是将这个分布拟合到一组训练样本 $\{x_n\}$ 上, 其中 $n = 1, \cdots, N$. 然而, 我们并不能通过优化似然函数来确定 w, 因为这通常无法以封闭形式来计算. 生成对抗网络 (Goodfellow et al., 2014; Ruthotto and Haber, 2021) 的关键思想是引入判别器 (critic) 网络, 判别器网络与生成器网络一起训练, 并为更新生成器的权重提供训练信号, 如图 17.1 所示.



成对抗网络的示意图。在这样的网络中，训练判别器网络 $d(x,\phi)$ 以区分来自训练集的真实样本（小猫由生成器网络 $g(z, w)$ 产生的合成样本。生成器网络的目标是通过产生逼真的图像来让判别器网络产

图 17.1: 生成对抗网络的示意图. 在这样的网络中, 训练判别器网络 $d(x, \phi)$ 以区分来自训练集的真实样本 (小猫的图像) 和由生成器网络 $g(z, w)$ 产生的合成样本. 生成器网络的目标是通过产生逼真的图像来让判别器网络产生更多的误判, 而判别器网络的目的是通过更好地区分真实样本和合成样本来减少误判.

判别器网络的目标是区分来自数据集的真实样本和由生成器网络的合成 (或 "假") 样本, 它通过最小化传统的分类误差函数来进行训练. 相反, 生成器网络的目标是通过生成与训练集同分布的样本来最大化这个误差. 因此, 生成器网络和判别器网络互相对抗,"生成对抗网络" 由此得名. 这是一种**零和博弈** (zero-sum game), 在这场零和博弈中, 任何一个网络的收益都代表另一个网络的损失. 生成对抗网络通过允许判别器网络提供一个训练信号来训练生成器网络, 从而将无监督的密度建模问题转换成了一种监督学习的形式.

17.1.1 损失函数

判别器网络拥有单个输出单元, 带有 sigmoid 激活函数, 其输出表示数据向量 x 为真实数据的概率:

$$P(t = 1) = d(x, \phi) \tag{17.2}$$

为了明确这一点, 我们定义一个二元目标变量 t:

$$t = 1, \text{代表真实数据} \tag{17.3}$$

$$t = 0, \text{代表合成数据} \tag{17.4}$$

我们可以使用如下标准的交叉熵损失函数来训练判别器网络:

$$E(w,\phi)=\frac{1}{N}\sum_{n=1}^{N}\Big\{t_n\ln d(x_n,\phi)+(1-t_n)\ln\big(1-d(g(z_n,w),\phi)\big)\Big\} \tag{17.5}$$

其中 $d_n=d(x_n,\phi)$ 是判别器网络的输出, 我们已经通过数据点的总数对它进行了归一化 (参见 1.2.4 小节). 训练集包括标记为 x_n 的真实样本和由生成器网络 $g(z,w)$ 输出的合成样本, 其中 z 是从潜空间分布 $p(z)$ 中采样的随机样本. 对于真实样本,$t_n=1$; 对于合成样本,$t_n=0$. 我们可以将损失函数写成以下形式:

$$E_{\text{GAN}}(w,\phi)=\frac{1}{N}\sum_{n=1}^{N}\ln d(x_n,\phi)+\frac{1}{N_G}\sum_{m=1}^{N_G}\ln\big(1-d(g(z_m,w),\phi)\big) \tag{17.6}$$

通常情况下, 真实数据点的数量 N 等于合成数据点的数量 N_G. 这种生成器网络和判别器网络的组合可以使用随机梯度下降进行端到端的训练 (参见第 7 章), 并通过反向传播来计算梯度. 然而, 这种训练方式的独特之处在于其对抗性, 即误差相对于 ϕ 最小化, 而相对于 w 最大化.

这种最大化可以通过标准的基于梯度的方法来完成, 通过反转梯度的符号, 参数更新为

$$\Delta\phi=-\lambda\nabla_{\phi}E(w,\phi) \tag{17.7}$$

$$\Delta w=+\lambda\nabla_{w}E(w,\phi) \tag{17.8}$$

其中 $E(w,\phi)$ 表示为数据点 n 定义的误差, 或者更一般地说, 是为小批量数据点定义的误差. 注意式 (17.7) 和式 (17.8) 右侧的符号不同, 因为判别器网络的训练目标是降低错误率, 而生成器网络的训练目标是提高错误率. 实际上, 训练交替地在更新生成器网络的参数和更新判别器网络的参数之间进行, 每次只使用一个小批量数据.

一旦 GAN 训练完毕, 判别器网络就会被舍弃, 生成器网络则用来从潜空间中采样并将这些样本经训练好的生成器网络加以传播, 从而在数据空间中合成新的样本. 我们可以证明, 对于具有无限灵活性的生成器网络和判别器网络, 完全优化的 GAN 将具有与数据分布完全匹配的生成分布 (见习题 17.1). 如果生成器网络成功找到完美解决方案, 则判别器网络将无法区分真实数据和合成数据, 因此其输出始终为 0.5.

到目前为止我们所讨论的 GAN 都是从无条件分布 $p(x)$ 中生成样本的. 例如, 如果我们用狗的图像对 GAN 进行训练, 它就会生成合成的狗的图像. 我们也可以创建**条件 GAN**(Mirza and Osindero, 2014), 它从条件分布 $p(x\mid c)$ 中采样, 其中条件向量 c 可能代表不同品种的狗. 为此, 生成器网络和判别器网络会将 c 作为额外输入, 并使用标有标签的图像实例 $\{x_n,c_n\}$ 对 GAN 进行训练. 一旦 GAN 训练好了, 我们就可以通过将 c 设置为相应的类向量来生成所需类别的图像. 与为每个类别训练单独的 GAN 相比, 这样做有一个优势, 就是可以跨所有类别学习共享的内部表示, 从而更有效地利用数据.

17.1.2 实战中的 GAN 训练

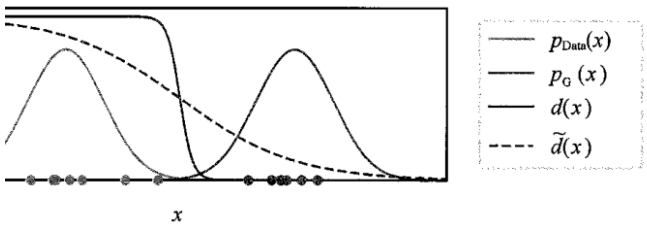
尽管 GAN 可以产生高质量的结果, 但由于对抗学习的特点, 其并不容易训练成功 (见习题 17.2). 此外, 与标准的损失函数最小化不同, 由于目标函数的值在训练过程中可能上升也可能下降, 因此并没有一个明确的进展指标.

GAN 在训练过程中可能出现**模式崩溃** (mode collapse) 的问题, 即在训练过程中, 生成器网络的权重发生了适应性调整, 导致所有潜变量样本 z 都被映射到有效输出的一个子集上. 在极端情况下, 输出可能只对应一个或少数几个输出值 x. 判别器网络随后将这些实例的值设置为 0.5 并停止训练. 例如, 一个生成手写数字训练的 GAN 可能学会只生成数字 “3” 的实例, 而判别器网络无法将这些实例与真正的数字 “3” 的实例区分开来, 它没能认识到生成器网络并没有生成所有可能的数字.

通过图 17.2, 我们可以更深入地理解训练 GAN 的困难. 该图显示了一个简单的一维数据空间 x, 其中样本 $\{x_n\}$ 来自固定但未知的数据分布 $p_{\text{data}}(x)$. 该图还显示了初始生成分布 $p_g(x)$ 以及从该分布中抽取的样本. 因为数据分布和生成分布之间差异显著, 最优的判别器函数 $d(x)$ 很容易学习, 其函数值下降态势急剧, 在真实样本或合成样本的邻近区域梯度几乎为零. 考虑 GAN 损失函数 [式 (17.6)] 中的第二项, 由于在生成样本跨越的区域内 $d(g(z,w),\phi)$ 值等于零, 生成器网络参数 w 的微小变化在判别器网络的输出上将产生很小的变化, 因此梯度较小, 导致学习进程缓慢.

这个问题可以通过使用判别器函数的平滑版本 $\tilde{d}(x)$ 来解决 (见图 17.2), 从而为生成器网络提供更强的梯

度以驱动训练. 最小二乘 GAN(least-squares GAN, Mao et al., 2016) 通过修改判别器网络输出一个实值而不是取值范围为 0 ~ 1 的概率, 并用平方和损失函数替换交叉熵损失函数来实现平滑. 另一种称为**实例噪声** (instance noise)(Sönderby et al., 2016) 的方法则向真实样本和合成样本中添加高斯噪声, 这同样能使判别器函数更加平滑.



什么训练 GAN 可能会有难度，这里展示了一个简单的一维数据空间 x，其中有固

图 17.2: 为了从概念上说明为什么训练 GAN 可能会有难度, 这里展示了一个简单的一维数据空间 x, 其中有固定但未知的数据分布 $p_{\text{data}}(x)$ 和初始生成分布 $p_g(x)$. 最优的判别函数 $d(x)$ 在训练或合成数据点附近几乎没有梯度, 使学习变得非常缓慢. 判别函数的平滑版本 $\tilde{d}(x)$ 可以使学习变得更快.

更直接的方法是修改误差标准, 以反映这两个分布在数据空间中的距离. 这可以使用 **Wasserstein 距离** (也称推土机距离) 来衡量. 设想生成分布 $p_g(x)$ 是一堆土, 我们需要分批运输这堆土以构建数据分布 $p_{\text{data}}(x)$.Wasserstein 距离就是移动的土壤总量和平均移动距离的积. 在重排这堆土以构建 $p_{\text{data}}(x)$ 的许多方式中, 能产生最小平均移动距离的方式被用于定义度量. 为确保生成分布 $p_g(x)$ 向数据分布 $p_{\text{data}}(x)$ 靠拢, 这种方法在实践中并不能直接实现, 需要通过使用一个具有实值输出的判别器网络来近似, 然后通过权重裁剪限制判别器函数关于 x 的梯度 $\nabla_x d(x, \phi)$, 从而产生了 Wasserstein GAN(Arjovsky, Chintala, and Bottou, 2017).

另一种改进的方法是对梯度引入惩罚, 从而产生了梯度惩罚 Wasserstein GAN(Gulrajani et al., 2017), 其损失函数由下式给出:

$$E_{\text{WGAN}}(w, \phi) = -\frac{1}{N}\sum_n d(x_n, \phi) + \frac{1}{N_G}\sum_m d(g(z_m, w), \phi) + \lambda\,\mathbb{E}\left[(\|\nabla_{\hat{x}} d(\hat{x}, \phi)\|_2 - 1)^2\right] \tag{17.9}$$

其中的 λ 控制了惩罚项的相对重要性.

为了改进 GAN 的训练, 研究者们提出了许多对损失函数和训练过程的修改方案 (Mescheder, Geiger, and Nowozin, 2018). 其中常见的一个建议是将原始损失函数中的生成器目标

$$\mathbb{E}_z \ln\left(1 - d(g(z, w), \phi)\right) \tag{17.10}$$

替换为

$$-\mathbb{E}_z \ln d(g(z, w), \phi) \tag{17.11}$$

式 (17.10) 所示的第一种形式最小化了图像为假的概率, 式 (17.11) 所示的第二种形式则最大化了图像为真的概率. 这两种形式的不同属性可以通过图 17.3 来加以理解. 当生成分布 $p_g(x)$ 与数据分布 $p_{\text{data}}(x)$ 完全不重叠时,$d(g(z, w))$ 接近于零, 因此第一种形式具有非常小的梯度, 而第二种形式具有较大的梯度, 从而能加快训练速度.

17.2 图像的生成对抗网络

GAN 的基本概念催生了大量的研究工作, 涵盖了许多算法和众多应用, 其中应用最广泛且成功的是图像生成. 早期的 GAN 使用全连接网络作为生成器网络和判别器网络 (参见第 10 章). 然而, 使用卷积网络有许多好处, 尤其是对于更高分辨率的图像而言. 判别器网络将图像作为输入, 并提供一个标量概率作为输出, 因此使用标准的卷积网络非常合适 (参见 10.5.3 小节). 生成器网络需要将低维潜空间映射成高分辨率图像, 因此使用了基于转置卷积的网络, 如图 17.4 所示.

从低分辨率开始, 随着训练的进行, 逐步添加新的卷积层, 就能模拟越来越精细的细节 (Karras et al., 2017). 这种方法不仅加速了训练过程, 而且能够从 4×4 大小的图像开始, 最终合成 1024×1024 大小的高分辨率图像.

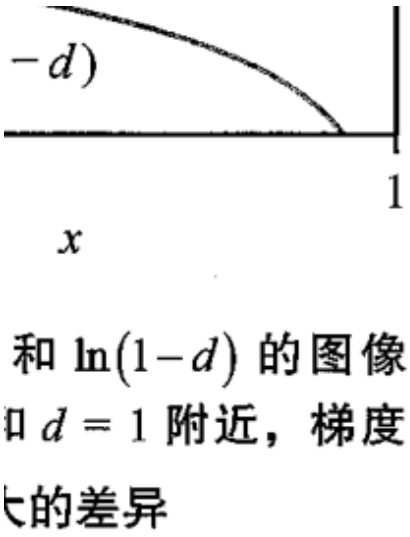
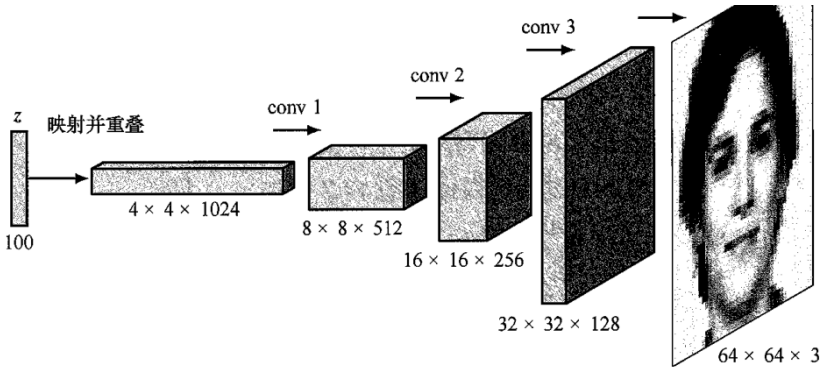


图 17.3: $-\ln(d)$ 和 $-\ln(1-d)$ 的图像. 在 $d=0$ 和 $d=1$ 附近, 两者在梯度上的表现存在非常大的差异:$-\ln(1-d)$ 在 $d \to 0$ 时梯度趋近于零, 而 $-\ln(d)$ 仍提供显著梯度.



度卷积 GAN 的示例架构，这里展示了如何在网络的连续模块中使用转置卷积来扩展维度

分辨率开始，随着训练的进行，逐步添加新的卷积层，就能模拟越来越精

图 17.4: 深度卷积 GAN 的示例架构, 这里展示了如何在网络的连续模块中使用转置卷积来扩展维度: 从 100 维潜向量出发, 经 $4\times4\times1024$、$8\times8\times512$、$16\times16\times256$、$32\times32\times128$, 最终得到 $64\times64\times3$ 的输出图像.

图 17.5 展示了用于类别条件图像生成的 GAN 模型—BigGAN, 这是体现 GAN 架构的规模和复杂性的一个典型例子.BigGAN 的生成器网络拥有超过 7000 万个参数, 而拥有 8800 万个参数的判别器网络具有类似的架构, 不同之处在于判别器网络使用平均池化层 (average pooling) 来降低维度, 而不是通过上采样 (upsampling) 来增加维度 (Brock, Donahue, and Simonyan, 2018).

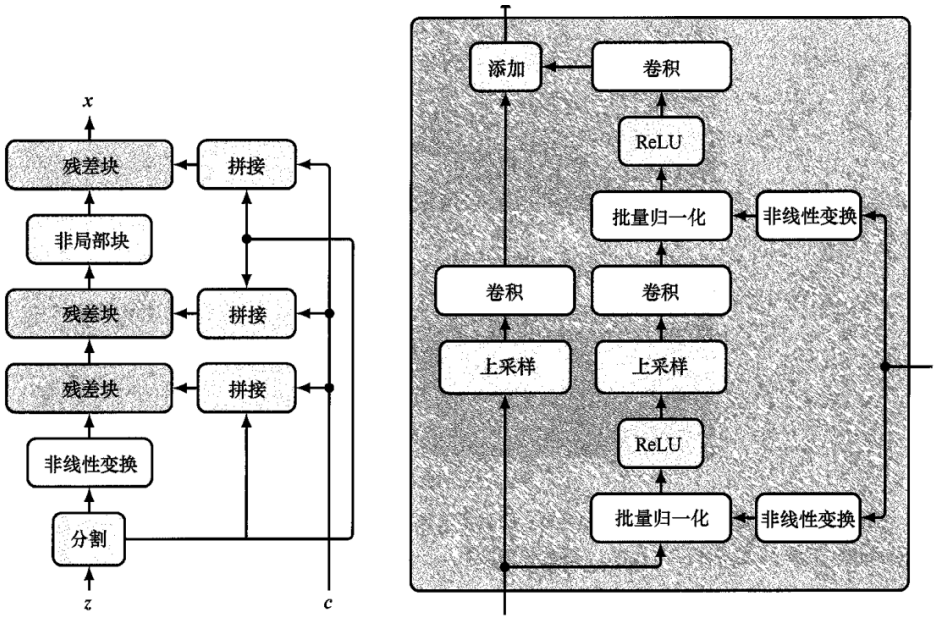


图 17.5 **(a) BigGAN 模型中生成器网络的架构，它拥有超过 7000 万个参数。(b) 生成器网络中每个残差块的细节。拥有 8800 万个参数的判别器网络具有类似的架构，不同之处在于判别器网络使用平均池化层（average pooling）来降低维度，而不是通过上采样（up-sampling）来增加维度［基于 Brock, Donahue, and Simonyan（2018）］**

图 17.5: (a) BigGAN 模型中生成器网络的架构.(b) 生成器网络中每个残差块的细节, 包含上采样、卷积、批量归一化、非线性变换、拼接与非局部块等组件.

17.2.1 CycleGAN

作为 GAN 多样性的一个例子, 考虑一种称为 CycleGAN(Zhu et al., 2017) 的 GAN 架构, 我们将其作为示例以展示深度学习技术如何用于解决传统任务 (如分类和密度估计) 之外的不同类型的问题. 考虑如下问题: 如何将一张照片转换为同一场景下的莫奈风格的画作 (或者反之)?

CycleGAN 的目标是学习两个双射 (一对一映射), 一个是从照片的域 X 到莫奈风格画作的域 Y, 另一个则相反. 为了实现这一点,CycleGAN 利用了两个条件生成器 (g_X 和 g_Y) 以及两个判别器 (d_X 和 d_Y). 生成器 $g_X(y, w_X)$ 以一幅画作 $y \in Y$ 为输入, 并生成相应的合成照片, 判别器 $d_X(x, \phi_X)$ 则区分合成照片和真实照片. 类似地, 生成器 $g_Y(x, w_Y)$ 以一张照片 $x \in X$ 为输入并生成一幅合成画作 y, 判别器 $d_Y(y, \phi_Y)$ 则区分合成画作和真实画作. 因此, 对判别器 d_X 的训练, 需要结合 g_X 生成的合成照片和真实照片来进行, 而判别器 d_Y 则是在 g_Y 生成的合成画作和真实画作的组合上进行训练的.

如果我们使用标准的 GAN 损失函数来训练这种架构,CycleGAN 就能学会生成逼真的合成画作和合成照片, 但没有什么机制能确保生成的画作看起来像对应的照片, 反之亦然. 因此, 我们在损失函数中引入了一个额外的术语, 称为**循环一致性误差** (cycle consistency error). 它包含两项, 其构建过程如图 17.7 所示. 这一过程的目标是确保当一张照片先被转换成一幅画, 再被转换回照片时, 结果与原始照片接近, 从而确保生成的画作保留了足以恢复原始照片的信息. 类似地, 当一幅画作先被转换成一张合成照片, 再被转换回一幅画作时, 结果也应该与原始画作接近.



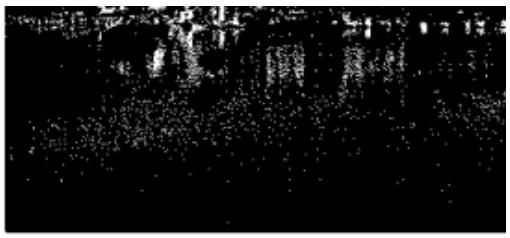
画作→合成照片



照片→合成画作

进行图像转换的示例：由莫奈风格画作合成摄影风格照片（上排），以及由照片合成莫

图 17.6: 使用 CycleGAN 进行图像转换的示例: 由莫奈风格画作合成摄影风格照片 (上排), 以及由照片合成莫奈风格画作 (下排)(Zhu et al., 2017, 经授权使用).



照片-

图 17.6 使用 CycleGAN 进行图像转换的示例：由莫

奈风格画作（下排）[Zhu et al.（2017），经授权使用

图 17.7: 为示例照片 x_n 计算循环一致性误差. 首先使用生成器 g_Y 将照片映射到画作域, 然后将得到的向量映射回照片域, 使用的是生成器 g_X. 结果照片与原始照片 x_n 之间的差异是由循环一致性误差贡献的.

将这个理念应用到训练集中的所有照片和画作, 就会得到形式如下的循环一致性误差:

$$E(w_X, w_Y) = \frac{1}{N_X}\sum_{n=1}^{N_X}\left\|g_X\left(g_Y(x_n, w_Y), w_X\right) - x_n\right\|_1 + \frac{1}{N_Y}\sum_{m=1}^{N_Y}\left\|g_Y\left(g_X(y_m, w_X), w_Y\right) - y_m\right\|_1 \tag{17.12}$$

其中 $\|\cdot\|_1$ 表示 L_1 范数. 将循环一致性误差添加到式 (17.6) 定义的 GAN 损失函数中, 便可得到总的损失函数:

$$E_{\text{total}} = E_{\text{GAN}}(w_X, \phi_X) + E_{\text{GAN}}(w_Y, \phi_Y) + \eta\, E(w_X, w_Y) \tag{17.13}$$

其中系数 η 决定了 GAN 误差和循环一致性误差的相对重要性. 在计算一个图像和一幅画作的损失函数时, 通过 CycleGAN 的信息流如图 17.8 所示.

其中系数η决定了 GAN 误差和循环一致性误差的相对重要性。在计算一个图像和一幅画作的损失函数时，通过 CycleGAN 的信息流如图 17.8 所示。



图 17.8: 通过 CycleGAN 的信息流. 数据点 x 和 y 的总误差是其 4 个组成部分误差的总和: 两个方向的 GAN 误差与两个方向的循环一致性误差.

17.2.2 表示学习与潜空间插值

我们已经看到,GAN 能胜任生成式模型的工作, 它也可用于**表示学习** (representation learning)(参见 6.3.3 小节), 其通过无监督学习揭示了数据集中丰富的统计结构. 当用图 17.4 所示的深度卷积 GAN 在一个卧室图像数据集上进行训练, 并且从潜空间中随机抽取样本, 然后通过已训练的网络加以传播时, 生成的图像看起来也像卧室, 正如我们预期的那样 (Radford, Metz, and Chintala, 2015).

此外, 潜空间已经以具有语义意义的方式组织起来. 例如, 如果我们沿着潜空间中的平滑轨迹进行跟踪, 并生成相应的一系列图像, 就可以实现从一个图像到另一个图像的平滑过渡, 如图 17.9 所示. 其中的每一行都是通过在潜空间内部随机生成的位置之间进行平滑移动得到的. 图像的过渡很平滑, 每幅图像看起来都像是一个合理的卧室. 在最下方一行, 我们可以看到墙上的电视逐渐变形成了窗户.

且从潜空间中随机抽取样本，然后通过已训练的网络加以传播时，生成的图像看起来也像卧室，正如我们预期的那样（Radford, Metz and Chintala, 2015）。此外，潜空间已经以具有语义意义的方式组织起来。例如，如果我们沿着潜空间中的平滑轨迹进行跟踪，并生成相应的一系列图像，就可以实现从一个图像到另一个图像的平滑过渡，如图 17.9 所示。



图 17.9: 由在卧室图像上训练的深度卷积 GAN 生成的样本. 每一行都是通过在潜空间内部随机生成的位置之间进行平滑移动得到的; 最下方一行中墙上的电视逐渐变形成了窗户 (Radford, Metz, and Chintala, 2015, 经授权使用).

此外, 识别潜空间中与具有语义意义的变换相对应的方向也是可能的. 例如, 对人脸图像, 某个方向可能对应面部朝向的变化, 而其他方向可能对应光照变化或面部微笑程度的变化, 这称为**解耦表示** (disentangled representation). 我们可以利用它们来合成具有特定属性的新图像.

图 17.10 是一个关于在面部图像上训练的 GAN 的例子. 对左侧三列中的每一列, 算法先对生成这些图像的潜空间向量取平均值, 再对具有中间值的结果进行向量运算, 这样就创建了右侧 3 × 3 阵列中正中的那个新向量. 然后向这个向量中添加噪声, 产生周围的 8 个样本图像. 最下方一行的 4 幅图像则展示了直接在数据空间中应用相同的运算会导致不对齐, 从而产生模糊的图像. 可以看到性别或 “戴眼镜与否” 等语义属性对应于潜空间中特定的方向.

特定属性的新图像。图 17.10 是一个关于在面部图像上训
则或 “戴眼镜与否” 等语义属性对应于潜空间中特定的方

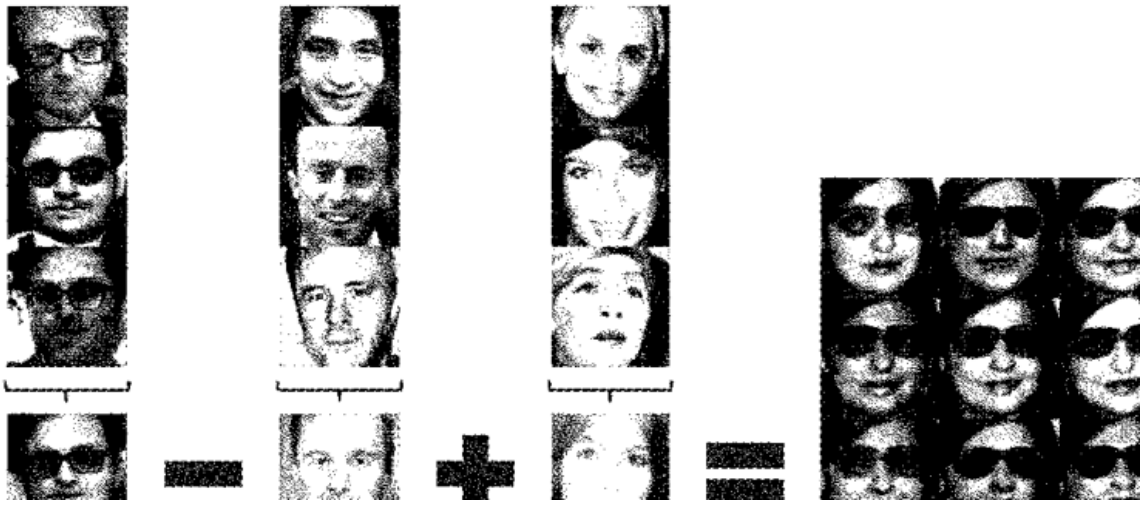


图 17.10: 在训练成功的 GAN 的潜空间中进行向量运算的例子: 戴眼镜的女士 − 不戴眼镜的女士 + 不戴眼镜的男士 ≈ 戴眼镜的男士 (Radford, Metz, and Chintala, 2015, 经授权使用).

17.2.3 理论性质

GAN 损失函数 [式 (17.6)] 可以重写为如下形式:

$$E(p_g, d) = -\int p_{\text{data}}(x) \ln d(x)\, \mathrm{d}x - \int p_g(x) \ln(1 - d(x))\, \mathrm{d}x \tag{17.14}$$

其中 $p_{\text{data}}(x)$ 是真实数据点的固定分布. 考虑对所有函数 $d(x)$ 进行变分优化 (见附录 B), 证明对于固定的生成器网络, 使 E 最小化的最优判别器函数 $d(x)$ 的解由下式给出:

$$d^*(x) = \frac{p_{\text{data}}(x)}{p_{\text{data}}(x) + p_g(x)} \tag{17.15}$$

进一步证明损失函数 E 可以作为生成器网络 $p_g(x)$ 的函数, 写成以下形式:

$$C(p_g) = -\ln 4 + 2\,\mathrm{JSD}\left(p_{\text{data}} \,\|\, p_g\right) \tag{17.16}$$

其中 Jensen–Shannon 散度可以用两个 Kullback–Leibler 散度的总和表示 (参见 2.5.5 小节). 根据只有在 $p(x) = q(x)$ 对所有 x 都成立时 $\mathrm{KL}(p\|q)$ 才等于零这一性质, 可以证明 $C(p_g)$ 的最小值发生在 $p_g(x) = p_{\text{data}}(x)$ 时. 和 Kullback–Leibler 散度一样,Jensen–Shannon 散度也是一个非负的量, 当且仅当两个分布相等时它会消失, 但不同于 KL 散度, 它对于两个分布是对称的.

第 18 章 标准化流

18.1 耦合流

本章将讨论训练非线性潜变量模型的 4 种方法中的第 2 种方法 (参见 16.4.4 小节), 它通过限制神经网络模型的形式, 在确保能够从训练好的模型中易于采样的前提下, 无须近似即可计算似然函数. 假设我们定义了一个分布 $p_z(z)$(有时也称**基础分布** (base distribution)), 这个分布定义在潜变量 z 上, 以及一个由深度神经网络给出的非线性函数 $x = f(z, w)$, 该函数要将潜空间转换成数据空间. 假设 $p_z(z)$ 是一个简单的分布, 比如高斯分布, 那么从这样的模型中抽样就很容易, 因为每一个潜样本 $z \sim p_z(z)$ 只需要通过神经网络就可以生成对应的数据样本 $x = f(z, w)$.

为了计算这个模型的似然函数, 我们需要知道数据空间分布, 而它依赖于神经网络函数的逆函数, 记为 $z = g(x, w)$, 并且满足 $z = g(f(z, w), w)$. 这要求对于每一个 w 值, 函数 $f(z, w)$ 和 $g(x, w)$ 都是可逆的, 也称为**双射** (bijective), 即每个 x 值都对应唯一的 z 值, 反之亦然. 然后我们可以使用变量变换公式来计算数据密度 (参见 2.4 节):

$$p_x(x \mid w) = p_z\big(g(x, w)\big)\,|\det J(x)| \tag{18.1}$$

其中 $J(x)$ 是偏导数的雅可比矩阵, 其元素由 $J_{ij} = \partial z_i / \partial x_j$ 给出. 式 (18.1) 中的 $|\cdot|$ 表示模或绝对值. 尽管确定性映射意味着任何给定的数据值 x 都对应一个唯一的 z 值,z 值因此不再不确定, 但我们仍继续将 z 称为 “潜变量”.

考虑一个训练集 $\mathcal{D} = \{x_1, \cdots, x_N\}$, 它由独立数据点组成, 根据式 (18.1) 得到的对数似然函数为

$$\ln p(\mathcal{D} \mid w) = \sum_{n=1}^{N} \ln p_z\big(g(x_n, w)\big) + \sum_{n=1}^{N} \ln |\det J(x_n)| \tag{18.2}$$

要求可逆映射的一个结果是潜空间的维度必须与数据空间的维度相同, 这可能导致像图像这样的高维数据会需要很大的模型. 另外, 评估 $D \times D$ 矩阵的行列式的成本通常是 $O(D^3)$, 所以我们需要对模型施加一些额外的限制, 以便更高效地计算雅可比行列式.

我们的目标是使用似然函数来训练神经网络. 为了能够建模各种各样的分布, 我们希望转换函数 $x = f(z, w)$ 高度灵活, 因而使用了深度神经网络架构. 如果能够保证网络的每一层都是可逆的, 那么整个函数就是可逆的 (见习题 18.2). 考虑三个连续的转换, 其中的每个转换对应于一个层, 形式如下:

$$x = f_1(f_2(f_3(z))) \tag{18.3}$$

逆函数为

$$z = g_3(g_2(g_1(x))) \tag{18.4}$$

其中 g_1, g_2, g_3 分别是 f_1, f_2, f_3 的逆函数. 此外, 可以使用微积分链式规则计算每个单独层的雅可比行列式, 这种分层结构的雅可比行列式也很容易得到. 式 (18.4) 的右侧为三个矩阵的乘积, 而乘积的行列式是行列式的乘积. 因此, 整体雅可比矩阵的对数行列式将是每层对应对数行列式之和 (参见附录 A).

这种对灵活分布进行建模的方法称为**标准化流** (normalizing flow), 因为通过一系列映射来转换概率分布的过程在某种程度上类似于流体的流动. 此外, 逆映射的效果是将复杂的数据分布转换成标准化形式, 通常是高斯分布. 我们将讨论实践中经常使用的两个主要类型的标准化流的核心概念: 耦合流 (coupling flow) 和自回归流 (autoregressive flow). 此外, 我们还将介绍使用神经微分方程定义可逆映射的方法, 并引入连续流 (continuous flow) 这一概念.

18.1.1 实值 NVP

我们的目标是设计一个单一的可逆函数层, 这样就可以将许多这样的层组合在一起, 从而定义一类高度灵活的可逆函数. 首先考虑以下形式的线性变换:

$$x = az + b \tag{18.5}$$

这个变换很容易求逆, 得到

$$z = (x - b)/a \tag{18.6}$$

然而, 线性变换在组合下是封闭的, 这意味着一系列线性变换等同于一个整体的线性变换 (见习题 3.6). 因为, 高斯分布的线性变换结果还是高斯分布. 所以即使我们有许多这样的线性变换 “层”, 我们也只能得到一个高斯分布. 问题在于我们能否保持线性变换的可逆性, 同时允许额外的灵活性, 从而使得最终结果可以是非高斯分布.

解决这个问题的一种方法是使用一个叫作**实值 NVP**(real-valued non-volume-preserving, RealNVP)(Dinh, Krueger, and Bengio, 2014; Dinh, Sohl-Dickstein, and Bengio, 2016) 的标准化流模型. 这里的 NVP 代表非体积保持 (non-volume-preserving). 其思想是将潜变量向量 z 划分为两部分,$z = (z_A, z_B)$, 如果 z 的维度为 D 且 z_A 的维度为 d, 则 z_B 的维度为 $D - d$. 我们可以类似地划分输出向量,$x = (x_A, x_B)$, 其中 x_A 的维度为 d,x_B 的维度为 $D - d$. 对于输出向量的第一部分, 我们简单地复制输入

$$x_A = z_A \tag{18.7}$$

输出向量的第二部分经历了一个线性变换, 但这个线性变换中的系数由 z_A 的非线性函数给出:

$$x_B = \exp\big(s(z_A, w)\big) \odot z_B + b(z_A, w) \tag{18.8}$$

其中 $s(z_A, w)$ 和 $b(z_A, w)$ 是神经网络的实值输出, 指数函数确保了乘法项是非负的. 这里的 $\odot$ 表示哈达玛积 (Hadamard product), 即对两个向量进行逐元素相乘. 在式 (18.8) 中, 指数也是逐元素计算的.

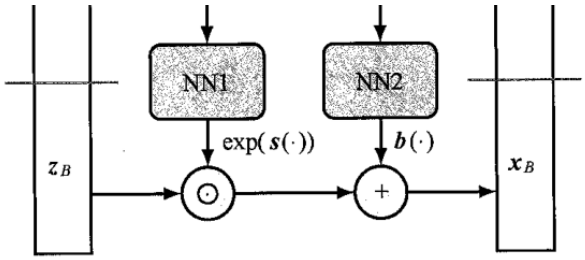
由于使用了神经网络函数,x_B 的值可以是 z_A 的一个非常灵活的函数. 尽管如此, 整体的变换仍然很容易逆推. 给定 $x = (x_A, x_B)$ 的值, 首先计算

$$z_A = x_A \tag{18.9}$$

然后计算 $s(z_A, w)$ 和 $b(z_A, w)$, 最后使用下式计算 z_B:

$$z_B = \exp\big(-s(z_A, w)\big) \odot \big(x_B - b(z_A, w)\big) \tag{18.10}$$

整体变换如图 18.1 所示. 注意, 单个神经网络函数, 如 $s(z_A, w)$ 和 $b(z_A, w)$, 并不要求是可逆的.



:模型的单层结构。在这里，网络 NN1 计算函数 $\exp\big(s(z_A, w)\big)$，而网络 NN2 计算函

18.10）和式（18.11）定义

图 18.1: 实值 NVP 标准化流模型的单层结构. 在这里, 网络 NN1 计算函数 $\exp\{s(z_A, w)\}$, 而网络 NN2 计算函数 $b(z_A, w)$.

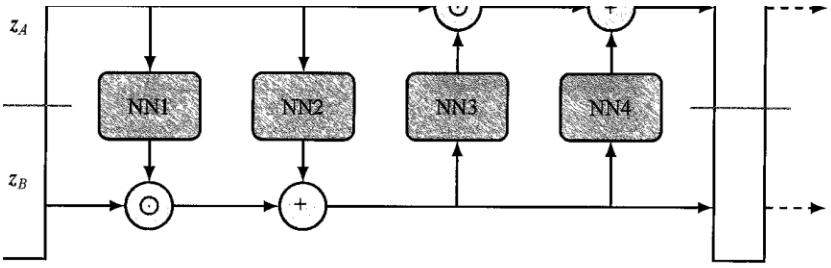
考虑对式 (18.1) 定义的雅可比矩阵及其行列式进行计算. 可以将该雅可比矩阵分成块, 对应于 z 和 x 的划分, 得到

$$J = \begin{pmatrix} \dfrac{\partial z_A}{\partial x_A} & \dfrac{\partial z_A}{\partial x_B} \\ \dfrac{\partial z_B}{\partial x_A} & \dfrac{\partial z_B}{\partial x_B} \end{pmatrix} \tag{18.11}$$

其中, 左上角的块对应于 z_A 关于 x_A 的导数, 根据式 (18.9), 结果为 $d \times d$ 的单位矩阵. 右上角的块对应于 z_A 关于 x_B 的导数, 根据式 (18.9), 结果为 0. 左下角的块对应于 z_B 关于 x_A 的导数, 根据式 (18.10), 这是关于神经网络函数的复杂表达式. 最后, 右下角的块对应于 z_B 关于 x_B 的导数, 根据式 (18.10), 这些导数由一个对角矩阵表示, 其对

角元素是 $\exp(-s(z_A, w))$. 因此我们看到该雅可比矩阵是一个下三角块矩阵, 其行列式只是沿主对角线元素的乘积, 而不依赖于左下角的块中的复杂表达式.

因此, 雅可比行列式可以简单地由 $\exp(s(z_A, w))$ 中元素的乘积给出. 这种方法的一个明显局限是,z_A 的值没有因变换而改变. 这可以通过添加另一层来轻松解决, 其中 z_A 和 z_B 的角色被颠倒过来, 如图 18.2 所示. 这种双层结构可以重复多次, 从而能构建一类非常灵活的生成式模型.



含两个图 18.1 所示的层，我们得到了一个更加灵活但仍然可逆的非线性层。其中的每个子层都是
有容易计算的雅可比矩阵，因此整个双层结构也具有相同的特性

图 18.2: 通过组合两个图 18.1 所示的层, 我们得到了一个更加灵活但仍然可逆的非线性层. 其中的每个子块都是可逆的, 并且拥有容易计算的雅可比矩阵, 因此整个双层结构也具有相同的特性.

实值 NVP 模型属于一类使用广泛的标准化流, 称为**耦合流** (coupling flows). 在耦合流中, 线性变换 [式 (18.8)] 被如下更一般的形式替换:

$$x_B = h\big(z_B, g(z_A, w)\big) \tag{18.12}$$

其中 $h(z_B, g)$ 是 z_B 的函数, 对于任何给定的 g 值, 它都可以高效地逆推, 称为**耦合函数** (coupling function).$g(z_A, w)$ 称为**条件函数** (conditioner), 通常用神经网络来表示.

整个训练过程包括创建数据点的一些小批量, 在这些小批量中, 每个数据点对对数似然函数的贡献可从式 (18.2) 中获得. 对于形式为 $\mathcal{N}(z \mid 0, I)$ 的潜分布, 对数密度就是 $-\|z\|^2/2$ 加一个加法常数. 逆变换 $z = g(x)$ 可以使用一系列形式为式 (18.10) 的逆变换计算得出. 同样, 雅可比行列式的对数由每层的对数行列式之和给出, 其中的每一项本身就是形式为 $-s_i(x, w)$ 的项之和.

我们可以使用一个简单的数据集来说明实值 NVP 标准化流, 这个数据集有时称为 “双月” 数据集, 如图 18.3 所示. 在这里, 一个二维高斯分布通过使用两个连续的层被转换成一个更复杂的分布, 其中每一层都对输入数据的两个维度交替进行耦合变换.

18.2 自回归流

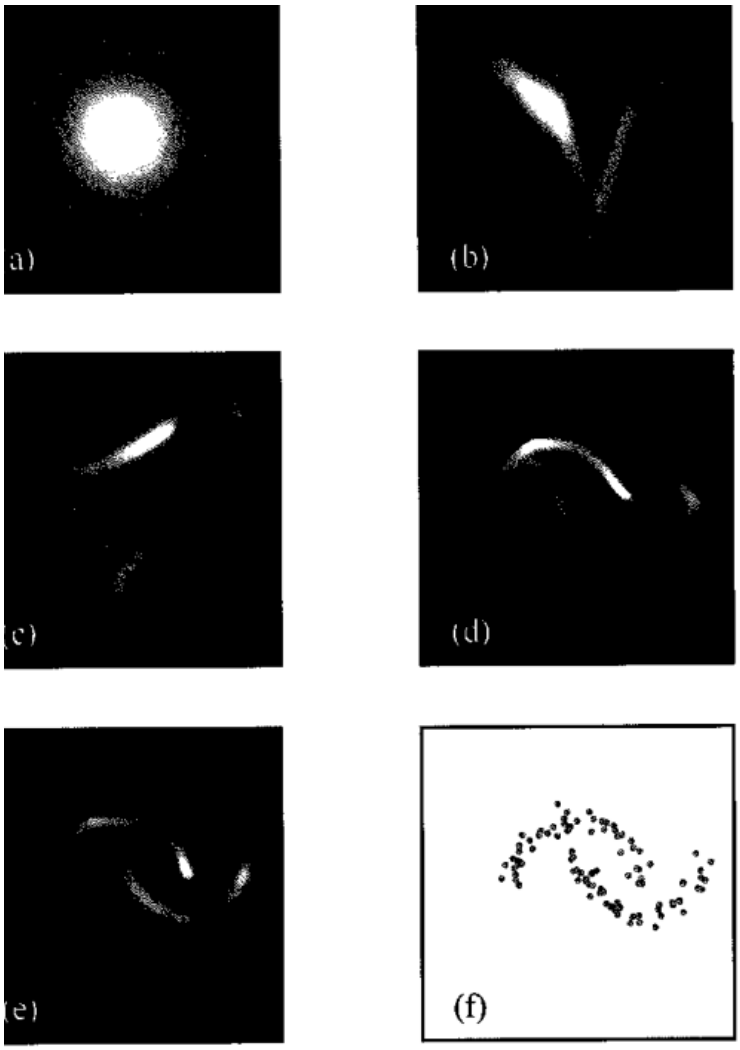
我们注意到一组变量的联合分布总是可以写成条件分布的乘积, 每个变量对应一个条件分布, 由此可以推导出相关的标准化流公式 (参见 11.1 节). 首先选择向量 x 中变量的顺序, 我们可以在不失一般性的情况下写出:

$$p(x) = \prod_{i=1}^{D} p(x_i \mid x_{1:i-1}) \tag{18.13}$$

其中 $x_{1:i-1}$ 表示 $x_1, \cdots, x_{i-1}$. 这种因式分解可以用来构建一类称为**掩码自回归流** (Masked Autoregressive Flow, MAF)(Papamakarios, Pavlakou, and Murray, 2017) 的标准化流, 如下所示:

$$x_i = h_i\big(z_i, g_i(x_{1:i-1}, w_i)\big) \tag{18.14}$$

图 18.4(a) 说明了这一点. 这里的 $h_i(z_i, g)$ 是耦合函数, 因为可以很容易地对 z_i 求逆. 另外,g_i 是条件函数, 通常用一个深度神经网络来表示. 术语 “掩码” 指的是使用单个神经网络来实现一组形式为式 (18.14) 的方程, 连同一个二进制掩码 (Germain et al., 2015) 来强制网络权重的一个子集为零, 以实现自回归约束 [式 (18.13)].



型中，通过交替地对不同维度应用变换，可以将基础的高斯分布逐步转换
分布：初始的二维高斯分布通常均值为零且协方差矩阵为单位矩阵，表示
的分布：第一层变换只对垂直轴应用耦合函数和条件函数，并保持水平轴不

图 18.3: 在实值 NVP 标准化流模型中, 通过交替地对不同维度应用变换, 可以将基础的高斯分布逐步转换为更复杂的数据分布:(a) 基础的高斯分布;(b) 垂直轴变换后的分布;(c) 水平轴变换后的分布;(d) 第二次垂直轴变换后的分布;(e) 第二次水平轴变换后的分布; 最终得到与训练集相匹配的复杂形状, 即所谓的 “双月” 数据集.

在这种情况下, 计算似然函数所需的逆运算如下:

$$z_i = h_i^{-1}\left(x_i, g_i(x_{1:i-1}, w_i)\right) \tag{18.15}$$

由于式 (18.15) 中 $z_1, \cdots, z_D$ 这些分项函数可以并行计算, 因此评估以上似然函数的操作在现代硬件上就能高效执行. 对应于一组变换 [式 (18.15)] 的雅可比矩阵的元素为 $\partial z_i / \partial x_i$, 这些元素形成一个下三角矩阵, 其行列式由对角线元素的乘积给出, 因此也可以高效计算 (见习题 18.4).

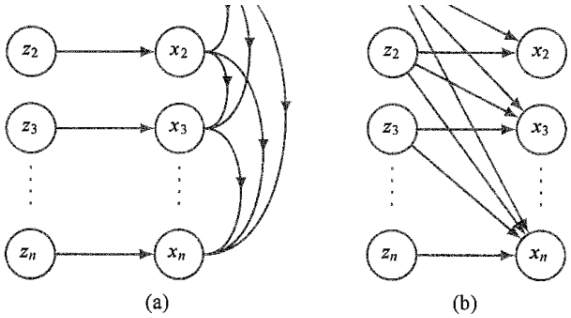
然而, 从该模型中抽样的任务必须通过计算式 (18.14) 来完成, 这本质上是一个顺序执行的过程, 因为在计算 x_i 之前必须先计算 $x_1, \cdots, x_{i-1}$ 的值, 所以速度较慢. 为了避免这种效率低下的抽样, 我们可以定义**逆自回归流** (Inverse Autoregressive Flow, IAF)(Kingma et al., 2016), 如下所示:

$$x_i = h_i\left(z_i, g_i(z_{1:i-1}, w_i)\right) \tag{18.16}$$

如图 18.4(b) 所示, 对于给定的 z, 可以使用式 (18.16) 并行计算元素 $x_1, \cdots, x_D$, 因此采样效率提高了. 然而, 计算似然所需的逆函数需要一系列如下形式的计算:

$$z_i = h_i^{-1}\left(x_i, g_i(z_{1:i-1}, w_i)\right) \tag{18.17}$$

这些计算本质上是顺序进行的, 因此速度很慢. 具体应该使用掩码自回归流还是逆自回归流取决于特定的应用.



化流的结构示意图。(a) 中显示的掩码自回归流允许对似然函数进行有效评估，而 (b) 中
牛进行高效抽样

是顺序进行的，因此速度很慢。具体应该使用掩码自回归流还是

图 18.4: 两种自回归标准化流的结构示意图.(a) 中显示的掩码自回归流允许对似然函数进行有效评估; 而 (b) 中显示的逆自回归流允许进行高效抽样.

耦合流和自回归流是密切相关的. 尽管自回归流引入了相当大的灵活性, 但由于需要顺序地依次采样, 这带来了随数据空间维数 D 线性增长的计算成本. 耦合流可以视为自回归流的一个特例, 它将变量分成两组而不是 D 组, 提升了效率, 但牺牲了一些通用性.

18.3 连续流

本章将要探讨的最后一种标准化流采用了常微分方程 (Ordinary Differential Equation, ODE) 定义的深度神经网络, 它可以看成具有无限多层数的深层网络. 我们首先介绍神经常微分方程 (简称神经 ODE) 的概念, 然后将其应用于标准化流模型的构建.

神经网络包含的处理层越多, 神经网络就越有用, 我们不妨探究一下, 如果层数无限多会发生什么. 考虑一个残差网络, 其中每一层生成的输出由输入向量加上该输入向量的某个参数化非线性函数给出:

$$z(t+1) = z(t) + f\left(z(t), w\right) \tag{18.18}$$

其中 $t = 1, \cdots, T$ 标记了网络中的层. 注意, 我们在每一层都使用了相同的函数, 并且共享参数向量 w, 因为这允许我们在参数数量有限的情况下考虑任意多的层. 想象一下, 增加层数, 同时确保在每一层引入的变化相应地变得更小. 在极限情况下, 隐藏单元激活向量变成了连续变量 t 的函数 $z(t)$, 我们可以将这个向量通过网络的演化表示为

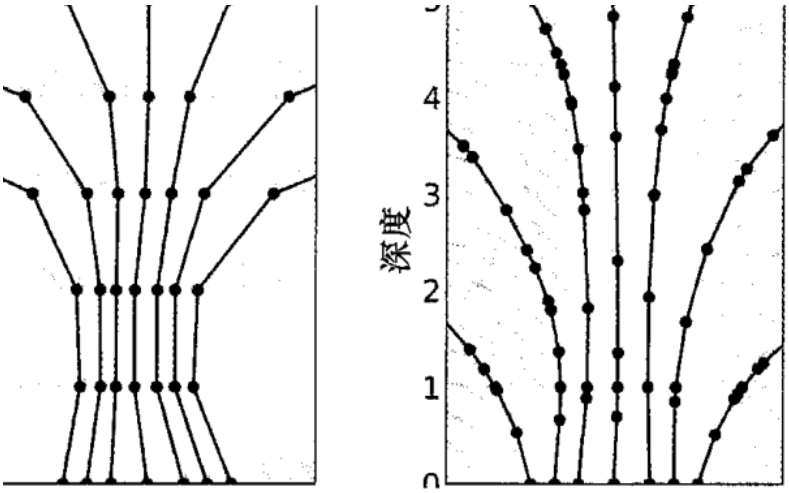
一个微分方程 (见习题 18.5):

$$\frac{\mathrm{d}z(t)}{\mathrm{d}t} = f\big(z(t), w\big) \tag{18.19}$$

其中 t 通常称为 “时间”. 式 (18.19) 所示的方程就称为**神经 ODE**(Chen et al., 2018). 常微分方程的 “常” 指的是只有单一变量 t. 如果将网络的输入表示为向量 $z(0)$, 则输出 $z(T)$ 可以通过积分微分方程来获得:

$$z(T) = z(0) + \int_0^T f\big(z(t), w\big)\,\mathrm{d}t \tag{18.20}$$

这个积分可以使用标准数值积分软件工具来计算. 求解微分方程最简单的方法是欧拉前向积分法 (forward integration), 对应于式 (18.18). 在实际操作中, 更强大的数值积分算法可以通过调整其函数计算来实现目标. 特别是, 它们可以自适应地选择通常为非均匀分布的 t 的值. 这种计算次数取代了传统分层网络中 “深度” 的概念. 传统分层网络与神经 ODE 的比较如图 18.5 所示.



ODE 的比较。左图对应一个有 5 层的残差网络，并展示了单标量输入的几个起
神经 ODE 的数值积分结果，同样是对单标量输入的几个起始值，我们看到函
值的。求值点由数值求解器自适应选择，并且取决于输入值的选择「经 Chen

图 18.5: 传统分层网络与神经 ODE 的比较. 左图对应一个有 5 层的残差网络, 展示了单标量输入的几个起始值的轨迹; 右图显示了连续神经 ODE 的数值积分结果, 同样是对单标量输入的几个起始值, 求值点由数值求解器自适应选择, 并且取决于输入值的选择 (Chen et al., 2018, 经授权使用).

18.3.1 神经 ODE 的反向传播

我们现在需要解决如何训练神经 ODE 的问题, 即如何通过优化损失函数来确定 w 的值. 给定一个数据集, 其中包括输入向量 $z(0)$ 的值以及相关的输出目标向量, 以及一个依赖于输出向量 $z(T)$ 的损失函数 $L(\cdot)$. 一种方法是使用自动微分来微分 ODE 求解器在前向传播期间执行的所有操作 (参见 8.2 节). 尽管这样做很直接, 但从内存角度来看却代价高昂, 并且在控制数值误差方面也不是最佳做法. 另一种方法 (Chen et al., 2018) 将 ODE 求解器视为一个黑箱, 并使用了一种称为**伴随灵敏度方法** (adjoint sensitivity method) 的技术, 这可以视为显式反向传播的连续类比.

回想一下, 反向传播涉及对每个数据点执行三个连续操作步骤 (参见第 8 章): 前向传播, 计算网络中每一层的激活向量; 反向传播, 从输出层开始利用微积分的链式法则计算损失函数对每一层激活的导数, 并向后传播; 参数梯度计算, 通过将前向传播的激活值和反向传播的梯度相乘来计算损失函数对网络参数的导数.

为了将反向传播应用于神经 ODE, 定义一个称为**伴随** (adjoint) 的量 (见习题 18.6):

$$a(t) = \frac{\partial L}{\partial z(t)} \tag{18.21}$$

$a(T)$ 对应于损失函数关于输出向量的常规导数.$a(t)$ 满足如下微分方程:

$$\frac{\mathrm{d}a(t)}{\mathrm{d}t} = -a(t)^{\mathsf{T}} \nabla_z f\big(z(t), w\big) \tag{18.22}$$

这是微积分链式法则的连续版本. 这可以通过从 $a(T)$ 开始向后积分来求解, 并且在计算神经 ODE 的梯度时可以执行类似的步骤.

反向传播的第三步通过计算激活值和梯度的适当乘积来计算损失函数对网络参数的导数. 当一个网络中的多个连接共享一个参数值时, 总导数由每个连接的导数相加而成 (见习题 9.7). 对于神经 ODE, 其中相同的参数向量 w 在整个网络中共享, 求和变成了对 t 的积分, 形式如下 (见习题 18.7):

$$\nabla_w L = -\int_0^T a(t)^{\mathsf{T}} \nabla_w f\big(z(t), w\big)\, \mathrm{d}t \tag{18.23}$$

式 (18.22) 中的 ∇_z 和式 (18.23) 中的 ∇_w 可以使用自动微分高效地求值 (见 8.2 节). 注意, 上述结果同样适用于更普遍的神经网络函数 $f(z(t), t, w)$, 它除了通过 $z(t)$ 的隐式依赖之外, 还显式地依赖于 t.

与传统分层网络相比, 使用伴随方法训练的神经 ODE 的一个好处是, 不需要存储前向传播的中间结果, 因此内存成本是恒定的. 此外, 神经 ODE 可以自然地处理连续时间数据, 其中观测可在任意时间发生. 如果损失函数 L 依赖于 $z(T)$ 的其他值而不仅仅是输出值, 则需要进行多次逆模型求解, 对每一对连续的输出进行一次逆模型求解, 以便单个解决方案可以分解为多个连续解决方案以访问中间状态. 请注意, 在训练阶段可以使用高精度的求解器; 而在推理阶段, 特别是在计算资源有限的应用场景中, 可以采用较低的精度, 从而减少所需的函数评估次数.

18.3.2 神经 ODE 流

我们可以利用神经 ODE 来定义一个旨在构建易处理的标准化流模型的替代方法. 神经 ODE 定义了一个从输入向量 $z(0)$ 到输出向量 $z(T)$ 的高度灵活的转换, 其形式为如下微分方程:

$$\frac{\mathrm{d}z(t)}{\mathrm{d}t} = f\big(z(t), w\big) \tag{18.24}$$

如果我们定义一个基于输入向量的基础分布 $p(z(0))$, 神经 ODE 就会随时间向前传播, 从而得到每个 t 值的分布 $p(z(t))$, 进而得到输出向量的分布 $p(z(T))$. 有研究 (Chen et al., 2018) 指出, 对于神经 ODE 来说, 密度转换可以通过积分如下微分方程来计算 (见习题 18.8):

$$\frac{\partial \log p\big(z(t)\big)}{\partial t} = -\operatorname{Tr}\left(\frac{\partial f\big(z(t), w\big)}{\partial z(t)}\right) \tag{18.25}$$

其中 $\partial f / \partial z(t)$ 表示雅可比矩阵, 其元素为 $\partial f_i / \partial z_j$. 这个积分操作可以使用标准 ODE 求解器来完成. 同样, 可以通过从基础分布 $p(z(0))$ 中采样来获得这个密度的样本, 这个基础分布可以是一个简单的分布, 如高斯分布, 并支持使用 ODE 求解器再次积分 [式 (18.24)], 以便将值传播到输出. 得到的框架称为**连续标准化流** (continuous normalizing flow), 如图 18.6 所示.

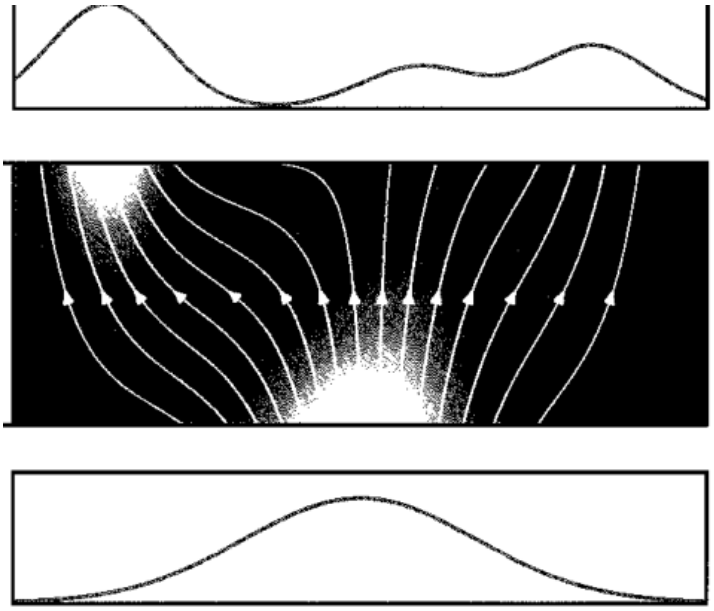
连续标准化流可以使用适用于神经 ODE 的伴随灵敏度方法进行训练, 该方法可以看作反向传播的连续时间等价物 (参见 18.3.1 小节).

由于式 (18.25) 涉及雅可比矩阵的**迹**而不是行列式 (行列式出现在离散标准化流中), 它可能看起来更具计算效率. 一般来说, 计算一个 $D \times D$ 矩阵的行列式需要 $O(D^3)$ 次运算, 计算该矩阵的迹则需要 $O(D^2)$ 次运算. 然而, 如果行列式是下三角的, 正如许多标准化流所呈现的那样, 则行列式是对角项的乘积, 因此也需要 $O(D)$ 次运算. 由于计算雅可比矩阵的各个元素需要进行单独的前向传播, 这本身需要 $O(D)$ 次运算, 因此计算矩阵的迹或行列式 (对于一个下三角矩阵来说) 总共需要 $O(D^2)$ 的成本.

然而, 通过使用 Hutchinson 的迹估计器 (Grathwohl et al., 2018), 我们可以将计算迹的成本降低到 $O(D)$. 对于矩阵 A, 迹的形式为

$$\operatorname{Tr}(A) = \mathbb{E}\big[\epsilon^{\mathsf{T}} A \epsilon\big] \tag{18.26}$$

其中 ϵ 是一个零均值、单位协方差的随机向量, 例如高斯分布 $\mathcal{N}(0, I)$. 对于一个特定的 ϵ, 矩阵–向量乘积 $A\epsilon$ 可以



里展示了 $t=0$ 时刻的一个简单高斯分布是如何通过连续转换变成 $t=T$
密度降低；当流线靠拢时，密度增加

可比矩阵的迹而不是行列式（行列式出现在离散标准化

图 18.6: 连续标准化流的示意图. 这里展示了 $t = 0$ 时刻的一个简单高斯分布是如何通过连续转换变成 $t = T$ 时刻的一个多峰分布的. 当流线分开时, 密度降低; 当流线靠拢时, 密度增加.

使用反向的自动微分在单次传递中高效计算. 然后我们可以使用有限数量的样本来近似矩阵的迹, 形式如下:

$$\text{Tr}(A) \approx \frac{1}{M}\sum_{m=1}^{M} \epsilon_m^{\mathsf{T}} A \epsilon_m \tag{18.27}$$

实践中, 我们可以设置 $M = 1$ 并只使用一个样本对每个新的数据点进行求值. 虽然这是一个有噪声的估计, 但由于它是噪声随机梯度下降过程的一部分, 这可能显得不太重要. 重要的是, 它是无偏的, 这意味着估计器的期望等于真实值 (参见习题 18.11).

通过使用一种名为**流匹配** (flow matching) 的技术 (参见第 20 章), 可以显著提高连续标准化流的训练效率 (Lipman et al., 2022). 这使得标准化流更接近于扩散模型, 避免了在积分器中进行反向传播, 同时大幅减少内存需求, 实现了更快的推理和更稳定的训练.

第 19 章 自编码器

19.1 确定性的自编码器

深度学习的一个核心目标是发现数据的表示方法, 为后续应用提供价值. 自联想神经网络 (auto-associative neural network) 或**自编码器** (autoencoder) 是获得广泛认可的学习数据内部表示的方法. 其中神经网络的输入单元和输出单元的数量相同, 训练目的是让输出 y 尽可能接近输入 x. 一旦训练完成, 神经网络中的一个内部层就为每个新的输入提供一个表示 $z(x)$. 这样的网络可以划分成两部分: 首先是编码器, 它将输入 x 映射到一个隐式表示 $z(x)$; 其次是解码器, 它将隐式表示映射到输出 $y(z)$.

一个网络把自己的输入复制到输出, 这就是一个很一般的自编码器, 不解决任何实际问题. 如果自编码器要找到非平凡解, 就需要引入某种形式的约束. 例如, 这种约束可以要求 z 的维度小于 x 的维度, 或者要求 z 具有稀疏表示. 我们也可以改变训练的方式, 迫使网络去探索非平凡解. 例如, 训练网络学会纠正输入向量的错误, 比如清除加入的噪声或者填补遗漏的信息. 这样的限制性条件能激励神经网络挖掘数据内部有价值的结构, 从而获得更好的训练效果.

我们可以利用神经网络的非线性特点来定义一种非线性 PCA, 其中的潜在流形不再是数据空间的线性子空间. 这是通过使用输出数量与输入数量相同的网络, 并针对每一组训练数据, 通过优化权重以最小化输入与输出之间的某种重建误差来实现的. 这种模型通过对输入向量进行线性变换, 将其映射到一个低维流形上, 然后通过再次线性变换, 将其近似地重构回原始数据空间中 (参见 16.1 节).

简单的自编码器在现代深度学习中很少直接使用, 因为它们在潜空间中既不能提供语义上有意义的表示, 也不能直接从数据分布中生成新的样本. 然而, 它们为一些更强大的深度生成式模型 (如变分自编码器) 提供了重要的概念基础 (参见 19.2 节).

19.1.1 线性自编码器

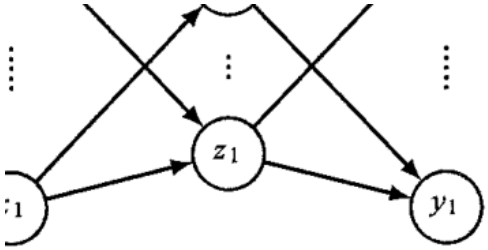
首先考虑图 19.1 所示形式的多层感知机, 它有 D 个输入单元、D 个输出单元和 M 个隐藏单元, 其中 $M < D$. 用来训练网络试图将每个输入向量映射到其自身. 我们称这样的网络形成了一个**自联想** (auto-associative) 映射. 由于隐藏单元的数量少于输入单元的数量, 它通常无法完美地重构所有输入向量; 因此, 我们先定义一个误差函数, 用来衡量输入向量及其重建之间的不匹配程度. 然后通过最小化这个误差函数, 来确定网络参数 w. 在这里, 我们使用以下形式的平方和误差函数:

$$E(w) = \frac{1}{2}\sum_{n=1}^{N} \|x_n - \widehat{x}_n\|^2 \tag{19.1}$$

其中 $\widehat{x}_n$ 是由网络产生的对输入 x_n 的重建.

如果隐藏单元具有线性激活函数, 则可以证明误差函数具有唯一的全局最小值, 并且在这个全局最小值处, 网络执行一个投影操作把数据映射到由数据的前 M 个主成分所张成 (span) 的 M 维子空间上 (Bourlard and Kamp, 1988; Baldi and Hornik, 1989). 因此, 图 19.1 中通向隐藏单元的权重向量形成了一个张成主子空间的基集. 请注意, 这些向量不必是正交或归一化的. 这个结果并不在意料之外, 因为 PCA 和神经网络都依赖于线性降维以及最小化相同的平方和误差函数. 即使隐藏层中有非线性单元, 这样的网络也相当于线性主成分分析.

有人可能会觉得, 对图 19.1 所示网络的隐藏单元使用非线性激活函数, 可以克服线性流形的局限性. 然而, 即使有非线性隐藏单元, 最小误差解也仍然会被投影到主成分子空间 (Bourlard and Kamp, 1988). 因此, 使用两层神经网络进行降维没有任何优势. 基于奇异值分解 (SVD) 的标准 PCA 技术可以保证在有限时间内给出正确解, 并且生成一组有序的特征值及对应的正交归一化特征向量.



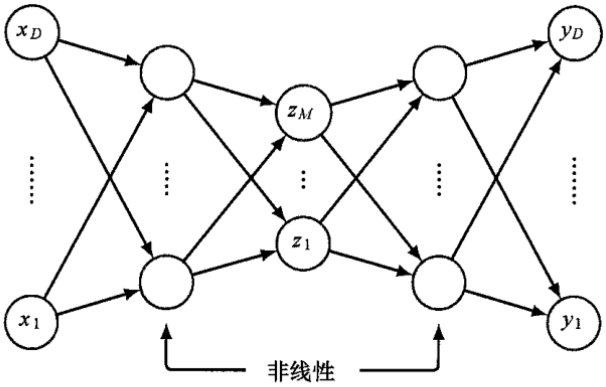
一个具有两层权重的多层感知机。这样
通过最小化平方和误差来训练的，使得
被映射到它们自身。即使隐藏层中有非
，这样的网络也相当于线性主成分分析。
起见，表示偏置参数的连接已省略

这样的网络
associative）映
输入单元的数
有输入向量；
数，用来衡量
配程度。然后
确定网络参数
的平方和误差

图 19.1: 一个具有两层权重的多层感知机. 这样的网络是通过最小化平方和误差来训练的, 使得目标向量就是输入向量本身, 因此该网络试图将每个输入向量映射到其自身. 为了清晰起见, 表示偏置参数的连接已省略.

19.1.2 深度自编码器

然而, 如果在网络中加入额外的非线性层, 情况就有所不同了. 以图 19.2 所示的四层自联想网络为例, 输出单元仍然是线性的, 并且第二层中的 M 个单元也可以是线性的. 然而, 第一层和第三层具有非线性激活函数. 网络再次通过最小化损失函数 [式 (19.1)] 来训练.



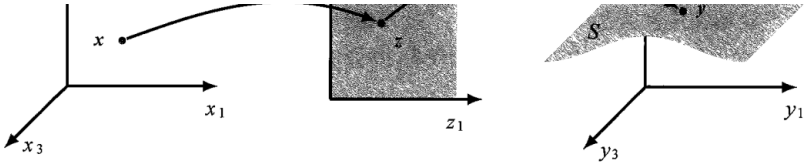
性单元隐藏层会产生一个自联想网络，它可以执行非线性降维



图 19.2: 添加额外的非线性单元隐藏层会产生一个自联想网络, 它可以执行非线性降维.

我们可以将这个网络视为两个连续的函数映射, 如图 19.2 中的 F_1 和 F_2. 第一个映射 F_1 将原始的 D 维数据投影到了由第二层单元的激活定义的 M 维子空间. 因为第一层非线性单元的存在, 这个映射非常通用, 不受限于线性映射. 类似地, 该网络的后半部分定义了从 M 维隐藏空间回到原始 D 维输入空间的任意映射. 这在几何上有一个简单的解释, 参见图 19.3, 其中 $D = 3$ 且 $M = 2$. 映射 F_1 定义了流形 S 在更高维数据空间中的嵌入方式. 由于映射 F_1 可以是非线性的, 因此流形 S 的嵌入可以是非平面的. 映射 F_2 则定义了从原始 D 维数据空间到 M 维潜空间的投影.

这样的网络实际上有效地执行了一种非线性形式的 PCA. 它具有不受限于线性变换的优势, 尽管标准 PCA 是它的一个特例. 然而, 现在训练网络需要进行非线性优化, 因为误差函数 [式 (19.1)] 不再是网络参数的二次函数. 因此必须使用计算密集型的非线性优化技术, 并且存在找到误差函数的次优局部极小值的风险. 此外, 在训练网络之前必须指定子空间的维数.



19.3 图 19.2 所示网络执行的映射的几何解释（具有 $D=3$ 个输入且第二层中有 $M=2$ 个单元的模型）从潜空到数据空间的映射 F_2 定义了流形 S 在更高维数据空间中的嵌入方式。由于映射 F_2 可以是非线性的，因此流形的嵌入可以是非平面的。映射 F_1 则定义了从原始 D 维数据空间到 M 维潜空间的投影

图 19.3: 图 19.2 所示网络执行的映射的几何解释 (具有 $D = 3$ 个输入且第二层中有 $M = 2$ 个单元的模型). 从潜空间到数据空间的映射 F_1 定义了流形 S 在更高维数据空间中的嵌入方式.

19.1.3 稀疏自编码器

与其限制网络中某个隐藏层的节点数量, 不如考虑另一种约束内部表示的方法: 使用正则化器鼓励稀疏表示, 从而降低有效维度. 一个简单的选择是 L_1 正则化器, 它能促进数据的稀疏性, 相应的正则化误差函数如下 (参见 9.2.2 小节):

$$E_{\text{reg}}(w) = E(w) + \frac{\lambda}{2}\sum_j |z_j| \tag{19.2}$$

其中 $E(w)$ 是未正则化的误差函数, 求和操作是在隐藏层中所有单元的激活值上进行的. 注意, 正则化通常应用于网络的参数, 而在这里它被用于单元激活值. 用于梯度下降训练的导数可以像往常一样使用自动微分来计算.

19.1.4 去噪自编码器

我们已经看到了在简单的自编码器中限制潜空间层的维度的重要性, 这样能避免模型直接学习恒等映射. **去噪自编码器** (denoising autoencoder)(Vincent, 2008) 也能迫使模型发现数据内部有趣的结构. 其背后的思想是用噪声破坏每个输入向量 x_n, 得到一个修改后的向量 $\tilde{x}_n$, 然后将其输入自编码器, 产生输出 $\hat{x}(x_n, w)$. 网络可以通过最小化平方和损失函数来训练, 其目的是重建原始的无噪声输入向量:

$$E(w) = \frac{1}{2}\sum_{n=1}^{N} \left\|x_n - \hat{x}(\tilde{x}_n, w)\right\|^2 \tag{19.3}$$

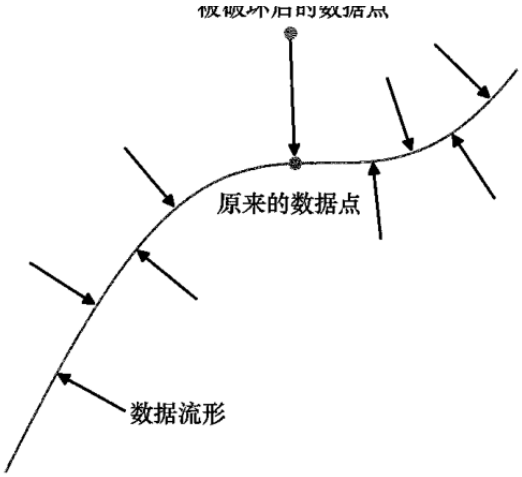
添加噪声的一种方法是随机地把一些输入变量设置为零. 此类输入所占的比例 v 代表噪声水平, 并且范围是 $0 \leq v \leq 1$. 添加噪声的另一种方法是为每个输入变量添加独立的零均值高斯噪声, 其中噪声的规模由高斯分布的方差设定. 通过学习去除输入数据中的噪声, 网络被迫学习数据方方面面的结构. 例如, 如果数据包含图像, 网络只需要学习到相邻像素值之间的强相关性, 就可以纠正被噪声损坏的像素.

更正式地讲, 去噪自编码器的训练与**得分匹配** (score matching)(Vincent, 2011) 相关, 得分由 $s(x) = \nabla_x \ln p(x)$ 定义. 图 19.4 说明了我们对这种关系的一些直观的理解. 自编码器通过学习逆转失真向量 $\tilde{x}_n - x_n$, 学习了数据空间中每个点的一个向量, 该向量指向流形, 从而指向高数据密度区域. 得分向量 $\nabla_x \ln p(x)$ 同样是一个指向高数据密度区域的向量. 等到讨论扩散模型时 (参见 20.3 节), 我们将更深入地探索得分匹配和去噪之间的关系. 我们将看到, 扩散模型也会学习从那些被噪声破坏的输入中去除噪声.

19.1.5 掩蔽自编码器

我们已经看到, 借助随机遮盖输入子集的自监督学习方式, 像 BERT 这样的 Transformer 模型能学习到自然语言的深层次内部表示 (参见 12.2 节). 人们很自然地会问, 类似的方法能应用于自然图像吗? **掩蔽自编码器** (masked autoencoder)(He et al., 2021) 旨在利用深度神经网络, 根据输入图像的损坏版本重建图像, 这与去噪自编码器类似. 在这种情况下, 损坏 (corruption) 的形式是掩蔽或删除输入图像的一部分.

这种技术通常可以与视觉 Transformer 架构结合起来使用 (参见 12.4.1 小节), 因为在这种情况下, 仅仅通过传递随机选定的输入补丁标记 (input patch token) 子集给编码器, 就可以轻松实现对输入图像的部分进行掩蔽.



中，数据点被假定存在于数据空间中的一个低维流形上，并且被加性噪声破坏了。自编码映射回它们原来的值，因此自编码器为数据空间中的每个点学习了一个指向流形的向量

图 19.4: 在去噪自编码器中, 数据点被假定存在于数据空间中的一个低维流形上, 并且被加性噪声破坏了. 自编码器会学习将损坏的数据点映射回它们原来的值, 因此自编码器为数据空间中的每个点学习了一个指向流形的向量.

图 19.5 总结了整个算法.

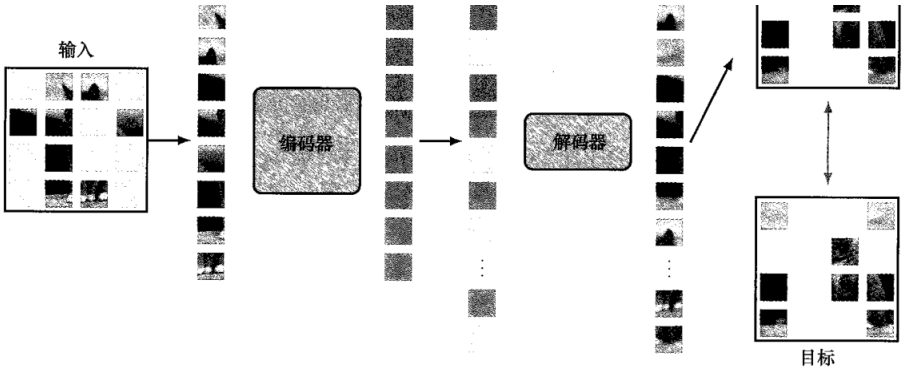


图 19.5 **掩蔽自编码器在训练阶段的架构。注意，目标是输入的补集，因为损失仅应用于被掩蔽的区块。训练结束后，解码器被抛弃，编码器用于将图像映射到内部表示，以便在后续任务中使用**

图 19.5: 掩蔽自编码器在训练阶段的架构. 注意, 目标是输入的补丁, 因为损失仅应用于被掩蔽的区块.

和语言不同, 图像具有更多的冗余, 并伴有强烈的局部相关性. 在一句话中省略一个单词可能会大大增加歧义, 然而随机移除图像中的一小块通常对图像的语义影响很小. 当输入图像有较高掩蔽比例 (通常为 75%) 时, 算法能学到最佳的内部表示; 而 BERT 中的掩蔽比例仅为 15%. 在 BERT 中, 掩蔽的输入被固定的掩蔽符号替换; 而在掩蔽自编码器中, 掩蔽的区块简单地被省略. 考虑到 Transformer 的计算效率随着输入序列长度的增加而下降, 通过省略大量的输入区块, 我们可以节省大量的计算资源, 因此掩蔽自编码器成为预训练大型 Transformer 编码器的优选.

由于解码器层同样是 Transformer, 它需要按照原始图像的维度来运作. 因为 Transformer 的输出与输入具有相同的维度, 我们需要在编码器输出和解码器输入之间恢复图像的维度. 这可以通过重新引入被掩蔽的区块来实现, 每个片段标记都增加了位置编码信息, 并且由固定的掩蔽标记向量表示. 由于解码器表示的维度远高于编码器, 解码器 Transformer 拥有的可学习参数远少于编码器 Transformer. 解码器的输出对接的是一个可学习的线性层, 用于将输出表示映射到像素值空间, 而训练损失函数的作用就是简单地对每个图像缺失区块的均方误差取平均值.

图 19.6 展示了使用训练成功的掩蔽自编码器重建图像的 4 个例子, 展示了训练后的自编码器在生成合理语

义上可信的重构能力. 然而, 我们最终的目标是学习对后续下游任务有用的内部表示, 因此我们舍弃了解码器, 而把编码器应用于没有遮蔽的完整图像, 并配备了一组针对所需应用而微调的新输出层.



使用训练成功的掩蔽自编码器重建图像的 4 个例子。在这些例子中，有 80% 的输入区块被掩蔽。在每个被掩蔽的图像在左侧，重建的图像在中间，原始图像在右侧 [图片来源于 He et al.(2021)，经授权使用]

图 19.6: 使用训练成功的掩蔽自编码器重建图像的 4 个例子. 在这些例子中, 有 80% 输入区块被掩蔽. 在每个例子中, 被掩蔽的图像在左侧, 重建的图像在中间, 原始图像在右侧 (He et al., 2021, 经授权使用).

19.2 变分自编码器

在本章中, 我们将从确定性的自编码器开始学习, 然后推广到学习编码器分布 $p(z \mid x)$ 以及解码器分布 $p(y \mid z)$ 的随机模型. 这些概率模型称为**变分自编码器** (variational autoencoder), 它们代表了我们训练非线性潜变量模型的 4 种方法中的第 3 种 (参见 16.4.4 小节).

考虑一个生成式模型, 其条件分布 $p(x \mid z, w)$ 覆盖了由深度神经网络输出控制的 D 维数据变量 x. 例如,$g(z, w)$ 可能表示高斯条件分布的均值. 同时考虑 M 维潜变量 z 的分布, 它由零均值、单位方差的高斯分布给出:

$$p(z) = \mathcal{N}(z \mid 0, I) \tag{19.4}$$

潜变量模型的似然函数由下式给出:

$$p(x \mid w) = \int p(x \mid z, w)\, p(z)\, \mathrm{d}z \tag{19.5}$$

其中 $p(x \mid z, w)$ 由一个深度神经网络定义. 这个似然函数是难以处理的, 因为关于 z 的积分无法解析计算. 变分自编码器 (Variational AutoEncoder, VAE)(Kingma and Welling, 2013; Rezende, Mohamed, and Wierstra, 2014; Doersch, 2016; Kingma and Welling, 2019) 在训练模型时使用了这个似然函数的近似表示来解决这一难题.VAE 背后的关键思想如下:(1) 使用证据下界来近似似然函数, 从而与 EM 算法形成紧密联系;(2) 借助摊销推断 (即使用编码器网络) 来近似 E 步骤中潜变量上的后验分布, 而不是精确评估每个数据点的后验分布;(3) 使用重参数化 (reparameterization trick) 技巧实现编码器模型的可微分训练 (参见 15.3 节).

为了推导出 VAE 的近似, 首先回想一下, 对于由潜变量 z 描述的空间上的任意概率分布 $q(z)$, 以下关系成立 (参见 15.4 节):

$$\ln p(x \mid w) = \mathcal{L}(w) + \mathrm{KL}\big(q(z) \,\|\, p(z \mid x, w)\big) \tag{19.6}$$

其中 $\mathcal{L}$ 是**证据下界** (evidence lower bound) 或 ELBO, 也称变分下界 (variational lower bound), 由下式给出:

$$\mathcal{L}(w) = \int q(z) \ln \left\{ \frac{p(x \mid z, w)\, p(z)}{q(z)} \right\} \mathrm{d}z \tag{19.7}$$

Kullback–Leibler 散度 (简称 KL 散度)$\mathrm{KL}(q \| p)$ 定义如下:

$$\mathrm{KL}\big(q(z) \,\|\, p(z)\big) = -\int q(z) \ln \left\{ \frac{p(z)}{q(z)} \right\} \mathrm{d}z \tag{19.8}$$

由于 KL 散度满足 $\mathrm{KL}(q \| p) \geq 0$, 可以得到

$$\ln p(x \mid w) \geq \mathcal{L} \tag{19.9}$$

所以 $\mathcal{L}$ 是 $\ln p(x \mid w)$ 的证据下界. 虽然对数似然函数 $\ln p(x \mid w)$ 难以直接计算, 但我们可以使用蒙特卡洛估计方法来评估这个下界, 从而使它近似于真实对数似然函数的值.

考虑从模型分布 $p(x)$ 中独立采样的训练数据集 $\mathcal{D} = \{x_1, \cdots, x_N\}$. 这个数据集的对数似然函数由下式给出:

$$\ln p(\mathcal{D} \mid w) = \sum_{n=1}^{N} \mathbb{E}_{q(z_n \mid x_n, \phi)}\Big[\ln p(x_n \mid z_n, w)\Big] - \sum_{n=1}^{N} \mathrm{KL}\big(q(z_n \mid x_n, \phi) \,\|\, p(z_n)\big) \tag{19.10}$$

19.2.1 摊销推理

请注意, 这引入了与每个数据向量 x_n 相对应的单独潜变量 z_n, 就像我们在混合模型和概率 PCA 模型中看到的那样. 因此, 每个潜变量都有自己的独立分布 $q_n(z_n)$, 且都可以单独优化 (参见 15.2 节和 16.2 节).

对于深度生成式模型, 精确后验分布由贝叶斯定理给出:

$$p(z \mid x, w) = \frac{p(x \mid z, w)\, p(z)}{\int p(x \mid z, w)\, p(z)\, \mathrm{d}z} \tag{19.11}$$

其中分子是可以直接计算的. 然而分母是由似然函数给出的, 难以解析计算. 因此, 我们需要找到一个对后验分布的近似. 原则上, 我们可以为每个分布 $q_n(z_n)$ 单独建立参数化模型, 并进行数值优化, 但计算成本非常高, 特别是对于大数据集来说, 此外还需要在每次更新 w 后重新计算分布. 因此, 我们决定换一个方向, 引入第二个神经网络来探索一个更有效的近似框架.

在变分自编码器中, 我们不再单独为每个数据点 x_n 计算一个后验分布 $p(z \mid x_n, w)$, 而是通过训练一个称为**编码器网络**的神经网络来近似所有这些分布. 这种技术称为**摊销推理** (amortized inference), 要求编码器生成单一的分布 $q(z \mid x, \phi)$, 该分布以 x 为条件, 其中 ϕ 代表网络参数. 由证据下界给出的目标函数现在同时依赖于 ϕ 和 w, 我们可以使用基于梯度的优化方法来联合优化这两组参数.

因此,VAE 包含两个参数独立但能联合训练的神经网络: 一个编码器网络, 用于将数据向量映射到潜空间; 一个解码器网络, 用于将潜空间向量映射回数据空间. 类似于简单的神经网络自编码器模型, 所不同的是, 我们在潜空间中定义了一个概率分布. 我们将看到, 编码器根据贝叶斯定理计算解码器的近似概率逆过程——即根据给定观测数据推断潜变量的分布.

在变分自编码器中, 编码器的一个典型选择是具有对角协方差矩阵的高斯分布, 其均值 μ_n 和方差 σ_n^2 由一个以 x 作为输入的神经网络给出:

$$q(z \mid x_n, \phi) = \mathcal{N}\big(z \mid \mu(x_n, \phi),\, \sigma^2(x_n, \phi)\, I\big) \tag{19.12}$$

请注意, 均值 $\mu(x, \phi)$ 位于区间 $(-\infty, \infty)$, 因此相应的输出单元激活函数可以是线性的; 而方差 $\sigma^2(x, \phi)$ 必须是非负的, 所以相关的输出单元通常使用指数函数 $\exp(\cdot)$ 作为激活函数.

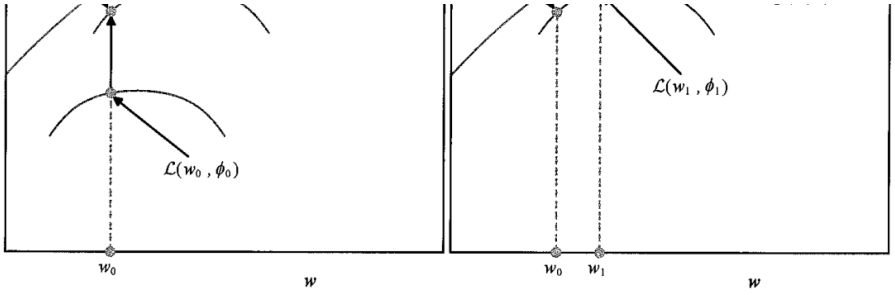
我们的优化目标是使用基于梯度的优化方法, 针对参数 ϕ 和 w 最大化这个下界, 这通常使用基于小批量数据的随机梯度下降法来实现. 尽管是联合优化参数, 但从概念上我们可以交替地优化 ϕ 和 w, 如图 19.7 所示.

ELBO 优化与 EM 算法的一个关键区别是, 对于给定的 w 值, 相对于编码器参数 ϕ 的优化通常不会将 KL 散度降低到零, 因为编码器网络并不能完美地预测后验潜在分布, 所以下界和真实对数似然之间存在残差. 尽管基于深度神经网络的编码器非常灵活, 但它并不能精确地模拟真实的后验分布, 原因如下:(1) 真实的条件后验分布未必服从因子分解的高斯分布;(2) 即使是一个大型神经网络, 也存在灵活性方面的限制;(3) 训练过程只是一个近似的优化.

19.2.2 重参数化技巧

遗憾的是, 到目前为止, 下界 [式 (19.7)] 仍然难以直接计算, 因为在潜变量 z 的积分过程中, 解码器网络会导致被积函数对潜变量存在复杂的依赖关系. 对于数据点 x_n, 其对下界的贡献可以表示为:

$$\mathcal{L}(w, \phi) = \int q(z \mid x_n, \phi) \ln p(x_n \mid z, w)\, \mathrm{d}z - \mathrm{KL}\big(q(z \mid x_n, \phi) \,\|\, p(z)\big) \tag{19.13}$$



8 对证据下界（ELBO）行进优化的示意图。(a) 对于解码器网络参数 w 的给定值 w_0，我们可以通过优化编码器网参数 ϕ 来提高这个下界的值。(b) 对于给定的 ϕ 值，我们可以通过优化 w 来增大 ELBO 函数的值。请注意，蓝色曲线的 ELBO 函数始终位于红色曲线所示的对数似然函数之下，这是因为编码器网络通常无法完全匹配真实的后验分布

图 19.7: 对证据下界 (ELBO) 进行优化的示意图.(a) 对于解码器网络参数 w 的给定值, 我们可以通过优化编码器网络的参数 ϕ 来提高这个下界的值.(b) 对于给定的 ϕ 值, 我们可以通过优化 w 来增大 ELBO 函数的值. 蓝色曲线所示的 ELBO 函数始终位于红色曲线所示的对数似然函数之下, 这是因为编码器网络通常无法完全匹配真实的后验分布.

对于式 (19.13), 我们可以尝试用一个简单的蒙特卡洛估计法来近似潜变量 z 上的积分:

$$\int q(z \mid x_n, \phi) \ln p(x_n \mid z, w) \, \mathrm{d}z \approx \frac{1}{L} \sum_{l=1}^{L} \ln p(x_n \mid z^{(l)}, w) \tag{19.14}$$

其中 $z^{(l)}$ 是从编码器分布 $q(z \mid x_n, \phi)$ 中采样. 关于 w 求导很容易, 但关于 ϕ 求梯度却存在问题, 因为参数 ϕ 的变化也会改变采样分布 $q(z \mid x_n, \phi)$. 然而这些样本是固定值, 所以我们没有办法获取这些样本相对于 ϕ 的导数. 从概念上讲, 我们可以认为将潜变量 z 固定到特定样本值的过程阻止了误差信号向编码器网络的反向传播, 如图 19.8 所示.

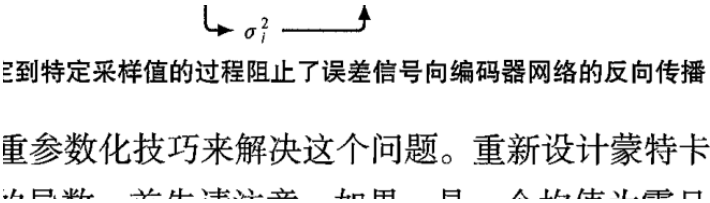


图 19.8: 将潜变量 z 固定到特定采样值的过程阻止了误差信号向编码器网络的反向传播.

我们可以使用**重参数化技巧**来解决这个问题. 重新设计蒙特卡洛采样过程, 以便明确计算相对于 ϕ 的导数. 首先请注意, 如果 ϵ 是一个均值为零且方差为单位方差的高斯随机变量, 那么 z 将具有均值为 μ、方差为 σ^2 的高斯分布 (见习题 19.2):

$$z = \mu + \sigma \odot \epsilon \tag{19.15}$$

将其应用于式 (19.12) 的采样过程, 其中 μ 和 σ 由编码器网络的输出 $\mu(x_n, \phi)$ 和 $\sigma^2(x_n, \phi)$ 确定. 我们不直接从潜变量 z 采样, 而是对 ϵ 采样, 并使用式 (19.15) 计算对应的 z 样本:

$$z^{(l)} = \mu(x_n, \phi) + \sigma(x_n, \phi) \odot \epsilon^{(l)}, \qquad \epsilon^{(l)} \sim \mathcal{N}(0, I) \tag{19.16}$$

其中 $l = 1, \cdots, L$ 是样本的索引. 该方法显式建立 ϕ 与采样值之间的依赖关系, 使梯度计算可行, 如图 19.9 所示.

使用我们特定的建模假设,VAE 的完整损失函数变为

$$\mathcal{L}(w, \phi) \approx \frac{1}{L} \sum_{l=1}^{L} \ln p(x_n \mid z^{(l)}, w) - \frac{1}{2} \sum_{j=1}^{M} \left(1 + \ln \sigma_j^2(x_n, \phi) - \mu_j^2(x_n, \phi) - \sigma_j^2(x_n, \phi)\right) \tag{19.17}$$

其中 $z^{(l)} = \sigma(x_n, \phi) \odot \epsilon^{(l)} + \mu(x_n, \phi)$. 每个数据点 x_n 的样本数 L 通常设置为 1, 即仅使用单个样本进行估计. 尽管这种设置会导致对证据下界的计算结果存在噪声, 但由于随机梯度优化过程本身就具有噪声, 该策略在整体上仍能实现更高效的参数优化.

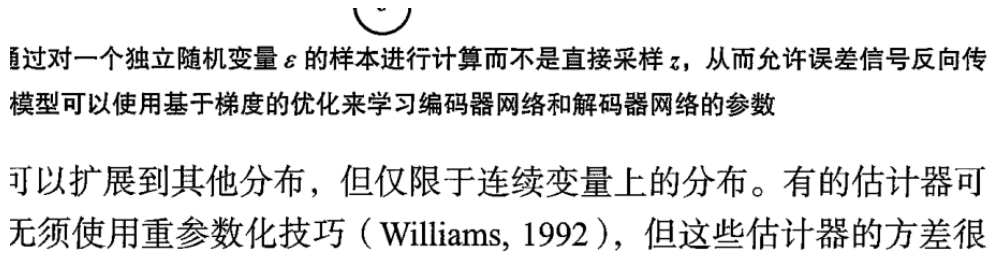


图 19.9: 重参数化技巧通过对一个独立随机变量 ϵ 的样本进行计算, 而不是直接采样 z, 从而允许误差信号反向传播到编码器网络. 得到的模型可以使用基于梯度的优化来学习编码器网络和解码器网络的参数.

我们可以将 VAE 训练总结如下. 对于小批量中的每个数据点, 通过编码器网络前向传播来计算近似潜变量分布的均值和方差, 并使用重参数化技巧从该分布中采样, 然后将这些样本通过解码器网络传播以计算证据下界 [式 (19.17)]. 最后使用自动微分计算相对于 w 和 ϕ 的梯度. 一旦模型训练完成, 编码器网络就被丢弃, 新的数据点将通过从先验分布 $p(z)$ 中采样并通过解码器网络前向传播来生成数据空间中的样本.

VAE 有许多变种. 当应用于图像数据时, 编码器通常基于卷积操作, 而解码器则基于转置卷积 (有时也称反卷积) 操作 (参见 10.5.3 小节). 在条件 VAE 中, 编码器和解码器都将一个条件变量 c 作为额外的输入. 例如, 我们可能想要生成物体的图像, 此时 c 代表物体的类别. 潜空间的先验分布 $p(z)$ 既可以是一个简单的高斯分布, 也可以扩展成另一个神经网络给出的条件分布 $p(z \mid c)$, 训练与测试过程与之前类似.

注意, 式 (19.13) 中的第一项鼓励编码器分布 $q(z \mid x, \phi)$ 接近先验分布 $p(z)$, 从而使解码器在生成模式下, 通过从 $p(z)$ 采样, 能够产生逼真的输出. 在训练 VAE 时, 可能会出现一个问题, 即变分分布 $q(z \mid x, \phi)$ 收敛到先验分布 $p(z)$, 这意味着变分分布没有从输入 x 中学到有价值的信息. 实际上, 潜在编码被忽略了, 因为它对输入 x 没有区分能力. 这称为**后验崩塌** (posterior collapse). 此时, 如果我们取一个输入并编码, 然后解码, 我们将得到一个看起来模糊不清的重建结果. 在这种情况下,KL 散度 $\mathrm{KL}(q(z \mid x, \phi) \| p(z))$ 接近于零.

另一种问题发生在潜在编码没有被压缩的情况下, 具体表现是, 即使能得到高度准确的重建结果, 但从 $p(z)$ 采样并通过解码器网络生成的输出质量却很差, 不像训练数据. 在这种情况下,KL 散度相对较大, 并且因为训练系统获得了一个与先验分布完全不同的变分分布, 从先验分布中采样的样本不会生成逼真的输出. 这两个问题都可以通过在式 (19.13) 的第一项前引入一个系数 β 来解决, 以控制 KL 散度的正则化效果, 通常 $\beta > 1$(Higgins et al., 2017). 如果重建结果很差, 则可以增大 β; 而如果生成的样本很差, 则可以减小 β.β 的值也可以根据退火调度来设置, 即它从一个小的值开始, 并在训练期间逐渐增大.

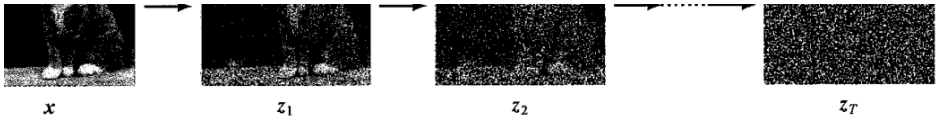
最后请注意, 虽然我们在本章中只讨论了一个表示高斯输出分布均值的解码器网络 $g(z, w)$, 但我们可以扩展 VAE, 让它包含代表高斯分布方差的输出, 或进一步让它表示其他更复杂分布的参数, 以描绘更广泛多变的数据特征 (参见 6.5 节).

第 20 章 扩散模型

20.1 前向编码器

构建丰富的生成式模型的一种行之有效的方法是引入一个潜变量 z 上的分布 $p(z)$, 然后使用深度神经网络将 z 转换到数据空间 x 中. 我们仅需要使用一个简单的固定分布来表示 $p(z)$, 例如高斯分布 $\mathcal{N}(z \mid 0, I)$, 因为神经网络的通用性能够将其转换成一组关于 x 的高度灵活的分布. 在前面的章节 (参见 16.4.4 小节) 中, 我们探讨了多种属于这一框架但基于不同方法定义和训练深度神经网络的模型, 包括生成对抗网络、变分自编码器和标准化流.

在本章中, 我们将讨论这一通用框架下的第 4 类模型—**扩散模型** (diffusion model), 也叫去噪扩散概率模型 (Denoising Diffusion Probabilistic Model, DDPM)(Sohl-Dickstein et al., 2015; Ho, Jain, and Abbeel, 2020), 在许多应用领域, 它们已经成为最先进的技术. 尽管这个框架有更广泛的适用性, 但我们仍将以图像数据为例讲述. 这类模型的核心思想是对每一个训练图像, 通过多步骤的噪声来损坏它, 将其转变为高斯分布的样本, 如图 20.1 所示. 然后训练一个深度神经网络来逆转这个过程, 一旦训练完成, 该网络就可以基于高斯分布的样本开始生成新的图像.



扩散模型中编码过程的示意图，一幅图像 x 逐渐被叠加的高斯噪声破坏，在经过多个阶段后，得到一系
嘈杂的图像。最后的结果与从高斯分布中抽取的样本已无法区分。我们想要训练一个深度神经网络来逆

图 20.1: 扩散模型中编码过程的示意图. 一幅图像 x 逐渐被叠加的高斯噪声破坏, 在经过多个阶段后, 得到一系列越来越嘈杂的图像. 最后的结果与从高斯分布中抽取的样本已无法区分. 我们想要训练一个深度神经网络来逆转这个过程.

扩散模型可以视为一种层次化的变分自编码器, 其中编码器分布是固定的, 并由噪声过程定义, 只有生成分布是学习得来的 (Luo, 2022)(参见 19.2 节). 它们易于训练, 在并行硬件上具有良好的可扩展性, 在避免对抗训练所带来的挑战和不稳定性的同时, 可以产生与生成对抗网络相当或更优的结果. 然而, 由于需要通过解码器网络进行多次前向传播, 它在生成新样本时会产生高昂的计算成本 (Dhariwal and Nichol, 2021).

从训练集中选取一个图像, 表示为 x, 然后将图像中的每个像素独立地与高斯噪声混合, 得到一个被噪声损坏的图像 z_1, 其定义如下:

$$z_1 = \sqrt{1-\beta_1}\, x + \sqrt{\beta_1}\, \epsilon_1 \tag{20.1}$$

其中 $\epsilon_1 \sim \mathcal{N}(\epsilon_1 \mid 0, I)$ 且 $\beta_1 < 1$ 是噪声分布的方差. 式 (20.1) 中的系数 $\sqrt{1-\beta_1}$ 和 $\sqrt{\beta_1}$ 确保了 z_1 的分布均值比 x 的分布均值更接近于零, 且 z_1 的方差比 x 的方差更接近单位矩阵 (见习题 20.1). 我们可以将式 (20.1) 写成以下形式 (参见习题 20.2):

$$q(z_1 \mid x) = \mathcal{N}\left(z_1 \mid \sqrt{1-\beta_1}\, x, \beta_1 I\right) \tag{20.2}$$

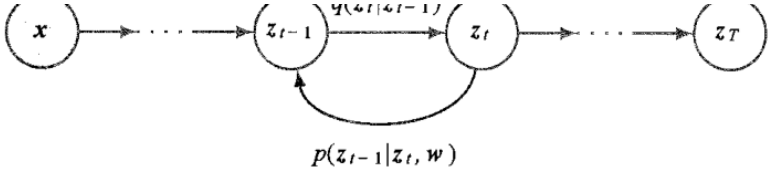
然后重复这个过程, 加入额外的独立高斯噪声, 从而生成一系列越来越多的加噪图像 $z_1, \cdots, z_T$. 在关于扩散模型的文献中, 这些加噪图像或潜变量有时表示为 $x_1, \cdots, x_T$, 而观测变量则表示为 x_0. 在本书中, 我们使用 z 来表示潜变量, 并使用 x 来表示观测变量. 每一个后续图像由下式给出:

$$z_t = \sqrt{1-\beta_t}\, z_{t-1} + \sqrt{\beta_t}\, \epsilon_t \tag{20.3}$$

其中 $\epsilon_t \sim \mathcal{N}(\epsilon_t \mid 0, I)$. 同样, 我们也可以将式 (20.3) 写成以下形式:

$$q(z_t \mid z_{t-1}) = \mathcal{N}\left(z_t \mid \sqrt{1-\beta_t}\, z_{t-1}, \beta_t I\right) \tag{20.4}$$

条件分布序列 [式 (20.4)] 形成了一条马尔可夫链, 并且可以表示为图 20.2 所示的概率图模型 (参见 11.3 节). 方差参数 $\beta_t \in (0, 1)$, 并且通常是手动设置的, 它们按预定的计划递增, 即 $\beta_1 < \beta_2 < \cdots < \beta_T$.



以表示为一个概率图模型。原始图像 x 表示为带有底色的节点，因为它是一个观测变量，而

!为潜变量。加噪过程由前向分布 $q(z_t \mid z_{t-1})$ 定义，它可以视为一个编码器。我们的目标是学

| x, w) 并试图逆转这个过程，可以把它视为一个解码器。正如我们稍后将看到的，条件分

图 20.2: 扩散过程可以表示为一个概率图模型. 原始图像 x 表示为带有底色的节点, 因为它是一个观测变量, 而加噪图像 $z_1, \cdots, z_T$ 则视为潜变量. 加噪过程由前向分布 $q(z_t \mid z_{t-1})$ 定义, 它可以视为一个编码器. 我们的目标是学习一个条件分布 $p(z_{t-1} \mid z_t, w)$ 并试图逆转这个过程, 可以把它视为一个解码器.

这个马尔可夫链 [式 (20.4)] 通常称为**前向过程** (forward process), 它类似于 VAE 中的编码器, 但这里的马尔可夫链是固定的而不是通过训练得来的.

20.1.1 扩散核

在观测到数据向量 x 的条件下, 潜变量的联合分布由下式给出:

$$q(z_1, z_2, \cdots, z_T \mid x) = \prod_{t=1}^{T} q(z_t \mid z_{t-1}) \tag{20.5}$$

对中间变量 $z_1, \cdots, z_{T-1}$ 进行边缘化, 即可得到**扩散核** (diffusion kernel)(参见习题 20.3):

$$q(z_t \mid x) = \mathcal{N}\left(z_t \mid \sqrt{\bar{\alpha}_t}\, x, (1 - \bar{\alpha}_t) I\right) \tag{20.6}$$

其中的 $\bar{\alpha}_t$ 由下式给出:

$$\bar{\alpha}_t = \prod_{s=1}^{t} (1 - \beta_s) \tag{20.7}$$

可以看到, 每一个中间分布都是一个简单的闭式的高斯分布, 我们可以直接从中采样, 这在训练 DDPM 时非常有用, 因为它允许我们使用马尔可夫链中随机选择的中间项进行高效的随机梯度下降, 而不必运行整个马尔可夫链. 我们也可以将式 (20.3) 写成以下形式:

$$z_t = \sqrt{\bar{\alpha}_t}\, x + \sqrt{1 - \bar{\alpha}_t}\, \epsilon_t \tag{20.8}$$

其中仍然有 $\epsilon_t \sim \mathcal{N}(\epsilon_t \mid 0, I)$. 请注意, 这里的 ϵ_t 代表添加到原始图像上的总噪声, 而不是在马尔可夫链这一步骤中添加的增量噪声.

在经过多个这样的步骤之后, 图像变得与高斯噪声无法区分. 在 $T \to \infty$ 的情况下, 我们有 (见习题 20.4)

$$q(z_T \mid x) = \mathcal{N}(z_T \mid 0, I) \tag{20.9}$$

由于式 (20.9) 的右侧独立于 x, 因此可以得出 z_T 的边缘分布:

$$q(z_T) = \mathcal{N}(z_T \mid 0, I) \tag{20.10}$$

因此, 关于原始图像的所有信息都丢失了. 在式 (20.3) 中, 系数 $\sqrt{1 - \beta_t}$ 和 $\sqrt{\beta_t}$ 确保了一旦马尔可夫链收敛到均值为零、协方差为单位矩阵的分布后, 就不会再受进一步更新的影响 (见习题 20.5).

20.1.2 条件分布

我们的目标是学会逆转加噪过程, 因此很自然地要考虑条件分布 $q(z_{t-1} \mid z_t)$ 的逆过程, 可以使用贝叶斯定理把它表示为

$$q(z_{t-1} \mid z_t) = \frac{q(z_t \mid z_{t-1})\, q(z_{t-1})}{q(z_t)} \tag{20.11}$$

我们可以将边缘分布 $q(z_t)$ 写成如下形式:

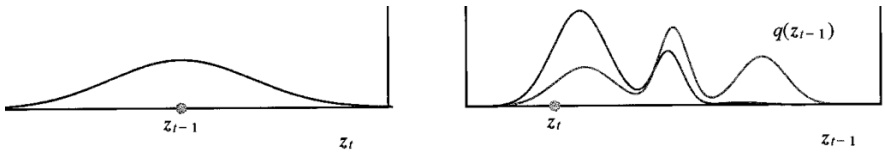
$$q(z_t) = \int q(z_t \mid x)\, p(x)\, \mathrm{d}x \tag{20.12}$$

其中 $q(z_t \mid x)$ 由条件高斯分布 [式 (20.6)] 给出. 然而, 这个分布是难以计算的, 因为我们必须对未知的数据密度 $p(x)$ 进行积分. 如果我们使用训练集中的样本来近似地进行积分, 则会得到一个以高斯函数混合形式表示的复杂分布.

换个思路, 考虑逆向分布的条件版本, 该分布以数据向量 x 为条件, 定义为 $q(z_{t-1} \mid z_t, x)$, 这实际上是一个简单的高斯分布. 从直观上看这是合理的, 因为我们很难猜测哪个低噪声图像演变为某个具体的加噪图像, 然而如果我们知道原始图像, 那么问题解决起来就会变得容易很多. 利用前向过程的马尔可夫性质, 可以得到

$$q(z_{t-1} \mid z_t, x) = \frac{q(z_t \mid z_{t-1}, x)\, q(z_{t-1} \mid x)}{q(z_t \mid x)} \tag{20.13}$$

从直观上看, 如果我们保持方差较小, 使得 β_t 远小于 1, 那么潜向量之间的步骤变化将相对较小, 因此学会逆转变换应该更容易. 更具体地说, 如果 β_t 远小于 1, 那么分布 $q(z_{t-1} \mid z_t)$ 将近似于关于 z_t 的高斯分布. 这一点从式 (20.11) 中可以看出, 因为等式右边是通过 $q(z_t \mid z_{t-1})$ 和 $q(z_{t-1})$ 与 z_t 相关联的. 如果 $q(z_t \mid z_{t-1})$ 是一个极其狭窄的高斯分布, 那么在 $q(z_{t-1} \mid z_t)$ 具有显著概率质量的区域,$q(z_{t-1})$ 的变化将非常小, 因此 $q(z_{t-1} \mid z_t)$ 也可以近似认为是高斯分布. 这种直觉可以通过图 20.3 所示的简单例子来证实. 然而, 由于每一步的方差都很小, 我们必须使用大量的步骤来确保从前向加噪过程中获得的最终潜变量 z_T 的分布仍然接近高斯分布, 这增加了生成新样本的成本. 在实践中,T 的值可能达到数千.



用于标量变量的贝叶斯定理［式（20.13）］的逆向分布 $q(z_{t-1}|z_t)$ 的计算示意图。右图中的红色曲线展示个高斯分布混合而成的边缘分布 $q(z_{t-1})$，左图则展示了以 z_{t-1} 为中心的高斯前向加噪过程 $q(z_t|z_{t-1})$。通相乘并归一化，我们得到了特定选择的 z_t 的分布 $q(z_{t-1}|z_t)$，如蓝色曲线所示。因为左侧的分布相对较于一个较大的方差 β_t，所以分布 $q(z_{t-1}|z_t)$ 具有复杂的多模态结构

图 20.3: 用于标量变量的贝叶斯定理 [式 (20.11)] 的逆向分布 $q(z_{t-1} \mid z_t)$ 的计算示意图. 右图中的红色曲线展示了使用三个高斯分布混合而成的边缘分布 $q(z_{t-1})$; 左图则展示了以 z_t 为中心的高斯前向加噪过程 $q(z_t \mid z_{t-1})$. 通过将这些相乘并归一化, 我们得到了特定选择的 z_t 的分布 $q(z_{t-1} \mid z_t)$, 如蓝色曲线所示. 因为左侧的分布相对较宽, 对应于一个较大的方差 β_t, 所以分布 $q(z_{t-1} \mid z_t)$ 具有复杂的多模态结构.

20.2 反向解码器

我们已经知道, 前向编码器是由一系列高斯条件分布 $q(z_t \mid z_{t-1})$ 定义的, 但直接逆转这一过程会导致一个难以处理的分布 $q(z_{t-1} \mid z_t)$, 因为需要对所有可能的起始向量 x 的值进行积分, 而 x 的分布就是我们希望建模的未知数据分布 $p(x)$. 为了解决这个问题, 我们将通过使用一个由深度神经网络控制的分布 $p(z_{t-1} \mid z_t, w)$ 来学习对逆向分布进行近似, 其中 w 表示网络的权重和偏置. 这个逆向步骤类似于变分自编码器中的解码器 (参见第 19 章), 可以通过图 20.2 来理解. 一旦网络训练完成, 我们就可以从简单的高斯分布 z_T 中进行采样, 并通过一系列逆向采样步骤将其变换成数据分布 $p(x)$ 的样本, 这些步骤是通过反复应用训练好的网络来完成的.

我们可以更形象地看到, 通过围绕点 z_t 将 $\ln q(z_{t-1} \mid z_t)$ 对 z_{t-1} 进行泰勒级数展开,$q(z_{t-1} \mid z_t)$ 接近于高斯分布, 并且具有与 $q(z_t \mid z_{t-1})$ 类似的方差. 这也表明对于小方差, 逆向分布 $q(z_{t-1} \mid z_t)$ 的协方差将接近前向加噪过程 $q(z_t \mid z_{t-1})$ 的协方差 $\beta_t I$. 因此, 我们可以使用以下形式的高斯分布来模拟逆向过程 (见习题 20.7):

$$p(z_{t-1} \mid z_t, w) = \mathcal{N}\left(z_{t-1} \mid \mu(z_t, w, t),\, \beta_t I\right) \tag{20.14}$$

其中 $\mu(z_t, w, t)$ 是由参数 w 控制的一个深度神经网络. 注意, 该深度神经网络显式地将步骤索引 t 作为输入, 以便

考虑到链中不同步骤之间方差 β_t 的变化. 这也使得我们能够使用单个网络来逆转马尔可夫链中的所有步骤, 而不必为每个步骤学习一个单独的网络. 此外, 还可以通过在网络中引入更多的输出来学习去噪过程的协方差 (Nichol and Dhariwal, 2021), 从而更好地描述 z_t 邻域内分布 $q(z_{t-1} \mid z_t)$ 的曲率. 用于建模 $\mu(z_t, w, t)$ 的神经网络在架构选择上有相当大的灵活性, 只要输出的维度与输入的维度相同即可 (参见 10.5.4 小节). 鉴于这一限制,U-Net 架构是图像处理应用的常见选择.

整个逆向去噪过程采用了一种由下式给出的马尔可夫链形式:

$$p(x, z_1, \cdots, z_T \mid w) = p(z_T) \prod_{t=1}^{T} p(z_{t-1} \mid z_t, w) \tag{20.15}$$

20.2.1 训练解码器

接下来, 我们需要决定用来训练神经网络的目标函数. 显而易见的选择是似然函数, 对于数据点 x, 似然函数由下式给出:

$$p(x \mid w) = \int \cdots \int p(x, z_1, \cdots, z_T \mid w) \, \mathrm{d}z_1 \cdots \mathrm{d}z_T \tag{20.16}$$

其中 $p(x, z_1, \cdots, z_T \mid w)$ 由式 (20.15) 定义. 这是一般潜变量模型 [式 (19.5)] 的一个实例, 其中潜变量是 $z = (z_1, \cdots, z_T)$, 观测变量是 x. 注意, 所有的潜变量与数据空间具有相同的维度, 这与标准化流一样, 但不适用于变分自编码器或生成对抗网络.

从式 (20.16) 中可以看到, 似然函数需要对所有可能生成观测数据点的噪声轨迹进行积分. 式 (20.16) 中的积分是难以计算的, 因为涉及对高度复杂的神经网络函数进行积分.

这里假设分布 $p(z_T)$ 与 $q(z_T)$ 相同, 因此也由 $\mathcal{N}(z_T \mid 0, I)$ 给出. 一旦模型训练好了, 采样就变得简单直接了, 因为我们可以首先从简单的高斯分布 $p(z_T)$ 中采样, 然后依次从每个条件分布 $p(z_{t-1} \mid z_t, w)$ 中采样, 最后从 $p(x \mid z_1, w)$ 中采样, 从而获得数据空间中的样本 x.

20.2.2 证据下界

鉴于精确的似然函数是难以计算的, 我们可以采取与变分自编码器相似的方法来最大化对数似然的下界, 这个下界称为**证据下界** (Evidence Lower BOund, ELBO). 对于任何选择的分布 $q(z)$, 以下关系恒成立:

$$\ln p(x \mid w) = \mathcal{L}(w) + \mathrm{KL}\big(q(z) \,\|\, p(z \mid x, w)\big) \tag{20.17}$$

其中 $\mathcal{L}$ 是证据下界, 也称变分下界 (variational lower bound), 由下式给出:

$$\mathcal{L}(w) = \int q(z) \ln \left\{ \frac{p(x \mid z, w) \, p(z)}{q(z)} \right\} \mathrm{d}z \tag{20.18}$$

由于 KL 散度满足 $\mathrm{KL}(q \| p) \geqslant 0$, 可以推导出

$$\ln p(x \mid w) \geqslant \mathcal{L}(w) \tag{20.19}$$

在定义下界时, 我们可以自由选择任何形式的 $q(z)$, 只要它是一个有效的概率分布即可, 即它是非负的并且积分值为 1. 在 ELBO 的许多应用 (如变分自编码器) 中, 我们可以选择具有可调参数的 $q(z)$, 这些参数通常以深度神经网络的形式存在, 然后相对于这些参数以及分布 $p(x, z \mid w)$ 的参数最大化 ELBO. 优化分布 $q(z)$ 使下界更接近真实值, 从而让 $p(x, z \mid w)$ 中参数的优化更接近最大似然估计. 然而在扩散模型中,$q(z)$ 由固定分布 $q(z_1, \cdots, z_T \mid x)$ 给出, 该分布由马尔可夫链 [式 (20.5)] 定义, 因此可调参数仅存在于用于逆马尔可夫链的模型 $p(z_{t-1} \mid z_t, w)$ 中.

为了做到这一点, 我们首先为扩散模型的下界推导出一种明确的形式. 在式 (20.5) 中, 我们可以使用前向过程的马尔可夫性质, 从而让 $p(x, z \mid w)$ 中参数的优化更接近最大似然估计. 由于对数似然函数是难以计算的, 所以我们通过最大化下界 $\mathcal{L}(w)$ 来训练神经网络.

20.2.3 重写 ELBO

在讨论了变分自编码器的 ELBO 之后, 我们的目标是用 KL 散度来表达 ELBO, 随后可以将其表示为闭式表达式. 神经网络建模的是逆向过程中的分布 $p(z_{t-1} \mid z_t, w)$, 而 q 分布是定义在前向过程 $q(z_t \mid z_{t-1}, x)$ 上, 因此我们使用贝叶斯定理来反转条件分布:

$$q(z_{t-1} \mid z_t, x) = \frac{q(z_t \mid z_{t-1}, x)\, q(z_{t-1} \mid x)}{q(z_t \mid x)} \tag{20.20}$$

这样我们就可以把 ELBO 中的第二项写为

$$q(z_{t-1} \mid z_t, x)\, q(z_t \mid x) = q(z_t \mid z_{t-1}, x)\, q(z_{t-1} \mid x) \tag{20.21}$$

式 (20.21) 右边的第二项与 w 无关, 因此可以省略. 将式 (20.21) 代入 ELBO, 最后我们可以把它写成下面的形式 (见习题 20.9):

$$\mathcal{L}(w) = \mathbb{E}_{q(z_T|x)}\Big[\ln p(z_T)\Big] + \sum_{t=2}^{T} \mathbb{E}_{q(z_t|x)}\Big[\mathrm{KL}\big(q(z_{t-1} \mid z_t, x) \,\|\, p(z_{t-1} \mid z_t, w)\big)\Big] + \mathbb{E}_{q(z_1|x)}\big[\ln p(x \mid z_1, w)\big] \tag{20.22}$$

式 (20.22) 右边的第一项 $\ln p(z_T)$ 是固定分布 $\mathcal{N}(z_T \mid 0, I)$, 它没有可训练的参数, 可以从 ELBO 中省略, 因为它代表一个固定的加性常数. 同样, 式 (20.22) 右边的第三项 $-\ln q(z_T \mid x)$ 与 w 无关, 所以也可以省略. 式 (20.22) 右边的第 4 项对应于变分自编码器中的重构项, 它可以从由式 (20.6) 定义的 z_t 的分布中抽取样本来获得蒙特卡洛估计, 从而通过近似期望 $\mathbb{E}[\cdot]$ 来评估.

现在的下界 [式 (20.22)] 与式 (19.13) 给出的变分自编码器的 ELBO 非常类似, 区别在于这里存在多个编码和解码阶段. 重构项通过提高观测数据样本的概率来优化模型, 并且可以采用与 VAE 中的相应项相同的方式, 通过采样近似来完成. 与变分自编码器不同, 我们不需要通过抽样值反向传播误差信号, 因为 q 分布是固定的, 所以这里不需要使用重参数化技巧 (参见 19.2.2 小节).

式 (20.22) 中的一致项是在高斯分布对之间定义的, 因此可以用解析形式来表达. 分布 $q(z_{t-1} \mid z_t, x)$ 由式 (20.20) 给出, 而分布 $p(z_{t-1} \mid z_t, w)$ 由式 (20.14) 给出, 因此 KL 散度变成式 (20.23) 的形式 (见习题 20.11):

$$\mathrm{KL}\big(q(z_{t-1} \mid z_t, x) \,\|\, p(z_{t-1} \mid z_t, w)\big) = \frac{\beta_t}{2\sigma_t^2}\big\|m_t(x, z_t) - \mu(z_t, w, t)\big\|^2 + \text{const} \tag{20.23}$$

其中 $m_t(x, z_t)$ 由逆条件分布的均值给出, 并且任何与网络参数 w 无关的加性项都已经被吸收到常数项 const 中, 在训练中不起作用. 式 (20.22) 中的每个一致项对 z_t 的剩余积分 (由 $q(z_t \mid x)$ 加权), 可以通过从 $q(z_t \mid x)$ 中采样来近似, 这可以通过扩散核 [式 (20.6)] 高效地完成.

观察式 (20.23) 可知,KL 散度表现为简单的平方损失函数的形式. 既然我们通过调整网络参数 w 来最大化式 (20.22) 中的下界, 而 ELBO 中 KL 散度项的前面带有负号, 因此这等价于最小化这个平方误差.

20.2.4 预测噪声

为了获得更高质量的结果, 我们可以改变神经网络的作用, 使其不是在马尔可夫链的每一步都预测去噪后的图像, 而是预测添加到原始图像上以创建该步骤中有噪声图像的总噪声成分 (Ho, Jain and Abbeel, 2020). 对式 (20.8) 重新整理后, 可以得到

$$x = \frac{1}{\sqrt{\bar{\alpha}_t}}\Big(z_t - \sqrt{1 - \bar{\alpha}_t}\,\epsilon_t\Big) \tag{20.24}$$

将式 (20.24) 代入逆条件分布的均值 $m_t(x, z_t)$, 即可用原始数据向量 x 和噪声 ϵ 重写逆条件分布 (见习题 20.12).

类似地, 我们不再使用神经网络 $\mu(z_t, w, t)$ 来预测去噪后的图像, 而是引入旨在预测添加到 x 上以生成 z_t 的总噪声的神经网络 $g(z_t, w, t)$. 按照推导式 (20.23) 的相同步骤, 我们看到, 这两个神经网络之间的关系如下:

$$\mu(z_t, w, t) = \frac{1}{\sqrt{\bar{\alpha}_t}}\Big(z_t - \sqrt{1 - \bar{\alpha}_t}\, g(z_t, w, t)\Big) \tag{20.25}$$

将简化后的式 (20.25) 代入式 (20.23), 可以得到 (见习题 20.13)

$$\mathcal{L}(w) = \frac{\beta_t^2}{2\bar{\alpha}_t(1 - \bar{\alpha}_t)(1 - \beta_t)}\big\|g\big(\sqrt{\bar{\alpha}_t}\, x + \sqrt{1 - \bar{\alpha}_t}\,\epsilon_t, w, t\big) - \epsilon_t\big\|^2 + \text{const} \tag{20.26}$$

有研究 (Ho, Jain, and Abbeel, 2020) 发现, 简单地省略掉式 (20.26) 前面的因子 $\beta_t^2/[2\bar{\alpha}_t(1-\bar{\alpha}_t)(1-\beta_t)]$, 马尔可夫链中的所有步骤就有了相等的权重, 这可以进一步提高性能. 将简化后的式 (20.25) 代入式 (20.23), 可以得到最终的损失函数:

$$\mathcal{L}(w) = \mathbb{E}_{t,x,\epsilon_t}\left[\left\|g\left(\sqrt{\bar{\alpha}_t}\,x + \sqrt{1-\bar{\alpha}_t}\,\epsilon_t, w, t\right) - \epsilon_t\right\|^2\right] \tag{20.27}$$

式 (20.27) 右侧的平方误差有一个非常简明的解释: 对于马尔可夫链中给定的步骤 t 和给定的训练数据点 x, 抽取一个噪声向量 ϵ_t, 并使用这个噪声向量来创建对应步骤 t 的噪声潜变量 z_t. 然后损失函数就是预测的噪声和实际噪声的平方差. 注意神经网络 $g(\cdot)$ 预测的是添加到原始数据向量 x 上的总噪声, 而不仅仅是步骤 t 中增加的增量噪声.

当使用随机梯度下降时, 针对从训练集中随机选取的数据点 x, 计算损失函数对网络参数的梯度向量. 另外, 对于每个这样的数据点, 随机选择沿着马尔可夫链的一个步骤 t, 而不是计算式 (20.27) 中对 t 求和的每一项的误差. 这些梯度在小批量数据样本上累积后, 用来更新网络权重. 另外, 这个损失函数自动地内置了一种数据增强机制, 因为每当特定的训练样本 x 被使用时, 它都会与一个新的噪声样本 ϵ_t 相结合.

20.2.5 生成新的样本

一旦网络训练好了, 我们就可以通过以下步骤生成数据空间中的新样本: 首先从高斯分布 $p(z_T)$ 中采样, 然后逐步通过马尔可夫链的每一步去噪. 给定步骤 t 的去噪样本 z_t, 我们可以通过三个步骤生成样本 z_{t-1}. 首先计算由 $g(z_t, w, t)$ 给出的神经网络输出. 根据这个输出, 使用式 (20.25) 计算均值 $\mu(z_t, w, t)$. 接下来, 通过在 $p(z_{t-1} \mid z_t, w) = \mathcal{N}(z_{t-1} \mid \mu(z_t, w, t), \beta_t I)$ 中添加经过方差缩放的噪声来生成样本 z_{t-1}:

$$z_{t-1} = \mu(z_t, w, t) + \sqrt{\beta_t}\,\epsilon_t, \qquad \epsilon_t \sim \mathcal{N}(\epsilon_t \mid 0, I) \tag{20.28}$$

其中 $g(\cdot)$ 预测的是生成 z_t 时添加到原始数据向量 x 上的总噪声. 但在采样步骤中, 我们只从 z_t 中减去一部分具有方差 $\beta_t/(1-\bar{\alpha}_t)$ 的噪声, 然后添加具有方差 β_t 的额外噪声来生成 z_{t-1}. 在最终计算合成数据样本 x 的步骤中, 由于我们的目标是产生一个无噪声输出, 因此我们不会添加额外的噪声.

使用扩散模型生成数据的主要缺点是, 需要在训练好的网络中多次顺序执行推理过程, 计算成本非常高. 加快采样过程的一种方法是, 首先将去噪过程转换为连续时间上的微分方程, 然后使用其他有效的离散化方法来高效求解该方程 (参见 20.3.4 小节). 研究人员 (Song, Meng, and Ermon, 2020) 提出了一种称为**去噪扩散隐式模型** (denoising diffusion implicit models) 的技术, 它放宽了加噪过程的马尔可夫假设条件, 同时保留保持训练目标函数不变. 这样就可以在不降低所生成样本质量的情况下, 使采样速度提升一个或两个数量级.

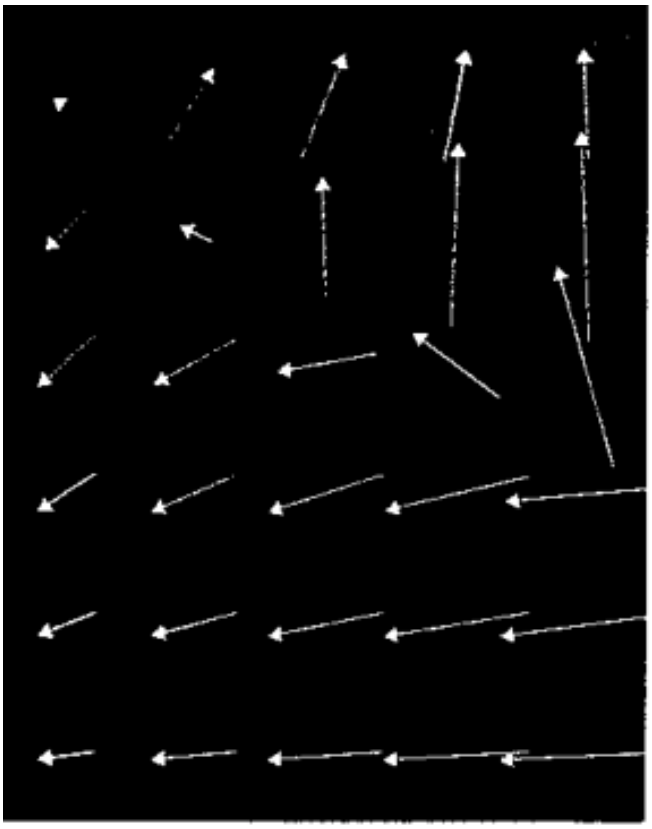
20.3 得分匹配

截至目前, 本章讨论的去噪扩散模型与另一类相对独立发展的深度生成式模型密切相关, 它们都基于**得分匹配** (score matching)(Hyvärinen, 2005; Song and Ermon, 2019). 这些模型利用的得分函数 (或斯坦因得分) 则定义为对数似然函数关于数据向量 x 的梯度:

$$s(x) = \nabla_x \ln p(x) \tag{20.29}$$

这里需要强调的是, 梯度是针对数据向量 x 计算的, 而不是针对任何参数向量. 注意 $s(x)$ 是一个与 x 维度相同的向量值函数, 并且其中的每个元素 $s_i(x) = \partial \ln p(x)/\partial x_i$ 都与 x 的相应元素 x_i 相关联. 例如, 如果 x 是一个图像, 那么 $s(x)$ 也可以表示为一个同尺寸的图像, 其中对应的像素是图像的得分.

为了理解得分函数的用处, 考虑两个函数 $q(x)$ 和 $p(x)$, 它们具有得分相等的属性, 所以对于 x 的所有值,$\nabla \ln q(x) = \nabla \ln p(x)$. 我们对这个等式的两边关于 x 积分并取指数, 则可以得到 $q(x) = Kp(x)$, 其中 K 是一个独立于 x 的常数. 所以如果我们能学习到一个得分函数的模型 $s(x, w)$, 就可以重建原始数据分布, 所得结果与之前的相比, 最多相差一个常数倍数.



的示意图，图中显示了两种
新混合分布构成的二维分布，
由式（20.42）定义的相应得
量绘制在 x 值的规则网格上

图 20.4: 得分函数的示意图, 图中显示了两种信息的融合: 由高斯混合分布构成的二维分布, 其作为向量绘制在 x 值的规则网格上, 表示为热图; 以及由式 (20.29) 定义的相应的概率密度示例以及相应的得分函数.

20.3.1 得分损失函数

为了训练这样的模型, 需要定义一个损失函数, 用于将模型得分函数 $s(x, w)$ 匹配到旨在生成数据的分布 $p(x)$ 的得分函数 $\nabla_x \ln p(x)$. 这样一个损失函数的例子是计算模型得分和真实得分之间的期望平方误差, 其由下式给出:

$$J(w) = \int \left\| s(x, w) - \nabla_x \ln p(x) \right\|^2 p(x) \, \mathrm{d}x \tag{20.30}$$

正如我们在讨论基于能量的模型时所看到的 (参见 14.3.1 小节), 得分函数不要求相关的概率密度被归一化, 因为归一化常数被梯度算子移除了, 所以我们在模型选择上有相当大的灵活性. 使用深度神经网络表示得分函数 $s(x, w)$ 的方法有两种.s 的每个元素 s_i 都对应于 x 中的一个元素 x_i, 所以第一种方法是使用一个输入数量和输出数量相同的网络. 然而, 得分函数定义为标量函数 (对数概率密度函数) 的梯度, 这是一类受限的函数 (见习题 20.14). 另一种方法是使用一个拥有单一输出的网络 $\phi(x, w)$, 然后使用自动微分计算 $\nabla_x \phi(x, w)$. 然而, 这方法需要进行两次反向传播, 因此计算成本更高. 大多数应用通常采用第一种方法 (见习题 20.15).

20.3.2 修改得分损失

损失函数 [式 (20.30)] 的一个问题是不能直接最小化, 因为我们不知道真实的数据得分 $\nabla_x \ln p(x)$. 我们所知的只是有限的数据集 $\mathcal{D} = \{x_1, \cdots, x_N\}$, 从中可以构建如下经验分布:

$$\widehat{p}(x) = \frac{1}{N} \sum_{n=1}^{N} \delta(x - x_n) \tag{20.31}$$

这里的 $\delta(x)$ 是狄拉克 delta 函数, 我们可以将其形象地理解为在 $x = 0$ 处无限高的 “尖峰”, 它具有以下性质:

$$\delta(x) = 0, \quad x \neq 0; \qquad \int \delta(x) \, \mathrm{d}x = 1 \tag{20.32}$$

由于式 (20.31) 所示的函数不是 x 的可微函数, 我们无法计算它的得分. 但我们可以通过引入噪声模型来 “抹平” 数据点并给出密度的平滑、可微表示来解决这个问题, 这称为 Parzen 估计器或核密度估计器 (参见 3.5.2 小节), 定义如下:

$$q(z \mid x, \sigma) = \int q(z \mid x, \sigma) \, \widehat{p}(x) \, \mathrm{d}x = \frac{1}{N} \sum_{n=1}^{N} q(z \mid x_n, \sigma) \tag{20.33}$$

其中 $q(z \mid x, \sigma)$ 是噪声核. 常见的核选择是高斯核:

$$q(z \mid x, \sigma) = \mathcal{N}(z \mid x, \sigma^2 I) \tag{20.34}$$

此时, 我们不再最小化损失函数式 (20.30), 而是使用对应于平滑后的 Parzen 密度的损失函数, 形式为

$$J(w) = \int \left\| s(z, w) - \nabla_z \ln q(z) \right\|^2 q(z) \, \mathrm{d}z \tag{20.35}$$

一个关键结果是, 通过将式 (20.33) 代入式 (20.35), 我们可以将这个损失函数重写为以下等价形式 (Vincent, 2011)(见习题 20.17):

$$J(w) = \frac{1}{N} \sum_{n=1}^{N} \int \left\| s(z, w) - \nabla_z \ln q(z \mid x_n, \sigma) \right\|^2 q(z \mid x_n, \sigma) \, \mathrm{d}z + \text{const} \tag{20.36}$$

对于高斯核式 (20.34), 得分函数变为

$$\nabla_z \ln q(z \mid x, \sigma) = -\frac{z - x}{\sigma^2} \tag{20.37}$$

其中 $\epsilon = z - x \sim \mathcal{N}(z \mid 0, I)$. 考虑特定的噪声模型 [式 (20.6)], 可以得到

$$J(w) = \frac{1}{N} \sum_{n=1}^{N} \mathbb{E}_{z \sim q(z \mid x_n, \sigma)} \left[\left\| s(z, w) + \epsilon / \sqrt{1 - \bar{\alpha}_t} \right\|^2 \right] \tag{20.38}$$

由此我们可以看出, 损失函数式 (20.38) 衡量了神经网络预测与噪声 ϵ 之间的差异. 所以, 该损失函数与去噪扩散模型中使用的损失函数式 (20.26) 拥有相同的最小值, 得分函数 $s(z, w)$ 扮演的角色与噪声预测网络 $g(z, w, t)$ 相同, 但前者做了常数缩放 $-1/\sqrt{1 - \bar{\alpha}_t}$(Song and Ermon, 2019). 最小化式 (20.38) 的过程称为**去噪得分匹配** (denoise

score matching), 并且我们看到它与去噪扩散模型有着密切的联系.

训练完基于得分的模型后, 我们需要生成新的样本. 朗之万动力学 (Langevin dynamics) 非常适用于基于得分函数的模型, 因为它以得分函数为基础进行运算, 无须依赖归一化的概率分布, 参见图 20.5.



图 20.5: 使用式 (**??**) 定义的朗之万动力学得到的采样轨迹, 这里显示了三条都从图中心开始的采样轨迹 (参见 14.3 节).

20.3.3 噪声方差

我们已经了解怎么从一组训练数据中学习得分函数, 以及如何使用朗之万动力学方法从学习到的分布中进行采样并生成新的样本. 然而, 这种方法存在三个潜在的问题 (Song and Ermon, 2019; Luo, 2022)(参见第 16 章). 第一, 如果数据分布位于比数据空间维度更低的流形上, 那么流形之外的点的概率密度将为零, 此时因为 $\ln p(x)$ 是未定义的, 所以得分函数是未定义的. 第二, 在数据密度低的区域, 得分函数的估计可能不准确, 因为损失函数 [式 (20.30)] 被密度加权了. 一个不准确的得分函数会导致使用朗之万动力学方法采样时轨迹不佳. 第三, 即使有一个准确的得分函数, 如果数据分布包括一系列不连续的混合分布, 通过朗之万动力学方法也可能无法正确采样 (见习题 20.18).

以上三个问题都可以通过选择核函数 [式 (20.34)] 中使用的噪声方差 σ^2 的足够大值来解决, 因为这会使数据分布平滑. 然而, 过大的方差会导致原始分布显著失真, 导致得分函数建模失准. 一个折中方案是引入方差序列 $\sigma_1^2 < \sigma_2^2 < \cdots < \sigma_L^2$(Song and Ermon, 2019), 其中 σ_1^2 足够小, 以便数据分布得到准确表示; 而 σ_L^2 则足够大, 以避免导致得分函数建模失准. 然后修改得分网络, 使其将方差作为额外的输入 $s(x, w, \sigma^2)$, 并通过使用一个加权和形

式的损失函数 [式 (20.38)] 对其进行训练, 其中的每一项表示相关网络与相应扰动数据集之间的误差. 训练完成后, 可以通过依次对 $i = L, L-1, \cdots, 2, 1$ 的每个模型逐个执行几步朗之万动力学方法进行采样并生成样本. 这种技术称为**退火朗之万动力学** (annealed Langevin dynamics), 类似于用来从去噪扩散概率模型中采样的算法 20.2.

20.3.4 随机微分方程

在构建扩散模型的加噪过程时, 使用大数量的步骤 (通常为几千步) 是有益处的. 因此, 我们需要考虑在无穷步数的极限情况下会发生什么, 就像我们在引入神经微分方程时为无限深度的神经网络所做的那样 (参见 18.3.1 小节). 在这种极限情况下, 我们需要确保每一步的噪声方差 β_t 随着步长变小而同步变小. 这引出了连续时间下扩散模型的理论框架, 即**随机微分方程** (Stochastic Differential Equation, SDE)(Song et al., 2020). 去噪扩散概率模型和得分匹配模型都可以视为连续时间 SDE 的离散化实现.

我们可以将一般的 SDE 写成向量 z 的无穷小更新:

$$\mathrm{d}z = f(z,t)\,\mathrm{d}t + g(t)\,\mathrm{d}w \tag{20.39}$$

其中漂移项 (drift term) 是确定性的, 类似于常微分方程 (ODE), 但扩散项 (diffusion term) 是随机的. 这里的参数 t 常常称为 "时间", 类比于物理系统中的时间. 扩散模型的前向加噪过程 [式 (20.3)], 可以通过采取连续时间极限而写成形如式 (20.39) 的 SDE(见习题 20.19).

对于 SDE [式 (20.39)], 存在一个对应的逆向 SDE(Song et al., 2020), 如下所示:

$$\mathrm{d}z = \left\{f(z,t) - g^2(t)\nabla_z \ln p(z)\right\}\mathrm{d}t + g(t)\,\mathrm{d}\bar{w} \tag{20.40}$$

其中 $\nabla_z \ln p(z)$ 是得分函数. 对于式 (20.39) 给出的 SDE, 应从 $t = T$ 倒序求解至 $t = 0$.

对于所有由 SDE 支配的扩散过程, 都存在一个相应的由 ODE 描述的确定性过程, 其轨迹具有与 SDE 相同的边缘概率密度 $p(z \mid t)$(Song et al., 2020). 对于形如式 (20.39) 的 SDE, 相应的 ODE 为

$$\frac{\mathrm{d}z}{\mathrm{d}t} = f(z,t) - \frac{1}{2}g^2(t)\nabla_z \ln p(z) \tag{20.41}$$

该 ODE 允许使用高效的自适应步长求解器以最大限度地减少函数评估次数. 此外,ODE 还允许将概率扩散模型与标准化流模型相关联, 后者允许使用变量替换式 (18.1) 来精确计算对数似然 (参见第 18 章).

要想数值求解 SDE, 我们需要对时间变量进行离散化. 最简单的方法是使用固定且等间距的时间步, 这称为 Euler–Maruyama 求解器. 对于反向 SDE, 我们可以先恢复朗之万方程的一种形式 (参见 14.3 节), 再采用更复杂的求解器, 这些求解器使用了更灵活的离散化形式 (Kloeden and Platen, 2013).

20.4 有引导的扩散

到目前为止, 我们讨论的扩散模型主要用于表示从一组训练样本 $x_1, \cdots, x_N$ 中学习到的无条件密度 $p(x)$, 这些样本是独立地从 $p(x)$ 中抽取的. 一旦模型训练好了, 我们就可以从这个分布中生成新的样本. 我们已经看到了一个从深度生成式模型中进行无条件采样的例子, 用于人脸图像.

然而在许多应用中, 我们希望从条件分布 $p(x \mid c)$ 中进行采样, 其中条件变量 c 可以是一个类标签或是描述目标图像内容的文字. 这也构成了诸如图像超分辨率、图像修复、视频生成等许多其他应用的基础. 实现这一点的最简单方法是将 c 视为去噪神经网络 $g(z, w, t, c)$ 的一个额外输入, 然后使用匹配的 $\{x_n, c_n\}$ 对来训练网络. 这种方法的主要问题是, 网络可能会对条件变量赋予的权重不足, 甚至忽略条件变量, 因此我们需要一种方式来控制给予条件变量多少权重, 并与样本多样性进行权衡. 这种用以匹配条件信息的约束机制称为**引导** (guidance). 根据是否使用单独的分类器, 引导主要有两种.

20.4.1 有分类器的引导

假设我们有一个训练好的分类器 $p(c \mid x)$, 下面从得分函数的角度考虑扩散模型. 使用贝叶斯定理, 我们可以将条件扩散模型的得分函数写成以下形式:

$$\nabla_x \ln p(x \mid c) = \nabla_x \ln p(x) + \nabla_x \ln p(c \mid x) \tag{20.42}$$

这里我们利用了 $\nabla_x \ln p(c) = 0$, 因为 $p(c)$ 不依赖于 x. 式 (20.42) 中的第一项是常见的无条件得分函数, 第二项则推动去噪过程朝着最大化给定标签 c 在分类器模型下的概率的方向前进 (Dhariwal and Nichol, 2021). 我们可以通过引入一个超参数 λ [称为引导尺度 (guidance scale)] 来控制赋予分类器梯度的权重. 用于采样的得分函数随后变为

$$\text{score}(x, c, \lambda) = \nabla_x \ln p(x) + \lambda \nabla_x \ln p(c \mid x) \tag{20.43}$$

如果 $\lambda = 0$, 则恢复原始的无条件扩散模型; 而如果 $\lambda = 1$, 则得到对应于条件分布 $p(x \mid c)$ 的得分. 当 $\lambda > 1$ 时, 模型被强烈引导遵守条件标签, 我们也可能使用更大的 λ 值, 例如 $\lambda = 10$. 然而, 这是以牺牲样本多样性为代价的, 因为模型更倾向于 "简单" 的样本, 即分类器能够正确分类的样本.

基于分类器的引导方法的一个问题是必须训练一个单独的分类器. 此外, 这个分类器需要能够对具有不同加噪程度的样本进行分类, 而标准分类器是在干净的样本上进行训练的. 因此, 我们转向另一种无需独立分类器的引导方法, 以避免使用单独的分类器.

20.4.2 无分类器的引导

更进一步, 我们无须分别训练多个网络来对 $p(x \mid c)$ 和 $p(x)$ 进行建模, 而是训练一个单独的条件模型, 并在该模型中将条件变量 c 在训练过程中以一定概率设置为零 (参见 9.6.1 小节). 这在某种程度上类似于 dropout,dropout 会将一部分训练向量的所有条件输入都设为空值, 例如 $c = 0$, 概率通常设置为 10% ~ 20%, 然后 $p(x)$ 由 $p(x \mid c = 0)$ 表示.

训练完成后, 得分函数 [式 (20.42)] 用来强化条件信息的权重. 实践表明, 无分类器的引导相比有分类器的引导可以提供更高的生成质量 (Nichol et al., 2021; Saharia et al., 2022). 原因是分类器 $p(c \mid x)$ 可以忽略大部分输入向量 x, 只要能对 c 做出良好预测即可; 而无分类器的引导基于条件密度 $p(x \mid c)$, 其必须为 x 的所有特征赋予高概率.

用式 (20.42) 替换式 (20.43) 中的 $\nabla_x \ln p(c \mid x)$, 即可将得分函数写成以下形式 (见习题 20.20):

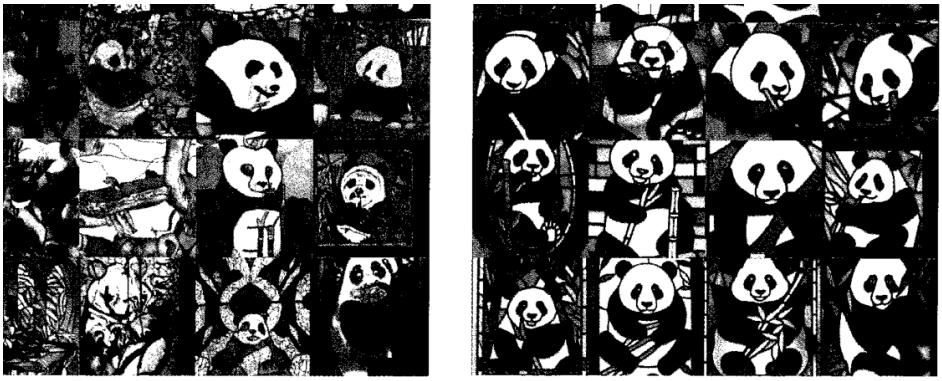
$$\text{score}(x, c, \lambda) = \lambda \nabla_x \ln p(x \mid c) + (1 - \lambda) \nabla_x \ln p(x) \tag{20.44}$$

对于 $0 < \lambda < 1$, 上式代表了条件对数密度 $\ln p(x \mid c)$ 和无条件对数密度 $\ln p(x)$ 的凸组合; 对于 $\lambda > 1$, 来自无条件得分项的贡献变为负值, 这意味着模型主动降低了生成那些忽略条件信息的样本的概率, 而更倾向于生成考虑条件信息的样本.

文本引导的扩散模型可以结合大语言模型的技术, 把条件输入扩展为通用文本序列, 即提示词 (prompt), 而非局限于预定义的类别标签 (参见第 12 章). 这允许文本输入以两种方式影响去噪过程: 第一, 将基于 Transformer 的大语言模型的内部表示与去噪网络的输入连接起来; 第二, 允许去噪网络内的交叉注意力层关注文本标记序列.

条件扩散模型的另一个应用是图像超分辨率, 旨在将低分辨率图像转换为相应的高分辨率图像. 这本质上是一个逆问题, 单个低分辨率图像对应于多个高分辨率图像的解. 实现方法是通过用低分辨率图像作为条件变量, 对由高斯分布采样生成的高分辨率样本进行去噪 (Saharia, Ho, et al., 2021).

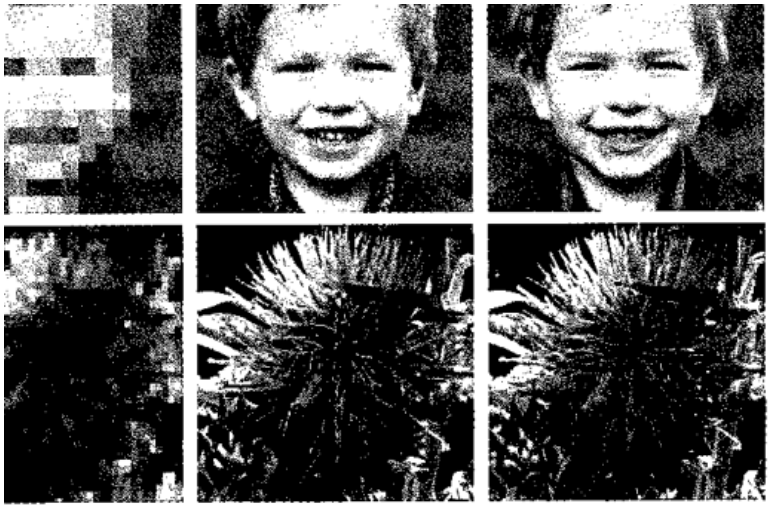
这种级联方式也适用于图像生成扩散模型, 在这种模型中, 首先在较低分辨率的图像上执行去噪操作, 之后使用另一个独立的网络 (可附加文本提示词输入) 进行上采样, 以生成最终的高分辨率图像 (Nichol et al., 2021; Saharia et al., 2022). 由于去噪过程可能需要调用去噪网络数百次, 与直接在高维空间中进行计算相比, 级联方法能够大幅减少计算量. 需要指出的是, 这种方法依旧是在图像空间中直接进行操作的, 只是分辨率较低而已. 这些模型可以级联起来, 以实现非常高的图像分辨率 (Ho et al., 2021), 例如从 64×64 到 256×256, 然后从 256×256 到 1024×1024. 每个阶段通常用一个 U-Net 架构 (参见 10.5.4 小节), 且当之前 U-Net 都以前一阶段的最终去噪结果



.7 基于文本提示的无分类器的引导，扩散模型由一个名为 GLIDE 的模型生成，使用的条件文本是“A
ed glass window of a panda eating bamboo”（一只熊猫在吃竹子的彩绘玻璃窗）。左侧的样本是用 $\lambda=0$
的（无引导，仅仅是纯条件模型），而右侧的样本是用 $\lambda=3$ 生成的［图片来源于 Nichol et al. (2021)，经
使用］

条件扩散模型的另一个应用是图像超分辨率，旨在将低分辨率图像转换为相应的

图 20.6: 基于文本提示的无分类器的引导, 扩散模型由一个名为 GLIDE 的模型生成, 使用的条件文本是 “A stained glass window of a panda eating bamboo”(一只熊猫在吃竹子的彩绘玻璃窗). 左侧的样本是用 $\lambda = 0$ 生成的 (先引导, 仅仅是纯条件模型), 而右侧的样本是用 $\lambda = 3$ 生成的 (Nichol et al., 2021, 经授权使用).



入图像　　输出图像　　原始图像

扩散模型生成的相应高分辨率图像。上面一行显示了一个 16×16 的输入图

图 20.7: 两个低分辨率图像以及由扩散模型生成的相应高分辨率图像. 上面一行显示了一个 16×16 的输入图像和相应的 128×128 的输出图像, 以及原始图像. 下面一行显示了一个 64×64 的输入图像和一个 256×256 的输出图像, 并与原始图像进行了对比 (Saharia, Ho et al., 2021, 经授权使用).

为条件.

对于在高分辨率图像空间中直接应用扩散模型所带来的高计算成本问题, 另一种方法是使用**潜扩散模型** (latent diffusion model)(Rombach et al., 2021). 首先在无噪声图像上训练一个自编码器 (autoencoder), 以获得图像的低维表征并使参数固定. 接下来训练一个 U-Net 网络, 在低维空间中执行去噪操作, 注意, 这个空间本身并不能直接解释为图像. 最后使用固定自编码器网络的输出部分将去噪表征映射到高分辨率图像空间. 这种方法更有效地利用了低维空间, 该空间可以专注于图像语义, 并让解码器从去噪的低维表示中创建相应的清晰的高分辨率图像.

条件图像生成的其他应用还包括图像修复、去除裁剪、图像恢复、图像变形、图像风格转移、着色、去模糊和视频生成等 (Yang, Srivastava, and Mandt, 2022). 图 20.8 展示了一个图像修复的例子.



修复的一个例子。左侧是原始图像，中间是部分区域被移除的图像，右侧是经过修复处理的图像
aharia, Chan, Chang et al.(2021)，经授权使用]

图 20.8: 图像修复的一个例子. 左侧是原始图像, 中间是部分区域被移除的图像, 右侧是经过修复处理的图像 (Saharia, Chan, Chang et al., 2021, 经授权使用).

附录 A 线性代数

附录 A 列出了一些有用的性质以及涉及矩阵和行列式的恒等式. 我们假设读者已经熟悉了基本的线性代数知识, 所以附录 A 并不是介绍这些概念的教程. 对于某些结论, 我们会说明如何进行证明, 而在更复杂的情况下, 建议感兴趣的读者参考与该主题有关的教科书. 在所有情况下, 我们假设矩阵的逆存在, 并且假设矩阵的维度可以使公式得到正确定义. 关于线性代数的更详细且全面的讨论可以参见 Golub and Van Loan (1996), 并且 Lütkepohl (1996) 给出了大量的矩阵性质,Magnus and Neudecker (1999) 讨论了矩阵导数.

矩阵 A 由元素 A_{ij} 组成, 其中 i 索引行,j 索引列. 我们用 I 来表示 $N \times N$ 单位矩阵 (也称恒等矩阵). 当维度上没有歧义的时候, 我们可以直接用 I 代表单位矩阵. 转置矩阵 A^{T} 包含元素 $(A^{\mathsf{T}})_{ij} = A_{ji}$. 从转置的定义来看, 我们有

$$(AB)^{\mathsf{T}} = B^{\mathsf{T}}A^{\mathsf{T}} \tag{A.1}$$

对此我们可以通过写出索引来验证.A 的逆矩阵记为 A^{-1}, 满足

$$AA^{-1} = A^{-1}A = I \tag{A.2}$$

因为 $ABB^{-1}A^{-1} = I$, 所以有

$$(AB)^{-1} = B^{-1}A^{-1} \tag{A.3}$$

并且有

$$(A^{\mathsf{T}})^{-1} = (A^{-1})^{\mathsf{T}} \tag{A.4}$$

这通过取式 (A.3) 的转置并应用式 (A.1) 可以很容易得证.

A.0.1 矩阵恒等式

一个涉及矩阵的逆的有用恒等式如下:

$$(P^{-1} + B^{\mathsf{T}}R^{-1}B)^{-1}B^{\mathsf{T}}R^{-1} = PB(B^{\mathsf{T}}PB + R)^{-1} \tag{A.5}$$

这很容易通过对式 (A.5) 的两边右乘 $(B^{\mathsf{T}}PB + R)$ 得证. 假设 P 的维度为 $N \times N$、R 的维度为 $M \times M$, 则 B 的维度就是 $M \times N$. 如果 $M < N$, 那么计算式 (A.5) 右边的值要比计算其左边的值开销小得多. 有时也会出现如下特殊情况:

$$(I + AB)^{-1}A = A(I + BA)^{-1} \tag{A.6}$$

另一个有用的关于矩阵的逆的恒等式如下:

$$(A + BD^{-1}C)^{-1} = A^{-1} - A^{-1}B(D + CA^{-1}B)^{-1}CA^{-1} \tag{A.7}$$

这就是**伍德伯里恒等式** (Woodbury identity), 对式 (A.7) 的两边同时乘以 $(A + BD^{-1}C)$ 即可得证. 在某些情况下很有用, 例如, 当 A 很大且为对角矩阵时,A 的逆很容易求得, 或者当 B 有很多行但列较少时 (C 则相反), 式 (A.7) 右边的计算开销就比左边的计算开销小得多.

如果当且仅当所有 $\alpha_i = 0$ 时,$\sum_i \alpha_i a_i = 0$ 才成立, 则称向量 $\{a_1, \cdots, a_M\}$ 是**线性无关** (linearly independent) 的. 这意味着没有任何一个向量可以表示为其余向量的线性组合. 矩阵的秩等于线性无关行的最大数量 (也可等价地说, 矩阵的秩等于线性无关列的最大数量).

A.0.2 迹和行列式

方阵有迹和行列式. 矩阵 A 的迹 $\mathrm{tr}(A)$ 定义为其主对角线上元素的和. 通过写出索引, 我们可以得到

$$\mathrm{tr}(AB) = \mathrm{tr}(BA) \tag{A.8}$$

通过将式 (A.8) 应用于三个矩阵的乘积, 可以得到

$$\mathrm{tr}(ABC) = \mathrm{tr}(CAB) = \mathrm{tr}(BCA) \tag{A.9}$$

这就是迹运算的循环 (cyclic) 性质. 显然, 这一性质可以扩展到任意数量的矩阵的乘积.$N \times N$ 矩阵 A 的行列式 $|A|$ 由下式定义:

$$|A| = \sum (-1)^p A_{1i_1} A_{2i_2} \cdots A_{Ni_N} \tag{A.10}$$

其中, 行列式对每行和每列中元素的乘积求和, 并根据排列 $i_1 \cdots i_N$ 为偶数还是奇数来确定系数是 $+1$ 还是 -1. 注意 $|I| = 1$, 且对角矩阵的行列式是由其主对角线上元素的乘积给出的. 因此, 对于 2×2 的矩阵, 其行列式的形式如下:

$$|A| = a_{11}a_{22} - a_{12}a_{21} \tag{A.11}$$

两个矩阵乘积的行列式由下式给出:

$$|AB| = |A|\,|B| \tag{A.12}$$

式 (A.12) 可以从式 (A.10) 推导得出. 同样, 逆矩阵的行列式由下式给出:

$$|A^{-1}| = \frac{1}{|A|} \tag{A.13}$$

式 (A.13) 可以通过取式 (A.2) 的行列式并应用式 (A.12) 推导得出. 如果 A 和 B 都是大小为 $N \times M$ 的矩阵, 则有

$$|I_N + AB^{\mathsf{T}}| = |I_M + A^{\mathsf{T}}B| \tag{A.14}$$

一种有用的特殊情况如下:

$$|I + ab^{\mathsf{T}}| = 1 + a^{\mathsf{T}}b \tag{A.15}$$

其中 a 和 b 都是 N 维的列向量.

A.0.3 矩阵导数

有时我们需要考虑向量和矩阵关于标量的导数. 向量 a 关于标量 x 的导数也是一个向量, 其分量为

$$\frac{\partial a}{\partial x} = \left(\frac{\partial a_i}{\partial x}\right) \tag{A.16}$$

类似地:

$$\frac{\partial x^{\mathsf{T}}a}{\partial x} = a \tag{A.17}$$

标量 x 关于向量 a 的导数可以定义为

$$\frac{\partial x}{\partial a} = \left(\frac{\partial x}{\partial a_i}\right) \tag{A.18}$$

类似地:

$$\frac{\partial (a^{\mathsf{T}}x)}{\partial x} = a \tag{A.19}$$

矩阵关于标量的导数也有类似的定义. 使用乘积法则可得

$$\frac{\partial}{\partial x}(AB) = \frac{\partial A}{\partial x}B + A\frac{\partial B}{\partial x} \tag{A.20}$$

类似地, 矩阵的逆关于标量 x 的导数可以定义为

$$\frac{\partial}{\partial x}(A^{-1}) = -A^{-1}\frac{\partial A}{\partial x}A^{-1} \tag{A.21}$$

这可以通过对式 (A.2) 微分 $A^{-1}A = I$, 然后右乘 A^{-1} 得证.

如果我们选择 x 作为 A 的元素之一, 则有

$$\frac{\partial}{\partial A_{ij}}\mathrm{tr}(A) = I_{ij} \tag{A.22}$$

类似地:

$$\frac{\partial}{\partial A}\mathrm{tr}(AB) = B^{\mathsf{T}} \tag{A.23}$$

其可以通过使用索引表示法写出矩阵得到. 这个结论也可以写成如下更紧凑的形式:

$$\nabla_A \mathrm{tr}(AB) = B^{\mathsf{T}} \tag{A.24}$$

通过使用这种表示法, 我们可以得到以下性质:

$$\nabla_A \mathrm{tr}(AB) = B^{\mathsf{T}}, \qquad \nabla_A \mathrm{tr}(ABA^{\mathsf{T}}) = A(B + B^{\mathsf{T}}), \qquad \nabla_A |A| = |A|(A^{-1})^{\mathsf{T}} \tag{A.25}$$

这些性质同样可以通过写出矩阵的索引得证. 另外, 我们还可得到

$$\nabla_A \ln |A| = (A^{-1})^{\mathsf{T}} \tag{A.26}$$

式 (A.26) 可以由式 (A.22) 和式 (A.23) 推导得出.

A.0.4 特征向量

对于大小为 $M \times M$ 的方阵 A, 其特征向量方程定义为

$$Au_i = \lambda_i u_i \tag{A.27}$$

其中 $i = 1, \cdots, M$,u_i 是特征向量,λ_i 是对应的特征值. 这可以视为一组 M 个同时满足的齐次线性方程, 有解的条件是

$$|A - \lambda I| = 0 \tag{A.28}$$

这就是特征方程. 因为这是一个关于 λ 的 M 阶多项式, 所以它必须有 M 个解 (尽管这些解不必都是不同的).A 的秩等于非零特征值的数量.

需要特别讨论的是对称矩阵, 对称矩阵可能是协方差矩阵、核矩阵或黑塞矩阵. 对称矩阵具有性质 $A_{ij} = A_{ji}$, 或等价地有 $A^{\mathsf{T}} = A$. 对称矩阵的逆矩阵也是对称矩阵, 这可以通过取 $A^{-1}A = I$ 的转置并利用 $AA^{-1} = I$ 以及 I 的对称性得证.

通常, 矩阵的特征值是复数, 但对于对称矩阵, 其特征值 λ_i 是实数. 为了证明这一点, 可以首先对式 (A.27) 左乘 u_i^*(其中 $*$ 表示复共轭), 从而得到

$$(u_i^*)^{\mathsf{T}} A u_i = \lambda_i (u_i^*)^{\mathsf{T}} u_i \tag{A.29}$$

接下来取式 (A.27) 的复共轭并左乘 u_i^{T}, 从而得到

$$u_i^{\mathsf{T}} A u_i = \lambda_i^* u_i^{\mathsf{T}} u_i \tag{A.30}$$

其中使用了 $A^{\mathsf{T}} = A$, 因为我们仅考虑实矩阵 A. 对这两个等式取转置并相减, 利用对称性质 $A^{\mathsf{T}} = A$, 可得

$$(\lambda_i - \lambda_i^*) u_i^{\mathsf{T}} u_i = 0 \tag{A.31}$$

因此, 对于 $\lambda_i \neq \lambda_i^*$, 我们有 $u_i^{\mathsf{T}} u_i = 0$, 即 u_i 和 u_i^* 是正交的. 如果这两个特征值相等, 那么任何线性组合 $\alpha u_i + \beta u_j$ 都将是具有相同特征值的特征向量, 因此我们可以任意选择一个线性组合, 然后选择另一个线性组合并使其与第一个线性组合正交 (可以证明退化的特征向量永远不是线性相关的). 因此, 特征向量可以选择为正交向量, 并通过归一化变为单位长度. 由于有 M 个特征值, 而对应的 M 个正交特征向量构成了一个完备集, 因此任何 M 维向量都可以表示为特征向量的线性组合.

实对称矩阵的特征向量 u_i 可以选择为正交归一化的向量 (即正交的且为单位长度的向量), 从而有

$$u_i^{\mathsf{T}} u_j = I_{ij} \tag{A.32}$$

其中 I_{ij} 是单位矩阵 I 的元素. 为了说明这一点, 首先将式 (A.27) 左乘 u_j^{T}, 从而得到第一个方程

$$u_j^{\mathsf{T}} A u_i = \lambda_i u_j^{\mathsf{T}} u_i \tag{A.33}$$

然后交换索引, 从而得到第二个方程:

$$u_i^{\mathsf{T}} A u_j = \lambda_j u_i^{\mathsf{T}} u_j \tag{A.34}$$

取第二个方程的转置并利用对称性质 $A^{\mathsf{T}} = A$, 最后将这两个方程相减, 从而得到

$$(\lambda_i - \lambda_j)u_i^{\mathsf{T}}u_j = 0 \tag{A.35}$$

因此, 对于 $\lambda_i \neq \lambda_j$, 我们有 $u_i^{\mathsf{T}}u_j = 0$, 即 u_i 和 u_j 是正交的.

我们可以将特征向量 u_i 作为 $M \times M$ 矩阵 U 的列, 该矩阵由于正交归一性而满足

$$U^{\mathsf{T}}U = I \tag{A.36}$$

这样的矩阵称为**正交矩阵**. 有趣的是, 这个矩阵的行也是正交的, 所以有 $UU^{\mathsf{T}} = I$. 为了证明这一点, 请注意式 (A.36) 意味着 $UU^{\mathsf{T}}U^{-1} = U^{-1} = U^{\mathsf{T}}$, 从而有 $U^{\mathsf{T}}U^{-1} = U^{-1}U^{\mathsf{T}} = I$. 使用式 (A.12), 我们还可以推导出 $|U| = 1$.

特征向量方程 [式 (A.27)] 可以用以下关于 U 的形式来表示:

$$AU = U\Lambda \tag{A.37}$$

其中 Λ 是 $M \times M$ 的对角矩阵, 其对角元素由特征值 λ_i 给出.

假设我们通过使用正交矩阵 U 转换列向量 x 得到了一个新的向量:

$$x' = Ux \tag{A.38}$$

则向量的长度保持不变, 因为

$$x'^{\mathsf{T}}x' = x^{\mathsf{T}}U^{\mathsf{T}}Ux = x^{\mathsf{T}}x \tag{A.39}$$

同样, 任何两个此类向量之间的夹角也保持不变, 因为

$$x'^{\mathsf{T}}y' = x^{\mathsf{T}}U^{\mathsf{T}}Uy = x^{\mathsf{T}}y \tag{A.40}$$

因此, 向量乘以 U 可以解释为坐标系发生刚性旋转.

从式 (A.37) 可以看出

$$U^{\mathsf{T}}AU = \Lambda \tag{A.41}$$

又因为 Λ 是对角矩阵, 我们可以说矩阵 A 被矩阵 U 对角化了. 左乘 U 并右乘 U^{T}, 我们可以得到

$$A = U\Lambda U^{\mathsf{T}} \tag{A.42}$$

取这个方程的逆并利用式 (A.3) 以及 $U^{-1} = U^{\mathsf{T}}$, 可以得到

$$A^{-1} = U\Lambda^{-1}U^{\mathsf{T}} \tag{A.43}$$

以上两个方程也可以写成以下形式:

$$A = \sum_i \lambda_i u_i u_i^{\mathsf{T}}, \qquad A^{-1} = \sum_i \frac{1}{\lambda_i} u_i u_i^{\mathsf{T}} \tag{A.44}$$

如果我们取式 (A.42) 的行列式并使用式 (A.12), 则可以得到

$$|A| = \prod_i \lambda_i \tag{A.45}$$

类似地, 取式 (A.42) 的迹, 并利用迹运算的循环性质 [式 (A.9)] 以及 $U^{\mathsf{T}}U = I$, 我们可以得到

$$\mathrm{tr}(A) = \sum_i \lambda_i \tag{A.46}$$

作为一个练习, 请读者利用式 (A.32)、式 (A.42)、式 (A.43) 和式 (A.22) 验证式 (A.22).

如果对于所有非零向量 w, 有 $w^{\mathsf{T}}Aw > 0$, 则称矩阵 A 为**正定矩阵**, 记为 $A > 0$. 等价地, 正定矩阵对于其所有特征值, 有 $\lambda_i > 0$(这可以通过将 w 分别设置为每个特征向量来验证, 并注意到任意向量都可以表示为特征向量的线性组合而看出). 一个矩阵的所有元素均为正并不一定意味着该矩阵就是正定矩阵. 例如, 矩阵 $\begin{pmatrix} 1 & 2 \\ 2 & 1 \end{pmatrix}$ 具有特征值 $\lambda \approx 3$ 和 $\lambda \approx -1$. 如果对于所有的 w, 有 $w^{\mathsf{T}}Aw \geq 0$, 则称这个矩阵为半正定矩阵, 记为 $A \geq 0$, 等价于 $\lambda_i \geq 0$.

矩阵的条件数由下式给出:

$$\mathrm{cond}(A) = \frac{\lambda_{\max}}{\lambda_{\min}} \tag{A.47}$$

其中 λ_{max} 是最大的特征值,λ_{min} 是最小的特征值.

附录 B 变分法

我们可以将函数 $y(x)$ 视为如下运算: 对于任何输入值 x, 返回输出值 y. 同样, 我们也可以将**泛函** (functional)$F[y]$ 视为接收函数 $y(x)$ 并返回输出值 F 的运算. 泛函的一个示例是在二维平面上计算绘制的曲线的长度, 其中曲线的路径是由一个函数定义的. 在机器学习中, 一个广泛使用的泛函是连续变量 x 的熵 $H[x]$. 对于任意选择的概率密度函数 $p(x)$, 它都会返回一个标量值以表示该密度下 x 的熵. 因此,$p(x)$ 的熵也可以类似地写作 $H[p]$.

传统微积分中的一个常见问题, 就是找到一个 x 值以使函数 $y(x)$ 最大化 (或最小化). 类似地, 在**变分法** (calculus of variations) 中, 我们也要寻求一个函数 $y(x)$, 该函数能使泛函 $F[y]$ 最大化 (或最小化). 也就是说, 在所有可能的函数中, 我们希望找到能使泛函 $F[y]$ 取最大值 (或最小值) 的特定函数. 例如, 我们可以使用变分法来证明两点之间直线最短, 而最大熵分布是高斯分布.

如果不熟悉普通微积分的法则, 则可以对变量 x 施加一个小的改变 ϵ, 然后以 ϵ 的幂展开来计算传统导数 $\mathrm{d}y/\mathrm{d}x$, 从而有

$$y(x+\epsilon) = y(x) + \epsilon\frac{\mathrm{d}y}{\mathrm{d}x} + O(\epsilon^2) \tag{B.1}$$

最后取极限 $\epsilon \to 0$. 类似地, 对于具有多个变量的函数 $y(x_1, \cdots, x_D)$, 相应的偏导数由下式定义:

$$y(x_1+\epsilon_1, x_2+\epsilon_2) = y(x_1, x_2) + \sum_i \epsilon_i \frac{\partial y}{\partial x_i} + O(\epsilon^2) \tag{B.2}$$

当需要考虑对函数 $y(x)$ 进行微小改动 $\epsilon\eta(x)$ 会使泛函 $F[y]$ 发生多大的改变时, 就可以类似地定义泛函的导数, 其中 $\eta(x)$ 是 x 的任意函数. 用 $\delta F/\delta y(x)$ 表示 $F[y]$ 相对于 $y(x)$ 的**泛函导数** (functional derivative), 并用以下关系式加以定义:

$$F[y(x)+\epsilon\eta(x)] = F[y(x)] + \epsilon\int \frac{\delta F}{\delta y(x)}\eta(x)\,\mathrm{d}x + O(\epsilon^2) \tag{B.3}$$

式 (B.3) 可以看作式 (B.2) 的自然扩展, 其中 $F[y]$ 依赖于一组连续的变量, 即所有点 x 处 y 的值.

如果要求泛函相对于函数 $y(x)$ 的微小变化是平稳的, 则有

$$\int \frac{\delta F}{\delta y(x)}\eta(x)\,\mathrm{d}x = 0 \tag{B.4}$$

由于式 (B.4) 必须对任意选择的 $\eta(x)$ 都成立, 因此对于任意的 x, 泛函的导数都必须为零. 为了说明这一点, 选择一个扰动 $\eta(x)$, 其在除了点 x_0 的邻域之外的任意位置都为零. 在这种情况下, 泛函的导数在 $x = x_0$ 处必须为零. 但是, 由于对任意的 x 都必须满足此条件, 因此泛函的导数必须为零.

考虑一个定义在函数 $G(\dot{y}(x), y(x), x)$ 的积分之上的泛函, 它同时还依赖于 $y(x)$ 及其导数 $\dot{y}(x)$, 并且直接依赖于 x:

$$F[y] = \int G(\dot{y}(x), y(x), x)\,\mathrm{d}x \tag{B.5}$$

其中, 假设 $y(x)$ 的值在积分的边界处 (可能在无穷大处) 为定值. 考虑函数 $y(x)$ 的变动, 我们可以得到

$$F[y(x)+\epsilon\eta(x)] = F[y(x)] + \epsilon\int \left\{\frac{\partial G}{\partial y} - \frac{\mathrm{d}}{\mathrm{d}x}\left(\frac{\partial G}{\partial \dot{y}}\right)\right\}\eta(x)\,\mathrm{d}x + O(\epsilon^2) \tag{B.6}$$

让我们以式 (B.3) 的形式来表示式 (B.6). 为此, 对式 (B.6) 右侧的第二项进行分部积分, 并注意 $\eta(x)$ 必须在积分的边界处消除 (因为 $y(x)$ 在边界处为定值), 从而得到

$$F[y(x)+\epsilon\eta(x)] = F[y(x)] + \epsilon\int \frac{\delta F}{\delta y(x)}\eta(x)\,\mathrm{d}x + O(\epsilon^2) \tag{B.7}$$

与式 (B.3) 进行比较, 即可确定泛函的导数. 由于要求消除泛函的导数, 因此有

$$\frac{\partial G}{\partial y} - \frac{\mathrm{d}}{\mathrm{d}x}\left(\frac{\partial G}{\partial \dot{y}}\right) = 0 \tag{B.8}$$

这就是**欧拉-拉格朗日** (Euler-Lagrange) 方程. 例如, 如果

$$G = \dot{y}(x)^2 + y(x)^2 \tag{B.9}$$

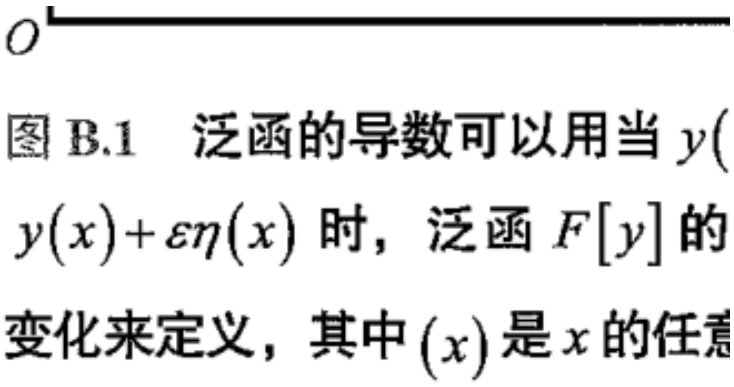


图 B.1: 泛函的导数可以用当 $y(x)$ 变为 $y(x)+\epsilon\eta(x)$ 时, 泛函 $F[y]$ 的值如何变化来定义, 其中 $\eta(x)$ 是 x 的任意函数.

则欧拉-拉格朗日方程的形式变为

$$y(x) - \ddot{y}(x) = 0 \tag{B.10}$$

这个二阶微分方程可以利用 $y(x)$ 的边界条件来求解.

考虑由积分定义的泛函, 其积分采用 $G(y, x)$ 的形式, 并且不依赖于 $\dot{y}(x)$. 在这种情况下, 只要 x 的所有值都满足 $\partial G/\partial y(x) = 0$, 就能保证解的稳定性.

如果我们关于概率分布优化一个泛函, 则需要保持概率的归一化约束. 满足该要求的最简单方式就是使用拉格朗日乘子 (参见附录 C), 其允许进行无约束的优化.

上述结果可以直接扩展到多维变量 x 上. 有关变分法的更全面讨论, 请参阅 Sagan (1969).

附录 C 拉格朗日乘子

拉格朗日乘子也称拉格朗日乘数或未定乘子, 用于在一个或多个约束条件下, 寻找多变量函数的驻点. 考虑在与 x_1, x_2 有关的约束条件下找到函数 $f(x_1, x_2)$ 最大值的问题. 该约束条件可以写成如下形式:

$$g(x_1, x_2) = 0 \tag{C.1}$$

一种方法是求解约束方程 [式 (C.1)], 从而将 x_2 表示为 x_1 的函数, 形式为 $x_2 = h(x_1)$. 然后将其代入 $f(x_1, x_2)$, 得到形式为 $f(x_1, h(x_1))$ 的仅关于 x_1 的函数. 接下来, 可以通过常用的微分方法找到关于 x_1 的最大值, 得到驻点值 x_1^*, 相应的 x_2^* 值由 $x_2^* = h(x_1^*)$ 给出.

这种方法存在的一个问题是, 找到约束方程的一个解析解可能会很困难 (该方程使得 x_2 可以表示为 x_1 的显式函数). 此外, 这种方法区别对待 x_1 和 x_2, 因而会破坏这些变量之间的自然对称性.

一种更优雅且通常更简单的方法是引入一个称为**拉格朗日乘子**的参数 λ. 我们将从几何角度来说明这一技术. 考虑一个具有 $x_1, \cdots, x_D$ 分量的 D 维向量 x. 约束方程 $g(x) = 0$ 表示 x 空间中的一个 $(D-1)$ 维曲面. 如果 x 是 D 维的, 则约束条件 $g(x) = 0$ 对应一个维度为 $D-1$ 的子空间. 我们想在约束条件 $g(x) = 0$ 下最大化函数 $f(x)$.

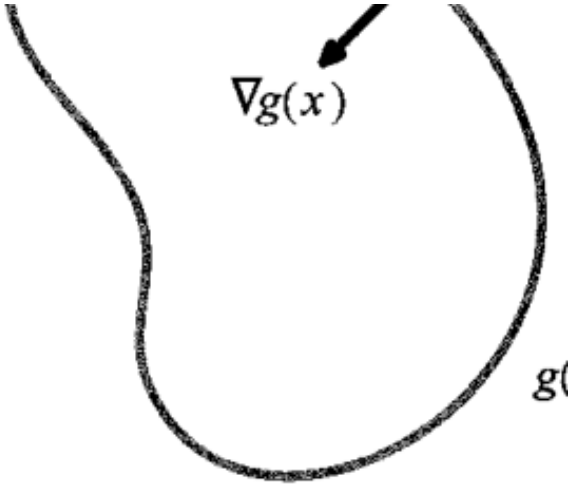


图 C.1 拉格朗日乘子法的几何解

要在约束条件 $g(x)=0$ 下最大化

如果 x 是 D 维的，则约束条件 g

一个维度为 $D-1$ 的子空间，如

图 C.1: 拉格朗日乘子法的几何解释. 我们想要在约束条件 $g(x) = 0$ 下最大化函数 $f(x)$. 约束曲面上任何一点, 约束函数的梯度 $\nabla g(x)$ 都与曲面正交; 在约束曲面上使 $f(x)$ 最大的点 x^* 处,$\nabla f(x^*)$ 也与约束曲面垂直, 因此 ∇f 与 ∇g 平行.

首先注意, 在该约束曲面上的任何一点, 约束函数的梯度 $\nabla g(x)$ 都与曲面正交. 为了说明这一点, 考虑一个位于该约束曲面上的点 x 以及另一个位于该约束曲面上的附近点 $x + \epsilon$. 如果我们在点 x 附近进行泰勒展开, 则有

$$g(x + \epsilon) \approx g(x) + \epsilon^{\mathsf{T}} \nabla g(x) \tag{C.2}$$

因为点 x 和点 $x + \epsilon$ 都位于约束曲面上, 所以我们有 $g(x) = g(x + \epsilon)$, 从而有 $\epsilon^{\mathsf{T}} \nabla g(x) \approx 0$. 当 $\epsilon \to 0$ 时, 我们有

$\epsilon^{\mathsf{T}}\nabla g(x) = 0$, 并且因为 ϵ 与约束曲面 $g(x) = 0$ 平行, 所以我们看到 ∇g 与约束曲面是垂直的.

接下来我们寻找约束曲面上使得 $f(x)$ 最大化的点 x^*. 这样的点满足 $\nabla f(x^*)$ 也与约束曲面垂直的性质, 否则我们可以通过沿约束曲面移动一小段距离来增大 $f(x)$ 的值. 因此,∇f 和 ∇g 是平行 (或反平行) 的, 并且必定存在一个参数 λ, 使得

$$\nabla f + \lambda \nabla g = 0 \tag{C.3}$$

其中 $\lambda \neq 0$ 称为拉格朗日乘子. 注意,λ 可正可负.

至此, 我们很容易引出下式定义的**拉格朗日函数** (Lagrangian):

$$L(x, \lambda) = f(x) + \lambda g(x) \tag{C.4}$$

为了找到在约束条件 $g(x) = 0$ 下函数 $f(x)$ 的最大值, 首先定义式 (C.4) 所示的拉格朗日函数, 然后寻找 $L(x, \lambda)$ 关于 x 和 λ 的驻点. 对于一个 D 维向量 x, 这会给出 $D+1$ 个方程, 用于确定驻点 x^* 以及 λ^* 的值. 可通过设置 $\nabla L = 0$ 得到带约束的驻点条件 [式 (C.3)]. 此外, 通过 $\partial L/\partial \lambda = 0$ 可推出约束条件 $g(x) = 0$. 如果只对 x 感兴趣, 则可以从驻点方程中消去 λ, 而不需要计算 λ 的值 ("未定乘子" 由此得名).

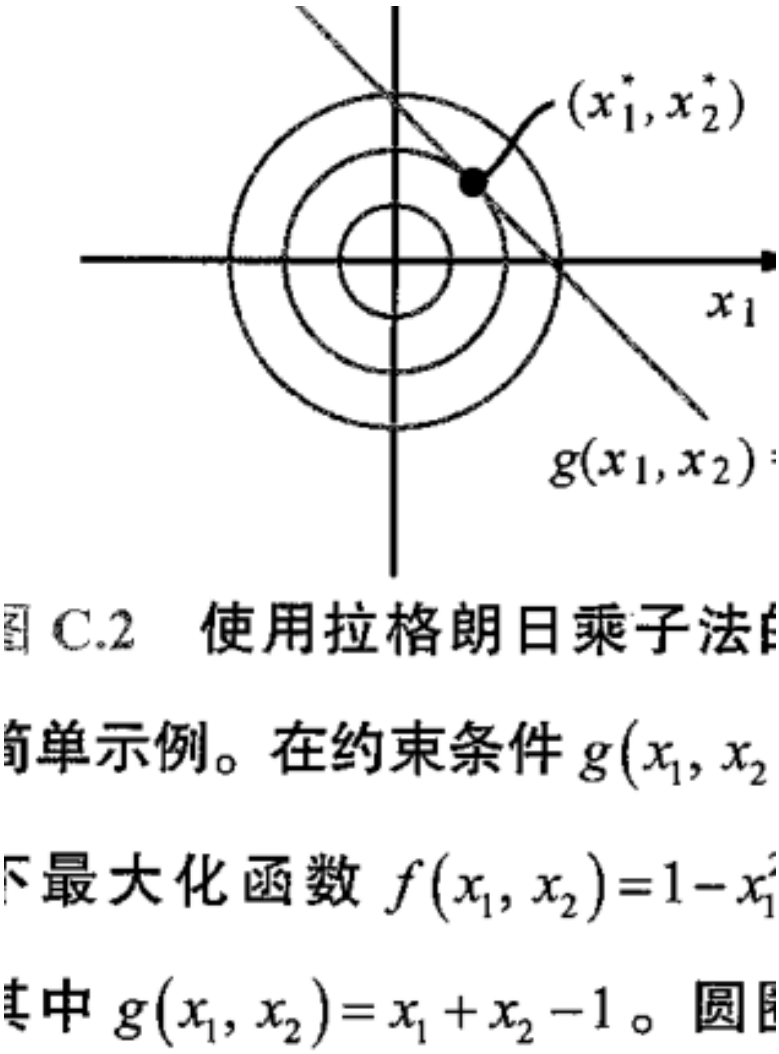


图 C.2: 使用拉格朗日乘子法的一个简单示例. 在约束条件 $g(x_1, x_2) = 0$ 下最大化函数 $f(x_1, x_2) = 1 - x_1^2 - x_2^2$, 其中 $g(x_1, x_2) = x_1 + x_2 - 1$. 椭圆表示函数 $f(x_1, x_2)$ 的等值线, 对角线则代表约束曲面 $g(x_1, x_2) = 0$.

举个简单的例子, 假设我们希望找到函数 $f(x_1, x_2) = 1 - x_1^2 - x_2^2$ 在约束条件 $g(x_1, x_2) = x_1 + x_2 - 1 = 0$ 下的驻点. 相应的拉格朗日函数由下式给出:

$$L(x, \lambda) = 1 - x_1^2 - x_2^2 + \lambda(x_1 + x_2 - 1) \tag{C.5}$$

这个拉格朗日函数关于 x_1、x_2 和 λ 的驻点条件给出了如下耦合方程:

$$-2x_1 + \lambda = 0 \tag{C.6}$$

$$-2x_2 + \lambda = 0 \tag{C.7}$$

$$x_1 + x_2 - 1 = 0 \tag{C.8}$$

解这些方程, 可以得到驻点 $(x_1^*, x_2^*) = (1/2, 1/2)$, 相应的拉格朗日乘子为 $\lambda = 1$.

到目前为止, 我们已经讨论了在形如 $g(x) = 0$ 的**等式约束** (equality constraint) 下如何最大化函数的问题. 接下来考虑在形如 $g(x) \geq 0$ 的**不等式约束** (inequality constraint) 下如何最大化 $f(x)$ 的问题.

根据带约束的驻点是否位于 $g(x) > 0$ 的区域内, 这个问题有两种可能的解. 如果驻点位于 $g(x) > 0$ 的区域内, 则约束是**非激活的** (inactive); 而如果驻点位于边界 $g(x) = 0$ 上, 则约束是**激活的** (active). 在前一种情况下, 函数 $g(x)$ 不起作用, 因此驻点条件是 $\nabla f(x) = 0$, 这对应于 $\lambda = 0$ 的拉格朗日函数 [式 (C.4)] 的驻点. 在后一种情况下, 驻点位于边界上, 类似于前面讨论的等式约束, 对应于 $\lambda \neq 0$ 的拉格朗日函数 [式 (C.4)] 的驻点. 可见拉格朗日乘子的符号是关键, 因为只有当 $f(x)$ 的梯度远离 $g(x) > 0$ 的区域时, 函数 $f(x)$ 才会达到最大值. 因此, 当 $\lambda > 0$ 时, 我们有 $\nabla f(x^*) = -\lambda \nabla g(x^*)$.

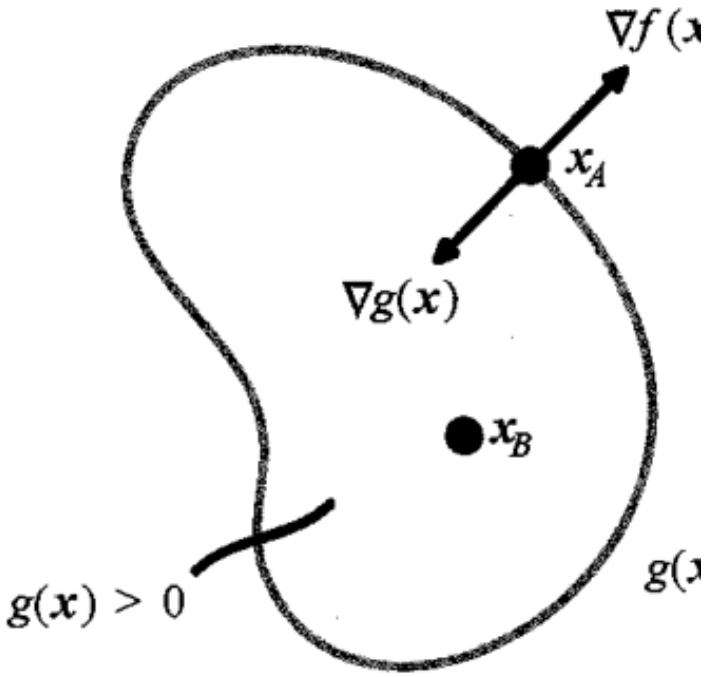


图 C.3 在不等式约束 $g(x) \geqslant 0$ 大化函数 $f(x)$ 的问题示意图

图 C.3: 在不等式约束 $g(x) \geq 0$ 下最大化 $f(x)$ 的问题示意图. 当驻点位于 $g(x) > 0$ 的区域内时约束非激活; 当驻点位于 $g(x) = 0$ 的边界上时约束激活.

对于这两种情况, 乘积 $\lambda g(x) = 0$. 因此, 在 $g(x) \geq 0$ 的不等式约束下最大化 $f(x)$ 的问题, 可通过在以下条件下关于 x 和 λ 优化拉格朗日函数 [式 (C.4)] 来求解:

$$g(x) \geq 0 \tag{C.9}$$

$$\lambda \geq 0 \tag{C.10}$$

$$\lambda g(x) = 0 \tag{C.11}$$

我们称它们为**卡鲁什-库恩-塔克** (Karush-Kuhn-Tucker, KKT) 条件 (Karush, 1939; Kuhn and Tucker, 1951). 注意, 如果我们希望在不等式约束 $g(x) \geq 0$ 下最小化 (而非最大化) 函数 $f(x)$, 则需要关于 x 最小化拉格朗日函数 $L(x, \lambda) = f(x) - \lambda g(x)$, 这同样要求 $\lambda \geq 0$.

最后, 拉格朗日乘子法可以很容易地扩展到具有多个等式约束和不等式约束的情况. 假设我们希望在约束条件 $g_j(x) = 0(j = 1, \cdots, J)$ 和 $h_k(x) \geq 0(k = 1, \cdots, K)$ 下最大化 $f(x)$, 则可以引入拉格朗日乘子 $\{\lambda_j\}$ 和 $\{\mu_k\}$, 并优化如下拉格朗日函数:

$$L(x, \{\lambda_j\}, \{\mu_k\}) = f(x) + \sum_j \lambda_j g_j(x) + \sum_k \mu_k h_k(x) \tag{C.12}$$

其中 $\mu_k \geq 0$ 且 $\mu_k h_k(x) = 0, k = 1, \cdots, K$. 关于拉格朗日乘子法更详细的讨论, 请参阅 Nocedal and Wright (1999).